

DSM MFront Branch Full Change Dump

Generated from git diff with syntax coloring

Generated on 2026-04-30 11:38:37 CEST

Range

```
Branch: dsm_mfront
Base: 874095d37c 874095d37c5478a48caf06d2115b9e62dcd1af5
Head: dd7cc48635 dd7cc48635a81ef7ad847609226f3abbe8d19dd
Range: 874095d37c..dd7cc48635
dd7cc48635a81ef7ad847609226f3abbe8d19dd (HEAD -> dsm_mfront, origin/dsm_mfront, origin/dsm-nb-mfront-transition) RM DSM: drop generated MFront benchmark outputs
70eaaaf8513769a56351e4dbd74dd71ad82c58824 RM DSM: add MFront BEACON ctests
5d4cc99475d4407902c17ae8aa920adf8c1253b4 RM DSM: add renamed bridge baselines
7bbe07925f2dc4e98e78924c0d6e6f3cb965ae53 (vgk2/dsm-nb-mfront-transition, github/dsm_mfront, github/dsm-nb-mfront-transition) benchmarks: stabilize calibrated BEACON 1a01 startup
1cc4c44a52184508b24e38004a32031380d7926 DSM: switch Bishop law to chi5_L in benchmark/calibration decks
126450c10b48a9a8f86f8ec4463357cce3fcd6dbfe docs: add benchmark/calibration script inventory
d575e778779b5c98cc93175b3c57e823f85c7649 benchmarks: add linear-law-calibrated DSM triplet comparison workflow
a731a2fb72675f3778b1082a4728ecfcb9970a5 dsm: fix EPFL parity mapping and refresh native-mfront benchmark tooling
2f93d44f5c5632e31af18080b8f0a162f7286285e DSM cleanup: remove stale potential_role_mapping leftovers and align docs
9a0c9d288ff066bbbe28c5d5f278754f224ba7e9 (ogs-2/dsm-nb-mfront-transition) docs(mpl): rename SaturationTuller DSM comment wording
eccd555f53a8e4edd2fbd12fddb8a6d4ed57c DSM MFront tests: drop notebook naming from bridge wiring
14958d2922caa53ca436ce6961034a4f198214f4 MFront DSM: rename notebook scope to dsm_micromacro
fa6d95693e885c777b5eae20835233f6d6dd0d956 Document post-rename automated checks and CTest status
2cf7b4734d81c9ea573461d0e65bf54fce0ef00d Rename vk BEACON project filenames and sync script references
69844051af36e80a9dba38d480ff551cf54762 Update BEACON/ANCHORS calibration assets and potential_exchange naming
720e773c679b0118added089bd0f823561d4f213 BEACON 1a01: align native Hamaker with bridge parity run
7d0bfa6c319e385657ea09bb546ce9348bd946 Use current-density scaling in MFront bridge and stabilize benchmark reruns
26d80c748b8872bdfb84e7efeb8865dee62c84 Add identical-parameter native-vs-MFront benchmark parity runner
f285203839acc3e0dae99ea3705ed212058e8d2 BEACON 1a01 native replay: switch to MCC carrier and reference n_5 mode for tighter native-MFront parity
96c229133e3a6004fe41478dca5f8380c2573df5 BEACON: align native 1a01/1b decks with calibrated native-vs-mfront parity run
d4fc0988e73eb474ca0ef33651dd9ad9a8df97421 Tests: expand DSM native-vs-MFront unit coverage and fix bridge-core anchor
3bc1d8979e2a078a10e12ea3ab38ad56de2c722 Organize calibrated benchmark artifacts and enforce biot=1.0 decks
15e01cd41aa05271651be5156a1ae911a645e docs(scripts): document remaining RichardsMechanics helper scripts
0387f6dfafad268cb0bf1e207bf08347f6879b docs(scripts): add inline documentation for RichardsMechanics helper workflows
26071fcbec6c19445ae87229cf31751b60bc2d anchors: drop baseline series from ciemat/villar helper plots
638c90e511b442b0955efd5a0f5139c8c4c1048 Tighten dense native-vs-MFront vdW fit to 1e-4 tolerance
f1acfd4b20a0cd27037d5c2ddccc47fe88dc83c Tighten dense native-vs-MFront vdW calibration to 0.2% tolerance
4e56ed3a1ade83087410d401e6ed930e35a83cbc Align dense vdW calibration with native mass-storage mode
fdea3674d01c8e1a18491d4e0693c8da25d8faff Fix ANCHORS run table hash for dense D0 parity rerun
a5ae4845e2cd5273da0ac14b00a916f655c10da Align native dense D0 notebook calibration and document parity status
a6431b12c681ae12ce943af1f08289487b66 Add dense Villar calibration artifacts and native-notebook comparison
f0a453f89c7d9987ce81bfc03977f8360c486c Rerun first benchmark set and refresh BEACON/EPFL summaries
1adfb426940d225b65999f394e06185bd8cfc Add BEACON, EPFL, and MS33 artefacts with run-hash table
d9effb0c5d634743589e98a4bb5c00296bbe119a Update BEACON parity checkpoint notes
d7bbdbf476c6500b882c7fa8belce58edcf80aff Add reusable BEACON plotting helpers
f02b321060ff3b0bdc6e8410b34e5961a9b23191 Add first D3.3 EPFL BGR benchmark attempt
6359d1f953436e943b6c2be9efc8a1b7625d13 Calibrate 1a01 bridge for axial BEACON match
c0db2f262af3227f060839cd6ca018854284a929 tests: add dense beacon native bridge report batch
48c5ab66ea3f51fd6f97db685b9aa8e39ed6ed docs: add beacon mesh and bc figures
2a0fa56789ba44d29c0594fd121f868e44104e6c docs: compare beacon 1a01 and 1b against report
f628cf5f6cbf8e39f5175a04ac6be0dd582410 rm: match beacon 1a01 inflow notebook bridge
39612779a6cc9e3453bb883c21b5ee201d11fe15 mfront: keep notebook swelling micro-driven
78111c384ae994feaea84af532509ea46aa25e3 docs: update beacon branch comparison
c349ee271348fcd8a1ea3a306d08081dda7e796 tests: add beacon notebook bridge smoke study
ea13205ea3f1e4b43f5043e7328ed8bde192ae0 mfront: expose notebook support state on hybrid bridge
aaaf3f14a3a7599839a974e1d994dcd1dfff66 tests: add notebook tuller benchmark parity gate
6a66d26c542bb391ab7da25aeb23e7cb7b77d6d mfront: add notebook swelling feedback to hybrid bridge
22500e25c75f41bd640f12514f09d4a65ad4207 mfront: add notebook-mcc hybrid bridge step
89a0093d2d3c8d77438f12e3d7e3bae73be229 tests: add MCC benchmark parity compare
de26d5cd311225457841375aea00b01ec3e678 mfront: add pressure-coupled constE MCC bridge
cd35e979355d9b7c17163cc0ca050be19857d mpl: align RM unsaturated parity to notebook law
2424588b1e2a933dc0a2a9d1b3e7cee3d2c1e7d7 rm: preserve pressure-coupled saturation state
77a06bd8c2fb1c941a8c2e0e4dcd98cb33a3079d tests: add exact elastic RM bridge parity gate
a56427fcb09b58655d3201e82535e1be916a9fc5 rm: consume bridge saturation-strain tangent
aa1fb3cfeccfbcadb39e5dd046fc1777a11192a0 docs: simplify RM bridge parity notes
450dce2ed88d7d8220de9dc158df97aed7735e3 rm: stabilize bridge benchmark shell
c15a0e3ac28e9ed8da5c84ae79c7f32b2c811a4 rm: tighten bridge benchmark boundary
0ce07a0364e2df38f2d5494a1ad4f2e370f17b20 docs: formalize RM bridge contract note
362eb8df95921dc229d4e759e507fbc4891ec2a2 rm: unblock bridge secondary vars
ef1c8dd89efc12ac57c29234b07a82ba536cca32 tests: add RM bridge parity compare
d0c4119f4075aca93a16dc3e81a6009156ac072 RM: stabilize bridge process parity shells
1c470be5111eb782771c3af3a0bc4be1b0a3811 RM: add plane-strain bridge initial-assembly regression
06c6af18430e2403cef14dc95a0080bbd36f80a3 RM: add one-element parity shells and zero-dt bridge unit
a0bb9d17beb2a069738b7d23ae0b1db1e4a347a4 RM: refresh bridge strain replay baseline
2549aae061f4f631ea87d9cc08b014f5e9081537 RM: add notebook overlap carrier regression
a50e35b0b12123375caa21a9eb2286ceaa0110 RM: add notebook strain-storage bridge unit
ebbf17b72ccf8faeb4f2406b6680a63e396b285 RM: add reduced notebook overlap bridge law
f062795fa5672503578f00b4c7fc88f785d58cf9 RM: add reduced notebook baseline regression
25633c0c9faf33ade18ab09f4ef852c715df0b4 RM: add bridge-local notebook microstate unit
c45df09267d8e3862d8fcbef9e9c890bd1421 RM: consume coupled bridge response in RM assembly
48570b7536aa91bc23693f860a69ae253796f13e RM: expose coupled bridge response via MechanicsBase
a2e3ada813a21e0b7969f11c9c5ccc766253fcdcd RM: expose bridge pressure-coupled response blocks
7957208d573c66f96a5d6e117191af091f3ba2c9 RM: add factory-path bridge regression
bac796c78af08b84ba441bc365062613df48e1 RM: add bridge history comparison regression
9b01d2905b5d7021ca9482e139fab0af3b6a866 build: restore vtkdiff import for OgsTest
6d5282a88ca723f518a03de64aae8ea73aad209e RM: add notebook bridge unit and wiring
0aaa46cf85e0ace120f644197a8e0538fa3349e Merge branch ogs:master into ogs-TPM_Swelling_MCC_Coupled
64108547cbb1edbbb844aaab95911e9c97b79c2f Added the first working unit test.
6d046fcff36e5c32280ced34187f413541ed78f Added the new MFront model.
dfa4b88a098d14e4e2276bcf6681dc30f6f3534e Added name ModCamClay_semiExpL_coupled to CMaKeLists.
```

Changed Files

```

.gitignore | 8 +
.../SaturationTuller/c_SaturationTuller.md | 1 +
.../SaturationTuller/t_area_factor_tuller.md | 1 +
.../SaturationTuller/t_characteristic_pore_size.md | 1 +
.../t_pore_area_shape_factor_tuller.md | 1 +
.../property/SaturationTuller/t_surface_tension.md | 1 +
.../RichardsMechanicsDSMBridgeParameterComparison.md | 320 +++
.../RichardsMechanicsDSMBridgeParityPlan.md | 1564 +++++
Documentation/figures/beacon_1a01_mesh_bc.png | Bin 0 -> 90256 bytes
Documentation/figures/beacon_1b_mesh_bc.png | Bin 0 -> 88703 bytes
MaterialLib/MPL/CreateInfoProperty.cpp | 4 +
.../CreateInfoSaturationTuller.cpp | 39 +
.../CreateInfoSaturationTuller.h | 29 +
.../SaturationTuller.cpp | 128 ++
.../CapillaryPressureSaturation/SaturationTuller.h | 86 ++
MaterialLib/MPL/Properties/CreateInfoProperties.h | 1 +
MaterialLib/MPL/Properties/Properties.h | 1 +
.../SolidModels/CreateInfoConstitutiveRelation.cpp | 12 +
MaterialLib/SolidModels/MFront/CMakeLists.txt | 8 +
.../SolidModels/MFront/CreateInfoFrontGeneric.cpp | 9 +-
.../MFront/CreateInfoFrontRichardsMechanics.cpp | 46 +
.../MFront/CreateInfoFrontRichardsMechanics.h | 41 +
MaterialLib/SolidModels/MFront/MFrontGeneric.h | 40 +-
.../SolidModels/MFront/MFrontRichardsMechanics.cpp | 16 +
.../SolidModels/MFront/MFrontRichardsMechanics.h | 228 +++
.../dCamClay_semiExpl_constE_pressureCoupled.mfront | 255 ++++
.../MFront/ModCamClay_semiExpl_coupled.mfront | 343 +++++
.../RichardsMechanicsDSMMicroMacroBridge.mfront | 587 ++++++
.../RichardsMechanicsDSMMicroMacroBridge_MCC.mfront | 1267 ++++++
MaterialLib/SolidModels/MechanicsBase.h | 27 +
ProcessLib/Output/SecondaryVariable.cpp | 6 +
ProcessLib/Output/SecondaryVariable.h | 2 +
.../ConstitutiveRelations/ConstitutiveData.h | 2 +
.../PressureCoupledSolidData.h | 55 +
.../RichardsMechanics/IntegrationPointData.h | 41 +-
.../RichardsMechanics/RichardsMechanicsFEM-impl.h | 441 +++++
.../RichardsMechanics/RichardsMechanicsProcess.cpp | 6 +
ProcessLib/RichardsMechanics/Tests.cmake | 353 +++++
Tests/CMakeLists.txt | 58 +-
.../MicroMacroBridge_overlap_transfer_baseline.csv | 6 +
.../MacroBridge_strain_coupled_overlap_baseline.csv | 6 +
.../RichardsMechanicsNotebookBridge_history_baseline.csv | 6 +
.../RichardsMechanicsNotebookBridge_overlap_transfer_baseline.csv | 6 +
.../eBookBridge_strain_coupled_overlap_baseline.csv | 6 +
.../EmbeddedAnchorSourceTerm/ms33_no_anchor.prj | 170 +++
.../EmbeddedAnchorSourceTerm/ms33_one_anchor.json | 13 +
.../EmbeddedAnchorSourceTerm/ms33_one_anchor.prj | 177 +++
.../EmbeddedAnchorSourceTerm/ms33_one_anchor.vtu | 30 +
.../ANCHORS_MS33_ModelI/.gitignore | 3 +
.../ANCHORS_MS33_ModelI/README.md | 88 ++
.../ANCHORS_MS33_ModelI/ms33_model_i_dd1400.prj | 318 ++++
.../ANCHORS_MS33_ModelI/ms33_model_i_dd1600.prj | 318 ++++
.../ANCHORS_MS33_ModelI/ms33_model_i_dd1800.prj | 318 ++++
.../ANCHORS_MS33_ModelI/ms33_model_i_history.csv | 40 +
.../ANCHORS_MS33_ModelI/ms33_model_i_summary.json | 749 ++++++
.../vs_ciemat_swelling_pressure_vs_dry_density.csv | 108 ++
.../vs_ciemat_swelling_pressure_vs_dry_density.png | Bin 0 -> 126233 bytes
.../plot_ms33_vs_ciemat_swelling_pressure_dd.py | 188 ++
.../run_villar_dense_dd_calibration.py | 891 ++++++
.../villar_dense_dd_native_dsm_micromacro_branch.py | 610 ++++++
.../r_dense_dd_native_dsm_micromacro_calibration.py | 741 ++++++
.../ANCHORS_MS33_ModelI/summarize_ms33_model_i.py | 158 ++
.../villar_dense_dd_calibration.csv | 18 +
.../villar_dense_dd_calibration_summary.json | 369 +++++
.../villar_dense_dd_native_dsm_micromacro_branch.csv | 18 +
.../se_dd_native_dsm_micromacro_branch_summary.json | 285 ++++
.../e_dd_native_dsm_micromacro_branch_vs_villar.png | Bin 0 -> 109764 bytes
.../dense_dd_native_dsm_micromacro_calibration.csv | 18 +
.../active_dsm_micromacro_calibration_comparison.png | Bin 0 -> 123872 bytes
.../native_dsm_micromacro_calibration_summary.json | 373 +++++
.../nse_dd_native_dsm_micromacro_vdw_multiplier.png | Bin 0 -> 85687 bytes
.../e_dd_native_vs_mfront_calibrated_comparison.csv | 18 +
.../ve_vs_mfront_calibrated_comparison_summary.json | 9 +
.../villar_dense_dd_swelling_pressure_comparison.png | Bin 0 -> 92085 bytes
.../villar_dense_dd_vdw_multiplier.png | Bin 0 -> 73186 bytes
.../BEACON_WP3_BGR_EPFL/.gitignore | 2 +
.../BEACON_WP3_BGR_EPFL/README.md | 13 +
.../BEACON_WP3_BGR_EPFL/analyze_bgr_wp3_abprime.py | 87 ++
.../bgr_wp3_epfl_domain_162e.vtu | 165 +++
.../BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_domain_2e.vtu | 26 +
.../BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_geometry.gml | 17 +
.../bgr_wp3_p2_1_abprime_fig4_5b_like.png | Bin 0 -> 86176 bytes
.../bgr_wp3_p2_1_abprime_fig4_6_like.png | Bin 0 -> 116095 bytes
.../bgr_wp3_p2_1_abprime_mfront.prj | 501 +++++
.../bgr_wp3_p2_1_abprime_native.prj | 356 +++++
.../bgr_wp3_p2_1_abprime_report.json | 38 +
.../bgr_wp3_p2_1_abprime_report_style_panel.png | Bin 0 -> 186326 bytes
.../bgr_wp3_p2_1_abprime_report_style_series.csv | 99 ++
.../BEACON_WP3_BGR_EPFL/generate_bgr_epfl_mesh.py | 127 ++
.../plot_bgr_wp3_epfl_report_figures.py | 392 +++++
.../BEACON_report_figures/.gitignore | 3 +
.../1a01/1a01_bridge_reportplot.prj | 218 +++
.../1a01/1a01_native_reportplot.prj | 187 +++
.../1b/1b_bridge_reportplot.prj | 138 ++
.../1b/1b_native_reportplot.prj | 104 ++
.../1c/1c_bridge_reportplot.prj | 209 +++
.../BEACON_report_figures/README.md | 53 +
.../BEACON_report_figures/beacon_1a01_profile.csv | 5 +
.../beacon_1a01_report_style_panel.png | Bin 0 -> 177103 bytes
.../BEACON_report_figures/beacon_1a01_series.csv | 25 +
.../BEACON_report_figures/beacon_1b_profile.csv | 11 +
.../beacon_1b_report_style_panel.png | Bin 0 -> 250260 bytes
.../BEACON_report_figures/beacon_1b_series.csv | 34 +
.../BEACON_report_figures/beacon_1c_profile.csv | 4 +
.../beacon_1c_report_style_panel.png | Bin 0 -> 162153 bytes
.../beacon_1c_zone_series.csv | 11 +
.../beacon_report_plot_summary.json | 51 +
.../ve_branch_dsm_micromacro_comparison_report.json | 151 ++
.../ve_branch_dsm_micromacro_comparison_summary.csv | 5 +
.../native_branch_dsm_micromacro_stress_compare.png | Bin 0 -> 87031 bytes
.../DSM_leftover_cleanup_2026-04-14.md | 34 +
.../Mathematica_DSM_Driver/DsmMicromacroDriver.wl | 427 +++++
.../Mathematica_DSM_Driver/README.md | 62 +

```

```

.../Mathematica_DSM_Driver/run_dsm_driver_demo.wl | 119 ++
Tests/Data/RichardsMechanics/SCRIPT_INVENTORY.md | 62 +
Tests/Data/RichardsMechanics/_outputs/.gitignore | 3 +
.../analyze_beacon_unstructured_batch.py | 153 ++
Tests/Data/RichardsMechanics/beacon_1a01.gml | 17 +
...beacon_1a01_axial_fit_native_bridge_report.json | 93 ++
.../Data/RichardsMechanics/beacon_1a01_domain.vtu | 24 +
.../beacon_1a01_domain_unstructured_162e.vtu | 216 +++
.../beacon_1a01_dsm_micromacro_mcc_bridge.prj | 214 +++
...cromacro_mcc_bridge_reference_t_1000.000000.vtu | 50 +
...eacon_1a01_dsm_micromacro_mcc_inflow_bridge.prj | 218 +++
...mcc_inflow_bridge_reference_t_100000.000000.vtu | 50 +
...sm_micromacro_mcc_inflow_unstructured_batch.prj | 218 +++
.../beacon_1a01_inflow_unstructured_batch.prj | 192 +++
Tests/Data/RichardsMechanics/beacon_1b.gml | 17 +
Tests/Data/RichardsMechanics/beacon_1b_domain.vtu | 24 +
.../beacon_1b_domain_unstructured_162e.vtu | 216 +++
.../beacon_1b_dsm_micromacro_mcc_bridge.prj | 138 ++
...cromacro_mcc_bridge_reference_t_1000.000000.vtu | 50 +
...on_1b_dsm_micromacro_mcc_unstructured_batch.prj | 138 ++
.../beacon_1b_unstructured_batch.prj | 110 ++
Tests/Data/RichardsMechanics/beacon_1c.gml | 19 +
Tests/Data/RichardsMechanics/beacon_1c_domain.vtu | 26 +
.../beacon_1c_dsm_micromacro_mcc_bridge.prj | 209 +++
...cromacro_mcc_bridge_reference_t_1000.000000.vtu | 50 +
.../calibrate_beacon_1b_ddcurve_probe.py | 403 +++++
.../generate_beacon_unstructured_mesher.py | 81 +
.../mfront_parity_lelement_bridge.prj | 368 +++++
...t_parity_lelement_dsm_micromacro_mcc_bridge.prj | 474 +++++
...y_lelement_dsm_micromacro_mcc_tuller_bridge.prj | 474 +++++
...y_lelement_dsm_micromacro_mcc_tuller_native.prj | 315 +++++
.../mfront_parity_lelement_elastic_bridge.prj | 405 +++++
.../mfront_parity_lelement_elastic_native.prj | 303 +++++
.../mfront_parity_lelement_mcc_bridge.prj | 339 +++++
.../mfront_parity_lelement_mcc_native.prj | 315 +++++
.../mfront_parity_lelement_native.prj | 304 +++++
.../mfront_parity_lelement_unsat_bridge.prj | 408 +++++
.../mfront_parity_lelement_unsat_native.prj | 306 +++++
...ont_restart_part1_dsm_micromacro_mcc_bridge.prj | 510 ++++++
...tart_part1_dsm_micromacro_mcc_tuller_bridge.prj | 461 ++++++
...tart_part1_dsm_micromacro_mcc_tuller_native.prj | 309 +++++
.../mfront_restart_part1_rm_bridge.prj | 382 +++++
.../plot_beacon_report_figures.py | 748 ++++++++
.../plot_calibrated_benchmark_comparison.py | 167 +++
.../run_beacon_native_bridge_report_batch.py | 110 ++
.../run_calibrated_beacon_anchors_comparison.py | 868 ++++++++
.../run_identical_parameter_benchmark_parity.py | 573 ++++++
.../run_linearLaw_calibrated_dsm_comparison.py | 759 ++++++++
.../run_linear_mcc_d01700_dsm_comparison.py | 623 ++++++
.../run_native_branch_dsm_micromacro_comparison.py | 355 +++++
Tests/Data/RichardsMechanics/test_run_table.csv | 6 +
.../MFront/TRM_MCC_coupled/.gitignore | 6 +
.../ModCamClay_semiExpl_coupled-generic.hxx | 96 ++
.../TFEL/Material/ModCamClay_semiExpl_coupled.hxx | 1228 ++++++++
.../ModCamClay_semiExpl_coupledBehaviourData.hxx | 348 +++++
.../ModCamClay_semiExpl_coupledIntegrationData.hxx | 188 ++
.../TRM_MCC_coupled/mod_cam_clay_coupled.ipynb | 852 ++++++++
.../MFront/TRM_MCC_coupled/saturation.prj | 452 +++++
.../MFront/TRM_MCC_coupled/saturation_coupled.prj | 485 +++++
.../MFront/TRM_MCC_coupled/square_1x1.gml | 31 +
.../MFront/TRM_MCC_coupled/square_1x1_quad_1e0.vtu | 25 +
.../MFront/TRM_MCC_coupled/src/Makefile.mfront | 52 +
.../src/ModCamClay_semiExpl_coupled-generic.cxx | 345 +++++
.../src/ModCamClay_semiExpl_coupled.cxx | 150 ++
.../MFront/TRM_MCC_coupled/src/targets.lst | 40 +
.../ModCamClaySemiExplConstPressureCoupled.cpp | 938 ++++++++
.../RichardsMechanicsDSMMicroMacroBridge.cpp | 1392 ++++++++
.../RichardsMechanicsDSMMicroMacroBridgeMCC.cpp | 1335 ++++++++
Tests/MaterialLib/TestMPLSaturationTuller.cpp | 81 +
.../RichardsMechanics/PressureCoupledSolidData.cpp | 49 +
mfront/tests/behaviours/CheckParamPassing1.mfront | 86 ++
.../CheckStiffnessMatrixAllLocks.mfront | 100 ++
.../CheckStiffnessMatrixAllLocksReordered.mfront | 101 ++
.../CheckStiffnessMatrixExcessLocks.mfront | 107 ++
.../CheckStiffnessMatrixMissingLocks.mfront | 54 +
scripts/cmake/Dependencies.cmake | 10 +
scripts/cmake/DependenciesExternalProject.cmake | 69 +
...nicsMFrontDSMMicroMacroMCCBenchmarkParity.cmake | 112 ++
...ardsMechanicsMFrontDSMMicroMacroMCCParity.cmake | 89 ++
...rontDSMMicroMacroMCCtullerBenchmarkParity.cmake | 112 ++
...hysicsMFrontDSMMicroMacroMCCtullerParity.cmake | 89 ++
...mpareRichardsMechanicsMFrontElasticParity.cmake | 84 ++
...RichardsMechanicsMFrontMCCBenchmarkParity.cmake | 112 ++
.../CompareRichardsMechanicsMFrontMCCParity.cmake | 89 ++
.../CompareRichardsMechanicsMFrontParity.cmake | 86 ++
...dsMechanicsMFrontUnsaturatedElasticParity.cmake | 84 ++
199 files changed, 38801 insertions(+), 109 deletions(-)

```

Full Diff

```

diff --git a/.gitignore b/.gitignore
index d174388ea6..e94c10eb73 100644
--- a/.gitignore
+++ b/.gitignore
@@ -41,3 +41,11 @@ nohup.out
/wheelhouse
.ipynb_checkpoints
/Applications/Python/_version.py
+
+## Local benchmark runtime outputs (kept on disk, not committed)
+beacon_*_reportplot.pvd
+beacon_*_reportplot_t_*.vtu
+bgr_wp3_p2_1_abprime_mfront.pvd
+bgr_wp3_p2_1_abprime_mfront_t_*.vtu
+!Tests/Data/RichardsMechanics/_outputs/**
+!Tests/Data/RichardsMechanics/_outputs/.gitignore
diff --git a/Documentation/ProjectFile/properties/property/SaturationTuller/c_SaturationTuller.md b/Documentation/ProjectFile/properties/property/SaturationTuller/c_SaturationTulle
r.md
new file mode 100644
index 0000000000..3367905b27

```

```

--- /dev/null
+++ b/Documentation/ProjectFile/properties/property/SaturationTuller/c_SaturationTuller.md
@@ -0,0 +1 @@
+\\copydoc MaterialPropertyLib::SaturationTuller
diff --git a/Documentation/ProjectFile/properties/property/SaturationTuller/t_area_factor_tuller.md b/Documentation/ProjectFile/properties/property/SaturationTuller/t_area_factor_t
uller.md
new file mode 100644
index 0000000000..7857234845
--- /dev/null
+++ b/Documentation/ProjectFile/properties/property/SaturationTuller/t_area_factor_tuller.md
@@ -0,0 +1 @@
+\\copydoc MaterialPropertyLib::SaturationTuller::area_factor_tuller_
diff --git a/Documentation/ProjectFile/properties/property/SaturationTuller/t_characteristic_pore_size.md b/Documentation/ProjectFile/properties/property/SaturationTuller/t_charact
eristic_pore_size.md
new file mode 100644
index 0000000000..97bd5455aa
--- /dev/null
+++ b/Documentation/ProjectFile/properties/property/SaturationTuller/t_characteristic_pore_size.md
@@ -0,0 +1 @@
+\\copydoc MaterialPropertyLib::SaturationTuller::characteristic_pore_size_
diff --git a/Documentation/ProjectFile/properties/property/SaturationTuller/t_pore_area_shape_factor_tuller.md b/Documentation/ProjectFile/properties/property/SaturationTuller/t_po
re_area_shape_factor_tuller.md
new file mode 100644
index 0000000000..604185bd83
--- /dev/null
+++ b/Documentation/ProjectFile/properties/property/SaturationTuller/t_pore_area_shape_factor_tuller.md
@@ -0,0 +1 @@
+\\copydoc MaterialPropertyLib::SaturationTuller::pore_area_shape_factor_tuller_
diff --git a/Documentation/ProjectFile/properties/property/SaturationTuller/t_surface_tension.md b/Documentation/ProjectFile/properties/property/SaturationTuller/t_surface_tension.
md
new file mode 100644
index 0000000000..566410a9d2
--- /dev/null
+++ b/Documentation/ProjectFile/properties/property/SaturationTuller/t_surface_tension.md
@@ -0,0 +1 @@
+\\copydoc MaterialPropertyLib::SaturationTuller::surface_tension_
diff --git a/Documentation/RichardsMechanicsDSMBridgeParameterComparison.md b/Documentation/RichardsMechanicsDSMBridgeParameterComparison.md
new file mode 100644
index 0000000000..10d17a3bd1
--- /dev/null
+++ b/Documentation/RichardsMechanicsDSMBridgeParameterComparison.md
@@ -0,0 +1,320 @@
+# RichardsMechanics DSM Native-vs-Bridge Parameter Comparison
+
+
+## What this note answers
+
+
+This note answers a simple question:
+
+
+Do the current native and bridge parity tests use the same parameters?
+
+
+Short answer:
+
+
+-- the shared OGS problem setup is aligned well
+-- the active constitutive parameter sets are not literally the same
+-- the current green compare test is therefore a reduced overlap test, not exact
+-- same-parameter MCC plasticity parity
+
+
+There is now also a separate aligned unsaturated elastic pair where both sides
+use the same Tuller saturation law. That pair is exact, but it is only an
+elastic saturation-law parity check, not MCC parity.
+
+
+There is also now a second bridge surface, the hybrid
+'RichardsMechanicsNotebookBridge_MCC', which keeps the verified MCC carrier
+surface but also stores notebook auxiliary state. That new hybrid surface is a
+staging step toward notebook-driven parity; it is not yet the final
+notebook-derived constitutive closure.
+
+
+In the current verified hybrid step:
+
+
+-- notebook microstate now feeds back into the returned effective stress through
+the swelling correction
+-- the returned saturation law is still intentionally kept equal to the
+verified MCC carrier surface
+-- the hybrid now also exposes the notebook support-state outputs 'phi', 'n_S',
+'n_L', 'rho_LR', 'omega_L', 'delta_epsilon_sw', and 'sigma_S'
+
+
+## Files compared
+
+
+-- native shell:
+| [mfront_parity_1element_native.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_native.prj)
+-- bridge shell:
+| [mfront_parity_1element_bridge.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_bridge.prj)
+
+
+For the aligned unsaturated elastic check:
+
+
+-- native shell:
+| [mfront_parity_1element_unsat_native.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_native.prj)
+-- bridge shell:
+| [mfront_parity_1element_unsat_bridge.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_bridge.prj)
+
+
+## What is the same
+
+
+These items are the same, or intentionally mapped to the same numerical value,
+in the reduced one-element parity pair.
+
+
+| Item | Native shell | Bridge shell | Meaning |
+| --- | --- | --- | --- |
+| Mesh and boundary meshes | same | same | Same geometry and same boundary subsets |
+| Axial symmetry | 'true' | 'true' | Same kinematics |
+| Time stepping | 't = 0..4', '4 x dt=1' | same | Same time grid |
+| Pressure history | same ramp | same ramp | Same loading path |
+| Initial macro pressure | '0' | '0' | Same macro pressure start |
+| Initial stress 'sigma0' | '(-5e3, -5e3, -5e3, 0)' | same | Same total initial stress |
+| 'YoungModulus' | '52e6' | '52e6' | Same elastic modulus |
+| 'PoissonRatio' | '0.3' | '0.3' | Same elastic Poisson ratio |
+| Mass exchange coefficient | '5e-15' | '5e-15' | Same numerical value, different hookup |
+| 'phi0' | '0.432' | '0.432' | Same initial total porosity |
+| 'phi_tr0' | '0.332' | '0.332' | Same initial transport porosity |
+| 'biot_coefficient' | '0.6' | '0.6' | Same Biot coupling |
+| 'permeability' | '2e-21' | '2e-21' | Same hydraulic conductivity input |
+| 'relative_permeability' | '1' | '1' | Same reduced permeability |
+| Macro saturation law | same van Genuchten law | same | Same macro saturation curve |
+| Bishop law | same cutoff law | same | Same effective-stress reduction law |
+| Liquid viscosity | '1e-3' | '1e-3' | Same viscosity |
+| Liquid density | '1e3' | '1e3' | Same macro liquid density |
+| Medium solid density | '2780' | '2780' | Same medium property |

```

```

+## What is not the same
+
+The main difference is the constitutive law itself.
+
+| Item | Native shell | Bridge shell | Meaning |
+| --- | --- | --- | --- |
+| Constitutive law | 'ModCamClay_semiExpl_constE' | 'RichardsMechanicsNotebookBridge' | Not the same model |
+| Active plastic state | MCC state variables are active | no matching MCC plastic state | Native has plasticity path; bridge does not expose the same one |
+| Active microstate inputs | process-side DSM + native law inputs | reduced bridge microstate inputs | Different parameter surfaces |
+
+## Native MCC parameters vs bridge status
+
+These parameters are active on the native side.
+Some of them are written into the bridge project file too, but they are not
+used by the current bridge law.
+
+| MCC-related item | Native shell | Bridge shell | Status |
+| --- | --- | --- | --- |
+| 'CriticalStateLineSlope' | active | defined only | inactive on bridge |
+| 'SwellingLineSlope' | active | defined only | inactive on bridge |
+| 'VirginConsolidationLineSlope' | active | defined only | inactive on bridge |
+| 'InitialPreConsolidationPressure' | active | defined only | inactive on bridge |
+| 'InitialVolumeRatio' | active | defined only | inactive on bridge |
+| Plastic outputs | requested and available | not available | native only |
+
+So matching numbers in the project files do not automatically mean matching
+constitutive behavior.
+
+## Bridge-only active parameters
+
+These parameters are active only in the current bridge law
+| RichardsMechanicsNotebookBridge.mfront[.../MaterialLib/SolidModels/MFront/RichardsMechanicsNotebookBridge.mfront].
+
+| Bridge-only item | Value | Meaning |
+| --- | --- | --- |
+| 'SwellingSlope' | '0.1' | swelling from micro-porosity change |
+| 'ReferenceLiquidDensityMacro' | '1000.0' | macro liquid density reference |
+| 'ReferenceLiquidDensityMicro' | '2072.8234319102588' | micro liquid density reference |
+| 'ReferenceDensitySolid' | '2470.0' | bridge microstate solid density |
+| 'MicroLiquidDensityA' | '1.3' | bridge EOS parameter |
+| 'MicroLiquidDensityB' | '1.0' | bridge EOS parameter |
+| 'HamakerConstant' | '-6e-20' | bridge micro-potential parameter |
+| 'SpecificSurface' | '100.0' | bridge micro-potential parameter |
+| 'AreaFactorTuller' | '1.0' | bridge saturation parameter |
+| 'PoreAreaShapeFactorTuller' | '0.8584073464102069' | bridge saturation parameter |
+| 'CharacteristicPoreSize' | '1e-5' | bridge saturation parameter |
+| 'SurfaceTension' | '0.0715' | bridge saturation parameter |
+| 'n_l0' | '0.1' | initial bridge micro-liquid content |
+| 'rho_lR0' | '2072.8234319102588' | initial bridge micro-liquid density |
+| 'epsilon_sw0' | '0.0' | initial bridge swelling strain |
+
+## Benchmark-only bridge initial state
+
+The benchmark shell uses a different pressure level than the reduced
+one-element test, so the bridge needs a different pressure-consistent initial
+microstate there.
+
+| Benchmark bridge IC item | Value | Meaning |
+| --- | --- | --- |
+| 'n_l0' | '0.012069019712402708' | pressure-consistent micro-liquid content at benchmark pressure |
+| 'rho_lR0' | '2267.4495975433856' | pressure-consistent micro-liquid density at benchmark pressure |
+
+These values are part of the run-level benchmark-shell fix.
+They do not make the benchmark pair an exact same-model parity pair.
+
+## What exact same-parameter MCC parity would mean
+
+For MCC, the yield function is usually written as
+
+
$$f(q, p, p_c) = q^2 + M^2 (p - p_c),$$

+
+with mean effective stress and deviatoric stress measure
+
+
$$p = -\frac{1}{3} \text{tr}(\boldsymbol{\sigma}_{eff}),$$


$$q = \sqrt{\frac{3}{2} \boldsymbol{s} : \boldsymbol{s}}.$$

+
+An exact same-parameter MCC parity test would therefore need:
+
+-- the same active MCC parameters on both sides
+-- the same active hardening state on both sides
+-- the same stress meaning on both sides
+-- the same plastic-state outputs on both sides
+
+In practice that means the bridge would need to consume and expose at least:
+
+-- 'CriticalStateLineSlope'
+-- 'SwellingLineSlope'
+-- 'VirginConsolidationLineSlope'
+-- 'InitialPreConsolidationPressure'
+-- 'InitialVolumeRatio'
+-- 'EquivalentPlasticStrain'
+-- 'PlasticVolumetricStrain'
+-- 'PreConsolidationPressure'
+-- 'VolumeRatio'
+
+The current bridge does not do that yet.
+
+## What the current green CTest is still good for
+
+The current green compare CTest is still valuable.
+
+It is a valid reduced DSM overlap test for:
+
+-- 'displacement'
+-- 'pressure'
+-- 'sigma'
+-- 'epsilon'
+-- 'saturation'
+-- 'swelling_stress'
+
+It is just important to describe it honestly:
+

```

```

+- good description:
+ *reduced native-vs-bridge overlap "parity
+- bad description:
+ *exact same-parameter MCC plasticity "parity
+
+## Next step for exact MCC parity
+
+1. Extend or replace the current bridge law so it uses the active MCC parameter
+ set.
+2. Expose matching plastic state variables and outputs on the bridge side.
+3. Build a yield-driving shell where both sides use the same active parameter
+ list.
+4. Only then extend the compare test to include plastic-state outputs.
+
+## What the new hybrid bridge changes
+
+The repository now has a new pressure-coupled bridge:
+
+- [RichardsMechanicsNotebookBridge_MCC.mfront](../MaterialLib/SolidModels/MFront/RichardsMechanicsNotebookBridge_MCC.mfront)
+
+This hybrid bridge uses the active MCC carrier parameters
+
+- 'YoungModulus'
+- 'PoissonRatio'
+- 'CriticalStateLineSlope'
+- 'SwellingLineSlope'
+- 'VirginConsolidationLineSlope'
+- 'CharacteristicPreConsolidationPressure'
+- 'ResidualLiquidSaturation'
+- 'ResidualGasSaturation'
+- 'BubblePressure'
+- 'VanGenuchtenExponent_m'
+- 'NotebookSaturationMode'
+
+and also stores notebook auxiliary-state inputs
+
+- 'SwellingSlope'
+- 'MassExchangeCoefficient'
+- 'ReferenceLiquidDensityMacro'
+- 'ReferenceLiquidDensityMicro'
+- 'ReferenceDensitySolid'
+- 'MicroLiquidDensityA'
+- 'MicroLiquidDensityB'
+- 'HamakerConstant'
+- 'SpecificSurface'
+- 'AreaFactorTuller'
+- 'PoreAreaShapeFactorTuller'
+- 'CharacteristicPoreSize'
+- 'SurfaceTension'
+- 'InitialPorosity'
+
+The widened hybrid bridge now exposes these notebook support-state outputs too:
+
+- 'phi'
+- 'n_S'
+- 'n_L'
+- 'rho_LR'
+- 'omega_l'
+- 'delta_epsilon_sw'
+- 'sigma_S'
+
+Those support-state outputs are locked against the committed notebook baseline
+histories in
+[RichardsMechanicsNotebookBridgeMCC.cpp](../Tests/MaterialLib/MFront/RichardsMechanicsNotebookBridgeMCC.cpp).
+- 'n_l0'
+- 'rho_LR0'
+- 'epsilon_sw0'
+
+So the hybrid bridge now has
+
+{
+  \eta_{\text{hybrid}} = (\eta_{\text{MCC}}, \eta_{\text{NB}}),
+}
+
+but in the current verified step it still returns the same constitutive
+surface as the pressure-coupled MCC bridge when notebook coupling is neutral.
+
+That means the remaining open work is no longer "add MCC state to the "bridge.
+That step is already done. The open work is to decide how notebook state should
+feed back into the returned stress, tangents, and possibly saturation without
+breaking the verified MCC carrier surface.
+
+That decision is now partially implemented:
+
+- stress-side notebook feedback is active in the hybrid bridge
+- saturation-side notebook feedback is now also available in an optional
+ reduced-shell Tuller mode
+- the same notebook Tuller saturation branch is now also benchmark-compared on
+ the part-1 shell
+- the neutral benchmark shell for the hybrid bridge is now tested in
+ [mfront_restart_part1_notebook_mcc_bridge.prj](../Tests/Data/RichardsMechanics/mfront_restart_part1_notebook_mcc_bridge.prj)
+
+The reduced same-law MCC + notebook saturation pair is now also tested in
+
+- [mfront_parity_1element_notebook_mcc_tuller_native.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_native.prj)
+- [mfront_parity_1element_notebook_mcc_tuller_bridge.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_bridge.prj)
+
+with the compare CTest
+
+- 'ogs-RichardsMechanics_mfront_parity_1element_notebook_mcc_tuller_compare'
+
+The benchmark same-law MCC + notebook saturation pair is now also tested in
+
+- [mfront_restart_part1_notebook_mcc_tuller_native.prj](../Tests/Data/RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_native.prj)
+- [mfront_restart_part1_notebook_mcc_tuller_bridge.prj](../Tests/Data/RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_bridge.prj)
+
+with the compare CTest
+
+- 'ogs-RichardsMechanics_mfront_restart_part1_notebook_mcc_tuller_compare'
+
+So the narrower remaining open question is now:
+
+- which notebook-owned non-neutral constitutive branches should be widened
+ beyond the now-tested Tuller saturation branch
+
+## What the new project files test
+

```

```

+| File | Purpose | Exact test meaning |
+| --- | --- | --- |
+| [mfront_parity_1element_notebook_mcc_tuller_native.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_native.prj) | Reduced native MCC shell with native MPL Tuller saturation | Native reduced reference for notebook-Tuller saturation on the one-element shell |
+| [mfront_parity_1element_notebook_mcc_tuller_bridge.prj](../Tests/Data/RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_bridge.prj) | Reduced hybrid bridge shell with 'NotebookSaturationMode = 1' | Bridge reduced reference for the same notebook-Tuller saturation branch |
+| [mfront_restart_part1_notebook_mcc_tuller_native.prj](../Tests/Data/RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_native.prj) | Native benchmark part-1 MCC shell with native MPL Tuller saturation | Native benchmark reference for the notebook-Tuller saturation branch on the full part-1 load scale |
+| [mfront_restart_part1_notebook_mcc_tuller_bridge.prj](../Tests/Data/RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_bridge.prj) | Hybrid bridge benchmark part-1 shell with 'NotebookSaturationMode = 1' and neutral notebook swelling/mass exchange | Bridge benchmark reference for the same notebook-Tuller saturation branch |
+
+For the benchmark Tuller pair, the same native part-1 geometry and load range
+are kept, but the branch is substepped with fixed '\Delta t = 1' and only the
+start and end states are compared. That keeps the benchmark-scale load path
+while making the non-neutral saturation branch loadable on both sides.
diff --git a/Documentation/RichardsMechanicsDSMBridgeParityPlan.md b/Documentation/RichardsMechanicsDSMBridgeParityPlan.md
new file mode 100644
index 0000000000..f4c5147b94
--- /dev/null
+++ b/Documentation/RichardsMechanicsDSMBridgeParityPlan.md
@@ -0,0 +1,1564 @@
+# RichardsMechanics DSM Native-vs-MFront Parity Plan
+
+## Goal
+
+Build an honest native-vs-MFront comparison for the RichardsMechanics
+double-structure model (DSM) inside the same CTest slice.
+
+The finished state should contain:
+
+- one native DSM run
+- one MFront bridge DSM run
+- one native-vs-bridge comparison test on the same output fields and time steps
+- zero tolerance where that is realistic, and only machine-scale tolerance
+- where exact bitwise equality is not realistic
+
+## Short version
+
+Two things are already true:
+
+- the reduced one-element native-vs-bridge compare works and is green
+- the saturated elastic one-element native-vs-bridge compare also works and is exact
+- the unsaturated elastic one-element native-vs-bridge compare also works and is exact once the native side uses the same Tuller saturation law as the notebook bridge
+- a reduced same-law MCC native-vs-bridge compare also works and is green once the bridge side uses a pressure-coupled variant of 'ModCamClay_semiExpL_constE'
+- a benchmark same-law MCC native-vs-bridge compare now also works and is green on the native part-1 shell
+- a widened notebook-MCC hybrid bridge can now carry notebook auxiliary state explicitly while still collapsing exactly to the verified MCC bridge when notebook coupling is neutral
+- the hybrid bridge now also feeds notebook swelling back into the returned effective stress and 'dsigma/dp_L', while still keeping the verified MCC saturation carrier surface unchanged
+- a neutral notebook-MCC benchmark compare now also works and is green on the native part-1 shell
+- a benchmark same-law MCC + notebook Tuller saturation compare now also works and is green on the native part-1 shell once that non-neutral saturation branch is substepped with '\Delta t = 1' over 't = 0..1000'
+
+One thing is still open:
+
+- full notebook-to-native DSM parity is still open, because the current notebook widening still covers only the Tuller saturation branch plus the swelling stress correction, not the full notebook-owned constitutive closure
+
+So the remaining problem is no longer "can the benchmark bridge run?" and it is no longer "can RM carry the native MFront MCC law through the pressure-coupled bridge?". The remaining problem is how far the notebook-derived bridge should be widened toward that now-verified MCC benchmark surface.
+
+## What is already verified
+
+As of 2026-03-27 on branch 'dsm-nb-mfront-transition':
+
+- the repository configures and builds with 'OGS_USE_MFRONT=ON'
+- this focused test slice passes:
+  - 'ogs-RichardsMechanics/mfront_restart_part1'
+  - 'ogs-RichardsMechanics/mfront_restart_part2'
+  - 'ogs-RichardsMechanics/mfront_restart_part1_rm_bridge'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_native'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_bridge'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_compare'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_elastic_compare'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_unsat_compare'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_mcc_compare'
+  - 'ogs-RichardsMechanics/mfront_restart_part1_mcc_compare'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_notebook_mcc_bridge'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_notebook_mcc_compare'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_native'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_bridge'
+  - 'ogs-RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_compare'
+  - 'ogs-RichardsMechanics/mfront_restart_part1_notebook_mcc_bridge'
+  - 'ogs-RichardsMechanics/mfront_restart_part1_notebook_mcc_compare'
+  - 'ogs-RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_native'
+  - 'ogs-RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_bridge'
+  - 'ogs-RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_compare'
+  - 'MaterialLib_RMBridgeMFront_NotebookMCC.NeutralNotebookStateMatchesVerifiedMCCBridge'
+  - 'MaterialLib_RMBridgeMFront_NotebookMCC.SwellingFeedbackChangesStressButKeepsCarrierSaturation'
+  - 'MaterialLib_RMBridgeMFront_NotebookMCC.NotebookSaturationModeMatchesTullerLawAndKeepsStressSurface'
+
+- the RM pressure-coupled carrier now keeps the bridge saturation derivative with respect to strain 'dS/dVarepsilon', and the pressure-equation displacement block now consumes that term instead of dropping it
+- the native MPL now has 'SaturationTuller', so RM can use the same notebook saturation law on the native side when that is the honest comparison target
+
+## Reduced one-element parity status
+
+The reduced one-element pair is the current parity gate.
+
+Compared fields:
+
+- 'displacement'

```

```

+-- 'pressure'
+-- 'sigma'
+-- 'epsilon'
+-- 'saturation'
+-- 'swelling_stress'
+
+Result:
+
+-- 'displacement': exact
+-- 'pressure': exact
+-- 'epsilon': exact
+-- 'saturation': exact
+-- 'swelling_stress': exact
+-- 'sigma': machine-scale residue only
+
+Observed 'sigma' residue:
+
+-- absolute max norm about '1.8e-12'
+-- relative max norm about '3.6e-16'
+
+So the reduced one-element compare is closed for practical purposes.
+The remaining 'sigma' difference is a floating-point issue, not a model-parity
+issue.
+
+## Saturated elastic parity status
+
+There is now also a separate saturated elastic one-element parity gate.
+
+This pair uses:
+
+-- the same one-element RM shell on both sides
+-- native 'LinearElasticIsotropic' on one side
+-- 'RichardsMechanicsNotebookBridge' with 'SwellingSlope = 0' on the other side
+-- a shared positive pressure ramp, so both sides stay on the saturated branch
+-- with 'S_L = 1'
+
+Compared fields:
+
+-- 'displacement'
+-- 'pressure'
+-- 'sigma'
+-- 'epsilon'
+-- 'saturation'
+-- 'velocity'
+
+Result:
+
+-- all compared fields match exactly on timesteps '0' through '4'
+
+What this means:
+
+-- the basic saturated elastic pressure-coupled RM bridge contract is working on
+-- this shared shell
+-- the remaining benchmark mismatch is therefore not explained by a generic
+-- failure of the bridge on the simplest elastic saturated branch
+-- the open benchmark problem must involve constitutive, unsaturated, or other
+-- process-level features beyond this branch
+
+## Unsaturated elastic parity status
+
+The first unsaturated elastic probe was useful because it showed that the old
+native shell and the notebook bridge were not using the same saturation law.
+
+That first probe used:
+
+-- the same geometry on both sides
+-- the same elastic stiffness on both sides
+-- zero displacement on all boundaries, so 'u = 0' and '\varepsilon = 0'
+-- the same prescribed liquid-pressure history
+-- 'p_L = (0, -2.5\times 10^5, -5\times 10^5, -7.5\times 10^5, -10^6)\,\text{Pa}'
+-- native 'LinearElasticIsotropic' with the native medium saturation law
+-- bridge 'RichardsMechanicsNotebookBridge' with
+-- 'SwellingSlope = 0' and 'MassExchangeCoefficient = 0'
+
+This removes MCC plasticity from the picture. In that first probe, any
+mismatch had to come from saturation, Bishop weighting, or the RM
+pressure-coupled carrier.
+
+### What the first probe showed
+
+At first, the native side still used van Genuchten while the bridge used the
+notebook Tuller law.
+
+On the native side, the medium saturation was
+
+{\[
+S_L^{\text{nat}} = S_{\text{VG}}(p_{\text{cap}}),
+\\quad
+p_{\text{cap}} = -p_L,
+\\]
+
+while on the bridge side the direct material response followed
+for 'p_L < 0'
+
+{\[
+S_L^{\text{bridge}}(p_L) = 1 - \exp\!\left(-\frac{A}{p_L^2}\right),
+\\]
+
+{\[
+A = \frac{4\,\beta_T\,\gamma^2}{a_T\,r_c^2},
+\\]
+
+where in the current bridge parameters
+
+{\[
+\beta_T = 0.8584073464102069,
+\\quad
+\gamma = 0.0715,
+\\quad
+a_T = 1,
+\\quad
+r_c = 10^{-5}\,\text{m}.
+\\]
+
+After fixing the RM secondary-variable/state path so that pressure-coupled
+materials keep their returned 'S_L' instead of falling back to the medium law,
+that first unsaturated elastic probe showed:

```



```

+
+ 'pressure': exact
+ 'displacement': exact
+ 'epsilon': exact
+ 'sigma': exact
+ 'velocity': exact
+ 'saturation': not equal from 'ts_1' onward
+
+Observed 'saturation' values:
+
+ 'ts_0': native '1', bridge '1'
+ 'ts_1': native '0.99956534554', bridge '0.0028046311451'
+ 'ts_2': native '0.99862233608', bridge '0.00070189642845'
+ 'ts_3': native '0.9972984069', bridge '0.00031201481239'
+ 'ts_4': native '0.99564897855', bridge '0.00017552031277'
+
+That result did not mean the bridge concept was wrong. It meant the native and
+bridge probes were still asking different saturation questions.
+
+### Alignment step that closes this gap
+
+To make the comparison honest, the native side is now aligned to the notebook
+law too.
+
+The native medium can now use the same Tuller law through the new
+'SaturationTuller' MPL property:
+
+ \[
+ S_L^{(Tuller)}(p_{\text{cap}}) =
+ \begin{cases}
+ 1 - \exp\left(-\text{dfrac}{A}(p_{\text{cap}}^2)\right), & p_{\text{cap}} > 0, \\
+ 1, & p_{\text{cap}} \leq 0,
+ \end{cases}
+ \quad
+ A = \frac{4\gamma\beta_T\gamma^2}{a_T r_c^2}.
+ \]
+
+Because 'p_cap' = -p_L', this is the same scalar law as the notebook bridge.
+
+There is now a dedicated unsaturated elastic one-element CTest pair:
+
+ native side:
+ 'LinearElasticIsotropic' + 'SaturationTuller'
+ bridge side:
+ 'RichardsMechanicsNotebookBridge' with
+ 'SwellingSlope' = 0', 'MassExchangeCoefficient' = 0'
+ same pressure path:
+ 'p_L' = (0, -2.5times 10^5, -5times 10^5, -7.5times 10^5, -10^6)\text{Pa}
+
+Compared fields:
+
+ 'displacement'
+ 'pressure'
+ 'sigma'
+ 'epsilon'
+ 'saturation'
+ 'velocity'
+
+Result:
+
+ all compared fields match exactly on timesteps '0' through '4'
+
+So the old unsaturated elastic mismatch is now closed. It was caused by using
+different saturation laws, not by a generic RM-vs-bridge inconsistency on the
+shared elastic unsaturated branch.
+
+## Reduced same-law MCC parity status
+
+There is now also a reduced one-element same-law MCC gate.
+
+This pair uses:
+
+ the same one-element RM shell on both sides
+ the same van Genuchten medium saturation law on both sides
+ native 'ModCamClay_semiExpL_constE' on one side
+ 'ModCamClay_semiExpL_constE_pressureCoupled' under
+ 'MFrontRichardsMechanics' on the other side
+
+Compared fields:
+
+ 'displacement'
+ 'pressure'
+ 'sigma'
+ 'epsilon'
+ 'saturation'
+ 'velocity'
+ 'ElasticStrain'
+ 'EquivalentPlasticStrain'
+ 'PreConsolidationPressure'
+ 'PlasticVolumetricStrain'
+ 'VolumeRatio'
+
+Result:
+
+ all compared history fields match up to machine-scale floating-point residue
+ the project-level parity CTest is green
+ the focused material-point test is green
+
+The observed residue is only round-off:
+
+ 'sigma' closes with '5e-11' absolute tolerance in the project-level compare
+ the material-point tangent match closes with '1e-8' absolute tolerance
+
+The focused material-point check also verifies the bridge-side pressure blocks:
+
+ 'od/dp_L' = 0
+ 'dS_L/d = 0
+ 'dS_L/dp_L' matches the native van Genuchten formula
+
+What this means:
+
+ native MFront MCC already works through the RM pressure-coupled bridge on a
+ reduced shell
+ no MCC law had to be reimplemented on the native OGS side
+ the remaining benchmark ambiguity is now narrower than before
+ the tracked benchmark gap is no longer a generic question of whether RM can
+ carry native 'ModCamClay_semiExpL_constE' through

```

```

+ 'MFrontRichardsMechanics'
+
+### RM carrier bug that was hiding this
+
+One RM bug was also found and fixed during this probe.
+
+Before the fix, the bridge run exported the medium saturation in the
+secondary-variable path, even after the constitutive update had returned a
+different pressure-coupled saturation. That made the bridge output look
+artificially close to the native output on unsaturated elastic shells.
+
+The fix is in
+ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h:
+
+the secondary-variable path now rebuilds the local
+ 'dS_L/dp_{cap}', '\Delta S_L / \Delta p_{cap}', and 'd\chi/dS_L' terms
+ after the constitutive update, it reapplies the pressure-coupled solid data
+ so the stored/output 'S_L' matches the bridge response
+
+This fix does not close the benchmark mismatch by itself. What it does is make
+the RM bridge state and output honest on unsaturated pressure-coupled runs.
+
+### Benchmark-shell same-law MCC parity status
+
+The tracked benchmark bridge shell now uses the same available MCC law as the
+native benchmark:
+
+native benchmark law:
+ 'ModCamClay_semiExpL_constE'
+ bridge benchmark law:
+ 'ModCamClay_semiExpL_constE_pressureCoupled'
+
+The dedicated benchmark compare CTest runs both project files on the native
+part-1 shell and compares:
+
+'displacement'
+'pressure'
+'sigma'
+'epsilon'
+'saturation'
+'velocity'
+'ElasticStrain'
+'EquivalentPlasticStrain'
+'PreConsolidationPressure'
+'PlasticVolumetricStrain'
+'VolumeRatio'
+'swelling_stress'
+'transport_porosity'
+'dry_density_solid'
+
+Result:
+
+at 'ts_0'
+ - all compared fields are exact except for machine-scale 'sigma'
+at 'ts_1'
+ - all compared fields still agree up to machine-scale residue only
+ - 'pressure' closes with absolute max norm about '1.82e-12'
+ - 'sigma' closes with absolute max norm about '5.82e-11'
+
+So the same-law MCC benchmark parity task is now closed in practice. The new
+project-level gate is:
+
+'ogs=RichardsMechanics_mfront_restart_part1_mcc_compare'
+
+What remains open is not this same-law benchmark surface. What remains open is
+the notebook-derived bridge law and how far it should be widened toward this
+now-verified MCC benchmark boundary.
+
+### Notebook-MCC hybrid bridge status
+
+There is now a first widening step between the old reduced notebook bridge and
+the verified MCC bridge.
+
+New behaviour:
+
+'MaterialLib/SolidModels/MFront/RichardsMechanicsNotebookBridge_MCC.mfront'
+
+New regression checks:
+
+material-point:
+ 'Tests/MaterialLib/MFront/RichardsMechanicsNotebookBridgeMCC.cpp'
+ project-level reduced shell:
+ 'Tests/Data/RichardsMechanics/mfront_parity_1element_notebook_mcc_bridge.prj'
+ compare driver:
+ 'scripts/cmake/test/CompareRichardsMechanicsMFrontNotebookMCCParity.cmake'
+
+### What this new bridge does
+
+It starts from the verified pressure-coupled MCC bridge surface and adds the
+notebook auxiliary state variables as persistent process-visible state:
+
+
+{
+  \eta_{MCC}
+  =
+  {
+    \left(
+      \varpi_{\epsilon}^{\{e\}},
+      \Lambda_p,
+      p_c,
+      \varpi_{v^p},
+      v^r
+    \right),
+  }
+  {
+    \eta_{NB}
+    =
+    {
+      \left(
+        n_l,
+        \rho_{\{lR\}},
+        \varpi_{\{sw\}},
+        \phi_M,
+        \phi_M,
+        \mu_{\{lR\}},
+        \hat{\rho}_l
+      \right),
+    }
+  }
+}

```

```

+The current hybrid bridge stores
+
+\[
+\eta_{\text{hybrid}} = (\eta_{\text{MCC}}, \eta_{\text{NB}}),
+\]
+
+but its returned constitutive surface is still the verified MCC one:
+
+\[
+\mathcal{C}_{\text{hybrid}}^{(\emptyset)}(\nu, p_L, \eta_{\text{hybrid}})
+=
+\mathcal{C}_{\text{MCC}}^{(pc)}(\nu, p_L, \eta_{\text{MCC}}).
+\]
+
+The first hybrid step only widened the state. The current hybrid step adds the
+first constitutive feedback too:
+
+\[
+\boldsymbol{\sigma}_{\text{eff}}^{\text{hybrid}}
+=
+\boldsymbol{\sigma}_{\text{eff}}^{\text{MCC}}
+- K, \Delta \nu_{\text{sw}}, \mathbf{I},
+\quad
+\Delta \nu_{\text{sw}}
+=
+\text{SwellingSlope}, (n_L^{\text{new}} - n_L^{\text{old}}).
+\]
+
+At the moment, the saturation surface is still intentionally kept equal to the
+verified MCC carrier surface:
+
+\[
+S_L^{\text{hybrid}} = S_L^{\text{MCC}},
+\quad
+\frac{\partial S_L^{\text{hybrid}}}{\partial p_L}
+=
+\frac{\partial S_L^{\text{MCC}}}{\partial p_L},
+\quad
+\frac{\partial S_L^{\text{hybrid}}}{\partial \nu}
+= 0.
+\]
+
+### What is already verified for the hybrid bridge
+
+When notebook coupling is neutral,
+
+\[
+\text{SwellingSlope} = 0,
+\quad
+\text{MassExchangeCoefficient} = 0,
+\]
+
+the hybrid bridge collapses to the verified MCC bridge on the same loading
+path.
+
+Verified equal outputs:
+
+- 'stress'
+- 'dStress_dStrain'
+- 'dStress_dLiquidPressure'
+- 'saturation'
+- 'dSaturation_dStrain'
+- 'dSaturation_dLiquidPressure'
+- 'ElasticStrain'
+- 'EquivalentPlasticStrain'
+- 'PreconsolidationPressure'
+- 'PlasticVolumetricStrain'
+- 'VolumeRatio'
+
+Verified carried notebook state:
+
+- 'n_L'
+- 'rho_LR'
+- 'epsilon_sw'
+- 'phi_m'
+- 'phi_M'
+- 'mu_LR'
+- 'rho_L_hat'
+
+Verified hybrid constitutive feedback:
+
+- the returned stress differs from the verified MCC bridge exactly by the
+ isotropic notebook swelling correction
+- the returned 'dStress_dLiquidPressure' matches a direct finite-difference
+ probe on the active-swelling hybrid branch
+- the returned 'saturation', 'dSaturation_dLiquidPressure', and
+ 'dSaturation_dStrain' stay identical to the verified MCC bridge on that same
+ active-swelling branch
+
+So this first widening step proves a narrow but important point:
+
+- notebook state can be added to the bridge without breaking the verified MCC
+ carrier surface
+- notebook swelling can be fed back into the returned stress without breaking
+ the verified MCC carrier saturation surface
+- the benchmark shell can still collapse exactly to the verified MCC benchmark
+ surface when notebook coupling is kept neutral
+
+### The pressure-coupled contract in plain language
+
+The key question is simple:
+
+When the bridge returns stress and pressure derivatives, does RM interpret them
+the same way the bridge means them?
+
+### Continuum form
+
+The quasi-static momentum balance is
+
+\[
+\nabla \cdot \boldsymbol{\sigma}_{\text{tot}} + \rho \mathbf{b} = \mathbf{0}.
+\]
+
+The liquid mass balance is
+
+\[
+\dot{m}_L + \nabla \cdot \mathbf{w}_L = q_L.
\]

```

```

+\\
+
+In RichardsMechanics, total stress is built from effective stress by
+
+\\[
+\\boldsymbol{\\sigma}_{\\text{tot}}
+=
+\\boldsymbol{\\sigma}_{\\text{eff}}
+- \\alpha \\chi(S_L) p_L \\mathbf{I}.
+\\]
+
+So if the bridge returns effective stress, RM should add the pore-pressure
+contribution once.
+
+If the bridge already returns total stress, but RM still applies the same
+pressure correction, the pore-pressure term is counted twice.
+
+### Weak form
+
+The weak form of momentum is
+
+\\[
+\\int_{\\Omega} \\boldsymbol{\\varepsilon}(\\delta \\mathbf{u}) :
+\\boldsymbol{\\sigma}_{\\text{tot}} \\, d\\Omega
+- \\int_{\\Omega} \\delta \\mathbf{u} \\cdot \\rho \\mathbf{b} \\, d\\Omega
+- \\int_{\\Gamma_t} \\delta \\mathbf{u} \\cdot \\bar{\\mathbf{t}} \\, d\\Gamma = 0.
+\\]
+
+The weak form of liquid mass balance is
+
+\\[
+\\int_{\\Omega} \\delta p \\, \\dot{m}_L \\, d\\Omega
++ \\int_{\\Omega} \\nabla \\delta p \\cdot \\mathbf{w}_L \\, d\\Omega
+- \\int_{\\Omega} \\delta p \\, q_L \\, d\\Omega = 0.
+\\]
+
+The important point is that the momentum equation depends on pressure through
+'\\sigma_tot'.
+So a wrong stress meaning or a wrong pressure derivative changes the coupled
+'u-p' Jacobian.
+
+### Semi-discrete form
+
+With standard FE interpolation,
+
+\\[
+\\mathbf{u}_h = N_u \\mathbf{d},
+\\quad
+p_h = N_p \\mathbf{p},
+\\quad
+\\boldsymbol{\\varepsilon}(\\mathbf{u}_h) = B \\mathbf{d}.
+\\]
+
+The momentum residual is
+
+\\[
+R_u(\\mathbf{d}, \\mathbf{p})
+=
+\\int_{\\Omega} B^T \\boldsymbol{\\sigma}_{\\text{tot}} \\, d\\Omega - f_{\\text{ext}}.
+\\]
+
+The pressure-to-momentum Jacobian block is
+
+\\[
+K_{up}
+=
+\\frac{\\partial R_u}{\\partial \\mathbf{p}}
+=
+\\int_{\\Omega} B^T
+\\left(
+\\frac{\\partial \\boldsymbol{\\sigma}_{\\text{tot}}}{\\partial p_L}
+\\right)
+N_p \\, d\\Omega.
+\\]
+
+The pressure equation also has a displacement-coupling block. If the liquid
+storage contains a term like
+
+\\[
+m_L \\supset \\phi \\rho_{LR} S_L,
+\\]
+then linearizing with respect to displacement gives the saturation-strain part
+
+\\[
+K_{pu}^{(S)}
+=
+\\frac{\\partial R_p}{\\partial \\mathbf{d}}
+\\supset
+\\int_{\\Omega} N_p^T \\phi \\rho_{LR}
+\\left(
+\\frac{\\partial S_L}{\\partial \\boldsymbol{\\varepsilon}}
+\\right)
+B \\, d\\Omega.
+\\]
+
+That first 'dS/d\\varepsilon' term was a real RM-side carrier omission. It is
+now assembled.
+
+This gave one useful negative result during the earlier notebook-bridge
+investigation:
+
+restoring the 'dS/d\\varepsilon' carrier term was a real fix
+but it did not remove the old notebook-bridge benchmark gap by itself
+
+So a notebook-derived benchmark gap can still come from a wrong meaning of:
+
+returned stress
+'dS/d\\varepsilon'
+'d\\sigma/dp_L'
+'dS_L/dp_L'
+
+even if the local bridge solve itself is finite and stable.
+
+## What the current parity gate does and does not prove
+
+## What it proves
+
+The current one-element compare proves that the native and bridge paths agree
+on the reduced shared shell that is actually being tested.

```

```

+
+### What it does not prove
+
+It does not prove:
+
+ full notebook-to-native constitutive equivalence
+ that the notebook-derived bridge law has already reached the same active
+ constitutive closure as the verified MCC bridge
+
+### Parameter status
+
+For a parameter-by-parameter comparison of the reduced one-element pair, see:
+
+ [RichardsMechanicsDSMBridgeParameterComparison.md](./RichardsMechanicsDSMBridgeParameterComparison.md)
+
+Short version:
+
+ the shared OGS setup is aligned well enough for the reduced parity CTest
+ the active constitutive parameter sets are not literally the same
+ the current green compare test is not an exact same-parameter MCC plasticity
+ proof
+
+### Work packages
+
+### WP1: Keep the reduced one-element parity slice green
+
+Status:
+
+ complete
+
+Acceptance criteria:
+
+ native one-element run stays green
+ bridge one-element run stays green
+ compare CTest stays green
+ only machine-scale tolerance is used where exact equality is not realistic
+
+### WP2: Keep the benchmark shell loadable
+
+Status:
+
+ complete for run-level loadability
+
+Guardrails:
+
+ keep the pressure-consistent benchmark bridge IC
+ keep the bridge microstate fallback
+ keep the benchmark damping unless tests show it is safe to remove
+ keep the direct benchmark-pressure bridge unit tests green
+
+### WP3: Close the same-law benchmark quantitative gap
+
+Status:
+
+ complete
+
+Completed result:
+
+ compare clean native and bridge benchmark outputs on the part-1 shell
+ enforce the comparison with a dedicated CTest
+ keep only machine-scale tolerance for 'pressure' and 'sigma'
+
+### WP4: Add the benchmark comparison CTest
+
+Status:
+
+ complete
+
+Delivered test:
+
+ 'ogs-RichardsMechanics_mfront_restart_part1_mcc_compare'
+
+### WP5: Add a notebook-MCC hybrid bridge that preserves the verified MCC surface
+
+Status:
+
+ complete
+
+Delivered result:
+
+ add 'RichardsMechanicsNotebookBridge_MCC'
+ carry notebook auxiliary state explicitly in the pressure-coupled bridge
+ prove, with a material-point test and a reduced-shell CTest, that the hybrid
+ bridge collapses to the verified MCC bridge when notebook coupling is neutral
+
+### WP6: Feed notebook swelling back into the constitutive response
+
+Status:
+
+ complete
+
+Delivered result:
+
+ keep the verified MCC plastic core
+ add the notebook isotropic swelling correction to the returned effective
+ stress
+ return the matching 'dStress_dLiquidPressure'
+ lock the active-swelling branch with a direct material-point regression
+
+Target form:
+
+
+
$$\begin{aligned} & \sigma_{\text{eff}}^{\text{hybrid}} \\ &= \\ & \sigma_{\text{eff}}^{\text{MCC}} \\ &+ K_{\text{sw}}(\eta_{\text{NB}}) \Delta \epsilon_{\text{sw}} \mathbf{I} \end{aligned}$$

+
+Current verified boundary:
+
+ the stress correction is active and tested
+ the pressure tangent is active and tested
+ the hybrid still preserves the verified MCC saturation carrier surface
+
+### WP7: Decide whether notebook microstate should modify saturation or stay auxiliary
+
+Status:

```

```

+
+← complete for the current reduced hybrid boundary
+
+Decision for the current branch:
+
+← keep the returned saturation law on the verified MCC carrier surface for now
+← keep notebook microstate as a stress-side widening step first
+← prove on a direct active-swellng material-point test that 'S_L',
+ 'dS_L/dp_L', and 'dS_L/d\varepsilon' stay equal to the verified MCC bridge
+
+What remains open later:
+
+← whether a future notebook-driven branch should also widen the returned
+ saturation law on the benchmark shell
+
+### WP8: Benchmark the widened notebook hybrid against the verified MCC boundary
+
+Status:
+
+← complete for the neutral notebook branch
+
+Delivered result:
+
+← add 'mfront_restart_part1_notebook_mcc_bridge.prj'
+← keep notebook coupling neutral on that benchmark shell
+← compare the shared benchmark fields against the native part-1 surface with
+ 'ogs-RichardsMechanics_mfront_restart_part1_notebook_mcc_compare'
+
+What remains open later:
+
+← rerun the benchmark on a non-neutral notebook branch once the intended final
+ notebook constitutive feedback is defined
+
+### WP9: Add a reduced same-law MCC + notebook saturation parity gate
+
+Status:
+
+← complete
+
+Delivered result:
+
+← add an optional notebook saturation mode to
+ 'RichardsMechanicsNotebookBridge_MCC'
+← keep the existing neutral benchmark branch unchanged by default
+← activate the notebook Tuller saturation law only when
+ 'NotebookSaturationMode = 1'
+← lock that mode with a direct material-point regression
+← add the reduced native-vs-bridge parity pair
+ 'mfront_parity_1element_notebook_mcc_tuller_native.prj' and
+ 'mfront_parity_1element_notebook_mcc_tuller_bridge.prj'
+← enforce the reduced compare with
+ 'ogs-RichardsMechanics_mfront_parity_1element_notebook_mcc_tuller_compare'
+
+Current verified boundary:
+
+← reduced same-law MCC + notebook Tuller saturation parity is green
+← the old neutral notebook benchmark path is still green
+← the optional notebook saturation mode introduces only machine-scale residue
+ in 'pressure' and 'sigma' on the reduced shell
+
+What remains open later:
+
+← benchmark-sized notebook widening beyond the current Tuller-only saturation
+ branch
+
+### WP10: Add a benchmark same-law MCC + notebook saturation parity gate
+
+Status:
+
+← complete for the current notebook Tuller saturation branch
+
+Delivered result:
+
+← add the native benchmark shell
+ 'mfront_restart_part1_notebook_mcc_tuller_native.prj'
+← add the bridge benchmark shell
+ 'mfront_restart_part1_notebook_mcc_tuller_bridge.prj'
+← keep the same native part-1 geometry, loads, and MCC material parameters
+← activate the notebook Tuller saturation branch on the bridge side with
+ 'NotebookSaturationMode = 1'
+← use the same Tuller saturation law and Bishop cutoff on the native side
+← substep that non-neutral saturation benchmark with fixed '\Delta t = 1'
+ over 't = 0..1000'
+← emit only the start and end states on that benchmark pair
+← enforce the benchmark compare with
+ 'ogs-RichardsMechanics_mfront_restart_part1_notebook_mcc_tuller_compare'
+
+Current verified boundary:
+
+← the native Tuller benchmark shell runs
+← the bridge Tuller benchmark shell runs
+← the native-vs-bridge benchmark compare is green
+← the final-time benchmark residue is small and measured, not guessed:
+ - 'displacement': about '9.75e-14'
+ - 'pressure': about '8.02e-7' absolute and '2.05e-10' relative
+ - 'sigma': about '8.15e-10'
+ - 'saturation': about '1.97e-13'
+ - 'dry_density_solid': about '2.99e-10'
+
+What remains open later:
+
+← notebook-owned non-neutral widening beyond the current Tuller saturation
+ branch and swelling-stress correction
+
+### WP11: Expose notebook support state on the hybrid MCC bridge
+
+Status:
+
+← complete
+
+Delivered result:
+
+← widen 'RichardsMechanicsNotebookBridge_MCC' so it exposes the notebook
+ support-state outputs 'phi', 'n_S', 'n_L', 'rho_LR', 'omega_L',
+ 'delta_epsilon_sw', and 'sigma_S'
+← keep these outputs notebook-owned even when the returned RM carrier
+ saturation stays on the verified MCC surface

```

```

+ reuse the committed notebook overlap-transfer and anchored strain-coupled
+ baseline CSVs as the direct support-state reference surface
+
+Delivered tests:
+
+ 'MaterialLib_RMBridgeMFront_NotebookMCC.NotebookSupportStateMatchesOverlapTransferBaseline'
+ 'MaterialLib_RMBridgeMFront_NotebookMCC.NotebookSupportStateMatchesStrainCoupledBaseline'
+
+Current verified boundary:
+
+ the hybrid bridge now exposes the full notebook support state needed by the
+ committed notebook overlap-transfer and strain-coupled histories
+ the direct notebook support-state regressions are green
+ the reduced and benchmark notebook-MCC compare CTests remain green after the
+ widening step
+
+What remains open later:
+
+ notebook-owned non-neutral widening beyond the current support-state,
+ swelling-stress, and Tuller-saturation steps
+
+## BEACON smoke native-vs-bridge check
+
+Status:
+
+ completed as a direct run-and-compare study
+ not promoted to CTest parity yet
+ native branch smoke runs succeed in a separate native build
+ bridge smoke copies run in the current MFront work tree
+
+What was run:
+
+ native side: direct 'ogs' runs from branch 'dsm-nb-transition'
+ at commit 'd46ellac00' using
+ - 'beacon_1a01_notebook_smoke.prj'
+ - 'beacon_1b_notebook_smoke.prj'
+ - 'beacon_1c_notebook_smoke.prj'
+ bridge side: direct 'ogs' runs from branch 'dsm-nb-mfront-transition'
+ at commit 'c349ee2713' using
+ - 'Tests/Data/RichardsMechanics/beacon_1a01_notebook_mcc_bridge.prj'
+ - 'Tests/Data/RichardsMechanics/beacon_1b_notebook_mcc_bridge.prj'
+ - 'Tests/Data/RichardsMechanics/beacon_1c_notebook_mcc_bridge.prj'
+ comparison time: 't = 1000 s'
+ comparison tool: 'vtkdiff'
+ overlap fields:
+ - '1a01': 'displacement', 'pressure', 'saturation', 'swelling_stress',
+   'sigma'
+ - '1b': 'displacement', 'pressure', 'saturation', 'swelling_stress',
+   'sigma'
+ - '1c': 'displacement', 'pressure', 'saturation', 'porosity',
+   'transport_porosity', 'swelling_stress', 'sigma'
+
+Bridge surface used:
+
+ 'RichardsMechanicsNotebookBridge_MCC'
+ notebook saturation mode left on the native Van Genuchten carrier
+ notebook swelling feedback kept neutral for this smoke check
+ preconsolidation pressure kept high so the bridge stays elastic on these
+ smoke cases
+
+Comparison summary:
+
+| Case | Native branch run | MFront branch run | Exact overlap fields | Nonzero overlap mismatch | Likely reason | Suggested solution |
+| --- | --- | --- | --- | --- | --- | --- |
+| '1a01' | yes | yes | 'displacement', 'swelling_stress', 'sigma' | 'pressure': abs max '4.61e2', rel max '4.78e-4'; 'saturation': abs max '1.84e-4', rel max '2.52e-4' | Only the top pressure is fixed, so the interior state exposes the remaining storage-carrier difference between the native VK path and the bridge notebook state path. | Add a BEACON-specific notebook storage-carrier mode before making '1a01' a parity gate. A secondary convenience step is to port 'potential_exchange' into the MFront tree so both project families can be run by one executable. |
+| '1b' | yes | yes | 'displacement', 'pressure', 'saturation', 'swelling_stress', 'sigma' | none on the compared overlap fields | One medium and two pressure Dirichlet boundaries suppress the remaining carrier ambiguity. | Promote '1b' to the first exact BEACON native-vs-bridge compare gate. |
+| '1c' | yes | yes | 'displacement', 'porosity', 'swelling_stress', 'sigma' | 'pressure': abs max '7.12e1', rel max '3.15e-4'; 'saturation': abs max '2.86e-5', rel max '3.00e-5'; 'transport_porosity': abs max '2.00e-2', rel max '8.00e-2' | The native VK path has a dedicated heterogeneous 'transport_porosity' split update; the bridge path still uses the generic pressure-coupled carrier there. | Widen the bridge/process contract so the bridge can drive 'phi_M' or 'transport_porosity' explicitly on heterogeneous VK cases. A secondary convenience step is to port 'potential_exchange' into the MFront tree so both project families can be run by one executable. |
+
+Main conclusion:
+
+ the bridge BEACON copies do run for '1a01', '1b', and '1c'
+ the native VK smoke decks also run, but in the separate native branch build
+ rather than in the MFront executable
+ the mismatch is not a generic elastic failure
+ '1b' is already exact on the overlap fields
+ the remaining BEACON work is concentrated on the native VK storage carrier
+ and, for '1c', the heterogeneous transport-porosity split
+
+## BEACON matching
+
+This section records the exact changes that were needed to make the bridge-side
+'1a01' stage-1 inflow case reproduce the native stage-1 end state.
+
+Scope:
+
+ native target:
+ ' /Users/vinaykumar/Documents/GitHub/ogs/Tests/Data/RichardsMechanics/beacon_1a01_notebook_inflow.prj'
+ native reference end state:
+ ' /Users/vinaykumar/Documents/GitHub/ogs/Tests/Data/RichardsMechanics/beacon_1a01_notebook_inflow_reference_t_100000.000000.vtu'
+ bridge run deck:
+ 'Tests/Data/RichardsMechanics/beacon_1a01_notebook_mcc_inflow_bridge.prj'
+ bridge executable:
+ ' /Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs'
+ comparison time:
+ 't = 100000 s'
+
+## What had to match conceptually
+
+The native '1a01' inflow case is sensitive to the local storage solve. The top
+pressure is fixed, but the interior state is not, so the bridge had to replay
+the same constitutive logic as the native scalar notebook-storage path.
+
+The final bridge matching setup uses:
+
+ the verified pressure-coupled MCC carrier for mechanics
+ notebook microstate evolution for swelling and exchange
+ a scalar notebook-storage local solve on 'n_l'
+ process-visible transfer of notebook exchange and notebook swelling stress
+
+The three key constitutive relations are:

```

```

+
+ \[
+ \sigma_{\mathrm{eff}}^{\mathrm{hybrid}}
+ =
+ \sigma_{\mathrm{eff}}^{\mathrm{MCC}}
+ \sigma_S,
+ \quad
+ \sigma_S = -K, \epsilon_{\mathrm{sw}}, \mathbf{I}
+ \]
+
+ \[
+ \hat{\rho}_l
+ =
+ \alpha_M^{\mathrm{eff}}
+ \left( \mu_{\mathrm{LR}} - \mu_{\mathrm{LR}} \right),
+ \quad
+ \alpha_M^{\mathrm{eff}} = \bar{\alpha}(\rho_{\mathrm{LR}}) / \mu
+ \]
+
+ \[
+ R_{\mathrm{n}_l}
+ =
+ n_l^{n+1}
+ - n_l^n
+ - \Delta t, \hat{\rho}_l, \rho_{\mathrm{LR}}^{\mathrm{ref}}
+ - n_l^{n+1} / \Delta t \epsilon_{\mathrm{sw}}
+ = 0
+ \]
+
+ The first equation ensures that the added swelling stress is micro-driven. The
+ second and third equations ensure that the notebook exchange term enters the
+ same storage balance that the native scalar notebook-storage path uses.
+
+ ### Code changes needed
+
+ | File | Change | Why it was needed for 'la01' matching |
+ | --- | --- | --- |
+ | MaterialLib/SolidModels/MFront/RichardsMechanicsNotebookBridge_MCC.mfront | Added material property 'NotebookLocalSolveMode'; added scalar local-storage branch 'use_scalar_notebook_storage'; kept 'rho_LR = rho_LR_ref' in that branch; exposed 'rho_l_hat' and 'sigma_S'; kept the hybrid stress rule 'sigma_eff^hybrid = sigma_eff^MCC + sigma_S' | The native 'la01' inflow path is driven by a scalar notebook-storage solve. Without this branch, the bridge used the older coupled '(n_l, rho_LR)' solve and drifted in 'pressure' and 'saturation'. |
+ | MaterialLib/SolidModels/MechanicsBase.h | Extended 'PressureCoupledResponse' with 'swelling_stress' and 'liquid_mass_exchange_source' | The notebook bridge already knew 'sigma_S' and 'rho_l_hat', but RM had no generic place to carry them out of the constitutive update. |
+ | ProcessLib/RichardsMechanics/ConstitutiveRelations/PressureCoupledSolidData.h | Added 'swelling_stress' and 'liquid_mass_exchange_source' to the bridge-side constitutive carrier | Needed to move the new bridge outputs into assembly and process-state storage. |
+ | MaterialLib/SolidModels/MFront/MFrontRichardsMechanics.h | Mapped internal variable 'sigma_S' to public 'swelling_stress'; mapped '-rho_l_hat' to 'liquid_mass_exchange_source' | This is the bridge-to-RM handoff that makes the notebook support stress and notebook exchange source visible to the process. |
+ | ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h | Added 'liquid_mass_exchange_source' to the pressure residual and wrote bridge 'swelling_stress' into the process-owned swelling state | Without the residual source, the pressure equation ignored notebook mass exchange. Without the state write-back, the user-facing 'swelling_stress' output stayed zero even when 'sigma_S' and 'sigma' were correct. |
+ | Tests/MaterialLib/MFront/RichardsMechanicsNotebookBridgeMCC.cpp | Added bridge regressions for the swelling-stress split and the native-aligned stage-1 local step | These tests prove the constitutive split before the FE run. |
+ | Tests/ProcessLib/RichardsMechanics/PressureCoupledSolidData.cpp | Added mapping checks for 'swelling_stress' and 'liquid_mass_exchange_source' | This protects the carrier path between MFront and RM assembly. |
+ | Tests/Data/RichardsMechanics/beacon_la01_notebook_mcc_inflow_bridge.prj | Added the dedicated bridge inflow deck | This is the first bridge BEACON deck that targets the full stage-1 end state at 't=100000 s', not just the smoke state at 't=1000 s'. |
+
+ ### Project and calibration choices used for the match
+
+ The 'la01' inflow match was not obtained by changing the geometry or the load
+ path. Those stay on the native stage-1 setup. The required calibration choices
+ were inside the bridge constitutive deck.
+
+ | Project quantity | Value in 'beacon_la01_notebook_mcc_inflow_bridge.prj' | Purpose |
+ | --- | --- | --- |
+ | 'InitialPreConsolidationPressure' | '1e10' | Keeps the MCC carrier elastic so the stage-1 swelling comparison is driven by the notebook microstate, not by plastic yielding. |
+ | 'InitialVolumeRatio' | '1.6666666666666667' | Keeps the pressure-coupled MCC carrier on the same reference surface as the verified MCC bridge decks. |
+ | 'NotebookLocalSolveMode' | '1' | Activates the scalar notebook-storage local solve that matches the native 'scalar_notebook_storage' logic. |
+ | 'MicroPotentialConvention' | '1' | Uses the native-aligned sign convention for the notebook micro-potential. |
+ | 'NotebookSwellingSlope' | '0.1' | Transfers notebook micro-swelling into the returned stress. This is not an independent dominant macro swelling law; it is the coefficient that maps the micro swelling strain into the support stress 'sigma_S = -K epsilon_sw I'. |
+ | 'MassExchangeCoefficient' | '1e-13' | Keeps the micro-to-macro exchange rate on the committed notebook scale. |
+ | 'MacroViscosity' | '1e-3' | Enters the native-style exchange scaling 'alpha_M^eff = bar alpha rho_LR / mu'. |
+ | 'n_l0' | '0.01' | Native-aligned initial notebook liquid content. |
+ | 'rho_LR0' | '2276.031917690513' | Native-aligned initial micro liquid density for the stage-1 inflow state. |
+ | 'epsilon_sw0' | '0.0' | Starts the notebook swelling strain from zero, so the stage-1 swelling stress is generated by the run itself. |
+
+ ### Why each change was necessary
+
+ The matching problem had three layers.
+
+ 1. The local storage solve had to match the native logic.
+   The old bridge solve evolved '(n_l, rho_LR)' together. That is acceptable on
+   reduced notebook tests, but it is not the same closure as the native scalar
+   notebook-storage branch used by 'la01' inflow. Switching to
+   'NotebookLocalSolveMode = 1' removed the remaining saturation mismatch.
+
+ 2. The exchange source had to enter the pressure equation.
+   The notebook bridge computes 'rho_l_hat', but until the bridge carrier was
+   widened, RM did not use it in the pressure residual. After the widening, the
+   pressure residual sees the source term
+   'R_p <- R_p + integral N_p^T (-rho_l_hat) dOmega'.
+
+ 3. The public swelling-stress output had to be populated from the bridge.
+   Before the new 'swelling_stress' carrier was added, the bridge already
+   produced the correct support stress internally as 'sigma_S', but the user
+   output 'swelling_stress' stayed zero because only the native MPL path wrote
+   that process-owned field.
+
+ ### Final 'la01' stage-1 comparison at 't = 100000 s'
+
+ The current bridge inflow run is close to the native reference:
+
+ | Field | Native vs bridge end-state comparison |
+ | --- | --- |
+ | 'pressure' | abs max '2.875603146608796e-02', rel max '1.686365460364637e-04' |
+ | 'saturation' | exact |
+ | 'swelling_stress' | abs max '1.350759650645159e-01', rel max '3.790781457621327e-05' |
+ | 'sigma' | abs max '1.35075965064524e-01', rel max '3.790781457646852e-05' |
+
+ This means the bridge now reproduces the stage-1 native end state to a very
+ good approximation, and it does so with micro-driven swelling:
+
+ - the swelling stress exposed to the user is the bridge notebook support stress
+   'sigma_S'
+ - the hybrid stress gap relative to the plain MCC carrier is also 'sigma_S'
+ - the remaining mismatch is small numerical residue in 'pressure', 'sigma', and

```



```

+ 'swelling_stress', not a missing constitutive branch
+
+### Practical boundary after the matching work
+
+For BEACON '1a01' stage 1, the remaining work is no longer "make swelling
+"appear or "make the bridge use the notebook exchange "term. Those are done.
+
+What is still open:
+
+-- add a tracked compare CTest for the bridge inflow case once the tolerated
+ thresholds are agreed
+-- extend the same storage-carrier treatment from '1a01' to the heterogeneous
+ '1c' path, where 'transport_porosity' is still native-specific
+-- port 'potential_exchange' into the MFront tree if one executable for both
+ native and bridge project families is still desired
+
+### BEACON report comparison for '1a01' and '1b'
+
+This section compares the current native and bridge OGS runs against the
+benchmark values reported in BEACON D5.1.1 for '1a01' and '1b'.
+
+The comparison rules used here are:
+
+-- axial swelling pressure from OGS:
+ 'p_sw'ax = |sigma_yy|
+-- radial swelling pressure from OGS:
+ 'p_sw'rad = |sigma_xx|
+-- dry density from OGS:
+ 'dry_density_solid'
+
+The current meshes for both '1a01' and '1b' are still one element through the
+height. That means the present OGS runs can only produce a uniform end-state
+dry-density field, not a resolved dry-density profile.
+
+### Meshes and boundary conditions
+
+The native and bridge comparisons in this section use the same specimen meshes
+and the same boundary-condition layout. The figures below show the actual mesh
+size and the active Dirichlet conditions for the compared '1a01' and '1b'
+project files.
+
+! [BEACON 1a01 mesh and boundary conditions] (figures/beacon_1a01_mesh_bc.png)
+
+'1a01': one quadrilateral element over a '25 mm x 20 mm' axisymmetric specimen.
+The radial displacement is fixed on 'left' and 'right', the axial displacement
+is fixed on 'bottom' and 'top', and the liquid pressure is prescribed on the
+'top' edge with 'p = 2e3 Pa'. The initial pressure is also '2e3 Pa'.
+
+! [BEACON 1b mesh and boundary conditions] (figures/beacon_1b_mesh_bc.png)
+
+'1b': one quadrilateral element over a '120 mm x 105.15 mm' axisymmetric
+specimen. The displacement constraints are the same as in '1a01'. The liquid
+pressure is prescribed on both axial faces: 'p = 0 Pa' at the 'top' and
+'p = 1e4 Pa' at the 'bottom'. The initial pressure is '-1e6 Pa'.
+
+### What was taken from the report
+
+For '1a01', D5.1.1 gives two different targets:
+
+-- Table 3-1 gives the end of stage 1 ('intermediate') at constant volume:
+ - axial stress '604 kPa'
+ - radial stress '994 kPa'
+ - dry density '1655 kg/m^3'
+-- Table 3-2 gives the final post-mortem density profile after the full test,
+ including stage 2 with added volume:
+ - from bottom '2.5 mm': '1466 kg/m^3'
+ - from bottom '7.5 mm': '1454 kg/m^3'
+ - from bottom '12.5 mm': '1427 kg/m^3'
+ - from bottom '17.5 mm': '1353 kg/m^3'
+ - average: '1425 kg/m^3'
+
+For '1b', D5.1.1 gives:
+
+-- Table 4-3 after the volume adjustment:
+ - dry density '1.52 g/cm^3 = 1520 kg/m^3'
+-- Figure 4-6 and the surrounding text:
+ - swelling pressure is nonzero
+ - a stabilized state is reached after about '500 days'
+
+The extracted report text does not provide a tabulated dry-density profile for
+'1b', and it does not provide a machine-readable swelling-pressure plateau
+value. So the '1b' report comparison is partly qualitative.
+
+### Current native and bridge runs used here
+
+| Case | Native run | Bridge run | End time used |
+| --- | --- | --- | --- |
+| '1a01' | 'beacon_1a01_notebook_inflow.prj' | 'beacon_1a01_notebook_mcc_inflow_bridge.prj' | '1e5 s' |
+| '1b' | temporary long-horizon copy of 'beacon_1b_notebook_smoke.prj' | temporary long-horizon copy of 'beacon_1b_notebook_mcc_bridge.prj' | '4.32e7 s = 500 days' |
+
+The '1b' long-horizon runs were temporary report-comparison runs only. They are
+not committed CTests.
+
+### Comparison table
+
+| Case | Quantity | BEACON report target | Native OGS | MFront OGS | Match status | Main reason |
+| --- | --- | --- | --- | --- | --- | --- |
+| '1a01' | Stage-1 axial swelling pressure | '604 kPa' | '3.563 kPa' | '3.563 kPa' | strong mismatch | The current '1a01' inflow implementation builds only a very small support stress on the stage-1 path. |
+| '1a01' | Stage-1 radial swelling pressure | '994 kPa' | '3.562 kPa' | '3.562 kPa' | strong mismatch | Same as above; both native and bridge remain far below the benchmark pressure level. |
+| '1a01' | Stage-1 dry density | '1655 kg/m^3' | '1668.009 kg/m^3' | '1668.009 kg/m^3' | close | The current stage-1 case keeps dry density near the initial compacted state. |
+| '1a01' | Final post-mortem dry-density profile after full test | '1466, 1454, 1427, 1353 kg/m^3' from bottom to top | uniform '1668.0 kg/m^3' field on a one-element mesh | uniform '1668.0 kg/m^3' field on a one-element mesh | mismatch and not fully comparable | The current OGS case is still stage-1 only, and the FE mesh has no axial resolution for a true profile. |
+| '1b' | Dry density after volume adjustment | '1520 kg/m^3' | '1389.975 kg/m^3' | '1389.975 kg/m^3' | mismatch | The current surrogate uses 'phi0 = 0.5', which fixes the bulk dry density near '1390 kg/m^3' instead of the report value '1520 kg/m^3'. |
+| '1b' | Swelling pressure after long saturation | nonzero and stabilized after about '500 days' | '0 kPa' | '0 kPa' | mismatch | The current '1b' native and bridge surrogates remain mechanically inactive even on the report-style time horizon. |
+| '1b' | Dry-density profile | not tabulated in the extracted D5.1.1 text | uniform '1389.975 kg/m^3' field | uniform '1389.975 kg/m^3' field | report value not available, OGS unresolved | The report text gives a requested profile definition, but not a tabulated profile; the current OGS mesh is also too coarse to resolve one. |
+
+### Interpretation
+
+The current picture is now clear:
+
+-- '1a01': native and bridge match each other well on the current OGS setup, but

```

```

+ both are still far below the BEACON stage-1 swelling-pressure target.
+ '1a01': dry density is close to the stage-1 report value, but the current
+ case cannot reproduce the final post-mortem profile because it does not yet
+ include the second benchmark stage and it has only one element over the
+ specimen height.
+ '1b': native and bridge again match each other, but they do not match the
+ report-facing dry density or the nonzero swelling-pressure development.
+
+So the current limiting factor is no longer native-vs-bridge disagreement for
+these two cases. The limiting factor is benchmark realism of the OGS surrogate.
+
+### Suggested next steps
+
+
+ '1a01': keep the current native/bridge matched stage-1 branch, but retune the
+ microscale swelling path until stage-1 swelling pressure reaches the reported
+ '604/994 kPa' range.
+ '1a01': add axial resolution and the second benchmark stage before claiming
+ any match to the post-mortem dry-density profile in Table 3-2.
+ '1b': set the initial dry density to the reported '1520 kg/m³' level first;
+ without that, a swelling-pressure comparison is already off on the initial
+ state.
+ '1b': after the density correction, check why the present constitutive branch
+ still gives zero stress on a '500 day' run.
+
+### BEACON report comparison on the shared unstructured batch
+
+
+The one-element comparison above is still useful as a smoke-level baseline, but
+it is not enough for any profile comparison. The new batch below repeats
+'1a01' and '1b' on shared unstructured meshes with '162' triangular elements
+and '100' points. The meshes are generated reproducibly by
+'Tests/Data/RichardsMechanics/generate_beacon_unstructured_meshes.py', and the
+report-facing metrics are extracted by
+'Tests/Data/RichardsMechanics/analyze_beacon_unstructured_batch.py'.
+
+### Batch files
+
+
+| Item | '1a01' | '1b' |
+| --- | --- | --- |
+| Unstructured mesh | 'Tests/Data/RichardsMechanics/beacon_1a01_domain_unstructured_162e.vtu' | 'Tests/Data/RichardsMechanics/beacon_1b_domain_unstructured_162e.vtu' |
+| Native project | 'Tests/Data/RichardsMechanics/beacon_1a01_inflow_unstructured_batch.prj' | 'Tests/Data/RichardsMechanics/beacon_1b_unstructured_batch.prj' |
+| Bridge project | 'Tests/Data/RichardsMechanics/beacon_1a01_notebook_mcc_inflow_unstructured_batch.prj' | 'Tests/Data/RichardsMechanics/beacon_1b_notebook_mcc_unstructured_batch.prj' |
+| Final time used here | '1e5 s' | '4.32e7 s = 500 days' |
+| Compared report quantity | stage-1 swelling pressure and density | long-time swelling pressure and density |
+
+### How the batch was compared
+
+
+ Axial swelling pressure from OGS:
+ 'p_sw^ax = |mean_top(sigma_yy)|'
+ Radial swelling pressure from OGS:
+ 'p_sw^rad = |mean_outer(sigma_xx)|'
+ Micro support stress diagnostic:
+ ' |mean(swelling_stress_yy)|' on the top boundary and
+ ' |mean(swelling_stress_xx)|' on the outer boundary
+ Dry-density profile:
+ average of 'dry_density_solid' in '4' equal-height bins for '1a01' and '10'
+ equal-height bins for '1b'
+
+### Unstructured batch comparison table
+
+
+| Case | Quantity | BEACON report target | Native unstructured | MFront unstructured | Native vs MFront | Match status | Main reason |
+| --- | --- | --- | --- | --- | --- | --- | --- |
+| '1a01' | Stage-1 axial swelling pressure | '604 kPa' | '2.846 kPa' | '2.847 kPa' | '0.00034 kPa' | strong mismatch to report, native vs MFront close | The dense mesh does not change the constitutive amplitude: both runs are still about two orders of magnitude below the report target. |
+| '1a01' | Stage-1 radial swelling pressure | '994 kPa' | '3.553 kPa' | '3.554 kPa' | '0.00032 kPa' | strong mismatch to report, native vs MFront close | Same as above; the bridge reproduces the native stage-1 stress level but that level is still far below the report. |
+| '1a01' | Top micro support stress 'swelling_stress_yy' | not reported separately | '3.563 kPa' | '3.564 kPa' | '0.00033 kPa' | native vs MFront close | The micro-driven swelling branch is active here, but it still generates only a few kilopascals on the current calibration. |
+| '1a01' | Mean dry density at stage 1 | '1655 kg/m³' | '1668.011 kg/m³' | '1668.011 kg/m³' | '1.07e-05 kg/m³' | close to report, native vs MFront exact for practical purposes | The dense mesh preserves the already-close stage-1 density level. |
+| '1b' | Swelling pressure after '500' days, axial | nonzero and stabilized | '1.288 kPa' | '6.201 kPa' | '4.914 kPa' | both nonzero, native vs MFront mismatch, report still unresolved | The dense mesh removes the earlier zero-stress result, but the bridge is now visibly stiffer than the native run on 'sigma_yy'. |
+| '1b' | Swelling pressure after '500' days, radial | nonzero and stabilized | '3.001 kPa' | '2.835 kPa' | '0.166 kPa' | qualitative agreement only | Both runs are nonzero, but the report does not provide a machine-readable plateau value for a strict quantitative check. |
+| '1b' | Top micro support stress 'swelling_stress_yy' | not reported separately | '0 kPa' | '0 kPa' | '0 kPa' | constitutive mismatch to intended mechanism | The stress that appears on '1b' is not coming from the exported micro swelling support field; both runs still show zero 'swelling_stress'. |
+| '1b' | Mean dry density after volume adjustment | '1520 kg/m³' | '1389.944 kg/m³' | '1389.860 kg/m³' | '0.084 kg/m³' | strong mismatch to report, native vs MFront close | The current mixture surrogate is still anchored near the old 'phi0 = 0.5' density level, far below the report target. |
+
+### Dry-density profile tables
+
+
+ '1a01' report vs unstructured OGS profile ('4' equal-height bins):
+
+
+| Height from bottom | BEACON report after full test | Native unstructured | MFront unstructured | Note |
+| --- | --- | --- | --- | --- |
+| '2.5 mm' | '1466 kg/m³' | '1668.020 kg/m³' | '1668.020 kg/m³' | OGS is much denser; current run is only stage 1, not the full two-stage test. |
+| '7.5 mm' | '1454 kg/m³' | '1668.017 kg/m³' | '1668.017 kg/m³' | Same reason as above. |
+| '12.5 mm' | '1427 kg/m³' | '1668.009 kg/m³' | '1668.009 kg/m³' | Same reason as above. |
+| '17.5 mm' | '1353 kg/m³' | '1667.997 kg/m³' | '1667.997 kg/m³' | Same reason as above. |
+
+ '1b' unstructured OGS dry-density profile ('10' equal-height bins). The
+extracted D5.1.1 text does not provide a tabulated profile, so only the report
+average can be checked directly:
+
+
+| Metric | BEACON report | Native unstructured | MFront unstructured | Note |
+| --- | --- | --- | --- | --- |
+| Average dry density | '1520 kg/m³' | '1389.944 kg/m³' | '1389.860 kg/m³' | Both OGS runs are about '130 kg/m³' below the report average. |
+| Bottom-bin dry density | not tabulated | '1389.896 kg/m³' | '1389.698 kg/m³' | Native and bridge are close, but no report-side profile is available. |
+| Top-bin dry density | not tabulated | '1389.994 kg/m³' | '1389.988 kg/m³' | The dense OGS profile stays almost flat over height. |
+| Profile range over height | not tabulated | '1389.896' to '1389.994 kg/m³' | '1389.698' to '1389.988 kg/m³' | The bridge shows a slightly wider gradient, but the overall level is still far below the report average. |
+
+### What the unstructured batch changes
+
+
+
+ '1a01': the dense unstructured mesh confirms that native and MFront remain
+ close on the current stage-1 constitutive branch. The mismatch to the BEACON
+ report is therefore not a mesh-resolution problem and not a native-vs-bridge
+ problem. It is still a constitutive-calibration problem.
+ '1a01': the dense profile is now available, but it should not be over-read.
+ The report profile is post-mortem after the full two-stage test, while the
+ current OGS run is only the stage-1 inflow branch.
+ '1b': the dense mesh changes the qualitative picture. The old zero-stress
+ result is gone. Both native and MFront now build nonzero total stress, but
+ the bridge is noticeably stiffer than the native run in 'sigma' even though
+ 'pressure' and 'saturation' still match essentially exactly.
+ '1b': the dense batch also shows that the present '1b' stress is still not
+ micro-swelling driven on the exported support-stress branch, because

```

```

+ 'swelling_stress' remains exactly zero in both runs.
+
+### Suggested next steps from the unstructured batch
+
+-- 'la01': keep the dense batch as the report-facing spatial benchmark, and tune
+ the notebook micro branch so that the stage-1 micro support stress reaches
+ the '604/994 kPa' report scale without handing the job over to a dominant
+ macro swelling carrier.
+-- 'la01': only compare against the post-mortem density profile after adding the
+ second benchmark stage to the dense mesh case.
+-- 'lb': reset the initial density to the reported '1520 kg/m^3' level before
+ reading too much into the long-time stress gap.
+-- 'lb': once the density is corrected, inspect why the bridge dense run stays
+ exact in 'pressure' and 'saturation' but drifts in 'sigma' by
+ 'max Δ|sigma| = 11.17 kPa'. That is now the main dense native-vs-MFront
+ mismatch on 'lb'.
+-- 'lb': activate a genuinely micro-driven swelling contribution for the pellet
+ mixture branch, because the current dense result still exports
+ 'swelling_stress = 0' in both native and bridge runs.
+
+### BEACON D3.3 Annex A BGR first attempt
+
+The next report-facing step after the 'la01' and 'lb' comparisons was the BGR
+appendix in BEACON D3.3. The current branch now contains a first reduced replay
+of the EPFL/BGR confined-swelling path 'P2-1', stage 'A-B'.
+
+### Scope of the committed replay
+
+-- source document:
+ BEACON H2020 deliverable D3.3, Annex A, BGR appendix
+-- target path:
+ 'P2-1 / A-B'
+-- current mesh reduction:
+ smallest nontrivial axisymmetric confined column that still runs both the
+ native and bridge decks
+-- native replay deck:
+ 'Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_native.prj'
+-- bridge replay deck:
+ 'Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_mfront.prj'
+-- mesh generator:
+ 'Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/generate_bgr_epfl_mesh.py'
+-- analyzer:
+ 'Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/analyze_bgr_wp3_abprime.py'
+-- committed report:
+ 'Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report.json'
+
+### Reduced replay targets used here
+
+-- BGR boundary-value swelling pressure:
+ '2.71 MPa'
+-- reported experimental swelling-pressure range:
+ '3.12' to '3.55 MPa'
+-- initial total suction range:
+ '90' to '110 MPa'
+-- initial void-ratio range:
+ '0.83' to '0.85'
+-- water-pressure boundary:
+ '20 kPa'
+
+### Reduced replay comparison at 't = 2e5 s'
+
+| Quantity | Native replay | MFront replay | BGR target or range | Interpretation |
+| --- | --- | --- | --- | --- |
+| Axial swelling pressure | '1.0103784801106894 MPa' | '0.7859139611933248 MPa' | '2.71 MPa' boundary value | Both runs underpredict the BGR appendix target; bridge is softer than native. |
+| Radial stress | '1.0101834147550586 MPa' | '0.7858119053590902 MPa' | same confined-swelling level expected | Same picture as axial: native and bridge stay below the report-facing scale. |
+| Mean dry density | '1502.8393244518954 kg/m^3' | '1502.8376146942085 kg/m^3' | not the primary appendix target | Native and bridge density levels are essentially identical on the reduced path. |
+| Mean saturation | '1.0' | '1.0' | full hydration expected on this reduced end state | The mismatch is not because one run stayed unsaturated. |
+| Native vs MFront axial gap | '0.22446451891736463 MPa' | same | '0 MPa' for exact parity | There is still a real constitutive gap between native and bridge on this replay. |
+
+### Interpretation
+
+The current first attempt closes one uncertainty and opens another:
+
+-- it proves that the D3.3 appendix path can be reduced to a reproducible native
+ and bridge pair in the current repositories
+-- it does not yet prove that the current constitutive branch matches the BGR
+ appendix target
+-- native and bridge are both too soft relative to the BGR boundary value and
+ the reported experimental range
+-- the bridge is also still softer than the native replay on the same reduced
+ path
+
+### Committed plotting boundary
+
+The branch now also contains the durable plotting helpers used to render the
+report-style figures that were added to the transition note:
+
+-- 'Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/plot_bgr_wp3_epfl_report_figures.py'
+-- 'Tests/Data/RichardsMechanics/plot_beacon_report_figures.py'
+
+These scripts are committed because they define reproducible extraction and
+plotting of the report-facing metrics. The generated output directories are not
+part of the committed boundary:
+
+-- do commit plotting code and note-facing figure assets
+-- do not commit ad hoc VTU/PVD/CSV/PNG output trees unless they are explicitly
+ promoted to tracked benchmark artifacts
+
+So the next D3.3 work should not be framed as "just add another report "run.
+The committed replay is already a report run. The real remaining tasks are:
+
+-- improve native-vs-MFront agreement on the reduced 'P2-1 / A-B' path
+-- decide whether the reduced path should be calibrated directly to the
+ '2.71 MPa' BGR boundary value or first tightened against the native replay
+-- only after that, consider enlarging the mesh or adding more of the appendix
+ setup details
+
+### Generated parity and benchmark files
+
+These are the generated project and compare files that define the current
+notebook-MCC saturation surfaces.
+
+| File | What it runs | What it tests exactly |
+| --- | --- | --- |

```

```

+| 'Tests/Data/RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_native.prj' | Native reduced one-element MCC shell with native MPL Tuller saturation and Bishop cutoff |
Native reduced reference for the notebook Tuller saturation branch, with no bridge state |
+| 'Tests/Data/RichardsMechanics/mfront_parity_1element_notebook_mcc_tuller_bridge.prj' | Reduced one-element 'RichardsMechanicsNotebookBridge_MCC' shell with 'NotebookSaturationMo
de = 1' and neutral notebook swelling/mass exchange | Bridge reduced reference for the same Tuller saturation branch |
+| 'scripts/cmake/test/CompareRichardsMechanicsMFrontNotebookMCCTullerParity.cmake' | Runs the reduced native and bridge Tuller shells and compares their VTU outputs | Enforces red
uced same-law MCC + notebook Tuller parity on 'displacement', 'pressure', 'sigma', 'epsilon', 'saturation', 'velocity', and MCC internal variables |
+| 'Tests/Data/RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_native.prj' | Native benchmark part-1 shell with native MPL Tuller saturation and Bishop cutoff | Native b
enchmark reference for the non-neutral notebook Tuller saturation branch on the part-1 geometry and load scale |
+| 'Tests/Data/RichardsMechanics/mfront_restart_part1_notebook_mcc_tuller_bridge.prj' | Benchmark part-1 'RichardsMechanicsNotebookBridge_MCC' shell with 'NotebookSaturationMode =
1', neutral notebook swelling/mass exchange, damped Newton, and fixed '\Delta t = 1' | Bridge benchmark reference for the same Tuller saturation branch |
+| 'scripts/cmake/test/CompareRichardsMechanicsMFrontNotebookMCCTullerBenchmarkParity.cmake' | Runs the benchmark native and bridge Tuller shells and compares the start and end sta
tes | Enforces benchmark same-law MCC + notebook Tuller parity on the part-1 shell |
+
+## Definition of done
+
+### Reduced one-element task
+
+Done means:
+
+  + the native and bridge one-element shells both run
+  + the compare CTest is green
+  + all compared fields match exactly except for justified machine-scale residue
+
+### Benchmark task
+
+Done means:
+
+  + the native and bridge benchmark shells both run on the native part-1 scale
+  + the benchmark fields match up to only justified machine-scale residue
+  + the benchmark comparison is enforced by a dedicated CTest
+
+Current state:
+
+  + done for the same-law MCC benchmark surface
+  + done for the same-law MCC + notebook Tuller benchmark surface
+  + not done for full notebook-to-native constitutive equivalence
+
+## Automated validation checkpoint (2026-04-10 12:12 CEST)
+
+Scope and commits validated in this checkpoint:
+
+  + native repo: '/Users/vinaykumar/git/ogs-native-dsm-transition', branch 'dsm-nb-transition', head '8fbd9a833'
+  + MFront repo: '/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled', branch 'dsm-nb-mfront-transition', head '2cf7b4734d'
+
+Automatic checks executed:
+
+  + Python syntax checks ('python3 -m py_compile') for updated BEACON/ANCHORS helper scripts: passed.
+  + XML syntax checks ('xllint --noout') for renamed BEACON project files: passed.
+  + CTest run on MFront build tree '/Users/vinaykumar/git/build/release-mfront-tpm' with
+  + "-R 'ogs-RichardsMechanics/double_porosity_swelling[ogs-RichardsMechanics/DoubleStructureBenchmark/double_porosity_swelling_RM]': passed ('2/2')".
+
+Native CTest status:
+
+  + Native reconfigure attempt on '/Users/vinaykumar/git/build/release-native-transition-mfront' failed before generation because the local CPM source cache is incomplete/offline fo
r required packages ('Phreeqc', 'OgsXdmf') and TFEEL version parsing fails on the currently resolved TFEEL path.
+  + Native CTest execution with the existing binary ('OpenGeoSys-6 version 6.5.7-169-ga8984e3d') fails at project parsing ('15/15' in the BEACON/DSM slice) with:
+  + 'Key <potential_exchange> has been read 1 time(s) less than it was present in the configuration tree.'
+  + This indicates the available native executable predates the 'potential_exchange' rename and cannot validate the renamed native decks until a fresh rebuild is completed from the
current branch state.
+
+Action required for full native confirmation:
+
+  + restore/populate local CPM cache content for missing dependencies used by the native build configuration,
+  + reconfigure and rebuild 'ogs-native-dsm-transition',
+  + rerun the native RichardsMechanics BEACON/DSM CTest slice on the rebuilt executable.
+
+## Useful commands
+
+Configure:
+
+'''bash
+cmake -S /Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled \
+  -B /Users/vinaykumar/git/build/release-mfront-tpm \
+  -DCMAKE_BUILD_TYPE=Release \
+  -DGS_USE_MFRONT=ON \
+  -DCPM_SOURCE_CACHE=/Users/vinaykumar/git/.cpm-cache \
+  -DTFELHOME=/Users/vinaykumar/.cache/CPM/_ext/TFEL/895fd7874cfa49079f7711831b3fa10069d8ee57388ab29ff17c265f8ffe777 \
+  -DMFRONT=/Users/vinaykumar/.cache/CPM/_ext/TFEL/895fd7874cfa49079f7711831b3fa10069d8ee57388ab29ff17c265f8ffe777/bin/mfront \
+  -DMFRONT_QUERY=/Users/vinaykumar/.cache/CPM/_ext/TFEL/895fd7874cfa49079f7711831b3fa10069d8ee57388ab29ff17c265f8ffe777/bin/mfront-query
+'''
+
+Focused CTest slice:
+
+'''bash
+ctest --test-dir /Users/vinaykumar/git/build/release-mfront-tpm \
+  --output-on-failure \
+  -R 'ogs-RichardsMechanics/mfront_restart_part1[ogs-RichardsMechanics/mfront_restart_part2[ogs-RichardsMechanics/mfront_restart_part1_rm_bridges[ogs-RichardsMechanics/mfront_re
start_part1_mcc_compares[ogs-RichardsMechanics_mfront_restart_part1_notebook_mcc_compares[ogs-RichardsMechanics_mfront_restart_part1_notebook_mcc_tuller_(native|bridge|compare)$]og
s-RichardsMechanics_mfront_parity_1element_natives[ogs-RichardsMechanics_mfront_parity_1element_bridges[ogs-RichardsMechanics_mfront_parity_1element_compares[ogs-RichardsMechanics_
mfront_parity_1element_notebook_mcc_bridges[ogs-RichardsMechanics_mfront_parity_1element_notebook_mcc_compares[ogs-RichardsMechanics_mfront_parity_1element_notebook_mcc_tuller_comp
ares'
+'''
+
+Native BEACON/DSM CTest slice after rebuild:
+
+'''bash
+ctest --test-dir /Users/vinaykumar/git/build/release-native-transition-mfront \
+  --output-on-failure \
+  -R 'ogs-RichardsMechanics_(double_porosity_swelling_vk_.*time|beacon_1a01_vk_.*time|beacon_1b_vk_.*time|beacon_1c_vk_.*time)$' \
+  -E 'omp|vtckdiff|history'
+'''
+
+Direct benchmark-pressure bridge guardrail:
+
+'''bash
+/Users/vinaykumar/git/build/release-mfront-tpm/bin/testrunner \
+  --gtest_filter='MaterialLib_RichardsMechanicsNotebookBridgeMFront.PlaneStrainFactoryPathZeroDtBenchmarkPressureConsistentExactRMSStateResponse:MaterialLib_RichardsMechanicsNotebo
okBridgeMFront.PlaneStrainFactoryPathBenchmarkPressureConsistentExactRMSStateFirstStepResponse:MaterialLib_RichardsMechanicsNotebookBridgeMFront.PlaneStrainFactoryPathBenchmarkPress
ureConsistentProcessFailuresStateResponse'
+'''
+
+Notebook-MCC hybrid guardrail:
+
+'''bash
+/Users/vinaykumar/git/build/release-mfront-tpm/bin/testrunner \
+  --gtest_filter='MaterialLib_RMBridgeMFront_NotebookMCC.*'
+'''

```

```

diff --git a/Documentation/figures/beacon_1a01_mesh_bc.png b/Documentation/figures/beacon_1a01_mesh_bc.png
new file mode 100644
index 0000000000..dbef366267
Binary files /dev/null and b/Documentation/figures/beacon_1a01_mesh_bc.png differ
diff --git a/Documentation/figures/beacon_1b_mesh_bc.png b/Documentation/figures/beacon_1b_mesh_bc.png
new file mode 100644
index 0000000000..c87e477552
Binary files /dev/null and b/Documentation/figures/beacon_1b_mesh_bc.png differ
diff --git a/MaterialLib/MPL/CreateProperty.cpp b/MaterialLib/MPL/CreateProperty.cpp
index b3d4350a94..d66f06542c 100644
--- a/MaterialLib/MPL/CreateProperty.cpp
+++ b/MaterialLib/MPL/CreateProperty.cpp
@@ -195,6 +195,10 @@ std::unique_ptr<MaterialPropertyLib::Property> createProperty(
{
    return createSaturationExponential(config);
}
+ if (boost::iequals(property_type, "SaturationTuller"))
+ {
+     return createSaturationTuller(config);
+ }
+ if (boost::iequals(property_type, "SaturationVanGenuchten"))
+ {
+     return createSaturationVanGenuchten(config);
+ }
diff --git a/MaterialLib/MPL/Properties/CapillaryPressureSaturation/CreateSaturationTuller.cpp b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/CreateSaturationTuller.cpp
new file mode 100644
index 0000000000..7fa401bf65
--- /dev/null
+++ b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/CreateSaturationTuller.cpp
@@ -0,0 +1,39 @@
+/**
+ * \file
+ * \copyright
+ * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */
+
+#include "BaseLib/ConfigTree.h"
+#include "SaturationTuller.h"
+
+namespace MaterialPropertyLib
+{
+std::unique_ptr<SaturationTuller> createSaturationTuller(
+    BaseLib::ConfigTree const& config)
+{
+    config.checkConfigParameter("type", "SaturationTuller");
+
+    auto property_name = config.peekConfigParameter<std::string>("name");
+
+    DEBUG("Create SaturationTuller medium property {s}.", property_name);
+
+    auto const area_factor_tuller =
+        config.getConfigParameter<double>("area_factor_tuller");
+    auto const pore_area_shape_factor_tuller =
+        config.getConfigParameter<double>("pore_area_shape_factor_tuller");
+    auto const characteristic_pore_size =
+        config.getConfigParameter<double>("characteristic_pore_size");
+    auto const surface_tension =
+        config.getConfigParameter<double>("surface_tension");
+
+    return std::make_unique<SaturationTuller>(
+        std::move(property_name), area_factor_tuller,
+        pore_area_shape_factor_tuller, characteristic_pore_size,
+        surface_tension);
+}
+} // namespace MaterialPropertyLib
diff --git a/MaterialLib/MPL/Properties/CapillaryPressureSaturation/CreateSaturationTuller.h b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/CreateSaturationTuller.h
new file mode 100644
index 0000000000..9feffa915
--- /dev/null
+++ b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/CreateSaturationTuller.h
@@ -0,0 +1,29 @@
+/**
+ * \file
+ * \copyright
+ * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */
+
+#pragma once
+
+#include <memory>
+
+namespace BaseLib
+{
+class ConfigTree;
+}
+
+namespace MaterialPropertyLib
+{
+class SaturationTuller;
+}
+
+namespace MaterialPropertyLib
+{
+std::unique_ptr<SaturationTuller> createSaturationTuller(
+    BaseLib::ConfigTree const& config);
+} // namespace MaterialPropertyLib
diff --git a/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.cpp b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.cpp
new file mode 100644
index 0000000000..e745208e47
--- /dev/null
+++ b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.cpp
@@ -0,0 +1,28 @@
+/**
+ * \file
+ * \copyright
+ * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */

```

```

+ */
+
+#include "SaturationTuller.h"
+
+#include <cmath>
+
+namespace MaterialPropertyLib
+{
+SaturationTuller::SaturationTuller(std::string name,
+    double const area_factor_tuller,
+    double const pore_area_shape_factor_tuller,
+    double const characteristic_pore_size,
+    double const surface_tension)
+ : area_factor_tuller_(area_factor_tuller),
+   pore_area_shape_factor_tuller_(pore_area_shape_factor_tuller),
+   characteristic_pore_size_(characteristic_pore_size),
+   surface_tension_(surface_tension),
+   capillary_prefactor_(4.0 * pore_area_shape_factor_tuller_ *
+       surface_tension_ * surface_tension_ /
+       (area_factor_tuller_ * characteristic_pore_size_ *
+       characteristic_pore_size_))
+{
+   name_ = std::move(name);
+
+   if (area_factor_tuller_ <= 0.0)
+   {
+       OGS_FATAL("SaturationTuller requires area_factor_tuller > 0.");
+   }
+   if (pore_area_shape_factor_tuller_ <= 0.0)
+   {
+       OGS_FATAL(
+           "SaturationTuller requires pore_area_shape_factor_tuller > 0.");
+   }
+   if (characteristic_pore_size_ <= 0.0)
+   {
+       OGS_FATAL("SaturationTuller requires characteristic_pore_size > 0.");
+   }
+   if (surface_tension_ <= 0.0)
+   {
+       OGS_FATAL("SaturationTuller requires surface_tension > 0.");
+   }
+}
+
+PropertyDataType SaturationTuller::value(VariableArray const& variable_array,
+    ParameterLib::SpatialPosition const&,
+    double const,
+    double const) const
+{
+   double const p_cap = variable_array.capillary_pressure;
+
+   if (p_cap <= pressure_tolerance_)
+   {
+       return 1.0;
+   }
+
+   auto const capillary_term = capillary_prefactor_ / (p_cap * p_cap);
+   return 1.0 - std::exp(-capillary_term);
+}
+
+PropertyDataType SaturationTuller::dValue(
+    VariableArray const& variable_array,
+    Variable const variable,
+    ParameterLib::SpatialPosition const&,
+    double const,
+    double const) const
+{
+   if (variable != Variable::capillary_pressure)
+   {
+       OGS_FATAL(
+           "SaturationTuller::dValue is implemented for derivatives with "
+           "respect to capillary pressure only.");
+   }
+
+   double const p_cap = variable_array.capillary_pressure;
+
+   if (p_cap <= pressure_tolerance_)
+   {
+       return 0.0;
+   }
+
+   auto const inv_p_cap = 1.0 / p_cap;
+   auto const exp_term =
+       std::exp(-capillary_prefactor_ * inv_p_cap * inv_p_cap);
+   return -2.0 * capillary_prefactor_ * exp_term * inv_p_cap * inv_p_cap *
+       inv_p_cap;
+}
+
+PropertyDataType SaturationTuller::d2Value(
+    VariableArray const& variable_array,
+    Variable const variable1,
+    Variable const variable2,
+    ParameterLib::SpatialPosition const&,
+    double const,
+    double const) const
+{
+   if (variable1 != Variable::capillary_pressure ||
+       variable2 != Variable::capillary_pressure)
+   {
+       OGS_FATAL(
+           "SaturationTuller::d2Value is implemented for second derivatives "
+           "with respect to capillary pressure only.");
+   }
+
+   double const p_cap = variable_array.capillary_pressure;
+
+   if (p_cap <= pressure_tolerance_)
+   {
+       return 0.0;
+   }
+
+   auto const p_cap2 = p_cap * p_cap;
+   auto const p_cap4 = p_cap2 * p_cap2;
+   auto const p_cap6 = p_cap4 * p_cap2;
+   auto const exp_term = std::exp(-capillary_prefactor_ / p_cap2);
+
+   return exp_term *
+       (6.0 * capillary_prefactor_ / p_cap4 -

```

```

+         4.0 * capillary_prefactor_ * capillary_prefactor_ / p_cap6);
+}
+} // namespace MaterialPropertyLib
diff --git a/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.h b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.h
new file mode 100644
index 0000000000..1a48917d33
--- /dev/null
+++ b/MaterialLib/MPL/Properties/CapillaryPressureSaturation/SaturationTuller.h
@@ -0,0 +1,86 @@
+/**
+ * \file
+ * \copyright
+ * Copyright (c) 2012–2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */
+
+//pragma once
+
+#include "MaterialLib/MPL/Property.h"
+
+namespace MaterialPropertyLib
+{
+/**
+ * \brief Tuller-style saturation law used by DSM micro-macro coupling.
+ *
+ * \details This property must be a medium property. It computes the liquid
+ * saturation from capillary pressure using the same scalar law as the
+ * DSM micro-scale unsaturated branch.
+ *
+ * For capillary pressure  $p_{\text{cap}}$  the saturation is
+ *
+ * \f[
+ * S_L(p_{\text{cap}}) =
+ * \begin{cases}
+ * 1 - \exp\left(-\dfrac{A_T}{p_{\text{cap}}^2}\right), & p_{\text{cap}} > p_{\text{tol}}, \\
+ * 1, & p_{\text{cap}} \leq p_{\text{tol}},
+ * \end{cases}
+ * \f]
+ *
+ * where
+ *
+ * \f[
+ * A_T =
+ * \frac{4 \beta_T \gamma_c}{a_T r_c^2}.
+ * \f]
+ */
+class SaturationTuller final : public Property
+{
+public:
+    SaturationTuller(std::string name,
+                     double area_factor_tuller,
+                     double pore_area_shape_factor_tuller,
+                     double characteristic_pore_size,
+                     double surface_tension);
+
+    void checkScale() const override
+    {
+        if (!std::holds_alternative<Medium>(scale_))
+        {
+            OGS_FATAL(
+                "The property 'SaturationTuller' is implemented on the "
+                "'media' scale only.");
+        }
+    }
+
+    PropertyDataType value(VariableArray const& variable_array,
+                           ParameterLib::SpatialPosition const& pos,
+                           double const t,
+                           double const dt) const override;
+
+    PropertyDataType dValue(VariableArray const& variable_array,
+                             Variable const variable,
+                             ParameterLib::SpatialPosition const& pos,
+                             double const t,
+                             double const dt) const override;
+
+    PropertyDataType d2Value(VariableArray const& variable_array,
+                              Variable const variable1,
+                              Variable const variable2,
+                              ParameterLib::SpatialPosition const& pos,
+                              double const t,
+                              double const dt) const override;
+
+private:
+    double const area_factor_tuller_;
+    double const pore_area_shape_factor_tuller_;
+    double const characteristic_pore_size_;
+    double const surface_tension_;
+    double const capillary_prefactor_;
+    static constexpr double pressure_tolerance_ = 1e-12;
+};
+
+} // namespace MaterialPropertyLib
diff --git a/MaterialLib/MPL/Properties/CreateProperties.h b/MaterialLib/MPL/Properties/CreateProperties.h
index 018922f3fe..924d3b75d9 100644
--- a/MaterialLib/MPL/Properties/CreateProperties.h
+++ b/MaterialLib/MPL/Properties/CreateProperties.h
@@ -16,6 +17,7 @@
+
+#include "CapillaryPressureSaturation/CreateSaturationBrooksCorey.h"
+#include "CapillaryPressureSaturation/CreateSaturationExponential.h"
+#include "CapillaryPressureSaturation/CreateSaturationLiakopoulos.h"
+#include "CapillaryPressureSaturation/CreateSaturationTuller.h"
+#include "CapillaryPressureSaturation/CreateSaturationVanGenuchten.h"
+#include "CapillaryPressureSaturation/CreateSaturationVanGenuchtenWithVolumetricStrain.h"
+#include "CreateAverageMolarMass.h"
diff --git a/MaterialLib/MPL/Properties/Properties.h b/MaterialLib/MPL/Properties/Properties.h
index 7080ec8d6a..a8df604523 100644
--- a/MaterialLib/MPL/Properties/Properties.h
+++ b/MaterialLib/MPL/Properties/Properties.h
@@ -17,6 +17,7 @@
+
+#include "CapillaryPressureSaturation/SaturationBrooksCorey.h"
+#include "CapillaryPressureSaturation/SaturationExponential.h"
+#include "CapillaryPressureSaturation/SaturationLiakopoulos.h"
+#include "CapillaryPressureSaturation/SaturationTuller.h"
+#include "CapillaryPressureSaturation/SaturationVanGenuchten.h"

```

```

#include "CapillaryPressureSaturation/SaturationVanGenuchtenWithVolumetricStrain.h"
#include "ClausiusClapeyron.h"
diff --git a/MaterialLib/SolidModels/CreateConstitutiveRelation.cpp b/MaterialLib/SolidModels/CreateConstitutiveRelation.cpp
index 87b23b52b2..6094eaf835 100644
--- a/MaterialLib/SolidModels/CreateConstitutiveRelation.cpp
+++ b/MaterialLib/SolidModels/CreateConstitutiveRelation.cpp
@@ -19,6 +19,7 @@
#include "CreateLubby2.h"
#ifdef OGS_USE_MFRONT
#include "MFront/CreateMFront.h"
#include "MFront/CreateMFrontRichardsMechanics.h"
#endif // OGS_USE_MFRONT

#include "CreateConstitutiveRelationsGeneric.h"
@@ -84,6 +85,17 @@ createConstitutiveRelation(
    OGS_FATAL(
        "OGS is compiled without MFront support. See OGS_USE_MFRONT CMake "
        "option.");
+endif // OGS_USE_MFRONT
+ }
+ if (type == "MFrontRichardsMechanics")
+ {
+ #ifdef OGS_USE_MFRONT
+ return MaterialLib::Solids::MFront::createMFrontRichardsMechanics<
+ DisplacementDim>(parameters, local_coordinate_system, config);
+ #else // OGS_USE_MFRONT
+ OGS_FATAL(
+ "OGS is compiled without MFront support. See OGS_USE_MFRONT CMake "
+ "option.");
+ #endif // OGS_USE_MFRONT
+ }
    OGS_FATAL("Cannot construct constitutive relation of given type '{s}'.",
diff --git a/MaterialLib/SolidModels/MFront/CMakeLists.txt b/MaterialLib/SolidModels/MFront/CMakeLists.txt
index 7f5f207131..e4e7be33b1 100644
--- a/MaterialLib/SolidModels/MFront/CMakeLists.txt
+++ b/MaterialLib/SolidModels/MFront/CMakeLists.txt
@@ -1,8 +1,12 @@
set(SOURCES
    CreateMFront.cpp
    CreateMFront.h
+    CreateMFrontRichardsMechanics.cpp
+    CreateMFrontRichardsMechanics.h
    MFront.cpp
    MFront.h
+    MFrontRichardsMechanics.cpp
+    MFrontRichardsMechanics.h
    MFrontGeneric.cpp
    MFrontGeneric.h
    CreateMFrontGeneric.cpp
@@ -37,6 +41,8 @@ set(_mfront_behaviours
    ModCamClay_semiExpl
    ModCamClay_semiExpl_absP
    ModCamClay_semiExpl_constE
+    ModCamClay_semiExpl_constE_pressureCoupled
+    ModCamClay_semiExpl_coupled
    MohrCoulombAbboSloan
    MohrCoulombAbboSloanAniso
    MohrCoulombAbboSloanOrtho
@@ -48,6 +54,8 @@ set(_mfront_behaviours
    StandardElasticityBrick
    StandardElasticityBrickOrtho
    ThermoPoreElasticity
+    RichardsMechanicsDSMicroMacroBridge
+    RichardsMechanicsDSMicroMacroBridge_MCC
    WeakPlaneModel
    WeakPlaneModelL_nonIter
)
diff --git a/MaterialLib/SolidModels/MFront/CreateMFrontGeneric.cpp b/MaterialLib/SolidModels/MFront/CreateMFrontGeneric.cpp
index b2c42aced5..2c8e9411b5 100644
--- a/MaterialLib/SolidModels/MFront/CreateMFrontGeneric.cpp
+++ b/MaterialLib/SolidModels/MFront/CreateMFrontGeneric.cpp
@@ -329,7 +329,14 @@ MFrontConfig createMFrontConfig(
    INFO("### MFRONT #####");

    //! \ogs_file_param(material__solid__constitutive_relation__type)
-    config.checkConfigParameter("type", "MFront");
+    auto const type = config.getConfigParameter<std::string>("type");
+    if (type != "MFront" && type != "MFrontRichardsMechanics")
+    {
+        OGS_FATAL(
+            "Expected constitutive relation type 'MFront' or "
+            "'MFrontRichardsMechanics', but got '{s}'.",
+            type);
+    }

    //! \ogs_file_param(material__solid__constitutive_relation__MFront__library)
    auto const library = config.getConfigSubtreeOptional("library");
diff --git a/MaterialLib/SolidModels/MFront/CreateMFrontRichardsMechanics.cpp b/MaterialLib/SolidModels/MFront/CreateMFrontRichardsMechanics.cpp
new file mode 100644
index 0000000000..16044f1bcd
--- /dev/null
+++ b/MaterialLib/SolidModels/MFront/CreateMFrontRichardsMechanics.cpp
@@ -0,0 +1,46 @@
+/**
+ * \file
+ * \copyright
+ * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */
+
+#include "CreateMFrontRichardsMechanics.h"
+
+#include "CreateMFrontGeneric.h"
+#include "MFrontRichardsMechanics.h"
+
+namespace MaterialLib::Solids::MFront
+{
+template <int DisplacementDim>
+std::unique_ptr<MechanicsBase<DisplacementDim>> createMFrontRichardsMechanics(
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+    std::optional<ParameterLib::CoordinateSystem> const& local_coordinate_system,
+    BaseLib::ConfigTree const& config)
+{
+    auto conf = createMFrontConfig(DisplacementDim, parameters, config);

```



```

+
+   return std::make_unique<MFrontRichardsMechanics<DisplacementDim>>({
+       std::move(conf.behaviour), std::move(conf.material_properties),
+       std::move(conf.state_variables_initial_properties),
+       local_coordinate_system});
+}
+} // namespace MaterialLib::Solids::MFront
+
+namespace MaterialLib::Solids::MFront
+{
+template std::unique_ptr<MechanicsBase<2>> createMFrontRichardsMechanics<2>({
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+    std::optional<ParameterLib::CoordinateSystem> const&
+        local_coordinate_system,
+    BaseLib::ConfigTree const& config});
+
+template std::unique_ptr<MechanicsBase<3>> createMFrontRichardsMechanics<3>({
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+    std::optional<ParameterLib::CoordinateSystem> const&
+        local_coordinate_system,
+    BaseLib::ConfigTree const& config});
+} // namespace MaterialLib::Solids::MFront
diff --git a/MaterialLib/SolidModels/MFront/CreateMFrontRichardsMechanics.h b/MaterialLib/SolidModels/MFront/CreateMFrontRichardsMechanics.h
new file mode 100644
index 0000000000..86d6fb2bcf
--- /dev/null
+++ b/MaterialLib/SolidModels/MFront/CreateMFrontRichardsMechanics.h
@@ -0,0 +1,41 @@
+/**
+ * * \file
+ * * \copyright
+ * * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * * Distributed under a Modified BSD License.
+ * * See accompanying file LICENSE.txt or
+ * * http://www.opengeosys.org/project/license
+ */
+
+#pragma once
+
+#include <memory>
+#include <vector>
+
+#include "BaseLib/ConfigTree-fwd.h"
+#include "MaterialLib/SolidModels/MechanicsBase.h"
+#include "ParameterLib/Parameter.h"
+
+namespace MaterialLib::Solids::MFront
+{
+template <int DisplacementDim>
+std::unique_ptr<MechanicsBase<DisplacementDim>> createMFrontRichardsMechanics(
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+    std::optional<ParameterLib::CoordinateSystem> const&
+        local_coordinate_system,
+    BaseLib::ConfigTree const& config);
+
+extern template std::unique_ptr<MechanicsBase<2>>
+createMFrontRichardsMechanics<2>({
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+    std::optional<ParameterLib::CoordinateSystem> const&
+        local_coordinate_system,
+    BaseLib::ConfigTree const& config});
+
+extern template std::unique_ptr<MechanicsBase<3>>
+createMFrontRichardsMechanics<3>({
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+    std::optional<ParameterLib::CoordinateSystem> const&
+        local_coordinate_system,
+    BaseLib::ConfigTree const& config});
+} // namespace MaterialLib::Solids::MFront
diff --git a/MaterialLib/SolidModels/MFront/MFrontGeneric.h b/MaterialLib/SolidModels/MFront/MFrontGeneric.h
index 8da07a8bf8..1792832f99 100644
--- a/MaterialLib/SolidModels/MFront/MFrontGeneric.h
+++ b/MaterialLib/SolidModels/MFront/MFrontGeneric.h
@@ -19,6 +19,7 @@
@@ -19,6 +19,7 @@
 #include <MGIS/Behaviour/BehaviourData.h>
 #include <MGIS/Behaviour/Integrate.h>
 #include <boost/mp11.hpp>
 #include <sstream>
 
 #include "MaterialLib/MPL/Utils/GetSymmetricTensor.h"
 #include "MaterialLib/MPL/Utils/Tensor.h"
@@ -568,9 +569,44 @@ public:
     auto const status = mgis::behaviour::integrate(v, _behaviour);
     if (status != 1)
     {
         throw NumLib::AssemblyException(
             auto const format_values = []() { auto const& values)
             {
                 std::ostringstream oss;
                 oss << "[";
                 for (std::size_t i = 0; i < values.size(); ++i)
                 {
                     if (i != 0)
                     {
                         oss << ", ";
                     }
                     oss << values[i];
                 }
                 oss << "]";
                 return oss.str();
             }
         );
     }
 
     std::string message =
         "MFront: integration failed with status " +
         std::to_string(status) + ".";
     std::to_string(status);
     if (behaviour_data.error_message != nullptr &&
         behaviour_data.error_message[0] != '\0')
     {
         message += ". ";
         message += behaviour_data.error_message;
     }
     message += fmt::format(
         " [dt={}, p_L_prev={}, p_L={}, S_L={}, T_prev={}, T={}, grads_prev={}, grads={}, forces_prev={}, isvs_prev={}]",
         dt, variable_array_prev.liquid_phase_pressure,
         variable_array_prev.liquid_phase_pressure,
         variable_array_prev.liquid_saturation,

```

```

+         variable_array.liquid_saturation,
+         variable_array_prev.temperature, variable_array.temperature,
+         format_values(behaviour_data.s0.gradients),
+         format_values(behaviour_data.s1.gradients),
+         format_values(behaviour_data.s0.thermodynamic_forces),
+         format_values(behaviour_data.s0.internal_state_variables));
+     message += ".";
+     throw NumLib::AssemblyException(message);
+ }

OGMFrontThermodynamicForcesData tdyn_forces_data;
diff --git a/MaterialLib/SolidModels/MFront/MFrontRichardsMechanics.cpp b/MaterialLib/SolidModels/MFront/MFrontRichardsMechanics.cpp
new file mode 100644
index 0000000000..326b0e0327
--- /dev/null
+++ b/MaterialLib/SolidModels/MFront/MFrontRichardsMechanics.cpp
@@ -0,0 +1,16 @@
+/**
+ * \file
+ * \copyright
+ * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */
+
+#include "MFrontRichardsMechanics.h"
+
+namespace MaterialLib::Solids::MFront
+{
+template class MFrontRichardsMechanics<2>;
+template class MFrontRichardsMechanics<3>;
+} // namespace MaterialLib::Solids::MFront
diff --git a/MaterialLib/SolidModels/MFront/MFrontRichardsMechanics.h b/MaterialLib/SolidModels/MFront/MFrontRichardsMechanics.h
new file mode 100644
index 0000000000..1ea697af56
--- /dev/null
+++ b/MaterialLib/SolidModels/MFront/MFrontRichardsMechanics.h
@@ -0,0 +1,228 @@
+/**
+ * \file
+ * \copyright
+ * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */
+
+#pragma once
+
+#include <algorithm>
+
+#include "MFrontGeneric.h"
+#include "Variable.h"
+
+namespace MaterialLib
+{
+namespace Solids
+{
+namespace MFront
+{
+template <int DisplacementDim>
+class MFrontRichardsMechanics
+ : private MFrontGeneric<DisplacementDim,
+     boost::mpl::mp_list<Strain, LiquidPressure>,
+     boost::mpl::mp_list<Stress, Saturation>,
+     boost::mpl::mp_list<Temperature>>,
+     public MechanicsBase<DisplacementDim>
+{
+public:
+    using Base = MFrontGeneric<DisplacementDim,
+        boost::mpl::mp_list<Strain, LiquidPressure>,
+        boost::mpl::mp_list<Stress, Saturation>,
+        boost::mpl::mp_list<Temperature>>;
+
+    using KelvinVector = typename Base::KelvinVector;
+    using KelvinMatrix = typename Base::KelvinMatrix;
+
+    public:
+        using PressureCoupledResponse =
+            typename MechanicsBase<DisplacementDim>::PressureCoupledResponse;
+
+        using Base::Base;
+
+        std::unique_ptr<
+            typename MechanicsBase<DisplacementDim>::MaterialStateVariables>
+        createMaterialStateVariables() const override
+        {
+            return Base::createMaterialStateVariables();
+        }
+
+        void initializeInternalStateVariables(
+            double const t,
+            ParameterLib::SpatialPosition const& x,
+            typename MechanicsBase<DisplacementDim>::MaterialStateVariables&
+            material_state_variables) const override
+        {
+            Base::initializeInternalStateVariables(t, x, material_state_variables);
+        }
+
+        std::optional<PressureCoupledResponse> integrateStressPressureCoupled(
+            MaterialPropertyLib::VariableArray const& variable_array_prev,
+            MaterialPropertyLib::VariableArray const& variable_array,
+            double const t,
+            ParameterLib::SpatialPosition const& x,
+            double const dt,
+            typename MechanicsBase<DisplacementDim>::MaterialStateVariables const&
+            material_state_variables) const override
+        {
+            auto res = Base::integrateStress(variable_array_prev,
+                variable_array,
+                t,
+                x,
+                dt,
+                material_state_variables);
+
+            if (!res)
+            {
+                return std::nullopt;
+            }
+        }
+    };
+}
+}
+}

```

```

+     }
+
+     auto& [forces_data, state, tangent_operator_data] = *res;
+     auto const view = this->createThermodynamicForcesView();
+     auto const swelling_stress =
+         getInternalVectorByName(*state, "sigma_5");
+     auto const liquid_mass_exchange_source =
+         -getInternalScalarByName(*state, "rho_l_hat");
+
+     PressureCoupledResponse response{
+         view.block(stress, forces_data),
+         view.block(saturation, forces_data),
+         blocks_view_.block(stress, strain, tangent_operator_data),
+         blocks_view_.block(stress, liquid_pressure, tangent_operator_data),
+         blocks_view_.block(saturation, strain, tangent_operator_data),
+         blocks_view_.block(saturation, liquid_pressure,
+             tangent_operator_data),
+
+         swelling_stress,
+         liquid_mass_exchange_source,
+         std::move(state));
+     return response;
+ }
+
+ std::optional<std::tuple<KelvinVector,
+     std::unique_ptr<typename MechanicsBase<
+         DisplacementDim>::MaterialStateVariables>,
+     KelvinMatrix>>
+ integrateStress(
+     MaterialPropertyLib::VariableArray const& variable_array_prev,
+     MaterialPropertyLib::VariableArray const& variable_array,
+     double const t,
+     ParameterLib::SpatialPosition const& x,
+     double const dt,
+     typename MechanicsBase<DisplacementDim>::MaterialStateVariables const&
+         material_state_variables) const override
+ {
+     auto response = integrateStressPressureCoupled(variable_array_prev,
+         variable_array,
+         t,
+         x,
+         dt,
+         material_state_variables);
+
+     if (!response)
+     {
+         return std::nullopt;
+     }
+
+     return std::optional<
+         std::tuple<KelvinVector,
+             std::unique_ptr<typename MechanicsBase<
+                 DisplacementDim>::MaterialStateVariables>,
+                 KelvinMatrix>>(<std::in_place,
+                     response->stress,
+                     std::move(response->state),
+                     response->dStress_dStrain);
+     }
+
+     std::vector<typename MechanicsBase<DisplacementDim>::InternalVariable>
+     getInternalVariables() const override
+     {
+         return Base::getInternalVariables();
+     }
+
+     double getBulkModulus(double const t,
+         ParameterLib::SpatialPosition const& x,
+         KelvinMatrix const& const C) const override
+     {
+         return Base::getBulkModulus(t, x, C);
+     }
+
+     double computeFreeEnergyDensity(
+         double const t,
+         ParameterLib::SpatialPosition const& x,
+         double const dt,
+         KelvinVector const& eps,
+         KelvinVector const& sigma,
+         typename MechanicsBase<DisplacementDim>::MaterialStateVariables const&
+             material_state_variables) const override
+     {
+         return Base::computeFreeEnergyDensity(
+             t, x, dt, eps, sigma, material_state_variables);
+     }
+
+ private:
+     KelvinVector getInternalVectorByName(
+         typename MechanicsBase<DisplacementDim>::MaterialStateVariables const&
+             material_state_variables,
+         std::string const& name) const
+     {
+         auto const internal_variables = Base::getInternalVariables();
+         auto const it =
+             std::find_if(internal_variables.begin(), internal_variables.end(),
+                 [&name](auto const& internal_variable)
+                 { return internal_variable.name == name; });
+
+         KelvinVector values = KelvinVector::Zero();
+         if (it == internal_variables.end() ||
+             it->num_components != values.rows())
+         {
+             return values;
+         }
+
+         std::vector<double> cache;
+         auto const& raw_values = it->getter(material_state_variables, cache);
+         for (Eigen::Index i = 0;
+             i < values.rows() &&
+             i < static_cast<Eigen::Index>(raw_values.size());
+             ++i)
+         {
+             values[i] = raw_values[static_cast<std::size_t>(i)];
+         }
+         return values;
+     }
+
+     double getInternalScalarByName(
+         typename MechanicsBase<DisplacementDim>::MaterialStateVariables const&

```

```

+         material_state_variables,
+         std::string const& name) const
+     {
+         auto const internal_variables = Base::getInternalVariables();
+         auto const it =
+             std::find_if(internal_variables.begin(), internal_variables.end(),
+                 [&name](auto const& internal_variable)
+                 { return internal_variable.name == name; });
+
+         if (it == internal_variables.end() || it->num_components != 1)
+         {
+             return 0.0;
+         }
+
+         std::vector<double> cache;
+         auto const& values = it->getter(material_state_variables, cache);
+         return values.empty() ? 0.0 : values[0];
+     }
+
+     OGSFrontTangentOperatorBlocksView<
+         DisplacementDim,
+         ForcesGradsCombinations<boost::mpl::mp_List<Strain, LiquidPressure>,
+             boost::mpl::mp_List<Stress, Saturation>,
+             boost::mpl::mp_List<Temperature>::type>
+     > blocks_view_ = this->createTangentOperatorBlocksView();
+ };
+
+extern template class MFrontRichardsMechanics<2>;
+extern template class MFrontRichardsMechanics<3>;
+
+} // namespace MFront
+} // namespace Solids
+} // namespace MaterialLib
diff --git a/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_constE_pressureCoupled.mfront b/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_constE_pressureCoupled.mfront
new file mode 100644
index 0000000000..e877bbcd5b
--- /dev/null
+++ b/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_constE_pressureCoupled.mfront
@@ -0,0 +1,255 @@
+@DSL ImplicitGenericBehaviour;
+@Behaviour ModCamClay_semiExpl_constE_pressureCoupled;
+@Author OpenGeoSys Community;
+@Date 26 / 03 / 2026;
+@Description{
+    Pressure-coupled RichardsMechanics bridge variant of
+    ModCamClay_semiExpl_constE. The mechanical return mapping stays on the
+    native effective-stress modified Cam-Clay path with constant elastic
+    parameters. Liquid pressure only enters through the returned van
+    Genuchten saturation and its tangent, so RichardsMechanics can assemble
+    the pore-pressure coupling outside the solid law exactly as on the native
+    path.
+}
+
+@Theta 1.0;
+@Epsilon 1e-14;
+@MaximumNumberOfIterations 200;
+@ModellingHypotheses("-.*");
+@Algorithm NewtonRaphson;
+
+@Gradient StrainTensor e**;
+e**.setGlossaryName("Strain");
+
+@ThermodynamicForce StressTensor sigma;
+sigma.setGlossaryName("Stress");
+
+@Gradient real 'p';
+'p'.setEntryName("LiquidPressure");
+
+@ThermodynamicForce real 's';
+'s'.setEntryName("Saturation");
+
+@StateVariable StrainTensor e*1;
+e*1.setGlossaryName("ElasticStrain");
+
+@StateVariable real A[];
+A[].setGlossaryName("EquivalentPlasticStrain");
+
+@IntegrationVariable strain rpc;
+
+@AuxiliaryStateVariable stress pc;
+pc.setEntryName("PreConsolidationPressure");
+
+@AuxiliaryStateVariable real e*[];
+e*[].setEntryName("PlasticVolumetricStrain");
+
+@AuxiliaryStateVariable real 'v';
+'v'.setEntryName("VolumeRatio");
+
+@Parameter stress pamb = 1e+3;
+@PhysicalBounds pamb in [0:*];
+pamb.setEntryName("AmbientPressure");
+
+@MaterialProperty stress young;
+@PhysicalBounds young in [0:*];
+young.setGlossaryName("YoungModulus");
+
+@MaterialProperty real v;
+@PhysicalBounds v in [-1:0.5];
+v.setGlossaryName("PoissonRatio");
+
+@MaterialProperty real M;
+@PhysicalBounds M in [0:*];
+M.setEntryName("CriticalStateLineSlope");
+
+@MaterialProperty real ka;
+@PhysicalBounds ka in [0:*];
+ka.setEntryName("SwellingLineSlope");
+
+@MaterialProperty real la;
+@PhysicalBounds la in [0:*];
+la.setEntryName("VirginConsolidationLineSlope");
+
+@MaterialProperty stress pc_char;
+@PhysicalBounds pc_char in [0:*];
+pc_char.setEntryName("CharacteristicPreConsolidationPressure");
+
+@MaterialProperty real 's';

```

```

+ 's.setEntryName("ResidualLiquidSaturation");
+
+ @MaterialProperty real "rS";
+ 's.setEntryName("ResidualGasSaturation");
+
+ @MaterialProperty real "p";
+ 'p.setEntryName("BubblePressure");
+
+ @MaterialProperty real "vcm";
+ 'vcm.setEntryName("VanGenuchtenExponent_m");
+
+ @LocalVariable bool withinElasticRange;
+ @LocalVariable real M2;
+ @LocalVariable real pc_min;
+ @LocalVariable real rpc_min;
+ @LocalVariable real K;
+ @LocalVariable StressTensor sigma;
+ @LocalVariable StiffnessTensor d0/depsilon;
+
+ @TangentOperatorBlocks d0/depsilon {, d0/dDelta*p, d-/dDelta*S, d-/dDelta*Sp};
+
+ @InitializeLocalVariables
+ {
+   const auto rS = [6](const auto "rP") {
+     return std::pow(
+       std::pow(std::max("rP", 0.0) / "p", 1.0 / (1.0 - "vcm")) + 1.0, -"vcm");
+   };
+
+   const auto S = rS(-("p" + Delta*p)) * (1.0 - "rS - 'S) + 'S;
+   'S = std::clamp(S, 'S, 1.0 - 'S);
+
+   tfel::raise_if((la < ka, "Invalid parameters: la<ka");
+   M2 = M * M;
+
+   epsilon = (1.0 + v) / young * sigma - v / young * trace0() * zI;
+   sigma = sigma;
+
+   rpc = pc / young;
+   pc_min = 1e-15 * pc_char;
+   rpc_min = pc_min / young;
+
+   K = young / (3.0 * (1.0 - 2.0 * v));
+   d0/depsilon = young / (1.0 + v) * Stensor4::K() + K * Stensor4::IxI();
+
+   const auto sigma = sigma + d0/depsilon * Delta*epsilon;
+   const auto 's = deviatoro("s");
+   const auto 'q = sqrt(1.5 * 's | 's);
+   const auto 'p = -traceo("l") / 3.0 + pamb;
+   const auto 'f = 'q * 'q + M2 * 'p * ('p - pc);
+   withinElasticRange = 'f < 0;
+ }
+
+ @ComputeThermodynamicForces
+ {
+   sigma = sigma + theta * d0/depsilon * Delta*epsilon;
+ }
+
+ @Integrator
+ {
+   constexpr const auto Pr4 = Stensor4::K();
+   const auto the = 'v / (la - ka);
+
+   if (withinElasticRange)
+   {
+     epsilon -= Delta*epsilon;
+     return true;
+   }
+
+   const auto epsr = strain(1.e-12);
+   const auto s = deviatoro();
+   const auto q = sqrt(1.5 * s | s);
+   const auto p = -trace0() / 3.0 + pamb;
+
+   const auto rpc_new = rpc + theta * Drpc;
+   const auto pc_new = rpc_new * young;
+
+   const auto f = q * q + M2 * p * (p - pc_new);
+   const auto d0/dp = M2 * (2.0 * p - pc_new);
+   const auto d0/dof = eval(3.0 * s - d0/dp * zI / 3.0);
+   auto norm = sqrt(6.0 * q * q + d0/dp * d0/dp / 3.0);
+   norm = max(norm, epsr * young);
+   const auto n = d0/dof / norm;
+   const auto ntr = d-/d0/dp / norm;
+
+   const auto Delta*epsilon = eval[Delta] ( * n);
+   const auto Delta*epsilon_v = traceo(Delta*epsilon);
+
+   const auto fchar = pc_char * young;
+
+   epsilon_f = Delta*epsilon + Delta*epsilon - Delta*epsilon;
+   [Delta] f = f / fchar;
+   frpc = Drpc + Delta*epsilon_v * the * (rpc_new - rpc_min);
+
+   const auto d0/danorm =
+     (9.0 * s - 2.0 * M2 / 9.0 * d0/dp * zI) / norm;
+   const auto d0/dDelta*epsilon_n =
+     (3.0 * Pr4 + 2.0 * M2 / 9.0 * (zI * zI - (n * d0/danorm)) / norm *
+     d0/depsilon * theta;
+   const auto d0/dAnrpc =
+     (zI + d0/dp * n / norm) * M2 / (3.0 * norm) * theta * young;
+   const auto d0/dDelta*epsilon_vfrpc = the * (rpc_new - rpc_min);
+
+   d0/depsilon_v/dDelta*epsilon_f += [Delta] * d0/dDelta*epsilon_n;
+   d0/depsilon_v/dDelta[Delta] f = n;
+   d0/depsilon_v/dDeltafrpc = [Delta] * d0/dAnrpc;
+
+   d0/dDelta*epsilon_f = d0/dof | d0/depsilon * theta / fchar;
+   d0/dDelta[Delta] f = strain(0);
+   d0/dDeltafrpc = -M2 * p * theta / fchar * young;
+
+   d0/dDelta[Delta] frpc = d0/dDelta*epsilon_vfrpc * ntr;
+   d0/dDelta*epsilon_vfrpc = d0/dDelta*epsilon_vfrpc * [Delta] * (zI | d0/dDelta*epsilon_n);
+   d0/dDeltafrpc =
+     1.0 + Delta*epsilon_v * the * theta + d0/dDelta*epsilon_vfrpc * [Delta] * traceo(d0/dAnrpc);
+ }
+
+ @ComputeFinalThermodynamicForces

```

```

+{
+  o = o + ∂o/∂εn * Δεn;
+}
+
+@UpdateAuxiliaryStateVariables
+{
+  pc += Δrpc * young;
+  εn ⋅ ⋅ += traceΔεn( - Δεn);
+  'v *= exp(traceΔεn());
+}
+
+@AdditionalConvergenceChecks
+{
+  if (converged)
+  {
+    if (!withinElasticRange)
+    {
+      if ΔA [ < 0)
+      {
+        converged = false;
+        withinElasticRange = true;
+      }
+    }
+  }
+}
+
+@TangentOperator
+{
+  const auto ∂2/∂2εnSp = [&](const auto **p) {
+    return (1.0 - εnS - 'S) *
+      ('S < (1.0 - εnS) && 'S > 'S
+        ? -vm / ('p * (1.0 - vm)) *
+          std::pow(std::max(std::max(**p, 0.0) / 'p,
+            1.0 / (1.0 - vm)) +
+            1.0,
+            -vm - 1.0) *
+          std::pow(std::max(**p, 0.0) / 'p,
+            1.0 / (1.0 - vm) - 1.0)
+        : 0.0);
+  };
+
+  ∂o/∂Δn*p = 0.0 * zI;
+  ∂2/∂Δεn*S = 0.0 * zI;
+  ∂2/∂Δn*Sp = ∂2/∂2εnSp(-('p + Δnp));
+
+  if ((smt == ELASTIC) || (smt == SECANTOPERATOR))
+  {
+    ∂o/∂Δεn = ∂o/∂εn;
+  }
+  else if (smt == CONSISTENTTANGENTOPERATOR)
+  {
+    Stensor4 Je;
+    getPartialJacobianInvert(Je);
+    ∂o/∂Δεn = ∂o/∂εn * Je;
+  }
+  else
+  {
+    return false;
+  }
+}
+
+diff --git a/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront b/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront
new file mode 100644
index 0000000000..9367067e16
--- /dev/null
+++ b/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront
@@ -0,0 +1,343 @@
+// DSL and behaviour declaration
+@DSL ImplicitGenericBehaviour;
+@Behaviour ModCamClay_semiExpl_coupled;
+@Author Aqeel Afzal Chaudhry, Christian Silbermann, Eric Simo, Miguel Mánica, Thomas Helfer, Thomas Nagel;
+@Date 01/09/2025;
+@Description{
+  + The modified cam-clay model according to Callari (1998):
+  + "A finite-strain cam-clay model in the framework of multiplicative elasto-plasticity"
+  + but here in a consistent geometrically linear form (linearized volume ratio evolution)
+  + semi-explicit due to explicit volume ratio update at the end of time step,
+  + nonlinear hypoelastic behavior: pressure-dependent bulk modulus, constant Poisson ratio,
+  + incremental formulation assuming constant elastic parameters over the time step,
+  + normalized plastic flow direction, lower limit for a minimal pre-consolidation pressure,
+  + with hydraulic and thermal coupling, saturation dependent swelling,
+}
+
+
+@Theta 1.0; // time integration scheme
+@Epsilon 1e-14; // tolerance of local stress integration algorithm
+@MaximumNumberOfIterations 100; // for local local stress integration algorithm
+@ModellingHypotheses(".*"); // supporting all stress and strain states
+@Algorithm NewtonRaphson; // NumericalJacobian;LevenbergMarquardt
+
+// Gradients and state variables
+@Gradient StrainTensor εn;
+εn-.setGlossaryName("Strain");
+
+@ThermodynamicForce StressTensor σ;
+σ-.setGlossaryName("Stress");
+
+
+@Gradient real 'p;
+'p-.setEntryName("LiquidPressure");
+
+
+@ThermodynamicForce real 'S;
+'S-.setEntryName("Saturation");
+
+
+@StateVariable StressTensor σn;
+σn-.setEntryName("swelling_stress");
+
+
+@StateVariable StrainTensor εn;
+εn-.setGlossaryName("ElasticStrain");
+
+
+@StateVariable real A [;
+A-.setGlossaryName("EquivalentPlasticStrain");
+
+
+@IntegrationVariable strain rpc;
+
+
+@AuxiliaryStateVariable stress pc;
+pc-.setEntryName("PreConsolidationPressure");
+
+
+@AuxiliaryStateVariable real εn ⋅ ⋅;
+εn ⋅ ⋅-.setEntryName("PlasticVolumetricStrain");

```

```

+
+@AuxiliaryStateVariable real v;
+v.setEntryName("VolumeRatio");
+
+
+@MaterialProperty real v;
+v.setGlossaryName("PoissonRatio");
+
+@MaterialProperty real M;
+M.setEntryName("CriticalStateLineSlope");
+
+@MaterialProperty real ka;
+ka.setEntryName("SwellingLineSlope");
+
+@MaterialProperty real la;
+la.setEntryName("VirginConsolidationLineSlope");
+
+@MaterialProperty stress pc_char;
+pc_char.setEntryName("CharacteristicPreConsolidationPressure");
+
+@MaterialProperty real v;
+v.setEntryName("InitialVolumeRatio");
+
+@MaterialProperty real a;
+a.setGlossaryName("ThermalExpansion");
+@MaterialProperty real a;
+a.setEntryName("BiotCoefficient");
+@MaterialProperty real S;
+S.setEntryName("ResidualLiquidSaturation");
+@MaterialProperty real S;
+S.setEntryName("ResidualGasSaturation");
+@MaterialProperty real p;
+p.setEntryName("BubblePressure");
+@MaterialProperty real m;
+m.setEntryName("VanGenuchtenExponent_m");
+
+@MaterialProperty real bishop_type;
+bishop_type.setEntryName("BishopsModelType");
+
+@MaterialProperty real bishop_value;
+bishop_value.setEntryName("BishopsModelValue");
+
+// Swelling properties (SaturationDependentSwelling as in OGS) but isotropic for the time being:
+@MaterialProperty real p;
+p.setEntryName("SwellingPressures");
+@MaterialProperty real m;
+m.setEntryName("Exponents");
+@MaterialProperty real S;
+S.setEntryName("LowerSaturationLimit");
+@MaterialProperty real S;
+S.setEntryName("UpperSaturationLimit");
+
+// local variables
+@LocalVariable real K;
+@LocalVariable real phi;
+@LocalVariable real phi;
+@LocalVariable real phi/a;
+@LocalVariable real Delta*p;
+@LocalVariable real x;
+@LocalVariable real x;
+@LocalVariable real dx/a;
+@LocalVariable StressTensor sigma;
+@LocalVariable StiffnessTensor da/depsilon;
+@LocalVariable bool withinElasticRange;
+@LocalVariable real M;
+@LocalVariable real young;
+@LocalVariable real pc_min;
+@LocalVariable real rpc_min;
+
+//TangentOperatorBlocks da/depsilon{{, da/delta*p, a-/delta*S}};
+@TangentOperatorBlocks da/depsilon{{, da/delta*T, da/delta*p, a-/delta*S}};
+
+@InitializeLocalVariables
+{
+    // retention
+    const auto s = [s](const auto **p) {
+        return std::pow(
+            std::pow(std::max(**p, 0.0) / *p, 1.0 / (1.0 - v_m)) + 1.0, -v_m);
+    };
+    // previous
+    const double s = s(-(**p)) * (1.0 - v_s - v_s) + v_s;
+    const double s = std::clamp(s, v_s, 1.0 - v_s);
+    // new
+    const double S = s(-(**p + Delta*p)) * (1.0 - v_s - v_s) + v_s;
+    S = std::clamp(S, v_s, 1.0 - v_s);
+
+    // Bishop power law or cutoff
+    const int model_type = static_cast<int>(bishop_type);
+    if (model_type == 0) {
+        x = std::pow(S, bishop_value);
+        dx/delta*S = bishop_value * std::pow(S, bishop_value - 1);
+        x = std::pow(s, bishop_value);
+    }
+    else if (model_type == 1) {
+        const double S_cut = bishop_value;
+        if (S < S_cut) {
+            x = 0.0;
+            dx/delta*S = 0.0;
+        } else if (S > 1.0) {
+            x = 1.0;
+            dx/delta*S = 0.0;
+        } else {
+            const double denom = std::max(1e-15, 1.0 - S_cut);
+            x = (S - S_cut) / denom;
+            dx/delta*S = 1.0 / denom;
+        }
+        x = (S < S_cut) ? 0.0 :
+            (S > 1.0) ? 1.0 :
+            (S - S_cut) / std::max(1e-15, 1.0 - S_cut);
+    }
+    else {
+        throw std::runtime_error("Invalid BishopsModelType: must be 0 (power law) or 1 (cutoff).");
+    }
+
+    // SaturationDependentSwelling
+    if (S <= S)
+    {

```

```

+      $\varphi = 0.0;$ 
+      $\partial\varphi/\partial^{\circ}S = 0.0;$ 
+ }
+ else if (°S >= °S)
+ {
+      $\varphi = 1.0;$ 
+      $\partial\varphi/\partial^{\circ}S = 0.0;$ 
+ }
+ else
+ {
+     const double normalized = (°S - °S) / (°S - °S);
+      $\varphi = \text{std::pow}(\text{normalized}, \text{"m"});$ 
+      $\partial\varphi/\partial^{\circ}S = \text{"m"} / (°S - °S) \cdot \text{std::pow}(\text{normalized}, \text{"m"} - 1);$ 
+ }
+
+  $\varphi_s = \text{std::pow}((1 + S - °S) / (°S - °S), \text{"m"});$ 
+
+ // Swelling stress contribution:
+  $\Delta^{\circ}p = \text{"p"} \cdot \varphi(1 - \varphi_s);$ 
+
+ tfel::raise_if((la < ka, "Invalid parameters: la<ka"));
+ M² = M · M;
+
+ // update  $\sigma_s \rightarrow$  effective
+  $\sigma_s = \sigma + \alpha^s \cdot \chi_s \cdot \text{"p"} \cdot zI;$ 
+
+ // compute elastic stiffness (constant during time step)
+ pc_min = 0.5e-8 · pc_char;
+ const auto p = -trace( $\sigma_s$ ) / 3;
+ K = °v / ka · max(p, pc_char);
+ const auto E = 3.0 · K · (1.0 - 2 ·  $\nu$ );
+
+ young = E;
+ rpc = pc / young;
+ rpc_min = pc_min / young;
+
+ // stress derivative
+  $\partial\sigma/\partial\epsilon^{\circ} = E / (1.0 + \nu) \cdot \text{Stensor4::K}() + K \cdot \text{Stensor4::IxI}();$ 
+
+ // elastic trial stress
+ const auto  $\sigma^{\circ} = \sigma_s + \partial\sigma/\partial\epsilon^{\circ} \cdot \Delta\epsilon^{\circ} - 3 \cdot K \cdot \alpha \cdot \Delta T \cdot zI - \Delta^{\circ}p \cdot zI;$ 
+
+ // elastic estimators
+ const auto °s = deviatoro°();
+ const auto °q = sqrt(1.5 · °s | °s);
+ const auto °p = -trace( $\sigma^{\circ}$ ) / 3;
+
+ const auto °pc = pc;
+ const auto °f = °q · °q + M² · °p · (°p - °pc);
+ withinElasticRange = °f < 0;
+}
+
+@ComputeThermodynamicForces{
+      $\sigma = \sigma_s + \theta \cdot \partial\sigma/\partial\epsilon^{\circ} \cdot \Delta\epsilon^{\circ} - 3 \cdot K \cdot \alpha \cdot \Delta T \cdot zI - \Delta^{\circ}p \cdot zI;$ 
+}
+
+@Integrator
+{
+     constexpr const auto Pr4 = Stensor4::K();
+     const auto the = °v / (la - ka);
+     // elastic range:
+     if (withinElasticRange)
+     {
+          $\epsilon^{\circ}f = \Delta\epsilon^{\circ};$ 
+          $\sigma/ = \sigma_s + \partial\sigma/\partial\epsilon^{\circ} \cdot \Delta\epsilon^{\circ} - 3 \cdot K \cdot \alpha \cdot \Delta T \cdot zI - \alpha^s \cdot (\text{"p"} + \Delta^{\circ}p) \cdot zI - \Delta^{\circ}p \cdot zI;$ 
+         return true;
+     }
+     // plastic range:
+     const auto epsr = strain(1.e-12);
+     // calculate invariants from current stress sig
+     const auto s = deviatoro();
+     const auto q = sqrt(1.5 · s | s);
+     const auto p = -trace() / 3;
+     // update the internal (state) variables (rpc holds the old value!)
+     const auto rpc_new = rpc +  $\theta \cdot \Delta\text{rpc};$ 
+     const auto pc_new = rpc_new · young;
+     // calculate the direction of plastic flow
+     const auto f = q · q + M² · p · (p - pc_new);
+     const auto  $\partial f/\partial p = M^2 \cdot (2 \cdot p - pc\_new);$ 
+     const auto  $\partial\sigma f = \text{eval}(3 \cdot s - \partial f/\partial p \cdot zI / 3);$ 
+     auto norm = sqrt(6 · q · q +  $\partial f/\partial p \cdot \partial f/\partial p / 3$ ); // = sqrt( $\partial(\sigma f)/\partial\sigma f$ );
+     norm = max(norm, epsr · young);
+     const auto n =  $\partial f/\partial\sigma f / \text{norm};$ 
+     const auto ntr =  $\partial - f/\partial p / \text{norm};$ 
+     // plastic strain and volumetric part
+     const auto  $\Delta\epsilon^{\circ} = \text{eval}[\Delta\Lambda] \cdot n;$ 
+     const auto  $\Delta\epsilon^{\circ}v = \text{trace}(\Delta\epsilon^{\circ});$ 
+
+     const auto fchar = pc_char · young;
+
+     // residual
+      $\epsilon^{\circ}f = \Delta\epsilon^{\circ} + \Delta\epsilon^{\circ} - \Delta\epsilon^{\circ};$ 
+      $\Lambda[] f = f / fchar;$ 
+     frpc =  $\Delta\text{rpc} + \Delta\epsilon^{\circ}v \cdot \text{the} \cdot (\text{rpc\_new} - \text{rpc\_min});$ 
+
+     // auxiliary derivatives
+     const auto  $\partial/\partial\sigma\text{norm} = (9 \cdot s - 2 \cdot M^2 / 9 \cdot \partial f/\partial p \cdot zI) / \text{norm};$ 
+     const auto  $\partial/\partial\Delta\epsilon^{\circ}n = (3 \cdot \text{Pr4} + 2 \cdot M^2 / 9 \cdot (zI \otimes zI) - (n \otimes \partial/\partial\sigma\text{norm})) / \text{norm} \cdot \partial\sigma/\partial\epsilon^{\circ} \cdot \theta;$ 
+     const auto  $\partial/\partial\Delta\text{rpc} = (zI + \partial f/\partial p \cdot n / \text{norm}) \cdot M^2 / (3 \cdot \text{norm}) \cdot \theta \cdot \text{young};$ 
+     const auto  $\partial/\partial\Delta\epsilon^{\circ}v\text{frpc} = \text{the} \cdot (\text{rpc\_new} - \text{rpc\_min});$ 
+
+     // jacobian (all other parts are zero)
+      $\partial\epsilon^{\circ}/\partial\Delta\epsilon^{\circ}f = \Delta\Lambda[] \cdot \partial/\partial\Delta\epsilon^{\circ}n;$ 
+      $\partial\epsilon^{\circ}/\partial\Delta\Lambda[] f = n;$ 
+      $\partial\epsilon^{\circ}/\partial\Delta\text{rpc} = \Delta\Lambda[] \cdot \partial/\partial\Delta\text{rpc};$ 
+
+      $\partial\Lambda[]/\partial\Delta\epsilon^{\circ}f = \partial(\partial\sigma f / \partial\sigma/\partial\epsilon^{\circ}) \cdot \theta / fchar;$  // in case of problems with zero use:
+      $\partial\Lambda[]/\partial\Delta\Lambda[] f = \text{strain}(\theta);$  // (q<epsr) ? strain(1) : strain(0);
+      $\partial\Lambda[]/\partial\Delta\text{rpc} = -M^2 \cdot p \cdot \theta / fchar \cdot \text{young};$ 
+
+      $\partial/\partial\Delta\Lambda[] \text{frpc} = \partial/\partial\Delta\epsilon^{\circ}v\text{frpc} \cdot \text{ntr};$ 
+      $\partial/\partial\Delta\epsilon^{\circ}v\text{frpc} = \partial/\partial\Delta\epsilon^{\circ}v\text{frpc} \cdot \Delta\Lambda[] \cdot (zI | \partial/\partial\Delta\epsilon^{\circ}n);$ 
+      $\partial/\partial\Delta\text{rpc}\text{frpc} = 1 + \Delta\epsilon^{\circ}v \cdot \text{the} \cdot \theta + \partial/\partial\Delta\epsilon^{\circ}v\text{frpc} \cdot \Delta\Lambda[] \cdot \text{trace}(\partial/\partial\Delta\text{rpc});$ 
+}
+
+@ComputeFinalThermodynamicForces{
+      $\sigma = \sigma_s + \partial\sigma/\partial\epsilon^{\circ} \cdot \Delta\epsilon^{\circ} - 3 \cdot K \cdot \alpha \cdot \Delta T \cdot zI - \alpha^s \cdot \chi \cdot \text{"p"} \cdot zI - \Delta^{\circ}p \cdot zI;$ 
+}

```



```

+}
+
+// explicit treatment as long as change of 'v (or e) during time increment is small
+@UpdateAuxiliaryStateVariables
+{
+  o** = Δ**p · zI;
+  pc += Δpc · young;
+  ε** += traceΔε**(- Δε** - α · ΔT · zI); //Thermal strain subtracted
+  'v += 'v · traceΔε**(- α · ΔT · zI); //Thermal strain subtracted
+}
+
+@AdditionalConvergenceChecks
+{
+  if (converged)
+  {
+    if (!withinElasticRange)
+    {
+      if ΔAQ ( < 0)
+      {
+        converged = false;
+        withinElasticRange = true;
+      }
+    }
+  }
+}
+
+@TangentOperator // because no Brick StandardElasticity
+{
+  const auto ∂/∂**Sp = [&](const auto **p) {
+    return (1.0 - ε**S - 'v**S) · (1 < (1 - ε**S) && 'v**S > 'v**S
+      ? -v**m / ('p · (1 - v**m)) · std::pow(
+        std::pow(std::max(**p, 0.0) / 'p,
+          1.0 / (1.0 - v**m)) +
+          1.0,
+        -v**m - 1) · std::pow(std::max(**p, 0.0) / 'p,
+          1.0 / (1.0 - v**m) - 1.0)
+      : 0.);
+  };
+  ∂o/∂Δ**p = - α* · χ(- ('*p + Δ**p) · ∂χ/∂**S · ∂/∂**Sp(-('*p + Δ**p))) · zI + 'p · ∂q/∂**S · ∂/∂**Sp(-('*p + Δ**p)) · zI;
+  ∂o/∂ΔT = -3 · K · α · zI;
+  ∂/∂Δ**Sp = - ∂/∂**Sp(-('*p + Δ**p));
+
+  if ((smt == ELASTIC) || (smt == SECANTOPERATOR))
+  {
+    ∂o/∂Δε** = ∂o/∂ε**;
+  }
+  else if (smt == CONSISTENTTANGENTOPERATOR)
+  {
+    Stensor4 Je;
+    getPartialJacobianInvert(Je);
+    ∂o/∂Δε** = ∂o/∂ε** · Je;
+  }
+  else
+  {
+    return false;
+  }
+}
+
+diff --git a/MaterialLib/SolidModels/MFront/RichardsMechanicsDSMMicroMacroBridge.mfront b/MaterialLib/SolidModels/MFront/RichardsMechanicsDSMMicroMacroBridge.mfront
new file mode 100644
index 0000000000..7823db047e
--- /dev/null
+++ b/MaterialLib/SolidModels/MFront/RichardsMechanicsDSMMicroMacroBridge.mfront
@@ -0,0 +1,587 @@
+@DSL DefaultGenericBehaviour;
+@Behaviour RichardsMechanicsDSMMicroMacroBridge;
+@Author OpenGeoSys Community;
+@Date 2026 / 03 / 24;
+@Description {
+  +Reduced dsm_micromacro-aligned RichardsMechanics bridge for the verified shared
+  +pressure-history boundary. This bridge replaces the earlier logistic pressure
+  +surrogate by a local microstate update with phi_m := n_l, a reduced micro-liquid
+  +EOS, a van-der-Waals micro potential, and an isotropic swelling-stress update.
+  +The verified regression boundary of this step is the nonnegative pressure path
+  +used by the committed dsm_micromacro overlap-transfer baseline.
+}
+
+@Gradient StrainTensor eps_to;
+eps_to.setGlossaryName("Strain");
+
+@Gradient real p_LR;
+p_LR.setEntryName("LiquidPressure");
+
+@Flux StressTensor sigma;
+sigma.setGlossaryName("Stress");
+
+@Flux real S_L;
+S_L.setEntryName("Saturation");
+
+@TangentOperatorBlocks{∂/∂Δsigmaeps_to, ∂/∂Δsigma_LR, ∂/∂ΔS_Lepts_to,
+  ∂/∂ΔS_Lp_LR};
+
+@MaterialProperty stress E;
+E.setGlossaryName("YoungModulus");
+@MaterialProperty real nu;
+nu.setGlossaryName("PoissonRatio");
+
+@MaterialProperty real swelling_slope;
+swelling_slope.setEntryName("SwellingSlope");
+@MaterialProperty real mass_exchange_coefficient;
+mass_exchange_coefficient.setEntryName("MassExchangeCoefficient");
+@MaterialProperty real rho_LR_ref;
+rho_LR_ref.setEntryName("ReferenceLiquidDensityMacro");
+@MaterialProperty real rho_l0;
+rho_l0.setEntryName("ReferenceLiquidDensityMicro");
+@MaterialProperty real rho_SR;
+rho_SR.setEntryName("ReferenceDensitySolid");
+@MaterialProperty real density_a;
+density_a.setEntryName("MicroLiquidDensityA");
+@MaterialProperty real density_b;
+density_b.setEntryName("MicroLiquidDensityB");
+@MaterialProperty real hamaker_constant;
+hamaker_constant.setEntryName("HamakerConstant");
+@MaterialProperty real specific_surface;
+specific_surface.setEntryName("SpecificSurface");
+@MaterialProperty real phi0;
+phi0.setEntryName("InitialPorosity");

```

```

+@MaterialProperty real area_factor_tuller;
+area_factor_tuller.setEntryName("AreaFactorTuller");
+@MaterialProperty real pore_area_shape_factor_tuller;
+pore_area_shape_factor_tuller.setEntryName("PoreAreaShapeFactorTuller");
+@MaterialProperty real characteristic_pore_size;
+characteristic_pore_size.setEntryName("CharacteristicPoreSize");
+@MaterialProperty real surface_tension;
+surface_tension.setEntryName("SurfaceTension");
+
+
+@StateVariable real n_l;
+n_l.setEntryName("n_l");
+@StateVariable real rho_LR;
+rho_LR.setEntryName("rho_LR");
+@StateVariable real epsilon_sw;
+epsilon_sw.setEntryName("epsilon_sw");
+
+
+@AuxiliaryStateVariable real phi_m;
+phi_m.setEntryName("phi_m");
+@AuxiliaryStateVariable real phi_M;
+phi_M.setEntryName("phi_M");
+@AuxiliaryStateVariable real mu_LR_value;
+mu_LR_value.setEntryName("mu_LR");
+@AuxiliaryStateVariable real rho_L_hat_value;
+rho_L_hat_value.setEntryName("rho_L_hat");
+
+
+@ProvidesTangentOperator;
+@Integrator {
+  constexpr const auto pi = 3.14159265358979323846;
+  constexpr const auto pressure_tolerance = 1e-12;
+  constexpr const auto minimum_n_l = 1e-14;
+  constexpr const auto minimum_density_gap = 1e-12;
+  // Finite-difference tangent probes around the benchmark equilibrium hit
+  // round-off before 1e-12, which can trigger a spurious line-search failure
+  // in an already converged microstate solve.
+  constexpr const auto newton_tolerance = 1e-11;
+  constexpr const auto max_iterations = 64;
+
+  + tfel::raise_if(phi0 <= 0.0 || phi0 >= 1.0,
+    "Invalid parameter: InitialPorosity must lie in (0,1).");
+  + tfel::raise_if(rho_LR_ref <= 0.0 || rho_L0 <= 0.0 || rho_SR <= 0.0,
+    "Invalid density parameter: all reference densities must be positive.");
+  + tfel::raise_if(specific_surface <= 0.0,
+    "Invalid parameter: SpecificSurface must be positive.");
+  + tfel::raise_if(characteristic_pore_size <= 0.0,
+    "Invalid parameter: CharacteristicPoreSize must be positive.");
+  + tfel::raise_if(surface_tension <= 0.0,
+    "Invalid parameter: SurfaceTension must be positive.");
+
+  + const auto lambda = computeLambda(E, nu);
+  + const auto mu = computeMu(E, nu);
+  + const auto bulk_modulus = E / (3.0 * (1.0 - 2.0 * nu));
+  + const auto p = p_LR + dp_LR;
+  + const auto delta_epsilon_v = trace(d(eps_to);
+  + const auto zero_dt = std::abs(dt) < 1e-15;
+  + const auto n_s = 1.0 - phi0;
+  + const auto n_l_prev = n_l;
+  + const auto rho_LR_prev = rho_LR;
+  + const auto phi_m_prev = n_l_prev;
+  + const auto previous_mass = n_l_prev * rho_LR_prev;
+
+  + const auto saturation_from_pressure = [&](const auto p_arg) {
+  +   if (p_arg < -pressure_tolerance)
+  +   {
+  +     const auto capillary_prefactor =
+  +       4.0 * pore_area_shape_factor_tuller * surface_tension * surface_tension /
+  +       (area_factor_tuller * characteristic_pore_size * characteristic_pore_size);
+  +     const auto capillary_term = capillary_prefactor / (p_arg * p_arg);
+  +     return 1.0 - std::exp(-capillary_term);
+  +   }
+  +   return 1.0;
+  + };
+
+  + const auto mu_macro_from_pressure = [&](const auto p_arg,
+    const auto rho_macro_arg) {
+  +   const auto rho_macro_safe =
+  +     std::max(rho_macro_arg, rho_LR_ref + minimum_density_gap);
+  +   return p_arg > -pressure_tolerance ? 0.0 : p_arg / rho_macro_safe;
+  + };
+
+  + const auto omega_from_state = [&](const auto n_l_arg, const auto rho_LR_arg) {
+  +   return n_l_arg * rho_LR_arg / (n_s * rho_SR);
+  + };
+
+  + const auto rho_micro_eos = [&](const auto n_l_arg, const auto rho_LR_arg) {
+  +   const auto omega = std::max(omega_from_state(n_l_arg, rho_LR_arg), minimum_n_l);
+  +   return rho_L0 * std::exp(-density_a * std::pow(omega, density_b)) + rho_LR_ref;
+  + };
+
+  + const auto mu_micro_from_state = [&](const auto n_l_arg, const auto rho_LR_arg) {
+  +   const auto omega = std::max(omega_from_state(n_l_arg, rho_LR_arg), minimum_n_l);
+  +   return (hamaker_constant * rho_LR_arg * rho_LR_arg * rho_LR_arg / (6.0 * pi)) *
+  +     std::pow(specific_surface / omega, 3.0);
+  + };
+
+  +
+  + const auto solve_microstate = [&](const auto p_arg,
+    const auto delta_epsilon_v_arg,
+    const auto n_l_prev_arg,
+    const auto rho_LR_prev_arg,
+    double& n_l_out,
+    double& rho_LR_out,
+    double& mu_LR_out,
+    double& rho_L_hat_out) {
+  +   const auto rho_macro_step =
+  +     std::max(rho_LR_prev_arg, rho_LR_ref + minimum_density_gap);
+  +   auto n_l_trial = std::max(n_l_prev_arg, minimum_n_l);
+  +   auto rho_LR_trial = std::max(rho_LR_prev_arg, rho_LR_ref + minimum_density_gap);
+  +   auto converged = false;
+
+  +   const auto evaluate = [&](const auto n_l_arg,
+     const auto rho_LR_arg,
+     double& residual_1,
+     double& residual_2,
+     double& mu_LR_arg,
+     double& rho_L_hat_arg) {
+  +     const auto mu_LR_arg = mu_macro_from_pressure(p_arg, rho_macro_step);
+  +     mu_LR_arg = mu_micro_from_state(n_l_arg, rho_LR_arg);
+  +     rho_L_hat_arg = mass_exchange_coefficient * (mu_LR_arg - mu_LR_arg);

```

```

+ residual_l = n_l_arg * rho_LR_arg - previous_mass - dt * rho_l_hat_arg -
+ n_l_arg * rho_LR_arg * delta_epsilon_v_arg;
+ residual_2 = rho_LR_arg - rho_micro_eos(n_l_arg, rho_LR_arg);
+ };
+
+ const auto solve_rho_for_n_l = [&](const auto n_l_arg,
+ const auto rho_initial_guess,
+ double& rho_LR_out_arg) {
+ auto rho_LR_trial_local =
+ std::max(rho_initial_guess, rho_LR_ref + minimum_density_gap);
+
+ for (unsigned int iteration = 0; iteration < 4 * max_iterations;
+ ++iteration)
+ {
+ const auto rho_candidate = rho_micro_eos(n_l_arg, rho_LR_trial_local);
+ const auto rho_change = std::abs(rho_candidate - rho_LR_trial_local);
+ rho_LR_trial_local = std::max(rho_candidate, rho_LR_ref + minimum_density_gap);
+
+ if (rho_change <
+ newton_tolerance * std::max(1.0, std::abs(rho_LR_trial_local)))
+ {
+ rho_LR_out_arg = rho_LR_trial_local;
+ return true;
+ }
+ }
+
+ rho_LR_out_arg = rho_LR_trial_local;
+ return false;
+ };
+
+ const auto evaluate_reduced = [&](const auto n_l_arg,
+ const auto rho_initial_guess,
+ double& residual_l,
+ double& rho_LR_arg,
+ double& mu_LR_arg,
+ double& rho_l_hat_arg) {
+ const auto rho_converged =
+ solve_rho_for_n_l(n_l_arg, rho_initial_guess, rho_LR_arg);
+ const auto mu_LR_arg = mu_macro_from_pressure(p_arg, rho_macro_step);
+ mu_LR_arg = mu_micro_from_state(n_l_arg, rho_LR_arg);
+ rho_l_hat_arg = mass_exchange_coefficient * (mu_LR_arg - mu_LR_arg);
+ residual_l =
+ n_l_arg * rho_LR_arg - previous_mass - dt * rho_l_hat_arg -
+ n_l_arg * rho_LR_arg * delta_epsilon_v_arg;
+ return rho_converged;
+ };
+
+ const auto solve_microstate_bracketed =
+ [&](double& n_l_bracketed_out,
+ double& rho_LR_bracketed_out,
+ double& mu_LR_bracketed_out,
+ double& rho_l_hat_bracketed_out) {
+ constexpr const auto bracket_scan_points = 256u;
+ auto n_l_lo = minimum_n_l;
+ auto n_l_hi = std::max(phi0, std::max(n_l_prev_arg, minimum_n_l * 10.0));
+
+ double residual_lo = 0.0;
+ double residual_hi = 0.0;
+ double rho_lo = 0.0;
+ double rho_hi = 0.0;
+ double mu_lo = 0.0;
+ double mu_hi = 0.0;
+ double rho_hat_lo = 0.0;
+ double rho_hat_hi = 0.0;
+
+ auto bracket_ok =
+ evaluate_reduced(n_l_lo, rho_LR_prev_arg, residual_lo, rho_lo,
+ mu_lo, rho_hat_lo) &&
+ evaluate_reduced(n_l_hi, rho_LR_prev_arg, residual_hi, rho_hi,
+ mu_hi, rho_hat_hi);
+
+ if (!bracket_ok)
+ {
+ return false;
+ }
+
+ if (residual_lo * residual_hi > 0.0)
+ {
+ const auto log_n_l_min = std::log10(minimum_n_l);
+ const auto log_n_l_max = std::log10(n_l_hi);
+ auto previous_n_l = n_l_lo;
+ auto previous_residual = residual_lo;
+ auto previous_rho = rho_lo;
+ auto previous_mu = mu_lo;
+ auto previous_rho_hat = rho_hat_lo;
+ auto found_bracket = false;
+
+ for (unsigned int i = 1; i <= bracket_scan_points; ++i)
+ {
+ const auto log_n_l =
+ log_n_l_min +
+ (log_n_l_max - log_n_l_min) *
+ (static_cast<double>(i) / bracket_scan_points);
+ const auto n_l_candidate = std::pow(10.0, log_n_l);
+
+ double residual_candidate = 0.0;
+ double rho_candidate = 0.0;
+ double mu_candidate = 0.0;
+ double rho_hat_candidate = 0.0;
+ if (!evaluate_reduced(n_l_candidate, rho_LR_prev_arg,
+ residual_candidate, rho_candidate,
+ mu_candidate, rho_hat_candidate))
+ {
+ return false;
+ }
+
+ if (previous_residual * residual_candidate <= 0.0)
+ {
+ n_l_lo = previous_n_l;
+ residual_lo = previous_residual;
+ rho_lo = previous_rho;
+ mu_lo = previous_mu;
+ rho_hat_lo = previous_rho_hat;
+
+ n_l_hi = n_l_candidate;
+ residual_hi = residual_candidate;
+ rho_hi = rho_candidate;

```

```

+         mu_hi = mu_candidate;
+         rho_hat_hi = rho_hat_candidate;
+         found_bracket = true;
+         break;
+     }
+
+     previous_n_l = n_l_candidate;
+     previous_residual = residual_candidate;
+     previous_rho = rho_candidate;
+     previous_mu = mu_candidate;
+     previous_rho_hat = rho_hat_candidate;
+ }
+
+ if (!found_bracket)
+ {
+     return false;
+ }
+ }
+
+ for (unsigned int iteration = 0; iteration < 4 * max_iterations;
+     ++iteration)
+ {
+     const auto n_l_mid = 0.5 * (n_l_lo + n_l_hi);
+     double residual_mid = 0.0;
+     double rho_mid = 0.0;
+     double mu_mid = 0.0;
+     double rho_hat_mid = 0.0;
+
+     if (!evaluate_reduced(n_l_mid, rho_LR_prev_arg, residual_mid,
+                         rho_mid, mu_mid, rho_hat_mid))
+     {
+         return false;
+     }
+
+     if (std::abs(residual_mid) < newton_tolerance ||
+         std::abs(n_l_hi - n_l_lo) <
+             newton_tolerance * std::max(1.0, std::abs(n_l_mid)))
+     {
+         n_l_bracketed_out = n_l_mid;
+         rho_LR_bracketed_out = rho_mid;
+         mu_LR_bracketed_out = mu_mid;
+         rho_l_hat_bracketed_out = rho_hat_mid;
+         return true;
+     }
+
+     if (residual_lo * residual_mid <= 0.0)
+     {
+         n_l_hi = n_l_mid;
+         residual_hi = residual_mid;
+     }
+     else
+     {
+         n_l_lo = n_l_mid;
+         residual_lo = residual_mid;
+     }
+ }
+
+ const auto n_l_mid = 0.5 * (n_l_lo + n_l_hi);
+ double residual_mid = 0.0;
+ double rho_mid = 0.0;
+ double mu_mid = 0.0;
+ double rho_hat_mid = 0.0;
+ if (!evaluate_reduced(n_l_mid, rho_LR_prev_arg, residual_mid,
+                     rho_mid, mu_mid, rho_hat_mid))
+ {
+     return false;
+ }
+
+ n_l_bracketed_out = n_l_mid;
+ rho_LR_bracketed_out = rho_mid;
+ mu_LR_bracketed_out = mu_mid;
+ rho_l_hat_bracketed_out = rho_hat_mid;
+ return true;
+ };
+
+ for (unsigned int iteration = 0; iteration < max_iterations; ++iteration)
+ {
+     double residual_1 = 0.0;
+     double residual_2 = 0.0;
+     double mu_LR_trial = 0.0;
+     double rho_l_hat_trial = 0.0;
+     evaluate(n_l_trial, rho_LR_trial, residual_1, residual_2, mu_LR_trial,
+             rho_l_hat_trial);
+
+     const auto residual_norm = std::abs(residual_1) + std::abs(residual_2);
+     if (residual_norm < newton_tolerance)
+     {
+         n_l_out = n_l_trial;
+         rho_LR_out = rho_LR_trial;
+         mu_LR_out = mu_LR_trial;
+         rho_l_hat_out = rho_l_hat_trial;
+         converged = true;
+         break;
+     }
+ }
+
+ const auto h_n_l = std::max(1e-8 * std::abs(n_l_trial), 1e-10);
+ const auto h_rho = std::max(1e-8 * std::abs(rho_LR_trial), 1e-8);
+
+ double residual_1_n = 0.0;
+ double residual_2_n = 0.0;
+ double residual_1_rho = 0.0;
+ double residual_2_rho = 0.0;
+ double dummy_1 = 0.0;
+ double dummy_2 = 0.0;
+ evaluate(n_l_trial + h_n_l, rho_LR_trial, residual_1_n, residual_2_n,
+         dummy_1, dummy_2);
+ evaluate(n_l_trial, rho_LR_trial + h_rho, residual_1_rho, residual_2_rho,
+         dummy_1, dummy_2);
+
+ const auto j11 = (residual_1_n - residual_1) / h_n_l;
+ const auto j21 = (residual_2_n - residual_2) / h_n_l;
+ const auto j12 = (residual_1_rho - residual_1) / h_rho;
+ const auto j22 = (residual_2_rho - residual_2) / h_rho;
+ const auto determinant = j11 * j22 - j12 * j21;
+ tfel::raise_if(std::abs(determinant) < 1e-24,
+               "DSMMicroMacro bridge microstate Newton system became singular.");

```

```

+ const auto delta_n_l = (-residual_1 * j22 + j12 * residual_2) / determinant;
+ const auto delta_rho = (j21 * residual_1 - j11 * residual_2) / determinant;
+
+ auto damping = 1.0;
+ auto accepted = false;
+ while (damping > 1e-6)
+ {
+     const auto n_l_candidate =
+     std::max(n_l_trial + damping * delta_n_l, minimum_n_l);
+     const auto rho_candidate =
+     std::max(rho_LR_trial + damping * delta_rho,
+             rho_LR_ref + minimum_density_gap);
+
+     double candidate_residual_1 = 0.0;
+     double candidate_residual_2 = 0.0;
+     double dummy_mu = 0.0;
+     double dummy_rho_hat = 0.0;
+     evaluate(n_l_candidate, rho_candidate, candidate_residual_1,
+             candidate_residual_2, dummy_mu, dummy_rho_hat);
+     const auto candidate_norm =
+     std::abs(candidate_residual_1) + std::abs(candidate_residual_2);
+
+     if (candidate_norm < residual_norm)
+     {
+         n_l_trial = n_l_candidate;
+         rho_LR_trial = rho_candidate;
+         accepted = true;
+         break;
+     }
+     damping *= 0.5;
+ }
+
+ if (!accepted)
+ {
+     if (solve_microstate_bracketed(n_l_out, rho_LR_out, mu_LR_out,
+                                   rho_l_hat_out))
+     {
+         converged = true;
+         break;
+     }
+     tfel::raise_if(true,
+                   "DSMMicroMacro bridge microstate Newton line search failed.");
+ }
+
+ if (!converged)
+ {
+     if (solve_microstate_bracketed(n_l_out, rho_LR_out, mu_LR_out,
+                                   rho_l_hat_out))
+     {
+         return;
+     }
+     tfel::raise_if(true,
+                   "DSMMicroMacro bridge microstate Newton solve did not converge.");
+ }
+ };
+
+ double n_l_trial = n_l_prev;
+ double rho_LR_trial = rho_LR_prev;
+ double mu_LR_trial = mu_micro_from_state(n_l_prev, rho_LR_prev);
+ double rho_l_hat_trial =
+     mass_exchange_coefficient *
+     (mu_macro_from_pressure(p, rho_LR_prev) - mu_LR_trial);
+ if (!zero_dt)
+ {
+     solve_microstate(p, delta_epsilon_v, n_l_prev, rho_LR_prev, n_l_trial,
+                     rho_LR_trial, mu_LR_trial, rho_l_hat_trial);
+ }
+
+ const auto phi_m_trial = n_l_trial;
+ const auto phi_M_trial = phi0 - phi_m_trial;
+ const auto delta_epsilon_sw = swelling_slope * (phi_m_trial - phi_m_prev);
+
+ sigma = sigma + lambda * trace(eps_to) * zI + 2.0 * mu * Δeps_to -
+     bulk_modulus * delta_epsilon_sw * zI;
+ S_L = saturation_from_pressure(p);
+
+ n_l = n_l_trial;
+ rho_LR = rho_LR_trial;
+ epsilon_sw = epsilon_sw + delta_epsilon_sw;
+ phi_m = phi_m_trial;
+ phi_M = phi_M_trial;
+ mu_LR_value = mu_LR_trial;
+ rho_l_hat_value = rho_l_hat_trial;
+
+ if (computeTangentOperator_)
+ {
+     ∂/∂σmaepsto = lambda * (zI ⊗ zI) + 2.0 * mu * zI;
+     ∂/∂SLepsto = 0.0 * zI;
+
+     if (zero_dt)
+     {
+         if (p > -pressure_tolerance)
+         {
+             ∂/∂σmapLR = 0.0 * zI;
+             ∂/∂SLp_LR = 0.0;
+         }
+         else
+         {
+             const auto dp_fd = std::max(1e-6, std::abs(p) * 1e-6);
+             const auto dS_dp =
+                 (saturation_from_pressure(p + dp_fd) - saturation_from_pressure(p - dp_fd)) /
+                 (2.0 * dp_fd);
+             ∂/∂σmapLR = 0.0 * zI;
+             ∂/∂SLp_LR = dS_dp;
+         }
+     }
+     else
+     {
+         const auto deps_v_fd = std::max(1e-8, std::abs(delta_epsilon_v) * 1e-6 + 1e-8);
+         double n_l_eps_plus = 0.0;
+         double rho_LR_eps_plus = 0.0;
+         double mu_LR_eps_plus = 0.0;
+         double rho_l_hat_eps_plus = 0.0;
+         double n_l_eps_minus = 0.0;
+         double rho_LR_eps_minus = 0.0;

```

```

+ double mu_LR_eps_minus = 0.0;
+ double rho_L_hat_eps_minus = 0.0;
+
+ solve_microstate(p, delta_epsilon_v + deps_v_fd, n_L_prev, rho_LR_prev,
+   n_L_eps_plus, rho_LR_eps_plus, mu_LR_eps_plus,
+   rho_L_hat_eps_plus);
+ solve_microstate(p, delta_epsilon_v - deps_v_fd, n_L_prev, rho_LR_prev,
+   n_L_eps_minus, rho_LR_eps_minus, mu_LR_eps_minus,
+   rho_L_hat_eps_minus);
+
+ const auto dn_L_dtrace_eps =
+   (n_L_eps_plus - n_L_eps_minus) / (2.0 * deps_v_fd);
+ a/dsigmaeps_to = a/dsigmaeps_to -
+   (bulk_modulus * swelling_slope * dn_L_dtrace_eps) *
+   (zI * zI);
+
+ if (p > -pressure_tolerance)
+ {
+   a/dsigma_LR = 0.0 * zI;
+   a/ds_Lp_LR = 0.0;
+ }
+ else
+ {
+   const auto dp_fd = std::max(1e-6, std::abs(p) * 1e-6);
+   double n_L_plus = 0.0;
+   double rho_LR_plus = 0.0;
+   double mu_LR_plus = 0.0;
+   double rho_L_hat_plus = 0.0;
+   double n_L_minus = 0.0;
+   double rho_LR_minus = 0.0;
+   double mu_LR_minus = 0.0;
+   double rho_L_hat_minus = 0.0;
+
+   solve_microstate(p + dp_fd, delta_epsilon_v, n_L_prev, rho_LR_prev,
+     n_L_plus, rho_LR_plus, mu_LR_plus, rho_L_hat_plus);
+   solve_microstate(p - dp_fd, delta_epsilon_v, n_L_prev, rho_LR_prev,
+     n_L_minus, rho_LR_minus, mu_LR_minus, rho_L_hat_minus);
+
+   const auto dS_dp =
+     (saturation_from_pressure(p + dp_fd) - saturation_from_pressure(p - dp_fd)) /
+     (2.0 * dp_fd);
+   const auto dn_L_dp = (n_L_plus - n_L_minus) / (2.0 * dp_fd);
+
+   a/dsigma_LR = -(bulk_modulus * swelling_slope * dn_L_dp) * zI;
+   a/ds_Lp_LR = dS_dp;
+ }
+ }
+ }
+ }
+
+ @UpdateAuxiliaryStateVariables
+ {
+   constexpr const auto pressure_tolerance = 1e-12;
+   constexpr const auto minimum_n_L = 1e-14;
+   constexpr const auto pi = 3.14159265358979323846;
+
+   const auto n_s = 1.0 - phi0;
+   const auto omega = std::max(n_L * rho_LR / (n_s * rho_SR), minimum_n_L);
+   phi_m = n_L;
+   phi_M = phi0 - n_L;
+   mu_LR_value = (hamaker_constant * rho_LR * rho_LR * rho_LR / (6.0 * pi)) *
+     std::pow(specific_surface / omega, 3.0);
+   const auto rho_macro = std::max(rho_LR, rho_LR_ref + 1e-12);
+   const auto mu_LR = p_LR > -pressure_tolerance ? 0.0 : p_LR / rho_macro;
+   rho_L_hat_value = mass_exchange_coefficient * (mu_LR - mu_LR_value);
+ }
+
+ diff --git a/MaterialLib/SolidModels/MFront/RichardsMechanicsDSM/MicroMacroBridge_MCC.mfront b/MaterialLib/SolidModels/MFront/RichardsMechanicsDSM/MicroMacroBridge_MCC.mfront
+ new file mode 100644
+ index 0000000000..e95ec4a5df
+ --- /dev/null
+ +++ b/MaterialLib/SolidModels/MFront/RichardsMechanicsDSM/MicroMacroBridge_MCC.mfront
+ @@ -0,0 +1,1267 @@
+ @DSL ImplicitGenericBehaviour;
+ @Behaviour RichardsMechanicsDSM/MicroMacroBridge_MCC;
+ @Author OpenGeoSys Community;
+ @Date 27 / 03 / 2026;
+ @Description{
+   Pressure-coupled RichardsMechanics bridge that keeps the verified
+   ModCamClay_semiExpl_constE mechanical and saturation surface, while also
+   carrying the dsm_micromacro microstate variables n_L, rho_LR, epsilon_sw, phi_m,
+   phi_M, mu_LR, rho_L_hat, phi, n_S, n_L, rho_LR, omega_L,
+   delta_epsilon_sw, and sigma_S as persistent dsm_micromacro support state. The
+   returned saturation matches either the verified MCC bridge or, when
+   explicitly enabled, the dsm_micromacro Tuller law. The returned effective stress
+   also includes the dsm_micromacro isotropic swelling correction driven by the
+   updated microstate.
+ }
+
+
+ @Theta 1.0;
+ @Epsilon 1e-14;
+ @MaximumNumberOfIterations 200;
+ @ModellingHypotheses("-.*");
+ @Algorithm NewtonRaphson;
+
+ @Gradient StrainTensor e*;
+ e*.setGlossaryName("Strain");
+
+ @ThermodynamicForce StressTensor sigma;
+ sigma.setGlossaryName("Stress");
+
+
+ @Gradient real "p";
+ "p".setEntryName("LiquidPressure");
+
+
+ @ThermodynamicForce real "S";
+ "S".setEntryName("Saturation");
+
+
+ @StateVariable StrainTensor e*;
+ e*^1.setGlossaryName("ElasticStrain");
+
+
+ @StateVariable real A[];
+ A[]^1.setGlossaryName("EquivalentPlasticStrain");
+
+
+ @IntegrationVariable strain rpc;
+
+
+ @AuxiliaryStateVariable stress pc;
+ pc.setEntryName("PreConsolidationPressure");
+
+

```

```

+@AuxiliaryStateVariable real e"[0]";
+e"[0]".setEntryName("PlasticVolumetricStrain");
+
+@AuxiliaryStateVariable real 'v;
+'v.setEntryName("VolumeRatio");
+
+@AuxiliaryStateVariable real n_L;
+n_L.setEntryName("n_L");
+
+@AuxiliaryStateVariable real rho_LR;
+rho_LR.setEntryName("rho_LR");
+
+@AuxiliaryStateVariable real epsilon_sw;
+epsilon_sw.setEntryName("epsilon_sw");
+
+@AuxiliaryStateVariable real phi_m;
+phi_m.setEntryName("phi_m");
+
+@AuxiliaryStateVariable real phi_M;
+phi_M.setEntryName("phi_M");
+
+@AuxiliaryStateVariable real phi_value;
+phi_value.setEntryName("phi");
+
+@AuxiliaryStateVariable real n_S_value;
+n_S_value.setEntryName("n_S");
+
+@AuxiliaryStateVariable real n_L_value;
+n_L_value.setEntryName("n_L");
+
+@AuxiliaryStateVariable real rho_LR_value;
+rho_LR_value.setEntryName("rho_LR");
+
+@AuxiliaryStateVariable real omega_L_value;
+omega_L_value.setEntryName("omega_L");
+
+@AuxiliaryStateVariable real mu_LR_value;
+mu_LR_value.setEntryName("mu_LR");
+
+@AuxiliaryStateVariable real rho_L_hat_value;
+rho_L_hat_value.setEntryName("rho_L_hat");
+
+@AuxiliaryStateVariable real delta_epsilon_sw_value;
+delta_epsilon_sw_value.setEntryName("delta_epsilon_sw");
+
+@AuxiliaryStateVariable Stensor sigma_S_value;
+sigma_S_value.setEntryName("sigma_S");
+
+@Parameter stress pamb = 1e+3;
+@PhysicalBounds pamb in [0:*=];
+pamb.setEntryName("AmbientPressure");
+
+@MaterialProperty stress young;
+@PhysicalBounds young in [0:*=];
+young.setGlossaryName("YoungModulus");
+
+@MaterialProperty real v;
+@PhysicalBounds v in [-1:0.5];
+v.setGlossaryName("PoissonRatio");
+
+@MaterialProperty real M;
+@PhysicalBounds M in [0:*=];
+M.setEntryName("CriticalStateLineSlope");
+
+@MaterialProperty real ka;
+@PhysicalBounds ka in [0:*=];
+ka.setEntryName("SwellingLineSlope");
+
+@MaterialProperty real la;
+@PhysicalBounds la in [0:*=];
+la.setEntryName("VirginConsolidationLineSlope");
+
+@MaterialProperty stress pc_char;
+@PhysicalBounds pc_char in [0:*=];
+pc_char.setEntryName("CharacteristicPreConsolidationPressure");
+
+@MaterialProperty real 'S;
+'S.setEntryName("ResidualLiquidSaturation");
+
+@MaterialProperty real 'S;
+'S.setEntryName("ResidualGasSaturation");
+
+@MaterialProperty real *p;
+*p.setEntryName("BubblePressure");
+
+@MaterialProperty real *m;
+*m.setEntryName("VanGenuchtenExponent_m");
+
+@MaterialProperty real macro_saturation_law_mode;
+@PhysicalBounds macro_saturation_law_mode in [0:*=];
+macro_saturation_law_mode.setEntryName("MacroSaturationLawMode");
+
+@MaterialProperty real microstate_local_solve_mode;
+@PhysicalBounds microstate_local_solve_mode in [0:*=];
+microstate_local_solve_mode.setEntryName("MicroStateLocalSolveMode");
+
+@MaterialProperty real micro_potential_convention;
+@PhysicalBounds micro_potential_convention in [0:*=];
+micro_potential_convention.setEntryName("MicroPotentialConvention");
+
+@MaterialProperty real swelling_slope;
+swelling_slope.setEntryName("SwellingSlope");
+
+@MaterialProperty real mass_exchange_coefficient;
+mass_exchange_coefficient.setEntryName("MassExchangeCoefficient");
+
+@MaterialProperty real macro_viscosity;
+macro_viscosity.setEntryName("MacroViscosity");
+
+@MaterialProperty real rho_LR_ref;
+rho_LR_ref.setEntryName("ReferenceLiquidDensityMacro");
+
+@MaterialProperty real rho_L0;
+rho_L0.setEntryName("ReferenceLiquidDensityMicro");
+
+@MaterialProperty real rho_SR;
+rho_SR.setEntryName("ReferenceDensitySoLid");

```

```

+
+@MaterialProperty real density_a;
+density_a.setEntryName("MicroLiquidDensityA");
+
+@MaterialProperty real density_b;
+density_b.setEntryName("MicroLiquidDensityB");
+
+@MaterialProperty real hamaker_constant;
+hamaker_constant.setEntryName("HamakerConstant");
+
+@MaterialProperty real specific_surface;
+specific_surface.setEntryName("SpecificSurface");
+
+@MaterialProperty real area_factor_tuller;
+area_factor_tuller.setEntryName("AreaFactorTuller");
+
+@MaterialProperty real pore_area_shape_factor_tuller;
+pore_area_shape_factor_tuller.setEntryName("PoreAreaShapeFactorTuller");
+
+@MaterialProperty real characteristic_pore_size;
+characteristic_pore_size.setEntryName("CharacteristicPoreSize");
+
+@MaterialProperty real surface_tension;
+surface_tension.setEntryName("SurfaceTension");
+
+@MaterialProperty real phi0;
+phi0.setEntryName("InitialPorosity");
+
+@LocalVariable bool withinElasticRange;
+@LocalVariable real M';
+@LocalVariable real pc_min;
+@LocalVariable real rpc_min;
+@LocalVariable real K;
+@LocalVariable StressTensor sigma;
+@LocalVariable StiffnessTensor d0/depsilon;
+@LocalVariable real n_l_trial_value;
+@LocalVariable real rho_LR_trial_value;
+@LocalVariable real epsilon_sw_increment_trial;
+@LocalVariable real phi_m_trial_value;
+@LocalVariable real phi_M_trial_value;
+@LocalVariable real mu_LR_trial_value;
+@LocalVariable real rho_hat_trial_value;
+@LocalVariable real dn_L_dtrace_eps_trial;
+@LocalVariable real dn_L_dp_trial;
+@LocalVariable bool use_macro_saturation_law;
+@LocalVariable bool use_scalar_microstate_storage;
+@LocalVariable real micro_potential_sign;
+@LocalVariable real tuller_capillary_prefactor;
+@LocalVariable real macro_support_saturation_trial;
+
+@TangentOperatorBlocks d0/depsilon({, d0/delta^*p, a^-/dDelta^*S, a^-/dDelta^*Sp};
+
+@InitializeLocalVariables
+{
+  const auto *S = [&](const auto **p) {
+    return std::pow(
+      std::pow(std::max(**p, 0.0) / *p, 1.0 / (1.0 - v^cm)) + 1.0, -v^cm);
+  };
+
+  constexpr const auto pressure_tolerance = 1e-12;
+
+  use_macro_saturation_law = macro_saturation_law_mode > 0.5;
+  use_scalar_microstate_storage = microstate_local_solve_mode > 0.5;
+  tfel::raise_if(area_factor_tuller <= 0.0,
+    "Invalid parameter: AreaFactorTuller must be positive.");
+  tfel::raise_if(pore_area_shape_factor_tuller <= 0.0,
+    "Invalid parameter: PoreAreaShapeFactorTuller must be positive.");
+  tfel::raise_if(characteristic_pore_size <= 0.0,
+    "Invalid parameter: CharacteristicPoreSize must be positive.");
+  tfel::raise_if(surface_tension <= 0.0,
+    "Invalid parameter: SurfaceTension must be positive.");
+  tuller_capillary_prefactor =
+    4.0 * pore_area_shape_factor_tuller * surface_tension * surface_tension /
+    (area_factor_tuller * characteristic_pore_size * characteristic_pore_size);
+
+  const auto macro_saturation_law = [&](const auto p_arg) {
+    if (p_arg < -pressure_tolerance)
+    {
+      const auto capillary_term = tuller_capillary_prefactor / (p_arg * p_arg);
+      return 1.0 - std::exp(-capillary_term);
+    }
+    return 1.0;
+  };
+
+  const auto S = use_macro_saturation_law
+    ? macro_saturation_law(*p + Delta^*p)
+    : *S - (*p + Delta^*p) * (1.0 - v^S - v^S) + v^S;
+  v^S = use_macro_saturation_law ? S : std::clamp(S, v^S, 1.0 - v^S);
+  macro_support_saturation_trial = macro_saturation_law(*p + Delta^*p);
+
+  tfel::raise_if((la < ka, "Invalid parameters: la<ka");
+  M' = M * M;
+
+  epsilon^1 = (1.0 + v) / young * sigma - v / young * traced() * zI;
+  sigma = sigma;
+
+  rpc = pc / young;
+  pc_min = 1e-15 * pc_char;
+  rpc_min = pc_min / young;
+
+  K = young / (3.0 * (1.0 - 2.0 * v));
+  d0/depsilon^1 = young / (1.0 + v) * Stensor4::K() + K * Stensor4::IXI();
+
+  const auto sigma^1 = sigma + d0/depsilon^1 * Delta^*epsilon;
+  const auto *is = deviatoro^1();
+  const auto *iq = sqrt(1.5 * *is | *is);
+  const auto *ip = -traced^1() / 3.0 * pamb;
+  const auto *if = *iq * *iq + M' * *ip * *ip * (*ip - pc);
+  withinElasticRange = *if < 0;
+
+  constexpr const auto pi = 3.14159265358979323846;
+  constexpr const auto minimum_n_l = 1e-14;
+  constexpr const auto minimum_density_gap = 1e-12;
+  constexpr const auto newton_tolerance = 1e-11;
+  constexpr const auto max_iterations = 64;
+  const auto hamaker_magnitude = std::abs(hamaker_constant);
+  micro_potential_sign =

```



```

+ (micro_potential_convention > 0.5 ? -1.0 : 1.0) *
+ (hamaker_constant >= 0.0 ? 1.0 : -1.0);
+
+ tfel::raise_if(phi0 <= 0.0 || phi0 >= 1.0,
+ "Invalid parameter: InitialPorosity must lie in (0,1).");
+ tfel::raise_if(rho_LR_ref <= 0.0 || rho_L0 <= 0.0 || rho_SR <= 0.0,
+ "Invalid density parameter: all reference densities must be positive.");
+ tfel::raise_if(macro_viscosity <= 0.0,
+ "Invalid parameter: MacroViscosity must be positive.");
+ tfel::raise_if(hamaker_magnitude <= 0.0,
+ "Invalid parameter: HamakerConstant must be non-zero.");
+ tfel::raise_if(specific_surface <= 0.0,
+ "Invalid parameter: SpecificSurface must be positive.");
+
+ const auto p = "p + Δ"p;
+ const auto delta_epsilon_v = traceΔε**();
+ const auto zero_dt = std::abs(dt) < 1e-15;
+ constexpr const auto porosity_upper = 1.0 - 1e-12;
+ const auto phi_total_from_state =
+ std::max(phi_m, 0.0) + std::max(phi_M, 0.0);
+ const auto phi_total_prev =
+ (std::isfinite(phi_total_from_state) &&
+ phi_total_from_state > minimum_n_l)
+ ? std::clamp(phi_total_from_state, minimum_n_l, porosity_upper)
+ : std::clamp(phi0, minimum_n_l, porosity_upper);
+ const auto porosity_denominator = 1.0 + delta_epsilon_v;
+ const auto phi_total_trial =
+ (std::isfinite(porosity_denominator) &&
+ std::abs(porosity_denominator) > 1e-12)
+ ? std::clamp(phi_total_prev + delta_epsilon_v /
+ porosity_denominator,
+ minimum_n_l, porosity_upper)
+ : phi_total_prev;
+ const auto n_s = std::max(minimum_n_l, 1.0 - phi_total_trial);
+ const auto n_l_upper_bound = std::max(phi_total_trial, minimum_n_l);
+ const auto n_l_prev = std::clamp(n_l, minimum_n_l, n_l_upper_bound);
+ const auto rho_LR_prev =
+ std::max(rho_LR, rho_LR_ref + minimum_density_gap);
+ const auto phi_m_prev = n_l_prev;
+ const auto previous_mass = n_l_prev * rho_LR_prev;
+
+ const auto macro_density_from_state = [&](const auto rho_macro_arg) {
+ return std::max(rho_macro_arg, rho_LR_ref + minimum_density_gap);
+ };
+
+ const auto alpha_M_effective_from_density = [&](const auto rho_macro_arg) {
+ const auto rho_macro_safe = macro_density_from_state(rho_macro_arg);
+ return mass_exchange_coefficient * rho_macro_safe / macro_viscosity;
+ };
+
+ const auto mu_macro_from_pressure = [&](const auto p_arg,
+ const auto rho_macro_arg) {
+ const auto rho_macro_safe = macro_density_from_state(rho_macro_arg);
+ return p_arg > -pressure_tolerance ? 0.0 : p_arg / rho_macro_safe;
+ };
+
+ const auto omega_from_state = [&](const auto n_l_arg,
+ const auto rho_LR_arg) {
+ return n_l_arg * rho_LR_arg / (n_s * rho_SR);
+ };
+
+ const auto rho_micro_eos = [&](const auto n_l_arg,
+ const auto rho_LR_arg) {
+ const auto omega =
+ std::max(omega_from_state(n_l_arg, rho_LR_arg), minimum_n_l);
+ return rho_L0 * std::exp(-density_a * std::pow(omega, density_b)) +
+ rho_LR_ref;
+ };
+
+ const auto mu_micro_from_state = [&](const auto n_l_arg,
+ const auto rho_LR_arg) {
+ const auto omega =
+ std::max(omega_from_state(n_l_arg, rho_LR_arg), minimum_n_l);
+ return (micro_potential_sign * hamaker_magnitude * rho_LR_arg *
+ rho_LR_arg * rho_LR_arg /
+ (6.0 * pi)) *
+ std::pow(specific_surface / omega, 3.0);
+ };
+
+ const auto solve_microstate = [&](const auto p_arg,
+ const auto delta_epsilon_v_arg,
+ const auto n_l_prev_arg,
+ const auto rho_LR_prev_arg,
+ double& n_l_out,
+ double& rho_LR_out,
+ double& mu_LR_out,
+ double& rho_l_hat_out) {
+ const auto rho_macro_step =
+ macro_density_from_state(rho_LR_prev_arg);
+
+ if (use_scalar_microstate_storage)
+ {
+ auto n_l_trial =
+ std::clamp(n_l_prev_arg, minimum_n_l, n_l_upper_bound);
+ const auto rho_scalar = rho_macro_step;
+
+ const auto evaluate_scalar = [&](const auto n_l_arg,
+ double& residual_arg,
+ double& mu_LR_arg,
+ double& rho_l_hat_arg) {
+ const auto mu_LR_arg =
+ mu_macro_from_pressure(p_arg, rho_scalar);
+ const auto alpha_M_effective =
+ alpha_M_effective_from_density(rho_scalar);
+ mu_LR_arg = mu_micro_from_state(n_l_arg, rho_scalar);
+ rho_l_hat_arg = alpha_M_effective * (mu_LR_arg - mu_LR_arg);
+ residual_arg = n_l_arg - n_l_prev_arg -
+ dt * rho_l_hat_arg / rho_scalar -
+ n_l_arg * delta_epsilon_v_arg;
+ };
+
+ const auto assign_scalar_state = [&](const auto n_l_candidate_arg) {
+ n_l_out = std::clamp(n_l_candidate_arg, minimum_n_l,
+ n_l_upper_bound);
+ double residual_dummy = 0.0;
+ evaluate_scalar(n_l_out, residual_dummy, mu_LR_out,
+ rho_l_hat_out);
+ }
+ }

```

```

+         rho_LR_out = rho_scalar;
+     });
+
+     const auto solve_scalar_bracketed =
+     [&](double& n_l_bracketed_out,
+        double& mu_LR_bracketed_out,
+        double& rho_l_hat_bracketed_out) {
+         constexpr const auto bracket_scan_points = 256u;
+         auto n_l_lo = minimum_n_l;
+         auto n_l_hi = n_l_upper_bound;
+
+         double residual_lo = 0.0;
+         double residual_hi = 0.0;
+         double mu_lo = 0.0;
+         double mu_hi = 0.0;
+         double rho_hat_lo = 0.0;
+         double rho_hat_hi = 0.0;
+
+         evaluate_scalar(n_l_lo, residual_lo, mu_lo, rho_hat_lo);
+         evaluate_scalar(n_l_hi, residual_hi, mu_hi, rho_hat_hi);
+
+         if (residual_lo * residual_hi > 0.0)
+         {
+             const auto log_n_l_min = std::log10(minimum_n_l);
+             const auto log_n_l_max = std::log10(n_l_hi);
+             auto previous_n_l = n_l_lo;
+             auto previous_residual = residual_lo;
+             auto previous_mu = mu_lo;
+             auto previous_rho_hat = rho_hat_lo;
+             auto found_bracket = false;
+
+             for (unsigned int i = 1; i <= bracket_scan_points; ++i)
+             {
+                 const auto log_n_l =
+                     log_n_l_min +
+                     (log_n_l_max - log_n_l_min) *
+                     (static_cast<double>(i) /
+                      bracket_scan_points);
+                 const auto n_l_candidate = std::clamp(
+                     std::pow(10.0, log_n_l), minimum_n_l,
+                     n_l_upper_bound);
+
+                 double residual_candidate = 0.0;
+                 double mu_candidate = 0.0;
+                 double rho_hat_candidate = 0.0;
+                 evaluate_scalar(n_l_candidate, residual_candidate,
+                     mu_candidate,
+                     rho_hat_candidate);
+
+                 if (previous_residual * residual_candidate <= 0.0)
+                 {
+                     n_l_lo = previous_n_l;
+                     residual_lo = previous_residual;
+                     mu_lo = previous_mu;
+                     rho_hat_lo = previous_rho_hat;
+
+                     n_l_hi = n_l_candidate;
+                     residual_hi = residual_candidate;
+                     mu_hi = mu_candidate;
+                     rho_hat_hi = rho_hat_candidate;
+                     found_bracket = true;
+                     break;
+                 }
+
+                 previous_n_l = n_l_candidate;
+                 previous_residual = residual_candidate;
+                 previous_mu = mu_candidate;
+                 previous_rho_hat = rho_hat_candidate;
+             }
+
+             if (!found_bracket)
+             {
+                 return false;
+             }
+         }
+
+         for (unsigned int iteration = 0;
+              iteration < 4 * max_iterations; ++iteration)
+         {
+             const auto n_l_mid = 0.5 * (n_l_lo + n_l_hi);
+             double residual_mid = 0.0;
+             double mu_mid = 0.0;
+             double rho_hat_mid = 0.0;
+             evaluate_scalar(n_l_mid, residual_mid, mu_mid,
+                             rho_hat_mid);
+
+             if (std::abs(residual_mid) < newton_tolerance ||
+                 std::abs(n_l_hi - n_l_lo) <
+                 newton_tolerance *
+                 std::max(1.0, std::abs(n_l_mid)))
+             {
+                 n_l_bracketed_out = n_l_mid;
+                 mu_LR_bracketed_out = mu_mid;
+                 rho_l_hat_bracketed_out = rho_hat_mid;
+                 return true;
+             }
+
+             if (residual_lo * residual_mid <= 0.0)
+             {
+                 n_l_hi = n_l_mid;
+                 residual_hi = residual_mid;
+             }
+             else
+             {
+                 n_l_lo = n_l_mid;
+                 residual_lo = residual_mid;
+             }
+         }
+
+         const auto n_l_mid = 0.5 * (n_l_lo + n_l_hi);
+         double residual_mid = 0.0;
+         double mu_mid = 0.0;
+         double rho_hat_mid = 0.0;
+         evaluate_scalar(n_l_mid, residual_mid, mu_mid,
+                         rho_hat_mid);
+         n_l_bracketed_out = n_l_mid;
+         mu_LR_bracketed_out = mu_mid;

```

```

+         rho_l_hat_bracketed_out = rho_hat_mid;
+         return true;
+     };
+
+     for (unsigned int iteration = 0; iteration < max_iterations;
+          ++iteration)
+     {
+         double residual = 0.0;
+         double mu_trial = 0.0;
+         double rho_hat_trial = 0.0;
+         evaluate_scalar(n_l_trial, residual, mu_trial, rho_hat_trial);
+
+         if (std::abs(residual) < newton_tolerance)
+         {
+             assign_scalar_state(n_l_trial);
+             return;
+         }
+
+         const auto dn_scale =
+             std::max(1e-8, std::abs(n_l_trial) * 1e-6);
+         double residual_n = 0.0;
+         double mu_n = 0.0;
+         double rho_hat_n = 0.0;
+         evaluate_scalar(
+             std::clamp(n_l_trial + dn_scale, minimum_n_l,
+                       n_l_upper_bound),
+             residual_n, mu_n, rho_hat_n);
+
+         const auto scalar_jacobian =
+             (residual_n - residual) / dn_scale;
+         if (!std::isfinite(scalar_jacobian) &&
+             std::abs(scalar_jacobian) > newton_tolerance)
+         {
+             break;
+         }
+
+         const auto delta_n = -residual / scalar_jacobian;
+         auto step_scale = 1.0;
+         auto accepted = false;
+
+         for (unsigned int line_search = 0; line_search < 8;
+              ++line_search)
+         {
+             const auto n_candidate =
+                 std::clamp(n_l_trial + step_scale * delta_n,
+                           minimum_n_l, n_l_upper_bound);
+             double residual_candidate = 0.0;
+             double mu_candidate = 0.0;
+             double rho_hat_candidate = 0.0;
+             evaluate_scalar(n_candidate, residual_candidate,
+                           mu_candidate, rho_hat_candidate);
+
+             if (std::abs(residual_candidate) <
+                 std::abs(residual) * (1.0 - 1e-4 * step_scale))
+             {
+                 n_l_trial = n_candidate;
+                 accepted = true;
+                 break;
+             }
+
+             step_scale *= 0.5;
+         }
+
+         if (!accepted)
+         {
+             if (solve_scalar_bracketed(n_l_out, mu_LR_out,
+                                       rho_l_hat_out))
+             {
+                 assign_scalar_state(n_l_out);
+                 return;
+             }
+
+             // Native-aligned robustness fallback: explicit one-step
+             // update when local Newton/bracketing does not converge.
+             double residual_prev = 0.0;
+             double mu_prev = 0.0;
+             double rho_hat_prev = 0.0;
+             auto const n_l_prev_safe = std::clamp(
+                 n_l_prev_arg, minimum_n_l, n_l_upper_bound);
+             evaluate_scalar(n_l_prev_safe, residual_prev, mu_prev,
+                           rho_hat_prev);
+             const auto n_l_explicit =
+                 n_l_prev_safe + dt * rho_hat_prev / rho_scalar +
+                 n_l_prev_safe * delta_epsilon_v_arg;
+             assign_scalar_state(n_l_explicit);
+             return;
+         }
+     }
+
+     if (solve_scalar_bracketed(n_l_out, mu_LR_out, rho_l_hat_out))
+     {
+         assign_scalar_state(n_l_out);
+         return;
+     }
+
+     // Native-aligned robustness fallback after scalar Newton loop.
+     double residual_prev = 0.0;
+     double mu_prev = 0.0;
+     double rho_hat_prev = 0.0;
+     auto const n_l_prev_safe =
+         std::clamp(n_l_prev_arg, minimum_n_l, n_l_upper_bound);
+     evaluate_scalar(n_l_prev_safe, residual_prev, mu_prev,
+                   rho_hat_prev);
+     const auto n_l_explicit =
+         n_l_prev_safe + dt * rho_hat_prev / rho_scalar +
+         n_l_prev_safe * delta_epsilon_v_arg;
+     assign_scalar_state(n_l_explicit);
+     return;
+ }
+
+ auto n_l_trial =
+     std::clamp(n_l_prev_arg, minimum_n_l, n_l_upper_bound);
+ auto rho_LR_trial =
+     std::max(rho_LR_prev_arg, rho_LR_ref + minimum_density_gap);
+ auto converged = false;
+
+ const auto evaluate = [&](const auto n_l_arg,

```

```

+         const auto rho_LR_arg,
+         double& residual_1,
+         double& residual_2,
+         double& mu_LR_arg,
+         double& rho_l_hat_arg) {
+
+     const auto mu_LR_arg =
+         mu_macro_from_pressure(p_arg, rho_macro_step);
+     const auto alpha_M_effective =
+         alpha_M_effective_from_density(rho_macro_step);
+     mu_LR_arg = mu_micro_from_state(n_l_arg, rho_LR_arg);
+     rho_l_hat_arg =
+         alpha_M_effective * (mu_LR_arg - mu_LR_arg);
+     residual_1 =
+         n_l_arg * rho_LR_arg - previous_mass - dt * rho_l_hat_arg -
+         n_l_arg * rho_LR_arg * delta_epsilon_v_arg;
+     residual_2 = rho_LR_arg - rho_micro_eos(n_l_arg, rho_LR_arg);
+
+ };
+
+ const auto solve_rho_for_n_l = [&](const auto n_l_arg,
+                                   const auto rho_initial_guess,
+                                   double& rho_LR_out_arg) {
+
+     auto rho_LR_trial_local =
+         std::max(rho_initial_guess, rho_LR_ref + minimum_density_gap);
+
+     for (unsigned int iteration = 0; iteration < 4 * max_iterations;
+          ++iteration)
+     {
+         const auto rho_candidate =
+             rho_micro_eos(n_l_arg, rho_LR_trial_local);
+         const auto rho_change =
+             std::abs(rho_candidate - rho_LR_trial_local);
+         rho_LR_trial_local =
+             std::max(rho_candidate, rho_LR_ref + minimum_density_gap);
+
+         if (rho_change < newton_tolerance *
+             std::max(1.0, std::abs(rho_LR_trial_local)))
+         {
+             rho_LR_out_arg = rho_LR_trial_local;
+             return true;
+         }
+     }
+
+     rho_LR_out_arg = rho_LR_trial_local;
+     return false;
+ };
+
+ const auto evaluate_reduced = [&](const auto n_l_arg,
+                                   const auto rho_initial_guess,
+                                   double& residual_1,
+                                   double& rho_LR_arg,
+                                   double& mu_LR_arg,
+                                   double& rho_l_hat_arg) {
+
+     const auto rho_converged = solve_rho_for_n_l(
+         n_l_arg, rho_initial_guess, rho_LR_arg);
+     const auto mu_LR_arg =
+         mu_macro_from_pressure(p_arg, rho_macro_step);
+     const auto alpha_M_effective =
+         alpha_M_effective_from_density(rho_macro_step);
+     mu_LR_arg = mu_micro_from_state(n_l_arg, rho_LR_arg);
+     rho_l_hat_arg =
+         alpha_M_effective * (mu_LR_arg - mu_LR_arg);
+     residual_1 =
+         n_l_arg * rho_LR_arg - previous_mass - dt * rho_l_hat_arg -
+         n_l_arg * rho_LR_arg * delta_epsilon_v_arg;
+     return rho_converged;
+ };
+
+ const auto assign_coupled_state = [&](const auto n_l_candidate_arg,
+                                       const auto rho_guess_arg) {
+
+     n_l_out = std::clamp(n_l_candidate_arg, minimum_n_l, n_l_upper_bound);
+     auto rho_resolved = std::max(rho_guess_arg, rho_LR_ref + minimum_density_gap);
+     if (!solve_rho_for_n_l(n_l_out, rho_resolved, rho_resolved))
+     {
+         rho_resolved = std::max(rho_micro_eos(n_l_out, rho_resolved),
+                                 rho_LR_ref + minimum_density_gap);
+     }
+     rho_LR_out = rho_resolved;
+     const auto mu_LR_arg = mu_macro_from_pressure(p_arg, rho_macro_step);
+     const auto alpha_M_effective =
+         alpha_M_effective_from_density(rho_macro_step);
+     mu_LR_out = mu_micro_from_state(n_l_out, rho_LR_out);
+     rho_l_hat_out = alpha_M_effective * (mu_LR_arg - mu_LR_out);
+
+ };
+
+ const auto fallback_coupled_state = [&]() {
+
+     const auto n_l_prev_safe =
+         std::clamp(n_l_prev_arg, minimum_n_l, n_l_upper_bound);
+     double residual_prev = 0.0;
+     double rho_prev =
+         std::max(rho_LR_prev_arg, rho_LR_ref + minimum_density_gap);
+     double mu_prev = 0.0;
+     double rho_hat_prev = 0.0;
+
+     const auto reduced_ok =
+         evaluate_reduced(n_l_prev_safe, rho_prev, residual_prev, rho_prev,
+                         mu_prev, rho_hat_prev);
+
+     (void)residual_prev;
+     if (!reduced_ok)
+     {
+         const auto mu_LR_arg =
+             mu_macro_from_pressure(p_arg, rho_macro_step);
+         const auto alpha_M_effective =
+             alpha_M_effective_from_density(rho_macro_step);
+         mu_prev = mu_micro_from_state(n_l_prev_safe, rho_prev);
+         rho_hat_prev = alpha_M_effective * (mu_LR_arg - mu_prev);
+     }
+
+     const auto n_l_explicit =
+         n_l_prev_safe +
+         dt * rho_hat_prev /
+         std::max(rho_prev, rho_LR_ref + minimum_density_gap) +
+         n_l_prev_safe * delta_epsilon_v_arg;
+     assign_coupled_state(n_l_explicit, rho_prev);
+
+ };
+
+ const auto solve_microstate_bracketed =
+     [&](double& n_l_bracketed_out,

```

```

+ double& rho_LR_bracketed_out,
+ double& mu_LR_bracketed_out,
+ double& rho_L_hat_bracketed_out) {
+ constexpr const auto bracket_scan_points = 256u;
+ auto n_l_lo = minimum_n_l;
+ auto n_l_hi = n_l_upper_bound;
+
+ double residual_lo = 0.0;
+ double residual_hi = 0.0;
+ double rho_lo = 0.0;
+ double rho_hi = 0.0;
+ double mu_lo = 0.0;
+ double mu_hi = 0.0;
+ double rho_hat_lo = 0.0;
+ double rho_hat_hi = 0.0;
+
+ auto bracket_ok =
+     evaluate_reduced(n_l_lo, rho_LR_prev_arg, residual_lo,
+                     rho_lo, mu_lo, rho_hat_lo) &&
+     evaluate_reduced(n_l_hi, rho_LR_prev_arg, residual_hi,
+                     rho_hi, mu_hi, rho_hat_hi);
+
+ if (!bracket_ok)
+ {
+     return false;
+ }
+
+ if (residual_lo * residual_hi > 0.0)
+ {
+     const auto log_n_l_min = std::log10(minimum_n_l);
+     const auto log_n_l_max = std::log10(n_l_hi);
+     auto previous_n_l = n_l_lo;
+     auto previous_residual = residual_lo;
+     auto previous_rho = rho_lo;
+     auto previous_mu = mu_lo;
+     auto previous_rho_hat = rho_hat_lo;
+     auto found_bracket = false;
+
+     for (unsigned int i = 1; i <= bracket_scan_points; ++i)
+     {
+         const auto log_n_l =
+             log_n_l_min +
+             (log_n_l_max - log_n_l_min) *
+             (static_cast<double>(i) / bracket_scan_points);
+         const auto n_l_candidate = std::clamp(
+             std::pow(10.0, log_n_l), minimum_n_l,
+             n_l_upper_bound);
+
+         double residual_candidate = 0.0;
+         double rho_candidate = 0.0;
+         double mu_candidate = 0.0;
+         double rho_hat_candidate = 0.0;
+         if (!evaluate_reduced(n_l_candidate, rho_LR_prev_arg,
+                             residual_candidate, rho_candidate,
+                             mu_candidate,
+                             rho_hat_candidate))
+         {
+             return false;
+         }
+
+         if (previous_residual * residual_candidate <= 0.0)
+         {
+             n_l_lo = previous_n_l;
+             residual_lo = previous_residual;
+             rho_lo = previous_rho;
+             mu_lo = previous_mu;
+             rho_hat_lo = previous_rho_hat;
+
+             n_l_hi = n_l_candidate;
+             residual_hi = residual_candidate;
+             rho_hi = rho_candidate;
+             mu_hi = mu_candidate;
+             rho_hat_hi = rho_hat_candidate;
+             found_bracket = true;
+             break;
+         }
+
+         previous_n_l = n_l_candidate;
+         previous_residual = residual_candidate;
+         previous_rho = rho_candidate;
+         previous_mu = mu_candidate;
+         previous_rho_hat = rho_hat_candidate;
+     }
+
+     if (!found_bracket)
+     {
+         return false;
+     }
+ }
+
+ for (unsigned int iteration = 0; iteration < 4 * max_iterations;
+     ++iteration)
+ {
+     const auto n_l_mid = 0.5 * (n_l_lo + n_l_hi);
+     double residual_mid = 0.0;
+     double rho_mid = 0.0;
+     double mu_mid = 0.0;
+     double rho_hat_mid = 0.0;
+
+     if (!evaluate_reduced(n_l_mid, rho_LR_prev_arg,
+                         residual_mid, rho_mid, mu_mid,
+                         rho_hat_mid))
+     {
+         return false;
+     }
+
+     if (std::abs(residual_mid) < newton_tolerance ||
+         std::abs(n_l_hi - n_l_lo) <
+         newton_tolerance *
+         std::max(1.0, std::abs(n_l_mid)))
+     {
+         n_l_bracketed_out = n_l_mid;
+         rho_LR_bracketed_out = rho_mid;
+         mu_LR_bracketed_out = mu_mid;
+         rho_L_hat_bracketed_out = rho_hat_mid;
+         return true;
+     }
+ }

```

```

+         if (residual_lo * residual_mid <= 0.0)
+         {
+             n_l_hi = n_l_mid;
+             residual_hi = residual_mid;
+         }
+         else
+         {
+             n_l_lo = n_l_mid;
+             residual_lo = residual_mid;
+         }
+     }
+
+     const auto n_l_mid = 0.5 * (n_l_lo + n_l_hi);
+     double residual_mid = 0.0;
+     double rho_mid = 0.0;
+     double mu_mid = 0.0;
+     double rho_hat_mid = 0.0;
+     if (levalute_reduced(n_l_mid, rho_LR_prev_arg, residual_mid,
+         rho_mid, mu_mid, rho_hat_mid))
+     {
+         return false;
+     }
+
+     n_l_bracketed_out = n_l_mid;
+     rho_LR_bracketed_out = rho_mid;
+     mu_LR_bracketed_out = mu_mid;
+     rho_l_hat_bracketed_out = rho_hat_mid;
+     return true;
+ };
+
+ for (unsigned int iteration = 0; iteration < max_iterations;
+     ++iteration)
+ {
+     double residual_1 = 0.0;
+     double residual_2 = 0.0;
+     double mu_LR_trial = 0.0;
+     double rho_l_hat_trial = 0.0;
+     evaluate(n_l_trial, rho_LR_trial, residual_1, residual_2,
+         mu_LR_trial, rho_l_hat_trial);
+
+     const auto residual_norm =
+         std::max(std::abs(residual_1), std::abs(residual_2));
+     if (residual_norm < newton_tolerance)
+     {
+         n_l_out = n_l_trial;
+         rho_LR_out = rho_LR_trial;
+         mu_LR_out = mu_LR_trial;
+         rho_l_hat_out = rho_l_hat_trial;
+         return;
+     }
+
+     const auto dn_scale =
+         std::max(1e-8, std::abs(n_l_trial) * 1e-6);
+     const auto drho_scale =
+         std::max(1e-6, std::abs(rho_LR_trial) * 1e-6);
+
+     double residual_1_n = 0.0;
+     double residual_2_n = 0.0;
+     double mu_dummy = 0.0;
+     double rho_hat_dummy = 0.0;
+     evaluate(std::clamp(n_l_trial + dn_scale, minimum_n_l,
+         n_l_upper_bound),
+         rho_LR_trial, residual_1_n, residual_2_n, mu_dummy,
+         rho_hat_dummy);
+
+     double residual_1_rho = 0.0;
+     double residual_2_rho = 0.0;
+     evaluate(n_l_trial, std::max(rho_LR_trial + drho_scale,
+         rho_LR_ref + minimum_density_gap),
+         residual_1_rho, residual_2_rho, mu_dummy, rho_hat_dummy);
+
+     const auto j11 = (residual_1_n - residual_1) / dn_scale;
+     const auto j21 = (residual_2_n - residual_2) / dn_scale;
+     const auto j12 = (residual_1_rho - residual_1) / drho_scale;
+     const auto j22 = (residual_2_rho - residual_2) / drho_scale;
+
+     const auto det = j11 * j22 - j12 * j21;
+     if (std::abs(det) < newton_tolerance)
+     {
+         break;
+     }
+
+     const auto delta_n = (-residual_1 * j22 + residual_2 * j12) / det;
+     const auto delta_rho =
+         (-j11 * residual_2 + j21 * residual_1) / det;
+
+     auto step_scale = 1.0;
+     auto accepted = false;
+
+     for (unsigned int line_search = 0; line_search < 8; ++line_search)
+     {
+         const auto n_candidate =
+             std::clamp(n_l_trial + step_scale * delta_n, minimum_n_l,
+                 n_l_upper_bound);
+         const auto rho_candidate = std::max(
+             rho_LR_trial + step_scale * delta_rho,
+             rho_LR_ref + minimum_density_gap);
+
+         double residual_1_candidate = 0.0;
+         double residual_2_candidate = 0.0;
+         double mu_candidate = 0.0;
+         double rho_hat_candidate = 0.0;
+         evaluate(n_candidate, rho_candidate, residual_1_candidate,
+             residual_2_candidate, mu_candidate, rho_hat_candidate);
+
+         const auto candidate_norm = std::max(
+             std::abs(residual_1_candidate),
+             std::abs(residual_2_candidate));
+         if (candidate_norm <
+             residual_norm * (1.0 - 1e-4 * step_scale))
+         {
+             n_l_trial = n_candidate;
+             rho_LR_trial = rho_candidate;
+             accepted = true;
+             break;
+         }
+     }
+ }

```

```

+         step_scale *= 0.5;
+     }
+
+     if (!accepted)
+     {
+         if (solve_microstate_bracketed(n_l_out, rho_LR_out, mu_LR_out,
+                                     rho_l_hat_out))
+         {
+             return;
+         }
+         fallback_coupled_state();
+         return;
+     }
+
+     if (std::abs(step_scale * delta_n) <
+         newton_tolerance * std::max(1.0, std::abs(n_l_trial)) &&
+         std::abs(step_scale * delta_rho) <
+         newton_tolerance * std::max(1.0, std::abs(rho_LR_trial)))
+     {
+         double residual_1 = 0.0;
+         double residual_2 = 0.0;
+         double mu_LR_trial = 0.0;
+         double rho_l_hat_trial = 0.0;
+         evaluate(n_l_trial, rho_LR_trial, residual_1, residual_2,
+                 mu_LR_trial, rho_l_hat_trial);
+
+         if (std::max(std::abs(residual_1), std::abs(residual_2)) <
+             newton_tolerance)
+         {
+             n_l_out = n_l_trial;
+             rho_LR_out = rho_LR_trial;
+             mu_LR_out = mu_LR_trial;
+             rho_l_hat_out = rho_l_hat_trial;
+             return;
+         }
+     }
+
+     if (solve_microstate_bracketed(n_l_out, rho_LR_out, mu_LR_out,
+                                 rho_l_hat_out))
+     {
+         return;
+     }
+     fallback_coupled_state();
+     return;
+ };
+
+ n_l_trial_value = n_l_prev;
+ rho_LR_trial_value = rho_LR_prev;
+ mu_LR_trial_value = mu_micro_from_state(n_l_prev, rho_LR_prev);
+ rho_l_hat_trial_value =
+     alpha_M_effective_from_density(rho_LR_prev) *
+     (mu_macro_from_pressure(p, rho_LR_prev) - mu_LR_trial_value);
+ dn_l_dtrace_eps_trial = 0.0;
+ dn_l_dp_trial = 0.0;
+
+ if (!zero_dt)
+ {
+     solve_microstate(p, delta_epsilon_v, n_l_prev, rho_LR_prev,
+                     n_l_trial_value, rho_LR_trial_value,
+                     mu_LR_trial_value, rho_l_hat_trial_value);
+
+     const auto deps_v_fd =
+         std::max(1e-8, std::abs(delta_epsilon_v) * 1e-6 + 1e-8);
+     double n_l_eps_plus = n_l_prev;
+     double rho_LR_eps_plus = rho_LR_prev;
+     double mu_LR_eps_plus = mu_LR_trial_value;
+     double rho_l_hat_eps_plus = rho_l_hat_trial_value;
+     double n_l_eps_minus = n_l_prev;
+     double rho_LR_eps_minus = rho_LR_prev;
+     double mu_LR_eps_minus = mu_LR_trial_value;
+     double rho_l_hat_eps_minus = rho_l_hat_trial_value;
+
+     solve_microstate(p, delta_epsilon_v + deps_v_fd, n_l_prev, rho_LR_prev,
+                     n_l_eps_plus, rho_LR_eps_plus, mu_LR_eps_plus,
+                     rho_l_hat_eps_plus);
+     solve_microstate(p, delta_epsilon_v - deps_v_fd, n_l_prev, rho_LR_prev,
+                     n_l_eps_minus, rho_LR_eps_minus, mu_LR_eps_minus,
+                     rho_l_hat_eps_minus);
+
+     dn_l_dtrace_eps_trial =
+         (n_l_eps_plus - n_l_eps_minus) / (2.0 * deps_v_fd);
+
+     if (p <= -pressure_tolerance)
+     {
+         const auto dp_fd = std::max(1e-6, std::abs(p) * 1e-6);
+         double n_l_plus = n_l_prev;
+         double rho_LR_plus = rho_LR_prev;
+         double mu_LR_plus = mu_LR_trial_value;
+         double rho_l_hat_plus = rho_l_hat_trial_value;
+         double n_l_minus = n_l_prev;
+         double rho_LR_minus = rho_LR_prev;
+         double mu_LR_minus = mu_LR_trial_value;
+         double rho_l_hat_minus = rho_l_hat_trial_value;
+
+         solve_microstate(p + dp_fd, delta_epsilon_v, n_l_prev, rho_LR_prev,
+                         n_l_plus, rho_LR_plus, mu_LR_plus,
+                         rho_l_hat_plus);
+         solve_microstate(p - dp_fd, delta_epsilon_v, n_l_prev, rho_LR_prev,
+                         n_l_minus, rho_LR_minus, mu_LR_minus,
+                         rho_l_hat_minus);
+
+         dn_l_dp_trial = (n_l_plus - n_l_minus) / (2.0 * dp_fd);
+     }
+ }
+
+ if (use_scalar_microstate_storage)
+ {
+     n_l_trial_value =
+         std::clamp(n_l_trial_value, minimum_n_l, n_l_upper_bound);
+     rho_LR_trial_value = macro_density_from_state(rho_LR_trial_value);
+     const auto mu_LR_final =
+         mu_macro_from_pressure(p, rho_LR_trial_value);
+     const auto alpha_M_effective_final =
+         alpha_M_effective_from_density(rho_LR_trial_value);
+     mu_LR_trial_value =

```

```

+      mu_micro_from_state(n_l_trial_value, rho_LR_trial_value);
+      rho_l_hat_trial_value = alpha_M_effective_final *
+      (mu_LR_final - mu_LR_trial_value);
+    }
+
+    phi_m_trial_value = std::clamp(n_l_trial_value, minimum_n_l, n_l_upper_bound);
+    phi_M_trial_value = std::max(0.0, phi_total_trial - phi_m_trial_value);
+    epsilon_sw_increment_trial =
+    swelling_slope * (phi_m_trial_value - phi_m_prev);
+  }
+
+  @ComputeThermodynamicForces
+  {
+    o = o_0 + 0 * ao/ae^1 * Δe^1;
+  }
+
+  @Integrator
+  {
+    constexpr const auto Pr4 = Stensor4::K();
+    const auto the = 'v' / (la - ka);
+
+    if (withinElasticRange)
+    {
+      e^1 f -= Δe^1;
+      return true;
+    }
+
+    const auto epsr = strain(1.e-12);
+    const auto s = deviator0();
+    const auto q = sqrt(1.5 * s | s);
+    const auto p = -traced() / 3.0 + pamb;
+
+    const auto rpc_new = rpc + 0 * Δrpc;
+    const auto pc_new = rpc_new * young;
+
+    const auto f = q * q + M^2 * p * (p - pc_new);
+    const auto ∂f/∂p = M^2 * (2.0 * p - pc_new);
+    const auto ∂f/∂o = eval(3.0 * s - ∂f/∂p * zI / 3.0);
+    auto norm = sqrt(6.0 * q * q + ∂f/∂p * ∂f/∂p / 3.0);
+    norm = max(norm, epsr * young);
+    const auto n = ∂f/∂o / norm;
+    const auto ntr = ∂-f/∂p / norm;
+
+    const auto Δe^1 = eval[ΔΛ] ( * n);
+    const auto Δe^1 v = tracede^1();
+
+    const auto fchar = pc_char * young;
+
+    e^1 f = Δe^1 + Δe^1 - Δe^1;
+    Λ[] f = f / fchar;
+    frpc = Δrpc + Δe^1 v * the * (rpc_new - rpc_min);
+
+    const auto ∂/∂anorm =
+    (9.0 * s - 2.0 * M^2 / 9.0 * ∂f/∂p * zI) / norm;
+    const auto ∂/∂Δe^1 n =
+    (3.0 * Pr4 + 2.0 * M^2 / 9.0 * (zI * zI) - (n * ∂/∂anorm)) / norm *
+    ao/ae^1 * 0;
+    const auto ∂/∂Δnrpc =
+    (zI + ∂f/∂p * n / norm) * M^2 / (3.0 * norm) * 0 * young;
+    const auto ∂/∂Δe^1 v frpc = the * (rpc_new - rpc_min);
+
+    ∂e^1 / ∂Δe^1 f += ΔΛ[] * ∂/∂Δe^1 n;
+    ∂e^1 / ∂ΔΛ[] f = n;
+    ∂e^1 / ∂Δfrpc = ΔΛ[] * ∂/∂Δnrpc;
+
+    ∂Λ[] / ∂Δe^1 f = ∂(∂f | ao/ae^1) * 0 / fchar;
+    ∂Λ[] / ∂ΔΛ[] f = strain(0);
+    ∂Λ[] / ∂Δfrpc = -M^2 * p * 0 / fchar * young;
+
+    ∂/∂ΔΛ[] frpc = ∂/∂Δe^1 v frpc * ntr;
+    ∂/∂Δe^1 v frpc = ∂/∂Δe^1 v frpc * ΔΛ[] * (zI | ∂/∂Δe^1 n);
+    ∂/∂Δfrpc frpc =
+    1.0 + Δe^1 v * the * 0 + ∂/∂Δe^1 v frpc * ΔΛ[] * traced(∂/∂Δnrpc);
+  }
+
+  @ComputeFinalThermodynamicForces
+  {
+    o = o_0 + ao/ae^1 * Δe^1 - K * epsilon_sw_increment_trial * zI;
+  }
+
+  @UpdateAuxiliaryStateVariables
+  {
+    pc += Δrpc * young;
+    e^1[] += tracede^1(- Δe^1);
+    'v' *= exp(tracede^1());
+
+    n_l = n_l_trial_value;
+    rho_LR = rho_LR_trial_value;
+    epsilon_sw += epsilon_sw_increment_trial;
+    phi_m = phi_m_trial_value;
+    phi_M = phi_M_trial_value;
+    phi_value = phi_m + phi_M;
+    n_S_value = 1.0 - phi_value;
+    n_l_value = phi_M * macro_support_saturation_trial;
+    rho_LR_value = rho_LR;
+    omega_l_value = n_l * rho_LR / (n_S_value * rho_SR);
+    mu_LR_value = mu_LR_trial_value;
+    rho_l_hat_value = rho_l_hat_trial_value;
+    delta_epsilon_sw_value = epsilon_sw_increment_trial;
+    sigma_S_value = -K * epsilon_sw * zI;
+  }
+
+  @AdditionalConvergenceChecks
+  {
+    if (converged)
+    {
+      if (!withinElasticRange)
+      {
+        if [ΔΛ] ( < 0)
+        {
+          converged = false;
+          withinElasticRange = true;
+        }
+      }
+    }
+  }
+}

```



```

+@TangentOperator
+{
+  const auto a-/a**Sp = [6](const auto **p) {
+    return (1.0 - **S - **S) *
+      (*S < (1.0 - **S) && *S > **S
+        ? -**m / (*p * (1.0 - **m)) *
+        std::pow(std::pow(std::max(**p, 0.0) / *p,
+          1.0 / (1.0 - **m)) +
+          1.0,
+          -**m - 1.0) *
+        std::pow(std::max(**p, 0.0) / *p,
+          1.0 / (1.0 - **m) - 1.0)
+      : 0.0);
+  };
+
+  const auto a-/a5_dsm_micromacrop = [6](const auto p_arg) {
+    if (p_arg >= -1e-12)
+    {
+      return 0.0;
+    }
+
+    const auto inv_p = 1.0 / p_arg;
+    const auto exp_term =
+      std::exp(-tuller_capillary_prefactor * inv_p * inv_p);
+    return -2.0 * tuller_capillary_prefactor * exp_term * inv_p * inv_p *
+      inv_p;
+  };
+
+  a0/aΔ*p = -(K * swelling_slope * dn_ldp_trial) * zI;
+  a-/aΔε**S = 0.0 * zI;
+  a-/aΔε**Sp = use_macro_saturation_law
+    ? a-/a5_dsm_micromacrop(*p + Δ*p)
+    : a-/a**Sp(-(*p + Δ*p));
+
+  if ((smt == ELASTIC) || (smt == SECANTOPERATOR))
+  {
+    a0/aΔε** = a0/aε**1;
+  }
+  else if (smt == CONSISTENTTANGENTOPERATOR)
+  {
+    Stensor4 Je;
+    getPartialJacobianInvert(Je);
+    a0/aΔε** = a0/aε**1 * Je;
+  }
+  else
+  {
+    return false;
+  }
+
+  a0/aΔε** = a0/aΔε** -
+    (K * swelling_slope * dn_ldtrace_eps_trial) * (zI * zI);
+}
diff --git a/MaterialLib/SolidModels/MechanicsBase.h b/MaterialLib/SolidModels/MechanicsBase.h
index 64c99f898e..fa937ec80f 100644
--- a/MaterialLib/SolidModels/MechanicsBase.h
+++ b/MaterialLib/SolidModels/MechanicsBase.h
@@ -82,6 +82,22 @@ struct MechanicsBase
   MathLib::KelvinVector::KelvinVectorType<DisplacementDim>;
   using KelvinMatrix =
     MathLib::KelvinVector::KelvinMatrixType<DisplacementDim>;
+  static constexpr int KelvinSize =
+    MathLib::KelvinVector::kelvin_vector_dimensions(DisplacementDim);
+  using SaturationStrainJacobian = Eigen::Matrix<double, 1, KelvinSize>;
+
+  struct PressureCoupledResponse
+  {
+    KelvinVector stress;
+    double saturation;
+    KelvinMatrix dStress_dStrain;
+    KelvinVector dStress_dLiquidPressure;
+    SaturationStrainJacobian dSaturation_dStrain;
+    double dSaturation_dLiquidPressure;
+    KelvinVector swelling_stress = KelvinVector::Zero();
+    double liquid_mass_exchange_source = 0.0;
+    std::unique_ptr<MaterialStateVariables> state;
+  };
+
+  /// Computation of the constitutive relation for specific material model.
+  /// This should be implemented in the derived model. Fixed Kelvin vector and
@@ -99,6 +115,17 @@ struct MechanicsBase
   double const dt,
   MaterialStateVariables const& material_state_variables) const = 0;
+
+  virtual std::optional<PressureCoupledResponse> integrateStressPressureCoupled(
+    MaterialPropertyLib::VariableArray const& /*variable_array_prev*/,
+    MaterialPropertyLib::VariableArray const& /*variable_array*/,
+    double const /*t*/,
+    ParameterLib::SpatialPosition const& /*x*/,
+    double const /*dt*/,
+    MaterialStateVariables const& /*material_state_variables*/) const
+  {
+    return std::nullopt;
+  }
+
+  /// Helper type for providing access to internal variables.
+  struct InternalVariable
+  {
diff --git a/ProcessLib/Output/SecondaryVariable.cpp b/ProcessLib/Output/SecondaryVariable.cpp
index 74a78ebe3e..eb3ffe486a 100644
--- a/ProcessLib/Output/SecondaryVariable.cpp
+++ b/ProcessLib/Output/SecondaryVariable.cpp
@@ -39,6 +39,12 @@ void SecondaryVariableCollection::addSecondaryVariable(
   }
 }
+
+bool SecondaryVariableCollection::hasSecondaryVariable(
+  std::string const& internal_name) const
+{
+  return _configured_secondary_variables.contains(internal_name);
+}
+
+std::map<std::string, std::string>::const_iterator
+SecondaryVariableCollection::begin() const
+{
diff --git a/ProcessLib/Output/SecondaryVariable.h b/ProcessLib/Output/SecondaryVariable.h
index 95d4f60fd5..6ee58bb8b4 100644
--- a/ProcessLib/Output/SecondaryVariable.h

```

```

+++ b/ProcessLib/Output/SecondaryVariable.h
@@ -132,6 +132,8 @@ public:
     void addSecondaryVariable(std::string const& internal_name,
                             SecondaryVariableFunctions&& fcts);

+    bool hasSecondaryVariable(std::string const& internal_name) const;
+
+    /*! Returns the secondary variable with the given external name.
+    SecondaryVariable const& get(std::string const& external_name) const;

diff --git a/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h b/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h
index 06cd01ee9c..9065e566a3 100644
--- a/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h
+++ b/ProcessLib/RichardsMechanics/ConstitutiveRelations/ConstitutiveData.h
@@ -15,6 +15,7 @@
@@ -15,6 +15,7 @@
#include "LiquidDensity.h"
#include "MicroPressure.h"
#include "MicroSaturation.h"
#include "PressureCoupledSolidData.h"
#include "ProcessLib/ConstitutiveRelations/Base.h"
#include "ProcessLib/ConstitutiveRelations/EffectiveStressData.h"
#include "ProcessLib/ConstitutiveRelations/MechanicalStrainData.h"
@@ -61,6 +62,7 @@ template <int DisplacementDim>
using ConstitutiveData = std::tuple<
// T000 (CL) check if all that data should stay here
StiffnessTensor<DisplacementDim>,
+ PressureCoupledSolidData<DisplacementDim>,
ProcessLib::ThermoRichardsMechanics::PorosityData, Density, LiquidDensity,
ProcessLib::ThermoRichardsMechanics::BiotData,
ProcessLib::ThermoRichardsMechanics::SaturationDataDeriv,

diff --git a/ProcessLib/RichardsMechanics/ConstitutiveRelations/PressureCoupledSolidData.h b/ProcessLib/RichardsMechanics/ConstitutiveRelations/PressureCoupledSolidData.h
new file mode 100644
index 0000000000..bed20b9719
--- /dev/null
+++ b/ProcessLib/RichardsMechanics/ConstitutiveRelations/PressureCoupledSolidData.h
@@ -0,0 +1,55 @@
+/**
+ * \file
+ * \copyright
+ * \copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * Distributed under a Modified BSD License.
+ * See accompanying file LICENSE.txt or
+ * http://www.opengeosys.org/project/license
+ */
+
+#pragma once
+
+#include <limits>
+
+#include "MaterialLib/SolidModels/MechanicsBase.h"
+
+namespace ProcessLib::RichardsMechanics
+{
+template <int DisplacementDim>
+struct PressureCoupledSolidData
+{
+    using SaturationStrainJacobian =
+        typename MaterialLib::Solids::MechanicsBase<
+            DisplacementDim>::SaturationStrainJacobian;
+
+    bool is_active = false;
+    double saturation = std::numeric_limits<double>::quiet_NaN();
+    double dS_l_dp_cap = std::numeric_limits<double>::quiet_NaN();
+    SaturationStrainJacobian dS_l_dStrain =
+        SaturationStrainJacobian::Zero();
+    MathLib::KelvinVector::KelvinVectorType<DisplacementDim>
+        dSigma_dLiquidPressure =
+            MathLib::KelvinVector::KelvinVectorType<DisplacementDim>::Zero();
+    MathLib::KelvinVector::KelvinVectorType<DisplacementDim> swelling_stress =
+        MathLib::KelvinVector::KelvinVectorType<DisplacementDim>::Zero();
+    double liquid_mass_exchange_source = 0.0;
+};
+
+template <int DisplacementDim>
+PressureCoupledSolidData<DisplacementDim> makePressureCoupledSolidData(
+    typename MaterialLib::Solids::MechanicsBase<
+        DisplacementDim>::PressureCoupledResponse const& response)
+{
+    return {
+        true,
+        response.saturation,
+        // RichardsMechanics assembles in capillary pressure p_cap = -p_L.
+        -response.dSaturation_dLiquidPressure,
+        response.dSaturation_dStrain,
+        response.dStress_dLiquidPressure,
+        response.swelling_stress,
+        response.liquid_mass_exchange_source,
+    };
+}
+
+} // namespace ProcessLib::RichardsMechanics
diff --git a/ProcessLib/RichardsMechanics/IntegrationPointData.h b/ProcessLib/RichardsMechanics/IntegrationPointData.h
index f3cb05a418..732905fdd3 100644
--- a/ProcessLib/RichardsMechanics/IntegrationPointData.h
+++ b/ProcessLib/RichardsMechanics/IntegrationPointData.h
@@ -11,8 +11,10 @@
@@ -11,8 +11,10 @@
#pragma once

#include <memory>
#include <optional>

#include "ConstitutiveRelations/Base.h"
#include "ConstitutiveRelations/PressureCoupledSolidData.h"
#include "MaterialLib/SolidModels/MechanicsBase.h"
#include "MathLib/KelvinVector.h"
#include "ProcessLib/ConstitutiveRelations/EffectiveStressData.h"
@@ -29,6 +31,13 @@ struct IntegrationPointData final
    typename ShapeMatrixTypeDisplacement::NodalRowVectorType N_u;
    typename ShapeMatrixTypeDisplacement::GlobalDimNodalMatrixType dNdx_u;

+    struct ConstitutiveRelationUpdate final
+    {
+        typename BMatricesType::KelvinMatrixType C;
+        std::optional<PressureCoupledSolidData<DisplacementDim>>
+            pressure_coupled_data;
+    };
+}

```

```

typename ShapeMatricesTypePressure::NodalRowVectorType N_p;
typename ShapeMatricesTypePressure::GlobalDimNodalMatrixType dIdx_p;

@@ -64,8 +73,9 @@ struct IntegrationPointData final
    return C;
}

- static typename BMatricesType::KelvinMatrixType updateConstitutiveRelation(
+ static ConstitutiveRelationUpdate updateConstitutiveRelation(
    MaterialPropertyLib::VariableArray const& variable_array,
+   MaterialPropertyLib::VariableArray const& variable_array_prev,
    double const t,
    ParameterLib::SpatialPosition const& x_position,
    double const dt,
@@ -84,15 +94,30 @@ struct IntegrationPointData final
    std::unique_ptr<typename MaterialLib::Solids::MechanicsBase<
        DisplacementDim>::MaterialStateVariables& material_state_variables)
    {
-   MaterialPropertyLib::VariableArray variable_array_prev;
-   variable_array_prev.stress = sigma_eff_prev->sigma_eff;
-   variable_array_prev.mechanical_strain
+   auto variable_array_prev_local = variable_array_prev;
+   variable_array_prev_local.stress = sigma_eff_prev->sigma_eff;
+   variable_array_prev_local.mechanical_strain
+   .emplace<MathLib::KelvinVector::KelvinVectorType<DisplacementDim>>(
+       eps_m_prev->eps_m);
-   variable_array_prev.temperature = temperature;
+   variable_array_prev_local.temperature = temperature;
+
+   if (auto pressure_coupled_response =
+       solid_material.integrateStressPressureCoupled(
+           variable_array_prev_local, variable_array, t, x_position,
+           dt, *material_state_variables))
+   {
+       auto pressure_coupled_data =
+           makePressureCoupledSolidData<DisplacementDim>(
+               *pressure_coupled_response);
+       sigma_eff.sigma_eff = pressure_coupled_response->stress;
+       material_state_variables =
+           std::move(pressure_coupled_response->state);
+       return {std::move(pressure_coupled_response->dStress_dStrain),
+               std::move(pressure_coupled_data)};
+   }

    auto&& solution = solid_material.integrateStress(
        variable_array_prev, variable_array, t, x_position, dt,
+       variable_array_prev_local, variable_array, t, x_position, dt,
        *material_state_variables);

    if (!solution)
@@ -100,11 +125,11 @@ struct IntegrationPointData final
        OGS_FATAL("Computation of local constitutive relation failed.");
    }

-   MathLib::KelvinVector::KelvinMatrixType<DisplacementDim> C;
+   typename BMatricesType::KelvinMatrixType C;
    std::tie(sigma_eff.sigma_eff, material_state_variables, C) =
        std::move(*solution);

-   return C;
+   return {std::move(C), std::nullopt};
}

EIGEN_MAKE_ALIGNED_OPERATOR_NEW;
diff --git a/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h b/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h
index ea87fd5614..e27e83bd21 100644
--- a/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h
+++ b/ProcessLib/RichardsMechanics/RichardsMechanicsFEM-impl.h
@@ -31,6 +31,44 @@ namespace ProcessLib
{
    namespace RichardsMechanics
    {
+   template <int DisplacementDim, typename Chi, typename DChi>
+   void applyPressureCoupledSolidData(
+       PressureCoupledSolidData<DisplacementDim> const& pressure_coupled_solid,
+       MaterialPropertyLib::VariableArray& variables,
+       MaterialPropertyLib::VariableArray& variables_prev,
+       double const p_cap_ip,
+       double const p_cap_prev_ip,
+       double const S_L_prev,
+       Chi const& chi,
+       DChi const& dchi,
+       double& S_L,
+       double& dS_L_dp_cap,
+       double& DeltaS_L_Deltap_cap,
+       double& chi_S_L,
+       double& chi_S_L_prev,
+       double& dchi_dS_L,
+       double& p_FR)
+   {
+       if (!pressure_coupled_solid.is_active)
+       {
+           return;
+       }
+
+       S_L = pressure_coupled_solid.saturation;
+       dS_L_dp_cap = pressure_coupled_solid.dS_L_dp_cap;
+       DeltaS_L_Deltap_cap = (p_cap_ip == p_cap_prev_ip)
+           ? dS_L_dp_cap
+           : (S_L - S_L_prev) / (p_cap_ip - p_cap_prev_ip);
+       chi_S_L = chi(S_L);
+       chi_S_L_prev = chi(S_L_prev);
+       dchi_dS_L = dchi(S_L);
+       p_FR = -chi_S_L * p_cap_ip;
+       variables.liquid_saturation = S_L;
+       variables_prev.liquid_saturation = S_L_prev;
+       variables.effective_pore_pressure = p_FR;
+       variables_prev.effective_pore_pressure = -chi_S_L_prev * p_cap_prev_ip;
+   }

+   template <int DisplacementDim>
+   void updateSwellingStressAndVolumetricStrain(
+       MaterialPropertyLib::Medium const& medium,
@@ -220,7 +425,9 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
    auto const [p_L, u] = localDOF(local_x);

```

```

- constexpr double dt = std::numeric_limits<double>::quiet_NaN();
+ // The initial tangent query must use a real zero step, not NaN, because
+ // the pressure-coupled MFront bridge treats dt as part of the local update.
+ constexpr double dt = 0.0;
+ auto const& medium =
+   this->process_data->media_map.getMedium(this->element->getID());
+ MPL::VariableArray variables;
@@ -273,6 +313,7 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+   S_L;
+   S_L_prev = medium->property(MPL::PropertyType::saturation)
+     .template value<double>(variables, x_position, t, dt);
+   variables.liquid_saturation = S_L_prev;

+   if (this->process_data->initial_stress.isTotalStress())
+   {
@@ -383,7 +424,17 @@ void RichardsMechanicsLocalAssembler<
+   assert(local_x.size() == pressure_size + displacement_size);

+   auto const [p_L, u] = localDOF(local_x);
+   - auto const [p_L_prev, u_prev] = localDOF(local_x_prev);
+   + auto const [p_L_prev_raw, u_prev_raw] = localDOF(local_x_prev);
+   + auto p_L_prev = p_L_prev_raw.eval();
+   + if (!p_L_prev.allFinite())
+   + {
+   +   p_L_prev = p_L;
+   + }
+   + auto u_prev = u_prev_raw.eval();
+   + if (!u_prev.allFinite())
+   + {
+   +   u_prev = u;
+   + }

+   auto K = MathLib::createZeroedMatrix<
+   typename ShapeMatricesTypeDisplacement::template MatrixType>
@@ -469,6 +520,9 @@ void RichardsMechanicsLocalAssembler<
+   // setting p_0 to 1 atm
+   // TODO : rewrite equations s.t. p_L = p_0 - p_cap
+   variables.gas_phase_pressure = 1.0e5;
+   + variables_prev.capillary_pressure = p_cap_prev_ip;
+   + variables_prev.liquid_phase_pressure = -p_cap_prev_ip;
+   + variables_prev.gas_phase_pressure = 1.0e5;

+   auto const temperature =
+     medium->property(MPL::PropertyType::reference_temperature)
@@ -478,6 +532,10 @@ void RichardsMechanicsLocalAssembler<
+   auto const alpha =
+     medium->property(MPL::PropertyType::biot_coefficient)
+     .template value<double>(variables, x_position, t, dt);
+   + S_L = medium->property(MPL::PropertyType::saturation)
+   +   .template value<double>(variables, x_position, t, dt);
+   + variables.liquid_saturation = S_L;
+   + variables_prev.liquid_saturation = S_L_prev;
+   + auto& SD = this->current_states_[ip];
+   + variables.stress =
+     std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData>
@@ -508,14 +566,14 @@ void RichardsMechanicsLocalAssembler<
+   variables_prev.liquid_saturation = S_L_prev;

+   // tangent derivative for Jacobian
+   - double const dS_L_dp_cap =
+   + double dS_L_dp_cap =
+     medium->property(MPL::PropertyType::saturation)
+     .template dValue<double>(variables,
+       MPL::Variable::capillary_pressure,
+       x_position, t, dt);
+   // secant derivative from time discretization for storage
+   // use tangent, if secant is not available
+   - double const DeltaS_L_Deltap_cap =
+   + double DeltaS_L_Deltap_cap =
+     (p_cap_ip == p_cap_prev_ip)
+     ? dS_L_dp_cap
+     : (S_L - S_L_prev) / (p_cap_ip - p_cap_prev_ip);
@@ -527,10 +585,19 @@ void RichardsMechanicsLocalAssembler<
+   return medium->property(MPL::PropertyType::bishops_effective_stress)
+     .template value<double>(vs, x_position, t, dt);
+   };
+   - double const chi_S_L = chi(S_L);
+   - double const chi_S_L_prev = chi(S_L_prev);
+   + auto const dchi = [medium, x_position, t, dt](double const S_L)
+   + {
+   +   MPL::VariableArray vs;
+   +   vs.liquid_saturation = S_L;
+   +   return medium->property(MPL::PropertyType::bishops_effective_stress)
+   +     .template dValue<double>(vs, MPL::Variable::liquid_saturation,
+   +       x_position, t, dt);
+   + };
+   + double chi_S_L = chi(S_L);
+   + double chi_S_L_prev = chi(S_L_prev);
+   + double dchi_dS_L = dchi(S_L);

+   - double const p_FR = -chi_S_L * p_cap_ip;
+   + double p_FR = -chi_S_L * p_cap_ip;
+   + variables.effective_pore_pressure = p_FR;
+   + variables_prev.effective_pore_pressure = -chi_S_L_prev * p_cap_prev_ip;

@@ -630,46 +697,12 @@ void RichardsMechanicsLocalAssembler<
+   }
+   }

+   - double const k_rel =
+   -   medium->property(MPL::PropertyType::relative_permeability)
+   -   .template value<double>(variables, x_position, t, dt);
+   - auto const mu =
+   -   liquid_phase.property(MPL::PropertyType::viscosity)
+   -   .template value<double>(variables, x_position, t, dt);
+   -
+   - auto const& sigma_sw =
+   -   std::get<ProcessLib::ThermoRichardsMechanics::
+   -     ConstitutiveStressStrainTemperature::
+   -     SwellingDataStatefulDisplacementDim>(&
+   -     this->current_states_[ip])
+   -   .sigma_sw;
+   - auto const& sigma_eff =
+   -   std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData>
+   -   DisplacementDim>(&this->current_states_[ip])
+   -   .sigma_eff;

```

```

- // Set mechanical variables for the intrinsic permeability model
- // For stress dependent permeability.
- {
-     auto const sigma_total =
-         (sigma_eff - alpha * p_FR * identity2).eval();
-
-     // For stress dependent permeability.
-     variables.total_stress.emplaceSymmetricTensor(
-         MathLib::KelvinVector::kelvinVectorToSymmetricTensor(
-             sigma_total));
- }
-
- variables.equivalent_plastic_strain =
-     this->material_states[ip]
-         .material_state_variables->getEquivalentPlasticStrain();
-
- auto const K_intrinsic = MPL::formEigenTensor<DisplacementDim>(
-     medium->property(MPL::PropertyType::permeability)
-         .value(variables, x_position, t, dt));
-
- GlobalDimMatrixType const rho_K_over_mu =
-     K_intrinsic * rho_LR * k_rel / mu;
-
- //
- // displacement equation, displacement part
@@ -688,6 +721,8 @@ void RichardsMechanicsLocalAssembler<
-     eps_m);
- }
-
+ std::optional<PressureCoupledSolidData<DisplacementDim>>
+ pressure_coupled_solid;
+ {
+     auto& SD = this->current_states[ip];
+     auto const& SD_prev = this->prev_states[ip];
@@ -706,12 +741,57 @@ void RichardsMechanicsLocalAssembler<
-         MechanicalStrainData<DisplacementDim>>(&
-             SD_prev);
-
-     ip_data[ip].updateConstitutiveRelation(
-         variables, t, x_position, dt, temperature, sigma_eff,
-         sigma_eff_prev, eps_m, eps_m_prev, this->solid_material_,
+     auto constitutive_update = ip_data[ip].updateConstitutiveRelation(
+         variables, variables_prev, t, x_position, dt, temperature,
+         sigma_eff, sigma_eff_prev, eps_m, eps_m_prev,
+         this->solid_material_,
+         this->material_states[ip].material_state_variables);
+     pressure_coupled_solid = constitutive_update.pressure_coupled_data;
+
+     if (pressure_coupled_solid)
+     {
+         applyPressureCoupledSolidData<DisplacementDim>(
+             *pressure_coupled_solid, variables, variables_prev,
+             p_cap_ip, p_cap_prev_ip, S_L_prev, chi, dchi, S_L,
+             dS_L_dp_cap, DeltaS_L_DeltaT_cap, chi_S_L,
+             chi_S_L_prev, dchi_dS_L, p_FR);
+         std::get<ProcessLib::ThermoRichardsMechanics::
+             ConstitutiveStressStrainTemperature::
+             SwellingDataStateful<DisplacementDim>>(&
+                 this->current_states[ip])
+             .sigma_sw = pressure_coupled_solid->swelling_stress;
+     }
+ }
+
+ auto const& sigma_eff =
+     std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
+         DisplacementDim>>(&this->current_states[ip])
+         .sigma_eff;
+
+ auto const mu =
+     liquid_phase.property(MPL::PropertyType::viscosity)
+         .template value<double>(variables, x_position, t, dt);
+
+ double const k_rel =
+     medium->property(MPL::PropertyType::relative_permeability)
+         .template value<double>(variables, x_position, t, dt);
+
+ {
+     auto const sigma_total =
+         (sigma_eff - alpha * p_FR * identity2).eval();
+     variables.total_stress.emplaceSymmetricTensor(
+         MathLib::KelvinVector::kelvinVectorToSymmetricTensor(
+             sigma_total));
+ }
+
+ variables.equivalent_plastic_strain =
+     this->material_states[ip]
+         .material_state_variables->getEquivalentPlasticStrain();
+
+ auto const K_intrinsic = MPL::formEigenTensor<DisplacementDim>(
+     medium->property(MPL::PropertyType::permeability)
+         .value(variables, x_position, t, dt));
+ GlobalDimMatrixType const rho_K_over_mu =
+     K_intrinsic * rho_LR * k_rel / mu;
+
+ // p_SR
+ variables.solid_grain_pressure =
+     p_FR - sigma_eff.dot(identity2) / (3 * (1 - phi));
@@ -755,9 +835,10 @@ void RichardsMechanicsLocalAssembler<
- //
- // displacement equation, pressure part
- //
- K.template block<displacement_size, pressure_size>(displacement_index,
-     pressure_index)
-     .noalias() += B.transpose() * alpha * chi_S_L * identity2 * N_p * w;
+ K.template block<displacement_size, pressure_size>(&
+     displacement_index, pressure_index)
+     .noalias() +=
+         B.transpose() * alpha * chi_S_L * identity2 * N_p * w;
+
+ //
+ // pressure equation, displacement part.
@@ -766,6 +847,16 @@ void RichardsMechanicsLocalAssembler<
-     displacement_index)
-     .noalias() += N_p.transpose() * S_L * rho_LR * alpha *
-         identity2.transpose() * B * w;
+
+ if (pressure_coupled_solid && pressure_coupled_solid->is_active &&
+     !this->process_data.explicit_hm_coupling_in_unsaturated_zone)
+ {

```

```

+      M.template block<pressure_size, displacement_size>{
+          pressure_index, displacement_index)
+      .noalias() +=
+      N.p.transpose() * phi * rho_LR *
+      pressure_coupled_solid->dS_L_dStrain * B * w;
+  }
+  }

+  if (this->process_data->apply_mass_lumping)
@@ -808,6 +899,9 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
double const temperature = T_data();
double const p_cap_ip = p_cap_data.p_cap;
double const p_cap_prev_ip = p_cap_data.p_cap_prev;
+ variables_prev.capillary_pressure = p_cap_prev_ip;
+ variables_prev.liquid_phase_pressure = -p_cap_prev_ip;
+ variables_prev.gas_phase_pressure = 1.0e5;

+ auto const& eps = std::get<StrainData<DisplacementDim>>(SD);
+ auto& S_L =
@@ -822,6 +916,11 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+ .template value<double>(variables, x_position, t, dt);
+ *std::get<ProcessLib::ThermoRichardsMechanics::BiotData>(CD) = alpha;

+ S_L = medium->property(MPL::PropertyType::saturation)
+ .template value<double>(variables, x_position, t, dt);
+ variables.liquid_saturation = S_L;
+ variables_prev.liquid_saturation = S_L_prev;
+
+ variables.stress =
+ std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
+ DisplacementDim>>(SD)
@@ -839,6 +938,8 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
variables.grain_compressibility = beta_SR;
std::get<ProcessLib::ThermoRichardsMechanics::SolidCompressibilityData>(CD)
+ .beta_SR = beta_SR;
+ auto& pressure_coupled_solid =
+ std::get<PressureCoupledSolidData<DisplacementDim>>(CD);

+ auto const rho_LR =
+ liquid_phase.property(MPL::PropertyType::density)
@@ -846,27 +947,18 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
variables.density = rho_LR;
+ *std::get<LiquidDensity>(CD) = rho_LR;

- S_L = medium->property(MPL::PropertyType::saturation)
- .template value<double>(variables, x_position, t, dt);
- variables.liquid_saturation = S_L;
- variables_prev.liquid_saturation = S_L_prev;
-
- // tangent derivative for Jacobian
- double const dS_L_dp_cap =
+ double dS_L_dp_cap =
+ medium->property(MPL::PropertyType::saturation)
+ .template dValue<double>(variables,
+ MPL::Variable::capillary_pressure,
+ x_position, t, dt);
- std::get<ProcessLib::ThermoRichardsMechanics::SaturationDataDeriv>(CD)
- .dS_L_dp_cap = dS_L_dp_cap;
- // secant derivative from time discretization for storage
- // use tangent, if secant is not available
- double const DeltaS_L_Deltap_cap =
+ double DeltaS_L_Deltap_cap =
+ (p_cap_ip == p_cap_prev_ip)
+ ? dS_L_dp_cap
+ : (S_L - S_L_prev) / (p_cap_ip - p_cap_prev_ip);
- std::get<SaturationSecantDerivative>(CD).DeltaS_L_Deltap_cap =
- DeltaS_L_Deltap_cap;

+ auto const chi = [medium, x_position, t, dt](double const S_L)
+ {
@@ -875,21 +967,19 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
return medium->property(MPL::PropertyType::bishops_effective_stress)
+ .template value<double>(vs, x_position, t, dt);
+ });
- double const chi_S_L = chi(S_L);
- std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD).chi_S_L =
- chi_S_L;
- double const chi_S_L_prev = chi(S_L_prev);
- std::get<PrevState<ProcessLib::ThermoRichardsMechanics::BishopsData>>(CD)
- ->chi_S_L = chi_S_L_prev;
-
- auto const dchi_dS_L =
- medium->property(MPL::PropertyType::bishops_effective_stress)
- .template dValue<double>{
- variables, MPL::Variable::liquid_saturation, x_position, t, dt);
- std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD).dchi_dS_L =
- dchi_dS_L;
+ auto const dchi = [medium, x_position, t, dt](double const S_L)
+ {
+ MPL::VariableArray vs;
+ vs.liquid_saturation = S_L;
+ return medium->property(MPL::PropertyType::bishops_effective_stress)
+ .template dValue<double>(vs, MPL::Variable::liquid_saturation,
+ x_position, t, dt);
+ });
+ double chi_S_L = chi(S_L);
+ double chi_S_L_prev = chi(S_L_prev);
+ double dchi_dS_L = dchi(S_L);

- double const p_FR = -chi_S_L * p_cap_ip;
+ double p_FR = -chi_S_L * p_cap_ip;
+ variables.effective_pore_pressure = p_FR;
+ variables_prev.effective_pore_pressure = -chi_S_L_prev * p_cap_prev_ip;

@@ -1067,14 +1157,44 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
MechanicalStrainData<DisplacementDim>>(
SD_prev);

- auto C = ip_data.updateConstitutiveRelation(
- variables, t, x_position, dt, temperature, sigma_eff,
- sigma_eff_prev, eps_m, eps_m_prev, solid_material,
+ auto constitutive_update = ip_data.updateConstitutiveRelation(
+ variables, variables_prev, t, x_position, dt, temperature,
+ sigma_eff, sigma_eff_prev, eps_m, eps_m_prev, solid_material,
+ material_state_data.material_state_variables);
+
+ *std::get<StiffnessTensor<DisplacementDim>>(CD) = std::move(C);

```

```

+         *std::get<StiffnessTensor<DisplacementDim>>(CD) =
+         std::move(constitutive_update.C);
+         pressure_coupled_solid = constitutive_update.pressure_coupled_data
+         .value_or(PressureCoupledSolidData<
+         DisplacementDim>{});
+         applyPressureCoupledSolidData<DisplacementDim>{
+         pressure_coupled_solid, variables, variables_prev, p_cap_ip,
+         p_cap_prev_ip, S_L_prev, chi, dchi, S_L, dS_L_dp_cap,
+         DeltaS_L_Deltap_cap, chi_S_L, chi_S_L_prev, dchi_dS_L, p_FR};
+         if (pressure_coupled_solid.is_active)
+         {
+             std::get<ProcessLib::ThermoRichardsMechanics::
+             ConstitutiveStressStrainTemperature::
+             SwellingDataStateful<DisplacementDim>>(SD)
+             .sigma_sw = pressure_coupled_solid.swelling_stress;
+         }
+     }
+
+     std::get<ProcessLib::ThermoRichardsMechanics::SaturationDataDeriv>(CD)
+     .dS_L_dp_cap = dS_L_dp_cap;
+     std::get<SaturationSecantDerivative>(CD).DeltaS_L_Deltap_cap =
+     DeltaS_L_Deltap_cap;
+     std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD).chi_S_L =
+     chi_S_L;
+     std::get<PrevState<ProcessLib::ThermoRichardsMechanics::BishopsData>>(CD)
+     ->chi_S_L = chi_S_L_prev;
+     std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD).dchi_dS_L =
+     dchi_dS_L;
+
+     auto const k_rel_effective =
+     medium->property(MPL::PropertyType::relative_permeability)
+     .template value<double>(variables, x_position, t, dt);
+
+     // p_SR
+     variables.solid_grain_pressure =
+     p_FR - std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
+     @ -1085,8 +1205,32 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+     solid_phase.property(MPL::PropertyType::density)
+     .template value<double>(variables, x_position, t, dt);
+
+     {
+         auto const sigma_total =
+         (std::get<ProcessLib::ConstitutiveRelations::EffectiveStressData<
+         DisplacementDim>>(SD)
+         .sigma_eff -
+         alpha * p_FR * identity2)
+         .eval();
+         variables.total_stress.emplace<SymmetricTensor>(
+         MathLib::KelvinVector::kelvinVectorToSymmetricTensor(sigma_total));
+     }
+     variables.equivalent_plastic_strain =
+     material_state_data.material_state_variables
+     ->getEquivalentPlasticStrain();
+
+     double const rho = rho_SR * (1 - phi) + S_L * phi * rho_LR;
+     *std::get<Density>(CD) = rho;
+
+     auto const K_intrinsic_effective = MPL::formEigenTensor<DisplacementDim>{
+     medium->property(MPL::PropertyType::permeability)
+     .value(variables, x_position, t, dt)};
+     std::get<ProcessLib::ThermoRichardsMechanics::PermeabilityData<
+     DisplacementDim>>(CD)
+     .k_rel = k_rel_effective;
+     std::get<ProcessLib::ThermoRichardsMechanics::PermeabilityData<
+     DisplacementDim>>(CD)
+     .Ki = K_intrinsic_effective;
+ }
+
+ template <typename ShapeFunctionDisplacement, typename ShapeFunctionPressure,
+ @ -1102,7 +1246,17 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+ assert(local_x.size() == pressure_size + displacement_size);
+
+     auto const [p_L, u] = localDOF(local_x);
+     - auto const [p_L_prev, u_prev] = localDOF(local_x_prev);
+     + auto const [p_L_prev_raw, u_prev_raw] = localDOF(local_x_prev);
+     + auto p_L_prev = p_L_prev_raw.eval();
+     + if (!p_L_prev.allFinite())
+     + {
+     +     p_L_prev = p_L;
+     + }
+     + auto u_prev = u_prev_raw.eval();
+     + if (!u_prev.allFinite())
+     + {
+     +     u_prev = u;
+     + }
+
+     auto local_Jac = MathLib::createZeroedMatrix<
+     typename ShapeMatricesTypeDisplacement::template MatrixType>
+     @ -1198,6 +1352,9 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+     // setting pG to 1 atm
+     // T000 : rewrite equations s.t. p_L = pG-p_cap
+     variables.gas_phase_pressure = 1.0e5;
+     + variables_prev.capillary_pressure = p_cap_prev_ip;
+     + variables_prev.liquid_phase_pressure = -p_cap_prev_ip;
+     + variables_prev.gas_phase_pressure = 1.0e5;
+
+     auto const temperature =
+     medium->property(MPL::PropertyType::reference_temperature)
+     @ -1248,22 +1405,36 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+     (CD)
+     .dS_L_dp_cap;
+
+     auto const& pressure_coupled_solid =
+     std::get<PressureCoupledSolidData<DisplacementDim>>(CD);
+
+     {
+         double const chi_S_L =
+         std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD)
+         .chi_S_L;
+         Kup.noalias() +=
+         B.transpose() * alpha * chi_S_L * identity2 * N_p * w;
+
+         double const dchi_dS_L =
+         std::get<ProcessLib::ThermoRichardsMechanics::BishopsData>(CD)
+         .dchi_dS_L;
+
+         - local_Jac
+         - .template block<displacement_size, pressure_size>{

```

```

-         displacement_index, pressure_index)
-         .noalias() -= B.transpose() * alpha *
-         (chi_S_L + dchi_dS_L * p_cap_ip * dS_L_dp_cap) *
-         identity2 * N_p * w;
+
+     auto local_Jac_up =
+     local_Jac.template block<displacement_size, pressure_size>(
+     displacement_index, pressure_index);
+
+     // The pressure-coupled bridge returns effective-stress quantities.
+     // RM must still assemble the process-owned Biot/Bishop contribution.
+     local_Jac_up.noalias() -=
+     B.transpose() * alpha *
+     (chi_S_L + dchi_dS_L * p_cap_ip * dS_L_dp_cap) * identity2 *
+     N_p * w;
+
+     if (pressure_coupled_solid.is_active)
+     {
+         local_Jac_up.noalias() -=
+         B.transpose() * pressure_coupled_solid.dsigma_dLiquidPressure *
+         N_p * w;
+     }
+
+     double const phi =
+     @ -1323,6 +1494,13 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+         identity2.transpose() * B * w;
+
+     }
+
+     if (pressure_coupled_solid.is_active &&
+         !this->process_data.explicit_hm_coupling_in_unsaturated_zone)
+     {
+         Kpu.noalias() += N_p.transpose() * phi * rho_LR *
+         pressure_coupled_solid.dS_L_dStrain * B * w;
+     }
+
+     //
+     // pressure equation, pressure part.
+     //
+     @ -1423,6 +1601,19 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+     local_rhs.template segment<pressure_size>(pressure_index).noalias() +=
+     dndx_p.transpose() * rho_LR * k_rel * rho_Ki_over_mu * b * w;
+
+     if (pressure_coupled_solid.is_active)
+     {
+         // Bridge-side DSM micro-macro exchange can contribute a macro liquid
+         // source term even when RM does not own a native micro-pressure
+         // carrier state. The current bridge exposes only the source value;
+         // a consistent Jacobian term can be added later once the
+         // pressure-coupled solid contract is widened accordingly.
+         local_rhs.template segment<pressure_size>(pressure_index)
+         .noalias() +=
+         N_p.transpose() *
+         pressure_coupled_solid.liquid_mass_exchange_source * w;
+     }
+
+     if (medium->hasProperty(MPL::PropertyType::saturation_micro))
+     {
+         double const alpha_bar =
+         @ -1544,7 +1735,17 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+             Eigen::VectorXd const& local_x_prev)
+         {
+             auto const [p_L, u] = localDOF(local_x);
+             - auto const [p_L_prev, u_prev] = localDOF(local_x_prev);
+             auto const [p_L_prev_raw, u_prev_raw] = localDOF(local_x_prev);
+             auto p_L_prev = p_L_prev_raw.eval();
+             if (!p_L_prev.allFinite())
+             {
+                 p_L_prev = p_L;
+             }
+             auto u_prev = u_prev_raw.eval();
+             if (!u_prev.allFinite())
+             {
+                 u_prev = u;
+             }
+
+             auto const& identity2 = MathLib::KelvinVector::Invariants<
+             MathLib::KelvinVector::kelvin_vector_dimensions>
+             @ -1597,12 +1798,26 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+             // setting p_0 to 1 atm
+             // TODO : rewrite equations s.t. p_L = p_0 - p_cap
+             variables.gas_phase_pressure = 1.0e5;
+             variables_prev.capillary_pressure = p_cap_prev_ip;
+             variables_prev.liquid_phase_pressure = -p_cap_prev_ip;
+             variables_prev.gas_phase_pressure = 1.0e5;
+
+             auto const temperature =
+             medium->property(MPL::PropertyType::reference_temperature)
+             .template value<double>(variables, x_position, t, dt);
+             variables.temperature = temperature;
+
+             auto const S_L_prev =
+             std::get<
+             PrevState<ProcessLib::ThermoRichardsMechanics::SaturationData>>(
+             this->prev_states[ip])
+             ->S_L;
+             auto const S_L_for_tangent =
+             medium->property(MPL::PropertyType::saturation)
+             .template value<double>(variables, x_position, t, dt);
+             variables.liquid_saturation = S_L_for_tangent;
+             variables_prev.liquid_saturation = S_L_prev;
+
+             auto& eps =
+             std::get<StrainData<DisplacementDim>>(this->current_states[ip])
+             .eps;
+             @ -1611,15 +1826,19 @ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+             std::get<ProcessLib::ThermoRichardsMechanics::SaturationData>(
+             this->current_states[ip])
+             .S_L;
+             - auto const S_L_prev =
+             - std::get<
+             - PrevState<ProcessLib::ThermoRichardsMechanics::SaturationData>>(
+             - this->prev_states[ip])
+             - ->S_L;
+
+             S_L = medium->property(MPL::PropertyType::saturation)
+             .template value<double>(variables, x_position, t, dt);
+             variables.liquid_saturation = S_L;
+             variables_prev.liquid_saturation = S_L_prev;

```



```

+ double dS_L_dp_cap =
+   medium->property(MPL::PropertyType::saturation)
+   .template dValue<double>(variables,
+   MPL::Variable::capillary_pressure,
+   x_position, t, dt);
+ double DeltaS_L_DeltaP_cap =
+   (p_cap_ip == p_cap_prev_ip)
+   ? dS_L_dp_cap
+   : (S_L - S_L_prev) / (p_cap_ip - p_cap_prev_ip);
+
+ auto const chi = [medium, x_position, t, dt](double const S_L)
+ {
+   @ -1628,8 +1847,17 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+   return medium->property(MPL::PropertyType::bishops_effective_stress)
+   .template value<double>(vs, x_position, t, dt);
+ };
+ - double const chi_S_L = chi(S_L);
+ - double const chi_S_L_prev = chi(S_L_prev);
+ + auto const dchi = [medium, x_position, t, dt](double const S_L)
+ + {
+ +   MPL::VariableArray vs;
+ +   vs.liquid_saturation = S_L;
+ +   return medium->property(MPL::PropertyType::bishops_effective_stress)
+ +   .template dValue<double>(vs, MPL::Variable::liquid_saturation,
+ +   x_position, t, dt);
+ + };
+ + double chi_S_L = chi(S_L);
+ + double chi_S_L_prev = chi(S_L_prev);
+ + double dchi_dS_L = dchi(S_L);
+
+ auto const alpha =
+   medium->property(MPL::PropertyType::biot_coefficient)
+ @ -1651,7 +1879,8 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+   t, x_position, &c_el);
+ variables.grain_compressibility = beta_SR;
+
+ - variables.effective_pore_pressure = -chi_S_L * p_cap_ip;
+ + double p_FR = -chi_S_L * p_cap_ip;
+ + variables.effective_pore_pressure = p_FR;
+ + variables_prev.effective_pore_pressure = -chi_S_L_prev * p_cap_prev_ip;
+
+ // Set volumetric strain rate for the general case without swelling.
+ @ -1776,7 +2085,6 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+
+ GlobalDimMatrixType const K_over_mu = k_rel * K_intrinsic / mu;
+
+ - double const p_FR = -chi_S_L * p_cap_ip;
+ // p_SR
+ variables.solid_grain_pressure =
+   p_FR - sigma_eff.dot(identity2) / (3 * (1 - phi));
+ @ -1823,10 +2051,29 @@ void RichardsMechanicsLocalAssembler<ShapeFunctionDisplacement,
+   MechanicalStrainData<DisplacementDim>>(&
+   SD_prev);
+
+ - ip_data[ip].updateConstitutiveRelation(
+ -   variables, t, x_position, dt, temperature, sigma_eff,
+ -   sigma_eff_prev, eps_m, eps_m_prev, this->solid_material_,
+ -   this->material_states[ip].material_state_variables);
+ + auto const constitutive_update =
+ +   ip_data[ip].updateConstitutiveRelation(
+ +   variables, variables_prev, t, x_position, dt,
+ +   temperature, sigma_eff, sigma_eff_prev, eps_m,
+ +   eps_m_prev, this->solid_material_,
+ +   this->material_states[ip].material_state_variables);
+ +
+ + if (constitutive_update.pressure_coupled_data)
+ + {
+ +   // Keep stored/output saturation consistent with the
+ +   // pressure-coupled material response.
+ +   applyPressureCoupledSolidData<DisplacementDim>(&
+ +   *constitutive_update.pressure_coupled_data, variables,
+ +   variables_prev, p_cap_ip, p_cap_prev_ip, S_L_prev, chi,
+ +   dchi, S_L, dS_L_dp_cap, DeltaS_L_DeltaP_cap, chi_S_L,
+ +   chi_S_L_prev, dchi_dS_L, p_FR);
+ +   std::get<ProcessLib::ThermoRichardsMechanics::
+ +   ConstitutiveStressStrainTemperature::
+ +   SwellingDataStateful><DisplacementDim>(&
+ +   this->current_states[ip])
+ +   .sigma_sw =
+ +   constitutive_update.pressure_coupled_data->swelling_stress;
+ + }
+ }
+
+ auto const& b = this->process_data_.specific_body_force;
+ diff --git a/ProcessLib/RichardsMechanics/RichardsMechanicsProcess.cpp b/ProcessLib/RichardsMechanics/RichardsMechanicsProcess.cpp
+ index 09662a3de3..6643ad751e 100644
+ --- a/ProcessLib/RichardsMechanics/RichardsMechanicsProcess.cpp
+ +++ b/ProcessLib/RichardsMechanics/RichardsMechanicsProcess.cpp
+ @ -181,6 +181,12 @@ void RichardsMechanicsProcess<DisplacementDim>::initializeConcreteProcess(
+   int const num_components,
+   auto get_ip_values_function)
+ {
+   if (_secondary_variables.hasSecondaryVariable(name))
+   {
+     DEBUG("Skipping duplicate secondary variable {:s}.", name);
+     return;
+   }
+
+   _secondary_variables.addSecondaryVariable(
+     name,
+     makeExtrapolator(num_components, getExtrapolator()),
+ diff --git a/ProcessLib/RichardsMechanics/Tests.cmake b/ProcessLib/RichardsMechanics/Tests.cmake
+ index a90e2696f8..cef7f76b0 100644
+ --- a/ProcessLib/RichardsMechanics/Tests.cmake
+ +++ b/ProcessLib/RichardsMechanics/Tests.cmake
+ @ -32,6 +32,359 @@ if (NOT OGS_USE_MPI AND OGS_USE_MFRONT)
+   OgsTest(PROJECTFILE RichardsMechanics/mfront_restart_part1.prj RUNTIME 1)
+   OgsTest(PROJECTFILE RichardsMechanics/mfront_restart_part2.xml RUNTIME 1)
+   OgsTest(PROJECTFILE RichardsMechanics/DoubleStructureBenchmark/double_porosity_swelling_RM.prj RUNTIME 1)
+
+ +
+ + # These bridge/parity inputs still run via AddTest because they do not
+ + # have source-side benchmark definitions. Native-vs-bridge parity for the
+ + # benchmark and reduced shells is enforced by explicit compare tests below.
+ + AddTest(
+ +   NAME RichardsMechanics_mfront_restart_part1_rm_bridge
+ +   PATH RichardsMechanics
+ +   EXECUTABLE ogs

```

```

+ EXECUTABLE_ARGS mfront_restart_part1_rm_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_restart_part1_dsm_micromacro_mcc_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_restart_part1_dsm_micromacro_mcc_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_native
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_native.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_elastic_native
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_elastic_native.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_elastic_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_elastic_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_unsat_native
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_unsat_native.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_unsat_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_unsat_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_mcc_native
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_mcc_native.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_mcc_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_mcc_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_dsm_micromacro_mcc_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_dsm_micromacro_mcc_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_dsm_micromacro_mcc_tuller_native
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_dsm_micromacro_mcc_tuller_native.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_parity_1element_dsm_micromacro_mcc_tuller_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_parity_1element_dsm_micromacro_mcc_tuller_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_restart_part1_dsm_micromacro_mcc_tuller_native
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_restart_part1_dsm_micromacro_mcc_tuller_native.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_mfront_restart_part1_dsm_micromacro_mcc_tuller_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ EXECUTABLE_ARGS mfront_restart_part1_dsm_micromacro_mcc_tuller_bridge.prj
+ RUNTIME 1
+ )
+ AddTest(
+ NAME RichardsMechanics_beacon_1a01_dsm_micromacro_mcc_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_mcc_bridge.prj
+ WRAPPER time
+ )
+ AddTest(
+ NAME RichardsMechanics_beacon_1a01_dsm_micromacro_mcc_inflow_bridge
+ PATH RichardsMechanics
+ EXECUTABLE ogs
+ RUNTIME 20
+ EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_mcc_inflow_bridge.prj
+ WRAPPER time
+ )

```

```

+ AddTest(
+   NAME RichardsMechanics_beacon_1a01_dsm_micromacro_mcc_bridge_reference
+   PATH RichardsMechanics
+   EXECUTABLE ogs
+   RUNTIME 20
+   EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_mcc_bridge.prj
+   WRAPPER time
+   TESTER vtkdiff
+   DIFF_DATA
+   beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_mcc_bridge_t_1000.000000.vtu displacement displacement 1e-16 0
+   beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_mcc_bridge_t_1000.000000.vtu pressure pressure 1e-16 1e-12
+   beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_mcc_bridge_t_1000.000000.vtu saturation saturation 1e-14 0
+   beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_mcc_bridge_t_1000.000000.vtu porosity porosity 1e-16 0
+   beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_mcc_bridge_t_1000.000000.vtu transport_porosity transport_porosity 1e-16 0
+   beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_mcc_bridge_t_1000.000000.vtu swelling_stress swelling_stress 1e-16 0
+   beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1a01_dsm_micromacro_mcc_bridge_t_1000.000000.vtu sigma sigma 1e-16 1e-10
+ )
+ AddTest(
+   NAME RichardsMechanics_beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference
+   PATH RichardsMechanics
+   EXECUTABLE ogs
+   RUNTIME 20
+   EXECUTABLE_ARGS beacon_1a01_dsm_micromacro_mcc_inflow_bridge.prj
+   WRAPPER time
+   TESTER vtkdiff
+   DIFF_DATA
+   beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_mcc_inflow_bridge_t_100000.000000.vtu displacement displacement 1e-16
+
0
+   beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_mcc_inflow_bridge_t_100000.000000.vtu pressure pressure 1e-16 1e-12
+   beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_mcc_inflow_bridge_t_100000.000000.vtu saturation saturation 1e-14 0
+   beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_mcc_inflow_bridge_t_100000.000000.vtu porosity porosity 1e-16 0
+   beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_mcc_inflow_bridge_t_100000.000000.vtu transport_porosity transport_po
rosity 1e-16 0
+   beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_mcc_inflow_bridge_t_100000.000000.vtu swelling_stress swelling_stress
1e-16 0
+   beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu beacon_1a01_dsm_micromacro_mcc_inflow_bridge_t_100000.000000.vtu sigma sigma 1e-16 1e-10
+ )
+ AddTest(
+   NAME RichardsMechanics_beacon_1b_dsm_micromacro_mcc_bridge
+   PATH RichardsMechanics
+   EXECUTABLE ogs
+   RUNTIME 20
+   EXECUTABLE_ARGS beacon_1b_dsm_micromacro_mcc_bridge.prj
+   WRAPPER time
+ )
+ AddTest(
+   NAME RichardsMechanics_beacon_1b_dsm_micromacro_mcc_bridge_reference
+   PATH RichardsMechanics
+   EXECUTABLE ogs
+   RUNTIME 20
+   EXECUTABLE_ARGS beacon_1b_dsm_micromacro_mcc_bridge.prj
+   WRAPPER time
+   TESTER vtkdiff
+   DIFF_DATA
+   beacon_1b_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_mcc_bridge_t_1000.000000.vtu displacement displacement 1e-16 0
+   beacon_1b_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_mcc_bridge_t_1000.000000.vtu pressure pressure 1e-16 1e-12
+   beacon_1b_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_mcc_bridge_t_1000.000000.vtu saturation saturation 1e-14 0
+   beacon_1b_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_mcc_bridge_t_1000.000000.vtu porosity porosity 1e-16 0
+   beacon_1b_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_mcc_bridge_t_1000.000000.vtu transport_porosity transport_porosity 1e-16 0
+   beacon_1b_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_mcc_bridge_t_1000.000000.vtu swelling_stress swelling_stress 1e-16 0
+   beacon_1b_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1b_dsm_micromacro_mcc_bridge_t_1000.000000.vtu sigma sigma 1e-16 1e-10
+ )
+ AddTest(
+   NAME RichardsMechanics_beacon_1c_dsm_micromacro_mcc_bridge
+   PATH RichardsMechanics
+   EXECUTABLE ogs
+   RUNTIME 20
+   EXECUTABLE_ARGS beacon_1c_dsm_micromacro_mcc_bridge.prj
+   WRAPPER time
+ )
+ AddTest(
+   NAME RichardsMechanics_beacon_1c_dsm_micromacro_mcc_bridge_reference
+   PATH RichardsMechanics
+   EXECUTABLE ogs
+   RUNTIME 20
+   EXECUTABLE_ARGS beacon_1c_dsm_micromacro_mcc_bridge.prj
+   WRAPPER time
+   TESTER vtkdiff
+   DIFF_DATA
+   beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_mcc_bridge_t_1000.000000.vtu displacement displacement 1e-16 0
+   beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_mcc_bridge_t_1000.000000.vtu pressure pressure 1e-16 1e-12
+   beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_mcc_bridge_t_1000.000000.vtu saturation saturation 1e-14 0
+   beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_mcc_bridge_t_1000.000000.vtu porosity porosity 1e-16 0
+   beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_mcc_bridge_t_1000.000000.vtu transport_porosity transport_porosity 1e-16 0
+   beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_mcc_bridge_t_1000.000000.vtu swelling_stress swelling_stress 1e-16 0
+   beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu beacon_1c_dsm_micromacro_mcc_bridge_t_1000.000000.vtu sigma sigma 1e-16 1e-10
+ )
+ add_test(
+   NAME ogs-RichardsMechanics_mfront_parity_1element_compare
+   COMMAND
+   ${CMAKE_COMMAND}
+   -DGS_EXE=${TARGET_FILE:ogs}
+   -DVTKDIFF_EXE=${TARGET_FILE:vtkdiff}
+   -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+   -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_parity_compare
+   -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontParity.cmake
+ )
+ set_tests_properties(
+   ogs-RichardsMechanics_mfront_parity_1element_compare
+   PROPERTIES COST 1
+   LABELS "RichardsMechanics;default;small"
+   WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+ )
+ add_test(
+   NAME ogs-RichardsMechanics_mfront_parity_1element_elastic_compare
+   COMMAND
+   ${CMAKE_COMMAND}
+   -DGS_EXE=${TARGET_FILE:ogs}
+   -DVTKDIFF_EXE=${TARGET_FILE:vtkdiff}
+   -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+   -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_parity_elastic_compare
+   -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontElasticParity.cmake
+ )
+ set_tests_properties(
+   ogs-RichardsMechanics_mfront_parity_1element_elastic_compare
+   PROPERTIES COST 1
+   LABELS "RichardsMechanics;default;small"

```

```

+         WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+     add_test(
+         NAME ogs-RichardsMechanics_mfront_parity_1element_unsat_compare
+         COMMAND
+             ${CMAKE_COMMAND}
+             -DGS_EXE=${TARGET_FILE:ogs}
+             -DVTKDIFFF_EXE=${TARGET_FILE:vtkdifff}
+             -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+             -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_parity_unsat_compare
+             -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontUnsatElasticParity.cmake
+     )
+     set_tests_properties(
+         ogs-RichardsMechanics_mfront_parity_1element_unsat_compare
+         PROPERTIES COST 1
+             LABELS "RichardsMechanics;default;small"
+             WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+     add_test(
+         NAME ogs-RichardsMechanics_mfront_parity_1element_mcc_compare
+         COMMAND
+             ${CMAKE_COMMAND}
+             -DGS_EXE=${TARGET_FILE:ogs}
+             -DVTKDIFFF_EXE=${TARGET_FILE:vtkdifff}
+             -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+             -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_parity_mcc_compare
+             -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCParity.cmake
+     )
+     set_tests_properties(
+         ogs-RichardsMechanics_mfront_parity_1element_mcc_compare
+         PROPERTIES COST 1
+             LABELS "RichardsMechanics;default;small"
+             WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+     add_test(
+         NAME ogs-RichardsMechanics_mfront_parity_1element_dsm_micromacro_mcc_compare
+         COMMAND
+             ${CMAKE_COMMAND}
+             -DGS_EXE=${TARGET_FILE:ogs}
+             -DVTKDIFFF_EXE=${TARGET_FILE:vtkdifff}
+             -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+             -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_parity_dsm_micromacro_mcc_compare
+             -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCParity.cmake
+     )
+     set_tests_properties(
+         ogs-RichardsMechanics_mfront_parity_1element_dsm_micromacro_mcc_compare
+         PROPERTIES COST 1
+             LABELS "RichardsMechanics;default;small"
+             WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+     add_test(
+         NAME ogs-RichardsMechanics_mfront_parity_1element_dsm_micromacro_mcc_tuller_compare
+         COMMAND
+             ${CMAKE_COMMAND}
+             -DGS_EXE=${TARGET_FILE:ogs}
+             -DVTKDIFFF_EXE=${TARGET_FILE:vtkdifff}
+             -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+             -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_parity_dsm_micromacro_mcc_tuller_compare
+             -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCTullerParity.cmake
+     )
+     set_tests_properties(
+         ogs-RichardsMechanics_mfront_parity_1element_dsm_micromacro_mcc_tuller_compare
+         PROPERTIES COST 1
+             LABELS "RichardsMechanics;default;small"
+             WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+     add_test(
+         NAME ogs-RichardsMechanics_mfront_restart_part1_mcc_compare
+         COMMAND
+             ${CMAKE_COMMAND}
+             -DGS_EXE=${TARGET_FILE:ogs}
+             -DVTKDIFFF_EXE=${TARGET_FILE:vtkdifff}
+             -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+             -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_restart_part1_mcc_compare
+             -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCBenchmarkParity.cmake
+     )
+     set_tests_properties(
+         ogs-RichardsMechanics_mfront_restart_part1_mcc_compare
+         PROPERTIES COST 1
+             LABELS "RichardsMechanics;default;small"
+             WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+     add_test(
+         NAME ogs-RichardsMechanics_mfront_restart_part1_dsm_micromacro_mcc_compare
+         COMMAND
+             ${CMAKE_COMMAND}
+             -DGS_EXE=${TARGET_FILE:ogs}
+             -DVTKDIFFF_EXE=${TARGET_FILE:vtkdifff}
+             -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+             -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_compare
+             -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCBenchmarkParity.cmake
+     )
+     set_tests_properties(
+         ogs-RichardsMechanics_mfront_restart_part1_dsm_micromacro_mcc_compare
+         PROPERTIES COST 1
+             LABELS "RichardsMechanics;default;small"
+             WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+     add_test(
+         NAME ogs-RichardsMechanics_mfront_restart_part1_dsm_micromacro_mcc_tuller_compare
+         COMMAND
+             ${CMAKE_COMMAND}
+             -DGS_EXE=${TARGET_FILE:ogs}
+             -DVTKDIFFF_EXE=${TARGET_FILE:vtkdifff}
+             -DSOURCE_PATH=${Data_SOURCE_DIR}/RichardsMechanics
+             -DBINARY_PATH=${Data_BINARY_DIR}/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_tuller_compare
+             -P ${PROJECT_SOURCE_DIR}/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCTullerBenchmarkParity.cmake
+     )
+     set_tests_properties(
+         ogs-RichardsMechanics_mfront_restart_part1_dsm_micromacro_mcc_tuller_compare
+         PROPERTIES COST 1
+             LABELS "RichardsMechanics;default;small"
+             WORKING_DIRECTORY ${Data_SOURCE_DIR}/RichardsMechanics
+     )
+ )
endif()

AddTest(

```

```

diff --git a/Tests/CMakeLists.txt b/Tests/CMakeLists.txt
index 84d0700bec..144e4c476f 100644
--- a/Tests/CMakeLists.txt
+++ b/Tests/CMakeLists.txt
@@ -120,9 +120,12 @@ target_link_libraries(
 $<${TARGET_EXISTS:InSituLib}:InSituLib>
 $<${TARGET_EXISTS:petsc}:petsc>
 $<${TARGET_EXISTS:Python:Python}:Python>
- PUBLIC $<${B00L:${OGS_USE_MFRONT}}>:OgsMFrontBehaviourForUnitTests>
 )

+if(OGS_USE_MFRONT)
+ add_dependencies(testrunner OgsMFrontBehaviourForUnitTests)
+endif()
+
+target_compile_definitions(
+ testrunner
+ PUBLIC # Enabled elements
@@ -194,6 +197,59 @@ else()
 DEPENDS testrunner tests-cleanup
 )
endif()

+
+## Focused DSM unit CTests (native + MFront bridge) for quick branch checks.
+add_test(
+ NAME unit-RichardsMechanicsDSM-native-core
+ COMMAND
+ $<TARGET_FILE:testrunner>
+ --gtest_filters=RichardsMechanics.computeMicroPorosity:RichardsMechanics.makePressureCoupledSolidData
+)
+set_tests_properties(
+ unit-RichardsMechanicsDSM-native-core
+ PROPERTIES
+ WORKING_DIRECTORY ${PROJECT_BINARY_DIR}
+ LABELS "unit;ds;richardsmechanics;native"
+)
+
+add_test(
+ NAME unit-RichardsMechanicsDSM-bridge-core
+ COMMAND
+ $<TARGET_FILE:testrunner>
+ --gtest_filters=MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFront.OneStepPressureResponse:MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFront.FactoryPathPressureCoupledBlocks:MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFront.MicrostateHistoryResponse:MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFront.ZeroTPressureCoupledInitialAssemblyResponse
+)
+set_tests_properties(
+ unit-RichardsMechanicsDSM-bridge-core
+ PROPERTIES
+ WORKING_DIRECTORY ${PROJECT_BINARY_DIR}
+ LABELS "unit;ds;richardsmechanics;mfront;bridge"
+)
+
+add_test(
+ NAME unit-RichardsMechanicsDSM-bridge-mcc
+ COMMAND
+ $<TARGET_FILE:testrunner>
+ --gtest_filter=MaterialLib_RMBridgeMFront_DSMMicroMacroMCC.SwellingFeedbackChangesStressButKeepsCarrierSaturation:MaterialLib_RMBridgeMFront_DSMMicroMacroMCC.MicroOnlySwellingKeepsMacroExchangeOffAndPreservesCarrierSaturation:MaterialLib_RMBridgeMFront_DSMMicroMacroMCC.NativeAlignedStageOneStressGapIsMicroSupportStress:MaterialLib_RMBridgeMFront_DSMMicroMacroMCC.PressureCoupledResponseExposesMacroExchangeSource
+)
+set_tests_properties(
+ unit-RichardsMechanicsDSM-bridge-mcc
+ PROPERTIES
+ WORKING_DIRECTORY ${PROJECT_BINARY_DIR}
+ LABELS "unit;ds;richardsmechanics;mfront;bridge;mcc"
+)
+
+add_test(
+ NAME unit-RichardsMechanicsDSM-native-vs-mfront-mcc
+ COMMAND
+ $<TARGET_FILE:testrunner>
+ --gtest_filter=MaterialLib_RMBridgeMFront_MCC.PressureCoupledConstEMatchesNativeMechanicalPath:MaterialLib_RMBridgeMFront_MCC.PressureCoupledConstEFactoryCreationWorks:MaterialLib_RMBridgeMFront_MCC.PressureCoupledConstEMatchesNativeAcrossDryDensitySweep:MaterialLib_RMBridgeMFront_MCC.PressureCoupledConstETangentConsistentNearPlasticOnset:MaterialLib_RMBridgeMFront_MCC.PressureCoupledConstERestartStateTransferMatchesContinuousPath
+)
+set_tests_properties(
+ unit-RichardsMechanicsDSM-native-vs-mfront-mcc
+ PROPERTIES
+ WORKING_DIRECTORY ${PROJECT_BINARY_DIR}
+ LABELS "unit;ds;richardsmechanics;native;mfront;bridge;mcc;comparison"
+)
+
+## cmake-format: on
+
+unset(CMAKE_FOLDER)
diff --git a/Tests/Data/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv b/Tests/Data/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv
new file mode 100644
index 0000000000..8ebcc0852
--- /dev/null
+++ b/Tests/Data/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv
@@ -0,0 +1,6 @@
+## 0,0 +1,6 @@
+*step*, "pressure", "S_L", "mu_LR", "n_L", "phi_m", "phi_M", "phi", "n_S", "n_L", "rho_LR", "rho_LR", "omega_L", "mu_LR", "rho_L_hat", "delta_epsilon_sw", "epsilon_sw", "sigma_S_xx", "sigma_S_yy", "sigma_S_zz", "sigma_S_xy", "sigma_S_yz", "sigma_S_xz"
+0,0,1,0,"0.059316781504527311","0.059316781504527311","0.14068321849547272","0.2,0.0,0.0,0.14068321849547272","2193.3876431815676",1000,0.06584245728002587,-0.11767337584908294,0.11767337584908294,-0.0040632184954727,-0.0040632184954727,-0.0040632184954727,27122145.663648464,27122145.663648464,0.0,0
+1,250,1,0,"0.0593627466696872","0.0593627466696872","0.140637252338313,0.2,0.0,0.0,0.140637252338313","2193.3876306177184",1000,0.06589473998593624,-0.117480234537086,0.117480234537086,0.0000459661244140899,-0.00406372523383129,27091501.555354193,27091501.555354193,27091501.555354193,0.0,0
+2,500,1,0,"0.059374212533624873","0.059374212533624873","0.1406258746637514,0.2,0.0,0.0,0.1406258746637514","2193.3876197482889",1000,0.06590626582163789,-0.11733223945503486,0.11733223945503486,0.000011464866656152322,-0.004062578746637514,27083858.90207911,27083858.90207911,27083858.90207911,0.0,0
+3,750,1,0,"0.059379941646881342","0.059379941646881342","0.14062005835311867,0.2,0.0,0.0,0.14062005835311867","2193.3876133501935",1000,0.06591256502516539,-0.1172982812791468,0.1172982812791468,0.000011256469636-7,-0.004062005835311867,27080038.90207911,27080038.90207911,27080038.90207911,0.0,0
+4,1000,1,0,"0.059382805374452955","0.059382805374452955","0.14061719462554706,0.2,0.0,0.0,0.14061719462554706","2193.3876099104677",1000,0.06591734369941794,-0.11728131201736554,0.11728131201736554,2.8637275161285066-7,-0.004061719462554706,27078129.750364702,27078129.750364702,27078129.750364702,0.0,0
diff --git a/Tests/Data/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_strain_coupled_overlap_baseline.csv b/Tests/Data/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_strain_coupled_overlap_baseline.csv
new file mode 100644
index 0000000000..e8ebcc0852
--- /dev/null
+++ b/Tests/Data/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_strain_coupled_overlap_baseline.csv
@@ -0,0 +1,6 @@
+## 0,0 +1,6 @@
+*step*, "epsilon_v_total", "delta_epsilon_v", "S_L", "mu_LR", "n_L", "phi_m", "phi_M", "phi", "n_S", "n_L", "rho_LR", "rho_LR", "omega_L", "mu_LR", "rho_L_hat", "delta_epsilon_sw", "epsilon_sw", "sigma_S_xx", "sigma_S_yy", "sigma_S_zz", "sigma_S_xy", "sigma_S_yz", "sigma_S_xz"
+0,1000,0.0001,0.0001,1,0,"0.059448758824633521","0.059448758824633521","0.1405512411753665,"2193.1397976177655",1000,0.0659814974176539,-0.11689140334452633,0.11689140334452633,0.000006595345018056698,-0.004055124117536649,27708027.450244326,27708027.450244326,27708027.450244326,0.0,0
+1,1000,0.0002,0.0001,1,0,"0.059510602148738246","0.059510602148738246","0.14048939785126177,0.2,0.0,0.0,0.14048939785126177","2193.0380881843352",1000,0.06604787346195905,-0.1165273618309234,0.1165273618309234,0.00006184332410472446,-0.004048939785126176,28326265.234174512,28326265.234174512,28326265.234174512,0.0,0
+2,1000,0.00030000000000000003,0.00010000000000000002,1.0,"0.059572284544791592","0.059572284544791592","0.14042771545520844,0.2,0.0,0.0,0.14042771545520844","2192.9366689925291",1000,0.06609925291,0.0,0

```

[illegible]

```

+
+ </processes>
+ <output>
+   <type>VTK</type>
+   <prefix>ms33_no_anchor</prefix>
+   <timesteps>
+     <pair>
+       <repeat>1</repeat>
+       <each_steps>1</each_steps>
+     </pair>
+   </timesteps>
+   <variables>
+     <variable>displacement</variable>
+     <variable>sigma</variable>
+     <variable>epsilon</variable>
+   </variables>
+   <suffix>_ts_{:timestep}_t_{:time}</suffix>
+ </output>
+ </time_loop>
+ <media>
+   <medium id="0,1">
+     <phases>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+   </medium>
+ </media>
+ <parameters>
+   <parameter>
+     <name>E</name>
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>nu</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>displacement0</name>
+     <type>Constant</type>
+     <values>0 0</values>
+   </parameter>
+   <parameter>
+     <name>dirichlet0</name>
+     <type>Constant</type>
+     <value>0</value>
+   </parameter>
+   <parameter>
+     <name>dirichlet</name>
+     <type>Constant</type>
+     <value>0.05</value>
+   </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>physical_group_LeftBoundary</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>dirichlet0</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>physical_group_LeftBoundary</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>dirichlet0</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>physical_group_RightBoundary</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>dirichlet</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver>
+     <name>basic_newton</name>
+     <type>Newton</type>
+     <max_iter>40</max_iter>
+     <linear_solver>general_linear_solver</linear_solver>
+   </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver>
+     <name>general_linear_solver</name>
+     <lis>-i cg -p jacobi -tol 1e-16 -maxiter 10000</lis>
+     <eigen>
+       <solver_type>CG</solver_type>
+       <precon_type>DIAGONAL</precon_type>
+       <max_iteration_step>10000</max_iteration_step>
+       <error_tolerance>1e-16</error_tolerance>
+     </eigen>
+     <petsc>
+       <prefix>sd</prefix>
+       <parameters>-sd_ksp_type cg -sd_pc_type bjacobi -sd_ksp_rtol 1e-16 -sd_ksp_max_it 10000</parameters>
+     </petsc>
+   </linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.json b/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.json
new file mode 100644
index 0000000000..dba77fd9f9

```

```

--- /dev/null
+++ b/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.json
@@ -0,0 +1,13 @@
+{
+  "anchor_start_points": [
+    [0.0, 0.0, 0.0]
+  ],
+  "anchor_end_points": [
+    [1.0, 0.0, 0.0]
+  ],
+  "maximum_anchor_stress": [5e28],
+  "initial_anchor_stress": [0.0],
+  "residual_anchor_stress": [5e28],
+  "anchor_cross_sectional_area": [1.0],
+  "anchor_stiffness": [3.2]
+}
diff --git a/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.prj b/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.prj
new file mode 100644
index 0000000000..2d9ee672db
--- /dev/null
+++ b/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.prj
@@ -0,0 +1,177 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+  <meshes>
+    <mesh>domain.vtu</mesh>
+    <mesh>physical_group_LeftBoundary.vtu</mesh>
+    <mesh>physical_group_RightBoundary.vtu</mesh>
+    <mesh>ms33_one_anchor.vtu</mesh>
+  </meshes>
+  <processes>
+    <process>
+      <name>SD</name>
+      <type>SMALL_DEFORMATION</type>
+      <integration_order>2</integration_order>
+      <constitutive_relation id="0">
+        <type>LinearElasticIsotropic</type>
+        <youngs_modulus>E</youngs_modulus>
+        <poissons_ratio>nu</poissons_ratio>
+      </constitutive_relation>
+      <constitutive_relation id="1">
+        <type>LinearElasticIsotropic</type>
+        <youngs_modulus>E</youngs_modulus>
+        <poissons_ratio>nu</poissons_ratio>
+      </constitutive_relation>
+      <specific_body_force>0 0</specific_body_force>
+      <process_variables>
+        <process_variable>displacement</process_variable>
+      </process_variables>
+      <secondary_variables>
+        <secondary_variable name="sigma"/>
+        <secondary_variable name="epsilon"/>
+      </secondary_variables>
+    </process>
+  </processes>
+  <time_loop>
+    <processes>
+      <process ref="SD">
+        <nonlinear_solver>basic_newton</nonlinear_solver>
+        <convergence_criterion>
+          <type>Delta</type>
+          <norm_type>NORM2</norm_type>
+          <abstol>1e-15</abstol>
+        </convergence_criterion>
+        <time_discretization>
+          <type>BackwardEuler</type>
+        </time_discretization>
+        <time_stepping>
+          <type>FixedTimeStepping</type>
+          <t_initial>0</t_initial>
+          <t_end>1</t_end>
+          <timesteps>
+            <pair>
+              <repeat>1</repeat>
+              <delta_t>1</delta_t>
+            </pair>
+          </timesteps>
+        </time_stepping>
+      </process>
+    </processes>
+    <output>
+      <type>VTK</type>
+      <prefix>ms33_one_anchor</prefix>
+      <timesteps>
+        <pair>
+          <repeat>1</repeat>
+          <each_steps>1</each_steps>
+        </pair>
+      </timesteps>
+      <variables>
+        <variable>displacement</variable>
+        <variable>sigma</variable>
+        <variable>epsilon</variable>
+      </variables>
+      <suffix>_ts_{:timestep}_t_{:time}</suffix>
+    </output>
+  </time_loop>
+  <media>
+    <medium id="0,1">
+      <phases>
+        <phase>
+          <type>Solid</type>
+          <properties>
+            <property>
+              <name>density</name>
+              <type>Constant</type>
+              <value>1</value>
+            </property>
+          </properties>
+        </phase>
+      </phases>
+    </medium>
+  </media>
+  <parameters>
+    <parameter>
+      <name>E</name>
+      <type>Constant</type>

```



```

+         <value>1</value>
+     </parameter>
+     <parameter>
+         <name>nu</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>dirichlet0</name>
+         <type>Constant</type>
+         <value>0</value>
+     </parameter>
+     <parameter>
+         <name>dirichlet</name>
+         <type>Constant</type>
+         <value>0.05</value>
+     </parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>
+                 <mesh>physical_group_LeftBoundary</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>dirichlet0</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>physical_group_LeftBoundary</mesh>
+                 <type>Dirichlet</type>
+                 <component>1</component>
+                 <parameter>dirichlet0</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>physical_group_RightBoundary</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>dirichlet</parameter>
+             </boundary_condition>
+         </boundary_conditions>
+         <source_terms>
+             <source_term>
+                 <mesh>ms33_one_anchor</mesh>
+                 <type>EmbeddedAnchor</type>
+             </source_term>
+         </source_terms>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>40</max_iter>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <lis>-i cg -p jacobi -tol 1e-16 -maxiter 10000</lis>
+         <eigen>
+             <solver_type>CG</solver_type>
+             <precon_type>DIAGONAL</precon_type>
+             <max_iteration_step>10000</max_iteration_step>
+             <error_tolerance>1e-16</error_tolerance>
+         </eigen>
+         <petsc>
+             <prefix>sd</prefix>
+             <parameters>-sd_ksp_type cg -sd_pc_type bjacobi -sd_ksp_rtol 1e-16 -sd_ksp_max_it 10000</parameters>
+         </petsc>
+     </linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.vtu b/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.vtu
new file mode 100644
index 0000000000..357f519b6a
--- /dev/null
+++ b/Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm/ms33_one_anchor.vtu
@@ -0,0 +1,30 @@
<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian" header_type="UInt32" compressor="vtkZLibDataCompressor">
+ <UnstructuredGrid>
+     <Piece NumberOfPoints="2"                NumberOfCells="1"                >
+         <PointData>
+             <DataArray type="Float64" Name="natural_coordinates" NumberOfComponents="3" format="appended" RangeMin="1.4142135624"                RangeMax="1.4142135624"                offset="0"
+             />
+             <DataArray type="UInt64" Name="bulk_element_ids" format="appended" RangeMin="0"                RangeMax="0"                offset="56"                />
+             <DataArray type="UInt64" Name="point_cloud_node_ids" format="appended" RangeMin="0"                RangeMax="1"                offset="96"                />
+         </PointData>
+         <CellData>
+             <DataArray type="Float64" Name="initial_anchor_stress" format="appended" RangeMin="0"                RangeMax="0"                offset="140"                />
+             <DataArray type="Float64" Name="maximum_anchor_stress" format="appended" RangeMin="5e+28"                RangeMax="5e+28"                offset="180"                />
+             <DataArray type="Float64" Name="residual_anchor_stress" format="appended" RangeMin="5e+28"                RangeMax="5e+28"                offset="228"                />
+             <DataArray type="Float64" Name="anchor_cross_sectional_area" format="appended" RangeMin="1"                RangeMax="1"                offset="276"                />
+         </CellData>
+         <DataArray type="Float64" Name="anchor_stiffness" format="appended" RangeMin="3.2"                RangeMax="3.2"                offset="320"                />
+     </CellData>
+     <Points>
+         <DataArray type="Float64" Name="Points" NumberOfComponents="3" format="appended" RangeMin="0"                RangeMax="1"                offset="364"
+     />
+ </Points>
+ <Cells>
+     <DataArray type="Int64" Name="connectivity" format="appended" RangeMin=""                RangeMax=""                offset="412"                />
+     <DataArray type="Int64" Name="offsets" format="appended" RangeMin=""                RangeMax=""                offset="456"                />
+     <DataArray type="UInt8" Name="types" format="appended" RangeMin=""                RangeMax=""                offset="496"                />
+ </Cells>
+ </Piece>
+ </UnstructuredGrid>

```

```

+ Appended data encoding='base64':
+   AQAIAACAAAwAAAAfGAAAA==eJxjYACB0/YMKDQHfN1PLg4GswfPQ==AQAIAACAAAwAAAAA==eJxjYEAfAAQAAE=AQAIAACAAAwAAAAA==eJxjYIAARigVAAAYAAI=AQAIAACAAAwAAAAA==eJxjYIAAAAA
IAAE=AQAIAACAAAwAAAAA==eJxjYACB0/YMKDQHfN1PLg4GswfPQ==AQAIAACAAAwAAAAA==eJxjYEAfAAQAAE=AQAIAACAAAwAAAAA==eJxjYIAARigVAAAYAAI=AQAIAACAAAwAAAAA==eJxjYIAAAAA
AFB5D40=AQAIAACAAAwAAAAA==eJxjYMAHPTijwAAFTBBMA==AQAIAACAAAwAAAAA==eJxjYIAARigVAAAYAAI=AQAIAACAAAwAAAAA==eJxjYACB0/YAAIcBMA=AQAIAACAAAwAAAAA==eJxjYIAAAAA
+ </AppendedData>
+ </VTKFile>
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI.gitignore b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI.gitignore
new file mode 100644
index 0000000000..252b26cc70
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI.gitignore
@@ -0,0 +1,3 @@
+# Runtime outputs (not committed)
+*.vtu
+*.pvd
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/README.md b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/README.md
new file mode 100644
index 0000000000..e477784a28
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/README.md
@@ -0,0 +1,88 @@
+# ANCHORS MS33 Model-I
+
+This folder contains a simplified MS33 Model-I confined swelling sweep at
+three dry-density points.
+
+Tracked artefacts include:
+
+- project files ('ms33_model_i_dd1400.prj', 'ms33_model_i_dd1600.prj',
+  'ms33_model_i_dd1800.prj'),
+- summarized outputs ('ms33_model_i_history.csv',
+  'ms33_model_i_summary.json'),
+- the summarizer script ('summarize_ms33_model_i.py'),
+- the CIEMAT/Villar overlay script and outputs
+  ('plot_ms33_vs_ciemat_swelling_pressure_dd.py',
+  'ms33_vs_ciemat_swelling_pressure_vs_dry_density.csv',
+  'ms33_vs_ciemat_swelling_pressure_vs_dry_density.png'),
+- the dense dry-density MFront-vs-native calibration artifacts
+  ('run_villar_dense_dd_calibration.py',
+  'villar_dense_dd_calibration.csv',
+  'villar_dense_dd_calibration_summary.json',
+  'villar_dense_dd_swelling_pressure_comparison.png',
+  'villar_dense_dd_vdw_multiplier.png'),
+- the dense dry-density native-dsm_micromacro-branch Villar-style sweep
+  ('run_villar_dense_dd_native_dsm_micromacro_branch.py',
+  'villar_dense_dd_native_dsm_micromacro_branch.csv',
+  'villar_dense_dd_native_dsm_micromacro_branch_summary.json',
+  'villar_dense_dd_native_dsm_micromacro_branch_vs_villar.png'),
+- the dense dry-density native-dsm_micromacro-branch effective-vdw calibration
+  run ('run_villar_dense_dd_native_dsm_micromacro_calibration.py',
+  'villar_dense_dd_native_dsm_micromacro_calibration.csv',
+  'villar_dense_dd_native_dsm_micromacro_calibration_summary.json',
+  'villar_dense_dd_native_dsm_micromacro_calibration_comparison.png',
+  'villar_dense_dd_native_dsm_micromacro_vdw_multiplier.png'),
+- direct calibrated native-vs-MFront parity summary
+  ('villar_dense_dd_native_vs_mfront_calibrated_comparison.csv',
+  'villar_dense_dd_native_vs_mfront_calibrated_comparison_summary.json').
+
+Runtime-only outputs (*.vtu, *.pvd) are intentionally not versioned.
+
+Run provenance (including git hashes) is tracked in:
+- 'Tests/Data/RichardsMechanics/test_run_table.csv'
+
+Dense calibration run command (17 points from 1400 to 1800 kg/m³):
+
+```bash
+HOME=/tmp MPLCONFIGDIR=/tmp/mpicache XDG_CACHE_HOME=/tmp \
+python3 run_villar_dense_dd_calibration.py --dd-step 25 --rel-tol 0.02
+```
+
+Native dsm_micromacro branch Villar-style sweep command (same density grid):
+
+```bash
+HOME=/tmp MPLCONFIGDIR=/tmp/mpicache XDG_CACHE_HOME=/tmp \
+python3 run_villar_dense_dd_native_dsm_micromacro_branch.py --dd-step 25
+```
+
+Native dsm_micromacro branch dense calibration command
+@pointwise effective-vdw multiplier per dry density):
+
+```bash
+HOME=/tmp MPLCONFIGDIR=/tmp/mpicache XDG_CACHE_HOME=/tmp \
+python3 run_villar_dense_dd_native_dsm_micromacro_calibration.py \
+  --native-ogs /Users/vinaykumar/git/build/release-native-beacon/bin/ogs \
+  --native-source /Users/vinaykumar/Documents/GitHub/ogs \
+  --dd-step 25 --rel-tol 0.02
+```
+
+Key result from the native dsm_micromacro branch baseline run ('d46ellac00'):
+
+- simulated swelling pressure stays around '0.050' to '0.062' MPa for
+  '1400' to '1800' kg/m³, while Villar Eq.(7) rises from about '1.50' to
+  '22.56' MPa.
+- mean mismatch over the 17 dry-density points is about '-7.99' MPa
+  (native below target).
+
+Key result from the updated native dsm_micromacro branch calibration run:
+
+- baseline mean relative error vs Villar: '98.69%'.
+- calibrated mean relative error vs Villar: '1.26%'.
+- calibrated max relative error vs Villar: '1.83%'.
+- calibrated pressure follows Villar closely over '1400' to '1800' kg/m³.
+
+Native vs MFront comparison notes (same dsm_micromacro-derived equations target):
+
+- native dense run now uses 'scalar_micro_macro_mass_storage_mode' with
+  EOS-driven 'rho_lR' carry-over and robust multiplier bracketing in log-space.
+- calibrated native and calibrated MFront curves are now close on the dense
+  grid (mean absolute pressure difference about '0.135 MPa', max about
+  '0.467 MPa').
+- strict parameter-level parity is still open: fitted 'vdw_multiplier'
+  remains branch-dependent (MFront/native ratio roughly
+  '3.1e5' to '8.2e5').
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1400.prj b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1400.prj
new file mode 100644
index 0000000000..0be6f8758a
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1400.prj
@@ -0,0 +1,13 @@
+<?xml version='1.0' encoding='ISO-8859-1'?>
+<OpenGeoSysProject>

```

```

+ <meshes>
+   <mesh axially_symmetric="true">../square_1x1_quad_1e0.vtu</mesh>
+   <mesh axially_symmetric="true">../square_1x1_quad_1e0_left.vtu</mesh>
+   <mesh axially_symmetric="true">../square_1x1_quad_1e0_right.vtu</mesh>
+   <mesh axially_symmetric="true">../square_1x1_quad_1e0_top.vtu</mesh>
+   <mesh axially_symmetric="true">../square_1x1_quad_1e0_bottom.vtu</mesh>
+ </meshes>
+ <processes>
+   <process>
+     <name>RM</name>
+     <type>RICHARDS_MECHANICS</type>
+     <integration_order>2</integration_order>
+     <jacobian_assembler>
+       <type>Analytical</type>
+     </jacobian_assembler>
+     <constitutive_relation>
+       <type>LinearElasticIsotropic</type>
+       <youngs_modulus>YoungModulus</youngs_modulus>
+       <poissons_ratio>PoissonRatio</poissons_ratio>
+     </constitutive_relation>
+     <process_variables>
+       <pressure>pressure</pressure>
+       <displacement>displacement</displacement>
+     </process_variables>
+     <secondary_variables>
+       <secondary_variable name="sigma"/>
+       <secondary_variable name="swelling_stress"/>
+       <secondary_variable name="epsilon"/>
+       <secondary_variable name="velocity"/>
+       <secondary_variable name="saturation"/>
+       <secondary_variable name="porosity"/>
+       <secondary_variable name="dry_density_solid"/>
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+   </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2700.0</value>
+           </property>
+           <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>2e6 2e6 2e6</swelling_pressures>
+             <exponents>1 1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+       <property>
+         <name>permeability</name>
+         <type>KozenyCarman</type>
+         <initial_permeability>IntrinsicPermeability0</initial_permeability>
+         <initial_porosity>phi0</initial_porosity>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.45</exponent>
+         <p_b>27e6</p_b>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityGeneralizedPower</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <min_relative_permeability>1e-12</min_relative_permeability>
+         <enhancement_factor>1</enhancement_factor>
+         <exponent>3</exponent>
+       </property>
+     </properties>
+   </medium>
+ </media>

```

```

+         <property>
+             <name>bishops_effective_stress</name>
+             <type>BishopsSaturationCutoff</type>
+             <cutoff_value>1</cutoff_value>
+         </property>
+     </properties>
+ </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>basic_newton</nonlinear_solver>
+             <convergence_criterion>
+                 <type>PerComponentDeltaX</type>
+                 <norm_type>NORM2</norm_type>
+                 <reltols>1e-10 1e-10 1e-9</reltols>
+             </convergence_criterion>
+             <time_discretization>
+                 <type>BackwardEuler</type>
+             </time_discretization>
+             <time_stepping>
+                 <type>FixedTimeStepping</type>
+                 <t_initial>0</t_initial>
+                 <t_end>10368000</t_end>
+                 <timesteps>
+                     <pair>
+                         <repeat>120</repeat>
+                         <delta_t>86400</delta_t>
+                     </pair>
+                 </timesteps>
+             </time_stepping>
+         </process>
+     </processes>
+     <output>
+         <type>VTK</type>
+         <prefix>ms33_model_i_dd1400</prefix>
+         <suffix>_ts_{:timestep}_t_{:time}</suffix>
+         <timesteps>
+             <pair>
+                 <repeat>120</repeat>
+                 <each_steps>10</each_steps>
+             </pair>
+         </timesteps>
+         <variables>
+             <variable>displacement</variable>
+             <variable>pressure</variable>
+             <variable>sigma</variable>
+             <variable>swelling_stress</variable>
+             <variable>epsilon</variable>
+             <variable>velocity</variable>
+             <variable>saturation</variable>
+             <variable>porosity</variable>
+             <variable>dry_density_solid</variable>
+         </variables>
+     </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>0</expression>
+         <expression>0</expression>
+         <expression>0</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.49640287769784175</value>
+     </parameter>
+     <parameter>
+         <name>IntrinsicPermeability0</name>
+         <type>Constant</type>
+         <value>1.2264011098493141e-20</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>zero</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>-100e6</value>
+     </parameter>
+     <parameter>
+         <name>pressure_bc_scale</name>
+         <type>Constant</type>
+         <value>1</value>
+     </parameter>
+     <parameter>
+         <name>pressure_bc</name>
+         <type>CurveScaled</type>
+         <curve>ms33_pressure_release</curve>
+         <parameter>pressure_bc_scale</parameter>
+     </parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>

```

```

+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_left</mesh>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_right</mesh>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_bottom</mesh>
+             <type>Dirichlet</type>
+             <component>1</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_top</mesh>
+             <type>Dirichlet</type>
+             <component>1</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+     </boundary_conditions>
+ </process_variable>
+ <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+     <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+     </boundary_condition>
+     <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+     </boundary_condition>
+     <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+     </boundary_condition>
+     <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+     </boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>60</max_iter>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+             <solver_type>SparseLU</solver_type>
+             <scaling>true</scaling>
+         </eigen>
+     </linear_solver>
+ </linear_solvers>
+ <curves>
+     <curve>
+         <name>ms33_pressure_release</name>
+         <coords>0 10368000</coords>
+         <values>-100000000 0</values>
+     </curve>
+ </curves>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1600.prj b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1600.prj
new file mode 100644
index 0000000000..6ccfc5c02d
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1600.prj
@@ -0,0 +1,318 @@
+<?xml version='1.0' encoding='ISO-8859-1'?>
+<OpenGeoSysProject>
+    <meshes>
+        <mesh axially_symmetric="true">../square_1x1_quad_1e0.vtu</mesh>
+        <mesh axially_symmetric="true">../square_1x1_quad_1e0_left.vtu</mesh>
+        <mesh axially_symmetric="true">../square_1x1_quad_1e0_right.vtu</mesh>
+        <mesh axially_symmetric="true">../square_1x1_quad_1e0_top.vtu</mesh>
+        <mesh axially_symmetric="true">../square_1x1_quad_1e0_bottom.vtu</mesh>
+    </meshes>
+    <processes>
+        <process>
+            <name>RM</name>
+            <type>RICHARDS_MECHANICS</type>
+            <integration_order>2</integration_order>
+            <jacobian_assembler>
+                <type>Analytical</type>
+            </jacobian_assembler>
+            <constitutive_relation>
+                <type>LinearElasticIsotropic</type>
+                <youngs_modulus>YoungModulus</youngs_modulus>
+                <poissons_ratio>PoissonRatio</poissons_ratio>
+            </constitutive_relation>
+            <process_variables>
+                <pressure>pressure</pressure>
+                <displacement>displacement</displacement>
+            </process_variables>
+            <secondary_variables>
+                <secondary_variable name="sigma"/>
+                <secondary_variable name="swelling_stress"/>
+            </secondary_variables>
+        </process>
+    </processes>
+</OpenGeoSysProject>

```

```

+         <secondary_variable name="epsilon"/>
+         <secondary_variable name="velocity"/>
+         <secondary_variable name="saturation"/>
+         <secondary_variable name="porosity"/>
+         <secondary_variable name="dry_density_solid"/>
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2780.0</value>
+           </property>
+           <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>2e6 2e6</swelling_pressures>
+             <exponents>1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+       <property>
+         <name>permeability</name>
+         <type>KozenyCarman</type>
+         <initial_permeability>IntrinsicPermeability0</initial_permeability>
+         <initial_porosity>phi0</initial_porosity>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.45</exponent>
+         <p_b>27e6</p_b>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityGeneralizedPower</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <min_relative_permeability>1e-12</min_relative_permeability>
+         <enhancement_factor>1</enhancement_factor>
+         <exponent>3</exponent>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsSaturationCutoff</type>
+         <cutoff_value>1</cutoff_value>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e-10 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>10368000</t_end>
+         <timesteps>
+           <pair>
+             <repeat>120</repeat>

```

```

+         <delta_t>86400</delta_t>
+     </pair>
+ </timesteps>
+ </time_stepping>
+ </process>
+ </processes>
+ <output>
+     <type>VTK</type>
+     <prefix>ms33_model_l_dd1600</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+         <pair>
+             <repeat>120</repeat>
+             <each_steps>10</each_steps>
+         </pair>
+     </timesteps>
+     <variables>
+         <variable>displacement</variable>
+         <variable>pressure</variable>
+         <variable>sigma</variable>
+         <variable>swelling_stress</variable>
+         <variable>epsilon</variable>
+         <variable>velocity</variable>
+         <variable>saturation</variable>
+         <variable>porosity</variable>
+         <variable>dry_density_solid</variable>
+     </variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>0</expression>
+         <expression>0</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.4244604316546763</value>
+     </parameter>
+     <parameter>
+         <name>IntrinsicPermeability0</name>
+         <type>Constant</type>
+         <value>5.870260900123441e-21</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>zero</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>-100e6</value>
+     </parameter>
+     <parameter>
+         <name>pressure_bc_scale</name>
+         <type>Constant</type>
+         <value>1</value>
+     </parameter>
+     <parameter>
+         <name>pressure_bc</name>
+         <type>CurveScaled</type>
+         <curve>ms33_pressure_release</curve>
+         <parameter>pressure_bc_scale</parameter>
+     </parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_left</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_right</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_bottom</mesh>
+                 <type>Dirichlet</type>
+                 <component>1</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_top</mesh>
+                 <type>Dirichlet</type>
+                 <component>1</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+         </boundary_conditions>
+     </process_variable>

```

```

+     </process_variable>
+   </process_variables>
+   <nonlinear_solvers>
+     <nonlinear_solver>
+       <name>basic_newton</name>
+       <type>Newton</type>
+       <max_iter>60</max_iter>
+       <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+   </nonlinear_solvers>
+   <linear_solvers>
+     <linear_solver>
+       <name>general_linear_solver</name>
+       <eigen>
+         <solver_type>SparseLU</solver_type>
+         <scaling>true</scaling>
+       </eigen>
+     </linear_solver>
+   </linear_solvers>
+   <curves>
+     <curve>
+       <name>ms33_pressure_release</name>
+       <coords>0 10368000</coords>
+       <values>-100000000 0</values>
+     </curve>
+   </curves>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1800.prj b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1800.prj
new file mode 100644
index 0000000000..c012df8d70
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_dd1800.prj
@@ -0,0 +1,318 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
  <meshes>
    <mesh axially_symmetric="true">../square_1x1_quad_1e0.vtu</mesh>
    <mesh axially_symmetric="true">../square_1x1_quad_1e0_left.vtu</mesh>
    <mesh axially_symmetric="true">../square_1x1_quad_1e0_right.vtu</mesh>
    <mesh axially_symmetric="true">../square_1x1_quad_1e0_top.vtu</mesh>
    <mesh axially_symmetric="true">../square_1x1_quad_1e0_bottom.vtu</mesh>
  </meshes>
  <processes>
    <process>
      <name>RM</name>
      <type>RICHARDS_MECHANICS</type>
      <integration_order>2</integration_order>
      <jacobian_assembler>
        <type>Analytical</type>
      </jacobian_assembler>
      <constitutive_relation>
        <type>LinearElasticIsotropic</type>
        <youngs_modulus>YoungModulus</youngs_modulus>
        <poissons_ratio>PoissonRatio</poissons_ratio>
      </constitutive_relation>
      <process_variables>
        <pressure>pressure</pressure>
        <displacement>displacement</displacement>
      </process_variables>
      <secondary_variables>
        <secondary_variable name="sigma"/>
        <secondary_variable name="swelling_stress"/>
        <secondary_variable name="epsilon"/>
        <secondary_variable name="velocity"/>
        <secondary_variable name="saturation"/>
        <secondary_variable name="porosity"/>
        <secondary_variable name="dry_density_solid"/>
      </secondary_variables>
      <specific_body_force>0 0</specific_body_force>
      <initial_stress>sigma0</initial_stress>
      <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
      <mass_lumping>true</mass_lumping>
    </process>
  </processes>
  <media>
    <medium>
      <phases>
        <phase>
          <type>AqueousLiquid</type>
          <properties>
            <property>
              <name>viscosity</name>
              <type>Constant</type>
              <value>1e-3</value>
            </property>
            <property>
              <name>density</name>
              <type>Constant</type>
              <value>1e3</value>
            </property>
          </properties>
        </phase>
      </phases>
    </medium>
  </media>

```



```

+         </property>
+     </properties>
+ </phase>
+ <phase>
+     <type>Solid</type>
+     <properties>
+         <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2780.0</value>
+         </property>
+         <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>2e6 2e6 2e6</swelling_pressures>
+             <exponents>1 1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+         </property>
+     </properties>
+ </phase>
+ </phases>
+ <properties>
+     <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+     </property>
+     <property>
+         <name>permeability</name>
+         <type>KozenyCarman</type>
+         <initial_permeability>IntrinsicPermeability0</initial_permeability>
+         <initial_porosity>phi0</initial_porosity>
+     </property>
+     <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+     </property>
+     <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+     </property>
+     <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.45</exponent>
+         <p_b>27e6</p_b>
+     </property>
+     <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityGeneralizedPower</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <min_relative_permeability>1e-12</min_relative_permeability>
+         <enhancement_factor>1</enhancement_factor>
+         <exponent>3</exponent>
+     </property>
+     <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsSaturationCutoff</type>
+         <cutoff_value>1</cutoff_value>
+     </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>basic_newton</nonlinear_solver>
+             <convergence_criterion>
+                 <type>PerComponentDeltaX</type>
+                 <norm_type>NORM2</norm_type>
+                 <reltols>1e-10 1e-10 1e-9</reltols>
+             </convergence_criterion>
+             <time_discretization>
+                 <type>BackwardEuler</type>
+             </time_discretization>
+             <time_stepping>
+                 <type>FixedTimeStepping</type>
+                 <t_initial>0</t_initial>
+                 <t_end>10368000</t_end>
+                 <timesteps>
+                     <pair>
+                         <repeat>120</repeat>
+                         <delta_t>86400</delta_t>
+                     </pair>
+                 </timesteps>
+             </time_stepping>
+         </process>
+     </processes>
+ </output>
+     <type>VTK</type>
+     <prefix>ms33_model_l_dd1800</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+         <pair>
+             <repeat>120</repeat>
+             <each_steps>10</each_steps>
+         </pair>
+     </timesteps>
+ </variables>
+     <variable>displacement</variable>
+     <variable>pressure</variable>
+     <variable>sigma</variable>
+     <variable>swelling_stress</variable>
+     <variable>epsilon</variable>
+     <variable>velocity</variable>
+     <variable>saturation</variable>
+     <variable>porosity</variable>
+     <variable>dry_density_solid</variable>
+ </variables>

```

```

+ </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>sigma0</name>
+     <type>Function</type>
+     <expression>0</expression>
+     <expression>0</expression>
+     <expression>0</expression>
+     <expression>0</expression>
+   </parameter>
+   <parameter>
+     <name>YoungModulus</name>
+     <type>Constant</type>
+     <value>52e6</value>
+   </parameter>
+   <parameter>
+     <name>PoissonRatio</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>phi0</name>
+     <type>Constant</type>
+     <value>0.3525179856115108</value>
+   </parameter>
+   <parameter>
+     <name>IntrinsicPermeability0</name>
+     <type>Constant</type>
+     <value>2.6569572325678573e-21</value>
+   </parameter>
+   <parameter>
+     <name>displacement0</name>
+     <type>Constant</type>
+     <values>0 0</values>
+   </parameter>
+   <parameter>
+     <name>zero</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>pressure_ic</name>
+     <type>Constant</type>
+     <value>-100e6</value>
+   </parameter>
+   <parameter>
+     <name>pressure_bc_scale</name>
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>pressure_bc</name>
+     <type>CurveScaled</type>
+     <curve>ms33_pressure_release</curve>
+     <parameter>pressure_bc_scale</parameter>
+   </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>

```

```

+
+      </boundary_conditions>
+    </process_variable>
+  </process_variables>
+  <nonlinear_solvers>
+    <nonlinear_solver>
+      <name>basic_newton</name>
+      <type>Newton</type>
+      <max_iter>60</max_iter>
+      <linear_solver>general_linear_solver</linear_solver>
+    </nonlinear_solver>
+  </nonlinear_solvers>
+  <linear_solvers>
+    <linear_solver>
+      <name>general_linear_solver</name>
+      <eigen>
+        <solver_type>SparseLU</solver_type>
+        <scaling>true</scaling>
+      </eigen>
+    </linear_solver>
+  </linear_solvers>
+  <curves>
+    <curve>
+      <name>ms33_pressure_release</name>
+      <coords>0 1036000</coords>
+      <values>-100000000 0</values>
+    </curve>
+  </curves>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_history.csv b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_history.csv
new file mode 100644
index 0000000000..fc48db3f21
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_history.csv
@@ -0,0 +1,40 @@
+case,specific_weight_MN_m3,rho_dry_kg_m3,time_days,suction_MPa,mean_total_stress_MPa,mean_swelling_stress_MPa,saturation,porosity
+dd1400,14.0,1400.0,0.0,100.0,0.0,0.0,0.3292027868296924,0.4964028776978417
+dd1400,14.0,1400.0,10.0,91.66666666666666,0.044012616096298245,0.044012616096298245,0.3512090948778415,0.4964028776978417
+dd1400,14.0,1400.0,20.0,83.33333333333334,0.09474298316171385,0.09474298316171381,0.3765742784105493,0.4964028776978417
+dd1400,14.0,1400.0,30.0,75.0,0.15379720471088573,0.1537972047108857,0.4061013891851352,0.4964028776978417
+dd1400,14.0,1400.0,40.0,66.66666666666667,0.2232842925105747,0.2232842925105746,0.44084493308497974,0.4964028776978417
+dd1400,14.0,1400.0,50.0,58.33333333333333,0.3059822750145542,0.3059822750145542,0.4821939243369694,0.4964028776978417
+dd1400,14.0,1400.0,60.0,50.0,0.40551666277651843,0.4055166627765184,0.5319611182179516,0.4964028776978417
+dd1400,14.0,1400.0,70.0,41.666666666666664,0.5264364792612226,0.5264364792612225,0.5924210264603837,0.4964028776978417
+dd1400,14.0,1400.0,80.0,33.33333333333336,0.6737789323219373,0.6737789323219374,0.6660922529906612,0.4964028776978417
+dd1400,14.0,1400.0,90.0,25.0,0.8508469703911241,0.8508469703911239,0.7546262720252543,0.4964028776978417
+dd1400,14.0,1400.0,100.0,16.666666666666666,1.051828129576864,1.0518281295768634,0.8551168516181242,0.4964028776978417
+dd1400,14.0,1400.0,110.0,0.33333333333337,1.2437152554932764,1.2437152554932762,0.9510604145763306,0.4964028776978417
+dd1400,14.0,1400.0,120.0,0.0,0.13415944263406152,1.3415944263406145,0.9999999999999999,0.4964028776978417
+dd1600,16.0,1600.0,0.0,100.0,0.0,0.0,0.3292027868296924,0.42446043165467623
+dd1600,16.0,1600.0,10.0,91.66666666666666,0.044012616096298245,0.044012616096298245,0.3512090948778415,0.42446043165467623
+dd1600,16.0,1600.0,20.0,83.33333333333334,0.09474298316171385,0.09474298316171381,0.3765742784105493,0.42446043165467623
+dd1600,16.0,1600.0,30.0,75.0,0.15379720471088573,0.1537972047108857,0.4061013891851352,0.42446043165467623
+dd1600,16.0,1600.0,40.0,66.66666666666667,0.2232842925105747,0.2232842925105746,0.44084493308497974,0.42446043165467623
+dd1600,16.0,1600.0,50.0,58.33333333333333,0.3059822750145542,0.3059822750145542,0.4821939243369694,0.42446043165467623
+dd1600,16.0,1600.0,60.0,50.0,0.40551666277651843,0.4055166627765184,0.5319611182179516,0.42446043165467623
+dd1600,16.0,1600.0,70.0,41.666666666666664,0.5264364792612226,0.5264364792612225,0.5924210264603837,0.42446043165467623
+dd1600,16.0,1600.0,80.0,33.33333333333336,0.6737789323219373,0.6737789323219374,0.6660922529906612,0.42446043165467623
+dd1600,16.0,1600.0,90.0,25.0,0.8508469703911241,0.8508469703911239,0.7546262720252543,0.42446043165467623
+dd1600,16.0,1600.0,100.0,16.666666666666666,1.051828129576864,1.0518281295768634,0.8551168516181242,0.42446043165467623
+dd1600,16.0,1600.0,110.0,0.33333333333337,1.2437152554932764,1.2437152554932762,0.9510604145763306,0.42446043165467623
+dd1600,16.0,1600.0,120.0,0.0,0.13415944263406152,1.3415944263406145,0.9999999999999999,0.42446043165467623
+dd1800,18.0,1800.0,0.0,100.0,0.0,0.0,0.3292027868296924,0.35251798561151076
+dd1800,18.0,1800.0,10.0,91.66666666666666,0.044012616096298245,0.044012616096298245,0.3512090948778415,0.35251798561151076
+dd1800,18.0,1800.0,20.0,83.33333333333334,0.09474298316171385,0.09474298316171381,0.3765742784105493,0.35251798561151076
+dd1800,18.0,1800.0,30.0,75.0,0.15379720471088573,0.1537972047108857,0.4061013891851352,0.35251798561151076
+dd1800,18.0,1800.0,40.0,66.66666666666667,0.2232842925105747,0.2232842925105746,0.44084493308497974,0.35251798561151076
+dd1800,18.0,1800.0,50.0,58.33333333333333,0.3059822750145542,0.3059822750145542,0.4821939243369694,0.35251798561151076
+dd1800,18.0,1800.0,60.0,50.0,0.40551666277651843,0.4055166627765184,0.5319611182179516,0.35251798561151076
+dd1800,18.0,1800.0,70.0,41.666666666666664,0.5264364792612226,0.5264364792612225,0.5924210264603837,0.35251798561151076
+dd1800,18.0,1800.0,80.0,33.33333333333336,0.6737789323219373,0.6737789323219374,0.6660922529906612,0.35251798561151076
+dd1800,18.0,1800.0,90.0,25.0,0.8508469703911241,0.8508469703911239,0.7546262720252543,0.35251798561151076
+dd1800,18.0,1800.0,100.0,16.666666666666666,1.051828129576864,1.0518281295768634,0.8551168516181242,0.35251798561151076
+dd1800,18.0,1800.0,110.0,0.33333333333337,1.2437152554932764,1.2437152554932762,0.9510604145763306,0.35251798561151076
+dd1800,18.0,1800.0,120.0,0.0,0.13415944263406152,1.3415944263406145,0.9999999999999999,0.35251798561151076
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_summary.json b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_summary.json
new file mode 100644
index 0000000000..de0f659091
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_model_i_summary.json
@@ -0,0 +1,749 @@
+{
+  "rho_solid_kg_m3": 2700.0,
+  "cases": {
+    "dd1400": {
+      "rho_dry": 1400.0,
+      "specific_weight_MN_m3": 14.0,
+      "phi0": 0.49640287769784175,
+      "k0": 1.2264011098493141e-20,
+      "history": {
+        {
+          "pressure_Pa_avg": -100000000.0,
+          "pressure_Pa_min": -100000000.0,
+          "pressure_Pa_max": -100000000.0,
+          "suction_MPa_avg": 100.0,
+          "saturation_avg": 0.3292027868296924,
+          "porosity_avg": 0.4964028776978417,
+          "porosity_spread": 0.0,
+          "dry_density_avg_kg_m3": 1399.9999999999998,
+          "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+          "mean_total_stress_MPa": 0.0,
+          "mean_swelling_stress_MPa": 0.0,
+          "timestep": 0,
+          "time_s": 0.0,
+          "time_days": 0.0
+        },
+        {
+          "pressure_Pa_avg": -9166666.66666666,
+          "pressure_Pa_min": -9166666.66666666,
+          "pressure_Pa_max": -9166666.66666666,
+          "suction_MPa_avg": 91.66666666666666,
+          "saturation_avg": 0.3512090948778415,
+          "porosity_avg": 0.4964028776978417,
+          "porosity_spread": 0.0,
+          "dry_density_avg_kg_m3": 1399.9999999999998,
+          "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+          "mean_total_stress_MPa": 0.044012616096298245,
+          "mean_swelling_stress_MPa": 0.044012616096298245,

```

```

+         "timestep": 10,
+         "time_s": 864000.0,
+         "time_days": 10.0
+     },
+     {
+         "pressure_Pa_avg": -83333333.33333334,
+         "pressure_Pa_min": -83333333.33333334,
+         "pressure_Pa_max": -83333333.33333334,
+         "suction_MPa_avg": 83.33333333333334,
+         "saturation_avg": 0.3765742784105493,
+         "porosity_avg": 0.4964028776978417,
+         "porosity_spread": 0.0,
+         "dry_density_avg_kg_m3": 1399.9999999999998,
+         "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+         "mean_total_stress_MPa": 0.09474298316171385,
+         "mean_swelling_stress_MPa": 0.09474298316171381,
+         "timestep": 20,
+         "time_s": 1728000.0,
+         "time_days": 20.0
+     },
+     {
+         "pressure_Pa_avg": -75000000.0,
+         "pressure_Pa_min": -75000000.0,
+         "pressure_Pa_max": -75000000.0,
+         "suction_MPa_avg": 75.0,
+         "saturation_avg": 0.4061013891851352,
+         "porosity_avg": 0.4964028776978417,
+         "porosity_spread": 0.0,
+         "dry_density_avg_kg_m3": 1399.9999999999998,
+         "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+         "mean_total_stress_MPa": 0.15379720471088573,
+         "mean_swelling_stress_MPa": 0.1537972047108857,
+         "timestep": 30,
+         "time_s": 2592000.0,
+         "time_days": 30.0
+     },
+     {
+         "pressure_Pa_avg": -66666666.66666667,
+         "pressure_Pa_min": -66666666.66666667,
+         "pressure_Pa_max": -66666666.66666667,
+         "suction_MPa_avg": 66.66666666666667,
+         "saturation_avg": 0.44084493388497974,
+         "porosity_avg": 0.4964028776978417,
+         "porosity_spread": 0.0,
+         "dry_density_avg_kg_m3": 1399.9999999999998,
+         "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+         "mean_total_stress_MPa": 0.2232842925105747,
+         "mean_swelling_stress_MPa": 0.2232842925105746,
+         "timestep": 40,
+         "time_s": 3456000.0,
+         "time_days": 40.0
+     },
+     {
+         "pressure_Pa_avg": -58333333.33333333,
+         "pressure_Pa_min": -58333333.33333333,
+         "pressure_Pa_max": -58333333.33333333,
+         "suction_MPa_avg": 58.33333333333333,
+         "saturation_avg": 0.4821939243369694,
+         "porosity_avg": 0.4964028776978417,
+         "porosity_spread": 0.0,
+         "dry_density_avg_kg_m3": 1399.9999999999998,
+         "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+         "mean_total_stress_MPa": 0.3059822750145542,
+         "mean_swelling_stress_MPa": 0.3059822750145542,
+         "timestep": 50,
+         "time_s": 4320000.0,
+         "time_days": 50.0
+     },
+     {
+         "pressure_Pa_avg": -50000000.0,
+         "pressure_Pa_min": -50000000.0,
+         "pressure_Pa_max": -50000000.0,
+         "suction_MPa_avg": 50.0,
+         "saturation_avg": 0.531961182179516,
+         "porosity_avg": 0.4964028776978417,
+         "porosity_spread": 0.0,
+         "dry_density_avg_kg_m3": 1399.9999999999998,
+         "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+         "mean_total_stress_MPa": 0.40551666277651843,
+         "mean_swelling_stress_MPa": 0.4055166627765184,
+         "timestep": 60,
+         "time_s": 5184000.0,
+         "time_days": 60.0
+     },
+     {
+         "pressure_Pa_avg": -41666666.66666664,
+         "pressure_Pa_min": -41666666.66666664,
+         "pressure_Pa_max": -41666666.66666664,
+         "suction_MPa_avg": 41.66666666666664,
+         "saturation_avg": 0.5924210264603837,
+         "porosity_avg": 0.4964028776978417,
+         "porosity_spread": 0.0,
+         "dry_density_avg_kg_m3": 1399.9999999999998,
+         "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+         "mean_total_stress_MPa": 0.5264364792612226,
+         "mean_swelling_stress_MPa": 0.5264364792612225,
+         "timestep": 70,
+         "time_s": 6048000.0,
+         "time_days": 70.0
+     },
+     {
+         "pressure_Pa_avg": -33333333.33333336,
+         "pressure_Pa_min": -33333333.33333336,
+         "pressure_Pa_max": -33333333.33333336,
+         "suction_MPa_avg": 33.33333333333336,
+         "saturation_avg": 0.6660922529906612,
+         "porosity_avg": 0.4964028776978417,
+         "porosity_spread": 0.0,
+         "dry_density_avg_kg_m3": 1399.9999999999998,
+         "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+         "mean_total_stress_MPa": 0.6737789323219373,
+         "mean_swelling_stress_MPa": 0.6737789323219374,
+         "timestep": 80,
+         "time_s": 6912000.0,
+         "time_days": 80.0
+     },
+     {

```

```

+      "pressure_Pa_avg": -25000000.0,
+      "pressure_Pa_min": -25000000.0,
+      "pressure_Pa_max": -25000000.0,
+      "suction_MPa_avg": 25.0,
+      "saturation_avg": 0.7546262728252543,
+      "porosity_avg": 0.4964028776978417,
+      "porosity_spread": 0.0,
+      "dry_density_avg_kg_m3": 1399.9999999999998,
+      "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+      "mean_total_stress_MPa": 0.8508469703911241,
+      "mean_swelling_stress_MPa": 0.8508469703911239,
+      "timestep": 90,
+      "time_s": 7776000.0,
+      "time_days": 90.0
+    },
+    {
+      "pressure_Pa_avg": -16666666.666666662,
+      "pressure_Pa_min": -16666666.666666662,
+      "pressure_Pa_max": -16666666.666666662,
+      "suction_MPa_avg": 16.666666666666666,
+      "saturation_avg": 0.8551168516181242,
+      "porosity_avg": 0.4964028776978417,
+      "porosity_spread": 0.0,
+      "dry_density_avg_kg_m3": 1399.9999999999998,
+      "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+      "mean_total_stress_MPa": 1.051828129576864,
+      "mean_swelling_stress_MPa": 1.0518281295768634,
+      "timestep": 100,
+      "time_s": 8640000.0,
+      "time_days": 100.0
+    },
+    {
+      "pressure_Pa_avg": -8333333.333333337,
+      "pressure_Pa_min": -8333333.333333337,
+      "pressure_Pa_max": -8333333.333333337,
+      "suction_MPa_avg": 8.333333333333337,
+      "saturation_avg": 0.9510604145763306,
+      "porosity_avg": 0.4964028776978417,
+      "porosity_spread": 0.0,
+      "dry_density_avg_kg_m3": 1399.9999999999998,
+      "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+      "mean_total_stress_MPa": 1.2437152554932764,
+      "mean_swelling_stress_MPa": 1.2437152554932762,
+      "timestep": 110,
+      "time_s": 9504000.0,
+      "time_days": 110.0
+    },
+    {
+      "pressure_Pa_avg": 0.0,
+      "pressure_Pa_min": 0.0,
+      "pressure_Pa_max": 0.0,
+      "suction_MPa_avg": 0.0,
+      "saturation_avg": 0.9999999999999999,
+      "porosity_avg": 0.4964028776978417,
+      "porosity_spread": 0.0,
+      "dry_density_avg_kg_m3": 1399.9999999999998,
+      "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+      "mean_total_stress_MPa": 1.3415944263406152,
+      "mean_swelling_stress_MPa": 1.3415944263406145,
+      "timestep": 120,
+      "time_s": 10368000.0,
+      "time_days": 120.0
+    }
+  ],
+  "final": {
+    "pressure_Pa_avg": 0.0,
+    "pressure_Pa_min": 0.0,
+    "pressure_Pa_max": 0.0,
+    "suction_MPa_avg": 0.0,
+    "saturation_avg": 0.9999999999999999,
+    "porosity_avg": 0.4964028776978417,
+    "porosity_spread": 0.0,
+    "dry_density_avg_kg_m3": 1399.9999999999998,
+    "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+    "mean_total_stress_MPa": 1.3415944263406152,
+    "mean_swelling_stress_MPa": 1.3415944263406145,
+    "timestep": 120,
+    "time_s": 10368000.0,
+    "time_days": 120.0
+  },
+  "initial": {
+    "pressure_Pa_avg": -100000000.0,
+    "pressure_Pa_min": -100000000.0,
+    "pressure_Pa_max": -100000000.0,
+    "suction_MPa_avg": 100.0,
+    "saturation_avg": 0.3292027868296924,
+    "porosity_avg": 0.4964028776978417,
+    "porosity_spread": 0.0,
+    "dry_density_avg_kg_m3": 1399.9999999999998,
+    "dry_density_spread_kg_m3": 1.5916157281026244e-12,
+    "mean_total_stress_MPa": 0.0,
+    "mean_swelling_stress_MPa": 0.0,
+    "timestep": 0,
+    "time_s": 0.0,
+    "time_days": 0.0
+  }
+},
+ "dd1600": {
+   "rho_dry": 1600.0,
+   "specific_weight_kN_m3": 16.0,
+   "phi0": 0.4244604316546763,
+   "k0": 5.870260900123441e-21,
+   "history": [
+     {
+       "pressure_Pa_avg": -100000000.0,
+       "pressure_Pa_min": -100000000.0,
+       "pressure_Pa_max": -100000000.0,
+       "suction_MPa_avg": 100.0,
+       "saturation_avg": 0.3292027868296924,
+       "porosity_avg": 0.42446043165467623,
+       "porosity_spread": 0.0,
+       "dry_density_avg_kg_m3": 1599.9999999999995,
+       "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+       "mean_total_stress_MPa": 0.0,
+       "mean_swelling_stress_MPa": 0.0,
+       "timestep": 0,
+       "time_s": 0.0,
+       "time_days": 0.0,

```

```

+     "time_days": 0.0
+ },
+ {
+     "pressure_Pa_avg": -91666666.66666666,
+     "pressure_Pa_min": -91666666.66666666,
+     "pressure_Pa_max": -91666666.66666666,
+     "suction_MPa_avg": 91.66666666666666,
+     "saturation_avg": 0.3512090948778415,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+     "mean_total_stress_MPa": 0.044012616096298245,
+     "mean_swelling_stress_MPa": 0.044012616096298245,
+     "timestep": 10,
+     "time_s": 864000.0,
+     "time_days": 10.0
+ },
+ {
+     "pressure_Pa_avg": -83333333.33333334,
+     "pressure_Pa_min": -83333333.33333334,
+     "pressure_Pa_max": -83333333.33333334,
+     "suction_MPa_avg": 83.33333333333334,
+     "saturation_avg": 0.3765742784105493,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+     "mean_total_stress_MPa": 0.09474298316171385,
+     "mean_swelling_stress_MPa": 0.09474298316171381,
+     "timestep": 20,
+     "time_s": 1728000.0,
+     "time_days": 20.0
+ },
+ {
+     "pressure_Pa_avg": -75000000.0,
+     "pressure_Pa_min": -75000000.0,
+     "pressure_Pa_max": -75000000.0,
+     "suction_MPa_avg": 75.0,
+     "saturation_avg": 0.4061013891851352,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+     "mean_total_stress_MPa": 0.15379720471088573,
+     "mean_swelling_stress_MPa": 0.1537972047108857,
+     "timestep": 30,
+     "time_s": 2592000.0,
+     "time_days": 30.0
+ },
+ {
+     "pressure_Pa_avg": -66666666.66666667,
+     "pressure_Pa_min": -66666666.66666667,
+     "pressure_Pa_max": -66666666.66666667,
+     "suction_MPa_avg": 66.66666666666667,
+     "saturation_avg": 0.44084493308497974,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+     "mean_total_stress_MPa": 0.2232842925105747,
+     "mean_swelling_stress_MPa": 0.2232842925105746,
+     "timestep": 40,
+     "time_s": 3456000.0,
+     "time_days": 40.0
+ },
+ {
+     "pressure_Pa_avg": -58333333.33333333,
+     "pressure_Pa_min": -58333333.33333333,
+     "pressure_Pa_max": -58333333.33333333,
+     "suction_MPa_avg": 58.33333333333333,
+     "saturation_avg": 0.4821939243369694,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+     "mean_total_stress_MPa": 0.3059822750145542,
+     "mean_swelling_stress_MPa": 0.3059822750145542,
+     "timestep": 50,
+     "time_s": 4320000.0,
+     "time_days": 50.0
+ },
+ {
+     "pressure_Pa_avg": -50000000.0,
+     "pressure_Pa_min": -50000000.0,
+     "pressure_Pa_max": -50000000.0,
+     "suction_MPa_avg": 50.0,
+     "saturation_avg": 0.5319611182179516,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+     "mean_total_stress_MPa": 0.40551666277651843,
+     "mean_swelling_stress_MPa": 0.4055166627765184,
+     "timestep": 60,
+     "time_s": 5184000.0,
+     "time_days": 60.0
+ },
+ {
+     "pressure_Pa_avg": -41666666.666666664,
+     "pressure_Pa_min": -41666666.666666664,
+     "pressure_Pa_max": -41666666.666666664,
+     "suction_MPa_avg": 41.666666666666664,
+     "saturation_avg": 0.5924210264603837,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642428526593924e-12,
+     "mean_total_stress_MPa": 0.5264364792612226,
+     "mean_swelling_stress_MPa": 0.5264364792612225,
+     "timestep": 70,
+     "time_s": 6048000.0,
+     "time_days": 70.0
+ },
+ {
+     "pressure_Pa_avg": -33333333.333333336,
+     "pressure_Pa_min": -33333333.333333336,

```

```

+     "pressure_Pa_max": -33333333.33333336,
+     "suction_MPa_avg": 33.33333333333336,
+     "saturation_avg": 0.666092259906612,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642420526593924e-12,
+     "mean_total_stress_MPa": 0.6737789323219373,
+     "mean_swelling_stress_MPa": 0.6737789323219374,
+     "timestep": 80,
+     "time_s": 6912000.0,
+     "time_days": 80.0
+   },
+   {
+     "pressure_Pa_avg": -25000000.0,
+     "pressure_Pa_min": -25000000.0,
+     "pressure_Pa_max": -25000000.0,
+     "suction_MPa_avg": 25.0,
+     "saturation_avg": 0.7546262720252543,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642420526593924e-12,
+     "mean_total_stress_MPa": 0.8580469703911241,
+     "mean_swelling_stress_MPa": 0.8580469703911239,
+     "timestep": 90,
+     "time_s": 7776000.0,
+     "time_days": 90.0
+   },
+   {
+     "pressure_Pa_avg": -16666666.666666662,
+     "pressure_Pa_min": -16666666.666666662,
+     "pressure_Pa_max": -16666666.666666662,
+     "suction_MPa_avg": 16.666666666666666,
+     "saturation_avg": 0.8551168516181242,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642420526593924e-12,
+     "mean_total_stress_MPa": 1.051828129576864,
+     "mean_swelling_stress_MPa": 1.0518281295768634,
+     "timestep": 100,
+     "time_s": 8640000.0,
+     "time_days": 100.0
+   },
+   {
+     "pressure_Pa_avg": -8333333.333333337,
+     "pressure_Pa_min": -8333333.333333337,
+     "pressure_Pa_max": -8333333.333333337,
+     "suction_MPa_avg": 8.333333333333337,
+     "saturation_avg": 0.9510604145763306,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642420526593924e-12,
+     "mean_total_stress_MPa": 1.2437152554932764,
+     "mean_swelling_stress_MPa": 1.2437152554932762,
+     "timestep": 110,
+     "time_s": 9504000.0,
+     "time_days": 110.0
+   },
+   {
+     "pressure_Pa_avg": 0.0,
+     "pressure_Pa_min": 0.0,
+     "pressure_Pa_max": 0.0,
+     "suction_MPa_avg": 0.0,
+     "saturation_avg": 0.9999999999999999,
+     "porosity_avg": 0.42446043165467623,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1599.9999999999995,
+     "dry_density_spread_kg_m3": 1.3642420526593924e-12,
+     "mean_total_stress_MPa": 1.3415944263406152,
+     "mean_swelling_stress_MPa": 1.3415944263406145,
+     "timestep": 120,
+     "time_s": 10368000.0,
+     "time_days": 120.0
+   }
+ ],
+ "final": {
+   "pressure_Pa_avg": 0.0,
+   "pressure_Pa_min": 0.0,
+   "pressure_Pa_max": 0.0,
+   "suction_MPa_avg": 0.0,
+   "saturation_avg": 0.9999999999999999,
+   "porosity_avg": 0.42446043165467623,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1599.9999999999995,
+   "dry_density_spread_kg_m3": 1.3642420526593924e-12,
+   "mean_total_stress_MPa": 1.3415944263406152,
+   "mean_swelling_stress_MPa": 1.3415944263406145,
+   "timestep": 120,
+   "time_s": 10368000.0,
+   "time_days": 120.0
+ },
+ "initial": {
+   "pressure_Pa_avg": -100000000.0,
+   "pressure_Pa_min": -100000000.0,
+   "pressure_Pa_max": -100000000.0,
+   "suction_MPa_avg": 100.0,
+   "saturation_avg": 0.3292027868296924,
+   "porosity_avg": 0.42446043165467623,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1599.9999999999995,
+   "dry_density_spread_kg_m3": 1.3642420526593924e-12,
+   "mean_total_stress_MPa": 0.0,
+   "mean_swelling_stress_MPa": 0.0,
+   "timestep": 0,
+   "time_s": 0.0,
+   "time_days": 0.0
+ }
+ },
+ "dd1800": {
+   "rho_dry": 1800.0,
+   "specific_weight_kN_m3": 18.0,
+   "phi0": 0.3525179856115108,
+   "k0": 2.6569572325678573e-21,
+   "history": [

```

```

+
+ {
+   "pressure_Pa_avg": -100000000.0,
+   "pressure_Pa_min": -100000000.0,
+   "pressure_Pa_max": -100000000.0,
+   "suction_MP_a_avg": 100.0,
+   "saturation_avg": 0.3292027868296924,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.501184298755527e-12,
+   "mean_total_stress_MPa": 0.0,
+   "mean_swelling_stress_MPa": 0.0,
+   "timestep": 0,
+   "time_s": 0.0,
+   "time_days": 0.0
+ },
+
+ {
+   "pressure_Pa_avg": -91666666.66666666,
+   "pressure_Pa_min": -91666666.66666666,
+   "pressure_Pa_max": -91666666.66666666,
+   "suction_MP_a_avg": 91.66666666666666,
+   "saturation_avg": 0.3512090948778415,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.501184298755527e-12,
+   "mean_total_stress_MPa": 0.044012616096298245,
+   "mean_swelling_stress_MPa": 0.044012616096298245,
+   "timestep": 10,
+   "time_s": 864000.0,
+   "time_days": 10.0
+ },
+
+ {
+   "pressure_Pa_avg": -83333333.33333334,
+   "pressure_Pa_min": -83333333.33333334,
+   "pressure_Pa_max": -83333333.33333334,
+   "suction_MP_a_avg": 83.33333333333334,
+   "saturation_avg": 0.3765742784105493,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.501184298755527e-12,
+   "mean_total_stress_MPa": 0.09474298316171385,
+   "mean_swelling_stress_MPa": 0.09474298316171381,
+   "timestep": 20,
+   "time_s": 1728000.0,
+   "time_days": 20.0
+ },
+
+ {
+   "pressure_Pa_avg": -75000000.0,
+   "pressure_Pa_min": -75000000.0,
+   "pressure_Pa_max": -75000000.0,
+   "suction_MP_a_avg": 75.0,
+   "saturation_avg": 0.4061013891851352,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.501184298755527e-12,
+   "mean_total_stress_MPa": 0.15379720471088573,
+   "mean_swelling_stress_MPa": 0.1537972047108857,
+   "timestep": 30,
+   "time_s": 2592000.0,
+   "time_days": 30.0
+ },
+
+ {
+   "pressure_Pa_avg": -66666666.66666667,
+   "pressure_Pa_min": -66666666.66666667,
+   "pressure_Pa_max": -66666666.66666667,
+   "suction_MP_a_avg": 66.66666666666667,
+   "saturation_avg": 0.44084493308497974,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.501184298755527e-12,
+   "mean_total_stress_MPa": 0.2232842925105747,
+   "mean_swelling_stress_MPa": 0.2232842925105746,
+   "timestep": 40,
+   "time_s": 3456000.0,
+   "time_days": 40.0
+ },
+
+ {
+   "pressure_Pa_avg": -58333333.33333333,
+   "pressure_Pa_min": -58333333.33333333,
+   "pressure_Pa_max": -58333333.33333333,
+   "suction_MP_a_avg": 58.33333333333333,
+   "saturation_avg": 0.482193924369694,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.501184298755527e-12,
+   "mean_total_stress_MPa": 0.3059822750145542,
+   "mean_swelling_stress_MPa": 0.3059822750145542,
+   "timestep": 50,
+   "time_s": 4320000.0,
+   "time_days": 50.0
+ },
+
+ {
+   "pressure_Pa_avg": -50000000.0,
+   "pressure_Pa_min": -50000000.0,
+   "pressure_Pa_max": -50000000.0,
+   "suction_MP_a_avg": 50.0,
+   "saturation_avg": 0.531961182179516,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.501184298755527e-12,
+   "mean_total_stress_MPa": 0.40551666277651843,
+   "mean_swelling_stress_MPa": 0.4055166627765184,
+   "timestep": 60,
+   "time_s": 5184000.0,
+   "time_days": 60.0
+ },
+
+ {
+   "pressure_Pa_avg": -41666666.66666664,
+   "pressure_Pa_min": -41666666.66666664,
+   "pressure_Pa_max": -41666666.66666664,
+   "suction_MP_a_avg": 41.66666666666664,

```



```

+     "saturation_avg": 0.5924210264603037,
+     "porosity_avg": 0.35251798561151076,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1799.9999999999995,
+     "dry_density_spread_kg_m3": 2.5011184298755527e-12,
+     "mean_total_stress_MPa": 0.5264364792612226,
+     "mean_swelling_stress_MPa": 0.5264364792612225,
+     "timestep": 70,
+     "time_s": 6048000.0,
+     "time_days": 70.0
+   },
+   {
+     "pressure_Pa_avg": -33333333.333333336,
+     "pressure_Pa_min": -33333333.333333336,
+     "pressure_Pa_max": -33333333.333333336,
+     "suction_MPa_avg": 33.333333333333336,
+     "saturation_avg": 0.6660922529906612,
+     "porosity_avg": 0.35251798561151076,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1799.9999999999995,
+     "dry_density_spread_kg_m3": 2.5011184298755527e-12,
+     "mean_total_stress_MPa": 0.6737789323219373,
+     "mean_swelling_stress_MPa": 0.6737789323219374,
+     "timestep": 80,
+     "time_s": 6912000.0,
+     "time_days": 80.0
+   },
+   {
+     "pressure_Pa_avg": -25000000.0,
+     "pressure_Pa_min": -25000000.0,
+     "pressure_Pa_max": -25000000.0,
+     "suction_MPa_avg": 25.0,
+     "saturation_avg": 0.7546262720252543,
+     "porosity_avg": 0.35251798561151076,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1799.9999999999995,
+     "dry_density_spread_kg_m3": 2.5011184298755527e-12,
+     "mean_total_stress_MPa": 0.8508469703911241,
+     "mean_swelling_stress_MPa": 0.8508469703911239,
+     "timestep": 90,
+     "time_s": 7776000.0,
+     "time_days": 90.0
+   },
+   {
+     "pressure_Pa_avg": -16666666.666666662,
+     "pressure_Pa_min": -16666666.666666662,
+     "pressure_Pa_max": -16666666.666666662,
+     "suction_MPa_avg": 16.666666666666666,
+     "saturation_avg": 0.8551168516181242,
+     "porosity_avg": 0.35251798561151076,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1799.9999999999995,
+     "dry_density_spread_kg_m3": 2.5011184298755527e-12,
+     "mean_total_stress_MPa": 1.051828129576864,
+     "mean_swelling_stress_MPa": 1.0518281295768634,
+     "timestep": 100,
+     "time_s": 8640000.0,
+     "time_days": 100.0
+   },
+   {
+     "pressure_Pa_avg": -8333333.333333337,
+     "pressure_Pa_min": -8333333.333333337,
+     "pressure_Pa_max": -8333333.333333337,
+     "suction_MPa_avg": 8.333333333333337,
+     "saturation_avg": 0.9510604145763306,
+     "porosity_avg": 0.35251798561151076,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1799.9999999999995,
+     "dry_density_spread_kg_m3": 2.5011184298755527e-12,
+     "mean_total_stress_MPa": 1.2437152554932764,
+     "mean_swelling_stress_MPa": 1.2437152554932762,
+     "timestep": 110,
+     "time_s": 9504000.0,
+     "time_days": 110.0
+   },
+   {
+     "pressure_Pa_avg": 0.0,
+     "pressure_Pa_min": 0.0,
+     "pressure_Pa_max": 0.0,
+     "suction_MPa_avg": 0.0,
+     "saturation_avg": 0.9999999999999999,
+     "porosity_avg": 0.35251798561151076,
+     "porosity_spread": 0.0,
+     "dry_density_avg_kg_m3": 1799.9999999999995,
+     "dry_density_spread_kg_m3": 2.5011184298755527e-12,
+     "mean_total_stress_MPa": 1.3415944263406152,
+     "mean_swelling_stress_MPa": 1.3415944263406145,
+     "timestep": 120,
+     "time_s": 10368000.0,
+     "time_days": 120.0
+   }
+ ],
+ "final": {
+   "pressure_Pa_avg": 0.0,
+   "pressure_Pa_min": 0.0,
+   "pressure_Pa_max": 0.0,
+   "suction_MPa_avg": 0.0,
+   "saturation_avg": 0.9999999999999999,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,
+   "dry_density_spread_kg_m3": 2.5011184298755527e-12,
+   "mean_total_stress_MPa": 1.3415944263406152,
+   "mean_swelling_stress_MPa": 1.3415944263406145,
+   "timestep": 120,
+   "time_s": 10368000.0,
+   "time_days": 120.0
+ },
+ "initial": {
+   "pressure_Pa_avg": -100000000.0,
+   "pressure_Pa_min": -100000000.0,
+   "pressure_Pa_max": -100000000.0,
+   "suction_MPa_avg": 100.0,
+   "saturation_avg": 0.3292827868296924,
+   "porosity_avg": 0.35251798561151076,
+   "porosity_spread": 0.0,
+   "dry_density_avg_kg_m3": 1799.9999999999995,

```

```

+      "dry_density_spread_kg_m3": 2.501104298755527e-12,
+      "mean_total_stress_MPa": 0.0,
+      "mean_swelling_stress_MPa": 0.0,
+      "timestep": 0,
+      "time_s": 0.0,
+      "time_days": 0.0
+    }
+  }
+ }
+)

\ No newline at end of file
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_Model1/ms33_vs_ciemat_swelling_pressure_vs_dry_density.csv b/Tests/Data/RichardsMechanics/ANCHORS_MS33_Model1/ms33_vs_ciemat_swelling_pressure_vs_dry_density.csv
new file mode 100644
index 0000000000..67c177230f
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_Model1/ms33_vs_ciemat_swelling_pressure_vs_dry_density.csv
@@ -0,0 +1,108 @@
+series,kind,dry_density_g_cm3,dry_density_kg_m3,swelling_pressure_MPa,basis
+CIEMAT/Villar Eq(7),curve,1.350000,1350.000,1.071972061,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.355000,1355.000,1.108879449,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.360000,1360.000,1.147057537,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.365000,1365.000,1.186550075,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.370000,1370.000,1.227402319,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.375000,1375.000,1.269661082,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.380000,1380.000,1.313374790,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.385000,1385.000,1.358593536,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.390000,1390.000,1.405369138,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.395000,1395.000,1.453755197,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.400000,1400.000,1.5037807161,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.405000,1405.000,1.555582386,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.410000,1410.000,1.609140201,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.415000,1415.000,1.664541982,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.420000,1420.000,1.721851215,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.425000,1425.000,1.781133572,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.430000,1430.000,1.842456987,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.435000,1435.000,1.905891732,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.440000,1440.000,1.971510500,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.445000,1445.000,2.039388484,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.450000,1450.000,2.109603468,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.455000,1455.000,2.182235914,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.460000,1460.000,2.257369055,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.465000,1465.000,2.335088986,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.470000,1470.000,2.415484770,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.475000,1475.000,2.498648536,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.480000,1480.000,2.584675582,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.485000,1485.000,2.673664491,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.490000,1490.000,2.765717237,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.495000,1495.000,2.860939307,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.500000,1500.000,2.959439819,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.505000,1505.000,3.061331647,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.510000,1510.000,3.166731553,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.515000,1515.000,3.275760317,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.520000,1520.000,3.388542880,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.525000,1525.000,3.505208482,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.530000,1530.000,3.625890814,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.535000,1535.000,3.750728171,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.540000,1540.000,3.879863606,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.545000,1545.000,4.013445101,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.550000,1550.000,4.151625731,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.555000,1555.000,4.294563842,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.560000,1560.000,4.442423230,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.565000,1565.000,4.595373332,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.570000,1570.000,4.753589420,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.575000,1575.000,4.917252797,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.580000,1580.000,5.086551011,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.585000,1585.000,5.261678067,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.590000,1590.000,5.442834647,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.595000,1595.000,5.630228345,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.600000,1600.000,5.824073902,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.605000,1605.000,6.024593450,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.610000,1610.000,6.232016773,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.615000,1615.000,6.446581562,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.620000,1620.000,6.668533695,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.625000,1625.000,6.898127514,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.630000,1630.000,7.135626117,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.635000,1635.000,7.381301662,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.640000,1640.000,7.635435676,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.645000,1645.000,7.898319379,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.650000,1650.000,8.170254018,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.655000,1655.000,8.451551213,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.660000,1660.000,8.742533309,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.665000,1665.000,9.043533754,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.670000,1670.000,9.354897472,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.675000,1675.000,9.676981266,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.680000,1680.000,10.010154222,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.685000,1685.000,10.354798133,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.690000,1690.000,10.711307937,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.695000,1695.000,11.080092171,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.700000,1700.000,11.461573437,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.705000,1705.000,11.856188885,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.710000,1710.000,12.264390718,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.715000,1715.000,12.686646708,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.720000,1720.000,13.123440731,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.725000,1725.000,13.575273324,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.730000,1730.000,14.042662256,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.735000,1735.000,14.526143123,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.740000,1740.000,15.026269960,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.745000,1745.000,15.543615880,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.750000,1750.000,16.078773726,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.755000,1755.000,16.632356751,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.760000,1760.000,17.204999325,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.765000,1765.000,17.797357656,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.770000,1770.000,18.410180548,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.775000,1775.000,19.043960173,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.780000,1780.000,19.699632879,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.785000,1785.000,20.377880024,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.790000,1790.000,21.079478831,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.795000,1795.000,21.805233287,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.800000,1800.000,22.555975054,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.805000,1805.000,23.332564434,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.810000,1810.000,24.135891343,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.815000,1815.000,24.966876341,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.820000,1820.000,25.826471679,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.825000,1825.000,26.715662396,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.830000,1830.000,27.635467442,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)
+CIEMAT/Villar Eq(7),curve,1.835000,1835.000,28.586940853,Lloret et al. 2007 Eq.(7): Ps=exp(6.77*pd-9.07)

```

```

+CIEMAT/Villar Eq(7),curve,1.840000,1840.000,29.571172951,Lloret et al. 2007 Eq.(7): P=exp(6.77*qd-9.07)
+CIEMAT/Villar Eq(7),curve,1.845000,1845.000,30.589291600,Lloret et al. 2007 Eq.(7): P=exp(6.77*qd-9.07)
+CIEMAT/Villar Eq(7),curve,1.850000,1850.000,31.642463495,Lloret et al. 2007 Eq.(7): P=exp(6.77*qd-9.07)
+CIEMAT/Villar Eq(7),reference_point,1.400000,1400.000,1.503807161,reference densities 1.40/1.60/1.80 g/cm^3
+CIEMAT/Villar Eq(7),reference_point,1.600000,1600.000,5.824073902,reference densities 1.40/1.60/1.80 g/cm^3
+CIEMAT/Villar Eq(7),reference_point,1.800000,1800.000,22.555975054,reference densities 1.40/1.60/1.80 g/cm^3
+OGS MS33 reduced Model-I,simulation_point,1.400000,1400.000,1.341594426,final mean total stress at 120 d (constant-volume reduced model)
+OGS MS33 reduced Model-I,simulation_point,1.600000,1600.000,1.341594426,final mean total stress at 120 d (constant-volume reduced model)
+OGS MS33 reduced Model-I,simulation_point,1.800000,1800.000,1.341594426,final mean total stress at 120 d (constant-volume reduced model)
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_vs_ciemat_swelling_pressure_vs_dry_density.png b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_vs_ciemat_
swelling_pressure_vs_dry_density.png
new file mode 100644
index 0000000000..9a2162bce6
Binary files /dev/null and b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/ms33_vs_ciemat_swelling_pressure_vs_dry_density.png differ
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/plot_ms33_vs_ciemat_swelling_pressure_dd.py b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/plot_ms33_vs_ciemat_swe
lling_pressure_dd.py
new file mode 100644
index 0000000000..93837c94c9
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/plot_ms33_vs_ciemat_swelling_pressure_dd.py
@@ -0,0 +1,188 @@
#!/usr/bin/env python3
+"""Plot the MS33 dry-density sweep against the CIEMAT/Villar reference fit.
+
+The script reads the reduced Model-I history, overlays the Villar Eq. (7)
+curve, and writes both a CSV export and the comparison plot.
+"""
+
+from __future__ import annotations
+
+import csv
+import json
+import os
+import tempfile
+from pathlib import Path
+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+import matplotlib
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+import numpy as np
+
+ROOT = Path(__file__).resolve().parent
+SUMMARY_JSON = ROOT / "ms33_model_i_summary.json"
+OUT_CSV = ROOT / "ms33_vs_ciemat_swelling_pressure_vs_dry_density.csv"
+OUT_PNG = ROOT / "ms33_vs_ciemat_swelling_pressure_vs_dry_density.png"
+
+def ciemat_villar_eq7_ps_mpa(qd_g_cm3: np.ndarray) -> np.ndarray:
+    """Compute the Villar Eq. (7) reference curve in MPa."""
+    return np.exp(6.77 * qd_g_cm3 - 9.07)
+
+def load_ms33_points() -> list[tuple[float, float]]:
+    """Load dry-density and final-stress pairs from the reduced Model-I summary."""
+    payload = json.loads(SUMMARY_JSON.read_text())
+    rows = []
+    for case_data in payload["cases"].values():
+        rho_d_kg_m3 = float(case_data["rho_dry"])
+        qd_g_cm3 = rho_d_kg_m3 / 1000.0
+        # In the reduced Model-I setup this is the isotropic confined response proxy.
+        ps_mpa = float(case_data["final"]["mean_total_stress_MPa"])
+        rows.append((qd_g_cm3, ps_mpa))
+    return sorted(rows, key=lambda item: item[0])
+
+def write_overlay_csv(ms33_points: list[tuple[float, float]]) -> None:
+    """Export the reference curve and simulation points as a flat CSV table."""
+    qd_ref = np.array([1.40, 1.60, 1.80], dtype=float)
+    ps_ref = ciemat_villar_eq7_ps_mpa(qd_ref)
+
+    qd_curve = np.linspace(1.35, 1.85, 101)
+    ps_curve = ciemat_villar_eq7_ps_mpa(qd_curve)
+
+    with OUT_CSV.open("w", newline="") as f:
+        writer = csv.writer(f)
+        writer.writerow(
+            [
+                "series",
+                "kind",
+                "dry_density_g_cm3",
+                "dry_density_kg_m3",
+                "swelling_pressure_MPa",
+                "basis",
+            ]
+        )
+        for qd, ps in zip(qd_curve, ps_curve, strict=True):
+            writer.writerow(
+                [
+                    "CIEMAT/Villar Eq(7)",
+                    "curve",
+                    f"{qd:.6f}",
+                    f"{qd * 1000.0:.3f}",
+                    f"{ps:.9f}",
+                    "Lloret et al. 2007 Eq.(7): P=exp(6.77*qd-9.07)",
+                ]
+            )
+        for qd, ps in zip(qd_ref, ps_ref, strict=True):
+            writer.writerow(
+                [
+                    "CIEMAT/Villar Eq(7)",
+                    "reference_point",
+                    f"{qd:.6f}",
+                    f"{qd * 1000.0:.3f}",
+                    f"{ps:.9f}",
+                    "reference densities 1.40/1.60/1.80 g/cm^3",
+                ]
+            )
+        for qd, ps in ms33_points:
+            writer.writerow(
+                [
+                    "OGS MS33 reduced Model-I",
+                    "simulation_point",

```

```

+         f"{qd:.6f}",
+         f"{qd * 1000.0:.3f}",
+         f"{ps:.9f}",
+         "final mean total stress at 120 d (constant-volume reduced model)",
+     ]
+ )
+
+
+def plot_overlay(ms33_points: list[tuple[float, float]]) -> None:
+    """Render the dry-density comparison plot used in the note."""
+    qd_curve = np.linspace(1.35, 1.85, 101)
+    ps_curve = ciemat_villar_eq7_ps_mpa(qd_curve)
+    qd_ref = np.array([1.40, 1.60, 1.80], dtype=float)
+    ps_ref = ciemat_villar_eq7_ps_mpa(qd_ref)
+
+    x_ms33 = np.array([row[0] for row in ms33_points], dtype=float)
+    y_ms33 = np.array([row[1] for row in ms33_points], dtype=float)
+
+    fig, ax = plt.subplots(figsize=(7.6, 4.8))
+
+    ax.plot(
+        qd_curve,
+        ps_curve,
+        color="black",
+        linewidth=2.0,
+        label="CIEMAT/Villar Eq. (7) fit",
+    )
+    ax.scatter(
+        qd_ref,
+        ps_ref,
+        marker="o",
+        facecolors="white",
+        edgecolors="black",
+        s=44,
+        zorder=3,
+        label="CIEMAT reference dry densities",
+    )
+    ax.plot(
+        x_ms33,
+        y_ms33,
+        color="#d62728",
+        linewidth=1.8,
+        marker="s",
+        markersize=6,
+        label="OGS reduced MS33 sweep (120 d)",
+    )
+
+    for x, y in zip(x_ms33, y_ms33, strict=True):
+        ax.annotate(
+            f"{int(round(1000.0 * x))}",
+            (x, y),
+            textcoords="offset points",
+            xytext=(0, 7),
+            ha="center",
+            fontsize=8.5,
+            color="#d62728",
+        )
+
+    ax.set_title("Swelling Pressure vs Dry Density: CIEMAT/Villar vs OGS MS33")
+    ax.set_xlabel("Dry density,  $\rho_d$  (g/cm3)")
+    ax.set_ylabel("Swelling pressure,  $p_s$  (MPa)")
+    ax.set_xlim(1.35, 1.85)
+    ax.set_ylim(0.0, max(float(ps_ref.max()), float(y_ms33.max())) * 1.08)
+    ax.grid(True, alpha=0.3)
+    ax.legend(loc="upper left", frameon=True)
+
+    # Secondary x-axis in kg/m3 for direct use with project files.
+    secax = ax.secondary_xaxis("top", functions=(lambda x: 1000.0 * x, lambda x: x / 1000.0))
+    secax.set_xlabel("Dry density,  $\rho_d$  (kg/m3)")
+
+    fig.tight_layout()
+    fig.savefig(OUT_PNG, dpi=220)
+    plt.close(fig)
+
+
+def main() -> None:
+    """Load reduced results, refresh the overlay CSV, and write the figure."""
+    if not SUMMARY_JSON.exists():
+        raise FileNotFoundError(
+            f"Missing {SUMMARY_JSON.name}. Run summarize_ms33_model_i.py first."
+        )
+
+    ms33_points = load_ms33_points()
+    write_overlay_csv(ms33_points)
+    plot_overlay(ms33_points)
+
+    print(OUT_CSV)
+    print(OUT_PNG)
+
+
+if __name__ == "__main__":
+    main()
+
+diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_calibration.py b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_calibration.py
+py
+new file mode 100755
+index 0000000000..b19d6ff6e8
+--- /dev/null
++++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_calibration.py
+@@ -0,0 +1,891 @@
+#!/usr/bin/env python3
+"""Calibrate the MFront dense dry-density sweep against the Villar curve.
+
+The workflow runs one OGS case per dry density, solves for an effective vdW
+multiplier, and writes the calibration table, JSON summary, and plots.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import json
+import math
+import re
+import subprocess
+from dataclasses import dataclass
+from pathlib import Path
+

```

```

+import matplotlib
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+
+
+
+ROOT = Path(__file__).resolve().parent
+
+
+RHO_SOLID = 2780.0 # kg/m^3
+RHO_SOLID_MFRONT = 2780.0 # kg/m^3
+RHO_LR_REF = 1000.0 # kg/m^3
+
+
+MS33_K0_REF = 5.6e-21
+MS33_PHI_REF = 0.42
+MS33_PB = 27e6
+MS33_M = 0.45
+
+
+HAMAKER_LITERATURE = 5.1e-21 # J, montmorillonite-water-montmorillonite
+SPECIFIC_SURFACE_MASS = 523.0 # m^2/g
+
+
+TOTAL_SUCTION_MPA = 100.0
+MACRO_SUCTION_MPA = 1.0
+MICRO_SUCTION_MPA = TOTAL_SUCTION_MPA - MACRO_SUCTION_MPA
+
+
+DSM_MICROMACRO_SWELLING_SLOPE = 0.1
+MASS_EXCHANGE_COEFFICIENT = 1e-13
+LIQUID_VISCOSITY = 1e-3
+DSM_MICROMACRO_SATURATION_MODE = 1
+# 0 => coupled dsm_micromacro mass-storage branch in RichardsMechanicsDSMMicroMacroBridge_MCC
+# (aligns with native scalar_micro_macro_mass_storage_mode solve intent).
+DSM_MICROMACRO_LOCAL_SOLVE_MODE = 0
+MICRO_POTENTIAL_CONVENTION = 1
+
+
+PORE_AREA_SHAPE_FACTOR_TULLER = 0.8584073464102069
+AREA_FACTOR_TULLER = 1.0
+CHARACTERISTIC_PORE_SIZE = 1e-5
+SURFACE_TENSION = 0.0715
+
+
+DENSITY_A = 0.0
+DENSITY_B = 1.0
+RHO_L0 = 1e-6
+RHO_LR0 = 1000.000001
+
+
+PRESSURE_IC_PA = -MACRO_SUCTION_MPA * 1e6
+TIME_END_S = 120 * 86400
+
+
+
+@dataclass(frozen=True)
+class Case:
+    dry_density: float # kg/m^3
+
+    @property
+    def dry_density_g_cm3(self) -> float:
+        return self.dry_density / 1000.0
+
+    @property
+    def phi0(self) -> float:
+        return 1.0 - self.dry_density / RHO_SOLID
+
+    @property
+    def phi_micro_assumed(self) -> float:
+        return 0.99 * self.phi0
+
+    @property
+    def phi_macro_assumed(self) -> float:
+        return 0.01 * self.phi0
+
+    @property
+    def initial_volume_ratio(self) -> float:
+        return 1.0 / (1.0 - self.phi0)
+
+    @property
+    def intrinsic_permeability(self) -> float:
+        return (
+            MS33_K0_REF
+            * ((1.0 - MS33_PHI_REF) ** 2 / MS33_PHI_REF**3)
+            * (self.phi0**3 / (1.0 - self.phi0) ** 2)
+        )
+
+    @property
+    def specific_surface_volumetric(self) -> float:
+        # Convert m^2/g -> m^2/m^3 using dry density (kg/m^3 = 1000 g/L).
+        return SPECIFIC_SURFACE_MASS * self.dry_density * 1000.0
+
+    @property
+    def villar_target_swelling_mpa(self) -> float:
+        # Lloret/Villar (2007), Eq. (7): Ps [MPa], rho_d [g/cm^3].
+        return math.exp(6.77 * self.dry_density_g_cm3 - 9.07)
+
+def read_grid(path: Path) -> vtk.vtkUnstructuredGrid:
+    """Read a VTU snapshot into a VTK unstructured grid."""
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    return reader.GetOutput()
+
+def get_array(grid: vtk.vtkUnstructuredGrid, name: str):
+    arr = grid.GetPointData().GetArray(name)
+    if arr is None:
+        arr = grid.GetCellData().GetArray(name)
+    if arr is None:
+        raise KeyError(name)
+    return vtk_to_numpy(arr)
+
+def mean_total_stress_mpa(vtu_path: Path) -> float:
+    """Compute the mean isotropic stress in MPa from the final VTU."""
+    grid = read_grid(vtu_path)
+    sigma = get_array(grid, "sigma")
+    p_mean = float((-sigma[:, 0] - sigma[:, 1] - sigma[:, 2]).mean() / 3.0)
+    return p_mean / 1e6

```



```

+         <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+         <mass_lumping>true</mass_lumping>
+     </process>
+ </processes>
+ <media>
+     <medium>
+         <phases>
+             <phase>
+                 <type>AqueousLiquid</type>
+                 <properties>
+                     <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+                     <property><name>density</name><type>Constant</type><value>1e3</value></property>
+                 </properties>
+             </phase>
+             <phase>
+                 <type>Solid</type>
+                 <properties>
+                     <property><name>density</name><type>Constant</type><value>{RHO_SOLID:1.16g}</value></property>
+                 </properties>
+             </phase>
+         </phases>
+         <properties>
+             <property>
+                 <name>biot_coefficient</name>
+                 <type>Constant</type>
+                 <value>1.0</value>
+             </property>
+             <property>
+                 <name>permeability</name>
+                 <type>KozenyCarman</type>
+                 <initial_permeability>IntrinsicPermeability0</initial_permeability>
+                 <initial_porosity>phi0</initial_porosity>
+             </property>
+             <property>
+                 <name>porosity</name>
+                 <type>PorosityFromMassBalance</type>
+                 <initial_porosity>phi0</initial_porosity>
+                 <minimal_porosity>0</minimal_porosity>
+                 <maximal_porosity>1</maximal_porosity>
+             </property>
+             <property>
+                 <name>reference_temperature</name>
+                 <type>Constant</type>
+                 <value>293.15</value>
+             </property>
+             <property>
+                 <name>saturation</name>
+                 <type>SaturationTuller</type>
+                 <area_factor_tuller>{AREA_FACTOR_TULLER}</area_factor_tuller>
+                 <pore_area_shape_factor_tuller>{PORE_AREA_SHAPE_FACTOR_TULLER}</pore_area_shape_factor_tuller>
+                 <characteristic_pore_size>{CHARACTERISTIC_PORE_SIZE}</characteristic_pore_size>
+                 <surface_tension>{SURFACE_TENSION}</surface_tension>
+             </property>
+             <property>
+                 <name>relative_permeability</name>
+                 <type>Constant</type>
+                 <value>1</value>
+             </property>
+             <property>
+                 <name>bishops_effective_stress</name>
+                 <type>BishopsPowerLaw</type>
+                 <exponent>1</exponent>
+             </property>
+         </properties>
+     </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>basic_newton</nonlinear_solver>
+             <convergence_criterion>
+                 <type>PerComponentDeltaX</type>
+                 <norm_type>NORM2</norm_type>
+                 <reltols>1e-10 1e-10 1e-9</reltols>
+             </convergence_criterion>
+             <time_discretization><type>BackwardEuler</type></time_discretization>
+             <time_stepping>
+                 <type>FixedTimeStepping</type>
+                 <t_initial>0</t_initial>
+                 <t_end>{TIME_END_S}</t_end>
+                 <timesteps><pair><repeat>120</repeat><delta_t>86400</delta_t></pair></timesteps>
+             </time_stepping>
+         </process>
+     </processes>
+     <output>
+         <type>VTK</type>
+         <prefix>{prefix}</prefix>
+         <suffix>_ts_{timestep}_t_{time}</suffix>
+         <fixed_output_times>{TIME_END_S}</fixed_output_times>
+         <variables>
+             <variable>pressure</variable>
+             <variable>sigma</variable>
+             <variable>swelling_stress</variable>
+             <variable>saturation</variable>
+             <variable>porosity</variable>
+             <variable>dry_density_solid</variable>
+             <variable>n_l</variable>
+             <variable>phi_m</variable>
+             <variable>phi_Mc</variable>
+             <variable>mu_LR</variable>
+         </variables>
+     </output>
+ </time_loop>
+ <parameters>
+     <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+     <parameter><name>YoungModulus</name><type>Constant</type><value>52e6</value></parameter>
+     <parameter><name>PoissonRatio</name><type>Constant</type><value>0.3</value></parameter>
+     <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+     <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+     <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+     <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+     <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>{case.initial_volume_ratio:1.16g}</value></parameter>
+     <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+     <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+     <parameter><name>BubblePressure</name><type>Constant</type><value>{MS33_PB:1.16g}</value></parameter>
+     <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>{MS33_M:1.16g}</value></parameter>
+     <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>{DSM_MICROMACRO_SATURATION_MODE}</value></parameter>

```

```

+ <parameter><name>MicroStateLocalSolveMode</name><type>Constant</type><value>{DSM_MICROMACRO_LOCAL_SOLVE_MODE}</value></parameter>
+ <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>{MICRO_POTENTIAL_CONVENTION}</value></parameter>
+ <parameter><name>MicroSwellingStrainSlope</name><type>Constant</type><value>{DSM_MICROMACRO_SWELLING_SLOPE:.16g}</value></parameter>
+ <parameter><name>MassExchangeCoefficient</name><type>Constant</type><value>{MASS_EXCHANGE_COEFFICIENT:.16g}</value></parameter>
+ <parameter><name>MacroViscosity</name><type>Constant</type><value>{LIQUID_VISCOSITY:.16g}</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>{RHO_LR_REF:.16g}</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMicro</name><type>Constant</type><value>{RHO_L0:.16g}</value></parameter>
+ <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>{RHO_SOLID_MFRONT:.16g}</value></parameter>
+ <parameter><name>MicroLiquidDensityA</name><type>Constant</type><value>{DENSITY_A:.16g}</value></parameter>
+ <parameter><name>MicroLiquidDensityB</name><type>Constant</type><value>{DENSITY_B:.16g}</value></parameter>
+ <parameter><name>HamakerConstant</name><type>Constant</type><value>{hamaker_eff:.16g}</value></parameter>
+ <parameter><name>SpecificSurface</name><type>Constant</type><value>{SPECIFIC_SURFACE_MASS:.16g}</value></parameter>
+ <parameter><name>AreaFactorTuller</name><type>Constant</type><value>{AREA_FACTOR_TULLER:.16g}</value></parameter>
+ <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>{PORE_AREA_SHAPE_FACTOR_TULLER:.16g}</value></parameter>
+ <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>{CHARACTERISTIC_PORE_SIZE:.16g}</value></parameter>
+ <parameter><name>SurfaceTension</name><type>Constant</type><value>{SURFACE_TENSION:.16g}</value></parameter>
+ <parameter><name>phi0</name><type>Constant</type><value>{case.phi0:.16g}</value></parameter>
+ <parameter><name>IntrinsicPermeability0</name><type>Constant</type><value>{case.intrinsic_permeability:.16g}</value></parameter>
+ <parameter><name>n_l0</name><type>Constant</type><value>{n_l0_fixed:.16g}</value></parameter>
+ <parameter><name>rho_LR0</name><type>Constant</type><value>{RHO_LR0:.16g}</value></parameter>
+ <parameter><name>epsilon_sv0</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>displacement0</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>zero</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>pressure_ic</name><type>Constant</type><value>{PRESSURE_IC_PA:.16g}</value></parameter>
+ <parameter><name>pressure_bc_scale</name><type>Constant</type><value>1</value></parameter>
+ <parameter><name>pressure_bc</name><type>CurveScaled</type><curve>pressure_release</curve><parameter>pressure_bc_scale</parameter></parameter>
+ </parameters>
+ <process_variables>
+ <process_variable>
+ <name>displacement</name>
+ <components>2</components>
+ <order>1</order>
+ <initial_condition>displacement0</initial_condition>
+ <boundary_conditions>
+ <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+ <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+ <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+ <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ <process_variable>
+ <name>pressure</name>
+ <components>1</components>
+ <order>1</order>
+ <initial_condition>pressure_ic</initial_condition>
+ <boundary_conditions>
+ <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+ <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+ <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+ <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+ <nonlinear_solver><name>basic_newton</name><type>Newton</type><max_iter>60</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+ <linear_solver><name>general_linear_solver</name><eigen>solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ <curves>
+ <curve><name>pressure_release</name><coords>0 {TIME_END_S}</coords><values>{PRESSURE_IC_PA:.16g} 0</values></curve>
+ </curves>
+ </OpenGeoSysProject>
+ """
+ project_path.write_text(xml)
+ return {
+ "hamaker_effective": hamaker_eff,
+ "n_l0": n_l0_fixed,
+ }
+
+ def write_native_project(case: Case, project_path: Path) -> None:
+ """Create the equivalent native project for comparison runs."""
+ prefix = project_path.stem
+ xml = f'<?xml version="1.0" encoding="ISO-8859-1"?>
+ <OpenGeoSysProject>
+ <meshes>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_left.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_right.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_top.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_bottom.vtu</mesh>
+ </meshes>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <jacobian_assembler><type>Analytical</type></jacobian_assembler>
+ <constitutive_relation>
+ <type>LinearElasticIsotropic</type>
+ <youngs_modulus>YoungModulus</youngs_modulus>
+ <poissons_ratio>PoissonRatio</poissons_ratio>
+ </constitutive_relation>
+ <process_variables>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable name="sigma"/>
+ <secondary_variable name="swelling_stress"/>
+ <secondary_variable name="saturation"/>
+ <secondary_variable name="porosity"/>
+ <secondary_variable name="dry_density_solid"/>
+ </secondary_variables>
+ <specific_body_force>0.0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+ <medium>
+ <phases>
+ <phase>
+ <type>AqueousLiquid</type>
+ <properties>

```



```

+         <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+         <property><name>density</name><type>Constant</type><value>1e3</value></property>
+     </properties>
+ </phase>
+ <phase>
+     <type>Solid</type>
+     <properties>
+         <property><name>density</name><type>Constant</type><value>(RHO_SOLID:1.16g)</value></property>
+         <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>2e6 2e6 2e6</swelling_pressures>
+             <exponents>1 1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+         </property>
+     </properties>
+ </phase>
+ </phases>
+ <properties>
+     <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+     </property>
+     <property>
+         <name>permeability</name>
+         <type>KozenyCarman</type>
+         <initial_permeability>IntrinsicPermeability0</initial_permeability>
+         <initial_porosity>phi0</initial_porosity>
+     </property>
+     <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+     </property>
+     <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+     </property>
+     <property>
+         <name>saturation</name>
+         <type>SaturationTuller</type>
+         <area_factor_tuller>(AREA_FACTOR_TULLER)</area_factor_tuller>
+         <pore_area_shape_factor_tuller>(PORE_AREA_SHAPE_FACTOR_TULLER)</pore_area_shape_factor_tuller>
+         <characteristic_pore_size>(CHARACTERISTIC_PORE_SIZE)</characteristic_pore_size>
+         <surface_tension>(SURFACE_TENSION)</surface_tension>
+     </property>
+     <property>
+         <name>relative_permeability</name>
+         <type>Constant</type>
+         <value>1</value>
+     </property>
+     <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+     </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>basic_newton</nonlinear_solver>
+             <convergence_criterion><type>PerComponentDeltaX</type><norm_type>NORM2</norm_type><reltols>1e-10 1e-10 1e-9</reltols></convergence_criterion>
+             <time_discretization><type>BackwardEuler</type></time_discretization>
+             <time_stepping>
+                 <type>FixedTimeStepping</type>
+                 <t_initial>0</t_initial>
+                 <t_end>(TIME_END_S)</t_end>
+                 <timesteps><pair><repeat>120</repeat><delta_t>86400</delta_t></pair></timesteps>
+             </time_stepping>
+         </process>
+     </processes>
+     <output>
+         <type>VTK</type>
+         <prefix>[prefix]</prefix>
+         <suffix>_ts_{(timestep)}_t_{(time)}</suffix>
+         <fixed_output_times>(TIME_END_S)</fixed_output_times>
+         <variables>
+             <variable>pressure</variable>
+             <variable>sigma</variable>
+             <variable>swelling_stress</variable>
+             <variable>saturation</variable>
+             <variable>porosity</variable>
+             <variable>dry_density_solid</variable>
+         </variables>
+     </output>
+ </time_loop>
+ <parameters>
+     <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+     <parameter><name>YoungModulus</name><type>Constant</type><value>52e6</value></parameter>
+     <parameter><name>PoissonRatio</name><type>Constant</type><value>0.3</value></parameter>
+     <parameter><name>phi0</name><type>Constant</type><value>(case.phi0:1.16g)</value></parameter>
+     <parameter><name>IntrinsicPermeability0</name><type>Constant</type><value>(case.intrinsic_permeability:1.16g)</value></parameter>
+     <parameter><name>displacement0</name><type>Constant</type><value>0 0</values></parameter>
+     <parameter><name>zero</name><type>Constant</type><value>0.0</value></parameter>
+     <parameter><name>pressure_ic</name><type>Constant</type><value>(PRESSURE_IC_PA:1.16g)</value></parameter>
+     <parameter><name>pressure_bc_scale</name><type>Constant</type><value>1</value></parameter>
+     <parameter><name>pressure_bc</name><type>CurveScaled</type><curve>pressure_release</curve><parameter>pressure_bc_scale</parameter></parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>

```

```

+         </process_variable>
+     </process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+         <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+         <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+         <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+         <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+     </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver><name>basic_newton</name><type>Newton</type><max_iter>60</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ <curves>
+     <curve><name>pressure_release</name><coords>0 {TIME_END_S}</coords><values>{PRESSURE_IC_PA.16g} 0</values></curve>
+ </curves>
+ </OpenGeoSysProject>
+ """
+     project_path.write_text(xml)
+
+
+def run_ogs(ogs_bin: Path, project_path: Path) -> None:
+    """Execute OGS for a single temporary project."""
+    subprocess.run(
+        [str(ogs_bin), str(project_path)],
+        cwd=ROOT,
+        check=True,
+        stdout=subprocess.DEVNULL,
+        stderr=subprocess.STDOUT,
+    )
+
+
+def git_short_hash(repo: Path) -> str:
+    """Return a short Git hash for provenance tracking."""
+    try:
+        out = subprocess.check_output(
+            ["git", "rev-parse", "--short", "HEAD"], cwd=repo, text=True
+        ).strip()
+    except Exception:
+        return ""
+    return out
+
+
+def run_mfront_case(
+    ogs_bin: Path, case: Case, multiplier: float, n_l0_fixed: float, idx: int
+) -> dict:
+    """Run one MFront calibration trial and return the extracted metric."""
+    case_tag = f"dd{int(case.dry_density)}_mfront_{idx:02d}"
+    project_path = ROOT / f"{case_tag}.prj"
+    meta = write_mfront_project(case, multiplier, n_l0_fixed, project_path)
+
+    try:
+        run_ogs(ogs_bin, project_path)
+        last_vtu = extract_last_vtu(case_tag)
+        pressure_mpa = mean_total_stress_mpa(last_vtu)
+    finally:
+        cleanup_runtime(case_tag)
+        project_path.unlink(missing_ok=True)
+
+    return {
+        "case_tag": case_tag,
+        "multiplier": multiplier,
+        "pressure_mpa": pressure_mpa,
+        **meta,
+    }
+
+
+def run_native_case(ogs_bin: Path, case: Case) -> float:
+    """Run the native reference project and return the final swelling pressure."""
+    case_tag = f"dd{int(case.dry_density)}_native"
+    project_path = ROOT / f"{case_tag}.prj"
+    write_native_project(case, project_path)
+
+    try:
+        run_ogs(ogs_bin, project_path)
+        last_vtu = extract_last_vtu(case_tag)
+        pressure_mpa = mean_total_stress_mpa(last_vtu)
+    finally:
+        cleanup_runtime(case_tag)
+        project_path.unlink(missing_ok=True)
+
+    return pressure_mpa
+
+
+def calibrate_multiplier_for_case(
+    ogs_bin: Path,
+    case: Case,
+    target_mpa: float,
+    rel_tol: float = 0.02,
+    max_iter: int = 12,
+) -> dict:
+    """Solve for the multiplier that matches the Villar target."""
+    n_l0_fixed = n_l0_from_micro_suction(case.phi0, HAMAKER_LITERATURE)
+
+    # 1) Baseline at multiplier=1.
+    run0 = run_mfront_case(ogs_bin, case, 1.0, n_l0_fixed, 0)
+    p0 = run0["pressure_mpa"]
+
+    # 2) First scaled guess from near-linear response.
+    m1 = target_mpa / max(abs(p0), 1e-9)
+    m1 = float(np.clip(m1, 1e-6, 1e18))
+    run1 = run_mfront_case(ogs_bin, case, m1, n_l0_fixed, 1)
+    p1 = run1["pressure_mpa"]
+
+    best = min(
+        [run0, run1],
+        key=lambda r: abs(r["pressure_mpa"] - target_mpa) / max(target_mpa, 1e-12),
+    )
+
+    if abs(p1 - target_mpa) / max(target_mpa, 1e-12) < rel_tol:

```

```

+         return best
+
+     m_prev, p_prev = 1.0, p0
+     m_curr, p_curr = m1, p1
+     i = 2
+
+     while i <= max_iter:
+         if abs(p_curr - p_prev) > 1e-12:
+             m_new = m_curr + (target_mpa - p_curr) * (m_curr - m_prev) / (
+                 p_curr - p_prev
+             )
+         else:
+             m_new = m_curr * (target_mpa / max(abs(p_curr), 1e-9))
+             m_new = float(np.clip(m_new, 1e-6, 1e18))
+
+         run_new = run_mfront_case(ogs_bin, case, m_new, n_l0_fixed, i)
+         p_new = run_new["pressure_mpa"]
+         err_new = abs(p_new - target_mpa) / max(target_mpa, 1e-12)
+         if err_new < abs(best["pressure_mpa"] - target_mpa) / max(target_mpa, 1e-12):
+             best = run_new
+         if err_new < rel_tol:
+             return run_new
+
+         m_prev, p_prev = m_curr, p_curr
+         m_curr, p_curr = m_new, p_new
+         i += 1
+
+     return best
+
+
+def main() -> None:
+    """Run the full dense calibration sweep and write all summaries."""
+    parser = argparse.ArgumentParser(
+        description=(
+            "Dense dry-density calibration of effective micro vdw multiplier "
+            "against Lloret/Villar swelling-pressure curve."
+        )
+    )
+    parser.add_argument(
+        "--ogs-bin",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs"),
+        help="Path to OGS binary (must support MFront).",
+    )
+    parser.add_argument("--dd-min", type=float, default=1400.0)
+    parser.add_argument("--dd-max", type=float, default=1800.0)
+    parser.add_argument("--dd-step", type=float, default=25.0)
+    parser.add_argument("--rel-tol", type=float, default=0.02)
+    args = parser.parse_args()
+
+    dd_values = np.arange(args.dd_min, args.dd_max + 0.5 * args.dd_step, args.dd_step)
+    cases = [Case(float(dd)) for dd in dd_values]
+
+    rows = []
+    for case in cases:
+        target = case.villar_target_swelling_mpa
+        calibrated = calibrate_multiplier_for_case(
+            args.ogs_bin, case, target, rel_tol=args.rel_tol
+        )
+        native_pressure = run_native_case(args.ogs_bin, case)
+        print(
+            f"dd={case.dry_density:.0f} kg/m3: "
+            f"target={target:.3f} MPa, "
+            f"mfront={calibrated['pressure_mpa']:.3f} MPa, "
+            f"native={native_pressure:.3f} MPa, "
+            f"mult={calibrated['multiplier']:.3e}"
+        )
+
+        rows.append(
+            {
+                "dry_density_kg_m3": case.dry_density,
+                "dry_density_g_cm3": case.dry_density_g_cm3,
+                "phi0": case.phi0,
+                "phi_micro_assumed": case.phi_micro_assumed,
+                "phi_macro_assumed": case.phi_macro_assumed,
+                "k0_m2": case.intrinsic_permeability,
+                "specific_surface_mass_m2_g": SPECIFIC_SURFACE_MASS,
+                "specific_surface_volumetric_m2_m3": case.specific_surface_volumetric,
+                "target_villar_MPa": target,
+                "mfront_calibrated_MPa": calibrated["pressure_mpa"],
+                "native_MPa": native_pressure,
+                "vdw_multiplier": calibrated["multiplier"],
+                "hamaker_literature_J": HAMAKER_LITERATURE,
+                "hamaker_effective_J": calibrated["hamaker_effective"],
+                "n_l0": calibrated["n_l0"],
+                "macro_suction_MPa": MACRO_SUCTION_MPA,
+                "micro_suction_MPa": MICRO_SUCTION_MPA,
+                "mfront_minus_target_MPa": calibrated["pressure_mpa"] - target,
+                "native_minus_target_MPa": native_pressure - target,
+            }
+        )
+
+    rows = sorted(rows, key=lambda r: r["dry_density_kg_m3"])
+
+    csv_path = ROOT / "villar_dense_dd_calibration.csv"
+    with csv_path.open("w", newline="") as f:
+        writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+        writer.writeheader()
+        writer.writerows(rows)
+
+    summary = {
+        "ogs_repo_hash": git_short_hash(ROOT.parents[3]),
+        "materialmodels_repo_hash": git_short_hash(
+            Path("/Users/vinaykumar/Documents/GitHub/materialmodels")
+        ),
+        "hamaker_literature_J": HAMAKER_LITERATURE,
+        "specific_surface_mass_m2_g": SPECIFIC_SURFACE_MASS,
+        "total_initial_suction_MPa": TOTAL_SUCTION_MPA,
+        "macro_initial_suction_MPa": MACRO_SUCTION_MPA,
+        "micro_initial_suction_MPa": MICRO_SUCTION_MPA,
+        "notes": (
+            "Effective multiplier is applied as A_eff = multiplier * A_literature. "
+            "n_l0 is recomputed for each (density, multiplier) to preserve the "
+            "micro suction split in the initial state."
+        ),
+    },
+    "results": rows,
+}

```

```

+ (ROOT / "villar_dense_dd_calibration_summary.json").write_text(
+     json.dumps(summary, indent=2)
+ )
+
+ x = np.array([r["dry_density_kg_m3"] for r in rows])
+ y_target = np.array([r["target_villar_MPa"] for r in rows])
+ y_mfront = np.array([r["mfront_calibrated_MPa"] for r in rows])
+ y_mult = np.array([r["vdw_multiplier"] for r in rows])
+
+ plt.figure(figsize=(8.0, 5.2))
+ plt.plot(x, y_target, "k-", linewidth=2.0, label="Villar Eq. (7) target")
+ plt.plot(
+     x,
+     y_mfront,
+     color="#d62728",
+     marker="s",
+     linestyle="--",
+     linewidth=1.7,
+     markersize=4.5,
+     label="MFront calibrated (effective vdw)",
+ )
+ plt.xlabel("Dry density (kg/m³)")
+ plt.ylabel("Swelling pressure at full saturation (MPa)")
+ plt.grid(True, alpha=0.35)
+ plt.legend(frameon=False)
+ plt.tight_layout()
+ plt.savefig(ROOT / "villar_dense_dd_swelling_pressure_comparison.png", dpi=200)
+ plt.close()
+
+ plt.figure(figsize=(8.0, 5.2))
+ plt.semilogy(
+     x,
+     y_mult,
+     color="#2ca02c",
+     marker="D",
+     linestyle="--",
+     linewidth=1.8,
+     markersize=4.5,
+ )
+ plt.xlabel("Dry density (kg/m³)")
+ plt.ylabel("Effective micro vdw multiplier (-)")
+ plt.grid(True, which="both", alpha=0.35)
+ plt.tight_layout()
+ plt.savefig(ROOT / "villar_dense_dd_vdw_multiplier.png", dpi=200)
+ plt.close()
+
+ print(f"Wrote: {csv_path}")
+ print(f"Wrote: {ROOT / 'villar_dense_dd_calibration_summary.json'}")
+ print(f"Wrote: {ROOT / 'villar_dense_dd_swelling_pressure_comparison.png'}")
+ print(f"Wrote: {ROOT / 'villar_dense_dd_vdw_multiplier.png'}")
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_native_dsm_micromacro_branch.py b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dens
e_dd_native_dsm_micromacro_branch.py
new file mode 100644
index 0000000000..fb08449d25
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_native_dsm_micromacro_branch.py
@@ -0,0 +1,610 @@
+#!/usr/bin/env python3
+"""Run the native dsm_micromacro-style dense dry-density sweep.
+
+
+The script builds per-density projects, runs OGS, and records the calibrated
+native-branch response along with a comparison curve.
+"""
+
+from __future__ import annotations
+
+
+import argparse
+import csv
+import json
+import math
+import os
+import re
+import subprocess
+import tempfile
+
+from dataclasses import dataclass
+from pathlib import Path
+
+
+import matplotlib
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+
+
+ROOT = Path(__file__).resolve().parent
+RICHARDS_DATA_ROOT = ROOT.parent
+
+
+RHO_SOLID = 2780.0 # kg/m³
+RHO_SOLID_REFERENCE = 2780.0 # kg/m³ (aligned with MFront bridge run)
+RHO_LR_REF = 1000.0 # kg/m³
+
+
+MS33_K0_REF = 5.6e-21
+MS33_PHI_REF = 0.42
+
+
+HAMAKER_LITERATURE = 5.1e-21 # J
+SPECIFIC_SURFACE_MASS = 523.0 # m²/g
+
+
+TOTAL_SUCTION_MPA = 100.0
+MACRO_SUCTION_MPA = 1.0
+MICRO_SUCTION_MPA = TOTAL_SUCTION_MPA - MACRO_SUCTION_MPA
+
+
+MASS_EXCHANGE_COEFFICIENT = 1e-13
+MICRO_SWELLING_SLOPE = 0.1
+PRESSURE_IC_PA = -MACRO_SUCTION_MPA * 1e6
+TIME_END_S = 120 * 86400
+PRESSURE_TOLERANCE_PA = 1e-12
+
+
+AREA_FACTOR_TULLER = 1.0

```

```

+PORE_AREA_SHAPE_FACTOR_TULLER = 0.8584073464102069
+CHARACTERISTIC_PORE_SIZE = 1e-5
+SURFACE_TENSION = 0.0715
+MICRO_LIQUID_DENSITY_REFERENCE = 1e-6
+MICRO_LIQUID_DENSITY_A = 1e-16
+MICRO_LIQUID_DENSITY_B = 1.0
+
+NATIVE_DSM_MICROMACRO_SOURCE = Path("/Users/vinaykumar/git/ogs-native-dsm-transition")
+DEFAULT_NATIVE_OGS = Path("/Users/vinaykumar/git/build/release-native-beacon/bin/ogs")
+DEFAULT_MFRONT_CALIBRATION_CSV = ROOT / "villar_dense_dd_calibration.csv"
+
+
+@dataclass(frozen=True)
+class Case:
+    dry_density: float
+
+    @property
+    def dry_density_g_cm3(self) -> float:
+        return self.dry_density / 1000.0
+
+    @property
+    def phi0(self) -> float:
+        return 1.0 - self.dry_density / RHO_SOLID
+
+    @property
+    def phi_micro_assumed(self) -> float:
+        return 0.99 * self.phi0
+
+    @property
+    def phi_macro_assumed(self) -> float:
+        return 0.01 * self.phi0
+
+    @property
+    def intrinsic_permeability(self) -> float:
+        return (
+            MS33_K0_REF
+            * ((1.0 - MS33_PHI_REF) ** 2 / MS33_PHI_REF**3)
+            * (self.phi0**3 / (1.0 - self.phi0) ** 2)
+        )
+
+    @property
+    def initial_volume_ratio(self) -> float:
+        return 1.0 / (1.0 - self.phi0)
+
+    @property
+    def villar_target_swelling_mpa(self) -> float:
+        return math.exp(6.77 * self.dry_density_g_cm3 - 9.07)
+
+
+def git_short_hash(repo: Path) -> str:
+    """Return a short Git hash for provenance tracking."""
+    try:
+        return (
+            subprocess.check_output(
+                ["git", "rev-parse", "--short", "HEAD"], cwd=repo, text=True
+            )
+            .strip()
+        )
+    except Exception:
+        return ""
+
+
+def read_grid(path: Path) -> vtk.vtkUnstructuredGrid:
+    """Read a VTU snapshot into a VTK unstructured grid."""
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    return reader.GetOutput()
+
+
+def get_array(grid: vtk.vtkUnstructuredGrid, name: str):
+    """Fetch a field from point or cell data."""
+    arr = grid.GetPointData().GetArray(name)
+    if arr is None:
+        arr = grid.GetCellData().GetArray(name)
+    if arr is None:
+        raise KeyError(name)
+    return vtk_to_numpy(arr)
+
+
+def mean_total_stress_mpa(vtu_path: Path) -> float:
+    """Compute the mean isotropic stress in MPa from the final VTU."""
+    grid = read_grid(vtu_path)
+    sigma = get_array(grid, "sigma")
+    p_mean = float((-sigma[:, 0] - sigma[:, 1] - sigma[:, 2]).mean() / 3.0)
+    return p_mean / 1e6
+
+
+def extract_last_vtu(prefix: str) -> Path:
+    """Select the final VTU produced by a case run."""
+    candidates = sorted(ROOT.glob(f"{prefix}_ts_*_t*.vtu"))
+    if not candidates:
+        raise FileNotFoundError(f"No VTU outputs found for prefix {prefix}")
+
+    def key(path: Path):
+        stem = path.stem
+        match = re.search(r"ts_(\d+)_t_([0-9E+.]*)$", stem)
+        if not match:
+            return (-1, -1.0)
+        return (int(match.group(1)), float(match.group(2)))
+
+    return sorted(candidates, key=key)[-1]
+
+
+def cleanup_runtime(prefix: str, project_path: Path) -> None:
+    """Remove transient outputs from a calibration trial."""
+    for pattern in (f"{prefix}.pvd", f"{prefix}_ts_*_t*.vtu"):
+        for p in ROOT.glob(pattern):
+            p.unlink(missing_ok=True)
+    project_path.unlink(missing_ok=True)
+
+
+def n_l0_from_micro_suction(phi0: float, hamaker_eff: float) -> float:
+    """Derive the initial micro-scale water content from the suction split."""
+    # Target micro suction potential: mu = p / rho.
+    mu_abs = MICRO_SUCTION_MPA * 1e6 / RHO_LR_REF
+    n_s = 1.0 - phi0

```

```

+ prefactor = (
+     abs(hamaker_eff)
+     * (SPECIFIC_SURFACE_MASS * n_s * RHO_SOLID_REFERENCE) ** 3
+     / (6.0 * math.pi)
+ )
+ n_l0 = (prefactor / mu_abs) ** (1.0 / 3.0)
+ return max(1e-12, n_l0)
+
+
+def write_native_dsm_micromacro_project(case: Case, project_path: Path) -> dict:
+    """Create a temporary native dsm_micromacro project for one density case."""
+    prefix = project_path.stem
+    hamaker_eff = HAMAKER_LITERATURE
+    n_l0 = n_l0_from_micro_suction(case.phi0, hamaker_eff)
+    n_s = 1.0 - case.phi0
+
+    xml = f'<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+
+  <meshes>
+    <mesh axially_symmetric="true">../square_1x1_quad_1e0.vtu</mesh>
+    <mesh axially_symmetric="true">../square_1x1_quad_1e0_left.vtu</mesh>
+    <mesh axially_symmetric="true">../square_1x1_quad_1e0_right.vtu</mesh>
+    <mesh axially_symmetric="true">../square_1x1_quad_1e0_top.vtu</mesh>
+    <mesh axially_symmetric="true">../square_1x1_quad_1e0_bottom.vtu</mesh>
+  </meshes>
+
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <jacobian_assembler><type>Analytical</type></jacobian_assembler>
+      <micro_porosity>
+        <mass_exchange_coefficient>{MASS_EXCHANGE_COEFFICIENT: .16g}</mass_exchange_coefficient>
+        <nonlinear_solver>
+          <maximum_iterations>100</maximum_iterations>
+          <residuum_tolerance>1e-8</residuum_tolerance>
+          <increment_tolerance>1e-20</increment_tolerance>
+        </nonlinear_solver>
+      </micro_porosity>
+      <potential_exchange>
+        <enabled>true</enabled>
+        <mode>full_potential</mode>
+        <pressure_tolerance>{PRESSURE_TOLERANCE_PA: .16g}</pressure_tolerance>
+        <hamaker_constant>{hamaker_eff: .16g}</hamaker_constant>
+        <specific_surface>{SPECIFIC_SURFACE_MASS: .16g}</specific_surface>
+        <micro_solid_density_reference>{RHO_SOLID_REFERENCE: .16g}</micro_solid_density_reference>
+        <micro_solid_volume_fraction_reference>{n_s: .16g}</micro_solid_volume_fraction_reference>
+        <micro_liquid_density_reference>{MICRO_LIQUID_DENSITY_REFERENCE: .16g}</micro_liquid_density_reference>
+        <micro_liquid_density_a>{MICRO_LIQUID_DENSITY_A: .16g}</micro_liquid_density_a>
+        <micro_liquid_density_b>{MICRO_LIQUID_DENSITY_B: .16g}</micro_liquid_density_b>
+        <initial_micro_water_content>{n_l0: .16g}</initial_micro_water_content>
+        <local_nonlinear_solve_mode>scalar_micro_macro_mass_storage_mode</local_nonlinear_solve_mode>
+        <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+        <micro_potential_convention>negative_attractive</micro_potential_convention>
+        <micro_water_content_swelling_slope>{MICRO_SWELLING_SLOPE: .16g}</micro_water_content_swelling_slope>
+      </potential_exchange>
+      <constitutive_relation>
+        <type>MFront</type>
+        <behaviour>ModCamClay_semiExpl_const</behaviour>
+        <material_properties>
+          <material_property name="YoungModulus" parameter="YoungModulus"/>
+          <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+          <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+          <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+          <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+          <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+        </material_properties>
+        <initial_values>
+          <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+          <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+        </initial_values>
+      </constitutive_relation>
+      <process_variables>
+        <pressure>pressure</pressure>
+        <displacement>displacement</displacement>
+      </process_variables>
+      <secondary_variables>
+        <secondary_variable name="sigma"/>
+        <secondary_variable name="swelling_stress"/>
+        <secondary_variable name="saturation"/>
+        <secondary_variable name="porosity"/>
+        <secondary_variable name="dry_density_solid"/>
+        <secondary_variable name="micro_pressure"/>
+        <secondary_variable name="micro_saturation"/>
+        <secondary_variable name="micro_water_content"/>
+        <secondary_variable name="micro_porosity"/>
+        <secondary_variable name="micro_exchange_source"/>
+      </secondary_variables>
+      <specific_body_force>0 0</specific_body_force>
+      <initial_stress>sigma0</initial_stress>
+      <mass_lumping>true</mass_lumping>
+    </process>
+  </processes>
+
+  <media>
+    <medium>
+      <phases>
+        <phase>
+          <type>AqueousLiquid</type>
+          <properties>
+            <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+            <property><name>density</name><type>Constant</type><value>1e3</value></property>
+          </properties>
+        </phase>
+        <phase>
+          <type>Solid</type>
+          <properties>
+            <property><name>density</name><type>Constant</type><value>{RHO_SOLID: .16g}</value></property>
+            <property>
+              <name>swelling_stress_rate</name>
+              <type>SaturationDependentSwelling</type>
+              <swelling_pressures>0 0</swelling_pressures>
+              <exponents>1 1</exponents>
+              <lower_saturation_limit>0</lower_saturation_limit>
+              <upper_saturation_limit>1</upper_saturation_limit>
+            </property>
+          </properties>
+        </phase>
+      </phases>
+    </medium>
+  </media>
+</OpenGeoSysProject>

```

```

+
+ </phases>
+ <properties>
+   <property>
+     <name>biot_coefficient</name>
+     <type>Constant</type>
+     <value>1.0</value>
+   </property>
+   <property>
+     <name>permeability</name>
+     <type>KozenyCarman</type>
+     <initial_permeability>IntrinsicPermeability0</initial_permeability>
+     <initial_porosity>phi0</initial_porosity>
+   </property>
+   <property>
+     <name>porosity</name>
+     <type>PorosityFromMassBalance</type>
+     <initial_porosity>phi0</initial_porosity>
+     <minimal_porosity>0</minimal_porosity>
+     <maximal_porosity>1</maximal_porosity>
+   </property>
+   <property>
+     <name>reference_temperature</name>
+     <type>Constant</type>
+     <value>293.15</value>
+   </property>
+   <property>
+     <name>relative_permeability</name>
+     <type>Constant</type>
+     <value>1</value>
+   </property>
+   <property>
+     <name>saturation</name>
+     <type>SaturationTuller</type>
+     <area_factor_tuller>(AREA_FACTOR_TULLER:.16g)</area_factor_tuller>
+     <pore_area_shapefactor_tuller>(PORE_AREA_SHAPE_FACTOR_TULLER:.16g)</pore_area_shapefactor_tuller>
+     <characteristic_pore_size>(CHARACTERISTIC_PORE_SIZE:.16g)</characteristic_pore_size>
+     <surface_tension>(SURFACE_TENSION:.16g)</surface_tension>
+     <residual_liquid_saturation>0</residual_liquid_saturation>
+     <residual_gas_saturation>0</residual_gas_saturation>
+   </property>
+   <property>
+     <name>bishops_effective_stress</name>
+     <type>BishopsPowerLaw</type>
+     <exponent>1</exponent>
+   </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <abstols>5e-8 1e-13 1e-13</abstols>
+       </convergence_criterion>
+       <time_discretization><type>BackwardEuler</type></time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>(TIME_END_S)</t_end>
+         <timesteps><pair><repeat>120</repeat><delta_t>86400</delta_t></pair></timesteps>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>[prefix]</prefix>
+     <suffix>_ts_{(timestep)}_t_{(time)}</suffix>
+     <fixed_output_times>(TIME_END_S)</fixed_output_times>
+     <variables>
+       <variable>pressure</variable>
+       <variable>sigma</variable>
+       <variable>swelling_stress</variable>
+       <variable>saturation</variable>
+       <variable>porosity</variable>
+       <variable>dry_density_solid</variable>
+       <variable>micro_pressure</variable>
+       <variable>micro_saturation</variable>
+       <variable>micro_water_content</variable>
+       <variable>micro_porosity</variable>
+       <variable>micro_exchange_source</variable>
+     </variables>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>YoungModulus</name><type>Constant</type><value>526</value></parameter>
+   <parameter><name>PoissonRatio</name><type>Constant</type><value>0.3</value></parameter>
+   <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+   <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+   <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+   <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+   <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>(case.initial_volume_ratio:.16g)</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>(case.phi0:.16g)</value></parameter>
+   <parameter><name>IntrinsicPermeability0</name><type>Constant</type><value>(case.intrinsic_permeability:.16g)</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><value>0 0</value></parameter>
+   <parameter><name>zero</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><value>(PRESSURE_IC_PA:.16g)</value></parameter>
+   <parameter><name>pressure_bc_scale</name><type>Constant</type><value>1</value></parameter>
+   <parameter><name>pressure_bc</name><type>CurveScaled</type><curve>pressure_release</curve><parameter>pressure_bc_scale</parameter></parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+       <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+       <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+       <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variable>

```

```

+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver><name>basic_newton</name><type>Newton</type><max_iter>60</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ <curves>
+     <curve><name>pressure_release</name><coords>0 (TIME_END_S)</coords><values>(PRESSURE_IC_PA.16g) 0</values></curve>
+ </curves>
+ </OpenGeoSysProject>
+ """
+     project_path.write_text(xml)
+     return {
+         "hamaker_effective_3": hamaker_eff,
+         "n_l0": n_l0,
+         "micro_solid_volume_fraction_reference": n_s,
+     }
+
+
+def run_ogs(ogs_bin: Path, project_path: Path) -> None:
+    """Execute OGS for a single temporary project."""
+    subprocess.run(
+        [str(ogs_bin), str(project_path)],
+        cwd=ROOT,
+        check=True,
+        stdout=subprocess.DEVNULL,
+        stderr=subprocess.STDOUT,
+    )
+
+
+def run_native_dsm_micromacro_case(ogs_bin: Path, case: Case) -> dict:
+    case_tag = f"dd{int(case.dry_density)}_native_dsm_micromacro"
+    project_path = ROOT / f"{case_tag}.prj"
+    meta = write_native_dsm_micromacro_project(case, project_path)
+    try:
+        run_ogs(ogs_bin, project_path)
+        last_vtu = extract_last_vtu(case_tag)
+        pressure_mpa = mean_total_stress_mpa(last_vtu)
+    finally:
+        cleanup_runtime(case_tag, project_path)
+    return {"pressure_mpa": pressure_mpa, **meta}
+
+
+def load_mfront_calibrated_curve(path: Path) -> tuple[np.ndarray, np.ndarray] | None:
+    if not path.exists():
+        return None
+    x = []
+    y = []
+    with path.open(newline="") as f:
+        reader = csv.DictReader(f)
+        for row in reader:
+            x.append(float(row["dry_density_kg_m3"]))
+            y.append(float(row["mfront_calibrated_MPa"]))
+    if not x:
+        return None
+    order = np.argsort(np.asarray(x))
+    return np.asarray(x)[order], np.asarray(y)[order]
+
+
+def main() -> None:
+    """Run the dense sweep and write the CSV, JSON, and comparison plots."""
+    parser = argparse.ArgumentParser(
+        description=(
+            "Villar-style dry-density sweep for the native_dsm_micromacro branch "
+            "(micro-enabled native implementation).",
+        )
+    )
+    parser.add_argument("--native-ogs", type=Path, default=DEFAULT_NATIVE_OGS)
+    parser.add_argument(
+        "--native-source",
+        type=Path,
+        default=NATIVE_DSM_MICROMACRO_SOURCE,
+        help="Native OGS source tree for commit-hash provenance in summary JSON.",
+    )
+    parser.add_argument("--dd-min", type=float, default=1400.0)
+    parser.add_argument("--dd-max", type=float, default=1800.0)
+    parser.add_argument("--dd-step", type=float, default=25.0)
+    parser.add_argument(
+        "--mfront-calibration-csv",
+        type=Path,
+        default=DEFAULT_MFRONT_CALIBRATION_CSV,
+        help="Optional MFront calibration curve for overlay.",
+    )
+    args = parser.parse_args()
+
+    dd_values = np.arange(args.dd_min, args.dd_max + 0.5 * args.dd_step, args.dd_step)
+    cases = [Case(float(dd)) for dd in dd_values]
+
+    rows = []
+    for case in cases:
+        result = run_native_dsm_micromacro_case(args.native_ogs, case)
+        target = case.villar_target_swelling_mpa
+        native_p = result["pressure_mpa"]
+        print(
+            f"dd={case.dry_density:.0f} kg/m3: "
+            f"Villars={target:.3f} MPa, "
+            f"native-dsm_micromacro={native_p:.3f} MPa"
+        )
+        rows.append(
+            {
+                "dry_density_kg_m3": case.dry_density,
+                "dry_density_g_cm3": case.dry_density_g_cm3,
+                "phi0": case.phi0,
+                "phi_micro_assumed": case.phi_micro_assumed,
+                "phi_macro_assumed": case.phi_macro_assumed,
+            }
+        )

```



```

+         "k0_m2": case.intrinsic_permeability,
+         "hamaker_literature_3": HAMAKER_LITERATURE,
+         "hamaker_effective_3": result["hamaker_effective_3"],
+         "specific_surface_mass_m2_g": SPECIFIC_SURFACE_MASS,
+         "n_l0": result["n_l0"],
+         "micro_solid_volume_fraction_reference": result[
+             "micro_solid_volume_fraction_reference"
+         ],
+         "target_villar_MPa": target,
+         "native_dsm_micromacro_branch_MPa": native_p,
+         "native_minus_target_MPa": native_p - target,
+     }
+ )
+
+ rows = sorted(rows, key=lambda r: r["dry_density_kg_m3"])
+
+ out_csv = ROOT / "villar_dense_dd_native_dsm_micromacro_branch.csv"
+ with out_csv.open("w", newline="") as f:
+     writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+     writer.writeheader()
+     writer.writerows(rows)
+
+ summary = {
+     "ogs_repo_hash": git_short_hash(RICHARDS_DATA_ROOT.parents[2]),
+     "native_dsm_micromacro_source_hash": git_short_hash(args.native_source),
+     "native_dsm_micromacro_source_path": str(args.native_source),
+     "native_ogs_version": subprocess.check_output(
+         [str(args.native_ogs), "--version"], text=True
+     ),
+     "total_initial_suction_MPa": TOTAL_SUCTION_MPA,
+     "macro_initial_suction_MPa": MACRO_SUCTION_MPA,
+     "micro_initial_suction_MPa": MICRO_SUCTION_MPA,
+     "hamaker_literature_3": HAMAKER_LITERATURE,
+     "specific_surface_mass_m2_g": SPECIFIC_SURFACE_MASS,
+     "results": rows,
+ }
+
+ out_summary = ROOT / "villar_dense_dd_native_dsm_micromacro_branch_summary.json"
+ out_summary.write_text(json.dumps(summary, indent=2))
+
+ x = np.array([r["dry_density_kg_m3"] for r in rows])
+ y_target = np.array([r["target_villar_MPa"] for r in rows])
+ y_native = np.array([r["native_dsm_micromacro_branch_MPa"] for r in rows])
+
+ plt.figure(figsize=(8.0, 5.2))
+ plt.plot(x, y_target, "k-", linewidth=2.0, label="Villar Eq. (7) target")
+ plt.plot(
+     x,
+     y_native,
+     color="#d62728",
+     marker="s",
+     linestyle=":",
+     linewidth=1.6,
+     markersize=3.8,
+     label="Native dsm_micromacro branch",
+ )
+
+ mfront_curve = load_mfront_calibrated_curve(args.mfront_calibration_csv)
+ if mfront_curve is not None:
+     x_m, y_m = mfront_curve
+     plt.plot(
+         x_m,
+         y_m,
+         color="black",
+         marker="o",
+         linestyle="-",
+         linewidth=1.4,
+         markersize=3.4,
+         label="Villar fit reference",
+     )
+
+ plt.xlabel("Dry density (kg/m³)")
+ plt.ylabel("Swelling pressure at full saturation (MPa)")
+ plt.grid(True, alpha=0.35)
+ plt.legend(frameon=False)
+ plt.tight_layout()
+ out_png = ROOT / "villar_dense_dd_native_dsm_micromacro_branch_vs_villar.png"
+ plt.savefig(out_png, dpi=220)
+ plt.close()
+
+ print(f"Wrote: {out_csv}")
+ print(f"Wrote: {out_summary}")
+ print(f"Wrote: {out_png}")
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_native_dsm_micromacro_calibration.py b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_native_dsm_micromacro_calibration.py
new file mode 100644
index 0000000000..d4bce5da21
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_native_dsm_micromacro_calibration.py
@@ -0,0 +1,741 @@
+#!/usr/bin/env python3
+
+"""Calibrate the native dsm_micromacro dense dry-density sweep against Villar data.
+
+
+This workflow mirrors the MFront calibration logic so the two implementations
+can be compared with the same dry-density sampling and output schema.
+
+"""
+
+from __future__ import annotations
+
+
+import argparse
+import csv
+import json
+import math
+import os
+import re
+import subprocess
+import tempfile
+from dataclasses import dataclass
+from pathlib import Path
+
+
+import matplotlib
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy

```

```

+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+
+
+
+ROOT = Path(__file__).resolve().parent
+RICHARDS_DATA_ROOT = ROOT.parent
+
+
+RHO_SOLID = 2780.0 # kg/m^3
+RHO_SOLID_REFERENCE = 2780.0 # kg/m^3 (aligned with Mfront bridge run)
+RHO_LR_REF = 1000.0 # kg/m^3
+
+
+MS33_K0_REF = 5.6e-21
+MS33_PHI_REF = 0.42
+
+
+HAMAKER_LITERATURE = 5.1e-21 # J
+SPECIFIC_SURFACE_MASS = 523.0 # m^2/g
+
+
+TOTAL_SUCTION_MPA = 100.0
+MACRO_SUCTION_MPA = 1.0
+MICRO_SUCTION_MPA = TOTAL_SUCTION_MPA - MACRO_SUCTION_MPA
+
+
+MASS_EXCHANGE_COEFFICIENT = 1e-13
+MICRO_SWELLING_SLOPE = 0.1
+PRESSURE_IC_PA = -MACRO_SUCTION_MPA * 1e6
+TIME_END_S = 120 * 86400
+PRESSURE_TOLERANCE_PA = 1e-12
+
+
+AREA_FACTOR_TULLER = 1.0
+PORE_AREA_SHAPE_FACTOR_TULLER = 0.8584073464102069
+CHARACTERISTIC_PORE_SIZE = 1e-5
+SURFACE_TENSION = 0.0715
+MICRO_LIQUID_DENSITY_REFERENCE = 1e-6
+MICRO_LIQUID_DENSITY_A = 1e-16
+MICRO_LIQUID_DENSITY_B = 1.0
+
+
+NATIVE_DSM_MICROMACRO_SOURCE = Path("/Users/vinaykumar/git/ogs-native-dsm-transition")
+DEFAULT_NATIVE_OGS = Path("/Users/vinaykumar/git/build/release-native-beacon/bin/ogs")
+DEFAULT_MFRONT_CALIBRATION_CSV = ROOT / "villar_dense_dd_calibration.csv"
+
+
+
+@dataclass(frozen=True)
+class Case:
+    dry_density: float
+
+
+    @property
+    def dry_density_g_cm3(self) -> float:
+        return self.dry_density / 1000.0
+
+
+    @property
+    def phi0(self) -> float:
+        return 1.0 - self.dry_density / RHO_SOLID
+
+
+    @property
+    def phi_micro_assumed(self) -> float:
+        return 0.99 * self.phi0
+
+
+    @property
+    def phi_macro_assumed(self) -> float:
+        return 0.01 * self.phi0
+
+
+    @property
+    def intrinsic_permeability(self) -> float:
+        return (
+            MS33_K0_REF
+            * ((1.0 - MS33_PHI_REF) ** 2 / MS33_PHI_REF ** 3)
+            * (self.phi0 ** 3 / (1.0 - self.phi0) ** 2)
+        )
+
+
+    @property
+    def initial_volume_ratio(self) -> float:
+        return 1.0 / (1.0 - self.phi0)
+
+
+    @property
+    def villar_target_swelling_mpa(self) -> float:
+        return math.exp(6.77 * self.dry_density_g_cm3 - 9.07)
+
+
+
+def git_short_hash(repo: Path) -> str:
+    """Return a short Git hash for provenance tracking."""
+    try:
+        return (
+            subprocess.check_output(
+                ["git", "rev-parse", "--short", "HEAD"], cwd=repo, text=True
+            )
+            .strip()
+        )
+    except Exception:
+        return ""
+
+
+
+def read_grid(path: Path) -> vtk.vtkUnstructuredGrid:
+    """Read a VTU snapshot into a VTK unstructured grid."""
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    return reader.GetOutput()
+
+
+
+def get_array(grid: vtk.vtkUnstructuredGrid, name: str):
+    """Fetch a field from point or cell data."""
+    arr = grid.GetPointData().GetArray(name)
+    if arr is None:
+        arr = grid.GetCellData().GetArray(name)
+    if arr is None:
+        raise KeyError(name)
+    return vtk_to_numpy(arr)
+
+
+
+def mean_total_stress_mpa(vtu_path: Path) -> float:
+    """Compute the mean isotropic stress in MPa from the final VTU."""
+    grid = read_grid(vtu_path)
+    sigma = get_array(grid, "sigma")
+    p_mean = float((-sigma[:, 0] - sigma[:, 1] - sigma[:, 2]).mean() / 3.0)

```

```

+ return p_mean / 1e6
+
+
+def extract_last_vtu(prefix: str) -> Path:
+ """Select the final VTU produced by a case run."""
+ candidates = sorted(ROOT.glob(f"{prefix}_ts_*_t*.vtu"))
+ if not candidates:
+ raise FileNotFoundError(f"No VTU outputs found for prefix {prefix}")
+
+ def key(path: Path):
+ stem = path.stem
+ match = re.search(r"^(?P<ts>ts_)(?P<t>t_)([0-9eE+]+)$", stem)
+ if not match:
+ return (-1, -1.0)
+ return (int(match.group(1)), float(match.group(2)))
+
+ return sorted(candidates, key=key)[-1]
+
+
+def cleanup_runtime(prefix: str, project_path: Path) -> None:
+ """Remove transient outputs from a calibration trial."""
+ for pattern in (f"{prefix}.pvd", f"{prefix}_ts_*_t*.vtu"):
+ for p in ROOT.glob(pattern):
+ p.unlink(missing_ok=True)
+ project_path.unlink(missing_ok=True)
+
+
+def n_l0_from_micro_suction(phi0: float, hamaker_eff: float) -> float:
+ """Derive the initial micro-scale water content from the suction split."""
+ # Target micro suction potential:  $\mu = p / \rho$ .
+ mu_abs = MICRO_SUCTION_MPA * 1e6 / RHO_LR_REF
+ n_s = 1.0 - phi0
+ prefactor = (
+ abs(hamaker_eff)
+ * (SPECIFIC_SURFACE_MASS * n_s * RHO_SOLID_REFERENCE) ** 3
+ / (6.0 * math.pi)
+ )
+ n_l0 = (prefactor / mu_abs) ** (1.0 / 3.0)
+ return max(1e-12, n_l0)
+
+
+def write_native_dsm_micromacro_project(
+ case: Case, project_path: Path, vdw_multiplier: float, n_l0_fixed: float
+) -> dict:
+ """Create a temporary native dsm_micromacro project for one multiplier trial."""
+ prefix = project_path.stem
+ hamaker_eff = HAMAKER_LITERATURE * vdw_multiplier
+ n_l0 = n_l0_fixed
+ n_s = 1.0 - case.phi0
+
+ xml = f"""<?xml version='1.0' encoding='ISO-8859-1'?>
+<OpenGeoSysProject>
+ <meshes>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_left.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_right.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_top.vtu</mesh>
+ <mesh axially_symmetric="true">../square_1x1_quad_1e0_bottom.vtu</mesh>
+ </meshes>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <jacobian_assembler><type>Analytical</type></jacobian_assembler>
+ <micro_porosity>
+ <mass_exchange_coefficient>{MASS_EXCHANGE_COEFFICIENT: .16g}</mass_exchange_coefficient>
+ <nonlinear_solver>
+ <maximum_iterations>100</maximum_iterations>
+ <residuum_tolerance>1e-8</residuum_tolerance>
+ <increment_tolerance>1e-20</increment_tolerance>
+ </nonlinear_solver>
+ </micro_porosity>
+ <potential_exchange>
+ <enabled>true</enabled>
+ <mode>full_potential</mode>
+ <pressure_tolerance>{PRESSURE_TOLERANCE_PA: .16g}</pressure_tolerance>
+ <hamaker_constant>{hamaker_eff: .16g}</hamaker_constant>
+ <specific_surface>{SPECIFIC_SURFACE_MASS: .16g}</specific_surface>
+ <micro_solid_density_reference>{RHO_SOLID_REFERENCE: .16g}</micro_solid_density_reference>
+ <micro_solid_volume_fraction_reference>{n_s: .16g}</micro_solid_volume_fraction_reference>
+ <micro_liquid_density_reference>{MICRO_LIQUID_DENSITY_REFERENCE: .16g}</micro_liquid_density_reference>
+ <micro_liquid_density_a>{MICRO_LIQUID_DENSITY_A: .16g}</micro_liquid_density_a>
+ <micro_liquid_density_b>{MICRO_LIQUID_DENSITY_B: .16g}</micro_liquid_density_b>
+ <initial_micro_water_content>{n_l0: .16g}</initial_micro_water_content>
+ <local_nonlinear_solve_mode>scalar_micro_macro_mass_storage_mode</local_nonlinear_solve_mode>
+ <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+ <micro_potential_convention>negative_attractive</micro_potential_convention>
+ <micro_water_content_swelling_slope>{MICRO_SWELLING_SLOPE: .16g}</micro_water_content_swelling_slope>
+ </potential_exchange>
+ <constitutive_relation>
+ <type>MFront</type>
+ <behaviour>ModCamClay_semiExpl_constE</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="YoungModulus"/>
+ <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable name="sigma"/>
+ <secondary_variable name="swelling_stress"/>
+ <secondary_variable name="saturation"/>
+ <secondary_variable name="porosity"/>
+ <secondary_variable name="dry_density_solid"/>
+ <secondary_variable name="micro_pressure"/>
+

```

```

+         <secondary_variable name="micro_saturation"/>
+         <secondary_variable name="micro_water_content"/>
+         <secondary_variable name="micro_porosity"/>
+         <secondary_variable name="micro_exchange_source"/>
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+           <property><name>density</name><type>Constant</type><value>1e3</value></property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property><name>density</name><type>Constant</type><value>{RHO_SOLID:.16g}</value></property>
+           <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>0 0</swelling_pressures>
+             <exponents>1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+       <property>
+         <name>permeability</name>
+         <type>KozenyCarman</type>
+         <initial_permeability>IntrinsicPermeability0</initial_permeability>
+         <initial_porosity>phi0</initial_porosity>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>Constant</type>
+         <value>1</value>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationTuller</type>
+         <area_factor_tuller>{AREA_FACTOR_TULLER:.16g}</area_factor_tuller>
+         <pore_area_shapefactor_tuller>{PORE_AREA_SHAPE_FACTOR_TULLER:.16g}</pore_area_shapefactor_tuller>
+         <characteristic_pore_size>{CHARACTERISTIC_PORE_SIZE:.16g}</characteristic_pore_size>
+         <surface_tension>{SURFACE_TENSION:.16g}</surface_tension>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <abstols>5e-8 1e-13 1e-13</abstols>
+       </convergence_criterion>
+       <time_discretization><type>BackwardEuler</type></time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>{TIME_END_S}</t_end>
+         <timesteps><pair><repeat>120</repeat><delta_t>86400</delta_t></pair></timesteps>
+       </time_stepping>
+     </process>
+   </processes>
+ </time_loop>
+ <output>
+   <type>VTK</type>
+   <prefix>{prefix}</prefix>
+   <suffix>_ts_{timestep}_t_{time}</suffix>
+   <fixed_output_times>{TIME_END_S}</fixed_output_times>
+   <variables>
+     <variable>pressure</variable>
+     <variable>sigma</variable>
+     <variable>swelling_stress</variable>
+     <variable>saturation</variable>
+     <variable>porosity</variable>
+     <variable>dry_density_solid</variable>
+     <variable>micro_pressure</variable>
+     <variable>micro_saturation</variable>
+     <variable>micro_water_content</variable>
+     <variable>micro_porosity</variable>
+     <variable>micro_exchange_source</variable>

```

```

+         </variables>
+     </output>
+ </time_loop>
+ <parameters>
+     <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+     <parameter><name>YoungModulus</name><type>Constant</type><value>526</value></parameter>
+     <parameter><name>PoissonRatio</name><type>Constant</type><value>0.3</value></parameter>
+     <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+     <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+     <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+     <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+     <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>(case.initial_volume_ratio:.16g)</value></parameter>
+     <parameter><name>phi0</name><type>Constant</type><value>(case.phi0:.16g)</value></parameter>
+     <parameter><name>IntrinsicPermeability0</name><type>Constant</type><value>(case.intrinsic_permeability:.16g)</value></parameter>
+     <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+     <parameter><name>zero</name><type>Constant</type><value>0.0</value></parameter>
+     <parameter><name>pressure_ic</name><type>Constant</type><value>(PRESSURE_IC_PA:.16g)</value></parameter>
+     <parameter><name>pressure_bc_scale</name><type>Constant</type><value>1</value></parameter>
+     <parameter><name>pressure_bc</name><type>CurveScaled</type><curve>pressure_release</curve><parameter>pressure_bc_scale</parameter></parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+     <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+             <boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver><name>basic_newton</name><type>Newton</type><max_iter>60</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ <curves>
+     <curve><name>pressure_release</name><coords>0 (TIME_END_S)</coords><values>(PRESSURE_IC_PA:.16g) 0</values></curve>
+ </curves>
+ </OpenGeoSysProject>
+ """
+     project_path.write_text(xml)
+     return {
+         "hamaker_effective_3": hamaker_eff,
+         "n_l0": n_l0,
+         "micro_solid_volume_fraction_reference": n_s,
+     }
+
+
+ def run_ogs(ogs_bin: Path, project_path: Path) -> None:
+     """Execute OGS for a single temporary project."""
+     subprocess.run(
+         [str(ogs_bin), str(project_path)],
+         cwd=ROOT,
+         check=True,
+         stdout=subprocess.DEVNULL,
+         stderr=subprocess.STDOUT,
+     )
+
+
+ def run_native_dsm_micromacro_case(
+     ogs_bin: Path, case: Case, multiplier: float, n_l0_fixed: float, idx: int
+ ) -> dict:
+     case_tag = f"d{int(case.dry_density)}_native_dsm_micromacro_{idx:02d}"
+     project_path = ROOT / f"{case_tag}.prj"
+     meta = write_native_dsm_micromacro_project(case, project_path, multiplier, n_l0_fixed)
+     try:
+         run_ogs(ogs_bin, project_path)
+         last_vtu = extract_last_vtu(case_tag)
+         pressure_mpa = mean_total_stress_mpa(last_vtu)
+     finally:
+         cleanup_runtime(case_tag, project_path)
+     return {"pressure_mpa": pressure_mpa, "multiplier": multiplier, **meta}
+
+
+ def calibrate_multiplier_for_case(
+     ogs_bin: Path,
+     case: Case,
+     n_l0_fixed: float,
+     target_mpa: float,
+     rel_tol: float = 0.02,
+     max_iter: int = 18,
+ ) -> dict:
+     """Solve for the multiplier that matches the Villar target."""
+     target_scale = max(target_mpa, 1e-12)
+     max_multiplier = 1e18
+
+     def rel_err(run: dict) -> float:
+         pressure = run["pressure_mpa"]
+         if not math.isfinite(pressure):
+             return float("inf")
+         return abs(pressure - target_mpa) / target_scale
+
+     # Baseline.
+     run_id = 0
+     run_lo = run_native_dsm_micromacro_case(ogs_bin, case, 1.0, n_l0_fixed, run_id)
+     run_id += 1
+     best = run_lo
+     if rel_err(run_lo) < rel_tol:
+         return run_lo

```

```

+ # Bracket from above with geometric expansion.
+ m_lo = 1.0
+ m_hi = max(2.0, target_mpa / max(abs(run_lo["pressure_mpa"]), 1e-12))
+ m_hi = float(np.clip(m_hi, 1e-6, max_multiplier))
+ if m_hi <= m_lo:
+     m_hi = 2.0
+
+ run_hi = run_native_dsm_micromacro_case(ogs_bin, case, m_hi, n_l0_fixed, run_id)
+ run_id += 1
+ if rel_err(run_hi) < rel_err(best):
+     best = run_hi
+ p_hi = run_hi["pressure_mpa"] if math.isfinite(run_hi["pressure_mpa"]) else float("inf")
+
+ expand_limit = max_iter + 16
+ while p_hi < target_mpa and m_hi < max_multiplier and run_id < expand_limit:
+     m_lo, run_lo = m_hi, run_hi
+     m_hi = min(max_multiplier, m_hi * 10.0)
+     if m_hi <= m_lo:
+         break
+     run_hi = run_native_dsm_micromacro_case(ogs_bin, case, m_hi, n_l0_fixed, run_id)
+     run_id += 1
+     if rel_err(run_hi) < rel_err(best):
+         best = run_hi
+     p_hi = (
+         run_hi["pressure_mpa"]
+         if math.isfinite(run_hi["pressure_mpa"])
+         else float("inf")
+     )
+
+ if p_hi < target_mpa:
+     return best
+
+ # Log-space bisection inside bracket [m_lo, m_hi].
+ for _ in range(max_iter):
+     m_mid = math.sqrt(m_lo * m_hi)
+     if m_mid <= m_lo * (1.0 + 1e-12) or m_mid >= m_hi * (1.0 - 1e-12):
+         break
+     run_mid = run_native_dsm_micromacro_case(ogs_bin, case, m_mid, n_l0_fixed, run_id)
+     run_id += 1
+     err_mid = rel_err(run_mid)
+     if err_mid < rel_err(best):
+         best = run_mid
+     if err_mid < rel_tol:
+         return run_mid
+
+     p_mid = (
+         run_mid["pressure_mpa"]
+         if math.isfinite(run_mid["pressure_mpa"])
+         else float("inf")
+     )
+
+     if p_mid < target_mpa:
+         m_lo, run_lo = m_mid, run_mid
+     else:
+         m_hi, run_hi = m_mid, run_mid
+
+ return best
+
+def load_mfront_calibrated_curve(path: Path) -> tuple[np.ndarray, np.ndarray] | None:
+    if not path.exists():
+        return None
+    x = []
+    y = []
+    with path.open(newline="") as f:
+        reader = csv.DictReader(f)
+        for row in reader:
+            x.append(float(row["dry_density_kg_m3"]))
+            y.append(float(row["mfront_calibrated_MPa"]))
+    if not x:
+        return None
+    order = np.argsort(np.asarray(x))
+    return np.asarray(x)[order], np.asarray(y)[order]
+
+def main() -> None:
+    """Run the dense sweep and write the CSV, JSON, and comparison plots."""
+    parser = argparse.ArgumentParser(
+        description=(
+            "Dense dry-density calibration of effective micro vdw multiplier "
+            "for the native dsm_micromacro branch against Villar Eq.(7). "
+        )
+    )
+    parser.add_argument("--native-ogs", type=Path, default=DEFAULT_NATIVE_OGS)
+    parser.add_argument(
+        "--native-source",
+        type=Path,
+        default=NATIVE_DSM_MICROMACRO_SOURCE,
+        help="Native OGS source tree for commit-hash provenance in summary JSON.",
+    )
+    parser.add_argument("--dd-min", type=float, default=1400.0)
+    parser.add_argument("--dd-max", type=float, default=1800.0)
+    parser.add_argument("--dd-step", type=float, default=25.0)
+    parser.add_argument("--rel-tol", type=float, default=0.02)
+    parser.add_argument(
+        "--mfront-calibration-csv",
+        type=Path,
+        default=DEFAULT_MFRONT_CALIBRATION_CSV,
+        help="Optional MFront calibration curve for overlay.",
+    )
+    args = parser.parse_args()
+
+    dd_values = np.arange(args.dd_min, args.dd_max + 0.5 * args.dd_step, args.dd_step)
+    cases = [Case(float(dd)) for dd in dd_values]
+
+    rows = []
+    for case in cases:
+        target = case.villar_target_swelling_mpa
+        # Keep n_l0 fixed per dry density from literature Hamaker baseline,
+        # matching the MFront dense calibration workflow.
+        n_l0_fixed = n_l0_from_micro_suction(case.phi0, HAMAKER_LITERATURE)
+        baseline = run_native_dsm_micromacro_case(args.native_ogs, case, 1.0, n_l0_fixed, 90)
+        calibrated = calibrate_multiplier_for_case(
+            args.native_ogs, case, n_l0_fixed, target, rel_tol=args.rel_tol
+        )
+        print(
+            f"dd={case.dry_density:.0f} kg/m3: "
+            f"target={target:.3f} MPa, "

```

```

+         f"native_baseline={baseline['pressure_mpa']:.3f} MPa, "
+         f"native_calibrated={calibrated['pressure_mpa']:.3f} MPa, "
+         f"mult={calibrated['multiplier']:.3e}"
+     )
+     rows.append(
+         {
+             "dry_density_kg_m3": case.dry_density,
+             "dry_density_g_cm3": case.dry_density_g_cm3,
+             "phi0": case.phi0,
+             "phi_micro_assumed": case.phi_micro_assumed,
+             "phi_macro_assumed": case.phi_macro_assumed,
+             "k0_m2": case.intrinsic_permeability,
+             "target_villar_MPa": target,
+             "native_baseline_MPa": baseline["pressure_mpa"],
+             "native_calibrated_MPa": calibrated["pressure_mpa"],
+             "vdw_multiplier": calibrated["multiplier"],
+             "hamaker_literature_J": HAMAKER_LITERATURE,
+             "hamaker_effective_J": calibrated["hamaker_effective_J"],
+             "specific_surface_mass_m2_g": SPECIFIC_SURFACE_MASS,
+             "n_l0": calibrated["n_l0"],
+             "micro_solid_volume_fraction_reference": calibrated[
+                 "micro_solid_volume_fraction_reference"
+             ],
+             "macro_suction_MPa": MACRO_SUCTION_MPA,
+             "micro_suction_MPa": MICRO_SUCTION_MPA,
+             "native_baseline_minus_target_MPa": baseline["pressure_mpa"] - target,
+             "native_calibrated_minus_target_MPa": calibrated["pressure_mpa"] - target,
+         }
+     )
+
+ rows = sorted(rows, key=lambda r: r["dry_density_kg_m3"])
+
+ out_csv = ROOT / "villar_dense_dd_native_dsm_micromacro_calibration.csv"
+ with out_csv.open("w", newline="") as f:
+     writer = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
+     writer.writeheader()
+     writer.writerows(rows)
+
+ rel_err_baseline = [
+     abs(r["native_baseline_minus_target_MPa"]) / max(r["target_villar_MPa"], 1e-12)
+     for r in rows
+ ]
+ rel_err_calibrated = [
+     abs(r["native_calibrated_minus_target_MPa"]) / max(r["target_villar_MPa"], 1e-12)
+     for r in rows
+ ]
+
+ summary = {
+     "ogs_repo_hash": git_short_hash(RICHARDS_DATA_ROOT.parents[2]),
+     "native_dsm_micromacro_source_hash": git_short_hash(args.native_source),
+     "native_dsm_micromacro_source_path": str(args.native_source),
+     "native_ogs_version": subprocess.check_output(
+         [str(args.native_ogs), "--version"], text=True
+     ),
+     "total_initial_suction_MPa": TOTAL_SUCTION_MPA,
+     "macro_initial_suction_MPa": MACRO_SUCTION_MPA,
+     "micro_initial_suction_MPa": MICRO_SUCTION_MPA,
+     "hamaker_literature_J": HAMAKER_LITERATURE,
+     "specific_surface_mass_m2_g": SPECIFIC_SURFACE_MASS,
+     "mean_relative_error_baseline_percent": 100.0 * float(np.mean(rel_err_baseline)),
+     "mean_relative_error_calibrated_percent": 100.0 * float(np.mean(rel_err_calibrated)),
+     "max_relative_error_calibrated_percent": 100.0 * float(np.max(rel_err_calibrated)),
+     "results": rows,
+ }
+
+ out_summary = ROOT / "villar_dense_dd_native_dsm_micromacro_calibration_summary.json"
+ out_summary.write_text(json.dumps(summary, indent=2))
+
+ x = np.array([r["dry_density_kg_m3"] for r in rows])
+ y_target = np.array([r["target_villar_MPa"] for r in rows])
+ y_calibrated = np.array([r["native_calibrated_MPa"] for r in rows])
+ y_mult = np.array([r["vdw_multiplier"] for r in rows])
+
+ plt.figure(figsize=(8.2, 5.2))
+ plt.plot(x, y_target, "k-", linewidth=2.0, label="Villar Eq. (7) target")
+ plt.plot(
+     x,
+     y_calibrated,
+     color="#2ca02c",
+     marker="D",
+     linestyle="--",
+     linewidth=1.8,
+     markersize=4.2,
+     label="Native dsm_micromacro calibrated",
+ )
+ mfront_curve = load_mfront_calibrated_curve(args.mfront_calibration_csv)
+ if mfront_curve is not None:
+     x_m, y_m = mfront_curve
+     plt.plot(
+         x_m,
+         y_m,
+         color="#d62728",
+         marker="s",
+         linestyle="--",
+         linewidth=1.5,
+         markersize=3.6,
+         label="MFront calibrated (existing run)",
+     )
+
+ plt.xlabel("Dry density (kg/m³)")
+ plt.ylabel("Swelling pressure at full saturation (MPa)")
+ plt.grid(True, alpha=0.35)
+ plt.legend(frameon=False)
+ plt.tight_layout()
+ out_cmp = ROOT / "villar_dense_dd_native_dsm_micromacro_calibration_comparison.png"
+ plt.savefig(out_cmp, dpi=220)
+ plt.close()
+
+ plt.figure(figsize=(8.2, 5.2))
+ plt.semilogy(
+     x,
+     y_mult,
+     color="#9467bd",
+     marker="^",
+     linestyle="--",
+     linewidth=1.8,
+     markersize=4.3,
+ )
+ plt.xlabel("Dry density (kg/m³)")

```

```

+ plt.ylabel("Native dsm_micromacro effective vdw multiplier (-)")
+ plt.grid(True, which="both", alpha=0.35)
+ plt.tight_layout()
+ out_mult = ROOT / "villar_dense_dd_native_dsm_micromacro_vdw_multiplier.png"
+ plt.savefig(out_mult, dpi=220)
+ plt.close()
+
+ print(f"Wrote: {out_csv}")
+ print(f"Wrote: {out_summary}")
+ print(f"Wrote: {out_cmp}")
+ print(f"Wrote: {out_mult}")
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/summarize_ms33_model_i.py b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/summarize_ms33_model_i.py
new file mode 100644
index 0000000000..133feb1ff8
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/summarize_ms33_model_i.py
@@ -0,0 +1,158 @@
#!/usr/bin/env python3
+"""Reduce MS33 Model-I benchmark snapshots into compact history tables.
+
+The script scans the case VTUs, extracts mean state variables, and writes a
+CSV history plus a JSON summary for later comparison and plotting.
+"""
+
+from __future__ import annotations
+
+import csv
+import json
+from pathlib import Path
+
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+ROOT = Path(__file__).resolve().parent
+RHO_SOLID = 2780.0
+CASES = {
+    "dd1400": {
+        "rho_dry": 1400.0,
+        "specific_weight_kg_m3": 14.0,
+        "phi0": 0.49640287769784175,
+        "k0": 1.2264011098493141e-20,
+    },
+    "dd1600": {
+        "rho_dry": 1600.0,
+        "specific_weight_kg_m3": 16.0,
+        "phi0": 0.4244604316546763,
+        "k0": 5.870260900123441e-21,
+    },
+    "dd1800": {
+        "rho_dry": 1800.0,
+        "specific_weight_kg_m3": 18.0,
+        "phi0": 0.3525179856115108,
+        "k0": 2.6569572325678573e-21,
+    },
+}
+
+def read_grid(path: Path) -> vtk.vtkUnstructuredGrid:
+    """Read a VTU snapshot into a VTK unstructured grid."""
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    return reader.GetOutput()
+
+def get_array(grid: vtk.vtkUnstructuredGrid, name: str):
+    """Fetch a field from point or cell data, preferring point data."""
+    arr = grid.GetPointData().GetArray(name)
+    if arr is None:
+        arr = grid.GetCellData().GetArray(name)
+    if arr is None:
+        raise KeyError(name)
+    return vtk_to_numpy(arr)
+
+def clean_scalar(value: float, tol: float = 1e-12) -> float:
+    """Clamp near-zero floating-point noise to exactly zero."""
+    return 0.0 if abs(value) < tol else value
+
+def reduce_snapshot(path: Path):
+    """Compute the aggregate diagnostics recorded for one snapshot."""
+    grid = read_grid(path)
+    pressure = get_array(grid, "pressure")
+    saturation = get_array(grid, "saturation")
+    porosity = get_array(grid, "porosity")
+    dry_density = get_array(grid, "dry_density_solid")
+    sigma = get_array(grid, "sigma")
+    swelling = get_array(grid, "swelling_stress")
+
+    mean_total_stress = float((-sigma[:, 0] - sigma[:, 1] - sigma[:, 2]).mean() / 3.0)
+    mean_swelling_stress = float((-swelling[:, 0] - swelling[:, 1] - swelling[:, 2]).mean() / 3.0)
+
+    return {
+        "pressure_Pa_avg": clean_scalar(float(pressure.mean())),
+        "pressure_Pa_min": clean_scalar(float(pressure.min())),
+        "pressure_Pa_max": clean_scalar(float(pressure.max())),
+        "suction_MPa_avg": clean_scalar(float(-pressure.mean() / 1e6)),
+        "saturation_avg": clean_scalar(float(saturation.mean())),
+        "porosity_avg": clean_scalar(float(porosity.mean())),
+        "porosity_spread": clean_scalar(float(porosity.max() - porosity.min())),
+        "dry_density_avg_kg_m3": clean_scalar(float(dry_density.mean())),
+        "dry_density_spread_kg_m3": clean_scalar(float(dry_density.max() - dry_density.min())),
+        "mean_total_stress_MPa": clean_scalar(mean_total_stress / 1e6),
+        "mean_swelling_stress_MPa": clean_scalar(mean_swelling_stress / 1e6),
+    }
+
+def main() -> None:
+    """Collect all snapshots, write the summary JSON, and emit the CSV."""
+    history_rows = []
+    summary = {"rho_solid_kg_m3": RHO_SOLID, "cases": {}}

```



```

+ "macro_initial_suction_MPa": 1.0,
+ "micro_initial_suction_MPa": 99.0,
+ "notes": "Effective multiplier is applied as A_eff = multiplier * A_literature. n_10 is recomputed for each (density, multiplier) to preserve the micro suction split in the initial state.",
+ "results": [
+ {
+   "dry_density_kg_m3": 1400.0,
+   "dry_density_g_cm3": 1.4,
+   "phi10": 0.4964827769784175,
+   "phi_micro_assumed": 0.4914388489208633,
+   "phi_macro_assumed": 0.00496482776978418,
+   "k0_m2": 1.2264011098493141e-20,
+   "specific_surface_mass_m2_g": 523.0,
+   "specific_surface_volumetric_m2_m3": 732200000.0,
+   "target_villar_MPa": 1.5038071611701085,
+   "mfront_calibrated_MPa": 1.5038065460524999,
+   "native_MPa": 1.9996489593744509,
+   "vdw_multiplier": 99961.04975655653,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.098013537584383e-16,
+   "n_10": 0.001023703867471382,
+   "macro_suction_MPa": 1.0,
+   "micro_suction_MPa": 99.0,
+   "mfront_minus_target_MPa": -6.151176086266474e-07,
+   "native_minus_target_MPa": 0.4958417982043424
+ },
+ {
+   "dry_density_kg_m3": 1425.0,
+   "dry_density_g_cm3": 1.425,
+   "phi10": 0.487410071942446,
+   "phi_micro_assumed": 0.48253597122302155,
+   "phi_macro_assumed": 0.00487410071942446,
+   "k0_m2": 1.1205713227338159e-20,
+   "specific_surface_mass_m2_g": 523.0,
+   "specific_surface_volumetric_m2_m3": 745275000.0,
+   "target_villar_MPa": 1.78113357233691,
+   "mfront_calibrated_MPa": 1.7811318756447714,
+   "native_MPa": 1.9996489593744509,
+   "vdw_multiplier": 171319.29503996955,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 8.737284047038447e-16,
+   "n_10": 0.001041984293676228,
+   "macro_suction_MPa": 1.0,
+   "micro_suction_MPa": 99.0,
+   "mfront_minus_target_MPa": -1.6966921385641598e-06,
+   "native_minus_target_MPa": 0.21851538703754092
+ },
+ {
+   "dry_density_kg_m3": 1450.0,
+   "dry_density_g_cm3": 1.45,
+   "phi10": 0.4784172661870504,
+   "phi_micro_assumed": 0.47363309352517985,
+   "phi_macro_assumed": 0.004784172661870504,
+   "k0_m2": 1.02345856648015e-20,
+   "specific_surface_mass_m2_g": 523.0,
+   "specific_surface_volumetric_m2_m3": 758350000.0,
+   "target_villar_MPa": 2.1096034680651288,
+   "mfront_calibrated_MPa": 2.109599067594651,
+   "native_MPa": 1.9996489593744509,
+   "vdw_multiplier": 294184.00258070176,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 1.500338412794379e-15,
+   "n_10": 0.0010602647198810742,
+   "macro_suction_MPa": 1.0,
+   "micro_suction_MPa": 99.0,
+   "mfront_minus_target_MPa": -4.400470477783358e-06,
+   "native_minus_target_MPa": -0.109954580869067791
+ },
+ {
+   "dry_density_kg_m3": 1475.0,
+   "dry_density_g_cm3": 1.475,
+   "phi10": 0.46942446043165464,
+   "phi_micro_assumed": 0.4647302158273381,
+   "phi_macro_assumed": 0.004694244604316547,
+   "k0_m2": 9.343268757119368e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "specific_surface_volumetric_m2_m3": 771425000.0,
+   "target_villar_MPa": 2.4986485357374417,
+   "mfront_calibrated_MPa": 2.498637692288117,
+   "native_MPa": 1.9996489593744509,
+   "vdw_multiplier": 506418.4683587369,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 2.582734188629558e-15,
+   "n_10": 0.00107854514608859203,
+   "macro_suction_MPa": 1.0,
+   "micro_suction_MPa": 99.0,
+   "mfront_minus_target_MPa": -1.0843449324671184e-05,
+   "native_minus_target_MPa": -0.49899957636299086
+ },
+ {
+   "dry_density_kg_m3": 1500.0,
+   "dry_density_g_cm3": 1.5,
+   "phi10": 0.460431654676259,
+   "phi_micro_assumed": 0.45582733812949644,
+   "phi_macro_assumed": 0.00460431654676259,
+   "k0_m2": 8.525885392608007e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "specific_surface_volumetric_m2_m3": 784500000.0,
+   "target_villar_MPa": 2.959439818739489,
+   "mfront_calibrated_MPa": 2.9594142426949315,
+   "native_MPa": 1.9996489593744509,
+   "vdw_multiplier": 874548.6094340767,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 4.4601979081137915e-15,
+   "n_10": 0.0010968255722907663,
+   "macro_suction_MPa": 1.0,
+   "micro_suction_MPa": 99.0,
+   "mfront_minus_target_MPa": -2.5576044557329425e-05,
+   "native_minus_target_MPa": -0.9597908593650379
+ },
+ {
+   "dry_density_kg_m3": 1525.0,
+   "dry_density_g_cm3": 1.525,
+   "phi10": 0.4514388489208633,
+   "phi_micro_assumed": 0.4469244604316546,
+   "phi_macro_assumed": 0.004514388489208633,
+   "k0_m2": 7.773969584065612e-21,

```

```

+
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 797575000.0,
+ "target_villar_MPa": 3.505208481894766,
+ "mfront_calibrated_MPa": 3.505150467835943,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 1516270.1832831167,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 7.732977934743895e-15,
+ "n_l0": 0.0011151059984956127,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -5.801405882310817e-05,
+ "native_minus_target_MPa": -1.505559522520315
+ },
+ {
+ "dry_density_kg_m3": 1550.0,
+ "dry_density_g_cm3": 1.55,
+ "phi0": 0.44244604316546765,
+ "phi_micro_assumed": 0.438021582733813,
+ "phi_macro_assumed": 0.004424460431654676,
+ "k0_m2": 7.084403435649412e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 810650000.0,
+ "target_villar_MPa": 4.151625731244024,
+ "mfront_calibrated_MPa": 4.151498844974241,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 2641230.585113023,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 1.3470275984076417e-14,
+ "n_l0": 0.0011333864247004585,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -0.00012688626978274442,
+ "native_minus_target_MPa": -2.151976771869573
+ },
+ {
+ "dry_density_kg_m3": 1575.0,
+ "dry_density_g_cm3": 1.575,
+ "phi0": 0.4334532374100719,
+ "phi_micro_assumed": 0.4291187050359712,
+ "phi_macro_assumed": 0.0043345323741007195,
+ "k0_m2": 6.4513614015419514e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 823725000.0,
+ "target_villar_MPa": 4.917252797188951,
+ "mfront_calibrated_MPa": 4.916904828259917,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 4625294.549720161,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 2.358900220361362e-14,
+ "n_l0": 0.001151666850905305,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -0.00026796892903391267,
+ "native_minus_target_MPa": -2.9176038378145
+ },
+ {
+ "dry_density_kg_m3": 1600.0,
+ "dry_density_g_cm3": 1.6,
+ "phi0": 0.4244604316546763,
+ "phi_micro_assumed": 0.4202158273381295,
+ "phi_macro_assumed": 0.004244604316546763,
+ "k0_m2": 5.8702609001234435e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 836800000.0,
+ "target_villar_MPa": 5.824073901819978,
+ "mfront_calibrated_MPa": 5.8235270115504925,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 8146393.729953618,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 4.154660802276345e-14,
+ "n_l0": 0.0011699472771101508,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -0.0005468902694856226,
+ "native_minus_target_MPa": -3.8244249424455274
+ },
+ {
+ "dry_density_kg_m3": 1625.0,
+ "dry_density_g_cm3": 1.625,
+ "phi0": 0.41546762589928055,
+ "phi_micro_assumed": 0.41131294964028775,
+ "phi_macro_assumed": 0.004154676258992806,
+ "k0_m2": 5.33691847941765e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 849875000.0,
+ "target_villar_MPa": 6.898127514056516,
+ "mfront_calibrated_MPa": 6.898121706273423,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 14442267.448706342,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 7.365556398840235e-14,
+ "n_l0": 0.001188227703314997,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -5.807783093558783e-06,
+ "native_minus_target_MPa": -4.8984785546820655
+ },
+ {
+ "dry_density_kg_m3": 1650.0,
+ "dry_density_g_cm3": 1.65,
+ "phi0": 0.4064748201438849,
+ "phi_micro_assumed": 0.40241007194244605,
+ "phi_macro_assumed": 0.004064748201438849,
+ "k0_m2": 4.84751084136096e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 862950000.0,
+ "target_villar_MPa": 8.170254018465311,
+ "mfront_calibrated_MPa": 8.1702390088907213,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 25752873.196480487,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 1.3133965330205048e-13,
+ "n_l0": 0.0012065081295198432,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -1.5009558097744957e-05,

```

```

+ "native_minus_target_MPa": -6.17060505909086
+ },
+ {
+ "dry_density_kg_m3": 1675.0,
+ "dry_density_g_cm3": 1.675,
+ "phi0": 0.3974820143884892,
+ "phi_micro_assumed": 0.3935071942446043,
+ "phi_macro_assumed": 0.003974820143884892,
+ "k0_m2": 4.3985401273197395e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 876025000.0,
+ "target_villar_MPa": 9.676981266325395,
+ "mfront_calibrated_MPa": 9.676944431483228,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 46200630.570207246,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 2.3562321590805697e-13,
+ "n_l0": 0.00122478855724689,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -3.6834842166655335e-05,
+ "native_minus_target_MPa": -7.6773323069509445
+ },
+ {
+ "dry_density_kg_m3": 1700.0,
+ "dry_density_g_cm3": 1.7,
+ "phi0": 0.38848920863309355,
+ "phi_micro_assumed": 0.38460431654676264,
+ "phi_macro_assumed": 0.0038848920863309355,
+ "k0_m2": 3.986802948009508e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 889100000.0,
+ "target_villar_MPa": 11.461573436660695,
+ "mfront_calibrated_MPa": 11.461487424771605,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 83369754.91317607,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 4.25185750057198e-13,
+ "n_l0": 0.001243068981929535,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -8.601188900818731e-05,
+ "native_minus_target_MPa": -9.461924477286244
+ },
+ {
+ "dry_density_kg_m3": 1725.0,
+ "dry_density_g_cm3": 1.725,
+ "phi0": 0.3794964028776978,
+ "phi_micro_assumed": 0.3757014388489208,
+ "phi_macro_assumed": 0.0037949640287769783,
+ "k0_m2": 3.609362709827281e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 902175000.0,
+ "target_villar_MPa": 13.575273324245082,
+ "mfront_calibrated_MPa": 13.575081768006122,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 151276551.05668733,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 7.715104103891054e-13,
+ "n_l0": 0.0012613494081343815,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -0.00019155623895983354,
+ "native_minus_target_MPa": -11.575624364870631
+ },
+ {
+ "dry_density_kg_m3": 1750.0,
+ "dry_density_g_cm3": 1.75,
+ "phi0": 0.3705035971223022,
+ "phi_micro_assumed": 0.36679856115107917,
+ "phi_macro_assumed": 0.003705035971223022,
+ "k0_m2": 3.2635248484816226e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 915250000.0,
+ "target_villar_MPa": 16.078773725647544,
+ "mfront_calibrated_MPa": 16.07836577146872,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 275913339.38553035,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 1.4071580380662049e-12,
+ "n_l0": 0.0012796298343392273,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -0.0004079541788222296,
+ "native_minus_target_MPa": -14.079124766273093
+ },
+ {
+ "dry_density_kg_m3": 1775.0,
+ "dry_density_g_cm3": 1.775,
+ "phi0": 0.36151079136690645,
+ "phi_micro_assumed": 0.35789568345323736,
+ "phi_macro_assumed": 0.0036151079136690647,
+ "k0_m2": 2.9468146312555356e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 928325000.0,
+ "target_villar_MPa": 19.043960172709813,
+ "mfront_calibrated_MPa": 19.043127052822083,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 505627558.19738066,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 2.578700546806641e-12,
+ "n_l0": 0.0012979102605440737,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -0.0008331198877300494,
+ "native_minus_target_MPa": -17.044311213335362
+ },
+ {
+ "dry_density_kg_m3": 1800.0,
+ "dry_density_g_cm3": 1.8,
+ "phi0": 0.3525179856115108,
+ "phi_micro_assumed": 0.3489928057553957,
+ "phi_macro_assumed": 0.0035251798561151083,
+ "k0_m2": 2.656957232567873e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "specific_surface_volumetric_m2_m3": 941400000.0,
+ "target_villar_MPa": 22.555975054319816,

```

```

+ "mfront_calibrated_MPa": 22.554338942449818,
+ "native_MPa": 1.9996489593744509,
+ "vdw_multiplier": 930588660.5966507,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 4.746002169042919e-12,
+ "n_l0": 0.0013161906867489197,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "mfront_minus_target_MPa": -0.0016361118707983735,
+ "native_minus_target_MPa": -20.556326094945366
+ }
+ ]
+*)
\ No newline at end of file
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_branch.csv b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_na
tive_dsm_micromacro_branch.csv
new file mode 100644
index 0000000000..93859e24d0
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_branch.csv
@@ -0,0 +1,180 @@
+dry_density_kg_m3,dry_density_g_cm3,phi0,phi_micro_assumed,phi_macro_assumed,k0_m2,hamaker_literature_J,hamaker_effective_J,specific_surface_mass_m2_g,n_l0,micro_solid_volume_frac
tion_reference,target_villar_MPa,native_notebook_branch_MPa,native_minus_target_MPa
+1400,0.1,4,0.49640287769784175,0.4914388489208633,0.004964028776978418,1.2264011098493141e-20,5.1e-21,5.1e-21,523,0,0.001023703867471382,0.5035971223021583,1.5038071611701885,0.05
202948126516802,-1.451776799049404
+1425,0.1,425,0.487410071942446,0.48253597122302155,0.00487410071942446,1.1205713227338159e-20,5.1e-21,5.1e-21,523,0,0.001041984293676228,0.512589928057554,1.78113357233691,0.05281
120854032262,-1.7283223637965874
+1450,0.1,45,0.4784172661870504,0.47363309352517985,0.004784172661870504,1.02345856648015e-20,5.1e-21,5.1e-21,523,0,0.0010602647198810742,0.5215827338129496,2.1096034680651288,0.05
3500764415021336,-2.0560127036501075
+1475,0.1,475,0.46942446043165464,0.4647302158273381,0.004694244604316547,9.343268757119368e-21,5.1e-21,5.1e-21,523,0,0.001078541460859203,0.5305755395683454,2.4986485357374417,0.
054368191784850896,-2.444280343952591
+1500,0.1,5,0.460431654676259,0.45582733812949644,0.00460431654676259,8.525085392608067e-21,5.1e-21,5.1e-21,523,0,0.001096825722907663,0.539568345323741,2.959439818739489,0.055143
53198496419,-2.9042962867545246
+1525,0.1,525,0.4514388489208633,0.4469244604316546,0.004514388489208633,7.77369958406512e-21,5.1e-21,5.1e-21,523,0,0.0011151059984956127,0.548561150791367,3.505208481894766,0.05
591682487191039,-3.449291657022856
+1550,0.1,55,0.4424604316546763,0.438021582733813,0.00442460431654676,7.08440345649412e-21,5.1e-21,5.1e-21,523,0,0.0011333864247004585,0.5575539568345323,4.151625731244024,0.056
68810889902,-4.094937622344122
+1575,0.1,575,0.4334532374100719,0.4291187058359712,0.0043345323741007195,6.4513614015419514e-21,5.1e-21,5.1e-21,523,0,0.001151666850905305,0.5665467625899281,4.917252797188951,0.0
57457421191983486,-4.859795375996067
+1600,0.1,6,0.424604316546763,0.4202158273381295,0.00424604316546763,5.8702609001234435e-21,5.1e-21,5.1e-21,523,0,0.0011699472771101508,0.5755395683453237,5.824073901819978,0.058
2247976065116,-5.7658491042134665
+1625,0.1,625,0.41546762589928055,0.41131294964028775,0.004154676258992806,5.33691847941765e-21,5.1e-21,5.1e-21,523,0,0.001188227703314997,0.5845323741007195,6.898127514056516,0.05
899027279933617,-6.839137241257182
+1650,0.1,65,0.4064748201438849,0.40247007194244605,0.004064748201438849,4.847510841360996e-21,5.1e-21,5.1e-21,523,0,0.0012065081295198432,0.5935251798561151,8.170254018465311,0.059
753880281995145,-8.110500138183315
+1675,0.1,675,0.3974820143884892,0.3935071942446043,0.003974820143884892,4.3985401273197395e-21,5.1e-21,5.1e-21,523,0,0.00122478855724689,0.6025179856115108,9.676981266325395,0.06
0515652476293036,-9.616465613849103
+1700,0.1,7,0.38848920863309355,0.38460431654676264,0.0038848920863309355,3.986802948098980e-21,5.1e-21,5.1e-21,523,0,0.001243068981929535,0.6115107913669064,11.461573436606095,0.0
612756207654512,-11.400297815895243
+1725,0.1,725,0.3794964028776978,0.3757014388489208,0.0037949640287769783,3.609362709827281e-21,5.1e-21,5.1e-21,523,0,0.0012613494081343815,0.6205035971223022,13.575273324245082,0.
06203381542176504,-13.513239508702904
+1750,0.1,75,0.3705035971223022,0.36679856115107917,0.003705035971223022,3.2635248484816226e-21,5.1e-21,5.1e-21,523,0,0.0012796298343392273,0.6294964028776978,16.078773725647544,0.
06279026625382632,-16.015983459393716
+1775,0.1,775,0.36151079136690645,0.35789568345323736,0.0036151079136690647,2.9468146312555356e-21,5.1e-21,5.1e-21,523,0,0.0012979102605440737,0.6384892086330936,19.043960712709813
,0.0635450014490069,-18.980415171264912
+1800,0.1,8,0.3525179856115108,0.3489928057553957,0.0035251798561151083,2.6569572325678573e-21,5.1e-21,5.1e-21,523,0,0.0013161906867489197,0.6474820143884892,22.555975054319816,0.0
6429804879705307,-22.491677005522764
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_branch_summary.json b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_de
nse_dd_native_dsm_micromacro_branch_summary.json
new file mode 100644
index 0000000000..cd976d5b6
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_branch_summary.json
@@ -0,0 +1,285 @@
+{
+  "ogs_repo_hash": "a6431b12c6",
+  "native_notebook_source_hash": "d46e1lac00",
+  "native_notebook_source_path": "/Users/vinaykumar/Documents/GitHub/ogs",
+  "native_ogs_version": "vnogs version: 6.5.7-168-gd46e1lac.dirty\n\nOmake arguments: -D0GS_BUILD_TESTING=\"OFF\" -D0GS_USE_MFRONT=\"OFF\"\\n\\n",
+  "total_initial_suction_MPa": 100.0,
+  "macro_initial_suction_MPa": 1.0,
+  "micro_initial_suction_MPa": 99.0,
+  "hamaker_literature_J": 5.1e-21,
+  "specific_surface_mass_m2_g": 523.0,
+  "results": [
+    {
+      "dry_density_kg_m3": 1400.0,
+      "dry_density_g_cm3": 1.4,
+      "phi0": 0.49640287769784175,
+      "phi_micro_assumed": 0.4914388489208633,
+      "phi_macro_assumed": 0.004964028776978418,
+      "k0_m2": 1.2264011098493141e-20,
+      "hamaker_literature_J": 5.1e-21,
+      "hamaker_effective_J": 5.1e-21,
+      "specific_surface_mass_m2_g": 523.0,
+      "n_l0": 0.001023703867471382,
+      "micro_solid_volume_fraction_reference": 0.5035971223021583,
+      "target_villar_MPa": 1.5038071611701885,
+      "native_notebook_branch_MPa": 0.05202948126516802,
+      "native_minus_target_MPa": -1.451776799049404
+    },
+    {
+      "dry_density_kg_m3": 1425.0,
+      "dry_density_g_cm3": 1.425,
+      "phi0": 0.487410071942446,
+      "phi_micro_assumed": 0.48253597122302155,
+      "phi_macro_assumed": 0.00487410071942446,
+      "k0_m2": 1.1205713227338159e-20,
+      "hamaker_literature_J": 5.1e-21,
+      "hamaker_effective_J": 5.1e-21,
+      "specific_surface_mass_m2_g": 523.0,
+      "n_l0": 0.001041984293676228,
+      "micro_solid_volume_fraction_reference": 0.512589928057554,
+      "target_villar_MPa": 1.78113357233691,
+      "native_notebook_branch_MPa": 0.0528120854032262,
+      "native_minus_target_MPa": -1.7283223637965874
+    },
+    {
+      "dry_density_kg_m3": 1450.0,
+      "dry_density_g_cm3": 1.45,
+      "phi0": 0.4784172661870504,
+      "phi_micro_assumed": 0.47363309352517985,
+      "phi_macro_assumed": 0.004784172661870504,
+      "k0_m2": 1.02345856648015e-20,
+      "hamaker_literature_J": 5.1e-21,
+      "hamaker_effective_J": 5.1e-21,
+      "specific_surface_mass_m2_g": 523.0,
+

```

```

+
+ "n_l0": 0.0010602647198810742,
+ "micro_solid_volume_fraction_reference": 0.5215827338129496,
+ "target_villar_MPa": 2.1096034680651288,
+ "native_notebook_branch_MPa": 0.053590764415021336,
+ "native_minus_target_MPa": -2.0560127036501075
+ },
+ {
+ "dry_density_kg_m3": 1475.0,
+ "dry_density_g_cm3": 1.475,
+ "phi0": 0.46942446043165464,
+ "phi_micro_assumed": 0.4647302158273381,
+ "phi_macro_assumed": 0.004694244604316547,
+ "k0_m2": 0.343268757119368e-21,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 5.1e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0010785451460859203,
+ "micro_solid_volume_fraction_reference": 0.5305755395683454,
+ "target_villar_MPa": 2.4986485357374417,
+ "native_notebook_branch_MPa": 0.054368191784850896,
+ "native_minus_target_MPa": -2.444280343952591
+ },
+ {
+ "dry_density_kg_m3": 1500.0,
+ "dry_density_g_cm3": 1.5,
+ "phi0": 0.460431654676259,
+ "phi_micro_assumed": 0.45582733812949644,
+ "phi_macro_assumed": 0.00460431654676259,
+ "k0_m2": 0.525085392608067e-21,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 5.1e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0010968255722907663,
+ "micro_solid_volume_fraction_reference": 0.539568345323741,
+ "target_villar_MPa": 2.959439818739489,
+ "native_notebook_branch_MPa": 0.05514353198496419,
+ "native_minus_target_MPa": -2.9042962867545246
+ },
+ {
+ "dry_density_kg_m3": 1525.0,
+ "dry_density_g_cm3": 1.525,
+ "phi0": 0.4514388489208633,
+ "phi_micro_assumed": 0.4469244604316546,
+ "phi_macro_assumed": 0.004514388489208633,
+ "k0_m2": 7.773969584005612e-21,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 5.1e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001151059984956127,
+ "micro_solid_volume_fraction_reference": 0.5485611510791367,
+ "target_villar_MPa": 3.505208481894766,
+ "native_notebook_branch_MPa": 0.05591682487191039,
+ "native_minus_target_MPa": -3.449291657022856
+ },
+ {
+ "dry_density_kg_m3": 1550.0,
+ "dry_density_g_cm3": 1.55,
+ "phi0": 0.44244604316546765,
+ "phi_micro_assumed": 0.438021582733813,
+ "phi_macro_assumed": 0.004424460431654676,
+ "k0_m2": 7.084403435649412e-21,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 5.1e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0011333864247004585,
+ "micro_solid_volume_fraction_reference": 0.5575539568345323,
+ "target_villar_MPa": 4.151625731244024,
+ "native_notebook_branch_MPa": 0.0566881088999022,
+ "native_minus_target_MPa": -4.094937622344122
+ },
+ {
+ "dry_density_kg_m3": 1575.0,
+ "dry_density_g_cm3": 1.575,
+ "phi0": 0.4334532374100719,
+ "phi_micro_assumed": 0.42911870580359712,
+ "phi_macro_assumed": 0.0043345323741007195,
+ "k0_m2": 6.4513614015419514e-21,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 5.1e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001151666850905305,
+ "micro_solid_volume_fraction_reference": 0.5665467625899281,
+ "target_villar_MPa": 4.917252797188951,
+ "native_notebook_branch_MPa": 0.057457421191913486,
+ "native_minus_target_MPa": -4.859795375996967
+ },
+ {
+ "dry_density_kg_m3": 1600.0,
+ "dry_density_g_cm3": 1.6,
+ "phi0": 0.4244604316546763,
+ "phi_micro_assumed": 0.4202158273381295,
+ "phi_macro_assumed": 0.004244604316546763,
+ "k0_m2": 5.8702609001234435e-21,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 5.1e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0011699472771101508,
+ "micro_solid_volume_fraction_reference": 0.5755395683453237,
+ "target_villar_MPa": 5.824073901819978,
+ "native_notebook_branch_MPa": 0.0582247976065116,
+ "native_minus_target_MPa": -5.7658491842134665
+ },
+ {
+ "dry_density_kg_m3": 1625.0,
+ "dry_density_g_cm3": 1.625,
+ "phi0": 0.41546762589928055,
+ "phi_micro_assumed": 0.41131294964028775,
+ "phi_macro_assumed": 0.004154676258992806,
+ "k0_m2": 5.33691847941765e-21,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 5.1e-21,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001188227703314997,
+ "micro_solid_volume_fraction_reference": 0.5845323741007195,
+ "target_villar_MPa": 6.898127514056516,
+ "native_notebook_branch_MPa": 0.05899027279933617,
+ "native_minus_target_MPa": -6.839137241257182

```

```

+ },
+ {
+   "dry_density_kg_m3": 1650.0,
+   "dry_density_g_cm3": 1.65,
+   "phi0": 0.4064748201438849,
+   "phi_micro_assumed": 0.40241007194244605,
+   "phi_macro_assumed": 0.004064748201438849,
+   "k0_m2": 4.84751084136096e-21,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.1e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "n_l0": 0.0012865081295198432,
+   "micro_solid_volume_fraction_reference": 0.5935251798561151,
+   "target_villar_MPa": 8.178254018465311,
+   "native_notebook_branch_MPa": 0.059753880281995145,
+   "native_minus_target_MPa": -8.110500138183315
+ },
+ {
+   "dry_density_kg_m3": 1675.0,
+   "dry_density_g_cm3": 1.675,
+   "phi0": 0.3974820143884892,
+   "phi_micro_assumed": 0.3935071942446043,
+   "phi_macro_assumed": 0.003974820143884892,
+   "k0_m2": 4.3985401273197395e-21,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.1e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "n_l0": 0.001224788555724689,
+   "micro_solid_volume_fraction_reference": 0.6025179856115108,
+   "target_villar_MPa": 9.676981266325395,
+   "native_notebook_branch_MPa": 0.060515652476293036,
+   "native_minus_target_MPa": -9.616465613849103
+ },
+ {
+   "dry_density_kg_m3": 1700.0,
+   "dry_density_g_cm3": 1.7,
+   "phi0": 0.38848920863309355,
+   "phi_micro_assumed": 0.38460431654676264,
+   "phi_macro_assumed": 0.0038848920863309355,
+   "k0_m2": 3.986802948009508e-21,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.1e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "n_l0": 0.001243068981929535,
+   "micro_solid_volume_fraction_reference": 0.6115107913669064,
+   "target_villar_MPa": 11.461573436660695,
+   "native_notebook_branch_MPa": 0.0612756207654512,
+   "native_minus_target_MPa": -11.400297815895243
+ },
+ {
+   "dry_density_kg_m3": 1725.0,
+   "dry_density_g_cm3": 1.725,
+   "phi0": 0.3794964028776978,
+   "phi_micro_assumed": 0.3757014388489208,
+   "phi_macro_assumed": 0.0037949640287769783,
+   "k0_m2": 3.609362709827281e-21,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.1e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "n_l0": 0.0012613494081343815,
+   "micro_solid_volume_fraction_reference": 0.6205835971223022,
+   "target_villar_MPa": 13.575273324245082,
+   "native_notebook_branch_MPa": 0.062033815542176504,
+   "native_minus_target_MPa": -13.513239508702904
+ },
+ {
+   "dry_density_kg_m3": 1750.0,
+   "dry_density_g_cm3": 1.75,
+   "phi0": 0.3705035971223022,
+   "phi_micro_assumed": 0.36679856115107917,
+   "phi_macro_assumed": 0.003705035971223022,
+   "k0_m2": 3.2635248484816226e-21,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.1e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "n_l0": 0.0012796298343392273,
+   "micro_solid_volume_fraction_reference": 0.6294964028776978,
+   "target_villar_MPa": 16.078773725647544,
+   "native_notebook_branch_MPa": 0.06279026625382632,
+   "native_minus_target_MPa": -16.015983459393716
+ },
+ {
+   "dry_density_kg_m3": 1775.0,
+   "dry_density_g_cm3": 1.775,
+   "phi0": 0.36151079136690645,
+   "phi_micro_assumed": 0.35789568345323736,
+   "phi_macro_assumed": 0.0036151079136690647,
+   "k0_m2": 2.9468146312555356e-21,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.1e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "n_l0": 0.0012979102605440737,
+   "micro_solid_volume_fraction_reference": 0.6384892086330936,
+   "target_villar_MPa": 19.043960172709813,
+   "native_notebook_branch_MPa": 0.06354508144490069,
+   "native_minus_target_MPa": -18.980415171264912
+ },
+ {
+   "dry_density_kg_m3": 1800.0,
+   "dry_density_g_cm3": 1.8,
+   "phi0": 0.3525179856115108,
+   "phi_micro_assumed": 0.3489928057553957,
+   "phi_macro_assumed": 0.0035251798561151083,
+   "k0_m2": 2.6569572325678573e-21,
+   "hamaker_literature_J": 5.1e-21,
+   "hamaker_effective_J": 5.1e-21,
+   "specific_surface_mass_m2_g": 523.0,
+   "n_l0": 0.0013161906867489197,
+   "micro_solid_volume_fraction_reference": 0.6474820143884892,
+   "target_villar_MPa": 22.555975054319816,
+   "native_notebook_branch_MPa": 0.06429804879705307,
+   "native_minus_target_MPa": -22.4916777005522764
+ }
+ ]
+}
\ No newline at end of file
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_branch_vs_villar.png b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_d

```

```

ense_dd_native_dsm_micromacro_branch_vs_villar.png
new file mode 100644
index 0000000000..cb08707bde
Binary files /dev/null and b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_branch_vs_villar.png differ
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration.csv b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration.csv
new file mode 100644
index 0000000000..806b00f758
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration.csv
@@ -0,0 +1,18 @@
+dry_density_kg_m3,dry_density_g_cm3,phi0,phi_micro_assumed,phi_macro_assumed,k0_m2,target_villar_MPa,native_baseline_MPa,native_calibrated_MPa,vdw_multiplier,hamaker_literature_J,hamaker_effective_J,specific_surface_mass_m2_g,n_l0,micro_solid_volume_fraction_reference,macro_suction_MPa,micro_suction_MPa,native_baseline_minus_target_MPa,native_calibrated_minus_target_MPa
+1400,0,1.4,0.49640827769784175,0.4914388489208633,0.004964028776978418,1.2264011098493141e-20,1.5038071611701805,0.05202948126516802,1.5037372863370806,99944.98611697741,5.1e-21,5.097194291965847e-16,523,0,0.001823703867471382,0.5035971223021583,1,0,99,0,-1.451776799049404,-6.987483302789421e-05
+1425,0,1.425,0.487410071942446,0.48253597122302155,0.00487410071942446,1.1205713227338159e-20,1.78113357233691,0.05281120854032262,1.7811019378747084,171309.29605577976,5.1e-21,8.736774098844768e-16,523,0,0.001041984293676228,0.512589928057554,1,0,99,0,-1.7283223637965874,-3.163446220155741e-05
+1450,0,1.45,0.4784172661870504,0.47363309352517985,0.004784172661870504,1.02345856648015e-20,2.1096834680051288,0.053590764415021336,2.109635958824076,294202.2807242468,5.1e-21,1.5004316109365808e-15,523,0,0.0010602647198810742,0.5215827338129406,1,0,99,0,-2.0560127036501075,3.24900812778851836e-05
+1475,0,1.475,0.46942446043165464,0.4647302158273381,0.004694244604316547,9.343268757119368e-21,2.4986485357374417,0.054368191784850896,2.498779160968145,506520.0146394756,5.1e-21,2.5832520746613258e-15,523,0,0.0010785451460859203,0.530575595683454,1,0,99,0,-2.444280343952591,0.0001306252307031741
+1500,0,1.5,0.460431654676259,0.45582733812949644,0.00460431654676259,8.525085392608067e-21,2.959439818739489,0.05514353198496419,2.959584575474802,874728.2174185852,5.1e-21,4.46113908834784e-15,523,0,0.0010968255722907663,0.539568345323741,1,0,99,0,-2.9042962867545246,0.00014475673531322286
+1525,0,1.525,0.4514388489208633,0.4469244604316546,0.004514388489208633,7.773969584065612e-21,3.505200481894766,0.05591682407191039,3.5050109424591684,1516059.5934533828,5.1e-21,7.731903026612252e-15,523,0,0.001115105994056127,0.5485611510791367,1,0,99,0,-3.4492916576022856,0.00019753943559752685
+1550,0,1.55,0.44244604316546765,0.438021582733813,0.004424460431654676,7.084403435649412e-21,4.151625731244024,0.0566881088999022,4.151625427987911,2641527.473775587,5.1e-21,1.3471700114215492e-14,523,0,0.0011333864247004585,0.5575539568345323,1,0,99,0,-4.004937622344122,-3.0325611266590613e-07
+1575,0,1.575,0.4334532374100719,0.4291187058359712,0.0043345323741007195,6.4513614015419514e-21,4.91725797188951,0.057457421191983486,4.917441413231121,4626862.167731534,5.1e-21,2.359690755430823e-14,523,0,0.00115666850905305,0.5665467625899281,1,0,99,0,-4.859795375906967,0.00018616042169798
+1600,0,1.6,0.4244604316546763,0.4202158273381295,0.004244604316546763,5.8702609001234435e-21,5.824073901819978,0.0582247976065116,5.824575212203727,8151765.952654096,5.1e-21,4.157400635839974e-14,523,0,0.0011699472771101508,0.5755395683453237,1,0,99,0,-5.7658491042134605,0.0005013103837487876
+1625,0,1.625,0.41546762589928055,0.41131294964028775,0.004154676258992806,5.33691847941765e-21,6.898127514056516,0.05899027279933617,6.898599185183794,14446002.597567983,5.1e-21,7.367461324759672e-14,523,0,0.001188227703314997,0.5845323741007195,1,0,99,0,-6.839137241257182,0.00047167112727741767
+1650,0,1.65,0.4064748201438849,0.40241007194244605,0.004064748201438849,4.84751084136096e-21,8.170254810465311,0.059753808281995145,8.169955154552317,25749677.27675216,5.1e-21,1.312323541095136e-13,523,0,0.0012065081295198432,0.593251798561151,1,0,99,0,-8.110500138183315,0.0002988639129934967
+1675,0,1.675,0.3974820143884892,0.3935071942446043,0.003974820143884892,4.3985401273197395e-21,9.676981266325395,0.060515652476293036,9.677897135096762,46217843.76030728,5.1e-21,2.3571100317576715e-13,523,0,0.00122478855724689,0.6025179856115100,1,0,99,0,-9.616465613849103,0.0009164687713667519
+1700,0,1.7,0.38848920863309355,0.38460431654676264,0.0038848920863309355,3.986802948090508e-21,11.461573436660695,0.0612756207654512,11.460589161928073,83345528.14100471,5.1e-21,4.2506219351912403e-13,523,0,0.001243068981292535,0.6115107913669064,1,0,99,0,-11.400297815895243,0.0009842747326214152
+1725,0,1.725,0.3794964028776978,0.3757014388489208,0.0037949640287769783,3.609362709827281e-21,13.575273324245082,0.062033815542176504,13.574638734828673,151258468.0250105,5.1e-21,7.714181069275536e-13,523,0,0.001261349408134815,0.6205084610776978,1,0,99,0,-16.015983459393716,0.000143534163529122
+1750,0,1.75,0.3705835971223022,0.36679856115107917,0.003705835971223022,3.2635248484816226e-21,16.078773725647544,0.06279026625382632,16.07891719063897,275905074.13345504,5.1e-21,1.4073479280806208e-12,523,0,0.001279629834392273,0.6294964028776978,1,0,99,0,-16.015983459393716,0.000143534163529122
+1775,0,1.775,0.36151079136690645,0.35789568345323736,0.0036151079136690647,2.9468146312555356e-21,19.043960172709813,0.06354500144940009,19.044707295924226,505790645.06090796,5.1e-21,2.5795322098106306e-12,523,0,0.0012979102605440737,0.6384892086330936,1,0,99,0,-18.908415171264912,0.0007471232144133921
+1800,0,1.8,0.3525179856115108,0.3489928057553957,0.0035251798561151083,2.6569572325678573e-21,22.555975054319816,0.06429804879705307,22.55612324694008,930876221.3900098,5.1e-21,4.74746872080905e-12,523,0,0.0013161906867489197,0.6474820143884892,1,0,99,0,-22.491677085522764,0.0001481926210651352
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration_comparison.png b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration_comparison.png
new file mode 100644
index 0000000000..b0c85a36ed
Binary files /dev/null and b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration_comparison.png differ
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration_summary.json b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration_summary.json
new file mode 100644
index 0000000000..051d26c6ad
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_calibration_summary.json
@@ -0,0 +1,373 @@
+{
+  "ogs_repo_hash": "f1acfd4b2",
+  "native_notebook_source_hash": "d46e1ac0",
+  "native_notebook_source_path": "/Users/vinaykumar/git/ogs-native-dsm-transition",
+  "native_ogs_version": "vnogs version: 6.5.7-168-gd46e1ac.dirty\n\nOake arguments:  -DOGS_BUILD_TESTING=OFF -DOGS_USE_MFRONT=OFF\n\n",
+  "total_initial_suction_MPa": 100.0,
+  "macro_initial_suction_MPa": 1.0,
+  "micro_initial_suction_MPa": 99.0,
+  "hamaker_literature_J": 5.1e-21,
+  "specific_surface_mass_m2_g": 523.0,
+  "mean_relative_error_baseline_percent": 98.68552715276913,
+  "mean_relative_error_calibrated_percent": 0.004404057285252355,
+  "max_relative_error_calibrated_percent": 0.00947060603837376,
+  "results": [
+    {
+      "dry_density_kg_m3": 1400.0,
+      "dry_density_g_cm3": 1.4,
+      "phi0": 0.49640827769784175,
+      "phi_micro_assumed": 0.4914388489208633,
+      "phi_macro_assumed": 0.004964028776978418,
+      "k0_m2": 1.2264011098493141e-20,
+      "target_villar_MPa": 1.5038071611701805,
+      "native_baseline_MPa": 0.05202948126516802,
+      "native_calibrated_MPa": 1.5037372863370806,
+      "vdw_multiplier": 99944.98611697741,
+      "hamaker_literature_J": 5.1e-21,
+      "hamaker_effective_J": 5.097194291965847e-16,
+      "specific_surface_mass_m2_g": 523.0,
+      "n_l0": 0.001823703867471382,
+      "micro_solid_volume_fraction_reference": 0.5035971223021583,
+      "macro_suction_MPa": 1.0,
+      "micro_suction_MPa": 99.0,
+      "native_baseline_minus_target_MPa": -1.451776799049404,
+      "native_calibrated_minus_target_MPa": -6.987483302789421e-05
+    },
+    {
+      "dry_density_kg_m3": 1425.0,
+      "dry_density_g_cm3": 1.425,
+      "phi0": 0.487410071942446,
+      "phi_micro_assumed": 0.48253597122302155,
+      "phi_macro_assumed": 0.00487410071942446,
+      "k0_m2": 1.1205713227338159e-20,
+      "target_villar_MPa": 1.78113357233691,
+      "native_baseline_MPa": 0.05281120854032262,
+      "native_calibrated_MPa": 1.7811019378747084,
+      "vdw_multiplier": 171309.29605577976,
+      "hamaker_literature_J": 5.1e-21,
+      "hamaker_effective_J": 8.736774098844768e-16,
+      "specific_surface_mass_m2_g": 523.0,
+      "n_l0": 0.001041984293676228,
+      "micro_solid_volume_fraction_reference": 0.512589928057554,
+      "macro_suction_MPa": 1.0,
+      "micro_suction_MPa": 99.0,
+      "native_baseline_minus_target_MPa": -1.7283223637965874,
+      "native_calibrated_minus_target_MPa": -3.163446220155741e-05
+    }
+  ]
+}

```



```

+
+ "dry_density_kg_m3": 1450.0,
+ "dry_density_g_cm3": 1.45,
+ "phi0": 0.4784172661870504,
+ "phi_micro_assumed": 0.473633009352517985,
+ "phi_macro_assumed": 0.004784172661870504,
+ "k0_m2": 1.02345856648015e-20,
+ "target_villar_MPa": 2.1096034680651288,
+ "native_baseline_MPa": 0.053590764415021336,
+ "native_calibrated_MPa": 2.1096359588824076,
+ "vdw_multiplier": 294202.2807242460,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 1.5004316316936588e-15,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0010602647198810742,
+ "micro_solid_volume_fraction_reference": 0.5215827338129496,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -2.0560127036501075,
+ "native_calibrated_minus_target_MPa": 3.2490817278851836e-05
+ },
+ {
+
+ "dry_density_kg_m3": 1475.0,
+ "dry_density_g_cm3": 1.475,
+ "phi0": 0.46942446043165464,
+ "phi_micro_assumed": 0.4647302158273381,
+ "phi_macro_assumed": 0.004694244604316547,
+ "k0_m2": 0.343268757119368e-21,
+ "target_villar_MPa": 2.4986485357374417,
+ "native_baseline_MPa": 0.054368191784850896,
+ "native_calibrated_MPa": 2.498779160968145,
+ "vdw_multiplier": 506520.0146394756,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 2.5832520746613258e-15,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0010785451460859203,
+ "micro_solid_volume_fraction_reference": 0.5305755395683454,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -2.444280343952591,
+ "native_calibrated_minus_target_MPa": 0.0001306252387031741
+ },
+ {
+
+ "dry_density_kg_m3": 1500.0,
+ "dry_density_g_cm3": 1.5,
+ "phi0": 0.460431654676259,
+ "phi_micro_assumed": 0.45582733812949644,
+ "phi_macro_assumed": 0.00460431654676259,
+ "k0_m2": 8.525085392608007e-21,
+ "target_villar_MPa": 2.959439818739489,
+ "native_baseline_MPa": 0.05514353198496419,
+ "native_calibrated_MPa": 2.959584575474802,
+ "vdw_multiplier": 874728.2174185852,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 4.46113908834784e-15,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0010968255722907663,
+ "micro_solid_volume_fraction_reference": 0.539568345323741,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -2.9042962867545246,
+ "native_calibrated_minus_target_MPa": 0.00014475673531322286
+ },
+ {
+
+ "dry_density_kg_m3": 1525.0,
+ "dry_density_g_cm3": 1.525,
+ "phi0": 0.4514388489208633,
+ "phi_micro_assumed": 0.4469244604316546,
+ "phi_macro_assumed": 0.004514388489208633,
+ "k0_m2": 7.773969584005612e-21,
+ "target_villar_MPa": 3.505208481894766,
+ "native_baseline_MPa": 0.05591602487191039,
+ "native_calibrated_MPa": 3.5050109424591684,
+ "vdw_multiplier": 1516059.5934533828,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 7.731903926612253e-15,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001151059984956127,
+ "micro_solid_volume_fraction_reference": 0.5485611510791367,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -3.449291657022856,
+ "native_calibrated_minus_target_MPa": -0.00019753943559752685
+ },
+ {
+
+ "dry_density_kg_m3": 1550.0,
+ "dry_density_g_cm3": 1.55,
+ "phi0": 0.44244604316546765,
+ "phi_micro_assumed": 0.438021582733813,
+ "phi_macro_assumed": 0.004424460431654676,
+ "k0_m2": 7.084403435649412e-21,
+ "target_villar_MPa": 4.151625731244024,
+ "native_baseline_MPa": 0.0566881088999022,
+ "native_calibrated_MPa": 4.151625427987911,
+ "vdw_multiplier": 2641527.473375587,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 1.3471790114215492e-14,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001133386427004585,
+ "micro_solid_volume_fraction_reference": 0.5575539568345323,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -4.094937622344122,
+ "native_calibrated_minus_target_MPa": -3.0325611266590613e-07
+ },
+ {
+
+ "dry_density_kg_m3": 1575.0,
+ "dry_density_g_cm3": 1.575,
+ "phi0": 0.4334532374100719,
+ "phi_micro_assumed": 0.4291187050359712,
+ "phi_macro_assumed": 0.0043345323741007195,
+ "k0_m2": 6.451361401519514e-21,
+ "target_villar_MPa": 4.917252797188951,
+ "native_baseline_MPa": 0.057457421191983486,
+ "native_calibrated_MPa": 4.917441413231121,
+ "vdw_multiplier": 4626862.167731534,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 2.3596997055430823e-14,

```

```

+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001151666850905305,
+ "micro_solid_volume_fraction_reference": 0.5665467625899281,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -4.859795375996967,
+ "native_calibrated_minus_target_MPa": 0.000188616042169798
+ },
+ {
+ "dry_density_kg_m3": 1600.0,
+ "dry_density_g_cm3": 1.6,
+ "phi0": 0.4244604316546763,
+ "phi_micro_assumed": 0.4202158273381295,
+ "phi_macro_assumed": 0.004244604316546763,
+ "k0_m2": 5.8702609001234435e-21,
+ "target_villar_MPa": 5.824073901819978,
+ "native_baseline_MPa": 0.0582247976065116,
+ "native_calibrated_MPa": 5.824575212203727,
+ "vdw_multiplier": 8151765.952654896,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 4.1574006358539974e-14,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0011699472771101508,
+ "micro_solid_volume_fraction_reference": 0.5755395683453237,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -5.7658491042134665,
+ "native_calibrated_minus_target_MPa": 0.0005013103837487876
+ },
+ {
+ "dry_density_kg_m3": 1625.0,
+ "dry_density_g_cm3": 1.625,
+ "phi0": 0.41546762589928055,
+ "phi_micro_assumed": 0.41131294964028775,
+ "phi_macro_assumed": 0.004154676258992806,
+ "k0_m2": 5.33691847941765e-21,
+ "target_villar_MPa": 6.898127514056516,
+ "native_baseline_MPa": 0.058990272799333617,
+ "native_calibrated_MPa": 6.898599185183794,
+ "vdw_multiplier": 14446002.597567983,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 7.367461324759672e-14,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001188227703314997,
+ "micro_solid_volume_fraction_reference": 0.5845323741007195,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -6.839137241257182,
+ "native_calibrated_minus_target_MPa": 0.00047167112727741767
+ },
+ {
+ "dry_density_kg_m3": 1650.0,
+ "dry_density_g_cm3": 1.65,
+ "phi0": 0.4064748201438849,
+ "phi_micro_assumed": 0.40241007194244605,
+ "phi_macro_assumed": 0.004064748201438849,
+ "k0_m2": 4.84751084136096e-21,
+ "target_villar_MPa": 8.170254018465311,
+ "native_baseline_MPa": 0.059753880281995145,
+ "native_calibrated_MPa": 8.169955154552317,
+ "vdw_multiplier": 25749677.276375216,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 1.313233541095136e-13,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0012065081295198432,
+ "micro_solid_volume_fraction_reference": 0.5935251798561151,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -8.110500138183315,
+ "native_calibrated_minus_target_MPa": -0.0002988639129934967
+ },
+ {
+ "dry_density_kg_m3": 1675.0,
+ "dry_density_g_cm3": 1.675,
+ "phi0": 0.3974820143884892,
+ "phi_micro_assumed": 0.3935071942446043,
+ "phi_macro_assumed": 0.003974820143884892,
+ "k0_m2": 4.3985401273197395e-21,
+ "target_villar_MPa": 9.676981266325395,
+ "native_baseline_MPa": 0.06051652476293836,
+ "native_calibrated_MPa": 9.677897735096762,
+ "vdw_multiplier": 46217843.76080728,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 2.3571100317756715e-13,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.00122478855724689,
+ "micro_solid_volume_fraction_reference": 0.6025179856115108,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -9.616465613849103,
+ "native_calibrated_minus_target_MPa": 0.0009164687713667519
+ },
+ {
+ "dry_density_kg_m3": 1700.0,
+ "dry_density_g_cm3": 1.7,
+ "phi0": 0.38848920863309355,
+ "phi_micro_assumed": 0.38460431654676264,
+ "phi_macro_assumed": 0.0038848920863309355,
+ "k0_m2": 3.986802948009508e-21,
+ "target_villar_MPa": 11.461573436660695,
+ "native_baseline_MPa": 0.0612756207654512,
+ "native_calibrated_MPa": 11.460589161928073,
+ "vdw_multiplier": 83345528.14100471,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 4.2506219351912403e-13,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.001243068981929535,
+ "micro_solid_volume_fraction_reference": 0.6115107913669064,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -11.400297815895243,
+ "native_calibrated_minus_target_MPa": -0.0009842747326214152
+ },
+ {
+ "dry_density_kg_m3": 1725.0,
+ "dry_density_g_cm3": 1.725,
+ "phi0": 0.3794964028776978,

```

```

+ "phi_micro_assumed": 0.37570143884892,
+ "phi_macro_assumed": 0.0037949640287769783,
+ "k0_m2": 3.609362709827281e-21,
+ "target_villar_MPa": 13.575273324245082,
+ "native_baseline_MPa": 0.06283815542176504,
+ "native_calibrated_MPa": 13.574638734828673,
+ "vdw_multiplier": 151258468.0250105,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 7.714181869275536e-13,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0012613494081343815,
+ "micro_solid_volume_fraction_reference": 0.6285035971223022,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -13.513239508072904,
+ "native_calibrated_minus_target_MPa": -0.0006345894164088151
+ },
+ {
+ "dry_density_kg_m3": 1750.0,
+ "dry_density_g_cm3": 1.75,
+ "phi0": 0.3705835971223022,
+ "phi_micro_assumed": 0.36679856115107917,
+ "phi_macro_assumed": 0.003705835971223022,
+ "k0_m2": 3.2635248484816226e-21,
+ "target_villar_MPa": 16.07873725647544,
+ "native_baseline_MPa": 0.06279026625382632,
+ "native_calibrated_MPa": 16.078917179063897,
+ "vdw_multiplier": 275950574.13345504,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 1.40734792808086208e-12,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0012796298343392273,
+ "micro_solid_volume_fraction_reference": 0.6294964028776978,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -16.015983459393716,
+ "native_calibrated_minus_target_MPa": 0.0001435434163529122
+ },
+ {
+ "dry_density_kg_m3": 1775.0,
+ "dry_density_g_cm3": 1.775,
+ "phi0": 0.36151079136690645,
+ "phi_micro_assumed": 0.35789563845323736,
+ "phi_macro_assumed": 0.003615107913669064,
+ "k0_m2": 2.9468146312555356e-21,
+ "target_villar_MPa": 19.043960172709813,
+ "native_baseline_MPa": 0.06354500144490869,
+ "native_calibrated_MPa": 19.044707295924226,
+ "vdw_multiplier": 505790645.06090796,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 2.5795322898106306e-12,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0012979102605440737,
+ "micro_solid_volume_fraction_reference": 0.6384892086330936,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -18.980415171264912,
+ "native_calibrated_minus_target_MPa": 0.000747232144133921
+ },
+ {
+ "dry_density_kg_m3": 1800.0,
+ "dry_density_g_cm3": 1.8,
+ "phi0": 0.3525179856115108,
+ "phi_micro_assumed": 0.3489928057553957,
+ "phi_macro_assumed": 0.0035251798561151083,
+ "k0_m2": 2.6509572325678573e-21,
+ "target_villar_MPa": 22.555975054319816,
+ "native_baseline_MPa": 0.06429084879705307,
+ "native_calibrated_MPa": 22.55612324694088,
+ "vdw_multiplier": 930876221.3900098,
+ "hamaker_literature_J": 5.1e-21,
+ "hamaker_effective_J": 4.747468729089905e-12,
+ "specific_surface_mass_m2_g": 523.0,
+ "n_l0": 0.0013161906067409197,
+ "micro_solid_volume_fraction_reference": 0.6474820143884892,
+ "macro_suction_MPa": 1.0,
+ "micro_suction_MPa": 99.0,
+ "native_baseline_minus_target_MPa": -22.49167700552764,
+ "native_calibrated_minus_target_MPa": 0.0001481926210651352
+ }
+ ]
+ }
+ }
\ No newline at end of file
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_vdw_multiplier.png b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_vdw_multiplier.png
se diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_vs_mfront_calibrated_comparison.csv b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_vs_mfront_calibrated_comparison.csv
new file mode 100644
index 0000000000..dda6c792bd
Binary files /dev/null and b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_dsm_micromacro_vdw_multiplier.png differ
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_vs_mfront_calibrated_comparison.csv b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_vs_mfront_calibrated_comparison.csv
new file mode 100644
index 0000000000..caf8617610
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_vs_mfront_calibrated_comparison.csv
@@ -0,0 +1,18 @@
+dry_density_kg_m3,target_villar_MPa,mfront_calibrated_MPa,native_calibrated_MPa,native_minus_mfront_MPa,abs_native_minus_mfront_MPa,mfront_vdw_multiplier,native_vdw_multiplier,mfrnt_over_natural_ratio
+1400.,0,1.503807161781085,1.5038065468524999,1.5037372863370806,-6.925971541926756e-05,6.925971541926756e-05,99961.04975655653,99944.98611697741,1.0001607248167539
+1425.,0,1.7813357323691,1.7813318756447714,1.7811019378747084,-2.993770062993252e-05,2.993770062993252e-05,171319.2960557976,1.0000583680186657
+1450.,0,2.1096034680651288,2.109599067594651,2.1096359588824076,3.689128756635194e-05,3.689128756635194e-05,294184.002508070176,294202.2807242468,0.9999378719447719
+1475.,0,2.498648537374417,2.498637692288117,2.49877916098145,0.0001416860800278453,0.0001416860800278453,506418.4683587369,506520.01436394756,0.999795216816634
+1500.,0,2.959430818739489,2.9594142426949315,2.959584575474802,0.0001783327798705523,0.0001783327798705523,874548.6904340767,874728.2174185852,0.999794669546691
+1525.,0,3.505208481894766,3.505180467835943,3.5051094254591684,-0.00013952537677441867,0.00013952537677441867,1516270.1832831167,1516059.593453828,1.000130906836836
+1550.,0,4.15162537244024,4.151490844974241,4.1516254297987911,0.00012658301367007851,0.00012658301367007851,2641230.585113023,2641527.47375587,0.9998807607358067
+1575.,0,4.917522797180951,4.916984828259917,4.9174414132131121,0.00045658497120371067,0.00045658497120371067,4625294.549728161,4626862.167731534,0.9996611919814825
+1600.,0,5.824873001819978,5.8239217011580425,5.824575212307277,0.001048206532344101,0.001048206532344101,8146393.729953618,8151765.952654896,0.9993409734369305
+1625.,0,6.89817514650516,6.898211786273423,6.898599185183794,0.0004774789130789745,0.0004774789130789745,14442267.448706342,14446802.597575983,0.9997414406624661
+1650.,0,8.170258184053117,8.176230809807218,8.169955154552317,-0.000283854348957517,0.000283854348957517,25752873.196480487,25749677.276375216,1.0001241149576738
+1
```

```

index 0000000000..c101c50f72
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_native_vs_mfront_calibrated_comparison_summary.json
@@ -0,0 +1,9 @@
+{
+  "num_points": 17,
+  "mean_abs_native_minus_mfront_MPa": 0.000540627784528373,
+  "max_abs_native_minus_mfront_MPa": 0.0017843044918635087,
+  "mean_mfront_over_native_multiplier_ratio": 0.9998774646343804,
+  "min_mfront_over_native_multiplier_ratio": 0.9993409743689305,
+  "max_mfront_over_native_multiplier_ratio": 1.000290678728802,
+  "calibration_rel_tol": 0.0001
+}
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_swelling_pressure_comparison.png b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_swelling_pressure_comparison.png
new file mode 100644
index 0000000000..897d978080
Binary files /dev/null and b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_swelling_pressure_comparison.png differ
diff --git a/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_vdw_multiplier.png b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_vdw_multiplier.png
new file mode 100644
index 0000000000..c74eafb6b3
Binary files /dev/null and b/Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/villar_dense_dd_vdw_multiplier.png differ
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/.gitignore b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/.gitignore
new file mode 100644
index 0000000000..7a60b85e14
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/.gitignore
@@ -0,0 +1,2 @@
+_pycache_/
+*.pyc
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/README.md b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/README.md
new file mode 100644
index 0000000000..b7895a04c3
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/README.md
@@ -0,0 +1,13 @@
+# BEACON WP3_BGR_EPFL Replay
+
+*This folder contains the reduced replay assets for D3.3 Annex A (BGR path
+'P2-1', stage 'A-B').*
+
+*Tracked artefacts include:
+  - replay project files (*.prj),
+  - geometry/mesh inputs (*.gml, *.vtu),
+  - analysis and plotting scripts (*.py),
+  - report-style figures and series (*.png, *.csv, *.json).
+
+*Run provenance (including git hashes) is tracked in:
+  - Tests/Data/RichardsMechanics/test_run_table.csv*
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/analyze_bgr_wp3_abprime.py b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/analyze_bgr_wp3_abprime.py
new file mode 100644
index 0000000000..46579b1d70
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/analyze_bgr_wp3_abprime.py
@@ -0,0 +1,87 @@
+#!/usr/bin/env python3
+"""Summarize the BEACON WP3_BGR_EPFL AB' benchmark outputs.
+
+*The script reads native and MFront VTU files, extracts the key stress and
+saturation observables, and emits a compact JSON payload for the report.
+"""
+
+from __future__ import annotations
+
+import argparse
+import json
+from pathlib import Path
+
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+TARGETS = {
+  "experiment_swelling_pressure_mpa_range": [3.12, 3.55],
+  "bgr_boundary_value_swelling_pressure_mpa": 2.71,
+  "initial_void_ratio_range": [0.83, 0.85],
+  "initial_total_suction_mpa_range": [90.0, 110.0],
+  "water_pressure_boundary_kpa": 20.0,
+}
+
+def load(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
+  """Load point coordinates and point data arrays from a VTU file."""
+  reader = vtk.vtkXMLUnstructuredGridReader()
+  reader.SetFileName(str(path))
+  reader.Update()
+  grid = reader.GetOutput()
+  points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+  data = {
+    grid.GetPointData().GetArrayName(i): vtk_to_numpy(grid.GetPointData().GetArray(i))
+    for i in range(grid.GetPointData().GetNumberOfArrays())
+  }
+  return points, data
+
+def boundary_mean(points: np.ndarray, values: np.ndarray, axis: int, side: str) -> np.ndarray:
+  """Average a field on the extremal boundary in the selected direction."""
+  target = points[:, axis].max() if side == "max" else points[:, axis].min()
+  mask = np.isclose(points[:, axis], target, atol=1e-10)
+  return np.asarray(values[mask]).mean(axis=0)
+
+def summarise_single(path: Path) -> dict:
+  """Build a compact metrics summary for one AB' output file."""
+  points, data = load(path)
+  sigma_top = boundary_mean(points, data["sigma"], axis=1, side="max")
+  sigma_side = boundary_mean(points, data["sigma"], axis=0, side="max")
+  summary = {
+    "path": str(path),
+    "axial_swelling_pressure_mpa": abs(float(sigma_top[1])) / 1.0e6,
+    "radial_stress_mpa": abs(float(sigma_side[0])) / 1.0e6,
+    "mean_saturation": float(np.asarray(data["saturation"]).mean()),
+  }
+  for field in ("porosity", "transport_porosity", "dry_density_solid", "swelling_stress"):
+    if field in data:

```

```

+         summary["mean_{field}"] = float(np.asarray(data[field]).mean())
+     return summary
+
+
+def main() -> None:
+     """Parse inputs and write the comparison JSON to stdout or disk."""
+     parser = argparse.ArgumentParser()
+     parser.add_argument("--native", type=Path, required=True)
+     parser.add_argument("--mfront", type=Path, required=True)
+     parser.add_argument("--json-out", type=Path)
+     args = parser.parse_args()
+
+     result = {
+         "targets": TARGETS,
+         "native": summarise_single(args.native),
+         "mfront": summarise_single(args.mfront),
+     }
+
+     text = json.dumps(result, indent=2, sort_keys=True)
+     if args.json_out:
+         args.json_out.write_text(text + "\n")
+     print(text)
+
+
+if __name__ == "__main__":
+     main()
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_domain_162e.vtu b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_domain_162e.vtu
new file mode 100644
index 0000000000..76eea790d4
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_domain_162e.vtu
@@ -0,0 +1,165 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian" header_type="UInt32" compressor="vtkZLibDataCompressor">
+ <UnstructuredGrid>
+   <Piece NumberOfPoints="100" NumberOfCells="162">
+     <PointData>
+     </PointData>
+     <CellData>
+     </CellData>
+     <Points>
+       <DataArray type="Float32" Name="Points" NumberOfComponents="3" format="ascii" RangeMin="0" RangeMax="0.0320156211988001">0 0 0 0.00194444444 0 0 0.00388888899 0 0 0.005833
3338 0 0.00777777777 0 0.0097222223000000001 0 0 0.01166666676 0 0 0.0136111115 0 0 0.0155555561 0 0 0.0175 0 0 0.0013888889375 0 0.00225491714 0.001318599375 0 0.0040882527 0 0
013642385625 0 0.0056893634000000001 0.001575601375 0 0.0080636094 0.0015076705625 0 0.0100534462 0.00136993275 0 0.0118805134 0.001460380875 0 0.0135908535 0.001225783625 0 0.01583
1914 0.0012625998125 0 0.0175 0.0012625998125 0 0.0027777777875 0 0.0018676602 0.00270511725 0 0.00399526988 0.0026286315 0 0.0058386391000000001 0.002564099375 0 0.0074664198 0 0.00
2493039375 0 0.0099608964 0.00259838125 0 0.0117837951 0.002583675 0 0.0139400016 0.0027538053125 0 0.015377103 0.00278757875 0 0.0175 0.00278757875 0 0.004166666875 0 0.0022636
103 0.004309475625 0.0037974342000000001 0.00429443725 0 0.0058458120000000001 0.0044953840625 0 0.0079277849 0.0044444529375 0 0.009695651 0.0042406079375 0 0.0116610907 0.0041051
19125 0 0.013263032 0.0039007575 0 0.0153566336 0.004000320625 0 0.0175 0.004000320625 0 0.005555555625 0 0.0017198727 0.005589749375 0 0.00396072208 0.0058228818750000001 0 0.00
5804731 0.005960145625 0 0.0074522035000000001 0.0060324487500000001 0 0.01005843475 0.005800614 0 0.0119520219 0.005634029375 0 0.0137337858 0.00539666875 0 0.0152485809 0.005214016
25 0 0.0175 0.00521401625 0 0.006944444375 0 0.00225237056 0.00708588875 0 0.0040021408000000001 0.0071606175 0 0.0056889658 0.007174669999999999 0 0.0077650853 0.0074002275 0 0.00
09601651000000002 0.0075236893750000001 0 0.0115677233 0.007736401875 0 0 0.0138577719 0.007982344375 0 0.0153644855 0.0081675368749999999 0 0.0175 0.0081675368749999999 0 0.008333333
75 0 0.00192833767 0.0083380243749999999 0.0039181473800000001 0.008133944375 0 0.0061000877 0.008186141875 0 0.0075014926 0.0083056709375000001 0 0.0094516268000000001 0.00824629499
9999999 0 0.0171853344 0.0082289256249999999 0 0.0133065891 0.008290056875 0 0.015462265 0.0082307262500000001 0 0.0175 0.0082307262500000001 0 0.0097222225000000001 0 0.00170851058
0.009564178125 0 0.0042340510800000001 0.0096164009375 0 0.0058945306 0.0097568893749999999 0 0.00769625465 0.0095839693749999999 0 0.0097972378000000002 0.0095363862500000001 0 0.01159
90861 0.0093303818749999999 0 0.013555745 0.00910627 0 0.015836926 0.0090436949999999999 0 0.0175 0.0090436949999999999 0 0.01111111125 0 0.00209516699 0.011155275625 0 0.00384699 0
.010745025 0 0.0055056918 0.01115179875 0 0.0074973955000000001 0.011019199375 0 0.0100434579 0.01095711875 0 0.0115704141 0.011052063125 0 0.0134499344 0.010986158125 0 0.01542079
0.01108382 0 0.0175 0.01108382 0 0.0125 0 0.00194444444 0.0125 0 0.00388888899 0.0125 0 0.005833338 0.0125 0 0.0077777777 0.0125 0 0.0097222223000000001 0.0125 0 0.01166666676 0 0
0125 0 0.013611115 0.0125 0 0.0155555561 0.0125 0 0.0175 0.0125 0<InformationKey name="L2_NORM_RANGE" location="vtkDataArray" length="2">
+   <Value index="0">
+     0
+   </Value>
+   <Value index="1">
+     0.032015621199
+   </Value>
+ </InformationKey>
+ </DataArray>
+ </Points>
+ <Cells>
+   <DataArray type="Int64" Name="connectivity" format="ascii" RangeMin="0" RangeMax="99">
+     19 18 9 12 11 2
+     15 5 6 22 11 12
+     11 10 1 11 1 2
+     12 2 3 13 3 4
+     14 13 4 15 14 5
+     16 15 6 17 7 8
+     10 0 1 13 12 3
+     32 31 22 14 4 5
+     21 10 11 26 25 15
+     22 21 11 17 16 6
+     22 12 13 17 6 7
+     24 23 13 18 17 8
+     25 24 14 18 8 9
+     26 16 17 29 18 19
+     26 15 16 28 27 17
+     21 20 18 23 22 13
+     42 31 32 24 13 14
+     31 30 21 25 14 15
+     31 21 22 36 35 25
+     33 22 23 27 26 17
+     33 32 22 28 17 18
+     34 24 25 29 28 18
+     35 34 25 36 25 26
+     38 37 27 30 20 21
+     33 23 24 52 51 42
+     34 33 24 41 30 31
+     46 36 37 42 41 31
+     37 36 26 42 32 33
+     37 26 27 44 33 34
+     38 27 28 45 44 34
+     39 38 28 46 45 36
+     39 28 29 45 35 36
+     47 46 37 41 40 30
+     43 42 33 62 51 52
+     44 43 33 51 50 41
+     45 34 35 51 41 42
+     56 45 46 52 42 43
+     47 37 38 54 53 44
+     48 47 38 55 44 45
+     49 48 38 56 55 45
+     49 38 39 57 46 47
+     59 58 49 57 46 47
+     50 40 41 53 52 43
+     72 61 62 53 43 44
+     61 50 51 55 54 44
+     62 61 51 76 66 67
+     62 52 53 57 47 48
+     63 53 54 58 57 48

```



```

+         0 0.0125 0
+         0,0175 0.0125 0
+
+       </DataArray>
+     </Points>
+     <Cells>
+       <DataArray type="Int64" Name="connectivity" format="ascii">0 1 3 2 2 3 5 4</DataArray>
+       <DataArray type="Int64" Name="offsets" format="ascii">4 8</DataArray>
+       <DataArray type="UInt8" Name="types" format="ascii">9 9</DataArray>
+     </Cells>
+   </Piece>
+ </UnstructuredGrid>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_geometry.gml b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_geometry.gml
new file mode 100644
index 0000000000..048ff1841d
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_epfl_geometry.gml
@@ -0,0 +1,17 @@
+<?xml version="1.0" encoding="ISO-8859-1"?">
+<?xml-stylesheet type="text/xml" href="OpenGeoSysGLI.xsl"?>
+<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+  <name>bgr_wp3_epfl_geometry</name>
+  <points>
+    <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
+    <point id="1" x="0.0" y="0.0125" z="0.0"/>
+    <point id="2" x="0.0175" y="0.0" z="0.0"/>
+    <point id="3" x="0.0175" y="0.0125" z="0.0"/>
+  </points>
+  <polyline>
+    <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt></polyline>
+    <polyline id="1" name="right"><pnt>2</pnt><pnt>3</pnt></polyline>
+    <polyline id="2" name="bottom"><pnt>0</pnt><pnt>2</pnt></polyline>
+    <polyline id="3" name="top"><pnt>1</pnt><pnt>3</pnt></polyline>
+  </polyline>
+</OpenGeoSysGLI>
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_fig4_5b_like.png b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_fig4_5b_like.png
new file mode 100644
index 0000000000..a76b39f608
Binary files /dev/null and b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_fig4_5b_like.png differ
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_fig4_6_like.png b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_fig4_6_like.png
new file mode 100644
index 0000000000..3a4012d00d
Binary files /dev/null and b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_fig4_6_like.png differ
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_mfront.prj b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_mfront.prj
new file mode 100644
index 0000000000..acec393484
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_mfront.prj
@@ -0,0 +1,501 @@
+<?xml version="1.0" encoding="ISO-8859-1"?">
+<OpenGeoSysProject>
+  <mesh axially_symmetric="true">bgr_wp3_epfl_domain_2e.vtu</mesh>
+  <geometry>bgr_wp3_epfl_geometry.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <constitutive_relation>
+        <type>MFrontRichardsMechanics</type>
+        <behaviour>RichardsMechanicsDSM<MicroMacroBridge_MCC</behaviour>
+        <material_properties>
+          <material_property name="YoungModulus" parameter="YoungModulus"/>
+          <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+          <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+          <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+          <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+          <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+          <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+          <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+          <material_property name="BubblePressure" parameter="BubblePressure"/>
+          <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+          <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode"/>
+          <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode"/>
+          <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention"/>
+          <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope"/>
+          <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient"/>
+          <material_property name="MacroViscosity" parameter="MacroViscosity"/>
+          <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro"/>
+          <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro"/>
+          <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid"/>
+          <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA"/>
+          <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB"/>
+          <material_property name="HamakerConstant" parameter="HamakerConstant"/>
+          <material_property name="SpecificSurface" parameter="SpecificSurface"/>
+          <material_property name="AreaFactorTuller" parameter="AreaFactorTuller"/>
+          <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller"/>
+          <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize"/>
+          <material_property name="SurfaceTension" parameter="SurfaceTension"/>
+          <material_property name="InitialPorosity" parameter="phi0"/>
+        </material_properties>
+        <initial_values>
+          <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+          <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+          <state_variable name="n_l" parameter="n_l0"/>
+          <state_variable name="rho_lr" parameter="rho_LR0"/>
+          <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+        </initial_values>
+        <constitutive_relation>
+          <process_variables>
+            <pressure>pressure</pressure>
+            <displacement>displacement</displacement>
+          </process_variables>
+          <secondary_variables>
+            <secondary_variable name="ElasticStrain"/>
+            <secondary_variable name="EquivalentPlasticStrain"/>
+            <secondary_variable name="PreConsolidationPressure"/>
+            <secondary_variable name="PlasticVolumetricStrain"/>
+            <secondary_variable name="sigma"/>
+            <secondary_variable name="swelling_stress"/>
+            <secondary_variable name="epsilon"/>
+            <secondary_variable name="velocity"/>
+            <secondary_variable name="saturation"/>
+            <secondary_variable name="porosity"/>
+            <secondary_variable name="transport_porosity"/>

```

```

+         <secondary_variable name="dry_density_solid"/>
+         <secondary_variable name="VolumeRatio"/>
+         <secondary_variable name="n_L"/>
+         <secondary_variable name="rho_LR"/>
+         <secondary_variable name="epsilon_sw"/>
+         <secondary_variable name="phi_m"/>
+         <secondary_variable name="phi_M"/>
+         <secondary_variable name="phi"/>
+         <secondary_variable name="n_S"/>
+         <secondary_variable name="n_L"/>
+         <secondary_variable name="rho_LR"/>
+         <secondary_variable name="omega_L"/>
+         <secondary_variable name="mu_LR"/>
+         <secondary_variable name="rho_l_hat"/>
+         <secondary_variable name="delta_epsilon_sw"/>
+         <secondary_variable name="sigma_S"/>
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2700.0</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>transport_porosity</name>
+         <type>TransportPorosityFromMassBalance</type>
+         <initial_porosity>phi_tr0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>9e-19</value>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>Constant</type>
+         <value>1</value>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.4189</exponent>
+         <p_b>1e6</p_b>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <abstols>5e-8 1e-13 1e-13</abstols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>IterationNumberBasedTimeStepping</type>

```



```

+         <t_initial>0.0</t_initial>
+         <t_end>2e5</t_end>
+         <initial_dt>10</initial_dt>
+         <minimum_dt>1e-6</minimum_dt>
+         <maximum_dt>2e4</maximum_dt>
+         <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+         <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+     </time_stepping>
+ </process>
+ </processes>
+ <output>
+     <type>VTK</type>
+     <prefix>bgr_vp3_p2_1_abprime_mfront</prefix>
+     <suffix>_t_{:time}</suffix>
+     <fixed_output_times>1e3 1e4 5e4 1e5 2e5</fixed_output_times>
+     <variables>
+         <variable>pressure</variable>
+         <variable>displacement</variable>
+         <variable>saturation</variable>
+         <variable>porosity</variable>
+         <variable>transport_porosity</variable>
+         <variable>dry_density_solid</variable>
+         <variable>swelling_stress</variable>
+         <variable>sigma</variable>
+         <variable>EquivalentPlasticStrain</variable>
+         <variable>PreConsolidationPressure</variable>
+         <variable>PlasticVolumetricStrain</variable>
+         <variable>VolumeRatio</variable>
+         <variable>n_l</variable>
+         <variable>rho_LR</variable>
+         <variable>epsilon_sw</variable>
+         <variable>phi_m</variable>
+         <variable>phi_M</variable>
+         <variable>phi</variable>
+         <variable>n_S</variable>
+         <variable>n_L</variable>
+         <variable>rho_LR</variable>
+         <variable>omega_l</variable>
+         <variable>mu_LR</variable>
+         <variable>rho_l_hat</variable>
+         <variable>delta_epsilon_sw</variable>
+         <variable>sigma_S</variable>
+     </variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>-2e5</expression>
+         <expression>-2e5</expression>
+         <expression>-2e5</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>180e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.2</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.4594</value>
+     </parameter>
+     <parameter>
+         <name>phi_tr0</name>
+         <type>Constant</type>
+         <value>0.1491</value>
+     </parameter>
+     <parameter>
+         <name>CriticalStateLineSlope</name>
+         <type>Constant</type>
+         <value>0.24</value>
+     </parameter>
+     <parameter>
+         <name>SwellingLineSlope</name>
+         <type>Constant</type>
+         <value>0.12</value>
+     </parameter>
+     <parameter>
+         <name>VirginConsolidationLineSlope</name>
+         <type>Constant</type>
+         <value>0.15</value>
+     </parameter>
+     <parameter>
+         <name>InitialPreConsolidationPressure</name>
+         <type>Constant</type>
+         <value>2e6</value>
+     </parameter>
+     <parameter>
+         <name>InitialVolumeRatio</name>
+         <type>Constant</type>
+         <value>1.849796522382538</value>
+     </parameter>
+     <parameter>
+         <name>ResidualLiquidSaturation</name>
+         <type>Constant</type>
+         <value>0</value>
+     </parameter>
+     <parameter>
+         <name>ResidualGasSaturation</name>
+         <type>Constant</type>
+         <value>0</value>
+     </parameter>
+     <parameter>
+         <name>BubblePressure</name>
+         <type>Constant</type>
+         <value>1e6</value>
+     </parameter>
+     <parameter>
+         <name>VanGenuchtenExponent_m</name>
+         <type>Constant</type>

```

```

+         <value>0.4109</value>
+     </parameter>
+ <parameter>
+     <name>MacroSaturationLawMode</name>
+     <type>Constant</type>
+     <value>0</value>
+ </parameter>
+ <parameter>
+     <name>MicroStateLocalSolveMode</name>
+     <type>Constant</type>
+     <value>1</value>
+ </parameter>
+ <parameter>
+     <name>MicroPotentialConvention</name>
+     <type>Constant</type>
+     <value>1</value>
+ </parameter>
+ <parameter>
+     <name>MicroSwellingStrainSlope</name>
+     <type>Constant</type>
+     <value>0.15606235939756805</value>
+ </parameter>
+ <parameter>
+     <name>MassExchangeCoefficient</name>
+     <type>Constant</type>
+     <value>5e-15</value>
+ </parameter>
+ <parameter>
+     <name>MacroViscosity</name>
+     <type>Constant</type>
+     <value>1e-3</value>
+ </parameter>
+ <parameter>
+     <name>ReferenceLiquidDensityMacro</name>
+     <type>Constant</type>
+     <value>1000.0</value>
+ </parameter>
+ <parameter>
+     <name>ReferenceLiquidDensityMicro</name>
+     <type>Constant</type>
+     <value>1e-6</value>
+ </parameter>
+ <parameter>
+     <name>ReferenceDensitySolid</name>
+     <type>Constant</type>
+     <value>2780.0</value>
+ </parameter>
+ <parameter>
+     <name>MicroLiquidDensityA</name>
+     <type>Constant</type>
+     <value>1.0</value>
+ </parameter>
+ <parameter>
+     <name>MicroLiquidDensityB</name>
+     <type>Constant</type>
+     <value>1.0</value>
+ </parameter>
+ <parameter>
+     <name>HamakerConstant</name>
+     <type>Constant</type>
+     <value>6.109794683334988e-16</value>
+ </parameter>
+ <parameter>
+     <name>SpecificSurface</name>
+     <type>Constant</type>
+     <value>523.0</value>
+ </parameter>
+ <parameter>
+     <name>AreaFactorTuller</name>
+     <type>Constant</type>
+     <value>1.0</value>
+ </parameter>
+ <parameter>
+     <name>PoreAreaShapeFactorTuller</name>
+     <type>Constant</type>
+     <value>0.8584073464102069</value>
+ </parameter>
+ <parameter>
+     <name>CharacteristicPoreSize</name>
+     <type>Constant</type>
+     <value>1e-5</value>
+ </parameter>
+ <parameter>
+     <name>SurfaceTension</name>
+     <type>Constant</type>
+     <value>0.0715</value>
+ </parameter>
+ <parameter>
+     <name>n_l0</name>
+     <type>Constant</type>
+     <value>0.0539922</value>
+ </parameter>
+ <parameter>
+     <name>rho_lR0</name>
+     <type>Constant</type>
+     <value>1000.000001</value>
+ </parameter>
+ <parameter>
+     <name>epsilon_sw0</name>
+     <type>Constant</type>
+     <value>0.0</value>
+ </parameter>
+ <parameter>
+     <name>displacement0</name>
+     <type>Constant</type>
+     <values>0 0</values>
+ </parameter>
+ <parameter>
+     <name>pressure_ic</name>
+     <type>Constant</type>
+     <value>-100e6</value>
+ </parameter>
+ <parameter>
+     <name>dirichlet0</name>
+     <type>Constant</type>
+     <value>0</value>
+ </parameter>

```

```

+       <parameter>
+         <name>water_pressure</name>
+         <type>Constant</type>
+         <value>20e3</value>
+       </parameter>
+     </parameters>
+     <process_variables>
+       <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+           <boundary_condition>
+             <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+             <geometry>left</geometry>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>dirichlet0</parameter>
+           </boundary_condition>
+           <boundary_condition>
+             <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+             <geometry>right</geometry>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>dirichlet0</parameter>
+           </boundary_condition>
+           <boundary_condition>
+             <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+             <geometry>bottom</geometry>
+             <type>Dirichlet</type>
+             <component>1</component>
+             <parameter>dirichlet0</parameter>
+           </boundary_condition>
+           <boundary_condition>
+             <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+             <geometry>top</geometry>
+             <type>Dirichlet</type>
+             <component>1</component>
+             <parameter>dirichlet0</parameter>
+           </boundary_condition>
+         </boundary_conditions>
+       </process_variable>
+       <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ics</initial_condition>
+         <boundary_conditions>
+           <boundary_condition>
+             <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+             <geometry>top</geometry>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>water_pressure</parameter>
+           </boundary_condition>
+           <boundary_condition>
+             <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+             <geometry>bottom</geometry>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>water_pressure</parameter>
+           </boundary_condition>
+         </boundary_conditions>
+       </process_variable>
+     </process_variables>
+     <nonlinear_solvers>
+       <nonlinear_solver>
+         <name>nonlinear_solver</name>
+         <type>Newton</type>
+         <max_iter>80</max_iter>
+         <linear_solver>general_linear_solver</linear_solver>
+       </nonlinear_solver>
+     </nonlinear_solvers>
+     <linear_solvers>
+       <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+           <solver_type>SparseLU</solver_type>
+           <scaling>true</scaling>
+         </eigen>
+       </linear_solver>
+     </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_native.prj b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_native.prj
new file mode 100644
index 0000000000..9c279f6316
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_native.prj
@@ -0,0 +1,356 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+  <mesh axially_symmetric="true">bgr_wp3_epfl_domain_2e.vtu</mesh>
+  <geometry>bgr_wp3_epfl_geometry.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <micro_porosity>
+        <mass_exchange_coefficient>5e-15</mass_exchange_coefficient>
+      </micro_porosity>
+      <nonlinear_solver>
+        <maximum_iterations>100</maximum_iterations>
+        <residuum_tolerance>1e-8</residuum_tolerance>
+        <increment_tolerance>1e-20</increment_tolerance>
+      </nonlinear_solver>
+    </process>
+    <constitutive_relation>
+      <type>MFront</type>
+      <behaviour>ModCamClay_semiExp_constE</behaviour>
+      <material_properties>
+        <material_property name="YoungModulus" parameter="YoungModulus"/>
+        <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+        <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+        <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+        <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+        <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+      </material_properties>
+    </constitutive_relation>
+  </processes>
+</OpenGeoSysProject>

```

```

+      </material_properties>
+      <initial_values>
+        <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+        <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+      </initial_values>
+    </constitutive_relation>
+    <process_variables>
+      <pressure>pressure</pressure>
+      <displacement>displacement</displacement>
+    </process_variables>
+    <secondary_variables>
+      <secondary_variable name="ElasticStrain"/>
+      <secondary_variable name="EquivalentPlasticStrain"/>
+      <secondary_variable name="PreConsolidationPressure"/>
+      <secondary_variable name="PlasticVolumetricStrain"/>
+      <secondary_variable name="sigma"/>
+      <secondary_variable name="swelling_stress"/>
+      <secondary_variable name="epsilon"/>
+      <secondary_variable name="velocity"/>
+      <secondary_variable name="saturation"/>
+      <secondary_variable name="porosity"/>
+      <secondary_variable name="transport_porosity"/>
+      <secondary_variable name="dry_density_solid"/>
+      <secondary_variable name="VolumeRatio"/>
+      <secondary_variable name="micro_saturation"/>
+      <secondary_variable name="micro_pressure"/>
+    </secondary_variables>
+    <specific_body_force>0 0</specific_body_force>
+    <initial_stress>sigma0</initial_stress>
+    <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+    <mass_lumping>true</mass_lumping>
+  </process>
+</processes>
+<media>
+  <medium>
+    <phases>
+      <phase>
+        <type>AqueousLiquid</type>
+        <properties>
+          <property>
+            <name>viscosity</name>
+            <type>Constant</type>
+            <value>1e-3</value>
+          </property>
+          <property>
+            <name>density</name>
+            <type>Constant</type>
+            <value>1e3</value>
+          </property>
+        </properties>
+      </phase>
+      <phase>
+        <type>Solid</type>
+        <properties>
+          <property>
+            <name>density</name>
+            <type>Constant</type>
+            <value>2700.0</value>
+          </property>
+          <property>
+            <name>swelling_stress_rate</name>
+            <type>SaturationDependentSwelling</type>
+            <swelling_pressures>4e6 4e6 4e6</swelling_pressures>
+            <exponents>1 1 1</exponents>
+            <lower_saturation_limit>0</lower_saturation_limit>
+            <upper_saturation_limit>1</upper_saturation_limit>
+          </property>
+        </properties>
+      </phase>
+    </phases>
+    <properties>
+      <property>
+        <name>biot_coefficient</name>
+        <type>Constant</type>
+        <value>1.0</value>
+      </property>
+      <property>
+        <name>permeability</name>
+        <type>Constant</type>
+        <value>9e-19</value>
+      </property>
+      <property>
+        <name>transport_porosity</name>
+        <type>TransportPorosityFromMassBalance</type>
+        <initial_porosity>phi_tr0</initial_porosity>
+        <minimal_porosity>0</minimal_porosity>
+        <maximal_porosity>1</maximal_porosity>
+      </property>
+      <property>
+        <name>porosity</name>
+        <type>PorosityFromMassBalance</type>
+        <initial_porosity>phi0</initial_porosity>
+        <minimal_porosity>0</minimal_porosity>
+        <maximal_porosity>1</maximal_porosity>
+      </property>
+      <property>
+        <name>reference_temperature</name>
+        <type>Constant</type>
+        <value>293.15</value>
+      </property>
+      <property>
+        <name>relative_permeability</name>
+        <type>Constant</type>
+        <value>1</value>
+      </property>
+      <property>
+        <name>saturation</name>
+        <type>SaturationVanGenuchten</type>
+        <residual_liquid_saturation>0</residual_liquid_saturation>
+        <residual_gas_saturation>0</residual_gas_saturation>
+        <exponent>0.4189</exponent>
+        <p_b>1e6</p_b>
+      </property>
+      <property>
+        <name>saturation_micro</name>
+        <type>SaturationVanGenuchten</type>
+        <residual_liquid_saturation>0</residual_liquid_saturation>

```

```

+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.5181</exponent>
+         <p_b>2e7</p_b>
+     </property>
+     <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+     </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+             <convergence_criterion>
+                 <type>PerComponentDeltaX</type>
+                 <norm_type>NORM2</norm_type>
+                 <abstols>5e-8 1e-13 1e-13</abstols>
+             </convergence_criterion>
+             <time_discretization>
+                 <type>BackwardEuler</type>
+             </time_discretization>
+             <time_stepping>
+                 <type>IterationNumberBasedTimeStepping</type>
+                 <t_initial>0.0</t_initial>
+                 <t_end>2e5</t_end>
+                 <initial_dt>10</initial_dt>
+                 <minimum_dt>1e-6</minimum_dt>
+                 <maximum_dt>2e4</maximum_dt>
+                 <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+                 <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+             </time_stepping>
+         </process>
+     </processes>
+ </output>
+     <type>VTK</type>
+     <prefix>bgr_wp3_p2_1_abprime_native</prefix>
+     <suffix>_t(:time)</suffix>
+     <fixed_output_times>1e3 1e4 5e4 1e5 2e5</fixed_output_times>
+     <variables>
+         <variable>pressure</variable>
+         <variable>displacement</variable>
+         <variable>saturation</variable>
+         <variable>micro_pressure</variable>
+         <variable>micro_saturation</variable>
+         <variable>swelling_stress</variable>
+         <variable>sigma</variable>
+         <variable>porosity</variable>
+         <variable>transport_porosity</variable>
+         <variable>dry_density_solid</variable>
+         <variable>EquivalentPlasticStrain</variable>
+         <variable>PreConsolidationPressure</variable>
+         <variable>PlasticVolumetricStrain</variable>
+         <variable>VolumeRatio</variable>
+     </variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>-2e5</expression>
+         <expression>-2e5</expression>
+         <expression>-2e5</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>180e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.2</value>
+     </parameter>
+     <parameter>
+         <name>CriticalStateLineSlope</name>
+         <type>Constant</type>
+         <value>0.24</value>
+     </parameter>
+     <parameter>
+         <name>SwellingLineSlope</name>
+         <type>Constant</type>
+         <value>0.12</value>
+     </parameter>
+     <parameter>
+         <name>VirginConsolidationLineSlope</name>
+         <type>Constant</type>
+         <value>0.15</value>
+     </parameter>
+     <parameter>
+         <name>InitialPreConsolidationPressure</name>
+         <type>Constant</type>
+         <value>2e6</value>
+     </parameter>
+     <parameter>
+         <name>InitialVolumeRatio</name>
+         <type>Constant</type>
+         <value>1.849796522382538</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.4594</value>
+     </parameter>
+     <parameter>
+         <name>phi_tr0</name>
+         <type>Constant</type>
+         <value>0.1491</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>

```

```

+
+ </parameter>
+
+ <parameter>
+
+ <name>pressure_ic</name>
+
+ <type>Constant</type>
+
+ <value>-100e6</value>
+
+ </parameter>
+
+ <parameter>
+
+ <name>dirichlet0</name>
+
+ <type>Constant</type>
+
+ <value>0</value>
+
+ </parameter>
+
+ <parameter>
+
+ <name>water_pressure</name>
+
+ <type>Constant</type>
+
+ <value>20e3</value>
+
+ </parameter>
+
+ </parameters>
+
+ <process_variables>
+
+ <process_variable>
+
+ <name>displacement</name>
+
+ <components>2</components>
+
+ <order>1</order>
+
+ <initial_condition>displacement0</initial_condition>
+
+ <boundary_conditions>
+
+ <boundary_condition>
+
+ <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+
+ <geometry>left</geometry>
+
+ <type>Dirichlet</type>
+
+ <component>0</component>
+
+ <parameter>dirichlet0</parameter>
+
+ </boundary_condition>
+
+ <boundary_condition>
+
+ <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+
+ <geometry>right</geometry>
+
+ <type>Dirichlet</type>
+
+ <component>0</component>
+
+ <parameter>dirichlet0</parameter>
+
+ </boundary_condition>
+
+ <boundary_condition>
+
+ <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+
+ <geometry>bottom</geometry>
+
+ <type>Dirichlet</type>
+
+ <component>1</component>
+
+ <parameter>dirichlet0</parameter>
+
+ </boundary_condition>
+
+ <boundary_condition>
+
+ <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+
+ <geometry>top</geometry>
+
+ <type>Dirichlet</type>
+
+ <component>1</component>
+
+ <parameter>dirichlet0</parameter>
+
+ </boundary_condition>
+
+ </boundary_conditions>
+
+ </process_variable>
+
+ <process_variable>
+
+ <name>pressure</name>
+
+ <components>1</components>
+
+ <order>1</order>
+
+ <initial_condition>pressure_ic</initial_condition>
+
+ <boundary_conditions>
+
+ <boundary_condition>
+
+ <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+
+ <geometry>top</geometry>
+
+ <type>Dirichlet</type>
+
+ <component>0</component>
+
+ <parameter>water_pressure</parameter>
+
+ </boundary_condition>
+
+ <boundary_condition>
+
+ <geometrical_set>bgr_wp3_epfl_geometry</geometrical_set>
+
+ <geometry>bottom</geometry>
+
+ <type>Dirichlet</type>
+
+ <component>0</component>
+
+ <parameter>water_pressure</parameter>
+
+ </boundary_condition>
+
+ </boundary_conditions>
+
+ </process_variable>
+
+ </process_variables>
+
+ <nonlinear_solvers>
+
+ <nonlinear_solver>
+
+ <name>nonlinear_solver</name>
+
+ <type>Newton</type>
+
+ <max_iter>80</max_iter>
+
+ <linear_solver>general_linear_solver</linear_solver>
+
+ </nonlinear_solver>
+
+ </nonlinear_solvers>
+
+ <linear_solvers>
+
+ <linear_solver>
+
+ <name>general_linear_solver</name>
+
+ <eigen>
+
+ <solver_type>SparseLU</solver_type>
+
+ <scaling>true</scaling>
+
+ </eigen>
+
+ </linear_solver>
+
+ </linear_solvers>
+
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report.json b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report.json
new file mode 100644
index 0000000000..69a5ffd5cd
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report.json
@@ -0,0 +1,38 @@
+{
+  "mfront": {
+    "axial_swelling_pressure_mpa": 0.7859139611933248,
+    "mean_dry_density_solid": 1502.8376146942885,
+    "mean_porosity": 0.45941092996611177,
+    "mean_saturation": 0.9999999999999999,
+    "mean_swelling_stress": -439358.9290193177,
+    "mean_transport_porosity": 0.14911092996611197,
+    "path": "transient-run:mfront:bgr_wp3_p2_1_abprime_mfront_t_200000.000000.vtu",
+    "radial_stress_mpa": 0.7858119053590902
+  },
+  "native": {
+    "axial_swelling_pressure_mpa": 1.0103784801106894,
+    "mean_dry_density_solid": 1502.8393244518954,
+    "mean_porosity": 0.45941031494536116,
+    "mean_saturation": 0.9999999999999999,

```

```
+ "mean_swelling_stress": -607637.5610665366,
+ "mean_transport_porosity": 0.14797127160151743,
+ "path": "transient-run:native:bgr_wp3_p2_1_abprime_native_t_200000.000000.vtu",
+ "radial_stress_mpa": 1.0101834147550586
+ },
+ "targets": {
+ "bgr_boundary_value_swelling_pressure_mpa": 2.71,
+ "experiment_swelling_pressure_mpa_range": [
+ 3.12,
+ 3.55
+ ],
+ "initial_total_suction_mpa_range": [
+ 90.0,
+ 110.0
+ ],
+ "initial_void_ratio_range": [
+ 0.83,
+ 0.85
+ ],
+ "water_pressure_boundary_kpa": 20.0
+ }
+ })
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report_style_panel.png b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report_style_panel.png
new file mode 100644
index 0000000000..0dad39db05
Binary files /dev/null and b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report_style_panel.png differ
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report_style_series.csv b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report_style_series.csv
new file mode 100644
index 0000000000..db76ef92ca
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/bgr_wp3_p2_1_abprime_report_style_series.csv
@@ -0,0 +1,99 @@
+series,time_s,time_min,axial_swelling_pressure_mpa,vertical_stress_mpa,void_ratio,mean_saturation
+Native 0G5,0.0,0.0,0.19999999999999996,0.19999999999999996,0.8497965223825373,0.04026417585943573
+Native 0G5,60.0,1.0,0.20038521555240799,0.20038521555240799,0.8497965223825373,0.27132878800318706
+Native 0G5,71.302723,1.1883787166666666,0.20045776020092254,0.20045776020092254,0.8497965223825373,0.2874457193547989
+Native 0G5,84.734639,1.4122439833333333,0.2005439611900631,0.2005439611900631,0.8497965223825373,0.304959661747082
+Native 0G5,100.696842,1.6782807,0.2006463879476581,0.2006463879476581,0.8497965223825373,0.3240149314476142
+Native 0G5,119.665984,1.994330666666665,0.2007680905141062,0.2007680905141062,0.8497965223825373,0.3445340809502515
+Native 0G5,142.20851,2.3701418333333333,0.20091269215269056,0.20091269215269056,0.8497965223825373,0.3665620869398885
+Native 0G5,168.997567,2.816626116666667,0.20108449396447925,0.20108449396447925,0.8497965223825373,0.39005517151896524
+Native 0G5,200.833112,3.3472185333333333,0.20128860530239218,0.20128860530239218,0.8497965223825373,0.41511744903620595
+Native 0G5,238.665797,3.977763283333333,0.2015310893085823,0.2015310893085823,0.8497965223825373,0.44172216749974175
+Native 0G5,283.625355,4.727089250000001,0.20181915023707392,0.20181915023707392,0.8497965223825373,0.4700431140553642
+Native 0G5,337.454337,5.617572283333333,0.20216132662073918,0.20216132662073918,0.8497965223825373,0.4999189298839918
+Native 0G5,400.548202,6.675803666666667,0.2025677478909867,0.2025677478909867,0.8497965223825373,0.531181161826416
+Native 0G5,476.00296,7.933826666666666,0.20305842754468275,0.20305842754468275,0.8497965223825373,0.5637093689904419
+Native 0G5,565.671789,9.42786315,0.20362360787674594,0.20362360787674594,0.8497965223825373,0.5973170470694734
+Native 0G5,672.232317,11.20387195,0.20430414824523044,0.20430414824523044,0.8497965223825373,0.6316610209333667
+Native 0G5,798.866582,13.314443033333333,0.20511201591793105,0.20511201591793105,0.8497965223825373,0.6664125810098437
+Native 0G5,949.356847,15.822600783333334,0.20607095262454853,0.20607095262454853,0.8497965223825373,0.70159083885136456
+Native 0G5,1128.194526,18.8032421,0.20720888174497973,0.20720888174497973,0.8497965223825373,0.7364761596957258
+Native 0G5,1340.722368,22.3453728,0.208558784466956,0.208558784466956,0.8497965223825373,0.770460537652007
+Native 0G5,1593.285933,26.554765550000003,0.210159657895001,0.210159657895001,0.8497965223825373,0.8031527632461906
+Native 0G5,1893.4271,31.557118333333335,0.21205742789354798,0.21205742789354798,0.8497965223825373,0.834164642324247
+Native 0G5,2258.108477,37.501807950000001,0.21430650406491547,0.21430650406491547,0.8497965223825373,0.8633703104005939
+Native 0G5,2673.981034,44.56635866666664,0.2166967998203081,0.2166967998203081,0.8497965223825373,0.890619246277638
+Native 0G5,3177.702163,52.961702716666665,0.22012164491980976,0.22012164491980976,0.8497965223825373,0.9144929447651959
+Native 0G5,3776.313633,62.938505500000005,0.22385089862586705,0.22385089862586705,0.8497965223825373,0.9367573521536497
+Native 0G5,4487.690767,74.79484611666666,0.22825815691044388,0.22825815691044388,0.8497965223825373,0.9563607940829408
+Native 0G5,5333.076216,88.8846036,0.2334607053380644,0.2334607053380644,0.8497965223825373,0.9729253996239514
+Native 0G5,6337.714295,105.62857158333333,0.23959548132469727,0.23959548132469727,0.8497965223825373,0.9860975132794608
+Native 0G5,7531.604811,125.52674685000001,0.2468223567610878,0.2468223567610878,0.8497965223825373,0.995365722683795
+Native 0G5,8950.398895,149.17331491666667,0.25532229158944525,0.25532229158944525,0.849804463275077,0.9997858295429532
+Native 0G5,10636.463595,177.27439324999997,0.2653803619389791,0.2653803619389791,0.8498286193149812,0.9999999999999997
+Native 0G5,12640.147088,210.6691168,0.2770076709701143,0.2770076709701143,0.8498287133930162,0.9999999999999997
+Native 0G5,15021.28174,250.3546956666667,0.2906494744642556,0.2906494744642556,0.8498288200426888,0.9999999999999997
+Native 0G5,17850.971589,297.51619315,0.3066632636558199,0.3066632636558199,0.849828938766579,0.9999999999999997
+Native 0G5,21213.714793,353.56191321666665,0.32524327914078466,0.32524327914078466,0.8498280740993082,0.9999999999999997
+Native 0G5,25209.927262,420.165436666667,0.3467992440913052,0.3467992440913052,0.849829225621787,0.9999999999999997
+Native 0G5,29958.941127,499.31568545,0.37172886809138583,0.37172886809138583,0.8498293856156343,0.9999999999999997
+Native 0G5,35602.568152,593.3716138666667,0.40046249624689423,0.40046249624689423,0.8498295631851173,0.9999999999999997
+Native 0G5,42300.33442,705.1555736666667,0.43346157618856646,0.43346157618856646,0.8498297543008814,0.9999999999999997
+Native 0G5,50270.512742,837.8917893333334,0.47121651115425045,0.47121651115425045,0.8498299576138709,0.9999999999999997
+Native 0G5,59751.103607,995.851771833332,0.5142439867270523,0.5142439867270523,0.8498301723129145,0.9999999999999997
+Native 0G5,71006.939467,1183.4489911166668,0.5630840456611398,0.5630840456611398,0.849830396631738,0.9999999999999997
+Native 0G5,84383.135936,1406.358598933335,0.6182967929458124,0.6182967929458124,0.8498306276903214,0.9999999999999997
+Native 0G5,100279.123194,1671.3187199,0.6804587615004533,0.6804587615004533,0.8498308643661403,0.9999999999999997
+Native 0G5,119169.576207,1986.15960345,0.7501590579914197,0.7501590579914197,0.8498311031099131,0.9999999999999997
+Native 0G5,141618.58066,2360.3098101,0.827994587397729,0.827994587397729,0.849831343802076,0.9999999999999997
+Native 0G5,168296.517258,2804.9419543,0.9145636914326302,0.9145636914326302,0.8498315823044279,0.9999999999999997
+Native 0G5,200000,0,3333.333333333333,1.010458382945538,1.010458382945538,0.849831818186178,0.9999999999999997
+MFront 0G5,0.0,0.0,0.19999999999999996,0.19999999999999996,0.8497965223825373,0.04026417585943573
+MFront 0G5,60.0,1.0,0.20046783944075428,0.20046783944075428,0.8497965223825373,0.2713537126381543
+MFront 0G5,71.302723,1.1883787166666666,0.20055588003108543,0.20055588003108543,0.8497965223825373,0.2874521628277625
+MFront 0G5,84.734639,1.4122439833333333,0.20066048898874264,0.20066048898874264,0.8497965223825373,0.30501762927033954
+MFront 0G5,100.696842,1.6782807,0.20078474494834797,0.20078474494834797,0.8497965223825373,0.32405680941089293
+MFront 0G5,119.665984,1.994330666666665,0.20093233846021685,0.20093233846021685,0.8497965223825373,0.3464245622297384
+MFront 0G5,142.20851,2.3701418333333333,0.20110763309425947,0.20110763309425947,0.8497965223825373,0.3666817245703682
+MFront 0G5,168.997567,2.816626116666667,0.20131580643880006,0.20131580643880006,0.8497965223825373,0.3902840686609055
+MFront 0G5,200.833112,3.3472185333333333,0.20156299291676114,0.20156299291676114,0.8497965223825373,0.41544704695536866
+MFront 0G5,238.665797,3.977763283333333,0.20185645678943878,0.20185645678943878,0.8497965223825373,0.4421575250592795
+MFront 0G5,283.625355,4.727089250000001,0.2022047988158349,0.2022047988158349,0.8497965223825373,0.47040477778851324
+MFront 0G5,337.454337,5.617572283333333,0.20261817387352365,0.20261817387352365,0.8497965223825373,0.5000514857315509
+MFront 0G5,400.548202,6.675803666666667,0.20310859840932957,0.20310859840932957,0.849796522382538,0.531046136529492
+MFront 0G5,476.00296,7.933826666666666,0.20369026607274585,0.20369026607274585,0.849796522382538,0.5633847990626294
+MFront 0G5,565.671789,9.42786315,0.20437999955407687,0.20437999955407687,0.849796522382538,0.5968637171486848
+MFront 0G5,672.232317,11.20387195,0.2051973603293348,0.2051973603293348,0.849796522382538,0.631123845325501
+MFront 0G5,798.866582,13.31444303333333,0.2061658517274497,0.2061658517274497,0.8497965223825381,0.6662818797160034
+MFront 0G5,949.356847,15.822600783333334,0.20731233884125547,0.20731233884125547,0.849796522382538,0.7014268616569991
+MFront 0G5,1128.194526,18.8032421,0.2086689026161923,0.2086689026161923,0.8497965223825387,0.736371321776637
+MFront 0G5,1340.722368,22.3453728,0.2102757505179658,0.2102757505179658,0.8497965223825389,0.7705811526239964
+MFront 0G5,1593.285933,26.554765550000003,0.21216614547257323,0.21216614547257323,0.8497965223825389,0.8031889629397689
+MFront 0G5,1893.4271,31.557118333333335,0.21439979039016396,0.21439979039016396,0.8497965223825389,0.8341699583805483
+MFront 0G5,2258.108477,37.501807950000001,0.21703136772091308,0.21703136772091308,0.8497965223825388,0.8613853973128149
+MFront 0G5,2673.981034,44.56635866666664,0.2201274752231815,0.2201274752231815,0.8497965223825387,0.8900637113499873
+MFront 0G5,3177.702163,52.961702716666665,0.2237642455263582,0.2237642455263582,0.8497965223825387,0.9146811878280586
+MFront 0G5,3776.313633,62.938505500000005,0.2280264183609163,0.2280264183609163,0.8497965223825388,0.9368032143590528
+MFront 0G5,4487.690767,74.79484611666666,0.23300734905772186,0.23300734905772186,0.8497965223825389,0.9561485425224546
+MFront 0G5,5333.076216,88.8846036,0.23882009704024612,0.23882009704024612,0.8497965223825388,0.9727861539782588
+MFront 0G5,6337.714295,105.62857158333333,0.2455804001603115,0.2455804001603115,0.8497965223825389,0.986807519709658
+MFront 0G5,7531.604811,125.52674685000001,0.25341389540912257,0.25341389540912257,0.8497965223825388,0.9953883823910886
+MFront 0G5,8950.398895,149.17331491666667,0.26246431955062705,0.26246431955062705,0.8498038214325123,0.99968072671024
+MFront 0G5,10636.463595,177.27439324999997,0.2728662194493716,0.2728662194493716,0.8498298683836657,0.9999999999999997
+MFront 0G5,12640.147088,210.6691168,0.2847987031561826,0.2847987031561826,0.8498300782967497,0.9999999999999997
```

```

+MFront 0G5,15021.28174,250.3546956666667,0.2983933791912255,0.2983933791912255,0.849830408299126,0.999999999999999
+MFront 0G5,17850.971589,297.51619315,0.3137846531009978,0.3137846531009978,0.8498305457908836,0.999999999999999
+MFront 0G5,21213.714793,353.56191321666667,0.3311515532737935,0.3311515532737935,0.8498307996471562,0.999999999999999
+MFront 0G5,25209.927262,428.16545436666667,0.3506129781502663,0.3506129781502663,0.8498310633638295,0.999999999999999
+MFront 0G5,29958.941127,499.31568545,0.37232983190345975,0.37232983190345975,0.8498313342069209,0.999999999999999
+MFront 0G5,35602.568152,593.3761358666667,0.39643373836377843,0.39643373836377843,0.8498316088542919,0.999999999999999
+MFront 0G5,42309.33442,705.1555736666667,0.42298070963278117,0.42298070963278117,0.8498318832343541,0.999999999999999
+MFront 0G5,50279.512742,837.9918798333334,0.4523583919161826,0.4523583919161826,0.849832156687635,0.999999999999999
+MFront 0G5,59751.103867,995.8517177833332,0.4844071206537823,0.4844071206537823,0.8498324233239125,0.999999999999999
+MFront 0G5,71006.939467,1183.4489911166668,0.5191356349418456,0.5191356349418456,0.8498326800059941,0.999999999999999
+MFront 0G5,84383.135936,1406.3855989333335,0.5566283741644444,0.5566283741644444,0.8498329247547934,0.999999999999999
+MFront 0G5,100279.123194,1671.3187199,0.5968758392605812,0.5968758392605812,0.8498331555740519,0.999999999999999
+MFront 0G5,119169.576207,1986.15960345,0.6400291455340281,0.6400291455340281,0.8498333719542966,0.999999999999999
+MFront 0G5,141618.588606,2360.3090181,0.6864481825633046,0.6864481825633046,0.8498335744636337,0.999999999999999
+MFront 0G5,168296.517258,2804.9419543,0.7357266171600998,0.7357266171600998,0.849833760659186,0.999999999999999
+MFront 0G5,200000.0,3333.333333333335,0.7880079376113349,0.7880079376113349,0.8498339312101456,0.999999999999999
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/generate_bgr_epfl_mesh.py b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/generate_bgr_epfl_mesh.py
new file mode 100644
index 0000000000..0d3338c0c3
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/generate_bgr_epfl_mesh.py
@@ -0,0 +1,127 @@
#!/usr/bin/env python3
+"""Build the EPFL benchmark mesh and geometry files from the BEACON source mesh.
+
++This utility rescales the reference BEACON mesh to the EPFL dimensions and
+writes the matching QML and small structured VTU column used by the benchmark.
+"""
+
+from __future__ import annotations
+
+import xml.etree.ElementTree as ET
+from pathlib import Path
+
+
+HERE = Path(__file__).resolve().parent
+SOURCE_MESH = HERE.parent / "beacon_1a01_domain_unstructured_162e.vtu"
+TARGET_MESH = HERE / "bgr_wp3_epfl_domain_162e.vtu"
+TARGET_MESH_STRUCTURED = HERE / "bgr_wp3_epfl_domain_2e.vtu"
+TARGET_GML = HERE / "bgr_wp3_epfl_geometry.gml"
+
+SOURCE_WIDTH = 0.025
+SOURCE_HEIGHT = 0.02
+TARGET_WIDTH = 0.0175
+TARGET_HEIGHT = 0.0125
+
+
+def scale_mesh() -> None:
+    """Rescale the source mesh coordinates to the EPFL benchmark footprint."""
+    tree = ET.parse(SOURCE_MESH)
+    root = tree.getroot()
+    data_array = root.find("./Points/DataArray")
+    if data_array is None or data_array.text is None:
+        raise RuntimeError("Could not find point coordinates in source VTU.")
+
+    values = list(map(float, data_array.text.split()))
+    sx = TARGET_WIDTH / SOURCE_WIDTH
+    sy = TARGET_HEIGHT / SOURCE_HEIGHT
+
+    for i in range(0, len(values), 3):
+        values[i] *= sx
+        values[i + 1] *= sy
+
+    data_array.text = " ".join(f"{v:.16g}" for v in values)
+    tree.write(TARGET_MESH, encoding="ISO-8859-1", xml_declaration=True)
+
+
+def write_gml() -> None:
+    """Emit the geometry file describing the scaled EPFL rectangular domain."""
+    text = f'<?xml version="1.0" encoding="ISO-8859-1"?>
+<?xml-stylesheet type="text/xsl" href="OpenGeoSysGLI.xsl"?>
+<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+  <name>bgr_wp3_epfl_geometry</name>
+  <points>
+    <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
+    <point id="1" x="0.0" y="{TARGET_HEIGHT}" z="0.0"/>
+    <point id="2" x="{TARGET_WIDTH}" y="0.0" z="0.0"/>
+    <point id="3" x="{TARGET_WIDTH}" y="{TARGET_HEIGHT}" z="0.0"/>
+  </points>
+  <polylines>
+    <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt></polyline>
+    <polyline id="1" name="right"><pnt>2</pnt><pnt>3</pnt></polyline>
+    <polyline id="2" name="bottom"><pnt>0</pnt><pnt>2</pnt></polyline>
+    <polyline id="3" name="top"><pnt>1</pnt><pnt>3</pnt></polyline>
+  </polylines>
+</OpenGeoSysGLI>
+"""
+    TARGET_GML.write_text(text)
+
+
+def write_structured_column_mesh() -> None:
+    """Write the tiny structured reference mesh used by the EPFL report plots."""
+    ny = 2
+    points = []
+    for j in range(ny + 1):
+        y = TARGET_HEIGHT * j / ny
+        points.append(f"0 {y:.16g} 0")
+        points.append(f"1 {TARGET_WIDTH:.16g} {y:.16g} 0")
+
+    connectivity = []
+    offsets = []
+    for j in range(ny):
+        base = 2 * j
+        connectivity.extend([base, base + 1, base + 3, base + 2])
+        offsets.append(4 * (j + 1))
+
+    material_ids = " ".join("0" for _ in range(ny))
+    connectivity_text = " ".join(str(v) for v in connectivity)
+    offsets_text = " ".join(str(v) for v in offsets)
+    types_text = " ".join("9" for _ in range(ny))
+    points_text = "\n          ".join(points)
+
+    text = f'<?xml version="1.0" encoding="ISO-8859-1"?>
+<VTIFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian">
+  <UnstructuredGrid>
+    <Piece NumberOfPoints="{2 * (ny + 1)}" NumberOfCells="{ny}">
+      <PointData/>

```



```

+         <CellData Scalars="MaterialIDs">
+             <DataArray type="Int32" Name="MaterialIDs" format="ascii">{material_ids}</DataArray>
+         </CellData>
+         <Points>
+             <DataArray type="Float64" NumberOfComponents="3" format="ascii">
+                 {points_text}
+             </DataArray>
+         </Points>
+         <Cells>
+             <DataArray type="Int64" Name="connectivity" format="ascii">{connectivity_text}</DataArray>
+             <DataArray type="Int64" Name="offsets" format="ascii">{offsets_text}</DataArray>
+             <DataArray type="UInt8" Name="types" format="ascii">{types_text}</DataArray>
+         </Cells>
+     </Piece>
+ </UnstructuredGrid>
+</VTKFile>
+"""
+     TARGET_MESH_STRUCTURED.write_text(text)
+
+
+def main() -> None:
+    """Generate the EPFL mesh artifacts in one pass."""
+    scale_mesh()
+    write_structured_column_mesh()
+    write_gml()
+    print(TARGET_MESH)
+    print(TARGET_MESH_STRUCTURED)
+    print(TARGET_GML)
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/plot_bgr_wp3_epfl_report_figures.py b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/plot_bgr_wp3_epfl_report_figure
s.py
new file mode 100644
index 0000000000..02ed4a0db7
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/plot_bgr_wp3_epfl_report_figures.py
@@ -0,0 +1,392 @@
#!/usr/bin/env python3
"""Run, extract, and plot the BGR EPFL AB' benchmark comparison figures.

+
+The script executes both project variants, converts VTU outputs into CSV
+summaries, and regenerates the report-style figures and panel layouts.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import os
+import shutil
+import subprocess
+import tempfile
+import xml.etree.ElementTree as ET
+from dataclasses import dataclass
+from pathlib import Path
+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+import matplotlib
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+import numpy as np
+import vtk
+from PIL import Image, ImageDraw, ImageFont
+from vtk.util.numpy_support import vtk_to_numpy
+
+
+HERE = Path(__file__).resolve().parent
+MFRONT_DEFAULT = Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs")
+NATIVE_DEFAULT = MFRONT_DEFAULT
+NATIVE_PROJECT = HERE / "bgr_wp3_p2_1_abprime_native.prj"
+MFRONT_PROJECT = HERE / "bgr_wp3_p2_1_abprime_mfront.prj"
+EXPERIMENT_SWELLING_PRESSURE_RANGE_MPA = (3.12, 3.55)
+
+
+@dataclass
+class Series:
+    label: str
+    times_s: np.ndarray
+    times_min: np.ndarray
+    axial_swelling_pressure_mpa: np.ndarray
+    vertical_stress_mpa: np.ndarray
+    void_ratio: np.ndarray
+    saturation: np.ndarray
+
+
+def make_output_times(t_end_s: float, num_times: int) -> list[float]:
+    """Generate a log-spaced output schedule with an explicit initial sample."""
+    raw = np.geomspace(60.0, t_end_s, num=num_times)
+    times = np.unique(np.round(raw, 6))
+    times = np.concatenate([0.0], times)
+    times[-1] = t_end_s
+    return [float(t) for t in times]
+
+
+def write_project_copy(source: Path, target: Path, prefix: str, output_times: list[float]) -> None:
+    """Clone a project file while injecting a new output prefix and cadence."""
+    tree = ET.parse(source)
+    root = tree.getroot()
+
+    output = root.find("./time_loop/output")
+    if output is None:
+        raise RuntimeError(f"No output block found in {source}")
+
+    prefix_node = output.find("prefix")
+    if prefix_node is None:
+        raise RuntimeError(f"No output prefix found in {source}")
+    prefix_node.text = prefix
+
+    fixed_times = output.find("fixed_output_times")
+    if fixed_times is None:
+        fixed_times = ET.SubElement(output, "fixed_output_times")
+    fixed_times.text = " ".join(f"{t:.6f}" for t in output_times[1:])

```

```

+ tree.write(target, encoding="ISO-8859-1", xml_declaration=True)
+
+
+def run_ogs(ogs: Path, project: Path, workdir: Path) -> None:
+    """Run OGS for a project in the benchmark working directory."""
+    subprocess.run(
+        [str(ogs), str(project)],
+        cwd=workdir,
+        check=True,
+        stdout=subprocess.DEVNULL,
+        stderr=subprocess.STDOUT,
+    )
+
+
+def load_vtu(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
+    """Load coordinates and point data from a VTU output file."""
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    grid = reader.GetOutput()
+    points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+    data = {
+        grid.GetPointData().GetArrayName(i): vtk_to_numpy(grid.GetPointData().GetArray(i))
+        for i in range(grid.GetPointData().GetNumberOfArrays())
+    }
+    return points, data
+
+
+def boundary_mean(points: np.ndarray, values: np.ndarray, axis: int, side: str) -> np.ndarray:
+    """Compute the mean value on the max boundary of the selected axis."""
+    target = points[:, axis].max() if side == "max" else points[:, axis].min()
+    mask = np.isclose(points[:, axis], target, atol=1e-10)
+    return np.asarray(values[mask]).mean(axis=0)
+
+
+def time_from_filename(path: Path) -> float:
+    """Extract the physical time stamp from a benchmark VTU filename."""
+    stem = path.stem
+    marker = "_t_"
+    if marker not in stem:
+        raise RuntimeError(f"Could not parse time from {path.name}")
+    return float(stem.split(marker, 1)[1])
+
+
+def collect_series(run_dir: Path, prefix: str, label: str) -> Series:
+    """Collect boundary-stress and saturation series from a full run directory."""
+    paths = sorted(run_dir.glob(f"{prefix}_t_*.vtu"), key=time_from_filename)
+    if not paths:
+        raise RuntimeError(f"No VTU outputs found for {prefix} in {run_dir}")
+
+    times_s = []
+    axial_swelling_pressure_mpa = []
+    vertical_stress_mpa = []
+    void_ratio = []
+    saturation = []
+
+    for path in paths:
+        points, data = load_vtu(path)
+        sigma_top = boundary_mean(points, data["sigma"], axis=1, side="max")
+        phi = np.asarray(data["porosity"]).mean()
+        times_s.append(time_from_filename(path))
+        axial_swelling_pressure_mpa.append(abs(float(sigma_top[1])) / 1.0e6)
+        vertical_stress_mpa.append(abs(float(sigma_top[1])) / 1.0e6)
+        void_ratio.append(float(phi / (1.0 - phi)))
+        saturation.append(float(np.asarray(data["saturation"]).mean()))
+
+    times_s_arr = np.asarray(times_s)
+    order = np.argsort(times_s_arr)
+    times_s_arr = times_s_arr[order]
+
+    return Series(
+        label=label,
+        times_s=times_s_arr,
+        times_min=times_s_arr / 60.0,
+        axial_swelling_pressure_mpa=np.asarray(axial_swelling_pressure_mpa)[order],
+        vertical_stress_mpa=np.asarray(vertical_stress_mpa)[order],
+        void_ratio=np.asarray(void_ratio)[order],
+        saturation=np.asarray(saturation)[order],
+    )
+
+
+def write_series_csv(path: Path, series: list[Series]) -> None:
+    """Write the extracted benchmark series in a plot-friendly CSV format."""
+    with path.open("w", newline="") as f:
+        writer = csv.writer(f)
+        writer.writerow(
+            [
+                "series",
+                "time_s",
+                "time_min",
+                "axial_swelling_pressure_mpa",
+                "vertical_stress_mpa",
+                "void_ratio",
+                "mean_saturation",
+            ]
+        )
+        for s in series:
+            for row in zip(
+                s.times_s,
+                s.times_min,
+                s.axial_swelling_pressure_mpa,
+                s.vertical_stress_mpa,
+                s.void_ratio,
+                s.saturation,
+                strict=True,
+            ):
+                writer.writerow([s.label, *row])
+
+
+def read_series_csv(path: Path) -> list[Series]:
+    """Reload previously written series data to support plot-only reruns."""
+    buckets: dict[str, dict[str, list[float]]] = {}
+    with path.open(newline="") as f:
+        reader = csv.DictReader(f)
+        for row in reader:
+            label = row["series"]
+            if label not in buckets:

```

```

+         buckets[label] = {
+             "time_s": [],
+             "time_min": [],
+             "axial_swelling_pressure_mpa": [],
+             "vertical_stress_mpa": [],
+             "void_ratio": [],
+             "mean_saturation": [],
+         }
+         buckets[label]["time_s"].append(float(row["time_s"]))
+         buckets[label]["time_min"].append(float(row["time_min"]))
+         buckets[label]["axial_swelling_pressure_mpa"].append(float(row["axial_swelling_pressure_mpa"]))
+         buckets[label]["vertical_stress_mpa"].append(float(row["vertical_stress_mpa"]))
+         buckets[label]["void_ratio"].append(float(row["void_ratio"]))
+         buckets[label]["mean_saturation"].append(float(row["mean_saturation"]))
+
+     series = []
+     for label, values in buckets.items():
+         series.append(
+             Series(
+                 label=label,
+                 times_s=np.asarray(values["time_s"]),
+                 times_min=np.asarray(values["time_min"]),
+                 axial_swelling_pressure_mpa=np.asarray(values["axial_swelling_pressure_mpa"]),
+                 vertical_stress_mpa=np.asarray(values["vertical_stress_mpa"]),
+                 void_ratio=np.asarray(values["void_ratio"]),
+                 saturation=np.asarray(values["mean_saturation"]),
+             )
+         )
+     return series
+
+
+def plot_swelling_pressure(series: list[Series], path: Path) -> None:
+    """Plot the swelling-pressure time history for the report figure."""
+    fig, ax = plt.subplots(figsize=(7.2, 4.2))
+
+    ax.axhspan(
+        EXPERIMENT_SWELLING_PRESSURE_RANGE_MPA[0],
+        EXPERIMENT_SWELLING_PRESSURE_RANGE_MPA[1],
+        color="#d9d9d9",
+        alpha=0.5,
+        label="EPFL report range",
+    )
+
+    colors = {"Native OG5": "#d62728", "MFront OG5": "#1f77b4"}
+    for s in series:
+        ax.plot(
+            s.times_min,
+            s.axial_swelling_pressure_mpa,
+            marker="o",
+            markersize=3.2,
+            linewidth=1.9,
+            color=colors[s.label],
+            label=s.label,
+        )
+
+    ax.set_title("EPFL P2-1 / A-B: Vertical Swelling Pressure With Time")
+    ax.set_xlabel("Time (min)")
+    ax.set_ylabel("Vertical swelling pressure (MPa)")
+    ax.set_xlim(left=0.0)
+    ax.set_ylim(bottom=0.0)
+    ax.grid(True, alpha=0.3)
+    ax.legend(frameon=True, loc="lower right")
+    fig.tight_layout()
+    fig.savefig(path, dpi=220)
+    plt.close(fig)
+
+
+def plot_void_ratio(series: list[Series], path: Path) -> None:
+    fig, ax = plt.subplots(figsize=(7.2, 4.6))
+
+    colors = {"Native OG5": "#d62728", "MFront OG5": "#1f77b4"}
+    for s in series:
+        ax.plot(
+            s.vertical_stress_mpa,
+            s.void_ratio,
+            marker="o",
+            markersize=3.4,
+            linewidth=1.9,
+            color=colors[s.label],
+            label=s.label,
+        )
+        ax.scatter(
+            [s.vertical_stress_mpa[0], s.vertical_stress_mpa[-1]],
+            [s.void_ratio[0], s.void_ratio[-1]],
+            color=colors[s.label],
+            s=24,
+            zorder=3,
+        )
+
+    ax.axvspan(
+        EXPERIMENT_SWELLING_PRESSURE_RANGE_MPA[0],
+        EXPERIMENT_SWELLING_PRESSURE_RANGE_MPA[1],
+        color="#d9d9d9",
+        alpha=0.35,
+        label="EPFL B' stress range",
+    )
+
+    ax.set_xscale("log")
+    ax.set_title("EPFL P2-1 / A-B: Void Ratio - Vertical Stress Path")
+    ax.set_xlabel("Total vertical stress (MPa)")
+    ax.set_ylabel("Void ratio (-)")
+    ax.set_xlim(left=1.0e-2)
+    ax.grid(True, alpha=0.3, which="both")
+    ax.legend(frameon=True, loc="upper right")
+    fig.tight_layout()
+    fig.savefig(path, dpi=220)
+    plt.close(fig)
+
+
+def plot_composite(swelling_path: Path, void_ratio_path: Path, out_path: Path) -> None:
+    img1 = Image.open(swelling_path)
+    img2 = Image.open(void_ratio_path)
+    width = max(img1.width, img2.width)
+    margin = 24
+    title_h = 44
+    canvas = Image.new(
+        "RGB",

```

```

+         (width + 2 * margin, title_h + img1.height + img2.height + 3 * margin),
+         "white",
+     )
+     draw = ImageDraw.Draw(canvas)
+     draw.text(
+         (margin, 12),
+         "EPFL report-style figures for reduced BGR P2-1 / A-B'",
+         fill="black",
+         font=ImageFont.load_default(),
+     )
+     canvas.paste(img1, (margin, title_h))
+     canvas.paste(img2, (margin, title_h + img1.height + margin))
+     canvas.save(out_path)
+
+
+def main() -> None:
+    parser = argparse.ArgumentParser()
+    parser.add_argument("--native-ogs", type=Path, default=NATIVE_DEFAULT)
+    parser.add_argument("--mfront-ogs", type=Path, default=MFRONT_DEFAULT)
+    parser.add_argument("--num-times", type=int, default=48)
+    parser.add_argument("--t-end-s", type=float, default=2.0e5)
+    parser.add_argument("--output-dir", type=Path, default=HERE)
+    parser.add_argument("--from-csv", type=Path)
+    args = parser.parse_args()
+
+    output_dir = args.output_dir.resolve()
+    output_dir.mkdir(parents=True, exist_ok=True)
+    series_csv = output_dir / "bgr_wp3_p2_1_abprime_report_style_series.csv"
+    swelling_png = output_dir / "bgr_wp3_p2_1_abprime_fig4_5b_like.png"
+    void_ratio_png = output_dir / "bgr_wp3_p2_1_abprime_fig4_6_like.png"
+    composite_png = output_dir / "bgr_wp3_p2_1_abprime_report_style_panel.png"
+
+    if args.from_csv:
+        print(f"Loading series from {args.from_csv}", flush=True)
+        series = read_series_csv(args.from_csv)
+    else:
+        print("Running native and Mfront EPFL replays", flush=True)
+        output_times = make_output_times(args.t_end_s, args.num_times)
+
+        with tempfile.TemporaryDirectory(prefix="bgr-epfl-native-") as native_tmp, tempfile.TemporaryDirectory(
+            prefix="bgr-epfl-mfront-"
+        ) as mfront_tmp:
+            native_run = Path(native_tmp)
+            mfront_run = Path(mfront_tmp)
+
+            for src in (HERE / "bgr_wp3_epfl_domain_2e.vtu", HERE / "bgr_wp3_epfl_geometry.gml"):
+                shutil.copy2(src, native_run / src.name)
+                shutil.copy2(src, mfront_run / src.name)
+
+            native_prefix = "bgr_wp3_p2_1_abprime_native_dense"
+            mfront_prefix = "bgr_wp3_p2_1_abprime_mfront_dense"
+            native_project = native_run / NATIVE_PROJECT.name
+            mfront_project = mfront_run / MFRONT_PROJECT.name
+
+            write_project_copy(NATIVE_PROJECT, native_project, native_prefix, output_times)
+            write_project_copy(MFRONT_PROJECT, mfront_project, mfront_prefix, output_times)
+
+            run_ogs(args.native_ogs, native_project, native_run)
+            run_ogs(args.mfront_ogs, mfront_project, mfront_run)
+
+            native_series = collect_series(native_run, native_prefix, "Native OGS")
+            mfront_series = collect_series(mfront_run, mfront_prefix, "Mfront OGS")
+            series = [native_series, mfront_series]
+            write_series_csv(series_csv, series)
+
+        print(f"Writing {swelling_png.name}", flush=True)
+        plot_swelling_pressure(series, swelling_png)
+        print(f"Writing {void_ratio_png.name}", flush=True)
+        plot_void_ratio(series, void_ratio_png)
+        print(f"Writing {composite_png.name}", flush=True)
+        plot_composite(swelling_png, void_ratio_png, composite_png)
+
+        print(series_csv if series_csv.exists() else args.from_csv)
+        print(swelling_png)
+        print(void_ratio_png)
+        print(composite_png)
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/.gitignore b/Tests/Data/RichardsMechanics/BEACON_report_figures/.gitignore
new file mode 100644
index 0000000000..03fd4584b
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/.gitignore
@@ -0,0 +1,3 @@
+# Runtime outputs (not committed)
+***/*.vtu
+***/*.pvd
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/1a01/1a01_bridge_reportplot.prj b/Tests/Data/RichardsMechanics/BEACON_report_figures/1a01/1a01_bridge_reportplot.prj
new file mode 100644
index 0000000000..5e502385b0
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/1a01/1a01_bridge_reportplot.prj
@@ -0,0 +1,218 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+  <mesh axially_symmetric="true">/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon-1a01_domain_unstructured_162e.vtu</mesh>
+  <geometry>/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon-1a01.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <constitutive_relation>
+        <type>MFrontRichardsMechanics</type>
+        <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+        <material_properties>
+          <material_property name="YoungModulus" parameter="E" />
+          <material_property name="PoissonRatio" parameter="nu" />
+          <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope" />
+          <material_property name="SwellingLineSlope" parameter="SwellingLineSlope" />
+          <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope" />
+          <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+          <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation" />
+          <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation" />
+          <material_property name="BubblePressure" parameter="BubblePressure" />

```

```

+         <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m" />
+         <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode" />
+         <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode" />
+         <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention" />
+         <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope" />
+         <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient" />
+         <material_property name="MacroViscosity" parameter="MacroViscosity" />
+         <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro" />
+         <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro" />
+         <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid" />
+         <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA" />
+         <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB" />
+         <material_property name="HamakerConstant" parameter="HamakerConstant" />
+         <material_property name="SpecificSurface" parameter="SpecificSurface" />
+         <material_property name="AreaFactorTuller" parameter="AreaFactorTuller" />
+         <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller" />
+         <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize" />
+         <material_property name="SurfaceTension" parameter="SurfaceTension" />
+         <material_property name="InitialPorosity" parameter="phi0" />
+     </material_properties>
+     <initial_values>
+         <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+         <state_variable name="VolumeRatio" parameter="InitialVolumeRatio" />
+         <state_variable name="n_L" parameter="n_L0" />
+         <state_variable name="rho_LR" parameter="rho_LR0" />
+         <state_variable name="epsilon_sw" parameter="epsilon_sw0" />
+     </initial_values>
+     <constitutive_relation>
+     <process_variables>
+         <pressure>pressure</pressure>
+         <displacement>displacement</displacement>
+     </process_variables>
+     <secondary_variables>
+         <secondary_variable name="ElasticStrain" />
+         <secondary_variable name="EquivalentPlasticStrain" />
+         <secondary_variable name="PreConsolidationPressure" />
+         <secondary_variable name="PlasticVolumetricStrain" />
+         <secondary_variable name="sigma" />
+         <secondary_variable name="swelling_stress" />
+         <secondary_variable name="epsilon" />
+         <secondary_variable name="velocity" />
+         <secondary_variable name="saturation" />
+         <secondary_variable name="porosity" />
+         <secondary_variable name="transport_porosity" />
+         <secondary_variable name="dry_density_solid" />
+         <secondary_variable name="VolumeRatio" />
+         <secondary_variable name="n_L" />
+         <secondary_variable name="rho_LR" />
+         <secondary_variable name="epsilon_sw" />
+         <secondary_variable name="phi_m" />
+         <secondary_variable name="phi_M" />
+         <secondary_variable name="phi_I" />
+         <secondary_variable name="n_S" />
+         <secondary_variable name="n_L" />
+         <secondary_variable name="rho_LR" />
+         <secondary_variable name="omega_L" />
+         <secondary_variable name="mu_LR" />
+         <secondary_variable name="rho_l_hat" />
+         <secondary_variable name="delta_epsilon_sw" />
+         <secondary_variable name="sigma_S" />
+     </secondary_variables>
+     <specific_body_force>0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+     <medium id="0">
+         <phases>
+             <phase>
+                 <type>AqueousLiquid</type>
+                 <properties>
+                     <property<name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+                     <property<name>density</name><type>Constant</type><value>1e3</value></property>
+                 </properties>
+             </phase>
+             <phase>
+                 <type>Solid</type>
+                 <properties>
+                     <property<name>density</name><type>Constant</type><value>2780</value></property>
+                 </properties>
+             </phase>
+         </phases>
+         <properties>
+             <property<name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+             <property<name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+             <property<name>permeability</name><type>Constant</type><value>1e-18</value></property>
+             <property<name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+             <property<name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+             <property<name>relative_permeability</name><type>Constant</type><value>1</value></property>
+             <property<name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+             <property<name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+         </properties>
+     </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+             <convergence_criterion>
+                 <norm_type>NORM2</norm_type>
+                 <type>PerComponentDeltaX</type>
+                 <abstols>5e-8 1e-13 1e-13</abstols>
+             </convergence_criterion>
+             <time_discretization><type>BackwardEuler</type></time_discretization>
+             <time_stepping>
+                 <type>IterationNumberBasedTimeStepping</type>
+                 <t_initial>0.0</t_initial>
+                 <t_end>1e5</t_end>
+                 <initial_dt>10</initial_dt>
+                 <minimum_dt>0.1</minimum_dt>
+                 <maximum_dt>2e3</maximum_dt>
+                 <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>

```

```

+           <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+       </time_stepping>
+   </process>
+ </processes>
+ <output>
+   <type>VTK</type>
+   <prefix>beacon_1a01_bridge_reportplot</prefix>
+   <suffix>_t_(time)</suffix>
+   <fixed_output_times>10.000000 30.000000 100.000000 300.000000 1000.000000 3000.000000 10000.000000 30000.000000 50000.000000 75000.000000 100000.000000</fixed_output_t
imes>
+   <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>porosity</variable><variable>transport_porosity</vari
able><variable>dry_density_solid</variable><variable>swelling_stress</variable><variable>sigma</variable><variable>EquivalentPlasticStrain</variable><variable>PreConsolidationPress
ure</variable><variable>PlasticVolumetricStrain</variable><variable>VolumeRatio</variable><variable>n_1</variable><variable>rho_LR</variable><variable>epsilon_sw</variable><variabl
e>phi_m</variable><variable>phi_M</variable><variable>phi</variable><variable>n_5</variable><variable>n_1</variable><variable>rho_LR</variable><variable>omega_1</variable><variable>
>mu_LR</variable><variable>rho_1_hat</variable><variable>delta_epsilon_sw</variable><variable>sigma_5</variable></variables>
+ </output>
+ </time_loop>
+ <parameters>
+   <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+   <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+   <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</value></parameter>
+   <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+   <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+   <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+   <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+   <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+   <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>1.6666666666666667</value></parameter>
+   <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>BubblePressure</name><type>Constant</type><value>1e6</value></parameter>
+   <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>0.5</value></parameter>
+   <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>MicroStateLocalSolveMode</name><type>Constant</type><value>1</value></parameter>
+   <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>1</value></parameter>
+   <parameter><name>MicroSwellingStrainSlope</name><type>Constant</type><value>0.1</value></parameter>
+   <parameter><name>MassExchangeCoefficient</name><type>Constant</type><value>1e-13</value></parameter>
+   <parameter><name>MacroViscosity</name><type>Constant</type><value>1e-3</value></parameter>
+   <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+   <parameter><name>ReferenceLiquidDensityMicro</name><type>Constant</type><value>1300.0</value></parameter>
+   <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+   <parameter><name>MicroLiquidDensityA</name><type>Constant</type><value>1.3</value></parameter>
+   <parameter><name>MicroLiquidDensityB</name><type>Constant</type><value>1.0</value></parameter>
+   <parameter><name>HamakerConstant</name><type>Constant</type><value>1.9606596248030517e-15</value></parameter>
+   <parameter><name>SpecificSurface</name><type>Constant</type><value>523.0</value></parameter>
+   <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+   <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.8584073464102069</value></parameter>
+   <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+   <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+   <parameter><name>n_10</name><type>Constant</type><value>0.0015338535</value></parameter>
+   <parameter><name>rho_LR0</name><type>Constant</type><value>2296.26173839</value></parameter>
+   <parameter><name>epsilon_sw0</name><type>Constant</type><value>0.0</value></parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirich
let0</parameter></boundary_condition>
+       <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>diric
hlet0</parameter></boundary_condition>
+       <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>diri
chlet0</parameter></boundary_condition>
+       <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichl
et0</parameter></boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_copdition>
+     <boundary_conditions>
+       <boundary_condition>geometrical_set=beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pre
ssure</parameter></boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver><name>general_linear_solver</name><eigen>solver_type=SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/1a01/1a01_native_reportplot.prj b/Tests/Data/RichardsMechanics/BEACON_report_figures/1a01/1a01_native_reportplot.prj
new file mode 100644
index 0000000000..f576c43b1e
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/1a01/1a01_native_reportplot.prj
@@ -0,0 +1,187 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+  <mesh axially_symmetric="true">Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1a01_domain_unstructured_162e.vtu</mesh>
+  <geometry>Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1a01.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <micro_porosity>
+        <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+        <nonlinear_solver>
+          <maximum_iterations>100</maximum_iterations>
+          <residuum_tolerance>1e-8</residuum_tolerance>
+          <increment_tolerance>1e-20</increment_tolerance>
+        </nonlinear_solver>
+      </micro_porosity>
+
+      <potential_exchange>

```

```

+         <enabled=true</enabled>
+         <mode>full_potential</mode>
+         <pressure_tolerance>1e-12</pressure_tolerance>
+         <hamaker_constant>1.9606596248030517e-15</hamaker_constant>
+         <specific_surface>523</specific_surface>
+         <micro_solid_density_reference>2650</micro_solid_density_reference>
+         <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+         <initial_micro_water_content>0.0015338535</initial_micro_water_content>
+         <local_nonlinear_solve_mode>scalar_microstate_storage_mode</local_nonlinear_solve_mode>
+         <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+         <micro_potential_convention>negative_attractive</micro_potential_convention>
+         <micro_solid_volume_fraction_mode>reference</micro_solid_volume_fraction_mode>
+         <micro_water_content_swelling_slope>0.1</micro_water_content_swelling_slope>
+     </potential_exchange>
+     <constitutive_relation>
+         <type>MFfront</type>
+         <behaviour>ModCamClay_semiExpl_constE</behaviour>
+         <material_properties>
+             <material_property name="YoungModulus" parameter="E" />
+             <material_property name="PoissonRatio" parameter="nu" />
+             <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope" />
+             <material_property name="SwellingLineSlope" parameter="SwellingLineSlope" />
+             <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope" />
+             <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+         </material_properties>
+         <initial_values>
+             <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+             <state_variable name="VolumeRatio" parameter="InitialVolumeRatio" />
+         </initial_values>
+     </constitutive_relation>
+     <process_variables>
+         <pressure>pressure</pressure>
+         <displacement>displacement</displacement>
+     </process_variables>
+     <secondary_variables>
+         <secondary_variable name="sigma" />
+         <secondary_variable name="swelling_stress" />
+         <secondary_variable name="epsilon" />
+         <secondary_variable name="velocity" />
+         <secondary_variable name="saturation" />
+         <secondary_variable name="porosity" />
+         <secondary_variable name="transport_porosity" />
+         <secondary_variable name="dry_density_solid" />
+         <secondary_variable name="micro_saturation" />
+         <secondary_variable name="micro_pressure" />
+         <secondary_variable name="micro_water_content" />
+         <secondary_variable name="micro_porosity" />
+         <secondary_variable name="micro_exchange_source" />
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+     <medium id="0">
+         <phases>
+             <phase>
+                 <type>AqueousLiquid</type>
+                 <properties>
+                     <property<name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+                     <property<name>density</name><type>Constant</type><value>1e3</value></property>
+                 </properties>
+             </phase>
+             <phase>
+                 <type>Solid</type>
+                 <properties>
+                     <property<name>density</name><type>Constant</type><value>2780</value></property>
+                     <property>
+                         <name>swelling_stress_rate</name>
+                         <type>SaturationDependentSwelling</type>
+                         <swelling_pressures>1.3e7 1.3e7 1.3e7</swelling_pressures>
+                         <exponents>1 1 1</exponents>
+                         <lower_saturation_limit>0</lower_saturation_limit>
+                         <upper_saturation_limit>1</upper_saturation_limit>
+                     </property>
+                 </properties>
+             </phase>
+         </phases>
+         <properties>
+             <property<name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+             <property<name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+             <property<name>permeability</name><type>Constant</type><value>1</value></property>
+             <property<name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+             <property<name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+             <property<name>relative_permeability</name><type>Constant</type><value>1</value></property>
+             <property<name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+             <property<name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+             <property<name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+         </properties>
+     </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+             <convergence_criterion>
+                 <norm_type>NORM2</norm_type>
+                 <type>PerComponentDeltaX</type>
+                 <abstols>5e-8 1e-13 1e-13</abstols>
+             </convergence_criterion>
+             <time_discretization><type>BackwardEuler</type></time_discretization>
+             <time_stepping>
+                 <type>IterationNumberBasedTimeStepping</type>
+                 <t_initial>0.0</t_initial>
+                 <t_end>1e5</t_end>
+                 <initial_dt>10</initial_dt>
+                 <minimum_dt>0.1</minimum_dt>
+                 <maximum_dt>2e3</maximum_dt>
+                 <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+                 <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+             </time_stepping>
+         </process>
+     </processes>

```

```

+
+ </processes>
+ <output>
+
+ <type>VTK</type>
+ <prefix>beacon_1a01_native_reportplot</prefix>
+ <suffix>_t_{time}</suffix>
+ <fixed_output_times>0.000000 30.000000 100.000000 300.000000 1000.000000 3000.000000 10000.000000 30000.000000 50000.000000 75000.000000 100000.000000</fixed_output_t
imes>
+
+ <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</
variable><variable>micro_water_content</variable><variable>micro_porosity</variable><variable>micro_exchange_source</variable><variable>swelling_stress</variable><variable>sigma</v
ariable><variable>porosity</variable><variable>transport_porosity</variable><variable>dry_density_solid</variable></variables>
+
+ </output>
+ </time_loop>
+ <parameters>
+
+ <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+ <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+ <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+ <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+ <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+ <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+ <parameter><name>pressure_ic</name><type>Constant</type><values>-1e6</values></parameter>
+ <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+ <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+ <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+ <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+ <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+ <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>1.666666666666667</value></parameter>
+ </parameters>
+ <process_variables>
+
+ <process_variable>
+
+ <name>displacement</name>
+ <components>2</components>
+ <order>1</order>
+ <initial_condition>displacement0</initial_condition>
+ <boundary_conditions>
+
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirich
let0</parameter></boundary_condition>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>diric
hlet0</parameter></boundary_condition>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</components><parameter>diri
chlet0</parameter></boundary_condition>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichl
et0</parameter></boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ <process_variable>
+
+ <name>pressure</name>
+ <components>1</components>
+ <order>1</order>
+ <initial_condition>pressure_ic</initial_condition>
+ <boundary_conditions>
+
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pre
ssure</parameter></boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+
+ <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+
+ <linear_solver><name>general_linear_solver</name><eigen>solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/1b/1b_bridge_reportplot.prj b/Tests/Data/RichardsMechanics/BEACON_report_figures/1b/1b_bridge_reportplot.prj
new file mode 100644
index 0000000000..f8b1dd035d
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/1b/1b_bridge_reportplot.prj
@@ -0,0 +1,138 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+
+ <mesh axially_symmetric="true">Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1b_domain_unstructured_162e.vtu</mesh>
+ <geometry>Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1b.gml</geometry>
+ <processes>
+
+ <process>
+
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <constitutive_relation>
+
+ <type>MFronRichardsMechanics</type>
+ <behaviour>RichardsMechanicsDSM<MicroMacroBridge_MCC</behaviour>
+ <material_properties>
+
+ <material_property name="YoungModulus" parameter="E" />
+ <material_property name="PoissonRatio" parameter="nu" />
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope" />
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope" />
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope" />
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation" />
+ <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation" />
+ <material_property name="BubblePressure" parameter="BubblePressure" />
+ <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m" />
+ <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode" />
+ <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode" />
+ <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention" />
+ <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope" />
+ <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient" />
+ <material_property name="MacroViscosity" parameter="MacroViscosity" />
+ <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro" />
+ <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro" />
+ <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid" />
+ <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA" />
+ <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB" />
+ <material_property name="HamakerConstant" parameter="HamakerConstant" />
+ <material_property name="SpecificSurface" parameter="SpecificSurface" />
+ <material_property name="AreaFactorTuller" parameter="AreaFactorTuller" />
+ <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller" />
+ <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize" />
+ <material_property name="SurfaceTension" parameter="SurfaceTension" />
+ <material_property name="InitialPorosity" parameter="phi0" />
+ </material_properties>
+ <initial_values>
+
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio" />
+ <state_variable name="n_l" parameter="n_l0" />
+ <state_variable name="rho_lr" parameter="rho_lr0" />
+ <state_variable name="epsilon_sw" parameter="epsilon_sw0" />

```



```

+
+      </initial_values>
+
+      </constitutive_relation>
+
+      <process_variables>pressure=pressure</pressure><displacement>displacement</displacement></process_variables>
+
+      <secondary_variables>
+
+      <secondary_variable name="ElasticStrain" /><secondary_variable name="EquivalentPlasticStrain" /><secondary_variable name="PreConsolidationPressure" /><secondary_v
riable name="PlasticVolumetricStrain" /><secondary_variable name="sigma" /><secondary_variable name="swelling_stress" /><secondary_variable name="epsilon" /><secondary_variable nam
e="velocity" /><secondary_variable name="saturation" /><secondary_variable name="porosity" /><secondary_variable name="transport_porosity" /><secondary_variable name="dry_density_s
olid" /><secondary_variable name="VolumeRatio" /><secondary_variable name="n_l" /><secondary_variable name="rho_LR" /><secondary_variable name="epsilon_sw" /><secondary_variable na
me="phi_m" /><secondary_variable name="phi_M" /><secondary_variable name="phi" /><secondary_variable name="n_S" /><secondary_variable name="n_L" /><secondary_variable name="rho_LR"
 /><secondary_variable name="omega_l" /><secondary_variable name="mu_LR" /><secondary_variable name="rho_l_hat" /><secondary_variable name="delta_epsilon_sw" /><secondary_variable
name="sigma_S" />
+
+      </secondary_variables>
+
+      <specific_body_force>0</specific_body_force>
+
+      <initial_stress>sigma0</initial_stress>
+
+      <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+
+      <mass_lumping>true</mass_lumping>
+
+    </process>
+
+  </processes>
+
+  <media>
+
+    <medium id="0">
+
+      <phases>
+
+      <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>
Constant</type><value>1e3</value></property></properties></phase>
+
+      <phase><type>Solid</type><properties><property><name>density</name><type>Constant</type><value>2780</value></property></properties></phase>
+
+      </phases>
+
+      <properties>
+
+      <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_poro
sity><maximal_porosity>1</maximal_porosity></property>
+
+      <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+
+      <property><name>permeability</name><type>Constant</type><value>5e-16</value></property>
+
+      <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity
>1</maximal_porosity></property>
+
+      <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+
+      <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+
+      <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residua
l_gas_saturation><exponent>0.5</exponent><p_b>5e5</p_b></property>
+
+      <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+
+      </properties>
+
+    </medium>
+
+  </media>
+
+  <time_loop>
+
+    <processes><process ref="RM"><nonlinear_solver>nonlinear_solver</nonlinear_solver><convergence_criterion>norm_type=NORM2</norm_type><type>PerComponentDelta</type><abstol
s>5e-8 1e-13 1e-13</abstols><convergence_criterion>time_discretization<type>BackwardEuler</type><time_discretization>time_stepping<type>FixedTimeStepping</type><t_initial>0.0
</t_initial><t_end>4.32e7</t_end><time_steps>pair<repeat=400</repeat><delta_t>1.08e5</delta_t></pair></time_steps></time_stepping></process></processes>
+
+    <output><type>VTk</type><prefix>beacon_1b_bridge_report</prefix><suffix>_t_(time)</suffix><fixed_output_times>108000.000000 540000.000000 1080000.000000 2160000.00000
0 5400000.000000 10800000.000000 16200000.000000 21600000.000000 32400000.000000 43200000.000000</fixed_output_times><variables><variable>pressure</variable><variable>displacement<
/variable><variable>saturation</variable><variable>porosity</variable><variable>transport_porosity</variable><variable>dry_density_solid</variable><variable>swelling_stress</variab
le><variable>sigma</variable><variable>EquivalentPlasticStrain</variable><variable>PreConsolidationPressure</variable><variable>PlasticVolumetricStrain</variable><variable>VolumeRa
tio</variable><variable>n_l</variable><variable>rho_LR</variable><variable>epsilon_sw</variable><variable>phi_m</variable><variable>phi_M</variable><variable>phi</variable><variable>
n_S</variable><variable>n_L</variable><variable>rho_LR</variable><variable>omega_l</variable><variable>mu_LR</variable><variable>rho_l_hat</variable><variable>delta_epsilon_sw</v
ariable><variable>sigma_S</variable></variables></output>
+
+    </time_loop>
+
+    <parameters>
+
+    <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+
+    <parameter><name>E</name><type>Constant</type><value>40e6</value></parameter>
+
+    <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+
+    <parameter><name>phi0</name><type>Constant</type><value>0.5</value></parameter>
+
+    <parameter><name>phi_tr0</name><type>Constant</type><value>0.4</value></parameter>
+
+    <parameter><name>displacement0</name><type>Constant</type><value>0</value></parameter>
+
+    <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</value></parameter>
+
+    <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+
+    <parameter><name>top_pressure</name><type>Constant</type><value>0</value></parameter>
+
+    <parameter><name>bottom_pressure</name><type>Constant</type><value>1e4</value></parameter>
+
+    <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+
+    <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+
+    <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+
+    <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+
+    <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>2.0</value></parameter>
+
+    <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+
+    <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+
+    <parameter><name>BubblePressure</name><type>Constant</type><value>5e5</value></parameter>
+
+    <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>0.5</value></parameter>
+
+    <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+
+    <parameter><name>MicroStateLocalSolVMode</name><type>Constant</type><value>0</value></parameter>
+
+    <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>0</value></parameter>
+
+    <parameter><name>MicroSwellingStrainSlope</name><type>Constant</type><value>0.0</value></parameter>
+
+    <parameter><name>MassExchangeCoefficient</name><type>Constant</type><value>1e-13</value></parameter>
+
+    <parameter><name>MacroViscosity</name><type>Constant</type><value>1000.0</value></parameter>
+
+    <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+
+    <parameter><name>ReferenceLiquidDensityMicro</name><type>Constant</type><value>1e-6</value></parameter>
+
+    <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+
+    <parameter><name>MicroLiquidDensityA</name><type>Constant</type><value>0.0</value></parameter>
+
+    <parameter><name>MicroLiquidDensityB</name><type>Constant</type><value>1.0</value></parameter>
+
+    <parameter><name>HamakerConstant</name><type>Constant</type><value>1e-30</value></parameter>
+
+    <parameter><name>SpecificSurface</name><type>Constant</type><value>1.0</value></parameter>
+
+    <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+
+    <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.8584073464102069</value></parameter>
+
+    <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+
+    <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+
+    <parameter><name>n_l0</name><type>Constant</type><value>0.1</value></parameter>
+
+    <parameter><name>rho_LR0</name><type>Constant</type><value>1000.000001</value></parameter>
+
+    <parameter><name>epsilon_sw0</name><type>Constant</type><value>0.0</value></parameter>
+
+    </parameters>
+
+    <process_variables>
+
+    <process_variable><name>displacement</name><components>2</components><order>1</order><initial_condition>displacement0</initial_condition><boundary_conditions>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</
parameter></boundary_condition>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</p
arameter></boundary_condition>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</p
arameter></boundary_condition>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</p
arameter></boundary_condition>
+
+    </boundary_conditions></process_variable>
+
+    </process_variables>
+
+    <nonlinear_solvers><nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_sol
ver></nonlinear_solvers>
+
+    <linear_solvers><linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver></linear_solvers>
+
+  </OpenGeoSysProject>
+
+  diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/1b/1b_native_reportplot.prj b/Tests/Data/RichardsMechanics/BEACON_report_figures/1b/1b_native_reportplot.prj
+  new file mode 100644

```

```

index 0000000000..986550f49f
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/1b/1b_native_reportplot.prj
@@ -0,0 +1,104 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+ <mesh axially_symmetric="true">/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1b_domain_unstructured_162e.vtu</mesh>
+ <geometry>/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1b.gml</geometry>
+ <processes>
+   <process>
+     <name>RM</name>
+     <type>RICHARDS_MECHANICS</type>
+     <integration_order>2</integration_order>
+     <micro_porosity>
+       <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+       <nonlinear_solver><maximum_iterations>100</maximum_iterations><residual_tolerance>1e-8</residual_tolerance><increment_tolerance>1e-20</increment_tolerance></nonlin
ear_solver>
+     </micro_porosity>
+     <potential_exchange>
+       <enabled>true</enabled>
+       <mode>full_potential</mode>
+       <pressure_tolerance>0.0</pressure_tolerance>
+       <hamaker_constant>1e-30</hamaker_constant>
+       <specific_surface>1.0</specific_surface>
+       <micro_solid_density_reference>2650</micro_solid_density_reference>
+       <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+       <initial_micro_water_content>0.1</initial_micro_water_content>
+       <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+     </potential_exchange>
+     <constitutive_relation>
+       <type>MFront</type>
+       <behaviour>ModCamClay_semiExpl_constE</behaviour>
+       <material_properties>
+         <material_property name="YoungModulus" parameter="E" />
+         <material_property name="PoissonRatio" parameter="nu" />
+         <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope" />
+         <material_property name="SwellingLineSlope" parameter="SwellingLineSlope" />
+         <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope" />
+         <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+       </material_properties>
+       <initial_values>
+         <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+         <state_variable name="VolumeRatio" parameter="InitialVolumeRatio" />
+       </initial_values>
+     </constitutive_relation>
+     <process_variables><pressure>pressure</pressure><displacement>displacement</displacement></process_variables>
+     <secondary_variables>
+       <secondary_variable name="sigma" /><secondary_variable name="swelling_stress" /><secondary_variable name="epsilon" /><secondary_variable name="velocity" /><seconda
ry_variable name="saturation" /><secondary_variable name="porosity" /><secondary_variable name="transport_porosity" /><secondary_variable name="dry_density_solid" /><secondary_vari
able name="micro_saturation" /><secondary_variable name="micro_pressure" />
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <mass_lumping>true</mass_lumping>
+   </process>
+ </processes>
+ <media>
+   <medium id="0">
+     <phases>
+       <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>
Constant</type><value>1e3</value></property></properties></phase>
+       <phase><type>Solid</type><properties><property><name>density</name><type>Constant</type><value>2780</value></property><property><name>swelling_stress_rate</name><type>SaturationDependentSwelling</type><swelling_pressures>1.1e7 1.1e7 1.1e7</swelling_pressures><exponents>1 1 1</exponents><lower_saturation_limit>0</lower_saturation_limit><upper
_saturation_limit>1</upper_saturation_limit></property></properties></phase>
+     </phases>
+     <properties>
+       <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_poro
sity><maximal_porosity>1</maximal_porosity></property>
+       <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+       <property><name>permeability</name><type>Constant</type><value>5e-16</value></property>
+       <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity
>1</maximal_porosity></property>
+       <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+       <property><name>relative_permability</name><type>Constant</type><value>1</value></property>
+       <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residua
l_gas_saturation><exponent>0.5</exponent><p_b>5e5</p_b></property>
+       <property><name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</res
idual_gas_saturation><exponent>0.5</exponent><p_b>5e6</p_b></property>
+       <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes><process ref="RM"><nonlinear_solver>nonlinear_solver</nonlinear_solver><convergence_criterion>norm_type=NORM2</norm_type><type>PerComponentDelta</type><abstol
s>5e-8 1e-13 1e-13</abstols></convergence_criterion><time_discretization><type>BackwardEuler</type></time_discretization><time_stepping><type>FixedTimeStepping</type><t_initial>0.0
</t_initial><t_end>4.32e7</t_end><time_steps>pair<repeat>400</repeat><delta_t>1.08e5</delta_t></pair></time_steps></time_stepping></process></processes>
+   <output><type>VTU</type><prefix>beacon_1b_native_reportplot</prefix><suffix>t_{:time}</suffix><fixed_output_times>108000.000000 540000.000000 1080000.000000 2160000.00000
0 5400000.000000 10800000.000000 16200000.000000 21600000.000000 32400000.000000 43200000.000000</fixed_output_times><variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</variable><variable>swelling_stress</variable><variable>sigma</variable><vari
able>porosity</variable><variable>transport_porosity</variable><variable>dry_density_solid</variable></variables></output>
+ </time_loop>
+ <parameters>
+   <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>E</name><type>Constant</type><value>40e6</value></parameter>
+   <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>0.5</value></parameter>
+   <parameter><name>phi_tr0</name><type>Constant</type><value>0.4</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><value>0 0</values></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</value></parameter>
+   <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>top_pressure</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>bottom_pressure</name><type>Constant</type><value>1e4</value></parameter>
+   <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+   <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+   <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+   <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+   <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>2.0</value></parameter>
+ </parameters>
+ <process_variables>
+   <process_variable><name>displacement</name><components>2</components><order>1</order><initial_condition>displacement0</initial_condition><boundary_conditions>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</p

```

```

arameter</boundary_condition>
+ </boundary_conditions></process_variable>
+ <process_variable<name>pressure</name><components>1</components><order>1</order><initial_condition>pressure_ic</initial_condition><boundary_conditions>
+ <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+ <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>0</component><parameter>bottom_pre
ssure</parameter></boundary_condition>
+ </boundary_conditions></process_variable>
+ </process_variables>
+ <nonlinear_solvers><nonlinear_solver><name>nonLinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_sol
ver></nonlinear_solvers>
+ <linear_solvers><linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver></linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/1c/1c_bridge_reportplot.prj b/Tests/Data/RichardsMechanics/BEACON_report_figures/1c/1c_bridge_reportplot.prj
new file mode 100644
index 0000000000..174e1d5df6
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/1c/1c_bridge_reportplot.prj
@@ -0,0 +1,209 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
+<OpenGeoSysProject>
+ <mesh axially_symmetric="true">/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1c_domain.vtu</mesh>
+ <geometry>/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1c.gml</geometry>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <constitutive_relation id="0">
+ <type>MFrontRichardsMechanics</type>
+ <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="E" />
+ <material_property name="PoissonRatio" parameter="nu" />
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope" />
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope" />
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope" />
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation" />
+ <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation" />
+ <material_property name="BubblePressure" parameter="BubblePressure_block" />
+ <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m" />
+ <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode" />
+ <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode" />
+ <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention" />
+ <material_property name="SwellingSlope" parameter="DSMMicroMacroSwellingSlope_block" />
+ <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient_block" />
+ <material_property name="MacroViscosity" parameter="MacroViscosity_block" />
+ <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro" />
+ <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro_block" />
+ <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid" />
+ <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA_block" />
+ <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB_block" />
+ <material_property name="HamakerConstant" parameter="HamakerConstant" />
+ <material_property name="SpecificSurface" parameter="SpecificSurface_block" />
+ <material_property name="AreaFactorTuller" parameter="AreaFactorTuller" />
+ <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller" />
+ <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize" />
+ <material_property name="SurfaceTension" parameter="SurfaceTension" />
+ <material_property name="InitialPorosity" parameter="phi0_block" />
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio_block" />
+ <state_variable name="n_l" parameter="n_l0_block" />
+ <state_variable name="rho_LR" parameter="rho_LR0_block" />
+ <state_variable name="epsilon_sw" parameter="epsilon_sw0_block" />
+ </initial_values>
+ </constitutive_relation>
+ <constitutive_relation id="1">
+ <type>MFrontRichardsMechanics</type>
+ <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="E" />
+ <material_property name="PoissonRatio" parameter="nu" />
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope" />
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope" />
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope" />
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation" />
+ <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation" />
+ <material_property name="BubblePressure" parameter="BubblePressure_pellet" />
+ <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m" />
+ <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode" />
+ <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode" />
+ <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention" />
+ <material_property name="SwellingSlope" parameter="DSMMicroMacroSwellingSlope_pellet" />
+ <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient_pellet" />
+ <material_property name="MacroViscosity" parameter="MacroViscosity_pellet" />
+ <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro" />
+ <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro_pellet" />
+ <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid" />
+ <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA_pellet" />
+ <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB_pellet" />
+ <material_property name="HamakerConstant" parameter="HamakerConstant" />
+ <material_property name="SpecificSurface" parameter="SpecificSurface_pellet" />
+ <material_property name="AreaFactorTuller" parameter="AreaFactorTuller" />
+ <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller" />
+ <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize" />
+ <material_property name="SurfaceTension" parameter="SurfaceTension" />
+ <material_property name="InitialPorosity" parameter="phi0_pellet" />
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio_pellet" />
+ <state_variable name="n_l" parameter="n_l0_pellet" />
+ <state_variable name="rho_LR" parameter="rho_LR0_pellet" />
+ <state_variable name="epsilon_sw" parameter="epsilon_sw0_pellet" />
+ </initial_values>
+ </constitutive_relation>
+ <process_variables><pressure>pressure</pressure><displacement>displacement</displacement></process_variables>
+ <secondary_variables>
+ <secondary_variable name="ElasticStrain" /><secondary_variable name="EquivalentPlasticStrain" /><secondary_variable name="PreConsolidationPressure" /><secondary_va
riable name="PlasticVolumetricStrain" /><secondary_variable name="sigma" /><secondary_variable name="swelling_stress" /><secondary_variable name="epsilon" /><secondary_variable nam
e="velocity" /><secondary_variable name="saturation" /><secondary_variable name="porosity" /><secondary_variable name="transport_porosity" /><secondary_variable name="dry_density_s
olid" /><secondary_variable name="VolumeRatio" /><secondary_variable name="n_l" /><secondary_variable name="rho_LR" /><secondary_variable name="epsilon_sw" /><secondary_variable na
me="phi_m" /><secondary_variable name="phi_M" /><secondary_variable name="phi" /><secondary_variable name="n_S" /><secondary_variable name="n_l" /><secondary_variable name="rho_LR"

```

```

/><secondary_variable name="omega_l" /><secondary_variable name="mu_LR" /><secondary_variable name="rho_l_hat" /><secondary_variable name="delta_epsilon_sw" /><secondary_variable
name="sigma_5" />
+
+ </secondary_variables>
+ <specific_body_force>0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+ <mass_lumping>true</mass_lumping>
+
+ </process>
+ </processes>
+ <media>
+ <medium id="0">
+ <phases>
+ <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>
Constant</type><value>1e3</value></property></properties></phase>
+ <phase><type>Solid</type><properties><property><name>density</name><type>Constant</type><value>2780</value></property></properties></phase>
+ </phases>
+ </properties>
+ <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0_block</initial_porosity><minimal_porosity>0</minima
l_porosity><maximal_porosity>1</maximal_porosity></property>
+ <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+ <property><name>permeability</name><type>Constant</type><value>1e-18</value></property>
+ <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0_block</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_po
rosity>1</maximal_porosity></property>
+ <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+ <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+ <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residua
l_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+ <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+ </properties>
+ </medium>
+ <medium id="1">
+ <phases>
+ <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>
Constant</type><value>1e3</value></property></properties></phase>
+ <phase><type>Solid</type><properties><property><name>density</name><type>Constant</type><value>2780</value></property></properties></phase>
+ </phases>
+ </properties>
+ <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0_pellet</initial_porosity><minimal_porosity>0</minim
al_porosity><maximal_porosity>1</maximal_porosity></property>
+ <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+ <property><name>permeability</name><type>Constant</type><value>1e-15</value></property>
+ <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0_pellet</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_p
orosity>1</maximal_porosity></property>
+ <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+ <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+ <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residua
l_gas_saturation><exponent>0.5</exponent><p_b>5e5</p_b></property>
+ <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+ <processes><process ref="RM"><nonlinear_solver>nonlinear_solver</nonlinear_solver><convergence_criterion>norm_type=NORM2</norm_type><type>PerComponentDelta</type><abstol
s>5e-8 1e-13 1e-13</abstols><convergence_criterion>time_discretization</type><BackwardEuler</type><time_discretization><time_stepping><type>FixedTimeStepping</type><t_initial>0.0
</t_initial><t_end>1e3</t_end><time_steps><pair><repeat>20</repeat><delta_t>50</delta_t></pair></time_steps></time_stepping></process></processes>
+ <output><type>VTK</type><prefix>beacon_1c_bridge_reportplot</prefix><suffix>_t_(time)</suffix><fixed_output_times>50.000000 100.000000 150.000000 250.000000 400.000000 55
0.000000 700.000000 850.000000 1000.000000</fixed_output_times><variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>por
osity</variable><variable>transport_porosity</variable><variable>swelling_stress</variable><variable>sigma</variable><variable>EquivalentPlasticStrain</variable><variable>PreConsol
idationPressure</variable><variable>PlasticVolumetricStrain</variable><variable>VolumeRatio</variable><variable>n_L</variable><variable>rho_LR</variable><variable>epsilon_sw</varia
ble><variable>phi_m</variable><variable>phi_M</variable><variable>phi</variable><variable>n_L</variable><variable>rho_LR</variable><variable>omega_L</variab
le><variable>mu_LR</variable><variable>rho_L_hat</variable><variable>delta_epsilon_sw</variable><variable>sigma_5</variable></variables></output>
+ </time_loop>
+ <parameters>
+ <parameter><name>sigma0</name><type>Function</type><expression></expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+ <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+ <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+ <parameter><name>phi0_block</name><type>Constant</type><value>0.35</value></parameter>
+ <parameter><name>phi_tr0_block</name><type>Constant</type><value>0.25</value></parameter>
+ <parameter><name>phi0_pellet</name><type>Constant</type><value>0.675</value></parameter>
+ <parameter><name>phi_tr0_pellet</name><type>Constant</type><value>0.55</value></parameter>
+ <parameter><name>displacement0</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</value></parameter>
+ <parameter><name>dirichlet0</name><type>Constant</type><value>8</value></parameter>
+ <parameter><name>top_pressure</name><type>Constant</type><value>1e4</value></parameter>
+ <parameter><name>bottom_pressure</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+ <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+ <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+ <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+ <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>0.5</value></parameter>
+ <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>MicroStateLocalSolVMode</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+ <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+ <parameter><name>HamakerConstant</name><type>Constant</type><value>1e-30</value></parameter>
+ <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.8584073464102069</value></parameter>
+ <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+ <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+ <parameter><name>BubblePressure_block</name><type>Constant</type><value>1e6</value></parameter>
+ <parameter><name>BubblePressure_pellet</name><type>Constant</type><value>5e5</value></parameter>
+ <parameter><name>DSMICroMacroSwellingSlope_block</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>DSMICroMacroSwellingSlope_pellet</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>MassExchangeCoefficient_block</name><type>Constant</type><value>1e-13</value></parameter>
+ <parameter><name>MassExchangeCoefficient_pellet</name><type>Constant</type><value>1e-13</value></parameter>
+ <parameter><name>MacroViscosity_block</name><type>Constant</type><value>1000.0</value></parameter>
+ <parameter><name>MacroViscosity_pellet</name><type>Constant</type><value>1000.0</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMicro_block</name><type>Constant</type><value>1300.0</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMicro_pellet</name><type>Constant</type><value>1300.0</value></parameter>
+ <parameter><name>MicroLiquidDensityA_block</name><type>Constant</type><value>1.3</value></parameter>
+ <parameter><name>MicroLiquidDensityA_pellet</name><type>Constant</type><value>1.3</value></parameter>
+ <parameter><name>MicroLiquidDensityB_block</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>MicroLiquidDensityB_pellet</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>SpecificSurface_block</name><type>Constant</type><value>0.8</value></parameter>
+ <parameter><name>SpecificSurface_pellet</name><type>Constant</type><value>1.5</value></parameter>
+ <parameter><name>InitialVolumeRatio_block</name><type>Constant</type><value>1.5384615384615385</value></parameter>
+ <parameter><name>InitialVolumeRatio_pellet</name><type>Constant</type><value>3.0769230769230771</value></parameter>
+ <parameter><name>n_L_block</name><type>Constant</type><value>0.08</value></parameter>
+ <parameter><name>n_L_pellet</name><type>Constant</type><value>0.14</value></parameter>
+ <parameter><name>rho_LR_block</name><type>Constant</type><value>2142.722975249453</value></parameter>
+ <parameter><name>rho_LR_pellet</name><type>Constant</type><value>1874.755778731532</value></parameter>
+ <parameter><name>epsilon_sw_block</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>epsilon_sw_pellet</name><type>Constant</type><value>0.0</value></parameter>
+ </parameters>

```

```

+ <process_variables>
+ <process_variable<name=displacement</name><components>2</components><order>1</order><initial_condition>displacement0</initial_condition><boundary_conditions>
+ <boundary_condition<geometrical_set=beacon_1c_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+ <boundary_condition<geometrical_set=beacon_1c_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+ <boundary_condition<geometrical_set=beacon_1c_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+ <boundary_condition<geometrical_set=beacon_1c_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+ </boundary_conditions></process_variable>
+ <process_variable<name=pressure</name><components>1</components><order>1</order><initial_condition>pressure_ic</initial_condition><boundary_conditions>
+ <boundary_condition<geometrical_set=beacon_1c_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+ <boundary_condition<geometrical_set=beacon_1c_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>0</component><parameter>bottom_pressure</parameter></boundary_condition>
+ </boundary_conditions></process_variable>
+ </process_variables>
+ <nonlinear_solvers><nonlinear_solver<name=nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solvers></nonlinear_solvers>
+ <linear_solvers><linear_solver<name=general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver></linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/README.md b/Tests/Data/RichardsMechanics/BEACON_report_figures/README.md
new file mode 100644
index 0000000000..b35b959117
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/README.md
@@ -0,0 +1,53 @@
+# BEACON Report Figures
+
+
+This folder stores report-style native vs MFront comparison assets for:
+
+* '1a01'
+* '1b'
+* '1c'
+
+
+Tracked artefacts here are:
+
+* project files (*.prj) used to run the report plots,
+* plot summaries (*.csv, *.json),
+* figure panels (*.png) used in the transition note.
+
+
+Native-branch replay comparison outputs
+
+* 'native_branch_dsm_micromacro_comparison_report.json',
+* 'native_branch_dsm_micromacro_comparison_summary.csv',
+* 'native_branch_dsm_micromacro_stress_compare.png'.
+
+
+Runtime-only simulation outputs (*.vtu, *.pvd) are intentionally not
+versioned in this folder.
+
+
+Run provenance (including git hashes) is tracked in:
+
+* 'Tests/Data/RichardsMechanics/test_run_table.csv'
+
+
+## Native DSMMicroMacro Branch Validation
+
+
+The native dsm_micromacro implementation was replayed from branch hash
+'d4661lac00' ('/Users/vinaykumar/git/ogs-native-dsm-transition') and
+compared against the current MFront bridge branch hash 'f0a453fb89'
+('ogs-TPM_Swelling_MCC_Coupled').
+
+
+Run command:
+
+
+```bash
+python3 Tests/Data/RichardsMechanics/run_native_branch_dsm_micromacro_comparison.py \
+  --output-root /tmp/ogs_native_branch_validation \
+  --json-out Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_comparison_report.json \
+  --csv-out Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_comparison_summary.csv \
+  --plot-out Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_stress_compare.png
+```
+
+
+Summary (final-output comparison):
+
+
+| case | native axial sigma [kPa] | MFront axial sigma [kPa] | native radial sigma [kPa] | MFront radial sigma [kPa] |
+| --- | --- | --- | --- | --- |
+| '1a01_smoke' | 0.000 | 0.000 | 0.000 | 0.000 |
+| '1b_smoke' | 0.000 | 0.000 | 0.000 | 0.000 |
+| '1c_smoke' | 0.000 | 0.000 | 0.000 | 0.000 |
+| '1a01_inflow' | 3.563 | 607.172 | 3.562 | 603.935 |
+
+
+Observed behavior:
+
+* '1a01/1b/1c' smoke decks are stress-parity checks and pass with effectively
+  zero stress mismatch.
+* '1a01_inflow' is not parity-calibrated between branches and shows a large
+  stress gap (~604 kPa) and pressure-state mismatch.
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_profile.csv b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_profile.csv
new file mode 100644
index 0000000000..d2a8eef5ca
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_profile.csv
@@ -0,0 +1,5 @@
+#height_mm,report_density_kg_m3,native_density_kg_m3,bridge_density_kg_m3
+2.5,1466.0,1668.0180280606128,1668.0526326256886
+7.5,1454.0,1668.0128990118512,1668.027753141388
+12.5,1427.0,1668.0025695072263,1667.9836617426702
+17.5,1353.0,1667.960324458496,1667.9252635922821
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_report_style_panel.png b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_report_style_panel.png
new file mode 100644
index 0000000000..aacc18f78e1
Binary files /dev/null and b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_report_style_panel.png differ
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_series.csv b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_series.csv
new file mode 100644
index 0000000000..d88fa5dec7
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_series.csv
@@ -0,0 +1,25 @@
+#series,time_s,time_h,time_d,axial_kpa,radiak_kpa
+Native OGS,0.0,0.0,0.0,0.10829545847699626,0.10829545847699623
+Native OGS,10.0,0.0,0.002777777777777778,0.0001574074074074075,0.1096460663191405,0.10854713076316475
+Native OGS,30.0,0.0,0.008333333333333333,0.0003472222222222224,0.3076456442469696,0.30456799768244996
+Native OGS,100.0,0.0,0.027777777777777776,0.001574074074074073,0.8494137864736693,0.84078374936567543
+Native OGS,300.0,0.0,0.08333333333333333,0.003472222222222222,1.827962851787539,1.8081600450271011
+Native OGS,1000.0,0.2,0.2777777777777778,0.01574074074074073,3.566566198061394,3.523676792079789
+Native OGS,3000.0,0.8,0.8333333333333334,0.03472222222222224,5.738161525928457,5.662078591041058
+Native OGS,10000.0,2.7,0.7777777777777778,0.1574074074074074,8.782627961601627,8.643808086984794
+Native OGS,30000.0,8.3,0.3333333333333334,0.3472222222222222,12.572272108737856,12.359270761767752
+Native OGS,50000.0,13.88888888888889,0.5787037037037037,14.77927617057652,15.009100579577096
+Native OGS,75000.0,20.833333333333332,0.8680555555555556,16.586731288578225,17.616200323325206
+Native OGS,100000.0,27.77777777777778,0.1574074074074074,18.380502864583168,19.53557680837731
+MFront OGS,0.0,0.0,0.0,40.28534026942161,40.285340269421596

```

```

+MFront 0G5,10.0,0.002777777777777778,0.00011574074074074075,40.28547025617924,40.28529813227501
+MFront 0G5,30.0,0.008333333333333333,0.0003472222222222224,66.6581597556442,66.65732607215801
+MFront 0G5,100.0,0.027777777777777776,0.0011574074074074073,100.86981956816227,100.86604366901292
+MFront 0G5,300.0,0.08333333333333333,0.003472222222222222,136.99340863480836,136.98126470560734
+MFront 0G5,1000.0,0.2777777777777778,0.011574074074074073,188.2348084319906,188.19850602202345
+MFront 0G5,3000.0,0.8333333333333334,0.03472222222222224,249.197847410736,249.1103189108722
+MFront 0G5,10000.0,2.7777777777777777,0.11574074074074074,337.61229513414406,337.3984571152211
+MFront 0G5,30000.0,8.333333333333334,0.3472222222222222,445.70304715600537,445.33174311072287
+MFront 0G5,50000.0,13.88888888888889,0.5787037037037037,507.385640396358,506.9507532590511
+MFront 0G5,75000.0,20.833333333333332,0.8680555555555556,562.4314048794166,561.9636495920246
+MFront 0G5,100000.0,27.77777777777778,1.1574074074074074,605.0781050191957,604.6022624691241
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_profile.csv b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_profile.csv
new file mode 100644
index 0000000000..4ffc238819
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_profile.csv
@@ -0,0 +1,11 @@
+height_mm,report_density_kg_m3,native_density_kg_m3,bridge_density_kg_m3
+5.25749996304512,1387.685328068562,1387.6853280584205
+15.772499889135357,1387.802998278517,1387.8029982792264
+26.287499815225598,1388.009230639722,1388.009230661833
+36.802499741315835,1388.521752486329,1388.5217572643346
+47.31749966740608,1388.7057160047838,1388.7057160159134
+57.832499593496316,1389.1077446116647,1389.1077446159518
+68.34749951958655,1389.4022054449815,1389.4022054466423
+78.8624994456768,1389.668132289389,1389.6681322900963
+89.37749937176703,1389.7710407414543,1389.7710407410127
+99.89249929785728,1389.840256227886,1389.8402562263589
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_report_style_panel.png b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_report_style_panel.p
ng
new file mode 100644
index 0000000000..89271befb4
Binary files /dev/null and b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_report_style_panel.png differ
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_series.csv b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_series.csv
new file mode 100644
index 0000000000..2291c228b5
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_lb_series.csv
@@ -0,0 +1,34 @@
+series,time_s,time_h,time_d,axial_kpa,radial_kpa
+Native 0G5,0.0,0.0,0.0,0.0,0.0
+Native 0G5,86400.0,24.0,1.0,0.3731140603344415,0.737765497978679
+Native 0G5,108000.0,30.0,1.25,3.3987978218377806,1.4988879414112903
+Native 0G5,432000.0,120.0,5.0,1.2875902016776408,3.00095340789622
+Native 0G5,540000.0,150.0,6.25,12.076876638058051,4.686481908371812
+Native 0G5,864000.0,240.0,10.0,1.2875902016776708,3.0009534078963167
+Native 0G5,1080000.0,300.0,12.5,12.076876638058051,4.686481908650378
+Native 0G5,2160000.0,600.0,25.0,12.076876638058055,4.686481908650376
+Native 0G5,2592000.0,720.0,30.0,1.2875902016776708,3.000953407896318
+Native 0G5,5184000.0,1440.0,60.0,1.2875902016776708,3.000953407896317
+Native 0G5,5400000.0,1500.0,62.5,12.076876638058064,4.686481908650375
+Native 0G5,8640000.0,2400.0,100.0,1.2875902016776706,3.0009534078963167
+Native 0G5,10800000.0,3000.0,125.0,12.076876638058067,4.686481908650375
+Native 0G5,12960000.0,3600.0,150.0,1.2875902016776704,3.0009534078963167
+Native 0G5,16200000.0,4500.0,187.5,12.076876638058067,4.686481908650375
+Native 0G5,17280000.0,4800.0,200.0,1.2875902016776706,3.0009534078963154
+Native 0G5,21600000.0,6000.0,250.0,12.076876638058067,4.686481908650375
+Native 0G5,25920000.0,7200.0,300.0,1.2875902016776712,3.000953407896318
+Native 0G5,32400000.0,9000.0,375.0,12.076876638058067,4.686481908650375
+Native 0G5,34560000.0,9600.0,400.0,1.2875902016776708,3.0009534078963163
+Native 0G5,43200000.0,12000.0,500.0,12.076876638058067,4.686481908650375
+MFront 0G5,0.0,0.0,0.0,0.0,0.0
+MFront 0G5,86400.0,24.0,1.0,1.1134522659801358,0.7877393820481825
+MFront 0G5,108000.0,30.0,1.25,3.398635287116805,1.4988398684910695
+MFront 0G5,540000.0,150.0,6.25,12.076876637998605,4.686481908324026
+MFront 0G5,1080000.0,300.0,12.5,12.076876638254971,4.686481908675183
+MFront 0G5,2160000.0,600.0,25.0,12.076876638254971,4.686481908675183
+MFront 0G5,5400000.0,1500.0,62.5,12.076876638254971,4.686481908675183
+MFront 0G5,10800000.0,3000.0,125.0,12.076876638254971,4.686481908675183
+MFront 0G5,16200000.0,4500.0,187.5,12.076876638254971,4.686481908675183
+MFront 0G5,21600000.0,6000.0,250.0,12.076876638254971,4.686481908675183
+MFront 0G5,32400000.0,9000.0,375.0,12.076876638254971,4.686481908675183
+MFront 0G5,43200000.0,12000.0,500.0,12.076876638254971,4.686481908675183
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_profile.csv b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_profile.csv
new file mode 100644
index 0000000000..cc126e5240
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_profile.csv
@@ -0,0 +1,4 @@
+height_mm,porosity,transport_porosity,saturation
+0.0,0.34999999999999987,0.24999999999999994,1.00411730577963
+48.5,0.5125,0.3999999999999999,0.9514665907200397
+100.0,0.6749999999999999,0.55,1.016446014086417
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_report_style_panel.png b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_report_style_panel.p
ng
new file mode 100644
index 0000000000..b6f5d06874
Binary files /dev/null and b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_report_style_panel.png differ
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_zone_series.csv b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_zone_series.csv
new file mode 100644
index 0000000000..8fb543e83c
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_zone_series.csv
@@ -0,0 +1,11 @@
+series,time_s,time_min,block_axial_kpa,block_radial_kpa,pellet_axial_kpa,pellet_radial_kpa
+MFront 0G5,0.0,0.0,0.0,0.0,0.0,0.0
+MFront 0G5,50.0,0.8333333333333334,0.0,0.0,0.0,0.0
+MFront 0G5,100.0,1.6666666666666667,0.0,0.0,0.0,0.0
+MFront 0G5,150.0,2.5,0.0,0.0,0.0,0.0
+MFront 0G5,250.0,4.166666666666667,0.0,0.0,0.0,0.0
+MFront 0G5,400.0,6.666666666666667,0.0,0.0,0.0,0.0
+MFront 0G5,550.0,9.166666666666667,0.0,0.0,0.0,0.0
+MFront 0G5,700.0,11.666666666666667,0.0,0.0,0.0,0.0
+MFront 0G5,850.0,14.166666666666667,0.0,0.0,0.0,0.0
+MFront 0G5,1000.0,16.666666666666668,0.0,0.0,0.0,0.0
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_report_plot_summary.json b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_report_plot_summary.json
new file mode 100644
index 0000000000..af58ded185
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_report_plot_summary.json
@@ -0,0 +1,51 @@
+{
+  "case": "1a01",
+  "panel": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_report_style_panel.png",
+  "series_csv": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_series.csv",
+  "profile_csv": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1a01_profile.csv",

```

```

+   "native_final": {
+     "time_s": 100000.0,
+     "axial_kpa": 18.380502864583168,
+     "radial_kpa": 19.53557680837731,
+     "dry_density_mean_kg_m3": 1667.998455331385
+   },
+   "bridge_final": {
+     "time_s": 100000.0,
+     "axial_kpa": 605.0781050191957,
+     "radial_kpa": 604.6022624691241,
+     "dry_density_mean_kg_m3": 1667.9973277754873
+   }
+ },
+ {
+   "case": "1b",
+   "panel": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1b_report_style_panel.png",
+   "series_csv": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1b_series.csv",
+   "profile_csv": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1b_profile.csv",
+   "native_final": {
+     "time_s": 43200000.0,
+     "axial_kpa": 12.076876638058067,
+     "radial_kpa": 4.686481908650375,
+     "dry_density_mean_kg_m3": 1388.85144089555514
+   },
+   "bridge_final": {
+     "time_s": 43200000.0,
+     "axial_kpa": 12.076876638254971,
+     "radial_kpa": 4.686481908675183,
+     "dry_density_mean_kg_m3": 1388.85144089599788
+   }
+ },
+ {
+   "case": "1c",
+   "panel": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_report_style_panel.png",
+   "series_csv": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_zone_series.csv",
+   "profile_csv": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/BEACON_report_figures/beacon_1c_profile.csv",
+   "bridge_final": {
+     "time_s": 1000.0,
+     "block_axial_kpa": 0.0,
+     "block_radial_kpa": 0.0,
+     "pellet_axial_kpa": 0.0,
+     "pellet_radial_kpa": 0.0
+   }
+ }
+ ]
+}
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_comparison_report.json b/Tests/Data/RichardsMechanics/BEACON_report_figures/native_bran
ch_dsm_micromacro_comparison_report.json
new file mode 100644
index 0000000000..941c4b368a
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_comparison_report.json
@@ -0,0 +1,151 @@
+{
+  "cases": {
+    "1a01_inflow": {
+      "max_abs_diff": {
+        "pressure": 126661.88123622442,
+        "saturation": 0.006610123751136454,
+        "sigma": 603608.2298717289,
+        "swelling_stress": 603608.2298717286
+      },
+      "mfront_metrics": {
+        "axial_sigma_kpa": 607.1716398706767,
+        "axial_swelling_stress_kpa": 607.1716398706765,
+        "dry_density_mean_kg_m3": NaN,
+        "n_l_mean": 0.2189504078316514,
+        "phi_m_mean": 0.18104959296834855,
+        "phi_u_mean": 0.2189504078316514,
+        "pressure_mean": -62416.20099654723,
+        "radial_sigma_kpa": 603.9348707823654,
+        "radial_swelling_stress_kpa": 603.9348707823655,
+        "saturation_mean": 0.9973789159451503
+      },
+      "mfront_project": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1a01_notebook_mcc_inflow_bridge.prj",
+      "mfront_vtu": "/tmp/ogs_native_branch_validation/1a01_inflow/mfront/beacon_1a01_notebook_mcc_inflow_bridge_t_100000.000000.vtu",
+      "native_metrics": {
+        "axial_sigma_kpa": 3.5634099989482175,
+        "axial_swelling_stress_kpa": 3.563409998948218,
+        "dry_density_mean_kg_m3": NaN,
+        "micro_pressure_mean": 108338.81371973315,
+        "micro_saturation_mean": 1.128220031830204,
+        "pressure_mean": 914.7396215649775,
+        "radial_sigma_kpa": 3.56166750839007,
+        "radial_swelling_stress_kpa": 3.56166750839007,
+        "saturation_mean": 0.9999999999999998,
+        "micro_exchange_source_mean": 1.0833881371973315e-05,
+        "micro_porosity_mean": 0.01128220031830204,
+        "micro_water_content_mean": 0.01128220031830204
+      },
+      "native_project": "/Users/vinaykumar/git/ogs-native-dsm-transition/Tests/Data/RichardsMechanics/beacon_1a01_notebook_inflow.prj",
+      "native_vtu": "/tmp/ogs_native_branch_validation/1a01_inflow/native/beacon_1a01_notebook_inflow_t_100000.000000.vtu"
+    },
+    "1a01_smoke": {
+      "max_abs_diff": {
+        "pressure": 461.3327170734992,
+        "saturation": 0.0001838300558427708,
+        "sigma": 0.0,
+        "swelling_stress": 0.0
+      },
+      "mfront_metrics": {
+        "axial_sigma_kpa": 0.0,
+        "axial_swelling_stress_kpa": 0.0,
+        "dry_density_mean_kg_m3": NaN,
+        "n_l_mean": 0.1,
+        "phi_m_mean": 0.30000000000000004,
+        "phi_u_mean": 0.1,
+        "pressure_mean": -481308.4780032531,
+        "radial_sigma_kpa": 0.0,
+        "radial_swelling_stress_kpa": 0.0,
+        "saturation_mean": 0.888898150420123
+      },
+      "mfront_project": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1a01_notebook_mcc_bridge.prj",
+      "mfront_vtu": "/tmp/ogs_native_branch_validation/1a01_smoke/mfront/beacon_1a01_notebook_mcc_bridge_t_1000.000000.vtu",
+      "native_metrics": {
+        "axial_sigma_kpa": 0.0,
+        "axial_swelling_stress_kpa": 0.0,
+        "dry_density_mean_kg_m3": NaN,

```

```

+       "micro_pressure_mean": -2.1356420177407673e-16,
+       "micro_saturation_mean": 0.9995104392036152,
+       "pressure_mean": -481077.8116447164,
+       "radial_sigma_kpa": 0.0,
+       "radial_swelling_stress_kpa": 0.0,
+       "saturation_mean": 0.8881688709167219
+     },
+     "native_project": "/Users/vinaykumar/git/ogs-native-dsm-transition/Tests/Data/RichardsMechanics/beacon_1a01_notebook_smoke.prj",
+     "native_vtu": "/tmp/ogs_native_branch_validation/1a01_smoke/native/beacon_1a01_notebook_smoke_t_1000.000000.vtu"
+   },
+   "1b_smoke": {
+     "max_abs_diff": {
+       "pressure": 0.0,
+       "saturation": 0.0,
+       "sigma": 0.0,
+       "swelling_stress": 0.0
+     },
+     "mfront_metrics": {
+       "axial_sigma_kpa": 0.0,
+       "axial_swelling_stress_kpa": 0.0,
+       "dry_density_mean_kg_m3": NaN,
+       "n_l_mean": 0.1,
+       "phi_m_mean": 0.4,
+       "phi_n_mean": 0.1,
+       "pressure_mean": 5000.0,
+       "radial_sigma_kpa": 0.0,
+       "radial_swelling_stress_kpa": 0.0,
+       "saturation_mean": 0.9999999999999998
+     },
+     "mfront_project": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1b_notebook_mcc_bridge.prj",
+     "mfront_vtu": "/tmp/ogs_native_branch_validation/1b_smoke/mfront/beacon_1b_notebook_mcc_bridge_t_1000.000000.vtu",
+     "native_metrics": {
+       "axial_sigma_kpa": 0.0,
+       "axial_swelling_stress_kpa": 0.0,
+       "dry_density_mean_kg_m3": NaN,
+       "micro_pressure_mean": -2.1325059416422807e-16,
+       "micro_saturation_mean": 0.9999999999999998,
+       "pressure_mean": 5000.0,
+       "radial_sigma_kpa": 0.0,
+       "radial_swelling_stress_kpa": 0.0,
+       "saturation_mean": 0.9999999999999998
+     },
+     "native_project": "/Users/vinaykumar/git/ogs-native-dsm-transition/Tests/Data/RichardsMechanics/beacon_1b_notebook_smoke.prj",
+     "native_vtu": "/tmp/ogs_native_branch_validation/1b_smoke/native/beacon_1b_notebook_smoke_t_1000.000000.vtu"
+   },
+   "1c_smoke": {
+     "max_abs_diff": {
+       "pressure": 71.1516465570021,
+       "saturation": 2.858086912917014e-05,
+       "sigma": 0.0,
+       "swelling_stress": 0.0
+     },
+     "mfront_metrics": {
+       "axial_sigma_kpa": 0.0,
+       "axial_swelling_stress_kpa": 0.0,
+       "dry_density_mean_kg_m3": NaN,
+       "n_l_mean": 0.109999999999061016,
+       "phi_m_mean": 0.40250000000938985,
+       "phi_n_mean": 0.109999999999061016,
+       "pressure_mean": -71936.63612587041,
+       "radial_sigma_kpa": 0.0,
+       "radial_swelling_stress_kpa": 0.0,
+       "saturation_mean": 0.9906766368620289
+     },
+     "mfront_project": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1c_notebook_mcc_bridge.prj",
+     "mfront_vtu": "/tmp/ogs_native_branch_validation/1c_smoke/mfront/beacon_1c_notebook_mcc_bridge_t_1000.000000.vtu",
+     "native_metrics": {
+       "axial_sigma_kpa": 0.0,
+       "axial_swelling_stress_kpa": 0.0,
+       "dry_density_mean_kg_m3": NaN,
+       "micro_pressure_mean": -1.9899694660422202e-16,
+       "micro_saturation_mean": 0.9998356583388817,
+       "pressure_mean": -71912.91891035142,
+       "radial_sigma_kpa": 0.0,
+       "radial_swelling_stress_kpa": 0.0,
+       "saturation_mean": 0.9906823645769531
+     },
+     "native_project": "/Users/vinaykumar/git/ogs-native-dsm-transition/Tests/Data/RichardsMechanics/beacon_1c_notebook_smoke.prj",
+     "native_vtu": "/tmp/ogs_native_branch_validation/1c_smoke/native/beacon_1c_notebook_smoke_t_1000.000000.vtu"
+   }
+ },
+ "mfront_ogs": "/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs",
+ "mfront_source_dir": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled",
+ "native_ogs": "/Users/vinaykumar/git/build/release-native-beacon/bin/ogs",
+ "native_source_dir": "/Users/vinaykumar/git/ogs-native-dsm-transition"
+ )
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_comparison_summary.csv b/Tests/Data/RichardsMechanics/BEACON_report_figures/native_bran
ch_dsm_micromacro_comparison_summary.csv
new file mode 100644
index 0000000000..342c9b4cee
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_comparison_summary.csv
@@ -0,0 +1,5 @@
+case,native_axial_sigma_kpa,mfront_axial_sigma_kpa,native_radial_sigma_kpa,mfront_radial_sigma_kpa,native_dry_density_mean_kg_m3,mfront_dry_density_mean_kg_m3,native_pressure_mean
,mfront_pressure_mean,max_abs_diff_pressure,max_abs_diff_sigma,max_abs_diff_swelling_stress
+1a01_smoke,0.0,0.0,0.0,0.0,nan,nan,-481077.8116447164,-481308.4780032531,461.3327170734992,0.0,0.0
+1b_smoke,0.0,0.0,0.0,0.0,nan,nan,5000.0,5000.0,0.0,0.0,0.0
+1c_smoke,0.0,0.0,0.0,0.0,nan,nan,-71912.91891035142,-71936.63612587041,71.1516465570021,0.0,0.0
+1a01_inflow,3.5634899898482175,607.1716398706767,3.561667550839007,603.9348707823654,nan,nan,914.7396215649775,-62416.20099654723,126661.88123622442,603608.2298717289,603608.22987
17286
diff --git a/Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_stress_compare.png b/Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_d
sm_micromacro_stress_compare.png
new file mode 100644
index 0000000000..4961f292d0
Binary files /dev/null and b/Tests/Data/RichardsMechanics/BEACON_report_figures/native_branch_dsm_micromacro_stress_compare.png differ
diff --git a/Tests/Data/RichardsMechanics/DSM_leftover_cleanup_2026-04-14.md b/Tests/Data/RichardsMechanics/DSM_leftover_cleanup_2026-04-14.md
new file mode 100644
index 0000000000..61e8e6a75d
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/DSM_leftover_cleanup_2026-04-14.md
@@ -0,0 +1,34 @@
+# DSM Leftover Cleanup (2026-04-14)
+
+*Scope: DSM-related assets in `ProcessLib/RichardsMechanics`, `MaterialLib/SolidModels/MFront`, and `Tests/Data/RichardsMechanics`.
+
+## What was cleaned
+
+

```



```

+1. Removed stale XML key from BEACON report deck:
+ - File: 'BEACON_report_figures/1a01/1a01_native_reportplot.prj'
+ - Change: removed '<potential_role_mapping=micro_macro_potential_role_mapping_mode</potential_role_mapping>'.
+ - Reason: this key is no longer consumed by the current DSM parameter parser.
+
+2. Removed stale mention from ANCHORS notes:
+ - File: 'ANCHORS_MS533_ModelI/README.md'
+ - Change: dropped 'potential_role_mappings=...' from native-vs-MFront notes.
+ - Reason: documentation now matches the active parameter set.
+
+3. Removed outdated wording in CIEMAT overlay script:
+ - File: 'ANCHORS_MS533_ModelI/plot_ms33_vs_ciemat_swelling_pressure_dd.py'
+ - Change: replaced "scaffold" wording with "reduced model/setup".
+ - Reason: avoids temporary/prototype wording for now-versioned benchmark assets.
+
+## Intentionally kept (not leftovers)
+
+1. 'Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/*'
+ - Kept because these files explicitly document and run the Mathematica-side DSM driver.
+
+2. Generic OGS 'notebook' mentions outside DSM process logic
+ - Example: 'ProcessLib/Tests.cmake', 'ProcessLib/HeatConduction/Tests.cmake'.
+ - Kept because these are global OGS test infrastructure references, not DSM transfer leftovers.
+
+## Verification after cleanup
+
+- Re-scan for 'potential_role_mapping' in DSM code/tests: only historical/binary outputs were excluded and no active DSM source/deck usage remains.
+- No constitutive or solver logic changed in this cleanup; this pass is naming/config/documentation alignment only.
diff --git a/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/DsmMicromacroDriver.wl b/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/DsmMicromacroDriver.wl
new file mode 100644
index 0000000000..f63b8ab47d
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/DsmMicromacroDriver.wl
@@ -0,0 +1,427 @@
+BeginPackage["DsmMicromacroDriver"];
+
+CreateDefaultParameters::usage =
+ "CreateDefaultParameters[] returns an Association of default DSM driver parameters.";
+
+CreateDefaultInitialState::usage =
+ "CreateDefaultInitialState[params] returns a physically bounded initial microstate.";
+
+RunHistory::usage =
+ "RunHistory[steps, params, initialState] advances the local DSM micro-macro state over a step list.";
+
+RunDryDensitySweep::usage =
+ "RunDryDensitySweep[dryDensityList, params, opts] runs repeated local-driver histories over dry-density cases.";
+
+ExportHistory::usage =
+ "ExportHistory[result, basePath] writes result rows to basePath.csv and full data to basePath.json.";
+
+Begin["Private"];
+
+toNumeric[value_, fallback_] := If[NumericQ[value], N[value], fallback];
+clip[value_, bounds_List] := Clip[N[value], bounds];
+
+CreateDefaultParameters[] :=
+ <|
+ "PressureTolerance" -> 1.*^-12,
+ "PorosityUpper" -> 1. - 1.*^-12,
+ "NFloor" -> 1.*^-16,
+ "RhoFloor" -> 1.*^-12,
+ "RhoLRMacro" -> 1000.,
+ "RhoLMicro" -> 1300.,
+ "RhoSR" -> 2470.,
+ "DensityA" -> 1.3,
+ "DensityB" -> 1.,
+ "HamakerConstant" -> -6.*^-20,
+ "SpecificSurface" -> 100.,
+ "MicroPotentialSign" -> 1.,
+ "VdwMultiplier" -> 1.,
+ "MassExchangeCoefficient" -> 1.,
+ "MacroViscosity" -> 1.,
+ "SwellingSlope" -> 0.1,
+ "BulkModulus" -> 6.666666666666667*^9,
+ "UseKinematicPorosityUpdate" -> True,
+ "AreaFactorTuller" -> 1.,
+ "PoreAreaShapeFactorTuller" -> 0.8584073464102069,
+ "CharacteristicPoreSize" -> 1.*^-5,
+ "SurfaceTension" -> 0.0715,
+ "MechanicaMode" -> "MCCCarrier"
+ >;
+
+CreateDefaultInitialState[params_: Automatic] := Module[
+ {p, phiTotal, phiM, nL0, rho0},
+ p = If[AssociationQ[params], params, CreateDefaultParameters[]];
+ phiTotal = 0.2;
+ nL0 = 0.059382885374452955;
+ phiM = Max[0., phiTotal - nL0];
+ rho0 = 2193.3876899104677;
+ <|
+ "n_l" -> clip[nL0, {p["NFloor"], phiTotal}],
+ "rho_LR" -> Max[p["RhoFloor"], rho0],
+ "phi_m" -> clip[nL0, {p["NFloor"], phiTotal}],
+ "phi_M" -> phiM,
+ "epsilon_sw" -> -0.004061719462554706,
+ "sigma_s_xx" -> 2.7078129758364702*^7,
+ "sigma_s_yy" -> 2.7078129758364702*^7,
+ "sigma_s_zz" -> 2.7078129758364702*^7
+ >;
+
+saturationFromTuller[presure_, params_Association] := Module[
+ {ptol, areaFactor, shapeFactor, poreSize, gamma, capillaryTerm},
+ ptol = params["PressureTolerance"];
+ areaFactor = params["AreaFactorTuller"];
+ shapeFactor = params["PoreAreaShapeFactorTuller"];
+ poreSize = params["CharacteristicPoreSize"];
+ gamma = params["SurfaceTension"];
+ If[presure > -ptol,
+ 1.,
+ capillaryTerm =
+ (4. * shapeFactor * gamma^2) / (areaFactor * poreSize^2 * pressure^2);
+ 1. - Exp[-capillaryTerm]
+ ]
+];

```

```

+muMacroFromPressure[pressure_, rhoLR_, ptoL_] :=
+ If[pressure > -ptol, 0., pressure / rhoLR];
+
+omegaFromState[nL_, rhoLr_, nS_, rhoSR_] :=
+ Max[1.*^-30, nL * rhoLr / Max[1.*^-30, nS * rhoSR]];
+
+rhoMicroEOS[nL_, rhoLrGuess_, params_Association, nS_] := Module[
+ {rhoL0, rhoLR, aRho, bRho, rhoSR, omega},
+ rhoL0 = params["RhoL0Micro"];
+ rhoLR = params["RhoLRMacro"];
+ aRho = params["DensityA"];
+ bRho = params["DensityB"];
+ rhoSR = params["RhoSR"];
+ omega = omegaFromState[nL, rhoLrGuess, nS, rhoSR];
+ rhoL0 * Exp[-aRho * omega*bRho] + rhoLR
+];
+
+muMicroFromState[nL_, rhoLr_, params_Association, nS_, vdwMultiplier_: 1.] := Module[
+ {hamakerMagnitude, signFactor, specificSurface, rhoSR, omega},
+ hamakerMagnitude = Abs[params["HamakerConstant"]];
+ signFactor = params["MicroPotentialSign"] * Sign[params["HamakerConstant"]];
+ specificSurface = params["SpecificSurface"];
+ rhoSR = params["RhoSR"];
+ omega = omegaFromState[nL, rhoLr, nS, rhoSR];
+ signFactor * vdwMultiplier * hamakerMagnitude * rhoLr^3 / (6. * Pi) *
+ (specificSurface / omega)^3
+];
+
+computeTotalPorosityTrial[state_Association, deltaEpsV_, params_Association] := Module[
+ {phiPrev, phiTrial, denom},
+ phiPrev = clip[
+ toNumeric[state["phi_m"], 0.] + toNumeric[state["phi_M"], 0.],
+ {params["NFloor"], params["PorosityUpper"]}
+ ];
+ If[TrueQ[params["UseKinematicPorosityUpdate"]],
+ denom = 1. + deltaEpsV;
+ phiTrial =
+ If[NumericQ[denom] && Abs[denom] > 1.*^-12,
+ (phiPrev + deltaEpsV) / denom,
+ phiPrev
+ ],
+ phiTrial = phiPrev
+ ];
+ clip[phiTrial, {params["NFloor"], params["PorosityUpper"]}
+];
+
+solveRhoForNl[nL_, rhoInit_, params_Association, nS_] := Module[
+ {rho = Max[params["RhoFloor"], rhoInit], rhoNext, iter},
+ For[iter = 1, iter <= 100, iter++,
+ rhoNext = Max[params["RhoFloor"], rhoMicroEOS[nL, rho, params, nS]];
+ If[Abs[rhoNext - rho] <= 1.*^-12 * Max[1., Abs[rho]], Return[rhoNext]];
+ rho = rhoNext;
+ ];
+ rho
+];
+
+solveMicrostateStep[state_Association, step_Association, params_Association] := Module[
+ {
+ dt, pressure, deltaEpsV, phiTotalTrial, nUpper, nFloor, rhoFloor, nS,
+ nPrev, rhoPrev, prevMass, volRate, rhoLR, muLR, alphaEff, vdwMultiplier,
+ root, nVar, rhoVar, nCandidate, rhoCandidate, converged, residual1,
+ residual2, muMicro, rhoHat, nExplicit, rhoResolved
+ },
+ dt = Max[0., toNumeric[Lookup[step, "dt", 1.], 1.]];
+ pressure = toNumeric[Lookup[step, "pressure", 0.], 0.];
+ deltaEpsV = toNumeric[Lookup[step, "delta_epsilon_v", 0.], 0.];
+ vdwMultiplier = toNumeric[Lookup[step, "vdw_multiplier", params["VdwMultiplier"], params["VdwMultiplier"]]];
+
+ nFloor = params["NFloor"];
+ rhoFloor = params["RhoFloor"];
+ phiTotalTrial = computeTotalPorosityTrial[state, deltaEpsV, params];
+ nUpper = Max[nFloor, phiTotalTrial];
+ nS = Max[nFloor, 1. - phiTotalTrial];
+
+ nPrev = clip[toNumeric[state["n_l"], nFloor], {nFloor, nUpper}];
+ rhoPrev = Max[rhoFloor, toNumeric[state["rho_LR"], params["RhoLRMacro"]]];
+ prevMass = nPrev * rhoPrev;
+ volRate = If[dt > 0., deltaEpsV / dt, 0.];
+ rhoLR = params["RhoLRMacro"];
+ muLR = muMacroFromPressure[pressure, rhoLR, params["PressureTolerance"]];
+ alphaEff = params["MassExchangeCoefficient"] * rhoLR / params["MacroViscosity"];
+
+ converged = False;
+ root = Quiet @ Check[
+ FindRoot[
+ {
+ nVar * rhoVar - prevMass -
+ dt * alphaEff * (muLR - muMicroFromState[nVar, rhoVar, params, nS, vdwMultiplier]) -
+ dt * nVar * rhoVar * volRate == 0,
+ rhoVar - rhoMicroEOS[nVar, rhoVar, params, nS] == 0
+ },
+ {
+ {nVar, SetPrecision[nPrev, 50]},
+ {rhoVar, SetPrecision[rhoPrev, 50]}
+ },
+ WorkingPrecision -> 50,
+ AccuracyGoal -> 20,
+ PrecisionGoal -> 20,
+ MaxIterations -> 100
+ ],
+ $Failed,
+ {FindRoot::lstol, FindRoot::cvm1, FindRoot::jsing, FindRoot::nnum}
+ ];
+
+ If[root != $Failed && NumericQ[nVar /. root] && NumericQ[rhoVar /. root],
+ nCandidate = clip[N[nVar /. root], {nFloor, nUpper}];
+ rhoCandidate = Max[rhoFloor, N[rhoVar /. root]];
+ residual1 =
+ nCandidate * rhoCandidate - prevMass -
+ dt * alphaEff * (muLR - muMicroFromState[nCandidate, rhoCandidate, params, nS, vdwMultiplier]) -
+ dt * nCandidate * rhoCandidate * volRate;
+ residual2 = rhoCandidate - rhoMicroEOS[nCandidate, rhoCandidate, params, nS];
+ converged = Abs[residual1] < 1.*^-8 && Abs[residual2] < 1.*^-8;
+ ];
+
+ If[!converged,
+ muMicro = muMicroFromState[nPrev, rhoPrev, params, nS, vdwMultiplier];

```

```

+ rhoHat = alphaEff * (muLR - muMicro);
+ nExplicit = nPrev + dt * rhoHat / Max[rhoFloor, rhoPrev] + dt * nPrev * volRate;
+ nCandidate = clip[nExplicit, {nFloor, nUpper}];
+ rhoResolved = solveRhoForN[nCandidate, rhoPrev, params, nS];
+ rhoCandidate = Max[rhoFloor, rhoResolved];
+ ];
+
+ muMicro = muMicroFromState[nCandidate, rhoCandidate, params, nS, vdWMultiplier];
+ rhoHat = alphaEff * (muLR - muMicro);
+
+ <|
+ "n_l" -> nCandidate,
+ "rho_LR" -> rhoCandidate,
+ "mu_LR" -> muLR,
+ "mu_LR" -> muMicro,
+ "rho_l_hat" -> rhoHat,
+ "phi_total_trial" -> phiTotalTrial,
+ "n_upper_bound" -> nUpper,
+ "solver_status" -> If[converged, "newton", "fallback_explicit"]
+ |>
+];
+
+ advanceState[state_Association, step_Association, params_Association, stepIndex_Integer] := Module[
+ {
+ deltaEpsV, pressure, dt, micro, phiM, phiMPrev, epsSwPrev,
+ deltaEpsSw, epsSw, sigmaPrev, sigmaNew, satMacro, phiTotal, nS, nL
+ },
+ deltaEpsV = toNumeric[Lookup[step, "delta_epsilon_v", 0.], 0.];
+ pressure = toNumeric[Lookup[step, "pressure", 0.], 0.];
+ dt = Max[0., toNumeric[Lookup[step, "dt", 1.], 1.]];
+ phiMPrev = toNumeric[state["phi_m"], 0.];
+ epsSwPrev = toNumeric[state["epsilon_sw"], 0.];
+ sigmaPrev = toNumeric[state["sigma_s_xx"], 0.];
+
+ micro = solveMicrostateStep[state, step, params];
+ phiM = Max[0., micro["phi_total_trial"] - micro["n_l"]];
+ phiTotal = micro["n_l"] + phiM;
+ nS = Max[params["nFloor"], 1. - phiTotal];
+ satMacro = saturationFromTuller[pressure, params];
+ nL = phiM * satMacro;
+
+ deltaEpsSw = params["SwellingSlope"] * (micro["n_l"] - phiMPrev);
+ epsSw = epsSwPrev + deltaEpsSw;
+ sigmaNew =
+ sigmaPrev + params["BulkModulus"] * deltaEpsV -
+ params["BulkModulus"] * deltaEpsSw;
+
+ <|
+ "step" -> stepIndex,
+ "pressure" -> pressure,
+ "dt" -> dt,
+ "delta_epsilon_v" -> deltaEpsV,
+ "S_L" -> satMacro,
+ "mu_LR" -> micro["mu_LR"],
+ "n_l" -> micro["n_l"],
+ "phi_m" -> micro["phi_m"],
+ "phi_M" -> phiM,
+ "phi" -> phiTotal,
+ "n_S" -> nS,
+ "n_L" -> nL,
+ "rho_LR" -> micro["rho_LR"],
+ "rho_LR" -> params["RhoLRMacro"],
+ "omega_l" -> omegaFromState[micro["n_l"], micro["rho_LR"], nS, params["RhoSR"]],
+ "mu_LR" -> micro["mu_LR"],
+ "rho_l_hat" -> micro["rho_l_hat"],
+ "delta_epsilon_sw" -> deltaEpsSw,
+ "epsilon_sw" -> epsSw,
+ "sigma_s_xx" -> sigmaNew,
+ "sigma_s_yy" -> sigmaNew,
+ "sigma_s_zz" -> sigmaNew,
+ "sigma_s_xy" -> 0.,
+ "sigma_s_yz" -> 0.,
+ "sigma_s_xz" -> 0.,
+ "solver_status" -> micro["solver_status"],
+ "n_upper_bound" -> micro["n_upper_bound"],
+ "mechanical_mode" -> params["MechanicalMode"]
+ |>
+];
+
+
+ RunHistory[steps_List, params_: Automatic, initialState_: Automatic] := Module[
+ {p, state, rows = {}, i, stepAssoc, row, stateKeys},
+ p = If[AssociationQ[params], params, CreateDefaultParameters[]];
+ state = If[AssociationQ[initialState], initialState, CreateDefaultInitialState[p]];
+ stateKeys = {"n_l", "rho_LR", "phi_m", "phi_M", "epsilon_sw", "sigma_s_xx", "sigma_s_yy", "sigma_s_zz"};
+
+ For[i = 1, i <= Length[steps], i++,
+ stepAssoc = If[AssociationQ[steps[[i]]], steps[[i]], <|"pressure" -> toNumeric[steps[[i]], 0.]]>;
+ row = advanceState[state, stepAssoc, p, i - 1];
+ AppendTo[rows, row];
+ state = Join[state, KeyTake[row, stateKeys]];
+ ];
+
+ <|
+ "rows" -> rows,
+ "final_state" -> state,
+ "parameters" -> p
+ |>
+];
+
+ resolveMultiplier[dryDensity_, params_Association, mode_] := Module[
+ {rules, xVals, yVals, interp},
+ Which[
+ NumericQ[mode], N[mode],
+ Head[mode] == Function, N[mode[dryDensity]],
+ AssociationQ[mode] && Length[mode] > 1,
+ rules = SortBy[Normal[mode], First];
+ xVals = rules[[All, 1]];
+ yVals = rules[[All, 2]];
+ interp = Interpolation[Transpose[{xVals, yVals}], InterpolationOrder -> 1];
+ N[interp[dryDensity]],
+ AssociationQ[mode] && KeyExistsQ[mode, dryDensity], N[mode[dryDensity]],
+ True, N[params["VdWMultiplier"]]
+ ]
+];
+
+ Options[RunDryDensitySweep] = {
+ "SolidDensity" -> 2700.,

```

```

+ "InitialMicroWaterFraction" -> 0.99,
+ "CaseSteps" -> 120,
+ "Pressure" -> 0.,
+ "DeltaEpsilonV" -> 0.,
+ "Dt" -> 1.,
+ "MultiplierByDryDensity" -> Automatic
+);
+
+RunDryDensitySweep[dryDensityList_List, params_: Automatic, opts : OptionsPattern[]] := Module[
+ {
+ p, solidDensity, microWaterFraction, caseSteps, pressure, deltaEpsV, dt,
+ multiplierMode, rows = {}, dd, phiTotal, phiM0, init, caseParams, steps,
+ result, finalRow, solverFallbackCount, multiplier
+ },
+ p = If[AssociationQ[params], params, CreateDefaultParameters[]];
+ solidDensity = OptionValue["SolidDensity"];
+ microWaterFraction = OptionValue["InitialMicroWaterFraction"];
+ caseSteps = Round[OptionValue["CaseSteps"]];
+ pressure = OptionValue["Pressure"];
+ deltaEpsV = OptionValue["DeltaEpsilonV"];
+ dt = OptionValue["Dt"];
+ multiplierMode = OptionValue["MultiplierByDryDensity"];
+
+ Do[
+ phiTotal = clip[1. - dd / solidDensity, {p["NFloor"], p["PorosityUpper"]}];
+ phiM0 = clip[microWaterFraction * phiTotal, {p["NFloor"], phiTotal}];
+ multiplier = resolveMultiplier[dd, p, multiplierMode];
+
+ caseParams = Join[p, <|"VdwMultiplier" -> multiplier|>];
+ init = <|
+ "n_l" -> phiM0,
+ "rho_LR" -> Max[p["RhoFloor"], p["RhoLMicro"]],
+ "phi_m" -> phiM0,
+ "phi_M" -> Max[0., phiTotal - phiM0],
+ "epsilon_sw" -> 0.,
+ "sigma_S_xx" -> 0.,
+ "sigma_S_yy" -> 0.,
+ "sigma_S_zz" -> 0.
+ |>;
+ steps = Table[
+ <|
+ "pressure" -> pressure,
+ "delta_epsilon_v" -> deltaEpsV,
+ "dt" -> dt,
+ "vdw_multiplier" -> multiplier
+ |>,
+ {caseSteps}
+ ];
+
+ result = RunHistory[steps, caseParams, init];
+ finalRow = Last[result["rows"]];
+ solverFallbackCount =
+ Count[Lookup[result["rows"], "solver_status", ""], "fallback_explicit"];
+
+ AppendTo[
+ rows,
+ <|
+ "dry_density_kg_m3" -> dd,
+ "phi_total_initial" -> phiTotal,
+ "vdw_multiplier" -> multiplier,
+ "final_n_l" -> finalRow["n_l"],
+ "final_phi_m" -> finalRow["phi_m"],
+ "final_phi_M" -> finalRow["phi_M"],
+ "final_phi" -> finalRow["phi"],
+ "final_rho_LR" -> finalRow["rho_LR"],
+ "final_mu_LR" -> finalRow["mu_LR"],
+ "final_rho_l_hat" -> finalRow["rho_l_hat"],
+ "final_swelling_stress_mpa" -> finalRow["sigma_S_xx"] / 1.*^6,
+ "fallback_step_count" -> solverFallbackCount
+ |>
+ ];
+ ,
+ {dd, dryDensityList}
+ ];
+
+ <|
+ "rows" -> rows,
+ "parameters" -> p,
+ "options" -> <|
+ "SolidDensity" -> solidDensity,
+ "InitialMicroWaterFraction" -> microWaterFraction,
+ "CaseSteps" -> caseSteps,
+ "Pressure" -> pressure,
+ "DeltaEpsilonV" -> deltaEpsV,
+ "Dt" -> dt
+ |>
+ |>
+ ];
+
+ExportHistory[result_Association, basePath_String] := Module[
+ {rows, csvPath, jsonPath, header},
+ rows = Lookup[result, "rows", {}];
+ csvPath = basePath << ".csv";
+ jsonPath = basePath << ".json";
+ If[rows == {},
+ Export[csvPath, {}, "CSV"],
+ header = Keys[First[rows]];
+ Export[
+ csvPath,
+ Prepend[Lookup[#, header] &] /@ rows, header],
+ "CSV"
+ ];
+ Export[jsonPath, result, "RawJSON"];
+ <|"csv" -> csvPath, "json" -> jsonPath|>
+ ];
+
+End[];
+
+EndPackage[];
diff --git a/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/README.md b/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/README.md
new file mode 100644
index 0000000000..4eb8ae1dfa
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/README.md
@@ -0,0 +1,62 @@
+
+## Mathematica DSM Driver Project

```

```

+
+This is a clean, reversible Mathematica (Wolfram Language) driver project for
+the DSM micro-macro constitutive replay used in OGS transition work.
+
+It is intended for:
+- single-integration-point constitutive checks,
+- microstate evolution replay ('n_l', 'rho_LR', 'phi_m', 'phi_M'),
+- swelling-strain/stress updates,
+- dry-density sweep studies with a vdW multiplier curve.
+
+It is not a full FE/process replacement for OGS BEACON/ANCHORS runs.
+
+## What is transferred here
+
+The project includes the recent fine-tuning logic:
+- finite microstate ceiling ('n_l' never unbounded),
+- 'n_l <= phi_total' at every step,
+- 'phi_m + phi_M == phi_total',
+- total porosity update by kinematics
+- 'phi_trial = (phi_prev + delta_eps_v)/(1 + delta_eps_v)' when valid,
+- robust fallback when local Newton does not converge.
+
+## Files
+
+- 'DsmMicromacroDriver.wl'
+- reusable package with model equations and driver functions.
+- 'run_dsm_driver_demo.wl'
+- runnable script that generates:
+  - pressure-path history,
+  - strain-coupled history,
+  - dry-density sweep.
+- '_outputs/'
+- generated CSV/JSON files (created by the demo script).
+
+## Run
+
+From this directory:
+
+```bash
+wolframscript -file run_dsm_driver_demo.wl
+```
+
+Expected outputs:
+
+- '_outputs/overlap_history.csv'
+- '_outputs/overlap_history.json'
+- '_outputs/strain_coupled_history.csv'
+- '_outputs/strain_coupled_history.json'
+- '_outputs/dry_density_sweep.csv'
+- '_outputs/dry_density_sweep.json'
+
+## Scope and comparability
+
+Comparable to OGS at constitutive-driver level:
+- same local microstate closure form,
+- same porosity bounds logic,
+- same exchange-driven storage update structure,
+- same swelling increment form from microstate change.
+
+For full benchmark-level parity ('1a01', '1b', '1c', EPFL/BGR, ANCHORS FE
+geometry and BCs), OGS process equations and FE discretization remain required.
+diff --git a/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/run_dsm_driver_demo.wl b/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/run_dsm_driver_demo.wl
+new file mode 100644
+index 0000000000..2550e6f6dc
+--- /dev/null
++++ b/Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/run_dsm_driver_demo.wl
+@@ -0,0 +1,119 @@
+
+ClearAll[
+  here, outputDir, calibrationCsvPath, params, initialState,
+  overlapSteps, overlapResult, overlapExport,
+  strainSteps, strainResult, strainExport,
+  calibrationRaw, calibrationHeader, ddCol, multCol, multCurve,
+  ddList, sweepResult, sweepExport, summaryPath, summary
+];
+
+here = DirectoryName[InputFileName];
+outputDir = FileNameJoin[{here, "_outputs"}];
+If[!DirectoryQ[outputDir], CreateDirectory[outputDir]];
+
+Get[FileNameJoin[{here, "DsmMicromacroDriver.wl"}]];
+
+params = DsmMicromacroDriver`CreateDefaultParameters[];
+initialState = DsmMicromacroDriver`CreateDefaultInitialState[params];
+
+overlapSteps = Table[
+  <|
+    "pressure" -> 250. * i,
+    "delta_epsilon_v" -> 0.,
+    "dt" -> 1.
+  |>,
+  {i, 0, 4}
+];
+
+overlapResult =
+  DsmMicromacroDriver`RunHistory[overlapSteps, params, initialState];
+overlapExport =
+  DsmMicromacroDriver`ExportHistory[
+    overlapResult,
+    FileNameJoin[{outputDir, "overlap_history"}]
+  ];
+
+strainSteps = Table[
+  <|
+    "pressure" -> 1000.,
+    "delta_epsilon_v" -> 1.*^-4,
+    "dt" -> 1.
+  |>,
+  {5}
+];
+
+strainResult =
+  DsmMicromacroDriver`RunHistory[strainSteps, params, overlapResult["final_state"]];
+strainExport =
+  DsmMicromacroDriver`ExportHistory[
+    strainResult,
+    FileNameJoin[{outputDir, "strain_coupled_history"}]
+  ];
+
+];

```

```

+
+calibrationCsvPath = FileNameJoin[
+ {here, "..", "ANCHORS_MS33_ModelI", "villar_dense_dd_calibration.csv"}
+];
+
+If[FileExistsQ[calibrationCsvPath],
+ calibrationRaw = Import[calibrationCsvPath, "CSV"];
+ calibrationHeader = First[calibrationRaw];
+ ddCol = First @ FirstPosition[calibrationHeader, "dry_density_kg_m3"];
+ multCol = First @ FirstPosition[calibrationHeader, "vdm_multiplier"];
+ multCurve =
+ Association @ Map[
+ (N[ToExpression#[[ddCol]]]] -> N[ToExpression#[[multCol]]]]) &,
+ Rest[calibrationRaw]
+ ],
+ multCurve = <|
+ 1400. -> 1.*^5,
+ 1450. -> 3.*^5,
+ 1500. -> 9.*^5,
+ 1600. -> 8.*^6,
+ 1700. -> 8.*^7,
+ 1800. -> 9.*^8
+ |>
+];
+
+ddList = Sort[Keys[multCurve]];
+sweepResult =
+ DsmMicromacroDriver`RunDryDensitySweep[
+ ddList,
+ params,
+ "SolidDensity" -> 2700.,
+ "InitialMicroWaterFraction" -> 0.99,
+ "CaseSteps" -> 120,
+ "Pressure" -> 0.,
+ "DeltaEpsilonV" -> 0.,
+ "Dt" -> 1.,
+ "MultiplierByDryDensity" -> multCurve
+ ];
+
+sweepExport =
+ DsmMicromacroDriver`ExportHistory[
+ sweepResult,
+ FileNameJoin[{outputDir, "dry_density_sweep"}]
+ ];
+
+summaryPath = FileNameJoin[{outputDir, "run_summary.json"}];
+summary = <|
+ "project" -> "Mathematica_DSM_Driver",
+ "overlap_rows" -> Length[overlapResult["rows"]],
+ "strain_rows" -> Length[strainResult["rows"]],
+ "dry_density_cases" -> Length[sweepResult["rows"]],
+ "calibration_curve_source" ->
+ If[FileExistsQ[calibrationCsvPath], calibrationCsvPath, "internal_fallback_curve"],
+ "exports" -> <|
+ "overlap" -> overlapExport,
+ "strain" -> strainExport,
+ "dry_density_sweep" -> sweepExport
+ |>
+ |>;
+
+Export[summaryPath, summary, "RawJSON"];
+
+Print["Mathematica DSM driver demo completed."];
+Print["Overlap history CSV: ", overlapExport["csv"]];
+Print["Strain history CSV: ", strainExport["csv"]];
+Print["Dry-density sweep CSV: ", sweepExport["csv"]];
+Print["Summary JSON: ", summaryPath];
+
+Exit[0];
diff --git a/Tests/Data/RichardsMechanics/SCRIPT_INVENTORY.md b/Tests/Data/RichardsMechanics/SCRIPT_INVENTORY.md
new file mode 100644
index 0000000000..36392c63f4
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/SCRIPT_INVENTORY.md
@@ -0,0 +1,62 @@
+# RichardsMechanics Benchmark Script Inventory
+
+*Last updated: 2026-04-15 (CEST)*
+*Repository branch: 'dsm-nb-mfront-transition'*
+
+*This file documents and consolidates the scripts used for DSM/native/Mfront
+calibration and benchmark exercises (ANCHORS, BEACON, EPFL/BGR)*.
+
+## Main End-to-End Workflows
+
+| Script | Purpose | Primary outputs |
+|---|---|---|
+| 'run_linear_low_calibrated_dsm_comparison.py' | ANCHORS linear-law calibration + triplet comparison ('linear', 'native_dsm', 'mfront_dsm') for ANCHORS/BEACON/EPFL. | '_outputs/linear_low_calibrated_dsm_comparison/*' |
+| 'run_identical_parameter_benchmark_parity.py' | Same-parameter parity run for ANCHORS + BEACON + EPFL ('native' vs 'mfront') using shared dry-density multiplier curve. | '_outputs/identical_parameter_native_mfront_comparison_with_epfl/*' |
+| 'run_calibrated_beacon_anchors_comparison.py' | Dry-density-calibrated comparison for ANCHORS + BEACON ('native' vs 'mfront') with threshold assertions. | '_outputs/calibrated_native_mfront_comparison/*' |
+
+## Calibration Scripts (ANCHORS)
+
+| Script | Scope | Notes |
+|---|---|---|
+| 'ANCHORS_MS33_ModelI/run_villar_dense_dd_calibration.py' | MFront DSM calibration across dry-density sweep. | Produces 'villar_dense_dd_calibration.csv'. |
+| 'ANCHORS_MS33_ModelI/run_villar_dense_dd_native_dsm_micromacro_calibration.py' | Native DSM calibration across dry-density sweep. | Produces 'villar_dense_dd_native_dsm_micromacro_calibration.csv'. |
+| 'ANCHORS_MS33_ModelI/run_villar_dense_dd_native_dsm_micromacro_branch.py' | Native DSM branch sweep with fixed settings (non-iterative branch response). | Useful for sanity checks vs target curves. |
+
+## BEACON / EPFL Analysis and Plotting
+
+| Script | Scope | Notes |
+|---|---|---|
+| 'plot_calibrated_benchmark_comparison.py' | Plots calibrated ANCHORS/BEACON native-vs-Mfront comparisons. | Reads calibrated delta CSVs. |
+| 'plot_beacon_report_figures.py' | BEACON report-style figure generation ('1a01', '1b', '1c'). | Produces report-panel PNGs. |
+| 'analyze_beacon_unstructured_batch.py' | Postprocessing for BEACON unstructured runs. | Metrics extraction helper. |
+| 'calibrate_beacon_1b_ddcurve_probe.py' | Dry-density / multiplier sensitivity probing for BEACON '1b'. | Diagnostic calibration probe. |
+| 'BEACON_MP3_BGR_EPFL/plot_bgr_mp3_epfl_report_figures.py' | EPFL/BGR report-style figure generation. | Uses native + Mfront EPFL decks. |
+| 'BEACON_MP3_BGR_EPFL/analyze_bgr_mp3_abprime.py' | EPFL/BGR AB* quantitative analysis helper. | CSV/JSON support utilities. |
+
+## Mesh/Geometry Helper Scripts
+

```

```

+| Script | Scope |
+|---|---|
+| 'generate_beacon_unstructured_meshes.py' | BEACON unstructured mesh generation. |
+| 'BEACON_WP3_BGR_EPFL/generate_bgr_epfl_mesh.py' | EPFL/BGR mesh and geometry generation. |
+
+## Ancillary / Reference Utilities
+
+| Script | Scope |
+|---|---|
+| 'run_beacon_native_bridge_report_batch.py' | Legacy native/bridge BEACON batch driver. |
+| 'run_native_branch_dsm_micromacro_comparison.py' | Native-branch comparison helper. |
+| 'ANCHORS_MS33_ModelI/plot_ms33_vs_ciemat_swelling_pressure_dd.py' | MS33 vs CIEMAT/Villar pressure-density plotting utility. |
+| 'ANCHORS_MS33_ModelI/summarize_ms33_model_i.py' | ANCHORS Model I summary helper. |
+| 'Mathematica_DSM_Driver/run_dsm_driver_demo.wl' | Mathematica DSM constitutive demo driver. |
+| 'Mathematica_DSM_Driver/DsmMicromacroDriver.wl' | Mathematica DSM constitutive equations/driver. |
+
+## Storage Convention
+
+Script sources are stored under:
+ - `Tests/Data/RichardsMechanics/`
+ - `Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/`
+ - `Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/`
+ - `Tests/Data/RichardsMechanics/Mathematica_DSM_Driver/`
+Generated run artifacts are stored under `_outputs/` and are intentionally git-ignored.
+If a new benchmark workflow is added, update this inventory in the same commit.
diff --git a/Tests/Data/RichardsMechanics/_outputs/.gitignore b/Tests/Data/RichardsMechanics/_outputs/.gitignore
new file mode 100644
index 0000000000..532cd4b292
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/_outputs/.gitignore
@@ -0,0 +1,3 @@
+# Keep local benchmark outputs outside Git tracking.
**
+!.gitignore
diff --git a/Tests/Data/RichardsMechanics/analyze_beacon_unstructured_batch.py b/Tests/Data/RichardsMechanics/analyze_beacon_unstructured_batch.py
new file mode 100644
index 0000000000..d525ae2f5a
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/analyze_beacon_unstructured_batch.py
@@ -0,0 +1,153 @@
#!/usr/bin/env python3
"""Summarize BEACON unstructured-batch native vs bridge results.

The script reads native and bridge VTU outputs, extracts boundary stresses and
density profiles, and writes a JSON payload for the report note.
"""
+
+from __future__ import annotations
+
+import argparse
+import json
+from pathlib import Path
+
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+REPORT_TARGETS = {
+    "1a01": {
+        "axial_swelling_pressure_kpa": 604.0,
+        "radial_swelling_pressure_kpa": 994.0,
+        "stage1_dry_density": 1655.0,
+        "post_mortem_density_profile": [1466.0, 1454.0, 1427.0, 1353.0],
+    },
+    "1b": {
+        "dry_density_after_volume_adjustment": 1520.0,
+        "swelling_pressure_note": "nonzero and stabilized after about 500 days",
+    },
+}
+
+def load_point_data(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray], int]:
+    """Load point coordinates, point data arrays, and cell count from a VTU."""
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    grid = reader.GetOutput()
+
+    points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+    point_data = {
+        grid.GetPointData().GetArrayName(i): vtk_to_numpy(grid.GetPointData().GetArray(i))
+        for i in range(grid.GetPointData().GetNumberOfArrays())
+    }
+    return points, point_data, int(grid.GetNumberOfCells())
+
+def boundary_mean(points: np.ndarray, values: np.ndarray, axis: int, atol: float = 1e-8) -> np.ndarray:
+    """Average a field over the max boundary of the selected axis."""
+    target = points[:, axis].max()
+    mask = np.isclose(points[:, axis], target, atol=atol)
+    return np.asarray(values[mask]).mean(axis=0)
+
+def density_profile(points: np.ndarray, density: np.ndarray, n_bins: int) -> list[dict[str, float]]:
+    """Bin a scalar field along the vertical coordinate for post-mortem plots."""
+    y = points[:, 1]
+    ymin = float(y.min())
+    ymax = float(y.max())
+    edges = np.linspace(ymin, ymax, n_bins + 1)
+    out = []
+    for i in range(n_bins):
+        lo = edges[i]
+        hi = edges[i + 1]
+        if i == n_bins - 1:
+            mask = (y >= lo) & (y <= hi)
+        else:
+            mask = (y >= lo) & (y < hi)
+        center_mm = 1000.0 * 0.5 * (lo + hi)
+        out.append(
+            {
+                "center_mm": float(center_mm),
+                "mean_kg_m3": float(np.asarray(density[mask]).mean()),
+                "count": int(mask.sum()),
+            }
+        )
+    return out

```

```

+
+
+def max_abs_diff(a: np.ndarray, b: np.ndarray) -> float:
+    """Return the maximum absolute difference between two arrays."""
+    return float(np.abs(np.asarray(a) - np.asarray(b)).max())
+
+
+def summarise(case: str, native_path: Path, bridge_path: Path) -> dict:
+    """Build the comparison payload for a BEACON case."""
+    native_points, native, native_n_cells = load_point_data(native_path)
+    bridge_points, bridge, bridge_n_cells = load_point_data(bridge_path)
+
+    if not np.allclose(native_points, bridge_points):
+        raise RuntimeError("Native and bridge outputs are not on the same point set.")
+    if native_n_cells != bridge_n_cells:
+        raise RuntimeError("Native and bridge outputs are not on the same cell set.")
+
+    bins = 4 if case == "1a01" else 10
+    native_sigma_top = boundary_mean(native_points, native["sigma"], axis=1)
+    native_sigma_side = boundary_mean(native_points, native["sigma"], axis=0)
+    bridge_sigma_top = boundary_mean(bridge_points, bridge["sigma"], axis=1)
+    bridge_sigma_side = boundary_mean(bridge_points, bridge["sigma"], axis=0)
+
+    native_sw_top = boundary_mean(native_points, native["swelling_stress"], axis=1)
+    native_sw_side = boundary_mean(native_points, native["swelling_stress"], axis=0)
+    bridge_sw_top = boundary_mean(bridge_points, bridge["swelling_stress"], axis=1)
+    bridge_sw_side = boundary_mean(bridge_points, bridge["swelling_stress"], axis=0)
+
+    fields_to_compare = ["pressure", "saturation", "dry_density_solid", "sigma", "swelling_stress"]
+    field_diffs = {field: max_abs_diff(native[field], bridge[field]) for field in fields_to_compare}
+
+    return {
+        "case": case,
+        "report_targets": REPORT_TARGETS[case],
+        "mesh": {
+            "n_points": int(native_points.shape[0]),
+            "n_cells": native_n_cells,
+        },
+        "native": {
+            "axial_swelling_pressure_kpa": abs(float(native_sigma_top[1])) / 1000.0,
+            "radial_swelling_pressure_kpa": abs(float(native_sigma_side[0])) / 1000.0,
+            "top_swelling_stress_kpa": abs(float(native_sw_top[1])) / 1000.0,
+            "side_swelling_stress_kpa": abs(float(native_sw_side[0])) / 1000.0,
+            "dry_density_mean_kg_m3": float(np.asarray(native["dry_density_solid"]).mean()),
+            "dry_density_profile_kg_m3": density_profile(native_points, native["dry_density_solid"], bins),
+        },
+        "bridge": {
+            "axial_swelling_pressure_kpa": abs(float(bridge_sigma_top[1])) / 1000.0,
+            "radial_swelling_pressure_kpa": abs(float(bridge_sigma_side[0])) / 1000.0,
+            "top_swelling_stress_kpa": abs(float(bridge_sw_top[1])) / 1000.0,
+            "side_swelling_stress_kpa": abs(float(bridge_sw_side[0])) / 1000.0,
+            "dry_density_mean_kg_m3": float(np.asarray(bridge["dry_density_solid"]).mean()),
+            "dry_density_profile_kg_m3": density_profile(bridge_points, bridge["dry_density_solid"], bins),
+        },
+        "native_vs_bridge_max_abs_diff": field_diffs,
+    }
+
+
+def main() -> None:
+    """Parse CLI arguments and write the comparison summary as JSON."""
+    parser = argparse.ArgumentParser()
+    parser.add_argument("--case", choices=["1a01", "1b"], required=True)
+    parser.add_argument("--native", type=Path, required=True)
+    parser.add_argument("--bridge", type=Path, required=True)
+    parser.add_argument("--json-out", type=Path)
+    args = parser.parse_args()
+
+    result = summarise(args.case, args.native, args.bridge)
+
+    text = json.dumps(result, indent=2, sort_keys=True)
+    if args.json_out:
+        args.json_out.write_text(text + "\n")
+    print(text)
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01.gml b/Tests/Data/RichardsMechanics/beacon_1a01.gml
new file mode 100644
index 0000000000..e0a5f644b0
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01.gml
@@ -0,0 +1,17 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="OpenGeoSysGLI.xsl"?>
<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
  <name>beacon_1a01_geometry</name>
  <points>
    <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
    <point id="1" x="0.0" y="0.02" z="0.0"/>
    <point id="2" x="0.025" y="0.0" z="0.0"/>
    <point id="3" x="0.025" y="0.02" z="0.0"/>
  </points>
  <polylines>
    <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt></polyline>
    <polyline id="1" name="right"><pnt>2</pnt><pnt>3</pnt></polyline>
    <polyline id="2" name="bottom"><pnt>0</pnt><pnt>2</pnt></polyline>
    <polyline id="3" name="top"><pnt>1</pnt><pnt>3</pnt></polyline>
  </polylines>
</OpenGeoSysGLI>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_axial_fit_native_bridge_report.json b/Tests/Data/RichardsMechanics/beacon_1a01_axial_fit_native_bridge_report.json
new file mode 100644
index 0000000000..8e5870c513
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_axial_fit_native_bridge_report.json
@@ -0,0 +1,93 @@
{
  "artifacts": {
    "bridge_project": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1a01_notebook_mcc_inflow_unstructured_batch.prj",
    "bridge_vtu": "/private/tmp/beacon_probe/axial_fit_1a01/1a01/bridge/beacon_1a01_notebook_mcc_inflow_unstructured_batch_t_100000.000000.vtu",
    "native_project": "/Users/vinaykumar/git/ogs-TPM_Swelling_MCC_Coupled/Tests/Data/RichardsMechanics/beacon_1a01_inflow_unstructured_batch.prj",
    "native_vtu": "/private/tmp/beacon_probe/axial_fit_1a01/1a01/native/beacon_1a01_inflow_unstructured_batch_t_100000.000000.vtu"
  },
  "bridge": {
    "axial_swelling_pressure_kpa": 603.6770403555414,
    "dry_density_mean_kg_m3": 1667.9991971436366,
    "dry_density_profile_kg_m3": [

```



```

+         "center_mm": 2.4999999441206455,
+         "count": 30,
+         "mean_kg_m3": 1668.0216933121987
+     },
+     {
+         "center_mm": 7.499999832361937,
+         "count": 20,
+         "mean_kg_m3": 1668.0112656941496
+     },
+     {
+         "center_mm": 12.499999720603228,
+         "count": 24,
+         "mean_kg_m3": 1667.9930855162202
+     },
+     {
+         "center_mm": 17.49999960884452,
+         "count": 26,
+         "mean_kg_m3": 1667.9695980279025
+     }
+ ],
+ "radial_swelling_pressure_kpa": 603.1119780253586,
+ "side_swelling_stress_kpa": 603.1325514821193,
+ "top_swelling_stress_kpa": 607.43395516703
+ },
+ "case": "1a01",
+ "executables": {
+     "bridge ogs": "/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs",
+     "native ogs": "/Users/vinaykumar/git/build/release-native-beacon/bin/ogs"
+ },
+ "mesh": {
+     "n_cells": 162,
+     "n_points": 100
+ },
+ "native": {
+     "axial_swelling_pressure_kpa": 18.82567139023727,
+     "dry_density_mean_kg_m3": 1667.99730131982,
+     "dry_density_profile_kg_m3": [
+         {
+             "center_mm": 2.4999999441206455,
+             "count": 30,
+             "mean_kg_m3": 1668.0052464380253
+         },
+         {
+             "center_mm": 7.499999832361937,
+             "count": 20,
+             "mean_kg_m3": 1668.0034747933882
+         },
+         {
+             "center_mm": 12.499999720603228,
+             "count": 24,
+             "mean_kg_m3": 1668.000158337015
+         },
+         {
+             "center_mm": 17.49999960884452,
+             "count": 26,
+             "mean_kg_m3": 1667.9807551878885
+         }
+     ],
+     "radial_swelling_pressure_kpa": 19.46676389634277,
+     "side_swelling_stress_kpa": 19.47728874712114,
+     "top_swelling_stress_kpa": 20.316411211729843
+ },
+ "native_vs_bridge_max_abs_diff": {
+     "dry_density_solid": 0.017909231821249705,
+     "pressure": 125858.63724190708,
+     "saturation": 0.008211593848047927,
+     "sigma": 586563.5031569699,
+     "swelling_stress": 587117.7394787471
+ },
+ "report_targets": {
+     "axial_swelling_pressure_kpa": 604.0,
+     "post_mortem_density_profile": [
+         1466.0,
+         1454.0,
+         1427.0,
+         1353.0
+     ],
+     "radial_swelling_pressure_kpa": 994.0,
+     "stage1_dry_density": 1655.0
+ }
+ }
+ }
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_domain.vtu b/Tests/Data/RichardsMechanics/beacon_1a01_domain.vtu
new file mode 100644
index 0000000000..f0b1058060
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_domain.vtu
@@ -0,0 +1,24 @@
<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian">
+ <UnstructuredGrid>
+   <Piece NumberOfPoints="4" NumberOfCells="1">
+     <PointData/>
+     <CellData>
+       <DataArray type="Int32" Name="MaterialIDs" format="ascii">0</DataArray>
+     </CellData>
+     <Points>
+       <DataArray type="Float64" NumberOfComponents="3" format="ascii">
+         0.0 0.0 0.0
+         0.025 0.0 0.0
+         0.025 0.02 0.0
+         0.0 0.02 0.0
+       </DataArray>
+     </Points>
+     <Cells>
+       <DataArray type="Int32" Name="connectivity" format="ascii">0 1 2 3</DataArray>
+       <DataArray type="Int32" Name="offsets" format="ascii">4</DataArray>
+       <DataArray type="UInt8" Name="types" format="ascii">9</DataArray>
+     </Cells>
+   </Piece>
+ </UnstructuredGrid>
</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_domain_unstructured_162e.vtu b/Tests/Data/RichardsMechanics/beacon_1a01_domain_unstructured_162e.vtu
new file mode 100644
index 0000000000..7694bd2463
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_domain_unstructured_162e.vtu
@@ -0,0 +1,216 @@

```

```

<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian" header_type="UInt32" compressor="vtkZLibDataCompressor">
  <UnstructuredGrid>
    <Piece NumberOfPoints="100" NumberOfCells="162">
      <PointData>
      </PointData>
      <CellData>
      </CellData>
      <Points>
        <DataArray type="Float32" Name="Points" NumberOfComponents="3" format="ascii" RangeMin="0" RangeMax="0.0320156211988001">
          0 0 0 0.0027777778 0 0
          0.0055555557 0 0 0.008333334 0 0
          0.011111111 0 0 0.013888889 0 0
          0.016666668 0 0 0.019444445 0 0
          0.022222223 0 0 0.025 0 0
          0 0.0022222223 0 0.0032213102 0.002109759 0
          0.005840361 0.0021827817 0 0.008127662 0.0025209622 0
          0.011480442 0.0024122729 0 0.014362066 0.0021918924 0
          0.016972162 0.0023366094 0 0.019415505 0.0019612538 0
          0.02261702 0.0020201597 0 0.025 0.0020201597 0
          0 0.0044444446 0 0.002668086 0.0043281876 0
          0.0057075284 0.0042058104 0 0.008340913 0.004102559 0
          0.010666314 0.0039028863 0 0.014229852 0.00415741 0
          0.016833993 0.00413388 0 0.019914288 0.0044060885 0
          0.02196729 0.004460126 0 0.025 0.004460126 0
          0 0.006666667 0 0.003233729 0.006895161 0
          0.005424906 0.0068710996 0 0.00835116 0.0071926145 0
          0.011325407 0.0071111247 0 0.01385093 0.0067849727 0
          0.016658701 0.0065681906 0 0.019037576 0.006241212 0
          0.021038048 0.006400513 0 0.025 0.006400513 0
          0 0.008888889 0 0.002456961 0.008943599 0
          0.0056581744 0.009316611 0 0.008291533 0.009536233 0
          0.010646005 0.009651918 0 0.0143691925 0.009289824 0
          0.017074317 0.009014447 0 0.019619694 0.00863467 0
          0.021783687 0.008342426 0 0.025 0.008342426 0
          0 0.011111111 0 0.0031791008 0.011337422 0
          0.005717344 0.011456988 0 0.008127094 0.011479472 0
          0.011092979 0.011840364 0 0.013843093 0.012037903 0
          0.016525319 0.012378243 0 0.019796817 0.012771751 0
          0.021949265 0.013068059 0 0.025 0.013068059 0
          0 0.013333334 0 0.0027547681 0.013340839 0
          0.0055973534 0.013014311 0 0.008714411 0.013097827 0
          0.010716418 0.0132890735 0 0.013502324 0.013194072 0
          0.016836192 0.013166281 0 0.019009413 0.013264091 0
          0.02208895 0.013169162 0 0.025 0.013169162 0
          0 0.015555556 0 0.0024407294 0.015302685 0
          0.0060486444 0.0153862415 0 0.008420758 0.015611023 0
          0.0109946495 0.015334351 0 0.013996054 0.015258218 0
          0.016570123 0.014928483 0 0.01936535 0.014570032 0
          0.02262418 0.014469912 0 0.025 0.014469912 0
          0 0.017777778 0 0.0029930957 0.017848441 0
          0.0054957 0.017719204 0 0.007865274 0.017842878 0
          0.010710565 0.017630719 0 0.014347797 0.01753139 0
          0.016529163 0.017683301 0 0.019214192 0.017577853 0
          0.0220297 0.017734112 0 0.025 0.017734112 0
          0 0.02 0 0.002777778 0.02 0
          0.0055555557 0.02 0 0.008333334 0.02 0
          0.011111111 0.02 0 0.013888889 0.02 0
          0.016666668 0.02 0 0.019444445 0.02 0
          0.022222223 0.02 0 0.025 0.02 0
        </DataArray>
        <InformationKey name="L2_NORM_RANGE" location="vtkDataArray" length="2">
          <Value index="0">
            0
          </Value>
          <Value index="1">
            0.032015621199
          </Value>
        </InformationKey>
      </DataArray>
    </Points>
    <Cells>
      <DataArray type="Int64" Name="connectivity" format="ascii" RangeMin="0" RangeMax="99">
        19 18 9 12 11 2
        15 5 6 22 11 12
        11 10 1 11 1 2
        12 2 3 13 3 4
        14 13 4 15 14 5
        16 15 6 17 7 8
        10 0 1 13 12 3
        32 31 22 14 4 5
        21 10 11 26 25 15
        22 21 11 17 16 6
        22 12 13 17 6 7
        24 23 13 18 17 8
        25 24 14 18 8 9
        26 16 17 29 18 19
        26 15 16 28 27 17
        21 20 10 23 22 13
        42 31 32 24 13 14
        31 30 21 25 14 15
        31 21 22 36 35 25
        33 22 23 27 26 17
        33 32 22 28 17 18
        34 24 25 29 28 18
        35 34 25 36 25 26
        38 37 27 30 20 21
        33 23 24 52 51 42
        34 33 24 41 30 31
        46 36 37 42 41 31
        37 36 26 42 32 33
        37 26 27 44 33 34
        38 27 28 45 44 34
        39 38 28 46 45 36
        39 28 29 45 35 36
        47 46 37 41 40 30
        43 42 33 62 51 52
        44 43 33 51 50 41
        45 34 35 51 41 42
        56 45 46 52 42 43
        47 37 38 54 53 44
        48 47 38 55 44 45
        49 48 38 56 55 45
        49 38 39 57 56 46
        59 58 49 57 46 47
        50 40 41 53 52 43
        72 61 62 53 43 44
        61 50 51 55 54 44
      </DataArray>
    </Cells>
  </Piece>
</UnstructuredGrid>
</VTKFile>

```



```

+      </convergence_criterion>
+      <time_discretization><type>BackwardEuler</type></time_discretization>
+      <time_stepping>
+        <type>FixedTimeStepping</type>
+        <t_initial>0.0</t_initial>
+        <t_end>1e3</t_end>
+        <timesteps><pair><repeat>20</repeat><delta_t>50</delta_t></pair></timesteps>
+      </time_stepping>
+    </process>
+  </processes>
+  <output>
+    <type>VTK</type>
+    <prefix>beacon_1a01_dsm_micromacro_mcc_bridge</prefix>
+    <suffix>_t_{:time}</suffix>
+    <fixed_output_times>1e3</fixed_output_times>
+    <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>porosity</variable><variable>transport_porosity</variable><variable>swelling_stress</variable><variable>sigma</variable><variable>EquivalentPlasticStrain</variable><variable>PreConsolidationPressure</variable><variable>PlasticVolumetricStrain</variable><variable>VolumeRatio</variable><variable>n_l</variable><variable>rho_LR</variable><variable>epsilon_sw</variable><variable>phi_m</variable><variable>phi</variable><variable>n_S</variable><variable>n_L</variable><variable>rho_LR</variable><variable>omega_l</variable><variable>mu_LR</variable><variable>rho_l_hat</variable><variable>delta_epsilon_sw</variable><variable>sigma_S</variable></variables>
+  </output>
+  </time_loop>
+  <parameters>
+    <parameter><name>sigma0</name><type>Function</type><expression></expression><expression></expression><expression></expression><expression></expression></parameter>
+    <parameter><name>E</name><type>Constant</type><value>506</value></parameter>
+    <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+    <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+    <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+    <parameter><name>displacement0</name><type>Constant</type><values><0 0</values></parameter>
+    <parameter><name>pressure_ic</name><type>Constant</type><values>-1e6</values></parameter>
+    <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+    <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+    <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+    <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+    <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+    <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+    <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>1.666666666666667</value></parameter>
+    <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+    <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+    <parameter><name>BubblePressure</name><type>Constant</type><value>1e6</value></parameter>
+    <parameter><name>VanGenuchtenExponent_n</name><type>Constant</type><value>0.5</value></parameter>
+    <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+    <parameter><name>MicroStateLocalSolveMode</name><type>Constant</type><value>1</value></parameter>
+    <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>0</value></parameter>
+    <parameter><name>MicroSwellingStrainSlope</name><type>Constant</type><value>0.0</value></parameter>
+    <parameter><name>MassExchangeCoefficient</name><type>Constant</type><value>1e-13</value></parameter>
+    <parameter><name>MacroViscosity</name><type>Constant</type><value>1000.0</value></parameter>
+    <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+    <parameter><name>ReferenceLiquidDensityMicro</name><type>Constant</type><value>1e-6</value></parameter>
+    <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+    <parameter><name>MicroLiquidDensityA</name><type>Constant</type><value>0.0</value></parameter>
+    <parameter><name>MicroLiquidDensityB</name><type>Constant</type><value>1.0</value></parameter>
+    <parameter><name>HamakerConstant</name><type>Constant</type><value>1e-30</value></parameter>
+    <parameter><name>SpecificSurface</name><type>Constant</type><value>1.0</value></parameter>
+    <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+    <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.854073464102069</value></parameter>
+    <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+    <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+    <parameter><name>n_l0</name><type>Constant</type><value>0.1</value></parameter>
+    <parameter><name>rho_LR0</name><type>Constant</type><value>1000.000001</value></parameter>
+    <parameter><name>epsilon_sw0</name><type>Constant</type><value>0.0</value></parameter>
+  </parameters>
+  <process_variables>
+    <process_variable>
+      <name>displacement</name>
+      <components>2</components>
+      <order>1</order>
+      <initial_condition>displacement0</initial_condition>
+      <boundary_conditions>
+        <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+        <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+        <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+        <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+      </boundary_conditions>
+    </process_variable>
+    <process_variable>
+      <name>pressure</name>
+      <components>1</components>
+      <order>1</order>
+      <initial_condition>pressure_ic</initial_condition>
+      <boundary_conditions>
+        <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure0</parameter></boundary_condition>
+      </boundary_conditions>
+    </process_variable>
+  </process_variables>
+  <nonlinear_solvers>
+    <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+  </nonlinear_solvers>
+  <linear_solvers>
+    <linear_solver><name>general_linear_solver</name><eigen>solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+  </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_bridge_ref
erence_t_1000.000000.vtu
new file mode 100644
index 0000000000..b5c3b523d8
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu
@@ -0,0 +1,50 @@
<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="1.0" byte_order="LittleEndian" header_type="UInt64" compressor="vtkLibDataCompressor">
  <UnstructuredGrid>
    <FieldData>
      <DataArray type="Int8" Name="OGS_VERSION" NumberOfTuples="25" format="appended" RangeMin="45" RangeMax="121" offset="0" />
    </FieldData>
    <Piece NumberOfPoints="4" NumberOfCells="1" >
      <PointData>
        <DataArray type="Float64" Name="EquivalentPlasticStrain" format="appended" RangeMin="0" RangeMax="0" offset="88" />
        <DataArray type="Float64" Name="PlasticVolumetricStrain" format="appended" RangeMin="0" RangeMax="0" offset="148" />
        <DataArray type="Float64" Name="PreConsolidationPressure" format="appended" RangeMin="10000000000" RangeMax="10000000000" offset="288" />
        <DataArray type="Float64" Name="VolumeRatio" format="appended" RangeMin="1.666666667" RangeMax="1.666666667" offset="288" />
      </PointData>
    </Piece>
  </UnstructuredGrid>
</VTKFile>

```

158

```

+       <secondary_variable name="EquivalentPlasticStrain"/>
+       <secondary_variable name="PreConsolidationPressure"/>
+       <secondary_variable name="PlasticVolumetricStrain"/>
+       <secondary_variable name="sigma"/>
+       <secondary_variable name="swelling_stress"/>
+       <secondary_variable name="epsilon"/>
+       <secondary_variable name="velocity"/>
+       <secondary_variable name="saturation"/>
+       <secondary_variable name="porosity"/>
+       <secondary_variable name="transport_porosity"/>
+       <secondary_variable name="dry_density_solid"/>
+       <secondary_variable name="VolumeRatio"/>
+       <secondary_variable name="n_L"/>
+       <secondary_variable name="rho_LR"/>
+       <secondary_variable name="epsilon_sw"/>
+       <secondary_variable name="phi_m"/>
+       <secondary_variable name="phi_M"/>
+       <secondary_variable name="phi"/>
+       <secondary_variable name="n_S"/>
+       <secondary_variable name="n_L"/>
+       <secondary_variable name="rho_LR"/>
+       <secondary_variable name="omega_L"/>
+       <secondary_variable name="mu_LR"/>
+       <secondary_variable name="rho_L_hat"/>
+       <secondary_variable name="delta_epsilon_sw"/>
+       <secondary_variable name="sigma_S"/>
+
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+
+   </process>
+ </processes>
+ <media>
+   <medium id="0">
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+           <property><name>density</name><type>Constant</type><value>1e3</value></property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property><name>density</name><type>Constant</type><value>2780</value></property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+       <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+       <property><name>permeability</name><type>Constant</type><value>1e-18</value></property>
+       <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+       <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+       <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+       <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+       <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+       <convergence_criterion>
+         <norm_type>NORM2</norm_type>
+         <type>PerComponentDeltaX</type>
+         <abstols>5e-8 1e-13 1e-13</abstols>
+       </convergence_criterion>
+       <time_discretization><type>BackwardEuler</type></time_discretization>
+       <time_stepping>
+         <type>IterationNumberBasedTimeStepping</type>
+         <t_initial>0.0</t_initial>
+         <t_end>1e5</t_end>
+         <initial_dt>10</initial_dt>
+         <minimum_dt>0.1</minimum_dt>
+         <maximum_dt>2e3</maximum_dt>
+         <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+         <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>beacon_1a01_dsm_micromacro_mcc_inflow_bridge</prefix>
+     <suffix>t_{:time}</suffix>
+     <fixed_output_times>1e3 1e4 5e4 1e5</fixed_output_times>
+     <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>porosity</variable><variable>transport_porosity</variable><variable>swelling_stress</variable><variable>sigma</variable><variable>EquivalentPlasticStrain</variable><variable>PreConsolidationPressure</variable><variable>PlasticVolumetricStrain</variable><variable>VolumeRatio</variable><variable>n_L</variable><variable>rho_LR</variable><variable>epsilon_sw</variable><variable>phi_m</variable><variable>phi_M</variable><variable>phi</variable><variable>n_S</variable><variable>n_L</variable><variable>rho_LR</variable><variable>omega_L</variable><variable>mu_LR</variable><variable>rho_L_hat</variable><variable>delta_epsilon_sw</variable><variable>sigma_S</variable></variables>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+   <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+   <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><value>0 0</value></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</value></parameter>
+   <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+   <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+   <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+   <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+   <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+   <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>1.6666666666666667</value></parameter>
+   <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>BubblePressure</name><type>Constant</type><value>1e6</value></parameter>

```

```

+ <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>0.5</value></parameter>
+ <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>MicroStateLocalSolveMode</name><type>Constant</type><value>1</value></parameter>
+ <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>1</value></parameter>
+ <parameter><name>MicroSwellingStrainSlope</name><type>Constant</type><value>0.1</value></parameter>
+ <parameter><name>MassExchangeCoefficient</name><type>Constant</type><value>1e-13</value></parameter>
+ <parameter><name>MacroViscosity</name><type>Constant</type><value>1e-3</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMicro</name><type>Constant</type><value>1300.0</value></parameter>
+ <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+ <parameter><name>MicroLiquidDensityA</name><type>Constant</type><value>1.3</value></parameter>
+ <parameter><name>MicroLiquidDensityB</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>HamakerConstant</name><type>Constant</type><value>1.9606596248030517e-15</value></parameter>
+ <parameter><name>SpecificSurface</name><type>Constant</type><value>523.0</value></parameter>
+ <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.8584073464102069</value></parameter>
+ <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+ <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+ <parameter><name>n_l0</name><type>Constant</type><value>0.0015338535</value></parameter>
+ <parameter><name>rho_LR0</name><type>Constant</type><value>2296.26173839</value></parameter>
+ <parameter><name>epsilon_sw0</name><type>Constant</type><value>0.0</value></parameter>
+ </parameters>
+ <process_variables>
+ <process_variable>
+ <name>displacement</name>
+ <components>2</components>
+ <order>1</order>
+ <initial_condition>displacement0</initial_condition>
+ <boundary_conditions>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirich
let0</parameter></boundary_condition>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>diric
hlet0</parameter></boundary_condition>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>diri
chlet0</parameter></boundary_condition>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichl
et0</parameter></boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ <process_variable>
+ <name>pressure</name>
+ <components>1</components>
+ <order>1</order>
+ <initial_condition>pressure_ic0</initial_condition>
+ <boundary_conditions>
+ <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pre
ssure</parameter></boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+ <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+ <linear_solver><name>general_linear_solver</name><eigen>solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_i
nflow_bridge_reference_t_100000.000000.vtu
new file mode 100644
index 0000000000..ec70868ee7
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_inflow_bridge_reference_t_100000.000000.vtu
@@ -0,0 +1,50 @@
<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="1.0" byte_order="LittleEndian" header_type="UInt64" compressor="vtkZLibDataCompressor">
+ <UnstructuredGrid>
+ <FieldData>
+ <DataArray type="Int8" Name="OGS_VERSION" NumberOfTuples="25" format="appended" RangeMin="45" RangeMax="121" offset="0" />
+ </FieldData>
+ <Piece NumberOfPoints="4" NumberOfCells="1" >
+ <PointData>
+ <DataArray type="Float64" Name="EquivalentPlasticStrain" format="appended" RangeMin="0" RangeMax="0" offset="88" />
+ <DataArray type="Float64" Name="PlasticVolumetricStrain" format="appended" RangeMin="0" RangeMax="0" offset="148" />
+ <DataArray type="Float64" Name="PreConsolidationPressure" format="appended" RangeMin="10000000000" RangeMax="10000000000" offset="208" />
+ <DataArray type="Float64" Name="VolumeRatio" format="appended" RangeMin="1.666666667" RangeMax="1.666666667" offset="288" />
+ <DataArray type="Float64" Name="delta_epsilon_sw" format="appended" RangeMin="0.00011209760832" RangeMax="0.00011209760832" offset="364" />
+ <DataArray type="Float64" Name="displacement" NumberOfComponents="2" format="appended" RangeMin="0" RangeMax="0" offset="448" />
+ </PointData>
+ <DataArray type="Float64" Name="epsilon_sw" format="appended" RangeMin="0.021651504559" RangeMax="0.021856815881" offset="508" />
+ <DataArray type="Float64" Name="mu_LR" format="appended" RangeMin="-5768.9651951" RangeMax="-5609.0214074" offset="592" />
+ <DataArray type="Float64" Name="n_L" format="appended" RangeMin="-0.001381190823" RangeMax="0.0068653105286" offset="676" />
+ <DataArray type="Float64" Name="n_S" format="appended" RangeMin="0.6" RangeMax="0.6" offset="760" />
+ <DataArray type="Float64" Name="n_L1" format="appended" RangeMin="0.21804889914" RangeMax="0.22010201236" offset="832" />
+ <DataArray type="Float64" Name="omega_l1" format="appended" RangeMin="0.31490399080" RangeMax="0.31786907517" offset="916" />
+ <DataArray type="Float64" Name="phi" format="appended" RangeMin="0.4" RangeMax="0.4" offset="1000" />
+ <DataArray type="Float64" Name="phi_L1" format="appended" RangeMin="0.17989798764" RangeMax="0.18195110086" offset="1076" />
+ <DataArray type="Float64" Name="phi_m" format="appended" RangeMin="0.21804889914" RangeMax="0.22010201236" offset="1160" />
+ <DataArray type="Float64" Name="porosity" format="appended" RangeMin="0.4" RangeMax="0.4" offset="1244" />
+ <DataArray type="Float64" Name="pressure" format="appended" RangeMin="-376782.51875" RangeMax="2000" offset="1320" />
+ <DataArray type="Float64" Name="rho_LR" format="appended" RangeMin="2296.2617384" RangeMax="2296.2617384" offset="1400" />
+ <DataArray type="Float64" Name="rho_LR1" format="appended" RangeMin="2296.2617384" RangeMax="2296.2617384" offset="1476" />
+ <DataArray type="Float64" Name="rho_hat" format="appended" RangeMin="0.0012870271529" RangeMax="0.0012881781248" offset="1552" />
+ <DataArray type="Float64" Name="saturation" format="appended" RangeMin="0.94466979615" RangeMax="1.0109815675" offset="1636" />
+ <DataArray type="Float64" Name="sigma" NumberOfComponents="4" format="appended" RangeMin="1041708.4988" RangeMax="1051586.5444" offset="1720" />
+ <DataArray type="Float64" Name="sigma_S" NumberOfComponents="4" format="appended" RangeMin="1041708.4988" RangeMax="1051586.5444" offset="1824" />
+ <DataArray type="Float64" Name="swelling_stress" NumberOfComponents="4" format="appended" RangeMin="1041708.4988" RangeMax="1051586.5444" offset="1932" />
+ <DataArray type="Float64" Name="transport_porosity" format="appended" RangeMin="0.3" RangeMax="0.3" offset="2040" />
+ </PointData>
+ <CellData>
+ </CellData>
+ <Points>
+ <DataArray type="Float64" Name="Points" NumberOfComponents="3" format="appended" RangeMin="0" RangeMax="0.032015621187" offset="2112" />
+ </Points>
+ <Cells>
+ <DataArray type="Int64" Name="connectivity" format="appended" RangeMin="" RangeMax="" offset="2196" />
+ <DataArray type="Int64" Name="offsets" format="appended" RangeMin="" RangeMax="" offset="2268" />
+ <DataArray type="UInt8" Name="types" format="appended" RangeMin="" RangeMax="" offset="2328" />
+ </Cells>
+ </Piece>
+ </UnstructuredGrid>
+ <AppendedData encoding="base64">

```



```
+ _AQAAAAAAAAAAGAAAAAABkAAAAAAITQAAAAAA=ef4z8zPVM9M1NjXVTtDmMUl0tr00UvJLcqpBAMgbyAQAAAAAAAAAAGAAAAAAACwAAAAAA=ef5jYMApAAGAAE=AQAAAAAAAAAAGAAAAAAACAAAA
AAAAcWAAAAAA=ef5jYMApAAGAAE=AQAAAAAAAAAAGAAAAAAACAAAAAAAGwAAAAAA=ef7777+/fPwCJ1cG8gYfEM8CpF9CxQHFVauHAQAAAAAAAGAAAAAAACAAAAAAAFgAAAAAA=ef5btQ0efTmvNLroPrYKA0
AhqMUlQ==AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef5rUP4jv59J1r4Zsm87/7jK1I1Pwfg2UBgDsHA5cAQAAAAAAAGAAAAAAACAAAAAAADAAAAAA=ef5jYKMApAAGAAQABQAAAAAAAGAAAAAAACAAAA
AAAAHgAAAAAA=ef7zPh6/7zZNMsvKH37wRzRRYn7K9CaQAAxXB8AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef6Tmcum/r1j2wE5K3J14nMMV68ut8xKhNAAG8RHAQAAAAAAAGAAAAAAACAAAAAAH
AAAA=ef5bwTirTnFh2ZSNUIPkd7XDZRa7M9CaDQ/TA/aQAAAAAAAGAAAAAAACAAAAAAAFQAAAAAA=ef4zNaBx/ZGUnOShT1aQCSAA10AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef5zkojdy/bytL8L1P5
336/BV+em/RcoD0DZF9rAQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef6B0RMEbtPgtLz0PQK8AN8cSvA=AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef6B0RMEbtPgtLz0PQK8AN8cSvA=
=AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef57K3WVmfRuP07KGbmdTTU7j9nZQGGABKQ+LAQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef5zkojdy/bytL8L1P5336/BV+em/RcoD0DZF9rAQAAAAAA
AgAAAAAAACAAAAAAAGAAAAAAAHgAAAAAA=ef6B0RMEbtPgtLz0PQK8AN8cSvA=AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef7B7mkbPVf70B0KM0Aag7zHABALfvCT4=AQAAAAAAAGAAAAAAACAAAAAAAH
AAAAAA=ef4LJYJ1P12h0Mg1A6G8n50GgNyAz7AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef4LJYJ1P12h0Mg1A6G8n50GgNyAz7AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef771fz9oZSYqP8PK3
FumUXk3sovRqUBg1daPaQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef5bvuk/Zh6rd/ZL0hSg0rJtB50P9t50GgAr0B0AQAAAAAAAGAAAAAAAHgAAAAAA=ef678Ty9dm2U0sEraD0DFKCl0tP9m7
vsX5X0h0w+r70hXG1YPAE1fL0=AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef67/yT9dm2U0sEraD0DFNyb8TFpmhyv3uPdbv50XQaJt8G5aPTMHkARNcv0w=AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA
=ef67/yT9dm2U0sEraD0DFNyb8TFpmhyv3uPdbv50XQaJt8G5aPTMHkARNcv0w=AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA=ef4zNaBx/ZGUnOShT1aQCSAA10AQAAAAAAAGAAAAAAACAAAAAAAHgAAAAAA
=ef5jYMA0Z8EA3t4tU169wFvk3BKcXBwCmHBDQAAAAAAAGAAAAAAACAAAAAAAEwAAAAAA=ef5jYIAAR1jNBKMZ0tQAHAABw==AQAAAAAAAGAAAAAAACAAAAAAACwAAAAAA=ef5jYAAAAAQAUI=AQAAAA
AAAAAAGAAAAAAACAAAAAAACAAAAAA=ef7jBAACgAK
+ </AppendedData>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch.prj b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_inflow_unstruct
ured_batch.prj
new file mode 100644
index 0000000000..ef693cc31c
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch.prj
@@ -0,0 +1,218 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+ <mesh axially_symmetric="true">beacon_1a01_domain_unstructured_162e.vtu</mesh>
+ <geometry>beacon_1a01.gml</geometry>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <constitutive_relation>
+ <type>MFrontRichardsMechanics</type>
+ <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="E"/>
+ <material_property name="PoissonRatio" parameter="nu"/>
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+ <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+ <material_property name="BubblePressure" parameter="BubblePressure"/>
+ <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+ <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode"/>
+ <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode"/>
+ <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention"/>
+ <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope"/>
+ <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient"/>
+ <material_property name="MacroViscosity" parameter="MacroViscosity"/>
+ <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro"/>
+ <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro"/>
+ <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid"/>
+ <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA"/>
+ <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB"/>
+ <material_property name="HamakerConstant" parameter="HamakerConstant"/>
+ <material_property name="SpecificSurface" parameter="SpecificSurface"/>
+ <material_property name="AreaFactorTuller" parameter="AreaFactorTuller"/>
+ <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller"/>
+ <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize"/>
+ <material_property name="SurfaceTension" parameter="SurfaceTension"/>
+ <material_property name="InitialPorosity" parameter="phi0"/>
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+ <state_variable name="n_l" parameter="n_l0"/>
+ <state_variable name="rho_LR" parameter="rho_LR0"/>
+ <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable name="ElasticStrain"/>
+ <secondary_variable name="EquivalentPlasticStrain"/>
+ <secondary_variable name="PreConsolidationPressure"/>
+ <secondary_variable name="PlasticVolumetricStrain"/>
+ <secondary_variable name="sigma"/>
+ <secondary_variable name="swelling_stress"/>
+ <secondary_variable name="epsilon"/>
+ <secondary_variable name="velocity"/>
+ <secondary_variable name="saturation"/>
+ <secondary_variable name="porosity"/>
+ <secondary_variable name="transport_porosity"/>
+ <secondary_variable name="dry_density_solid"/>
+ <secondary_variable name="VolumeRatio"/>
+ <secondary_variable name="n_l"/>
+ <secondary_variable name="rho_LR"/>
+ <secondary_variable name="epsilon_sw"/>
+ <secondary_variable name="phi_m"/>
+ <secondary_variable name="phi_M"/>
+ <secondary_variable name="phi"/>
+ <secondary_variable name="n_S"/>
+ <secondary_variable name="n_L"/>
+ <secondary_variable name="rho_LR"/>
+ <secondary_variable name="omega_l"/>
+ <secondary_variable name="mu_LR"/>
+ <secondary_variable name="rho_l_hat"/>
+ <secondary_variable name="delta_epsilon_sw"/>
+ <secondary_variable name="sigma_S"/>
+ </secondary_variables>
+ <specific_body_force>0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+ <medium id="0">
+ <phases>
+ <phase>
+ <type>AqueousLiquid</type>

```

```

+
+       <properties>
+       <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+       <property><name>density</name><type>Constant</type><value>1e3</value></property>
+       </properties>
+
+       </phase>
+       <phase>
+       <type>Solid</type>
+       <properties>
+       <property><name>density</name><type>Constant</type><value>2780</value></property>
+       </properties>
+       </phase>
+       </phases>
+       <properties>
+       <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+       <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+       <property><name>permeability</name><type>Constant</type><value>1e-18</value></property>
+       <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+       <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+       <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+       <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><pb>1e6</pb></property>
+       <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+       </properties>
+       </medium>
+     </media>
+     <time_loop>
+       <processes>
+       <process ref="RM">
+       <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+       <convergence_criterion>
+       <norm_type>NORM2</norm_type>
+       <type>PerComponentDeltaX</type>
+       <abstols>5e-8 1e-13 1e-13</abstols>
+       </convergence_criterion>
+       <time_discretization><type>BackwardEuler</type></time_discretization>
+       <time_stepping>
+       <type>IterationNumberBasedTimeStepping</type>
+       <t_initial>0.0</t_initial>
+       <t_end>1e5</t_end>
+       <initial_dt>10</initial_dt>
+       <minimum_dt>0.1</minimum_dt>
+       <maximum_dt>2e3</maximum_dt>
+       <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+       <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+       </time_stepping>
+       </process>
+     </processes>
+     <output>
+     <type>VTK</type>
+     <prefix>beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch</prefix>
+     <suffix>_t(:time)</suffix>
+     <fixed_output_times>1e3 1e4 5e4 1e5</fixed_output_times>
+     <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>porosity</variable><variable>transport_porosity</variable><variable>dry_density_solid</variable><variable>swelling_stress</variable><variable>sigma</variable><variable>EquivalentPlasticStrain</variable><variable>PreConsolidationPressure</variable><variable>PlasticVolumetricStrain</variable><variable>VolumeRatio</variable><variable>n_l</variable><variable>rho_LR</variable><variable>epsilon_sw</variable><variable>phi_mc</variable><variable>phi_M</variable><variable>phi</variable><variable>n_S</variable><variable>n_L</variable><variable>rho_LR</variable><variable>omega_L</variable><variable>rho_LR</variable><variable>rho_L_hat</variable><variable>delta_epsilon_sw</variable><variable>sigma_S</variable></variables>
+     </output>
+   </time_loop>
+   <parameters>
+   <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+   <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+   <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</value></parameter>
+   <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+   <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+   <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+   <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+   <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+   <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>1.6666666666666667</value></parameter>
+   <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>BubblePressure</name><type>Constant</type><value>1e6</value></parameter>
+   <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>0.5</value></parameter>
+   <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>MicroStateLocalSolveMode</name><type>Constant</type><value>1</value></parameter>
+   <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>1</value></parameter>
+   <parameter><name>MicroSwellingStrainSlope</name><type>Constant</type><value>0.1</value></parameter>
+   <parameter><name>MassExchangeCoefficient</name><type>Constant</type><value>1e-13</value></parameter>
+   <parameter><name>MacroViscosity</name><type>Constant</type><value>1e-3</value></parameter>
+   <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+   <parameter><name>ReferenceLiquidDensityMicro</name><type>Constant</type><value>1300.0</value></parameter>
+   <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+   <parameter><name>MicroLiquidDensityA</name><type>Constant</type><value>1.3</value></parameter>
+   <parameter><name>MicroLiquidDensityB</name><type>Constant</type><value>1.0</value></parameter>
+   <parameter><name>HamakerConstant</name><type>Constant</type><value>1.960596248030517e-15</value></parameter>
+   <parameter><name>SpecificSurface</name><type>Constant</type><value>523.0</value></parameter>
+   <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+   <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.854073464102069</value></parameter>
+   <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+   <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+   <parameter><name>n_l0</name><type>Constant</type><value>0.00153385355</value></parameter>
+   <parameter><name>rho_LR0</name><type>Constant</type><value>2296.26173839</value></parameter>
+   <parameter><name>epsilon_sw0</name><type>Constant</type><value>0.0</value></parameter>
+   </parameters>
+   <process_variables>
+   <process_variable>
+   <name>displacement</name>
+   <components>2</components>
+   <order>1</order>
+   <initial_condition>displacement0</initial_condition>
+   <boundary_conditions>
+   <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+   <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+   <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+   <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+   </boundary_conditions>

```

```

+      </process_variable>
+    </process_variable>
+    <name>pressure</name>
+    <components>1</components>
+    <order>1</order>
+    <initial_condition>pressure_ic</initial_condition>
+    <boundary_conditions>
+      <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pre
ssure</parameter></boundary_condition>
+    </boundary_conditions>
+  </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1a01_inflow_unstructured_batch.prj b/Tests/Data/RichardsMechanics/beacon_1a01_inflow_unstructured_batch.prj
new file mode 100644
index 0000000000..2f5ff5993c
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1a01_inflow_unstructured_batch.prj
@@ -0,0 +1,192 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+  <mesh axially_symmetric="true">beacon_1a01_domain_unstructured_162e.vtu</mesh>
+  <geometry>beacon_1a01.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <micro_porosity>
+        <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+        <nonlinear_solver>
+          <maximum_iterations>100</maximum_iterations>
+          <residuum_tolerance>1e-8</residuum_tolerance>
+          <increment_tolerance>1e-20</increment_tolerance>
+        </nonlinear_solver>
+      </micro_porosity>
+      <!-- Literature-backed stage-1 suction split:
+      total suction = 100 MPa is represented as 1 MPa macro
+      capillary suction plus 99 MPa microscale vdW suction.
+      The macro part enters through pressure_ic, while the micro
+      part is encoded through the initial micro water content under
+      the literature-backed Hamaker and specific-surface values. -->
+      <potential_exchange>
+        <enabled>true</enabled>
+        <mode>full_potential</mode>
+        <pressure_tolerance>1e-12</pressure_tolerance>
+        <hamaker_constant>1.9606596248030517e-15</hamaker_constant>
+        <specific_surface>523</specific_surface>
+        <micro_solid_density_reference>2650</micro_solid_density_reference>
+        <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+        <initial_micro_water_content>0.0015338535</initial_micro_water_content>
+        <local_nonlinear_solve_mode>scalar_microstate_storage_mode</local_nonlinear_solve_mode>
+        <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+        <micro_potential_convention>negative_attractive</micro_potential_convention>
+        <micro_solid_volume_fraction_mode>reference</micro_solid_volume_fraction_mode>
+        <micro_water_content_swelling_slope>0.1</micro_water_content_swelling_slope>
+      </potential_exchange>
+      <constitutive_relation>
+        <type>MFront</type>
+        <behaviour>ModCamClay_semiExpl_constE</behaviour>
+        <material_properties>
+          <material_property name="YoungModulus" parameter="E"/>
+          <material_property name="PoissonRatio" parameter="nu"/>
+          <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+          <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+          <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+          <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+        </material_properties>
+        <initial_values>
+          <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+          <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+        </initial_values>
+      </constitutive_relation>
+      <process_variables>
+        <pressure>pressure</pressure>
+        <displacement>displacement</displacement>
+      </process_variables>
+      <secondary_variables>
+        <secondary_variable name="sigma"/>
+        <secondary_variable name="swelling_stress"/>
+        <secondary_variable name="epsilon"/>
+        <secondary_variable name="velocity"/>
+        <secondary_variable name="saturation"/>
+        <secondary_variable name="porosity"/>
+        <secondary_variable name="transport_porosity"/>
+        <secondary_variable name="dry_density_solid"/>
+        <secondary_variable name="micro_saturation"/>
+        <secondary_variable name="micro_pressure"/>
+        <secondary_variable name="micro_water_content"/>
+        <secondary_variable name="micro_porosity"/>
+        <secondary_variable name="micro_exchange_source"/>
+      </secondary_variables>
+      <specific_body_force>0 0</specific_body_force>
+      <initial_stress>sigma0</initial_stress>
+      <mass_lumping>true</mass_lumping>
+    </process>
+  </processes>
+  <media>
+    <medium id="0">
+      <phases>
+        <phase>
+          <type>AqueousLiquid</type>
+          <properties>
+            <property><name>viscosity</name><type>Constant</type><value>1e-3</value></property>
+            <property><name>density</name><type>Constant</type><value>1e3</value></property>
+          </properties>
+        </phase>
+        <phase>
+          <type>Solid</type>
+          <properties>
+            <property><name>density</name><type>Constant</type><value>2780</value></property>

```

```

+         <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>1.3e7 1.3e7 1.3e7</swelling_pressures>
+             <exponents>1 1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+         </property>
+     </properties>
+ </phase>
+ </phases>
+ <properties>
+     <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+     <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+     <property><name>permeability</name><type>Constant</type><value>1e-18</value></property>
+     <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+     <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+     <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+     <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+     <property><name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</residual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+     <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>nonlinear_solver</nonlinear_solver>
+             <convergence_criterion>
+                 <norm_type>NORM2</norm_type>
+                 <type>PerComponentDeltaX</type>
+                 <abstols>5e-8 1e-13 1e-13</abstols>
+             </convergence_criterion>
+             <time_discretization><type>BackwardEuler</type></time_discretization>
+             <time_stepping>
+                 <type>IterationNumberBasedTimeStepping</type>
+                 <t_initial>0.0</t_initial>
+                 <t_end>1e5</t_end>
+                 <initial_dt>10</initial_dt>
+                 <minimum_dt>0.1</minimum_dt>
+                 <maximum_dt>2e3</maximum_dt>
+                 <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+                 <multiplier>1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+             </time_stepping>
+         </process>
+     </processes>
+ </output>
+     <type>VTK</type>
+     <prefix>beacon_1a01_inflow_unstructured_batch</prefix>
+     <suffix>_t(:time)</suffix>
+     <fixed_output_times>1e3 1e4 5e4 1e5</fixed_output_times>
+     <variables><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</variable><variable>micro_water_content</variable><variable>micro_porosity</variable><variable>micro_exchange_source</variable><variable>swelling_stress</variable><variable>sigma</variable><variable>porosity</variable><variable>transport_porosity</variable><variable>dry_density_solid</variable></variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+     <parameter><name>E</name><type>Constant</type><value>50e6</value></parameter>
+     <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+     <parameter><name>phi0</name><type>Constant</type><value>0.4</value></parameter>
+     <parameter><name>phi_tr0</name><type>Constant</type><value>0.3</value></parameter>
+     <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+     <parameter><name>pressure_ic</name><type>Constant</type><values>-1e6</values></parameter>
+     <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+     <parameter><name>top_pressure</name><type>Constant</type><value>2e3</value></parameter>
+     <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+     <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+     <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+     <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+     <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>1.666666666666667</value></parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+     <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+             <boundary_condition><geometrical_set>beacon_1a01_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver><name>general_linear_solver</name><eigen>solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1b.gml b/Tests/Data/RichardsMechanics/beacon_1b.gml
new file mode 100644
index 0000000000..57a680ff8e
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1b.gml
@@ -0,0 +1,17 @@

```

```

+<?xml version="1.0" encoding="ISO-8859-1"?>
+<?xml-styleSheet type="text/xml" href="OpenGeoSysGLI.xsl"?>
+<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+  <name=beacon_1b_geometry</name>
+  <points>
+    <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
+    <point id="1" x="0.0" y="0.10515" z="0.0"/>
+    <point id="2" x="0.12" y="0.0" z="0.0"/>
+    <point id="3" x="0.12" y="0.10515" z="0.0"/>
+  </points>
+  <polyLines>
+    <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt></polyline>
+    <polyline id="1" name="right"><pnt>2</pnt><pnt>3</pnt></polyline>
+    <polyline id="2" name="bottom"><pnt>0</pnt><pnt>2</pnt></polyline>
+    <polyline id="3" name="top"><pnt>1</pnt><pnt>3</pnt></polyline>
+  </polyLines>
+</OpenGeoSysGLI>
diff --git a/Tests/Data/RichardsMechanics/beacon_1b_domain.vtu b/Tests/Data/RichardsMechanics/beacon_1b_domain.vtu
new file mode 100644
index 0000000000..ab8879784b
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1b_domain.vtu
@@ -0,0 +1,24 @@
+<?xml version="1.0"?>
+<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian">
+  <UnstructuredGrid>
+    <Piece NumberOfPoints="4" NumberOfCells="1">
+      <PointData>
+      <CellData>
+        <DataArray type="Int32" Name="MaterialIDs" format="ascii">0</DataArray>
+      </CellData>
+      <Points>
+        <DataArray type="Float64" NumberOfComponents="3" format="ascii">
+          0.0 0.0 0.0
+          0.12 0.0 0.0
+          0.12 0.10515 0.0
+          0.0 0.10515 0.0
+        </DataArray>
+      </Points>
+      <Cells>
+        <DataArray type="Int32" Name="connectivity" format="ascii">0 1 2 3</DataArray>
+        <DataArray type="Int32" Name="offsets" format="ascii">4</DataArray>
+        <DataArray type="UInt8" Name="types" format="ascii">9</DataArray>
+      </Cells>
+    </Piece>
+  </UnstructuredGrid>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1b_domain_unstructured_162e.vtu b/Tests/Data/RichardsMechanics/beacon_1b_domain_unstructured_162e.vtu
new file mode 100644
index 0000000000..4723449b48
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1b_domain_unstructured_162e.vtu
@@ -0,0 +1,216 @@
+<?xml version="1.0"?>
+<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian" header_type="UInt32" compressor="vtkZLibDataCompressor">
+  <UnstructuredGrid>
+    <Piece NumberOfPoints="100" NumberOfCells="162">
+      <PointData>
+      </PointData>
+      <CellData>
+      </CellData>
+      <Points>
+        <DataArray type="Float32" Name="Points" NumberOfComponents="3" format="ascii" RangeMin="0" RangeMax="0.15955100031207056">
+          0 0 0 0.013333334 0 0
+          0.026666667 0 0 0.04 0 0
+          0.053333335 0 0 0.066666667 0 0
+          0.08 0 0.093333334 0 0
+          0.10666667 0 0 0.12 0 0
+          0 0.011683334 0 0.012595513 0.01144635 0
+          0.02818229 0.01223494 0 0.039042044 0.011983177 0
+          0.053638186 0.013399495 0 0.06528618 0.015142898 0
+          0.07911394 0.016313247 0 0.093452096 0.015677791 0
+          0.10694172 0.015618344 0 0.12 0.015618344 0
+          0 0.023366667 0 0.011665196 0.023145778 0
+          0.028720314 0.02381108 0 0.038175773 0.024658423 0
+          0.054298395 0.02463714 0 0.06440779 0.024865607 0
+          0.07895525 0.02504524 0 0.09275876 0.024635464 0
+          0.10876673 0.026503198 0 0.12 0.026503198 0
+          0 0.03505 0 0.011761169 0.033985347 0
+          0.028992172 0.03422083 0 0.04090181 0.034591835 0
+          0.051520053 0.034813818 0 0.064473845 0.03271515 0
+          0.07769469 0.03272928 0 0.09136155 0.030773973 0
+          0.10473867 0.03087559 0 0.12 0.03087559 0
+          0 0.046733335 0 0.013969353 0.045536723 0
+          0.02865906 0.044198867 0 0.03967901 0.044787392 0
+          0.053929023 0.04354663 0 0.06521151 0.0425226 0
+          0.08031936 0.043946795 0 0.09176506 0.04389581 0
+          0.105325736 0.045072876 0 0.12 0.045072876 0
+          0 0.05841667 0 0.012831001 0.056963284 0
+          0.02749507 0.05713023 0 0.04078791 0.05594444 0
+          0.052204654 0.055194236 0 0.06831729 0.05431181 0
+          0.08090663 0.054852653 0 0.09415413 0.054611426 0
+          0.10789165 0.054533467 0 0.12 0.054533467 0
+          0 0.0701 0 0.012177786 0.07009593 0
+          0.027561523 0.06849423 0 0.04110511 0.06853418 0
+          0.05527294 0.06863539 0 0.068499245 0.06728224 0
+          0.07824339 0.067254 0 0.090950705 0.06723592 0
+          0.104772806 0.068635255 0 0.12 0.068635255 0
+          0 0.08178333 0 0.0145890275 0.08158359 0
+          0.024883153 0.082896344 0 0.042386886 0.082954444 0
+          0.05254661 0.08257858 0 0.068063736 0.081316605 0
+          0.07842056 0.08008477 0 0.09502599 0.08107417 0
+          0.1088265 0.082865775 0 0.12 0.082865775 0
+          0 0.09346667 0 0.013601928 0.092430145 0
+          0.027336007 0.09391938 0 0.041613974 0.09193082 0
+          0.055669285 0.09170697 0 0.06538936 0.09078866 0
+          0.08210076 0.08996065 0 0.092884205 0.08840833 0
+          0.10650624 0.08902084 0 0.12 0.08902084 0
+          0 0.10515 0 0.013333334 0.10515 0
+          0.026666667 0.10515 0 0.04 0.10515 0
+          0.053333335 0.10515 0 0.066666667 0.10515 0
+          0.08 0.10515 0 0.093333334 0.10515 0
+          0.10666667 0.10515 0 0.12 0.10515 0
+        </DataArray>
+        <InformationKey name="L2_NORM_RANGE" location="vtkDataArray" length="2">
+          <Value index="0">
+            0
+          </Value>

```

```

+         <Value index="1">
+             0.15955100031
+         </Value>
+     </InformationKey>
+ </DataArray>
+ </Points>
+ <Cells>
+     <DataArray types="Int64" Name="connectivity" format="ascii" RangeMin="0" RangeMax="99">
+         18 8 9 12 2 3
+         16 15 5 22 12 13
+         11 0 1 11 1 2
+         12 11 2 13 12 3
+         14 13 4 15 14 5
+         16 5 6 17 6 7
+         11 10 0 13 3 4
+         32 31 22 14 4 5
+         21 10 11 26 15 16
+         21 11 12 17 16 6
+         22 21 12 17 7 8
+         23 22 13 18 17 8
+         24 23 14 29 28 19
+         26 25 15 19 18 9
+         27 26 16 27 17 18
+         21 20 10 23 13 14
+         43 42 32 24 14 15
+         31 20 21 25 24 15
+         31 21 22 37 26 27
+         32 22 23 27 16 17
+         33 23 24 28 27 18
+         35 34 24 28 18 19
+         36 35 25 36 25 26
+         38 27 28 31 30 20
+         33 32 23 52 41 42
+         34 33 24 41 40 31
+         35 24 25 41 31 32
+         46 36 37 42 41 32
+         37 36 26 43 33 34
+         38 37 27 44 43 34
+         39 38 28 45 35 36
+         39 28 29 46 45 36
+         49 48 39 47 37 38
+         40 30 31 43 32 33
+         62 51 52 44 34 35
+         51 40 41 45 44 35
+         52 51 41 56 46 47
+         53 52 42 47 46 37
+         54 43 44 48 47 38
+         54 53 43 48 38 39
+         55 45 46 56 55 46
+         57 47 48 51 50 40
+         53 42 43 72 71 62
+         55 54 44 61 50 51
+         55 44 45 62 61 51
+         66 55 56 63 62 52
+         57 56 47 63 53 54
+         58 57 48 64 63 54
+         58 48 49 66 65 55
+         59 58 49 67 66 56
+         68 67 57 61 60 50
+         63 52 53 82 81 72
+         64 54 55 71 70 61
+         65 64 55 71 61 62
+         77 76 67 73 72 62
+         67 56 57 74 73 63
+         68 57 58 75 74 64
+         69 68 58 75 64 65
+         69 58 59 76 65 66
+         77 67 68 70 60 61
+         73 62 63 91 81 82
+         74 63 64 81 70 71
+         76 75 65 81 71 72
+         76 66 67 83 72 73
+         88 87 77 83 73 74
+         78 77 68 85 74 75
+         78 68 69 86 85 75
+         79 78 69 87 86 76
+         88 77 78 81 80 70
+         83 82 72 84 83 74
+         91 80 81 85 84 74
+         92 91 82 86 75 76
+         93 92 82 87 76 77
+         94 83 84 97 86 87
+         95 94 84 89 88 78
+         96 85 86 89 78 79
+         96 95 85 97 87 88
+         91 90 80 93 82 83
+         94 93 83 95 84 85
+         97 96 86 98 97 88
+         98 88 89 99 98 89
+     </DataArray>
+     <DataArray types="Int64" Name="offsets" format="ascii" RangeMin="3" RangeMax="486">
+         3 6 9 12 15 18
+         21 24 27 30 33 36
+         39 42 45 48 51 54
+         57 60 63 66 69 72
+         75 78 81 84 87 90
+         93 96 99 102 105 108
+         111 114 117 120 123 126
+         129 132 135 138 141 144
+         147 150 153 156 159 162
+         165 168 171 174 177 180
+         183 186 189 192 195 198
+         201 204 207 210 213 216
+         219 222 225 228 231 234
+         237 240 243 246 249 252
+         255 258 261 264 267 270
+         273 276 279 282 285 288
+         291 294 297 300 303 306
+         309 312 315 318 321 324
+         327 330 333 336 339 342
+         345 348 351 354 357 360
+         363 366 369 372 375 378
+         381 384 387 390 393 396
+         399 402 405 408 411 414
+         417 420 423 426 429 432
+         435 438 441 444 447 450
+         453 456 459 462 465 468

```


169

```

+
+      <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>Constant</type><value>1e3</value></property></properties></phase>
+
+      <phase><type>Solid</type><properties><property><name>density</name><type>Constant</type><value>2788</value></property></properties></phase>
+
+    </phases>
+
+    <properties>
+      <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+      <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+      <property><name>permeability</name><type>Constant</type><value>5e-16</value></property>
+      <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity>1</maximal_porosity></property>
+      <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+      <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+      <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0.5</residual_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+      <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+    </properties>
+
+  </medium>
+
+ </media>
+
+ <time_loop>
+   <processes><process ref="RM"><nonlinear_solver>nonlinear_solver</nonlinear_solver><convergence_criterion>norm_type=NORM2</norm_type><type>PerComponentDelta</type><absol_s>5e-8 1e-13</absol_s><convergence_criterion>time_discretization<type>BackwardEuler</type><time_discretization>time_stepping<type>FixedTimeStepping</type><t_initial>0.0</t_initial><t_end>4.32e7</t_end><time_steps>pairs</repeat>400</repeat><delta_t>1.08e5</delta_t></pairs></time_steps></time_stepping></processes></processes>
+   <output><type>VTK</type><prefix>beacon_1b_dsm_micromacro_mcc_unstructured_batch</prefix><suffix>t_(time)</suffix><fixed_output_times>4.32e7</fixed_output_times><variable_s><variable>pressure</variable><variable>displacement</variable><variable>saturation</variable><variable>porosity</variable><variable>transport_porosity</variable><variable>dry_density_solid</variable><variable>swelling_stress</variable><variable>sigma</variable><variable>EquivalentPlasticStrain</variable><variable>PreConsolidationPressure</variable><variable>PlasticVolumetricStrain</variable><variable>VolumeRatio</variable><variable>n_L</variable><variable>rho_LR</variable><variable>epsilon_sw</variable><variable>phi_m</variable><variable>phi_M</variable><variable>phi</variable><variable>n_5</variable><variable>n_L</variable><variable>rho_LR</variable><variable>omega_L</variable><variable>mu_LR</variable><variable>rho_L_hat</variable><variable>delta_epsilon_sw</variable><variable>sigma_5</variable></variables></output>
+ </time_loop>
+
+ <parameters>
+   <parameter><name>sigma0</name><type>Function</type><expression></expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+   <parameter><name>E</name><type>Constant</type><value>40e6</value></parameter>
+   <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+   <parameter><name>phi0</name><type>Constant</type><value>0.5</value></parameter>
+   <parameter><name>phi_tr0</name><type>Constant</type><value>0.4</value></parameter>
+   <parameter><name>displacement0</name><type>Constant</type><values>0 0</values></parameter>
+   <parameter><name>pressure_ic</name><type>Constant</type><values>-1e6</values></parameter>
+   <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>top_pressure</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>bottom_pressure</name><type>Constant</type><value>1e4</value></parameter>
+   <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+   <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+   <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+   <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+   <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>2.0</value></parameter>
+   <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>BubblePressure</name><type>Constant</type><value>1e6</value></parameter>
+   <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>0.5</value></parameter>
+   <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+   <parameter><name>MicroStateLocalSolveMode</name><type>Constant</type><value>1</value></parameter>
+   <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>1</value></parameter>
+   <parameter><name>MicroSwellingStrainSlope</name><type>Constant</type><value>0.0</value></parameter>
+   <parameter><name>MassExchangeCoefficient</name><type>Constant</type><value>1e-13</value></parameter>
+   <parameter><name>MacroViscosity</name><type>Constant</type><value>1e-3</value></parameter>
+   <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+   <parameter><name>ReferenceLiquidDensityMicro</name><type>Constant</type><value>1300.0</value></parameter>
+   <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+   <parameter><name>MicroLiquidDensityA</name><type>Constant</type><value>1.3</value></parameter>
+   <parameter><name>MicroLiquidDensityB</name><type>Constant</type><value>1.0</value></parameter>
+   <parameter><name>HamakerConstant</name><type>Constant</type><value>5.1e-21</value></parameter>
+   <parameter><name>SpecificSurface</name><type>Constant</type><value>523.0</value></parameter>
+   <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+   <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.8584073464102069</value></parameter>
+   <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+   <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+   <parameter><name>n_L0</name><type>Constant</type><value>0.00153385355</value></parameter>
+   <parameter><name>rho_LR0</name><type>Constant</type><value>2296.26173839</value></parameter>
+   <parameter><name>epsilon_sw0</name><type>Constant</type><value>0.0</value></parameter>
+ </parameters>
+
+ <process_variables>
+   <process_variable><name>displacement</name><components>2</components><order>1</order><initial_condition>displacement0</initial_condition><boundary_conditions>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+   </boundary_conditions></process_variable>
+   <process_variable><name>pressure</name><components>1</components><order>1</order><initial_condition>pressure_ic</initial_condition><boundary_conditions>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+     <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>0</component><parameter>bottom_pressure</parameter></boundary_condition>
+   </boundary_conditions></process_variable>
+ </process_variables>
+
+ <nonlinear_solvers><nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solvers></nonlinear_solvers>
+
+ <linear_solvers><linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver></linear_solvers>
+
+ </OpenGeoSysProject>
+
+ diff --git a/Tests/Data/RichardsMechanics/beacon_1b_unstructured_batch.prj b/Tests/Data/RichardsMechanics/beacon_1b_unstructured_batch.prj
+ new file mode 100644
+ index 0000000000..df38a9f6cf
+ --- /dev/null
+ +++ b/Tests/Data/RichardsMechanics/beacon_1b_unstructured_batch.prj
+ @@ -0,0 +1,110 @@
+ <?xml version="1.0" encoding="ISO-8859-1"?>
+ <OpenGeoSysProject>
+   <mesh axially_symmetric="true">beacon_1b_domain_unstructured_162e.vtu</mesh>
+   <geometry>beacon_1b.gml</geometry>
+   <processes>
+     <process>
+       <name>RM</name>
+       <type>RICHARDS_MECHANICS</type>
+       <integration_order>2</integration_order>
+       <micro_porosity>
+         <mass_exchange_coefficient>1e-13</mass_exchange_coefficient>
+         <nonlinear_solver><maximum_iterations>100</maximum_iterations><residuum_tolerance>1e-8</residuum_tolerance><increment_tolerance>1e-20</increment_tolerance></nonlinear_solver>
+       <micro_porosity>
+         <potential_exchange>
+           <enabled>true</enabled>
+           <mode>full_potential</mode>
+           <pressure_tolerance>1e-12</pressure_tolerance>
+         </potential_exchange>
+       </micro_porosity>
+     </process>
+   </processes>
+ </OpenGeoSysProject>

```

```

+
+      <hamaker_constant>5.1e-21</hamaker_constant>
+      <specific_surface>523</specific_surface>
+      <micro_solid_density_reference>2650</micro_solid_density_reference>
+      <micro_solid_volume_fraction_reference>0.6</micro_solid_volume_fraction_reference>
+      <initial_micro_water_content>0.0015338535</initial_micro_water_content>
+      <local_nonlinear_solve_mode>scalar_microstate_storage_mode</local_nonlinear_solve_mode>
+      <micro_potential_convention>negative_attractive</micro_potential_convention>
+      <micro_solid_volume_fraction_mode>reference</micro_solid_volume_fraction_mode>
+      <!-- micro_water_content_swelling_slope intentionally omitted here:
+           this native 1b deck fails at time step 2 when the slope term is enabled
+           with the current long-horizon setup and fixed stepping. -->
+      <fd_jacobian_for_exchange>false</fd_jacobian_for_exchange>
+
+    </potential_exchange>
+
+    <constitutive_relation>
+      <type>MFront</type>
+      <behaviour>ModCamClay_semiExpl_constE</behaviour>
+
+      <material_properties>
+        <material_property name="YoungModulus" parameter="E"/>
+        <material_property name="PoissonRatio" parameter="nu"/>
+        <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+        <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+        <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+        <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+      </material_properties>
+
+      <initial_values>
+        <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+        <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+      </initial_values>
+
+      </constitutive_relation>
+
+      <process_variables><pressure>pressure</pressure><displacement>displacement</displacement></process_variables>
+
+      <secondary_variables>
+        <secondary_variable name="sigma"><secondary_variable name="swelling_stress"><secondary_variable name="epsilon"><secondary_variable name="velocity"><secondary_v
+variable name="saturation"><secondary_variable name="porosity"><secondary_variable name="transport_porosity"><secondary_variable name="dry_density_solid"><secondary_variable nam
+e="micro_saturation"><secondary_variable name="micro_pressure"/>
+      </secondary_variables>
+
+      <specific_body_force>0 0</specific_body_force>
+      <initial_stress>sigma</initial_stress>
+      <mass_lumping>true</mass_lumping>
+
+    </process>
+
+  </processes>
+
+  <media>
+    <medium id="0">
+      <phases>
+        <phase><type>AqueousLiquid</type><properties><property><name>viscosity</name><type>Constant</type><value>1e-3</value></property><property><name>density</name><type>
+Constant</type><value>1e3</value></property></properties></phase>
+        <phase><type>Solid</type><properties><property><name>density</name><type>Constant</type><value>2780</value></property><property><name>swelling_stress_rate</name><type>
+SaturationDependentSwelling</type><swelling_pressures>1.3e7 1.3e7 1.3e7</swelling_pressures><exponents>1 1 1</exponents><lower_saturation_limit>0</lower_saturation_limit><upper
+_saturation_limit>1</upper_saturation_limit></property></properties></phase>
+      </phases>
+
+      <properties>
+        <property><name>transport_porosity</name><type>TransportPorosityFromMassBalance</type><initial_porosity>phi_tr0</initial_porosity><minimal_porosity>0</minimal_poro
+sity><maximal_porosity>1</maximal_porosity></property>
+        <property><name>biot_coefficient</name><type>Constant</type><value>1.0</value></property>
+        <property><name>permeability</name><type>Constant</type><value>5e-16</value></property>
+        <property><name>porosity</name><type>PorosityFromMassBalance</type><initial_porosity>phi0</initial_porosity><minimal_porosity>0</minimal_porosity><maximal_porosity
+>1</maximal_porosity></property>
+        <property><name>reference_temperature</name><type>Constant</type><value>293.15</value></property>
+        <property><name>relative_permeability</name><type>Constant</type><value>1</value></property>
+        <property><name>saturation</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0.0</residual_liquid_saturation><residual_gas_saturation>0</residua
+_gas_saturation><exponent>0.5</exponent><p_b>1e6</p_b></property>
+        <property><name>saturation_micro</name><type>SaturationVanGenuchten</type><residual_liquid_saturation>0</residual_liquid_saturation><residual_gas_saturation>0</res
+idual_gas_saturation><exponent>0.5</exponent><p_b>1e7</p_b></property>
+        <property><name>bishops_effective_stress</name><type>BishopsPowerLaw</type><exponent>1</exponent></property>
+      </properties>
+
+    </medium>
+
+  </media>
+
+  <time_loop>
+    <processes><process ref="RM"><nonlinear_solver>nonlinear_solver</nonlinear_solver><convergence_criterion><norm_type>NORM2</norm_type><type>PerComponentDelta</type><abstol
+s>5e-8 1e-13 1e-13</abstols></convergence_criterion><time_discretization><type>BackwardEuler</type></time_discretization><time_stepping><type>FixedTimeStepping</type><t_initial>0.0
+</t_initial><t_end>4.32e7</t_end><time_steps>pair</repeat>400</repeat><delta_t>1.08e5</delta_t></pair></time_steps></time_stepping></process></processes>
+
+    <output><type>VTX</type><prefix>beacon_1b_unstructured_batch</prefix><suffix>_t_{time}</suffix><fixed_output_times>4.32e7</fixed_output_times><variables><variable>pressur
+e</variable><variable>displacement</variable><variable>saturation</variable><variable>micro_pressure</variable><variable>micro_saturation</variable><variable>swelling_stress</varia
+ble><variable>sigma</variable><variable>porosity</variable><variable>transport_porosity</variable><variable>dry_density_solid</variable></variables></output>
+
+  </time_loop>
+
+  <parameters>
+    <parameter><name>sigma0</name><type>Function</type><expression>0</expression><expression>0</expression><expression>0</expression><expression>0</expression></parameter>
+    <parameter><name>E</name><type>Constant</type><value>40e6</value></parameter>
+    <parameter><name>nu</name><type>Constant</type><value>0.2</value></parameter>
+    <parameter><name>phi0</name><type>Constant</type><value>0.5</value></parameter>
+    <parameter><name>phi_tr0</name><type>Constant</type><value>0.4</value></parameter>
+    <parameter><name>displacement0</name><type>Constant</type><value>0 0</values></parameter>
+    <parameter><name>pressure_ic</name><type>Constant</type><value>-1e6</values></parameter>
+    <parameter><name>dirichlet0</name><type>Constant</type><value>0</value></parameter>
+    <parameter><name>top_pressure</name><type>Constant</type><value>0</value></parameter>
+    <parameter><name>bottom_pressure</name><type>Constant</type><value>1e4</value></parameter>
+    <parameter><name>CriticalStateLineSlope</name><type>Constant</type><value>1.2</value></parameter>
+    <parameter><name>SwellingLineSlope</name><type>Constant</type><value>6.6e-3</value></parameter>
+    <parameter><name>VirginConsolidationLineSlope</name><type>Constant</type><value>7.7e-2</value></parameter>
+    <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+    <parameter><name>InitialVolumeRatio</name><type>Constant</type><value>2.0</value></parameter>
+  </parameters>
+
+  <process_variables>
+    <process_variable><name>displacement</name><components>2</components><order>1</order><initial_condition>displacement0</initial_condition><boundary_conditions>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>Left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</
+parameter></boundary_condition>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</p
+parameter></boundary_condition>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0
+</parameter></boundary_condition>
+
+    <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</p
+parameter></boundary_condition>
+
+  </boundary_conditions></process_variable>
+
+  <process_variables><name>pressure</name><components>1</components><order>1</order><initial_condition>pressure_ic</initial_condition><boundary_conditions>
+
+  <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</
+parameter></boundary_condition>
+
+  <boundary_condition><geometrical_set>beacon_1b_geometry</geometrical_set><geometry>bottom</geometry><type>Dirichlet</type><component>0</component><parameter>bottom_pre
+ssure</parameter></boundary_condition>
+
+  </boundary_conditions></process_variable>
+
+  </process_variables>
+
+  <nonlinear_solvers><nonlinear_solver><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_sol
+ver></nonlinear_solvers>
+
+  <linear_solvers><linear_solver><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver></linear_solvers>
+
+  </OpenGeoSysProject>
+
+diff --git a/Tests/Data/RichardsMechanics/beacon_1c.gml b/Tests/Data/RichardsMechanics/beacon_1c.gml
+new file mode 100644
+index 0000000000..029f23e82c
+--- /dev/null

```

```

+++ b/Tests/Data/RichardsMechanics/beacon_1c.gml
@@ -0,0 +1,19 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<?xml-stylesheet type="text/xsl" href="OpenGeoSysGLI.xsl"?>
+<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+  <name>beacon_1c_geometry</name>
+  <points>
+    <point id="0" x="0.0" y="0.0" z="0.0" name="origin"/>
+    <point id="1" x="0.0" y="0.0485" z="0.0"/>
+    <point id="2" x="0.0" y="0.1" z="0.0"/>
+    <point id="3" x="0.05" y="0.0" z="0.0"/>
+    <point id="4" x="0.05" y="0.0485" z="0.0"/>
+    <point id="5" x="0.05" y="0.1" z="0.0"/>
+  </points>
+  <polylines>
+    <polyline id="0" name="left"><pnt>0</pnt><pnt>1</pnt><pnt>2</pnt></polyline>
+    <polyline id="1" name="right"><pnt>3</pnt><pnt>4</pnt><pnt>5</pnt></polyline>
+    <polyline id="2" name="bottom"><pnt>0</pnt><pnt>3</pnt></polyline>
+    <polyline id="3" name="top"><pnt>2</pnt><pnt>5</pnt></polyline>
+  </polylines>
+</OpenGeoSysGLI>
diff --git a/Tests/Data/RichardsMechanics/beacon_1c_domain.vtu b/Tests/Data/RichardsMechanics/beacon_1c_domain.vtu
new file mode 100644
index 0000000000..c3cee8524f
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1c_domain.vtu
@@ -0,0 +1,26 @@
+<?xml version="1.0"?>
+<VTKFile type="UnstructuredGrid" version="0.1" byte_order="LittleEndian">
+  <UnstructuredGrid>
+    <Piece NumberOfPoints="6" NumberOfCells="2">
+      <PointData>
+        <CellData>
+          <DataArray type="Int32" Name="MaterialIDs" format="ascii">0 1</DataArray>
+        </CellData>
+        <Points>
+          <DataArray type="Float64" NumberOfComponents="3" format="ascii">
+            0.0 0.0 0.0
+            0.05 0.0 0.0
+            0.0 0.0485 0.0
+            0.05 0.0485 0.0
+            0.0 0.1 0.0
+            0.05 0.1 0.0
+          </DataArray>
+        </Points>
+        <Cells>
+          <DataArray type="Int32" Name="connectivity" format="ascii">0 1 3 2 3 5 4</DataArray>
+          <DataArray type="Int32" Name="offsets" format="ascii">4 8</DataArray>
+          <DataArray type="UInt8" Name="types" format="ascii">9 9</DataArray>
+        </Cells>
+      </Piece>
+    </UnstructuredGrid>
+</VTKFile>
diff --git a/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_mcc_bridge.prj b/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_mcc_bridge.prj
new file mode 100644
index 0000000000..4424fe7439
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_mcc_bridge.prj
@@ -0,0 +1,209 @@
+<?xml version="1.0" encoding="ISO-8859-1"?>
+<OpenGeoSysProject>
+  <mesh axially_symmetric="true">beacon_1c_domain.vtu</mesh>
+  <geometry>beacon_1c.gml</geometry>
+  <processes>
+    <process>
+      <name>RM</name>
+      <type>RICHARDS_MECHANICS</type>
+      <integration_order>2</integration_order>
+      <constitutive_relation id="0">
+        <type>MFrontRichardsMechanics</type>
+        <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+        <material_properties>
+          <material_property name="YoungModulus" parameter="E"/>
+          <material_property name="PoissonRatio" parameter="nu"/>
+          <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+          <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+          <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+          <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+          <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+          <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+          <material_property name="BubblePressure" parameter="BubblePressure_block"/>
+          <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+          <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode"/>
+          <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode"/>
+          <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention"/>
+          <material_property name="SwellingSlope" parameter="DSMMicroMacroSwellingSlope_block"/>
+          <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient_block"/>
+          <material_property name="MacroViscosity" parameter="MacroViscosity_block"/>
+          <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro"/>
+          <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro_block"/>
+          <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid"/>
+          <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA_block"/>
+          <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB_block"/>
+          <material_property name="HamakerConstant" parameter="HamakerConstant"/>
+          <material_property name="SpecificSurface" parameter="SpecificSurface_block"/>
+          <material_property name="AreaFactorTuller" parameter="AreaFactorTuller"/>
+          <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller"/>
+          <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize"/>
+          <material_property name="SurfaceTension" parameter="SurfaceTension"/>
+          <material_property name="InitialPorosity" parameter="phi0_block"/>
+        </material_properties>
+        <initial_values>
+          <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+          <state_variable name="VolumeRatio" parameter="InitialVolumeRatio_block"/>
+          <state_variable name="n_l" parameter="n_l_block"/>
+          <state_variable name="rho_lr" parameter="rho_lr_block"/>
+          <state_variable name="epsilon_sw" parameter="epsilon_sw_block"/>
+        </initial_values>
+      </constitutive_relation>
+    </process>
+  </processes>
+  <constitutive_relation id="1">
+    <type>MFrontRichardsMechanics</type>
+    <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+    <material_properties>
+      <material_property name="YoungModulus" parameter="E"/>
+      <material_property name="PoissonRatio" parameter="nu"/>
+      <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+      <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>

```



```

+ <parameter><name>InitialPreConsolidationPressure</name><type>Constant</type><value>1e10</value></parameter>
+ <parameter><name>ResidualLiquidSaturation</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>ResidualGasSaturation</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>VanGenuchtenExponent_m</name><type>Constant</type><value>0.5</value></parameter>
+ <parameter><name>MacroSaturationLawMode</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>MicroStateLocalSolveMode</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>MicroPotentialConvention</name><type>Constant</type><value>0</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMacro</name><type>Constant</type><value>1000.0</value></parameter>
+ <parameter><name>ReferenceDensitySolid</name><type>Constant</type><value>2650.0</value></parameter>
+ <parameter><name>HamakerConstant</name><type>Constant</type><value>1e-30</value></parameter>
+ <parameter><name>AreaFactorTuller</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>PoreAreaShapeFactorTuller</name><type>Constant</type><value>0.8584073464102069</value></parameter>
+ <parameter><name>CharacteristicPoreSize</name><type>Constant</type><value>1e-5</value></parameter>
+ <parameter><name>SurfaceTension</name><type>Constant</type><value>0.0715</value></parameter>
+ <parameter><name>BubblePressure_block</name><type>Constant</type><value>1e6</value></parameter>
+ <parameter><name>BubblePressure_pellet</name><type>Constant</type><value>5e5</value></parameter>
+ <parameter><name>DSMMicroMacroSwellingSlope_block</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>DSMMicroMacroSwellingSlope_pellet</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>MassExchangeCoefficient_block</name><type>Constant</type><value>1e-13</value></parameter>
+ <parameter><name>MassExchangeCoefficient_pellet</name><type>Constant</type><value>1e-13</value></parameter>
+ <parameter><name>MacroViscosity_block</name><type>Constant</type><value>1000.0</value></parameter>
+ <parameter><name>MacroViscosity_pellet</name><type>Constant</type><value>1000.0</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMicro_block</name><type>Constant</type><value>1300.0</value></parameter>
+ <parameter><name>ReferenceLiquidDensityMicro_pellet</name><type>Constant</type><value>1300.0</value></parameter>
+ <parameter><name>MicroLiquidDensityA_block</name><type>Constant</type><value>1.3</value></parameter>
+ <parameter><name>MicroLiquidDensityA_pellet</name><type>Constant</type><value>1.3</value></parameter>
+ <parameter><name>MicroLiquidDensityB_block</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>MicroLiquidDensityB_pellet</name><type>Constant</type><value>1.0</value></parameter>
+ <parameter><name>SpecificSurface_block</name><type>Constant</type><value>0.8</value></parameter>
+ <parameter><name>SpecificSurface_pellet</name><type>Constant</type><value>1.5</value></parameter>
+ <parameter><name>InitialVolumeRatio_block</name><type>Constant</type><value>1.5384615384615385</value></parameter>
+ <parameter><name>InitialVolumeRatio_pellet</name><type>Constant</type><value>3.0769230769230771</value></parameter>
+ <parameter><name>n_l0_block</name><type>Constant</type><value>0.08</value></parameter>
+ <parameter><name>n_l0_pellet</name><type>Constant</type><value>0.14</value></parameter>
+ <parameter><name>rho_LR0_block</name><type>Constant</type><value>2142.272975249453</value></parameter>
+ <parameter><name>rho_LR0_pellet</name><type>Constant</type><value>1874.7557787315332</value></parameter>
+ <parameter><name>epsilon_sw0_block</name><type>Constant</type><value>0.0</value></parameter>
+ <parameter><name>epsilon_sw0_pellet</name><type>Constant</type><value>0.0</value></parameter>
+ </parameters>
+ <process_variables>
+ <process_variable><name>displacement</name><components>2</components><order>1</order><initial_condition>displacement0</initial_conditions><boundary_conditions>
+ <boundary_condition><name>geometrical_set-beacon_1c_geometry</name><geometrical_set>geometry</set><geometry>Left</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+ <boundary_condition><name>geometrical_set-beacon_1c_geometry</name><geometrical_set>geometry</set><geometry>right</geometry><type>Dirichlet</type><component>0</component><parameter>dirichlet0</parameter></boundary_condition>
+ <boundary_condition><name>geometrical_set-beacon_1c_geometry</name><geometrical_set>geometry</set><geometry>bottom</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+ <boundary_condition><name>geometrical_set-beacon_1c_geometry</name><geometrical_set>geometry</set><geometry>top</geometry><type>Dirichlet</type><component>1</component><parameter>dirichlet0</parameter></boundary_condition>
+ </boundary_conditions></process_variable>
+ <process_variable><name>pressure</name><components>1</components><order>1</order><initial_condition>pressure_ic</initial_conditions><boundary_conditions>
+ <boundary_condition><name>geometrical_set-beacon_1c_geometry</name><geometrical_set>geometry</set><geometry>top</geometry><type>Dirichlet</type><component>0</component><parameter>top_pressure</parameter></boundary_condition>
+ <boundary_condition><name>geometrical_set-beacon_1c_geometry</name><geometrical_set>geometry</set><geometry>bottom</geometry><type>Dirichlet</type><component>0</component><parameter>bottom_pressure</parameter></boundary_condition>
+ </boundary_conditions></process_variable>
+ </process_variables>
+ <nonlinear_solvers><name>nonlinear_solver</name><type>Newton</type><max_iter>30</max_iter><linear_solver>general_linear_solver</linear_solver></nonlinear_solvers>
+ <linear_solvers><name>general_linear_solver</name><eigen><solver_type>SparseLU</solver_type><scaling>true</scaling></eigen></linear_solver></linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu b/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_mcc_bridge_referen
ce_t_1000.000000.vtu
new file mode 100644
index 0000000000..548c350b1d
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/beacon_1c_dsm_micromacro_mcc_bridge_reference_t_1000.000000.vtu
@@ -0,0 +1,50 @@
<?xml version="1.0"?>
<VTKFile type="UnstructuredGrid" version="1.0" byte_order="LittleEndian" header_type="UInt64" compressor="vtkLibDataCompressor">
+ <UnstructuredGrid>
+ <FieldData>
+ <DataArray type="Int8" Name="OGS_VERSION" NumberOfTuples="25" format="appended" RangeMin="45" RangeMax="121" offset="0" />
+ </FieldData>
+ <Piece NumberOfPoints="6" NumberOfCells="2" >
+ <PointData>
+ <DataArray type="Float64" Name="EquivalentPlasticStrain" format="appended" RangeMin="0" RangeMax="0.000611866313" offset="88" />
+ <DataArray type="Float64" Name="PlasticVolumetricStrain" format="appended" RangeMin="0" RangeMax="0.001058662982" offset="172" />
+ <DataArray type="Float64" Name="PreConsolidationPressure" format="appended" RangeMin="955782819.5" RangeMax="1000000000" offset="252" />
+ <DataArray type="Float64" Name="VolumeRatio" format="appended" RangeMin="1.5379189665" RangeMax="3.0779453586" offset="356" />
+ <DataArray type="Float64" Name="delta_epsilon_sw" format="appended" RangeMin="0" RangeMax="0" offset="456" />
+ <DataArray type="Float64" Name="displacement" NumberOfComponents="2" format="appended" RangeMin="0" RangeMax="0" offset="516" />
+ </PointData>
+ <DataArray type="Float64" Name="epsilon_sw" format="appended" RangeMin="0" RangeMax="0" offset="596" />
+ <DataArray type="Float64" Name="mu_LR" format="appended" RangeMin="4.1586330292e-20" RangeMax="2.7174344856e-19" offset="656" />
+ <DataArray type="Float64" Name="n_L" format="appended" RangeMin="0.010840496311" RangeMax="0.075472013144" offset="756" />
+ <DataArray type="Float64" Name="n_S" format="appended" RangeMin="0.32489225855" RangeMax="0.65022977221" offset="856" />
+ <DataArray type="Float64" Name="n_L" format="appended" RangeMin="0.0799679525" RangeMax="0.14006380565" offset="956" />
+ <DataArray type="Float64" Name="omega_L" format="appended" RangeMin="0.099425789202" RangeMax="0.30495238586" offset="1056" />
+ <DataArray type="Float64" Name="phi" format="appended" RangeMin="0.34977022779" RangeMax="0.67510774145" offset="1156" />
+ <DataArray type="Float64" Name="phi_L" format="appended" RangeMin="0.26980227529" RangeMax="0.5350439358" offset="1256" />
+ <DataArray type="Float64" Name="phi_m" format="appended" RangeMin="0.0799679525" RangeMax="0.14006380565" offset="1356" />
+ <DataArray type="Float64" Name="porosity" format="appended" RangeMin="0.34977022779" RangeMax="0.67510774145" offset="1456" />
+ <DataArray type="Float64" Name="pressure" format="appended" RangeMin="-225769.64055" RangeMax="10000" offset="1556" />
+ <DataArray type="Float64" Name="rho_LR" format="appended" RangeMin="1874.5254897" RangeMax="2142.3764964" offset="1640" />
+ <DataArray type="Float64" Name="rho_LR" format="appended" RangeMin="1874.5254897" RangeMax="2142.3764964" offset="1740" />
+ <DataArray type="Float64" Name="rho_L_hat" format="appended" RangeMin="-2.2576964055e-11" RangeMax="1e-12" offset="1840" />
+ <DataArray type="Float64" Name="saturation" format="appended" RangeMin="0.95148277132" RangeMax="1.0164409813" offset="1940" />
+ <DataArray type="Float64" Name="sigma" NumberOfComponents="4" format="appended" RangeMin="20785.049435" RangeMax="36731.517872" offset="2040" />
+ <DataArray type="Float64" Name="sigma_S" NumberOfComponents="4" format="appended" RangeMin="0" RangeMax="0" offset="2240" />
+ <DataArray type="Float64" Name="swelling_stress" NumberOfComponents="4" format="appended" RangeMin="0" RangeMax="0" offset="2300" />
+ <DataArray type="Float64" Name="transport_porosity" format="appended" RangeMin="0.24977022779" RangeMax="0.55010774145" offset="2360" />
+ </PointData>
+ <CellData>
+ </CellData>
+ <Points>
+ <DataArray type="Float64" Name="Points" NumberOfComponents="3" format="appended" RangeMin="0" RangeMax="0.11180339887" offset="2460" />
+ </Points>
+ <Cells>
+ <DataArray type="Int64" Name="connectivity" format="appended" RangeMin="" RangeMax="" offset="2560" />
+ <DataArray type="Int64" Name="offsets" format="appended" RangeMin="" RangeMax="" offset="2640" />
+ <DataArray type="UInt8" Name="types" format="appended" RangeMin="" RangeMax="" offset="2704" />

```



```

+def absolutize_mesh_and_geometry(root: ET.Element, source_dir: Path) -> None:
+    mesh = root.find("./mesh")
+    geometry = root.find("./geometry")
+    if mesh is not None and mesh.text:
+        mesh.text = str((source_dir / mesh.text.strip()).resolve())
+    if geometry is not None and geometry.text:
+        geometry.text = str((source_dir / geometry.text.strip()).resolve())
+
+
+def set_output_prefix(root: ET.Element, prefix: str) -> None:
+    node = root.find("./time_loop/output/prefix")
+    if node is None:
+        raise RuntimeError("Could not find output prefix.")
+    node.text = prefix
+
+
+def set_solver_controls(root: ET.Element) -> None:
+    for node in root.findall("./nonlinear_solvers/nonlinear_solver/max_iter"):
+        node.text = "60"
+    for node in root.findall("./convergence_criterion/abstols"):
+        if node.text is None:
+            continue
+        parts = node.text.strip().split()
+        if not parts:
+            continue
+        parts[0] = "5e-7"
+        node.text = " ".join(parts)
+
+
+def build_project_copy(
+    *,
+    source: Path,
+    target: Path,
+    prefix: str,
+    implementation: str,
+    multiplier: float,
+    apply_dd_target: bool,
+    apply_micro_activation: bool,
+) -> None:
+    root = ET.parse(source).getroot()
+    absolutize_mesh_and_geometry(root, source.parent)
+    set_output_prefix(root, prefix)
+    set_solver_controls(root)
+
+    if apply_dd_target:
+        phi0 = 1.0 - TARGET_DD_KG_M3 / RHO_SOLID_KG_M3
+        set_parameter_value(root, "phi0", phi0)
+        # Keep transport porosity below total porosity.
+        set_parameter_value(root, "phi_tr0", max(0.0, phi0 - 0.1))
+
+    hamaker_j = HAMAKER_REFERENCE_J * multiplier
+    if implementation == "native":
+        set_native_hamaker(root, hamaker_j)
+    elif implementation == "mfront":
+        set_mfront_hamaker(root, hamaker_j)
+        if apply_micro_activation:
+            # Switch on the same micro branch used in dense dd calibration.
+            set_parameter_value(root, "MicroSwellingStrainSlope", 0.1)
+            set_parameter_value(root, "MacroSaturationLawMode", 1.0)
+            set_parameter_value(root, "MicroStateLocalSolveMode", 0.0)
+            set_parameter_value(root, "MicroPotentialConvention", 1.0)
+            set_parameter_value(root, "SpecificSurface", 523.0)
+            set_parameter_value(root, "n_l0", 0.00153385355)
+            set_parameter_value(root, "rho_lR0", 2276.031917690513)
+    else:
+        raise ValueError(implementation)
+
+    target.parent.mkdir(parents=True, exist_ok=True)
+    ET.ElementTree(root).write(target, encoding="ISO-8859-1", xml_declaration=True)
+
+
+def run_ogs(ogs_bin: Path, project: Path, output_dir: Path) -> None:
+    output_dir.mkdir(parents=True, exist_ok=True)
+    result = subprocess.run(
+        [str(ogs_bin), "-o", str(output_dir), str(project)],
+        cwd=ROOT,
+        text=True,
+        capture_output=True,
+        check=False,
+    )
+    if result.returncode != 0:
+        raise RuntimeError(
+            f"ogs failed for {project.name}\n"
+            f"stdout:\n{result.stdout}\n"
+            f"stderr:\n{result.stderr}"
+        )
+
+
+def parse_output_time(path: Path) -> float:
+    marker = "_t_"
+    if marker not in path.stem:
+        return -math.inf
+    return float(path.stem.split(marker, 1)[1])
+
+
+def latest_vtu(output_dir: Path, prefix: str) -> Path:
+    files = sorted(output_dir.glob(f"{prefix}_t_*.vtu"), key=parse_output_time)
+    if not files:
+        raise FileNotFoundError(f"No VTU files found for prefix {prefix}")
+    return files[-1]
+
+
+def load_vtu(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    grid = reader.GetOutput()
+    points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+    arrays: dict[str, np.ndarray] = {}
+    for i in range(grid.GetPointData().GetNumberOfArrays()):
+        arr = grid.GetPointData().GetArray(i)
+        name = grid.GetPointData().GetArrayName(i)
+        if arr is None or name is None:
+            continue
+        arrays[name] = vtk_to_numpy(arr)
+    return points, arrays

```



```

+
+def boundary_mean(points: np.ndarray, values: np.ndarray, axis: int, side: str) -> np.ndarray:
+    target = points[:, axis].max() if side == "max" else points[:, axis].min()
+    mask = np.isclose(points[:, axis], target, atol=1e-10)
+    return np.asarray(values[mask]).mean(axis=0)
+
+
+def evaluate_metrics(vtu_path: Path) -> tuple[float, float, float, float, float]:
+    points, arrays = load_vtu(vtu_path)
+    sigma = arrays["sigma"]
+    swelling = arrays.get("swelling_stress")
+    dry_density = arrays.get("dry_density_solid")
+
+    sigma_top = boundary_mean(points, sigma, axis=1, side="max")
+    sigma_side = boundary_mean(points, sigma, axis=0, side="max")
+
+    if swelling is None:
+        sw_top = np.array([math.nan, math.nan])
+        sw_side = np.array([math.nan, math.nan])
+    else:
+        sw_top = boundary_mean(points, swelling, axis=1, side="max")
+        sw_side = boundary_mean(points, swelling, axis=0, side="max")
+
+    dd_mean = float(np.asarray(dry_density).mean()) if dry_density is not None else math.nan
+    return (
+        abs(float(sigma_top[1])) / 1e3,
+        abs(float(sigma_side[0])) / 1e3,
+        abs(float(sw_top[1])) / 1e3,
+        abs(float(sw_side[0])) / 1e3,
+        dd_mean,
+    )
+
+
+def run_case(
+    *,
+    case_id: str,
+    implementation: str,
+    source_project: Path,
+    ogs_bin: Path,
+    multiplier: float,
+    apply_dd_target: bool,
+    apply_micro_activation: bool,
+    note: str,
+    tmpdir: Path,
+) -> CaseResult:
+    prefix = f"{case_id}_{implementation}"
+    project_copy = tmpdir / f"{prefix}.prj"
+    out_dir = tmpdir / f"out_{prefix}"
+    hamaker_j = HAMAKER_REFERENCE_J * multiplier
+
+    build_project_copy(
+        source=source_project,
+        target=project_copy,
+        prefix=prefix,
+        implementation=implementation,
+        multiplier=multiplier,
+        apply_dd_target=apply_dd_target,
+        apply_micro_activation=apply_micro_activation,
+    )
+
+    try:
+        run_ogs(ogs_bin, project_copy, out_dir)
+        ax, rad, ax_sw, rad_sw, dd = evaluate_metrics(latest_vtu(out_dir, prefix))
+        return CaseResult(
+            case_id=case_id,
+            implementation=implementation,
+            run_status="success",
+            dry_density_target_kg_m3=TARGET_DD_KG_M3 if apply_dd_target else math.nan,
+            multiplier_used=multiplier,
+            hamaker_constant_j=hamaker_j,
+            axial_sigma_kpa=ax,
+            radial_sigma_kpa=rad,
+            axial_swelling_stress_kpa=ax_sw,
+            radial_swelling_stress_kpa=rad_sw,
+            dry_density_mean_kg_m3=dd,
+            note=note,
+        )
+    except Exception as exc:
+        return CaseResult(
+            case_id=case_id,
+            implementation=implementation,
+            run_status="failed",
+            dry_density_target_kg_m3=TARGET_DD_KG_M3 if apply_dd_target else math.nan,
+            multiplier_used=multiplier,
+            hamaker_constant_j=hamaker_j,
+            axial_sigma_kpa=math.nan,
+            radial_sigma_kpa=math.nan,
+            axial_swelling_stress_kpa=math.nan,
+            radial_swelling_stress_kpa=math.nan,
+            dry_density_mean_kg_m3=math.nan,
+            note=f"{note}; error={str(exc).splitlines()[-1][:220]}",
+        )
+
+
+def write_rows_csv(rows: list[CaseResult], path: Path) -> None:
+    path.parent.mkdir(parents=True, exist_ok=True)
+    keys = list(CaseResult.__dataclass_fields__.keys())
+    with path.open("w", newline="", encoding="utf-8") as f:
+        writer = csv.DictWriter(f, fieldnames=keys)
+        writer.writeheader()
+        for row in rows:
+            writer.writerow(row.__dict__)
+
+
+def main() -> None:
+    dd_curve, mult_curve = load_multiplier_curve(ANCHORS_CALIB_CSV)
+    interpolated_multiplier = interpolate_multiplier(TARGET_DD_KG_M3, dd_curve, mult_curve)
+
+    tmpdir = Path(tempfile.mkdtemp(prefix="beacon_1b_ddcurve_probe_"))
+    rows: list[CaseResult] = []
+    try:
+        rows.append(
+            run_case(
+                case_id="baseline_1b",
+                implementation="native",
+                source_project=NATIVE_PROJECT,
+                ogs_bin=NATIVE_OGS,

```

```

+         multiplier=1.0,
+         apply_dd_target=False,
+         apply_micro_activation=False,
+         note="Committed native 1b deck (qualitative report comparison).",
+         tmpdir=tmpdir,
+     )
+ )
+ rows.append(
+     run_case(
+         case_id="baseline_1b",
+         implementation="mfront",
+         source_project=MFRONT_PROJECT,
+         ogs_bin=MFRONT_OGS,
+         multiplier=1.0,
+         apply_dd_target=False,
+         apply_micro_activation=False,
+         note="Committed Mfront 1b deck (qualitative report comparison).",
+         tmpdir=tmpdir,
+     )
+ )
+ rows.append(
+     run_case(
+         case_id="dd_target_probe",
+         implementation="native",
+         source_project=MFRONT_PROJECT,
+         ogs_bin=NATIVE_OGS,
+         multiplier=interpolated_multiplier,
+         apply_dd_target=True,
+         apply_micro_activation=False,
+         note=(
+             "Native 1b with DD target + interpolated multiplier from dense Villar curve; "
+             "native 1b swelling remains on saturation-dependent law."
+         ),
+         tmpdir=tmpdir,
+     )
+ )
+ rows.append(
+     run_case(
+         case_id="dd_target_probe",
+         implementation="mfront",
+         source_project=MFRONT_PROJECT,
+         ogs_bin=MFRONT_OGS,
+         multiplier=interpolated_multiplier,
+         apply_dd_target=True,
+         apply_micro_activation=True,
+         note=(
+             "Mfront 1b with DD target + micro activation + interpolated multiplier "
+             "from dense Villar curve."
+         ),
+         tmpdir=tmpdir,
+     )
+ )
+ finally:
+     shutil.rmtree(tmpdir, ignore_errors=True)
+
+ out_csv = OUT_ROOT / "beacon_1b_ddcurve_probe_rows.csv"
+ out_json = OUT_ROOT / "beacon_1b_ddcurve_probe_summary.json"
+ write_rows_csv(rows, out_csv)
+
+ summary = {
+     "target_dry_density_kg_m3": TARGET_DD_KG_M3,
+     "rho_solid_kg_m3": RHO_SOLID_KG_M3,
+     "interpolated_multiplier_from_dense_villar_curve": interpolated_multiplier,
+     "reference_curve_csv": str(ANCHORS_CALIB_CSV),
+     "rows": [r._dict_ for r in rows],
+     "notes": [
+         "BEACON DS.1.1 provides a qualitative 1b stress target ('nonzero and stabilized after about 500 days') but no tabulated plateau value.",
+         "The DD-target probe therefore evaluates sensitivity to DD-corrected setup plus multiplier transfer from the dense Villar calibration.",
+         "Native and Mfront 1b decks are not equivalent in swelling-source formulation by default.",
+     ],
+ }
+ out_json.parent.mkdir(parents=True, exist_ok=True)
+ out_json.write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")
+
+ print(f"Wrote: {out_csv}")
+ print(f"Wrote: {out_json}")
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/generate_beacon_unstructured_meshes.py b/Tests/Data/RichardsMechanics/generate_beacon_unstructured_meshes.py
new file mode 100644
index 0000000000..9ba2b24a78
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/generate_beacon_unstructured_meshes.py
@@ -0,0 +1,81 @@
#!/usr/bin/env python3
"""Generate the small unstructured BEACON meshes used by the report runs.

+The script rescales the source GML footprint into the 1a01 and 1b benchmark
+domains and writes both the VTU meshes and their matching geometry files.
+"""
+
+from pathlib import Path
+import xml.etree.ElementTree as ET
+
+import numpy as np
+import vtk
+
+BASE_DIR = Path(__file__).resolve().parent
+
+def _read_rectangle(gml_path: Path) -> tuple[float, float]:
+    """Extract rectangle width and height from a simple GML geometry."""
+    root = ET.parse(gml_path).getroot()
+    xs = [float(point.attrib["x"]) for point in root.findall("./point")]
+    ys = [float(point.attrib["y"]) for point in root.findall("./point")]
+    return max(xs) - min(xs), max(ys) - min(ys)
+
+def _build_mesh(width: float, height: float, *, nx: int, ny: int, seed: int) -> vtk.vtkUnstructuredGrid:
+    """Create a lightly perturbed triangulation of a rectangular domain."""
+    rng = np.random.default_rng(seed)
+    points = vtk.vtkPoints()
+
+    for j in range(ny):

```

```

+     y = height * j / (ny - 1)
+     for i in range(nx):
+         x = width * i / (nx - 1)
+         if 0 < i < nx - 1 and 0 < j < ny - 1:
+             x += rng.uniform(-0.18, 0.18) * width / (nx - 1)
+             y += rng.uniform(-0.18, 0.18) * height / (ny - 1)
+             points.InsertNextPoint(float(x), float(y), 0.0)
+
+     poly = vtk.vtkPolyData()
+     poly.SetPoints(points)
+
+     delaunay = vtk.vtkDelaunay2D()
+     delaunay.SetInputData(poly)
+     delaunay.SetTolerance(1e-8)
+     delaunay.Update()
+
+     triangulation = delaunay.GetOutput()
+     append = vtk.vtkAppendFilter()
+     append.AddInputData(triangulation)
+     append.Update()
+     return append.GetOutput()
+
+
+def _write_mesh(mesh: vtk.vtkUnstructuredGrid, path: Path) -> None:
+    """Persist a VTU mesh in ASCII form for reproducible diffs."""
+    writer = vtk.vtkXMLUnstructuredGridWriter()
+    writer.SetFileName(str(path))
+    writer.SetInputData(mesh)
+    writer.SetDataModeToAscii()
+    if writer.Write() != 1:
+        raise RuntimeError(f"failed to write {path}")
+
+
+def main() -> None:
+    """Generate all benchmark meshes and report their sizes."""
+    cases = [
+        ("beacon_1a01.gml", "beacon_1a01_domain_unstructured_162e.vtu", 10, 10, 101),
+        ("beacon_1b.gml", "beacon_1b_domain_unstructured_162e.vtu", 10, 10, 202),
+    ]
+
+    for gml_name, mesh_name, nx, ny, seed in cases:
+        width, height = _read_rectangle(BASE_DIR / gml_name)
+        mesh = _build_mesh(width, height, nx=nx, ny=ny, seed=seed)
+        out = BASE_DIR / mesh_name
+        _write_mesh(mesh, out)
+        print(f"{mesh_name}: {mesh.GetNumberOfPoints()} points, {mesh.GetNumberOfCells()} cells")
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_bridge.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_bridge.prj
new file mode 100644
index 0000000000..0d33bb7b4f
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_bridge.prj
@@ -0,0 +1,368 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+   <meshes>
+       <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+       <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+       <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+       <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+       <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+   </meshes>
+   <processes>
+       <process>
+           <name>RM</name>
+           <type>RICHARDS_MECHANICS</type>
+           <integration_order>2</integration_order>
+           <jacobian_assembler>
+               <type>Analytical</type>
+           </jacobian_assembler>
+           <constitutive_relation>
+               <type>MFrontRichardsMechanics</type>
+               <behaviour>RichardsMechanicsDSMMicroMacroBridge</behaviour>
+               <material_properties>
+                   <material_property name="YoungModulus" parameter="YoungModulus" />
+                   <material_property name="PoissonRatio" parameter="PoissonRatio" />
+                   <material_property name="SwellingSlope" parameter="SwellingSlope" />
+                   <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient" />
+                   <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro" />
+                   <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro" />
+                   <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid" />
+                   <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA" />
+                   <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB" />
+                   <material_property name="HamakerConstant" parameter="HamakerConstant" />
+                   <material_property name="SpecificSurface" parameter="SpecificSurface" />
+                   <material_property name="InitialPorosity" parameter="phi0" />
+                   <material_property name="AreaFactorTuller" parameter="AreaFactorTuller" />
+                   <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller" />
+                   <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize" />
+                   <material_property name="SurfaceTension" parameter="SurfaceTension" />
+               </material_properties>
+               <initial_values>
+                   <state_variable name="n_l" parameter="n_l0" />
+                   <state_variable name="rho_lR" parameter="rho_lR0" />
+                   <state_variable name="epsilon_sw" parameter="epsilon_sw0" />
+               </initial_values>
+           </constitutive_relation>
+       </process>
+       <process_variables>
+           <pressure>pressure</pressure>
+           <displacement>displacement</displacement>
+       </process_variables>
+       <secondary_variables>
+           <secondary_variable name="sigma" />
+           <secondary_variable name="swelling_stress" />
+           <secondary_variable name="epsilon" />
+           <secondary_variable name="velocity" />
+           <secondary_variable name="saturation" />
+           <secondary_variable name="transport_porosity" />
+           <secondary_variable name="dry_density_solid" />
+       </secondary_variables>
+       <specific_body_force>0</specific_body_force>
+       <initial_stress>sigma0</initial_stress>
+       <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+       <mass_lumping>true</mass_lumping>

```

```

+
+ </process>
+ </processes>
+ <media>
+ <medium>
+ <phases>
+ <phase>
+ <type>AqueousLiquid</type>
+ <properties>
+ <property>
+ <name>viscosity</name>
+ <type>Constant</type>
+ <value>1e-3</value>
+ </property>
+ <property>
+ <name>density</name>
+ <type>Constant</type>
+ <value>1e3</value>
+ </property>
+ </properties>
+ </phase>
+ <phase>
+ <type>Solid</type>
+ <properties>
+ <property>
+ <name>density</name>
+ <type>Constant</type>
+ <value>2780.0</value>
+ </property>
+ </properties>
+ </phase>
+ </phases>
+ <properties>
+ <property>
+ <name>permeability</name>
+ <type>Constant</type>
+ <value>2e-21</value>
+ </property>
+ <property>
+ <name>transport_porosity</name>
+ <type>TransportPorosityFromMassBalance</type>
+ <initial_porosity>phi_tr0</initial_porosity>
+ <minimal_porosity>0</minimal_porosity>
+ <maximal_porosity>1</maximal_porosity>
+ </property>
+ <property>
+ <name>porosity</name>
+ <type>PorosityFromMassBalance</type>
+ <initial_porosity>phi0</initial_porosity>
+ <minimal_porosity>0</minimal_porosity>
+ <maximal_porosity>1</maximal_porosity>
+ </property>
+ <property>
+ <name>reference_temperature</name>
+ <type>Constant</type>
+ <value>293.15</value>
+ </property>
+ <property>
+ <name>relative_permeability</name>
+ <type>Constant</type>
+ <value>1</value>
+ </property>
+ <property>
+ <name>saturation</name>
+ <type>SaturationVanGenuchten</type>
+ <residual_liquid_saturation>0</residual_liquid_saturation>
+ <residual_gas_saturation>0</residual_gas_saturation>
+ <exponent>0.4</exponent>
+ <p_b>15e6</p_b>
+ </property>
+ <property>
+ <name>bishops_effective_stress</name>
+ <type>BishopsSaturationCutoff</type>
+ <cutoff_value>1</cutoff_value>
+ </property>
+ <property>
+ <name>biot_coefficient</name>
+ <type>Constant</type>
+ <value>0.6</value>
+ </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+ <processes>
+ <process ref="RM">
+ <nonlinear_solver>basic_newton</nonlinear_solver>
+ <convergence_criterion>
+ <type>PerComponentDeltaX</type>
+ <norm_type>NORM2</norm_type>
+ <reltols>1e-10 1e0 1e-9</reltols>
+ </convergence_criterion>
+ <time_discretization>
+ <type>BackwardEuler</type>
+ </time_discretization>
+ <time_stepping>
+ <type>FixedTimeStepping</type><t_initial>0</t_initial><t_end>4</t_end><timesteps><pair><repeat>4</repeat><delta_t>1</delta_t></pair></timesteps></time_stepping>
+
+ </process>
+ </processes>
+ <output>
+ <type>VTK</type>
+ <prefix>wfront_parity_1element_bridge</prefix>
+ <suffix>_ts_{: timestep}_t_{: time}</suffix>
+ <timesteps>
+ <pair><repeat>10</repeat><each_steps>1</each_steps></pair></timesteps>
+ <variables>
+ <variable>displacement</variable><variable>pressure</variable><variable>sigma</variable><variable>epsilon</variable><variable>saturation</variable><variable>swelling_s
+ </variable></variables>
+ </output>
+ </time_loop>
+ <parameters>
+ <parameter>
+ <name>sigma0</name>
+ <type>Function</type>
+ <expression>-5e3</expression>
+ <expression>-5e3</expression>

```

```

+      <expression>=5e3</expression>
+      <expression>0</expression>
+    </parameter>
+    <parameter>
+      <name>YoungModulus</name>
+      <type>Constant</type>
+      <value>52e6</value>
+    </parameter>
+    <parameter>
+      <name>PoissonRatio</name>
+      <type>Constant</type>
+      <value>0.3</value>
+    </parameter>
+    <parameter>
+      <name>CriticalStateLineSlope</name>
+      <type>Constant</type>
+      <value>1.2</value>
+    </parameter>
+    <parameter>
+      <name>SwellingLineSlope</name>
+      <type>Constant</type>
+      <value>6.6e-3</value>
+    </parameter>
+    <parameter>
+      <name>VirginConsolidationLineSlope</name>
+      <type>Constant</type>
+      <value>7.7e-2</value>
+    </parameter>
+    <parameter>
+      <name>InitialPreConsolidationPressure</name>
+      <type>Constant</type>
+      <value>2e5</value>
+    </parameter>
+    <parameter>
+      <name>InitialVolumeRatio</name>
+      <type>Constant</type>
+      <value>1.78571428571428571429</value>
+    </parameter>
+    <parameter>
+      <name>SaturationPressureScale</name>
+      <type>Constant</type>
+      <value>1e3</value>
+    </parameter>
+    <parameter>
+      <name>SwellingSlope</name>
+      <type>Constant</type>
+      <value>0.1</value>
+    </parameter>
+    <parameter>
+      <name>MassExchangeCoefficient</name>
+      <type>Constant</type>
+      <value>5e-15</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMacro</name>
+      <type>Constant</type>
+      <value>1000.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMicro</name>
+      <type>Constant</type>
+      <value>1300.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceDensitySolid</name>
+      <type>Constant</type>
+      <value>2470.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityA</name>
+      <type>Constant</type>
+      <value>1.3</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityB</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>HamakerConstant</name>
+      <type>Constant</type>
+      <value>-6e-20</value>
+    </parameter>
+    <parameter>
+      <name>SpecificSurface</name>
+      <type>Constant</type>
+      <value>100.0</value>
+    </parameter>
+    <parameter>
+      <name>AreaFactorTuller</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>PoreAreaShapeFactorTuller</name>
+      <type>Constant</type>
+      <value>0.8584073464102069</value>
+    </parameter>
+    <parameter>
+      <name>CharacteristicPoreSize</name>
+      <type>Constant</type>
+      <value>1e-5</value>
+    </parameter>
+    <parameter>
+      <name>SurfaceTension</name>
+      <type>Constant</type>
+      <value>0.0715</value>
+    </parameter>
+    <parameter>
+      <name>n_l0</name>
+      <type>Constant</type>
+      <value>0.1</value>
+    </parameter>
+    <parameter>
+      <name>rho_LR0</name>
+      <type>Constant</type>
+      <value>2072.8234319102588</value>

```

```

+
+ </parameter>
+
+ <parameter>
+
+ <name>epsilon_sw0</name>
+
+ <type>Constant</type>
+
+ <value>0.0</value>
+
+ </parameter>
+
+
+ <parameter>
+
+ <name>displacement0</name>
+
+ <type>Constant</type>
+
+ <values>0 0</values>
+
+ </parameter>
+
+ <parameter>
+
+ <name>zero</name>
+
+ <type>Constant</type>
+
+ <value>0.0</value>
+
+ </parameter>
+
+ <parameter>
+
+ <name>phi0</name>
+
+ <type>Constant</type>
+
+ <value>0.432</value>
+
+ </parameter>
+
+ <parameter>
+
+ <name>phi_tr0</name>
+
+ <type>Constant</type>
+
+ <value>0.332</value>
+
+ </parameter>
+
+ <parameter>
+
+ <name>top_pressure</name>
+
+ <type>Constant</type>
+
+ <value>-1e5</value>
+
+ </parameter>
+
+ <parameter>
+
+ <name>pressure_ic</name>
+
+ <type>Constant</type>
+
+ <value>0</value>
+
+ </parameter>
+
+ <parameter><name>pressure_bc_scale</name><type>Constant</type><value>1000</value></parameter><parameter><name>pressure_bc</name><type>CurveScaled</type><curve>pressure_ramp_parity</curve><parameter>pressure_bc_scale</parameter></parameter></parameters>
+
+ <process_variables>
+
+ <process_variable>
+
+ <name>displacement</name>
+
+ <components>2</components>
+
+ <order>1</order>
+
+ <initial_condition>displacement0</initial_condition>
+
+ <boundary_conditions>
+
+
+ <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition></boundary_conditions>
+
+ </process_variable>
+
+ <process_variable>
+
+ <name>pressure</name>
+
+ <components>1</components>
+
+ <order>1</order>
+
+ <initial_condition>pressure_ic</initial_condition>
+
+ <boundary_conditions>
+
+ <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition></boundary_conditions>
+
+ </process_variable>
+
+ </process_variables>
+
+ <nonlinear_solvers>
+
+ <nonlinear_solver>
+
+ <name>basic_newton</name>
+
+ <type>Newton</type>
+
+ <max_iter>60</max_iter>
+
+ <linear_solver>general_linear_solver</linear_solver>
+
+ </nonlinear_solver>
+
+ </nonlinear_solvers>
+
+ <linear_solvers>
+
+ <linear_solver>
+
+ <name>general_linear_solver</name>
+
+ <eigen>
+
+ <solver_type>SparseLU</solver_type>
+
+ <scaling>true</scaling>
+
+ </eigen>
+
+ </linear_solver>
+
+ </linear_solvers>
+
+ <curves><curve><name>pressure_ramp_parity</name><coords>0 1 2 3 4</coords><values>0 0.25 0.5 0.75 1</values></curve></curves></OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_bridge.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_bridge.prj
new file mode 100644
index 0000000000..d4d1effbfa
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_bridge.prj
@@ -0,0 +1,474 @@
+<?xml version='1.0' encoding='ISO-8859-1'?>
+<OpenGeoSysProject>
+
+ <meshes>
+
+ <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+
+ </meshes>
+
+ <processes>
+
+ <process>
+
+ <name>RM</name>
+
+ <type>RICHARDS_MECHANICS</type>
+
+ <integration_order>2</integration_order>
+
+ <jacobian_assembler>
+
+ <type>Analytical</type>
+
+ </jacobian_assembler>
+
+ <constitutive_relation>
+
+ <type>MFrontRichardsMechanics</type>
+
+ <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+
+ <material_properties>
+
+ <material_property name="YoungModulus" parameter="YoungModulus"/>
+
+ <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+
+ <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+
+ <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>

```

```

+       <material_property name="BubblePressure" parameter="BubblePressure"/>
+       <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+       <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode"/>
+       <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode"/>
+       <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention"/>
+       <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope"/>
+       <material_property name="MassExchangeCoefficient" parameter="MicroMacroPotentialExchangeCoefficient"/>
+       <material_property name="MacroViscosity" parameter="MacroPotentialViscosity"/>
+       <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro"/>
+       <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro"/>
+       <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid"/>
+       <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA"/>
+       <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB"/>
+       <material_property name="HamakerConstant" parameter="HamakerConstant"/>
+       <material_property name="SpecificSurface" parameter="SpecificSurface"/>
+       <material_property name="AreaFactorTuller" parameter="AreaFactorTuller"/>
+       <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller"/>
+       <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize"/>
+       <material_property name="SurfaceTension" parameter="SurfaceTension"/>
+       <material_property name="InitialPorosity" parameter="phi0"/>
+
+     </material_properties>
+
+     <initial_values>
+       <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+       <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+       <state_variable name="n_l" parameter="n_l0"/>
+       <state_variable name="rho_LR" parameter="rho_LR0"/>
+       <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+     </initial_values>
+
+     </constitutive_relation>
+
+     <process_variables>
+       <pressure>pressure</pressure>
+       <displacement>displacement</displacement>
+     </process_variables>
+
+     <secondary_variables>
+       <secondary_variable name="ElasticStrain"/>
+       <secondary_variable name="EquivalentPlasticStrain"/>
+       <secondary_variable name="PreConsolidationPressure"/>
+       <secondary_variable name="PlasticVolumetricStrain"/>
+       <secondary_variable name="sigma"/>
+       <secondary_variable name="epsilon"/>
+       <secondary_variable name="velocity"/>
+       <secondary_variable name="saturation"/>
+       <secondary_variable name="VolumeRatio"/>
+       <secondary_variable name="n_l"/>
+       <secondary_variable name="rho_LR"/>
+       <secondary_variable name="epsilon_sw"/>
+       <secondary_variable name="phi_m"/>
+       <secondary_variable name="phi_M"/>
+       <secondary_variable name="mu_LR"/>
+       <secondary_variable name="rho_l_hat"/>
+     </secondary_variables>
+
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+
+   </process>
+
+ </processes>
+
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2700.0</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+
+     <properties>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>1e-12</value>
+       </property>
+
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+
+       <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.01</exponent>
+         <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+       </property>
+
+       <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>

```

```

+         <exponent>0.4</exponent>
+         <p_b>1e4</p_b>
+       </property>
+     <property>
+       <name>bishops_effective_stress</name>
+       <type>BishopsPowerLaw</type>
+       <exponent>1</exponent>
+     </property>
+     <property>
+       <name>biot_coefficient</name>
+       <type>Constant</type>
+       <value>1.0</value>
+     </property>
+   </properties>
+ </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e0 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>4</t_end>
+         <timesteps>
+           <pair>
+             <repeat>4</repeat>
+             <delta_t>1</delta_t>
+           </pair>
+         </timesteps>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>front_parity_1element_dsm_micromacro_mcc_bridge</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+       <pair>
+         <repeat>10</repeat>
+         <each_steps>1</each_steps>
+       </pair>
+     </timesteps>
+     <variables>
+       <variable>displacement</variable>
+       <variable>pressure</variable>
+       <variable>sigma</variable>
+       <variable>epsilon</variable>
+       <variable>saturation</variable>
+       <variable>velocity</variable>
+       <variable>ElasticStrain</variable>
+       <variable>EquivalentPlasticStrain</variable>
+       <variable>PreConsolidationPressure</variable>
+       <variable>PlasticVolumetricStrain</variable>
+       <variable>VolumeRatio</variable>
+       <variable>n_l</variable>
+       <variable>rho_lr</variable>
+       <variable>epsilon_sw</variable>
+       <variable>phi_mc</variable>
+       <variable>phi_Mc</variable>
+       <variable>mu_lr</variable>
+       <variable>rho_l_hat</variable>
+     </variables>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>sigma0</name>
+     <type>Function</type>
+     <expression>-5e3</expression>
+     <expression>-5e3</expression>
+     <expression>-5e3</expression>
+     <expression>0</expression>
+   </parameter>
+   <parameter>
+     <name>YoungModulus</name>
+     <type>Constant</type>
+     <value>52e6</value>
+   </parameter>
+   <parameter>
+     <name>PoissonRatio</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>CriticalStateLineSlope</name>
+     <type>Constant</type>
+     <value>1.2</value>
+   </parameter>
+   <parameter>
+     <name>SwellingLineSlope</name>
+     <type>Constant</type>
+     <value>6.6e-3</value>
+   </parameter>
+   <parameter>
+     <name>VirginConsolidationLineSlope</name>
+     <type>Constant</type>
+     <value>7.7e-2</value>
+   </parameter>
+   <parameter>
+     <name>InitialPreConsolidationPressure</name>
+     <type>Constant</type>
+     <value>2e5</value>
+   </parameter>
+   <parameter>
+     <name>InitialVolumeRatio</name>
+     <type>Constant</type>
+     <value>1.78571428571428571429</value>
+   </parameter>

```



```

+ <parameter>
+   <name>ResidualLiquidSaturation</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>ResidualGasSaturation</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>BubblePressure</name>
+   <type>Constant</type>
+   <value>1e4</value>
+ </parameter>
+ <parameter>
+   <name>VanGenuchtenExponent_m</name>
+   <type>Constant</type>
+   <value>0.4</value>
+ </parameter>
+ <parameter>
+   <name>MacroSaturationLawMode</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>MicroStateLocalSolveMode</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>MicroPotentialConvention</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>MicroSwellingStrainSlope</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>MicroMacroPotentialExchangeCoefficient</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>MacroPotentialViscosity</name>
+   <type>Constant</type>
+   <value>1000.0</value>
+ </parameter>
+ <parameter>
+   <name>ReferenceLiquidDensityMacro</name>
+   <type>Constant</type>
+   <value>1000.0</value>
+ </parameter>
+ <parameter>
+   <name>ReferenceLiquidDensityMicro</name>
+   <type>Constant</type>
+   <value>1300.0</value>
+ </parameter>
+ <parameter>
+   <name>ReferenceDensitySolid</name>
+   <type>Constant</type>
+   <value>2470.0</value>
+ </parameter>
+ <parameter>
+   <name>MicroLiquidDensityA</name>
+   <type>Constant</type>
+   <value>1.3</value>
+ </parameter>
+ <parameter>
+   <name>MicroLiquidDensityB</name>
+   <type>Constant</type>
+   <value>1.0</value>
+ </parameter>
+ <parameter>
+   <name>HamakerConstant</name>
+   <type>Constant</type>
+   <value>-6e-20</value>
+ </parameter>
+ <parameter>
+   <name>SpecificSurface</name>
+   <type>Constant</type>
+   <value>100.0</value>
+ </parameter>
+ <parameter>
+   <name>AreaFactorTuller</name>
+   <type>Constant</type>
+   <value>1.0</value>
+ </parameter>
+ <parameter>
+   <name>PoreAreaShapeFactorTuller</name>
+   <type>Constant</type>
+   <value>0.8584073464102069</value>
+ </parameter>
+ <parameter>
+   <name>CharacteristicPoreSize</name>
+   <type>Constant</type>
+   <value>1e-5</value>
+ </parameter>
+ <parameter>
+   <name>SurfaceTension</name>
+   <type>Constant</type>
+   <value>0.0715</value>
+ </parameter>
+ <parameter>
+   <name>phi0</name>
+   <type>Constant</type>
+   <value>0.432</value>
+ </parameter>
+ <parameter>
+   <name>n_l0</name>
+   <type>Constant</type>
+   <value>0.1</value>
+ </parameter>
+ <parameter>
+   <name>rho_LR0</name>

```

```

+         <type>Constant</type>
+         <value>1300.0</value>
+     </parameter>
+     <parameter>
+         <name>epsilon_sw0</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>zero</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>top_pressure</name>
+         <type>Constant</type>
+         <value>-1e5</value>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>-5e3</value>
+     </parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_left</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_bottom</mesh>
+                 <type>Dirichlet</type>
+                 <component>1</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_top</mesh>
+                 <type>Neumann</type>
+                 <component>1</component>
+                 <parameter>top_pressure</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_right</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+     <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_top</mesh>
+                 <type>Dirichlet</type>
+                 <parameter>pressure_ic</parameter>
+             </boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>60</max_iter>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+             <solver_type>SparseLU</solver_type>
+             <scaling>true</scaling>
+         </eigen>
+     </linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_tuller_bridge.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_tulle
r_bridge.prj
new file mode 100644
index 0000000000..9c353575c1
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_tuller_bridge.prj
@@ -0,0 +1,474 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+     <meshes>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+     </meshes>
+     <processes>
+         <process>
+             <name>RM</name>
+             <type>RICHARDS_MECHANICS</type>
+             <integration_order>2</integration_order>
+             <jacobian_assembler>
+                 <type>Analytical</type>

```

```

+ </jacobian_assembler>
+ <constitutive_relation>
+ <type>MFrontRichardsMechanics</type>
+ <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="YoungModulus"/>
+ <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+ <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+ <material_property name="BubblePressure" parameter="BubblePressure"/>
+ <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+ <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode"/>
+ <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode"/>
+ <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention"/>
+ <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope"/>
+ <material_property name="MassExchangeCoefficient" parameter="MicroMacroPotentialExchangeCoefficient"/>
+ <material_property name="MacroViscosity" parameter="MacroPotentialViscosity"/>
+ <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro"/>
+ <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro"/>
+ <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid"/>
+ <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA"/>
+ <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB"/>
+ <material_property name="HamakerConstant" parameter="HamakerConstant"/>
+ <material_property name="SpecificSurface" parameter="SpecificSurface"/>
+ <material_property name="AreaFactorTuller" parameter="AreaFactorTuller"/>
+ <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller"/>
+ <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize"/>
+ <material_property name="SurfaceTension" parameter="SurfaceTension"/>
+ <material_property name="InitialPorosity" parameter="phi0"/>
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+ <state_variable name="n_l" parameter="n_l0"/>
+ <state_variable name="rho_LR" parameter="rho_LR0"/>
+ <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable name="ElasticStrain"/>
+ <secondary_variable name="EquivalentPlasticStrain"/>
+ <secondary_variable name="PreConsolidationPressure"/>
+ <secondary_variable name="PlasticVolumetricStrain"/>
+ <secondary_variable name="sigma"/>
+ <secondary_variable name="epsilon"/>
+ <secondary_variable name="velocity"/>
+ <secondary_variable name="saturation"/>
+ <secondary_variable name="VolumeRatio"/>
+ <secondary_variable name="n_l"/>
+ <secondary_variable name="rho_LR"/>
+ <secondary_variable name="epsilon_sw"/>
+ <secondary_variable name="phi_m"/>
+ <secondary_variable name="phi_M"/>
+ <secondary_variable name="mu_LR"/>
+ <secondary_variable name="rho_hat"/>
+ </secondary_variables>
+ <specific_body_force>0 0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+ <medium>
+ <phases>
+ <phase>
+ <type>AqueousLiquid</type>
+ <properties>
+ <property>
+ <name>viscosity</name>
+ <type>Constant</type>
+ <value>1e-3</value>
+ </property>
+ <property>
+ <name>density</name>
+ <type>Constant</type>
+ <value>1e3</value>
+ </property>
+ </properties>
+ </phase>
+ <phase>
+ <type>Solid</type>
+ <properties>
+ <property>
+ <name>density</name>
+ <type>Constant</type>
+ <value>2780.0</value>
+ </property>
+ </properties>
+ </phase>
+ </phases>
+ <properties>
+ <property>
+ <name>permeability</name>
+ <type>Constant</type>
+ <value>1e-12</value>
+ </property>
+ <property>
+ <name>porosity</name>
+ <type>PorosityFromMassBalance</type>
+ <initial_porosity>phi0</initial_porosity>
+ <minimal_porosity>0</minimal_porosity>
+ <maximal_porosity>1</maximal_porosity>
+ </property>
+ <property>
+ <name>reference_temperature</name>
+ <type>Constant</type>
+ <value>293.15</value>
+ </property>

```

```

+       <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.01</exponent>
+         <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationTuller</type>
+         <area_factor_tuller>1.0</area_factor_tuller>
+         <pore_area_shape_factor_tuller>0.8584073464102069</pore_area_shape_factor_tuller>
+         <characteristic_pore_size>1e-5</characteristic_pore_size>
+         <surface_tension>0.0715</surface_tension>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+       </property>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e0 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>4</t_end>
+         <timesteps>
+           <pair>
+             <repeat>4</repeat>
+             <delta_t>1</delta_t>
+           </pair>
+         </timesteps>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>#front_parity_1element_dsm_micromacro_mcc_tuller_bridge</prefix>
+     <suffix>_ts_{timestep}_t_{time}</suffix>
+     <timesteps>
+       <pair>
+         <repeat>10</repeat>
+         <each_steps>1</each_steps>
+       </pair>
+     </timesteps>
+     <variables>
+       <variable>displacement</variable>
+       <variable>pressure</variable>
+       <variable>sigma</variable>
+       <variable>epsilon</variable>
+       <variable>saturation</variable>
+       <variable>velocity</variable>
+       <variable>ElasticStrain</variable>
+       <variable>EquivalentPlasticStrain</variable>
+       <variable>PreConsolidationPressure</variable>
+       <variable>PlasticVolumetricStrain</variable>
+       <variable>VolumeRatio</variable>
+       <variable>n_l</variable>
+       <variable>rho_lr</variable>
+       <variable>epsilon_sw</variable>
+       <variable>phi_mc</variable>
+       <variable>phi_Mc</variable>
+       <variable>mu_lr</variable>
+       <variable>rho_t_hat</variable>
+     </variables>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>sigma0</name>
+     <type>Function</type>
+     <expression>5e3</expression>
+     <expression>5e3</expression>
+     <expression>5e3</expression>
+     <expression>0</expression>
+   </parameter>
+   <parameter>
+     <name>YoungModulus</name>
+     <type>Constant</type>
+     <value>52e6</value>
+   </parameter>
+   <parameter>
+     <name>PoissonRatio</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>CriticalStateLineSlope</name>
+     <type>Constant</type>
+     <value>1.2</value>
+   </parameter>
+   <parameter>
+     <name>SwellingLineSlope</name>
+     <type>Constant</type>
+     <value>6.6e-3</value>
+   </parameter>
+   <parameter>
+     <name>VirginConsolidationLineSlope</name>

```

```

+      <type>Constant</type>
+      <value>7.7e-2</value>
+    </parameter>
+    <parameter>
+      <name>InitialPreConsolidationPressure</name>
+      <type>Constant</type>
+      <value>2e5</value>
+    </parameter>
+    <parameter>
+      <name>InitialVolumeRatio</name>
+      <type>Constant</type>
+      <value>1.78571428571428571429</value>
+    </parameter>
+    <parameter>
+      <name>ResidualLiquidSaturation</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>ResidualGasSaturation</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>BubblePressure</name>
+      <type>Constant</type>
+      <value>1e4</value>
+    </parameter>
+    <parameter>
+      <name>VanGenuchtenExponent_m</name>
+      <type>Constant</type>
+      <value>0.4</value>
+    </parameter>
+    <parameter>
+      <name>MacroSaturationLawMode</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroStateLocalSolveMode</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroPotentialConvention</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroSwellingStrainSlope</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroMacroPotentialExchangeCoefficient</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MacroPotentialViscosity</name>
+      <type>Constant</type>
+      <value>1000.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMacro</name>
+      <type>Constant</type>
+      <value>1000.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMicro</name>
+      <type>Constant</type>
+      <value>1300.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceDensitySolid</name>
+      <type>Constant</type>
+      <value>2470.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityA</name>
+      <type>Constant</type>
+      <value>1.3</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityB</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>HamakerConstant</name>
+      <type>Constant</type>
+      <value>-6e-20</value>
+    </parameter>
+    <parameter>
+      <name>SpecificSurface</name>
+      <type>Constant</type>
+      <value>100.0</value>
+    </parameter>
+    <parameter>
+      <name>AreaFactorTuller</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>PoreAreaShapeFactorTuller</name>
+      <type>Constant</type>
+      <value>0.8584073464102069</value>
+    </parameter>
+    <parameter>
+      <name>CharacteristicPoreSize</name>
+      <type>Constant</type>
+      <value>1e-5</value>
+    </parameter>
+    <parameter>
+      <name>SurfaceTension</name>
+      <type>Constant</type>
+      <value>0.0715</value>

```

```

+       </parameter>
+       <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.432</value>
+       </parameter>
+     </parameter>
+     <parameter>
+       <name>n_l0</name>
+       <type>Constant</type>
+       <value>0.1</value>
+     </parameter>
+     <parameter>
+       <name>rho_lR0</name>
+       <type>Constant</type>
+       <value>1300.0</value>
+     </parameter>
+     <parameter>
+       <name>epsilon_sw0</name>
+       <type>Constant</type>
+       <value>0.0</value>
+     </parameter>
+     <parameter>
+       <name>displacement0</name>
+       <type>Constant</type>
+       <values>0 0</values>
+     </parameter>
+     <parameter>
+       <name>zero</name>
+       <type>Constant</type>
+       <value>0.0</value>
+     </parameter>
+     <parameter>
+       <name>top_pressure</name>
+       <type>Constant</type>
+       <value>-1e5</value>
+     </parameter>
+     <parameter>
+       <name>pressure_ic</name>
+       <type>Constant</type>
+       <value>-5e3</value>
+     </parameter>
+   </parameters>
+   <process_variables>
+     <process_variable>
+       <name>displacement</name>
+       <components>2</components>
+       <order>1</order>
+       <initial_condition>displacement0</initial_condition>
+       <boundary_conditions>
+         <boundary_condition>
+           <mesh>square_1x1_quad_1e0_left</mesh>
+           <type>Dirichlet</type>
+           <component>0</component>
+           <parameter>zero</parameter>
+         </boundary_condition>
+         <boundary_condition>
+           <mesh>square_1x1_quad_1e0_bottom</mesh>
+           <type>Dirichlet</type>
+           <component>1</component>
+           <parameter>zero</parameter>
+         </boundary_condition>
+         <boundary_condition>
+           <mesh>square_1x1_quad_1e0_top</mesh>
+           <type>Neumann</type>
+           <component>1</component>
+           <parameter>top_pressure</parameter>
+         </boundary_condition>
+         <boundary_condition>
+           <mesh>square_1x1_quad_1e0_right</mesh>
+           <type>Dirichlet</type>
+           <component>0</component>
+           <parameter>zero</parameter>
+         </boundary_condition>
+       </boundary_conditions>
+     </process_variable>
+     <process_variable>
+       <name>pressure</name>
+       <components>1</components>
+       <order>1</order>
+       <initial_condition>pressure_ic</initial_condition>
+       <boundary_conditions>
+         <boundary_condition>
+           <mesh>square_1x1_quad_1e0_top</mesh>
+           <type>Dirichlet</type>
+           <parameter>pressure_ic</parameter>
+         </boundary_condition>
+       </boundary_conditions>
+     </process_variable>
+   </process_variables>
+   <nonlinear_solvers>
+     <nonlinear_solver>
+       <name>basic_newton</name>
+       <type>Newton</type>
+       <max_iter>60</max_iter>
+       <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+   </nonlinear_solvers>
+   <linear_solvers>
+     <linear_solver>
+       <name>general_linear_solver</name>
+       <eigen>
+         <solver_type>SparseLU</solver_type>
+         <scaling>true</scaling>
+       </eigen>
+     </linear_solver>
+   </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_tuller_native.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_tulle
r_native.prj
new file mode 100644
index 0000000000..f85a1bf76f
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_dsm_micromacro_mcc_tuller_native.prj
@@ -0,0 +1,315 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
  <meshes>

```

```

+ <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+ </meshes>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <jacobian_assembler>
+ <type>Analytical</type>
+ </jacobian_assembler>
+ <constitutive_relation>
+ <type>MFront</type>
+ <behaviour>ModCamClay_semiExpl_constE</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="YoungModulus"/>
+ <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable name="ElasticStrain"/>
+ <secondary_variable name="EquivalentPlasticStrain"/>
+ <secondary_variable name="PreConsolidationPressure"/>
+ <secondary_variable name="PlasticVolumetricStrain"/>
+ <secondary_variable name="sigma"/>
+ <secondary_variable name="epsilon"/>
+ <secondary_variable name="velocity"/>
+ <secondary_variable name="saturation"/>
+ <secondary_variable name="VolumeRatio"/>
+ </secondary_variables>
+ <specific_body_force>0 0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+ <medium>
+ <phases>
+ <phase>
+ <type>AqueousLiquid</type>
+ <properties>
+ <property>
+ <name>viscosity</name>
+ <type>Constant</type>
+ <value>1e-3</value>
+ </property>
+ <property>
+ <name>density</name>
+ <type>Constant</type>
+ <value>1e3</value>
+ </property>
+ </properties>
+ </phase>
+ <phase>
+ <type>Solid</type>
+ <properties>
+ <property>
+ <name>density</name>
+ <type>Constant</type>
+ <value>2780.0</value>
+ </property>
+ </properties>
+ </phase>
+ </phases>
+ <properties>
+ <property>
+ <name>permeability</name>
+ <type>Constant</type>
+ <value>1e-12</value>
+ </property>
+ <property>
+ <name>porosity</name>
+ <type>PorosityFromMassBalance</type>
+ <initial_porosity>phi0</initial_porosity>
+ <minimal_porosity>0</minimal_porosity>
+ <maximal_porosity>1</maximal_porosity>
+ </property>
+ <property>
+ <name>reference_temperature</name>
+ <type>Constant</type>
+ <value>293.15</value>
+ </property>
+ <property>
+ <name>relative_permeability</name>
+ <type>RelativePermeabilityVanGenuchten</type>
+ <residual_liquid_saturation>0.0</residual_liquid_saturation>
+ <residual_gas_saturation>0</residual_gas_saturation>
+ <exponent>0.01</exponent>
+ <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+ </property>
+ <property>
+ <name>saturation</name>
+ <type>SaturationTuller</type>
+ <area_factor_tuller>1.0</area_factor_tuller>
+ <pore_area_shape_factor_tuller>0.8584073464102069</pore_area_shape_factor_tuller>
+ <characteristic_pore_size>1e-5</characteristic_pore_size>
+ <surface_tension>0.0715</surface_tension>
+ </property>
+ <property>
+ <name>bishops_effective_stress</name>
+ <type>BishopsPowerLaw</type>

```

```

+         <exponent>1</exponent>
+     </property>
+     <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+     </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>basic_newton</nonlinear_solver>
+             <convergence_criterion>
+                 <type>PerComponentDeltaX</type>
+                 <norm_type>NORM2</norm_type>
+                 <reltols>1e-10 1e0 1e-9</reltols>
+             </convergence_criterion>
+             <time_discretization>
+                 <type>BackwardEuler</type>
+             </time_discretization>
+             <time_stepping>
+                 <type>FixedTimeStepping</type>
+                 <t_initial>0</t_initial>
+                 <t_end>4</t_end>
+                 <timesteps>
+                     <pair>
+                         <repeat>4</repeat>
+                         <delta_t>1</delta_t>
+                     </pair>
+                 </timesteps>
+             </time_stepping>
+         </process>
+     </processes>
+     <output>
+         <type>VTK</type>
+         <prefix>#front_parity_1element_dsm_micromacro_mcc_tuller_native</prefix>
+         <suffix>_ts_{:timestep}_t_{:time}</suffix>
+         <timesteps>
+             <pair>
+                 <repeat>10</repeat>
+                 <each_steps>1</each_steps>
+             </pair>
+         </timesteps>
+         <variables>
+             <variable>displacement</variable>
+             <variable>pressure</variable>
+             <variable>sigma</variable>
+             <variable>epsilon</variable>
+             <variable>saturation</variable>
+             <variable>velocity</variable>
+             <variable>ElasticStrain</variable>
+             <variable>EquivalentPlasticStrain</variable>
+             <variable>PreConsolidationPressure</variable>
+             <variable>PlasticVolumetricStrain</variable>
+             <variable>VolumeRatio</variable>
+         </variables>
+     </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>CriticalStateLineSlope</name>
+         <type>Constant</type>
+         <value>1.2</value>
+     </parameter>
+     <parameter>
+         <name>SwellingLineSlope</name>
+         <type>Constant</type>
+         <value>6.6e-3</value>
+     </parameter>
+     <parameter>
+         <name>VirginConsolidationLineSlope</name>
+         <type>Constant</type>
+         <value>7.7e-2</value>
+     </parameter>
+     <parameter>
+         <name>InitialPreConsolidationPressure</name>
+         <type>Constant</type>
+         <value>2e5</value>
+     </parameter>
+     <parameter>
+         <name>InitialVolumeRatio</name>
+         <type>Constant</type>
+         <value>1.78571428571428571429</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>zero</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>

```



```

+         <value>0.432</value>
+     </parameter>
+     <parameter>
+         <name>top_pressure</name>
+         <type>Constant</type>
+         <value>-1e5</value>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>-5e3</value>
+     </parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_left</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_bottom</mesh>
+                 <type>Dirichlet</type>
+                 <component>1</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_top</mesh>
+                 <type>Neumann</type>
+                 <component>1</component>
+                 <parameter>top_pressure</parameter>
+             </boundary_condition>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_right</mesh>
+                 <type>Dirichlet</type>
+                 <component>0</component>
+                 <parameter>zero</parameter>
+             </boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+     <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_top</mesh>
+                 <type>Dirichlet</type>
+                 <parameter>pressure_ic</parameter>
+             </boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>60</max_iter>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+             <solver_type>SparseLU</solver_type>
+             <scaling>true</scaling>
+         </eigen>
+     </linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_elastic_bridge.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_elastic_bridge.prj
new file mode 100644
index 0000000000..aad16e1750
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_elastic_bridge.prj
@@ -0,0 +1,405 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+     <meshes>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+     </meshes>
+     <processes>
+         <process>
+             <name>RM</name>
+             <type>RICHARDS_MECHANICS</type>
+             <integration_order>2</integration_order>
+             <jacobian_assembler>
+                 <type>Analytical</type>
+             </jacobian_assembler>
+             <constitutive_relation>
+                 <type>MFrontRichardsMechanics</type>
+                 <behaviour>RichardsMechanicsDSMMicroMacroBridge</behaviour>
+             <material_properties>
+                 <material_property name="YoungModulus" parameter="YoungModulus" />
+                 <material_property name="PoissonRatio" parameter="PoissonRatio" />
+                 <material_property name="SwellingSlope" parameter="SwellingSlope" />
+                 <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient" />
+                 <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro" />
+                 <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro" />
+                 <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid" />
+                 <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA" />
+                 <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB" />
+                 <material_property name="HamakerConstant" parameter="HamakerConstant" />
+                 <material_property name="SpecificSurface" parameter="SpecificSurface" />
+                 <material_property name="InitialPorosity" parameter="phi0" />
+             </material_properties>
+         </process>
+     </processes>

```

```

+      <material_property name="AreaFactorTuller" parameter="AreaFactorTuller" />
+      <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller" />
+      <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize" />
+      <material_property name="SurfaceTension" parameter="SurfaceTension" />
+    </material_properties>
+    <initial_values>
+      <state_variable name="n_l" parameter="n_l0" />
+      <state_variable name="rho_LR" parameter="rho_LR0" />
+      <state_variable name="epsilon_sw" parameter="epsilon_sw0" />
+    </initial_values>
+    </constitutive_relation>
+    <process_variables>
+      <pressure>pressure</pressure>
+      <displacement>displacement</displacement>
+    </process_variables>
+    <secondary_variables>
+      <secondary_variable name="sigma" />
+      <secondary_variable name="epsilon" />
+      <secondary_variable name="velocity" />
+      <secondary_variable name="saturation" />
+    </secondary_variables>
+    <specific_body_force>0 0</specific_body_force>
+    <initial_stress>sigma0</initial_stress>
+    <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+    <mass_lumping>true</mass_lumping>
+  </process>
+</processes>
+<media>
+  <medium>
+    <phases>
+      <phase>
+        <type>AqueousLiquid</type>
+        <properties>
+          <property>
+            <name>viscosity</name>
+            <type>Constant</type>
+            <value>1e-3</value>
+          </property>
+          <property>
+            <name>density</name>
+            <type>Constant</type>
+            <value>1e3</value>
+          </property>
+        </properties>
+      </phase>
+      <phase>
+        <type>Solid</type>
+        <properties>
+          <property>
+            <name>density</name>
+            <type>Constant</type>
+            <value>2700.0</value>
+          </property>
+        </properties>
+      </phase>
+    </phases>
+    <properties>
+      <property>
+        <name>biot_coefficient</name>
+        <type>Constant</type>
+        <value>0.6</value>
+      </property>
+      <property>
+        <name>permeability</name>
+        <type>Constant</type>
+        <value>2e-21</value>
+      </property>
+      <property>
+        <name>transport_porosity</name>
+        <type>TransportPorosityFromMassBalance</type>
+        <initial_porosity>phi_tr0</initial_porosity>
+        <minimal_porosity>0</minimal_porosity>
+        <maximal_porosity>1</maximal_porosity>
+      </property>
+      <property>
+        <name>porosity</name>
+        <type>PorosityFromMassBalance</type>
+        <initial_porosity>phi0</initial_porosity>
+        <minimal_porosity>0</minimal_porosity>
+        <maximal_porosity>1</maximal_porosity>
+      </property>
+      <property>
+        <name>reference_temperature</name>
+        <type>Constant</type>
+        <value>293.15</value>
+      </property>
+      <property>
+        <name>relative_permeability</name>
+        <type>Constant</type>
+        <value>1</value>
+      </property>
+      <property>
+        <name>saturation</name>
+        <type>Constant</type>
+        <value>1</value>
+      </property>
+      <property>
+        <name>bishops_effective_stress</name>
+        <type>BishopsSaturationCutoff</type>
+        <cutoff_value>1</cutoff_value>
+      </property>
+    </properties>
+  </medium>
+</media>
+<time_loop>
+  <processes>
+    <process ref="RM">
+      <nonlinear_solver>basic_newton</nonlinear_solver>
+      <convergence_criterion>
+        <type>PerComponentDeltaX</type>
+        <norm_type>NORM2</norm_type>
+        <reltols>1e-10 1e0 1e-9</reltols>
+      </convergence_criterion>
+      <time_discretization>
+        <type>BackwardEuler</type>
+      </time_discretization>
+      <time_stepping>

```

```

+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>4</t_end>
+         <timesteps>
+             <pair>
+                 <repeat>4</repeat>
+                 <delta_t>1</delta_t>
+             </pair>
+         </timesteps>
+     </time_stepping>
+ </process>
+ </processes>
+ <output>
+     <type>VTK</type>
+     <prefix>mfront_parity_1element_elastic_bridge</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+         <pair>
+             <repeat>10</repeat>
+             <each_steps>1</each_steps>
+         </pair>
+     </timesteps>
+     <variables>
+         <variable>displacement</variable>
+         <variable>pressure</variable>
+         <variable>sigma</variable>
+         <variable>epsilon</variable>
+         <variable>saturation</variable>
+         <variable>velocity</variable>
+     </variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>0</expression>
+         <expression>0</expression>
+         <expression>0</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>SwellingSlope</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>MassExchangeCoefficient</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>ReferenceLiquidDensityMacro</name>
+         <type>Constant</type>
+         <value>1000.0</value>
+     </parameter>
+     <parameter>
+         <name>ReferenceLiquidDensityMicro</name>
+         <type>Constant</type>
+         <value>1300.0</value>
+     </parameter>
+     <parameter>
+         <name>ReferenceDensitySolid</name>
+         <type>Constant</type>
+         <value>2470.0</value>
+     </parameter>
+     <parameter>
+         <name>MicroLiquidDensityA</name>
+         <type>Constant</type>
+         <value>1.3</value>
+     </parameter>
+     <parameter>
+         <name>MicroLiquidDensityB</name>
+         <type>Constant</type>
+         <value>1.0</value>
+     </parameter>
+     <parameter>
+         <name>HamakerConstant</name>
+         <type>Constant</type>
+         <value>-6e-20</value>
+     </parameter>
+     <parameter>
+         <name>SpecificSurface</name>
+         <type>Constant</type>
+         <value>100.0</value>
+     </parameter>
+     <parameter>
+         <name>AreaFactorTuller</name>
+         <type>Constant</type>
+         <value>1.0</value>
+     </parameter>
+     <parameter>
+         <name>PoreAreaShapeFactorTuller</name>
+         <type>Constant</type>
+         <value>0.8584073464102069</value>
+     </parameter>
+     <parameter>
+         <name>CharacteristicPoreSize</name>
+         <type>Constant</type>
+         <value>1e-5</value>
+     </parameter>
+     <parameter>
+         <name>SurfaceTension</name>
+         <type>Constant</type>
+         <value>0.0715</value>
+     </parameter>
+ </parameters>

```

```

+       <name>n_l0</name>
+       <type>Constant</type>
+       <value>0.1</value>
+     </parameter>
+   <parameter>
+     <name>rho_lR0</name>
+     <type>Constant</type>
+     <value>1300.0</value>
+   </parameter>
+   <parameter>
+     <name>epsilon_sw0</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>displacement0</name>
+     <type>Constant</type>
+     <values>0 0</values>
+   </parameter>
+   <parameter>
+     <name>zero</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>phi0</name>
+     <type>Constant</type>
+     <value>0.432</value>
+   </parameter>
+   <parameter>
+     <name>phi_tr0</name>
+     <type>Constant</type>
+     <value>0.332</value>
+   </parameter>
+   <parameter>
+     <name>pressure_ic</name>
+     <type>Constant</type>
+     <value>0</value>
+   </parameter>
+   <parameter>
+     <name>pressure_bc_scale</name>
+     <type>Constant</type>
+     <value>1000</value>
+   </parameter>
+   <parameter>
+     <name>pressure_bc</name>
+     <type>CurveScaled</type>
+     <curve>pressure_ramp_parity</curve>
+     <parameter>pressure_bc_scale</parameter>
+   </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver>
+     <name>basic_newton</name>
+     <type>Newton</type>

```

```

+         <max_iter>60</max_iter>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+             <solver_type>SparselU</solver_type>
+             <scaling>true</scaling>
+         </eigen>
+     </linear_solver>
+ </linear_solvers>
+ <curves>
+     <curve>
+         <name>pressure_ramp_parity</name>
+         <coords>0 1 2 3 4</coords>
+         <values>0 0.25 0.5 0.75 1</values>
+     </curve>
+ </curves>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_elastic_native.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_elastic_native.prj
new file mode 100644
index 0000000000..a4625d1a23
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_elastic_native.prj
@@ -0,0 +1,303 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+     <meshes>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+     </meshes>
+     <processes>
+         <process>
+             <name>RM</name>
+             <type>RICHARDS_MECHANICS</type>
+             <integration_order>2</integration_order>
+             <jacobian_assembler>
+                 <type>Analytical</type>
+             </jacobian_assembler>
+             <constitutive_relation>
+                 <type>LinearElasticIsotropic</type>
+                 <youngs_modulus>YoungModulus</youngs_modulus>
+                 <poissons_ratio>PoissonRatio</poissons_ratio>
+             </constitutive_relation>
+             <process_variables>
+                 <pressure>pressure</pressure>
+                 <displacement>displacement</displacement>
+             </process_variables>
+             <secondary_variables>
+                 <secondary_variable name="sigma" />
+                 <secondary_variable name="epsilon" />
+                 <secondary_variable name="velocity" />
+                 <secondary_variable name="saturation" />
+             </secondary_variables>
+             <specific_body_force>0</specific_body_force>
+             <initial_stress>sigma0</initial_stress>
+             <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+             <mass_lumping>true</mass_lumping>
+         </process>
+     </processes>
+     <media>
+         <medium>
+             <phases>
+                 <phase>
+                     <type>AqueousLiquid</type>
+                     <properties>
+                         <property>
+                             <name>viscosity</name>
+                             <type>Constant</type>
+                             <value>1e-3</value>
+                         </property>
+                         <property>
+                             <name>density</name>
+                             <type>Constant</type>
+                             <value>1e3</value>
+                         </property>
+                     </properties>
+                 </phase>
+                 <phase>
+                     <type>Solid</type>
+                     <properties>
+                         <property>
+                             <name>density</name>
+                             <type>Constant</type>
+                             <value>2780.0</value>
+                         </property>
+                     </properties>
+                 </phase>
+             </phases>
+             <properties>
+                 <property>
+                     <name>biot_coefficient</name>
+                     <type>Constant</type>
+                     <value>0.6</value>
+                 </property>
+                 <property>
+                     <name>permeability</name>
+                     <type>Constant</type>
+                     <value>2e-21</value>
+                 </property>
+                 <property>
+                     <name>transport_porosity</name>
+                     <type>TransportPorosityFromMassBalance</type>
+                     <initial_porosity>phi_tr0</initial_porosity>
+                     <minimal_porosity>0</minimal_porosity>
+                     <maximal_porosity>1</maximal_porosity>
+                 </property>
+                 <property>
+                     <name>porosity</name>
+                     <type>PorosityFromMassBalance</type>
+                     <initial_porosity>phi0</initial_porosity>
+                     <minimal_porosity>0</minimal_porosity>

```

```

+         <maximal_porosity>1</maximal_porosity>
+     </property>
+     <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+     </property>
+     <property>
+         <name>relative_permeability</name>
+         <type>Constant</type>
+         <value>1</value>
+     </property>
+     <property>
+         <name>saturation</name>
+         <type>Constant</type>
+         <value>1</value>
+     </property>
+     <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsSaturationCutoff</type>
+         <cutoff_value>1</cutoff_value>
+     </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>basic_newton</nonlinear_solver>
+             <convergence_criterion>
+                 <type>PerComponentDeltaX</type>
+                 <norm_type>NORM2</norm_type>
+                 <reltols>1e-10 1e-9</reltols>
+             </convergence_criterion>
+             <time_discretization>
+                 <type>BackwardEuler</type>
+             </time_discretization>
+             <time_stepping>
+                 <type>FixedTimeStepping</type>
+                 <t_initial>0</t_initial>
+                 <t_end>4</t_end>
+                 <timesteps>
+                     <pair>
+                         <repeat>4</repeat>
+                         <delta_t>1</delta_t>
+                     </pair>
+                 </timesteps>
+             </time_stepping>
+         </process>
+     </processes>
+     <output>
+         <type>VTK</type>
+         <prefix>#front_parity_1element_elastic_native</prefix>
+         <suffix>_ts_{:timestep}_t_{:time}</suffix>
+         <timesteps>
+             <pair>
+                 <repeat>10</repeat>
+                 <each_steps>1</each_steps>
+             </pair>
+         </timesteps>
+         <variables>
+             <variable>displacement</variable>
+             <variable>pressure</variable>
+             <variable>sigma</variable>
+             <variable>epsilon</variable>
+             <variable>saturation</variable>
+             <variable>velocity</variable>
+         </variables>
+     </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>0</expression>
+         <expression>0</expression>
+         <expression>0</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>zero</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.432</value>
+     </parameter>
+     <parameter>
+         <name>phi_tr0</name>
+         <type>Constant</type>
+         <value>0.332</value>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>0</value>
+     </parameter>
+     <parameter>
+         <name>pressure_bc_scale</name>
+         <type>Constant</type>

```

```

+         <value>1000</value>
+     </parameter>
+ <parameter>
+     <name>pressure_bc</name>
+     <type>CurveScaled</type>
+     <curve>pressure_ramp_parity</curve>
+     <parameter>pressure_bc_scale</parameter>
+ </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver>
+     <name>basic_newton</name>
+     <type>Newton</type>
+     <max_iter>60</max_iter>
+     <linear_solver>general_linear_solver</linear_solver>
+   </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver>
+     <name>general_linear_solver</name>
+     <eigen>
+       <solver_type>SparseLU</solver_type>
+       <scaling>true</scaling>
+     </eigen>
+   </linear_solver>
+ </linear_solvers>
+ <curves>
+   <curve>
+     <name>pressure_ramp_parity</name>
+     <coords>0 1 2 3 4</coords>
+     <values>0 0.25 0.5 0.75 1</values>
+   </curve>
+ </curves>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_mcc_bridge.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_mcc_bridge.prj
new file mode 100644
index 0000000000..2f1a979734
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_mcc_bridge.prj
@@ -0,0 +1,339 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+   <meshes>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+   </meshes>
+   <processes>
+     <process>
+       <name>RM</name>
+       <type>RICHARDS_MECHANICS</type>
+       <integration_order>2</integration_order>
+       <jacobian_assembler>

```

```

+         <type>Analytical</type>
+     </jacobian_assembler>
+ </constitutive_relation>
+ <type>MFrontRichardsMechanics</type>
+ <behaviour>ModCamClay_semiExpl_constE_pressureCoupled</behaviour>
+ <material_properties>
+     <material_property name="YoungModulus" parameter="YoungModulus"/>
+     <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+     <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+     <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+     <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+     <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+     <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+     <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+     <material_property name="BubblePressure" parameter="BubblePressure"/>
+     <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+ </material_properties>
+ <initial_values>
+     <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+     <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+     <pressure>pressure</pressure>
+     <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+     <secondary_variable name="ElasticStrain"/>
+     <secondary_variable name="EquivalentPlasticStrain"/>
+     <secondary_variable name="PreConsolidationPressure"/>
+     <secondary_variable name="PlasticVolumetricStrain"/>
+     <secondary_variable name="sigma"/>
+     <secondary_variable name="epsilon"/>
+     <secondary_variable name="velocity"/>
+     <secondary_variable name="saturation"/>
+     <secondary_variable name="VolumeRatio"/>
+ </secondary_variables>
+ <specific_body_force>0 0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+     <medium>
+         <phases>
+             <phase>
+                 <type>AqueousLiquid</type>
+                 <properties>
+                     <property>
+                         <name>viscosity</name>
+                         <type>Constant</type>
+                         <value>1e-3</value>
+                     </property>
+                     <property>
+                         <name>density</name>
+                         <type>Constant</type>
+                         <value>1e3</value>
+                     </property>
+                 </properties>
+             </phase>
+             <phase>
+                 <type>Solid</type>
+                 <properties>
+                     <property>
+                         <name>density</name>
+                         <type>Constant</type>
+                         <value>2700.0</value>
+                     </property>
+                 </properties>
+             </phase>
+         </phases>
+         <properties>
+             <property>
+                 <name>permeability</name>
+                 <type>Constant</type>
+                 <value>1e-12</value>
+             </property>
+             <property>
+                 <name>porosity</name>
+                 <type>PorosityFromMassBalance</type>
+                 <initial_porosity>phi0</initial_porosity>
+                 <minimal_porosity>0</minimal_porosity>
+                 <maximal_porosity>1</maximal_porosity>
+             </property>
+             <property>
+                 <name>reference_temperature</name>
+                 <type>Constant</type>
+                 <value>293.15</value>
+             </property>
+             <property>
+                 <name>relative_permeability</name>
+                 <type>RelativePermeabilityVanGenuchten</type>
+                 <residual_liquid_saturation>0.0</residual_liquid_saturation>
+                 <residual_gas_saturation>0</residual_gas_saturation>
+                 <exponent>0.01</exponent>
+                 <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+             </property>
+             <property>
+                 <name>saturation</name>
+                 <type>SaturationVanGenuchten</type>
+                 <residual_liquid_saturation>0.0</residual_liquid_saturation>
+                 <residual_gas_saturation>0</residual_gas_saturation>
+                 <exponent>0.4</exponent>
+                 <p_b>1e4</p_b>
+             </property>
+             <property>
+                 <name>bishops_effective_stress</name>
+                 <type>BishopsPowerLaw</type>
+                 <exponent>1</exponent>
+             </property>
+             <property>
+                 <name>biot_coefficient</name>
+                 <type>Constant</type>
+                 <value>1.0</value>
+             </property>
+         </properties>
+     </medium>
+ </media>

```



```

+ </medium>
+ </media>
+ <time_loop>
+ <processes>
+ <process ref="RM">
+ <nonlinear_solver>basic_newton</nonlinear_solver>
+ <convergence_criterion>
+ <type>PerComponentDeltaX</type>
+ <norm_type>NORM2</norm_type>
+ <reltols>1e-10 1e0 1e-9</reltols>
+ </convergence_criterion>
+ <time_discretization>
+ <type>BackwardEuler</type>
+ </time_discretization>
+ <time_stepping>
+ <type>FixedTimeStepping</type>
+ <t_initial>0</t_initial>
+ <t_end>4</t_end>
+ <timesteps>
+ <pair>
+ <repeat>4</repeat>
+ <delta_t>1</delta_t>
+ </pair>
+ </timesteps>
+ </time_stepping>
+ </process>
+ </processes>
+ <output>
+ <type>VTK</type>
+ <prefix>#front_parity_1element_mcc_bridge</prefix>
+ <suffix>_ts_{timestep}_t_{time}</suffix>
+ <timesteps>
+ <pair>
+ <repeat>10</repeat>
+ <each_steps>1</each_steps>
+ </pair>
+ </timesteps>
+ <variables>
+ <variable>displacement</variable>
+ <variable>pressure</variable>
+ <variable>sigma</variable>
+ <variable>epsilon</variable>
+ <variable>saturation</variable>
+ <variable>velocity</variable>
+ <variable>ElasticStrain</variable>
+ <variable>EquivalentPlasticStrain</variable>
+ <variable>PreConsolidationPressure</variable>
+ <variable>PlasticVolumetricStrain</variable>
+ <variable>VolumeRatio</variable>
+ </variables>
+ </output>
+ </time_loop>
+ <parameters>
+ <parameter>
+ <name>sigma0</name>
+ <type>Function</type>
+ <expression>-5e3</expression>
+ <expression>-5e3</expression>
+ <expression>-5e3</expression>
+ <expression>0</expression>
+ </parameter>
+ <parameter>
+ <name>YoungModulus</name>
+ <type>Constant</type>
+ <value>52e6</value>
+ </parameter>
+ <parameter>
+ <name>PoissonRatio</name>
+ <type>Constant</type>
+ <value>0.3</value>
+ </parameter>
+ <parameter>
+ <name>CriticalStateLineSlope</name>
+ <type>Constant</type>
+ <value>1.2</value>
+ </parameter>
+ <parameter>
+ <name>SwellingLineSlope</name>
+ <type>Constant</type>
+ <value>6.6e-3</value>
+ </parameter>
+ <parameter>
+ <name>VirginConsolidationLineSlope</name>
+ <type>Constant</type>
+ <value>7.7e-2</value>
+ </parameter>
+ <parameter>
+ <name>InitialPreConsolidationPressure</name>
+ <type>Constant</type>
+ <value>2e5</value>
+ </parameter>
+ <parameter>
+ <name>InitialVolumeRatio</name>
+ <type>Constant</type>
+ <value>1.7857142857142857</value>
+ </parameter>
+ <parameter>
+ <name>ResidualLiquidSaturation</name>
+ <type>Constant</type>
+ <value>0.0</value>
+ </parameter>
+ <parameter>
+ <name>ResidualGasSaturation</name>
+ <type>Constant</type>
+ <value>0.0</value>
+ </parameter>
+ <parameter>
+ <name>BubblePressure</name>
+ <type>Constant</type>
+ <value>1e4</value>
+ </parameter>
+ <parameter>
+ <name>VanGenuchtenExponent_m</name>
+ <type>Constant</type>
+ <value>0.4</value>
+ </parameter>
+ </parameters>

```

```

+       <name>displacement0</name>
+       <type>Constant</type>
+       <values>0 0</values>
+     </parameter>
+   <parameter>
+     <name>zero</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>phi0</name>
+     <type>Constant</type>
+     <value>0.432</value>
+   </parameter>
+   <parameter>
+     <name>top_pressure</name>
+     <type>Constant</type>
+     <value>-1e5</value>
+   </parameter>
+   <parameter>
+     <name>pressure_ic</name>
+     <type>Constant</type>
+     <value>-5e3</value>
+   </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Neumann</type>
+         <component>1</component>
+         <parameter>top_pressure</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_ic</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver>
+     <name>basic_newton</name>
+     <type>Newton</type>
+     <max_iter>60</max_iter>
+     <linear_solver>general_linear_solver</linear_solver>
+   </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver>
+     <name>general_linear_solver</name>
+     <eigen>
+       <solver_type>SparseLU</solver_type>
+       <scaling>true</scaling>
+     </eigen>
+   </linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_mcc_native.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_mcc_native.prj
new file mode 100644
index 0000000000..54dba959f6
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_mcc_native.prj
@@ -0,0 +1,315 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+   <meshes>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+     <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+   </meshes>
+   <processes>
+     <process>
+       <name>RM</name>
+       <type>RICHARDS_MECHANICS</type>
+       <integration_order>2</integration_order>
+       <jacobian_assembler>
+         <type>Analytical</type>
+       </jacobian_assembler>
+       <constitutive_relation>
+         <type>MFront</type>
+         <behaviour>ModCamClay_semiExp_constE</behaviour>
+         <material_properties>

```

```

+         <material_property name="YoungModulus" parameter="YoungModulus"/>
+         <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+         <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+         <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+         <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+         <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+     </material_properties>
+     <initial_values>
+         <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+         <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+     </initial_values>
+ </constitutive_relation>
+ <process_variables>
+     <pressure>pressure</pressure>
+     <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+     <secondary_variable name="ElasticStrain"/>
+     <secondary_variable name="EquivalentPlasticStrain"/>
+     <secondary_variable name="PreConsolidationPressure"/>
+     <secondary_variable name="PlasticVolumetricStrain"/>
+     <secondary_variable name="sigma"/>
+     <secondary_variable name="epsilon"/>
+     <secondary_variable name="velocity"/>
+     <secondary_variable name="saturation"/>
+     <secondary_variable name="VolumeRatio"/>
+ </secondary_variables>
+ <specific_body_force>0 0</specific_body_force>
+ <initial_stress>sigma0</initial_stress>
+ <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+ <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+     <medium>
+         <phases>
+             <phase>
+                 <type>AqueousLiquid</type>
+                 <properties>
+                     <property>
+                         <name>viscosity</name>
+                         <type>Constant</type>
+                         <value>1e-3</value>
+                     </property>
+                     <property>
+                         <name>density</name>
+                         <type>Constant</type>
+                         <value>1e3</value>
+                     </property>
+                 </properties>
+             </phase>
+             <phase>
+                 <type>Solid</type>
+                 <properties>
+                     <property>
+                         <name>density</name>
+                         <type>Constant</type>
+                         <value>2780.0</value>
+                     </property>
+                 </properties>
+             </phase>
+         </phases>
+         <properties>
+             <property>
+                 <name>permeability</name>
+                 <type>Constant</type>
+                 <value>1e-12</value>
+             </property>
+             <property>
+                 <name>porosity</name>
+                 <type>PorosityFromMassBalance</type>
+                 <initial_porosity>phi0</initial_porosity>
+                 <minimal_porosity>0</minimal_porosity>
+                 <maximal_porosity>1</maximal_porosity>
+             </property>
+             <property>
+                 <name>reference_temperature</name>
+                 <type>Constant</type>
+                 <value>293.15</value>
+             </property>
+             <property>
+                 <name>relative_permeability</name>
+                 <type>RelativePermeabilityVanGenuchten</type>
+                 <residual_liquid_saturation>0.0</residual_liquid_saturation>
+                 <residual_gas_saturation>0</residual_gas_saturation>
+                 <exponent>0.01</exponent>
+                 <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+             </property>
+             <property>
+                 <name>saturation</name>
+                 <type>SaturationVanGenuchten</type>
+                 <residual_liquid_saturation>0.0</residual_liquid_saturation>
+                 <residual_gas_saturation>0</residual_gas_saturation>
+                 <exponent>0.4</exponent>
+                 <p_b>1e4</p_b>
+             </property>
+             <property>
+                 <name>bishops_effective_stress</name>
+                 <type>BishopsPowerLaw</type>
+                 <exponent>1</exponent>
+             </property>
+             <property>
+                 <name>biot_coefficient</name>
+                 <type>Constant</type>
+                 <value>1.0</value>
+             </property>
+         </properties>
+     </medium>
+ </media>
+ <time_loop>
+     <processes>
+         <process ref="RM">
+             <nonlinear_solver>basic_newton</nonlinear_solver>
+             <convergence_criterion>
+                 <type>PerComponentDeltaX</type>
+                 <norm_type>NORM2</norm_type>
+                 <reltols>1e-10 1e0 1e-9</reltols>

```

```

+         </convergence_criterion>
+         <time_discretization>
+             <type>BackwardEuler</type>
+         </time_discretization>
+         <time_stepping>
+             <type>FixedTimeStepping</type>
+             <t_initial>0</t_initial>
+             <t_end>4</t_end>
+             <timesteps>
+                 <pair>
+                     <repeat>4</repeat>
+                     <delta_t>1</delta_t>
+                 </pair>
+             </timesteps>
+         </time_stepping>
+     </process>
+ </processes>
+ <output>
+     <type>VTK</type>
+     <prefix>#front_parity_1element_mcc_native</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+         <pair>
+             <repeat>10</repeat>
+             <each_steps>1</each_steps>
+         </pair>
+     </timesteps>
+     <variables>
+         <variable>displacement</variable>
+         <variable>pressure</variable>
+         <variable>sigma</variable>
+         <variable>epsilon</variable>
+         <variable>saturation</variable>
+         <variable>velocity</variable>
+         <variable>ElasticStrain</variable>
+         <variable>EquivalentPlasticStrain</variable>
+         <variable>PreConsolidationPressure</variable>
+         <variable>PlasticVolumetricStrain</variable>
+         <variable>VolumeRatio</variable>
+     </variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>CriticalStateLineSlope</name>
+         <type>Constant</type>
+         <value>1.2</value>
+     </parameter>
+     <parameter>
+         <name>SwellingLineSlope</name>
+         <type>Constant</type>
+         <value>6.6e-3</value>
+     </parameter>
+     <parameter>
+         <name>VirginConsolidationLineSlope</name>
+         <type>Constant</type>
+         <value>7.7e-2</value>
+     </parameter>
+     <parameter>
+         <name>InitialPreConsolidationPressure</name>
+         <type>Constant</type>
+         <value>2e5</value>
+     </parameter>
+     <parameter>
+         <name>InitialVolumeRatio</name>
+         <type>Constant</type>
+         <value>1.78571428571428571429</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>zero</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.432</value>
+     </parameter>
+     <parameter>
+         <name>top_pressure</name>
+         <type>Constant</type>
+         <value>-1e5</value>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>-5e3</value>
+     </parameter>
+ </parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>

```

```

+ <initial_condition>displacement0</initial_condition>
+ <boundary_conditions>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_left</mesh>
+ <type>Dirichlet</type>
+ <component>8</component>
+ <parameter>zero</parameter>
+ </boundary_condition>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_bottom</mesh>
+ <type>Dirichlet</type>
+ <component>1</component>
+ <parameter>zero</parameter>
+ </boundary_condition>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_top</mesh>
+ <type>Neumann</type>
+ <component>1</component>
+ <parameter>top_pressure</parameter>
+ </boundary_condition>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_right</mesh>
+ <type>Dirichlet</type>
+ <component>8</component>
+ <parameter>zero</parameter>
+ </boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ <process_variable>
+ <name>pressure</name>
+ <components>1</components>
+ <order>1</order>
+ <initial_condition>pressure_ic</initial_condition>
+ <boundary_conditions>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_top</mesh>
+ <type>Dirichlet</type>
+ <parameter>pressure_ic</parameter>
+ </boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+ <nonlinear_solver>
+ <name>basic_newton</name>
+ <type>Newton</type>
+ <max_iter>60</max_iter>
+ <linear_solver>general_linear_solver</linear_solver>
+ </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+ <linear_solver>
+ <name>general_linear_solver</name>
+ <eigen>
+ <solver_type>SparseLU</solver_type>
+ <scaling>true</scaling>
+ </eigen>
+ </linear_solver>
+ </linear_solvers>
+ <OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_native.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_native.prj
new file mode 100644
index 0000000000..085cd99048
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_native.prj
@@ -0,0 +1,304 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+ <meshes>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+ </meshes>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <micro_porosity>
+ <mass_exchange_coefficient>5e-15</mass_exchange_coefficient>
+ <nonlinear_solver>
+ <maximum_iterations>100</maximum_iterations>
+ <residual_tolerance>1e-8</residual_tolerance>
+ <increment_tolerance>1e-20</increment_tolerance>
+ </nonlinear_solver>
+ </micro_porosity>
+ <jacobian_assembler>
+ <type>Analytical</type>
+ </jacobian_assembler>
+ <constitutive_relation>
+ <type>MFront</type>
+ <behaviour>ModCamClay_semiExpl_constE</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="YoungModulus" />
+ <material_property name="PoissonRatio" parameter="PoissonRatio" />
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope" />
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope" />
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope" />
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure" />
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio" />
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable name="ElasticStrain" />
+ <secondary_variable name="EquivalentPlasticStrain" />
+ <secondary_variable name="PreConsolidationPressure" />
+ <secondary_variable name="PlasticVolumetricStrain" />
+ <secondary_variable name="sigma" />

```

```

+         <secondary_variable name="swelling_stress" />
+         <secondary_variable name="epsilon" />
+         <secondary_variable name="velocity" />
+         <secondary_variable name="saturation" />
+         <secondary_variable name="porosity" />
+         <secondary_variable name="VolumeRatio" />
+         <secondary_variable name="transport_porosity" />
+         <secondary_variable name="dry_density_solid" />
+         <secondary_variable name="micro_saturation" />
+         <secondary_variable name="micro_pressure" />
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2700.0</value>
+           </property>
+           <property>
+             <name>swelling_stress_rate</name>
+             <type>SaturationDependentSwelling</type>
+             <swelling_pressures>13e6 13e6 13e6</swelling_pressures>
+             <exponents>1 1 1</exponents>
+             <lower_saturation_limit>0</lower_saturation_limit>
+             <upper_saturation_limit>1</upper_saturation_limit>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>0.6</value>
+       </property>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>2e-21</value>
+       </property>
+       <property>
+         <name>transport_porosity</name>
+         <type>TransportPorosityFromMassBalance</type>
+         <initial_porosity>phi_tr0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>Constant</type>
+         <value>1</value>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.4</exponent>
+         <p_b>15e6</p_b>
+       </property>
+       <property>
+         <name>saturation_micro</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.4</exponent>
+         <p_b>300e6</p_b>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsSaturationCutoff</type>
+         <cutoff_value>1</cutoff_value>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>

```

```

+         <convergence_criterion>
+             <type>PerComponentDeltaX</type>
+             <norm_type>NORM2</norm_type>
+             <reltols>1e-10 1e0 1e-9</reltols>
+         </convergence_criterion>
+         <time_discretization>
+             <type>BackwardEuler</type>
+         </time_discretization>
+         <time_stepping>
+             <type>FixedTimeStepping</type><t_initial>0</t_initial><t_end>4</t_end><timesteps><pair><repeat>4</repeat><delta_t>1</delta_t></pair></timesteps></time_stepping>
+     </process>
+ </processes>
+ <output>
+     <type>VTK</type>
+     <prefix>mfront_parity_1element_native</prefix>
+     <suffix>_ts_{: timestep}_t_{: time}</suffix>
+     <timesteps>
+         <pair><repeat>10</repeat><each_steps>1</each_steps></pair></timesteps>
+     <variables>
+         <variable>displacement</variable><variable>pressure</variable><variable>sigma</variable><variable>epsilon</variable><variable>saturation</variable><variable>swelling_s
tress</variable></variables>
+     </output>
+ </time_loop>
+ <parameters>
+
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value>
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>CriticalStateLineSlope</name>
+         <type>Constant</type>
+         <value>1.2</value>
+     </parameter>
+     <parameter>
+         <name>SwellingLineSlope</name>
+         <type>Constant</type>
+         <value>6.6e-3</value>
+     </parameter>
+     <parameter>
+         <name>VirginConsolidationLineSlope</name>
+         <type>Constant</type>
+         <value>7.7e-2</value>
+     </parameter>
+     <parameter>
+         <name>InitialPreConsolidationPressure</name>
+         <type>Constant</type>
+         <value>2e5</value>
+     </parameter>
+     <parameter>
+         <name>InitialVolumeRatio</name>
+         <type>Constant</type>
+         <value>1.78571428571428571429</value>
+     </parameter>
+
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>zero</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>phi0</name>
+         <type>Constant</type>
+         <value>0.432</value>
+     </parameter>
+     <parameter>
+         <name>phi_tr0</name>
+         <type>Constant</type>
+         <value>0.332</value>
+     </parameter>
+     <parameter>
+         <name>top_pressure</name>
+         <type>Constant</type>
+         <value>-1e5</value>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>0</value>
+     </parameter>
+     <parameter><name>pressure_bc_scale</name><type>Constant</type><value>1000</value></parameter><parameter><name>pressure_bc</name><type>CurveScaled</type><curve>pressure_ramp_pa
rity</curve><parameter>pressure_bc_scale</parameter></parameter></parameters>
+ <process_variables>
+     <process_variable>
+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+
+             <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition><boundary_co
ndition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><component>0</component><parameter>zero</parameter></boundary_condition><boundary_condition><mesh>square_1x1_qua
d_1e0_bottom</mesh><type>Dirichlet</type><component>1</component><parameter>zero</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Diric
hlet</type><component>1</component><parameter>zero</parameter></boundary_condition></boundary_conditions>
+         </process_variable>
+     <process_variable>
+         <name>pressure</name>

```

```

+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+         <boundary_condition><mesh>square_1x1_quad_1e0_left</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_right</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_bottom</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition><boundary_condition><mesh>square_1x1_quad_1e0_top</mesh><type>Dirichlet</type><parameter>pressure_bc</parameter></boundary_condition></boundary_conditions>
+         </process_variable>
+         </process_variables>
+         <nonlinear_solvers>
+         <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>60</max_iter>
+         <linear_solver>general_linear_solver</linear_solver>
+         </nonlinear_solver>
+         </nonlinear_solvers>
+         <linear_solvers>
+         <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+         <solver_type>SparseLU</solver_type>
+         <scaling>true</scaling>
+         </eigen>
+         </linear_solver>
+         </linear_solvers>
+         <curves><curve><name>pressure_ramp_parity</name><coords>0 1 2 3 4</coords><values>0 0.25 0.5 0.75 1</values></curve></curves></OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_bridge.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_bridge.prj
new file mode 100644
index 0000000000..b0215964d9
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_bridge.prj
@@ -0,0 +1,408 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+   <meshes>
+   <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+   <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+   <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+   <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+   <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+   </meshes>
+   <processes>
+   <process>
+   <name>RM</name>
+   <type>RICHARDS_MECHANICS</type>
+   <integration_order>2</integration_order>
+   <jacobian_assembler>
+   <type>Analytical</type>
+   </jacobian_assembler>
+   <constitutive_relation>
+   <type>MFrontRichardsMechanics</type>
+   <behaviour>RichardsMechanicsDSMMicroMacroBridge</behaviour>
+   <material_properties>
+   <material_property name="YoungModulus" parameter="YoungModulus" />
+   <material_property name="PoissonRatio" parameter="PoissonRatio" />
+   <material_property name="SwellingSlope" parameter="SwellingSlope" />
+   <material_property name="MassExchangeCoefficient" parameter="MassExchangeCoefficient" />
+   <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro" />
+   <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro" />
+   <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid" />
+   <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA" />
+   <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB" />
+   <material_property name="HamakerConstant" parameter="HamakerConstant" />
+   <material_property name="SpecificSurface" parameter="SpecificSurface" />
+   <material_property name="InitialPorosity" parameter="phi0" />
+   <material_property name="AreaFactorTuller" parameter="AreaFactorTuller" />
+   <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller" />
+   <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize" />
+   <material_property name="SurfaceTension" parameter="SurfaceTension" />
+   </material_properties>
+   <initial_values>
+   <state_variable name="n_l" parameter="n_l0" />
+   <state_variable name="rho_LR" parameter="rho_LR0" />
+   <state_variable name="epsilon_sw" parameter="epsilon_sw0" />
+   </initial_values>
+   </constitutive_relation>
+   <process_variables>
+   <pressure>pressure</pressure>
+   <displacement>displacement</displacement>
+   </process_variables>
+   <secondary_variables>
+   <secondary_variable name="sigma" />
+   <secondary_variable name="epsilon" />
+   <secondary_variable name="velocity" />
+   <secondary_variable name="saturation" />
+   </secondary_variables>
+   <specific_body_force>0 0</specific_body_force>
+   <initial_stress>sigma0</initial_stress>
+   <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+   <mass_lumping>true</mass_lumping>
+   </process>
+   </processes>
+   <media>
+   <medium>
+   <phases>
+   <phase>
+   <type>AqueousLiquid</type>
+   <properties>
+   <property>
+   <name>viscosity</name>
+   <type>Constant</type>
+   <value>1e-3</value>
+   </property>
+   <property>
+   <name>density</name>
+   <type>Constant</type>
+   <value>1e3</value>
+   </property>
+   </properties>
+   </phase>
+   <phase>
+   <type>Solid</type>
+   <properties>
+   <property>
+   <name>density</name>

```



```

+         <type>Constant</type>
+         <value>2780.0</value>
+     </property>
+ </properties>
+ </phase>
+ </phases>
+ <properties>
+   <property>
+     <name>biot_coefficient</name>
+     <type>Constant</type>
+     <value>0.6</value>
+   </property>
+   <property>
+     <name>permeability</name>
+     <type>Constant</type>
+     <value>2e-21</value>
+   </property>
+   <property>
+     <name>transport_porosity</name>
+     <type>TransportPorosityFromMassBalance</type>
+     <initial_porosity>phi_tr0</initial_porosity>
+     <minimal_porosity>0</minimal_porosity>
+     <maximal_porosity>1</maximal_porosity>
+   </property>
+   <property>
+     <name>porosity</name>
+     <type>PorosityFromMassBalance</type>
+     <initial_porosity>phi0</initial_porosity>
+     <minimal_porosity>0</minimal_porosity>
+     <maximal_porosity>1</maximal_porosity>
+   </property>
+   <property>
+     <name>reference_temperature</name>
+     <type>Constant</type>
+     <value>293.15</value>
+   </property>
+   <property>
+     <name>relative_permeability</name>
+     <type>Constant</type>
+     <value>1</value>
+   </property>
+   <property>
+     <name>saturation</name>
+     <type>SaturationTuller</type>
+     <area_factor_tuller>1.0</area_factor_tuller>
+     <pore_area_shape_factor_tuller>0.8584073464102069</pore_area_shape_factor_tuller>
+     <characteristic_pore_size>1e-5</characteristic_pore_size>
+     <surface_tension>0.0715</surface_tension>
+   </property>
+   <property>
+     <name>bishops_effective_stress</name>
+     <type>BishopsSaturationCutoff</type>
+     <cutoff_value>1</cutoff_value>
+   </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltol>1e-10 1e0 1e-9</reltol>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>4</t_end>
+         <timesteps>
+           <pair>
+             <repeat>4</repeat>
+             <delta_t>1</delta_t>
+           </pair>
+         </timesteps>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>#front_parity_element_unsat_bridge</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+       <pair>
+         <repeat>10</repeat>
+         <each_steps>1</each_steps>
+       </pair>
+     </timesteps>
+     <variables>
+       <variable>displacement</variable>
+       <variable>pressure</variable>
+       <variable>sigma</variable>
+       <variable>epsilon</variable>
+       <variable>saturation</variable>
+       <variable>velocity</variable>
+     </variables>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>sigma0</name>
+     <type>Function</type>
+     <expression>0</expression>
+     <expression>0</expression>
+     <expression>0</expression>
+     <expression>0</expression>
+   </parameter>
+   <parameter>
+     <name>YoungModulus</name>
+     <type>Constant</type>
+     <value>52e6</value>
+   </parameter>
+ </parameters>

```

```

+      <name>PoissonRatio</name>
+      <type>Constant</type>
+      <value>0.3</value>
+    </parameter>
+    <parameter>
+      <name>SwellingSlope</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MassExchangeCoefficient</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMacro</name>
+      <type>Constant</type>
+      <value>1000.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMicro</name>
+      <type>Constant</type>
+      <value>1300.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceDensitySolid</name>
+      <type>Constant</type>
+      <value>2470.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityA</name>
+      <type>Constant</type>
+      <value>1.3</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityB</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>HamakerConstant</name>
+      <type>Constant</type>
+      <value>-6e-20</value>
+    </parameter>
+    <parameter>
+      <name>SpecificSurface</name>
+      <type>Constant</type>
+      <value>100.0</value>
+    </parameter>
+    <parameter>
+      <name>AreaFactorTuller</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>PoreAreaShapeFactorTuller</name>
+      <type>Constant</type>
+      <value>0.8584073464102069</value>
+    </parameter>
+    <parameter>
+      <name>CharacteristicPoreSize</name>
+      <type>Constant</type>
+      <value>1e-5</value>
+    </parameter>
+    <parameter>
+      <name>SurfaceTension</name>
+      <type>Constant</type>
+      <value>0.0715</value>
+    </parameter>
+    <parameter>
+      <name>n_l0</name>
+      <type>Constant</type>
+      <value>0.1</value>
+    </parameter>
+    <parameter>
+      <name>rho_lR0</name>
+      <type>Constant</type>
+      <value>1300.0</value>
+    </parameter>
+    <parameter>
+      <name>epsilon_sw0</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>displacement0</name>
+      <type>Constant</type>
+      <values>0 0</values>
+    </parameter>
+    <parameter>
+      <name>zero</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>phi0</name>
+      <type>Constant</type>
+      <value>0.432</value>
+    </parameter>
+    <parameter>
+      <name>phi_tr0</name>
+      <type>Constant</type>
+      <value>0.332</value>
+    </parameter>
+    <parameter>
+      <name>pressure_ic</name>
+      <type>Constant</type>
+      <value>0</value>
+    </parameter>
+    <parameter>
+      <name>pressure_bc_scale</name>
+      <type>Constant</type>
+      <value>1000000</value>
+    </parameter>
+    <parameter>
+      <name>pressure_bc</name>
+      <type>CurveScaled</type>

```

```

+         <curve>pressure_ramp_unsat_parity</curve>
+         <parameter>pressure_bc_scale</parameter>
+     </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ics</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_bc</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver>
+     <name>basic_newton</name>
+     <type>Newton</type>
+     <max_iter>60</max_iter>
+     <linear_solver>general_linear_solver</linear_solver>
+   </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver>
+     <name>general_linear_solver</name>
+     <eigen>
+       <solver_type>SparseLU</solver_type>
+       <scaling>true</scaling>
+     </eigen>
+   </linear_solver>
+ </linear_solvers>
+ <curves>
+   <curve>
+     <name>pressure_ramp_unsat_parity</name>
+     <coords>0 1 2 3 4</coords>
+     <values>0 -0.25 -0.5 -0.75 -1</values>
+   </curve>
+ </curves>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_native.prj b/Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_native.prj
new file mode 100644
index 0000000000..7a2416459b
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_parity_1element_unsat_native.prj
@@ -0,0 +1,306 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
  <meshes>
    <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
  </meshes>
  <processes>
    <process>
      <name>RM</name>
      <type>RICHARDS_MECHANICS</type>
      <integration_order>2</integration_order>
      <jacobian_assembler>
        <type>Analytical</type>
      </jacobian_assembler>
      <constitutive_relation>
        <type>LinearElasticIsotropic</type>
        <youngs_modulus>YoungModulus</youngs_modulus>

```

```

+       <poissons_ratio>PoissonRatio</poissons_ratio>
+     </constitutive_relation>
+     <process_variables>
+       <pressure>pressure</pressure>
+       <displacement>displacement</displacement>
+     </process_variables>
+     <secondary_variables>
+       <secondary_variable name="sigma" />
+       <secondary_variable name="epsilon" />
+       <secondary_variable name="velocity" />
+       <secondary_variable name="saturation" />
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+   </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2780.0</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>0.6</value>
+       </property>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>2e-21</value>
+       </property>
+       <property>
+         <name>transport_porosity</name>
+         <type>TransportPorosityFromMassBalance</type>
+         <initial_porosity>phi_tr0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>Constant</type>
+         <value>1</value>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationTuller</type>
+         <area_factor_tuller>1.0</area_factor_tuller>
+         <pore_area_shape_factor_tuller>0.8584073464102069</pore_area_shape_factor_tuller>
+         <characteristic_pore_size>1e-5</characteristic_pore_size>
+         <surface_tension>0.0715</surface_tension>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsSaturationCutoff</type>
+         <cutoff_value>1</cutoff_value>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e0 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>4</t_end>
+         <timesteps>
+           <pair>
+             <repeat>4</repeat>

```

```

+         <delta_t>1</delta_t>
+     </pair>
+ </timesteps>
+ </time_stepping>
+ </process>
+ </processes>
+ <output>
+   <type>VTK</type>
+   <prefix>#front_parity_1element_unsat_natives</prefix>
+   <suffix>_ts_{timestep}_t_{time}</suffix>
+   <timesteps>
+     <pair>
+       <repeat>10</repeat>
+       <each_steps>1</each_steps>
+     </pair>
+   </timesteps>
+   <variables>
+     <variable>displacement</variable>
+     <variable>pressure</variable>
+     <variable>sigma</variable>
+     <variable>epsilon</variable>
+     <variable>saturation</variable>
+     <variable>velocity</variable>
+   </variables>
+ </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>sigma0</name>
+     <type>Function</type>
+     <expression>0</expression>
+     <expression>0</expression>
+     <expression>0</expression>
+     <expression>0</expression>
+   </parameter>
+   <parameter>
+     <name>YoungModulus</name>
+     <type>Constant</type>
+     <value>52e6</value>
+   </parameter>
+   <parameter>
+     <name>PoissonRatio</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>displacement0</name>
+     <type>Constant</type>
+     <values>0 0</values>
+   </parameter>
+   <parameter>
+     <name>zero</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>phi0</name>
+     <type>Constant</type>
+     <value>0.432</value>
+   </parameter>
+   <parameter>
+     <name>phi_tr0</name>
+     <type>Constant</type>
+     <value>0.332</value>
+   </parameter>
+   <parameter>
+     <name>pressure_ic</name>
+     <type>Constant</type>
+     <value>0</value>
+   </parameter>
+   <parameter>
+     <name>pressure_bc_scale</name>
+     <type>Constant</type>
+     <value>1000000</value>
+   </parameter>
+   <parameter>
+     <name>pressure_bc</name>
+     <type>CurveScaled</type>
+     <curve>pressure_ramp_unsat_parity</curve>
+     <parameter>pressure_bc_scale</parameter>
+   </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>

```

```

+ <components>1</components>
+ <order>1</order>
+ <initial_condition>pressure_ic</initial_condition>
+ <boundary_conditions>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_left</mesh>
+ <type>Dirichlet</type>
+ <parameter>pressure_bc</parameter>
+ </boundary_condition>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_right</mesh>
+ <type>Dirichlet</type>
+ <parameter>pressure_bc</parameter>
+ </boundary_condition>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_bottom</mesh>
+ <type>Dirichlet</type>
+ <parameter>pressure_bc</parameter>
+ </boundary_condition>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0_top</mesh>
+ <type>Dirichlet</type>
+ <parameter>pressure_bc</parameter>
+ </boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+ <nonlinear_solver>
+ <name>basic_newton</name>
+ <type>Newton</type>
+ <max_iter>60</max_iter>
+ <linear_solver>general_linear_solver</linear_solver>
+ </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+ <linear_solver>
+ <name>general_linear_solver</name>
+ <eigen>
+ <solver_type>SparseLU</solver_type>
+ <scaling>true</scaling>
+ </eigen>
+ </linear_solver>
+ </linear_solvers>
+ <curves>
+ <curve>
+ <name>pressure_ramp_unsat_parity</name>
+ <coords>0 1 2 3 4</coords>
+ <values>0 -0.25 -0.5 -0.75 -1</values>
+ </curve>
+ </curves>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_bridge.prj b/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_bridge.prj
new file mode 100644
index 0000000000..97daec65af
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_bridge.prj
@@ -0,0 +1,510 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+ <meshes>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+ </meshes>
+ <processes>
+ <process>
+ <name>RM</name>
+ <type>RICHARDS_MECHANICS</type>
+ <integration_order>2</integration_order>
+ <jacobian_assembler>
+ <type>Analytical</type>
+ </jacobian_assembler>
+ <constitutive_relation>
+ <type>MFrontRichardsMechanics</type>
+ <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="YoungModulus"/>
+ <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+ <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+ <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+ <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+ <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+ <material_property name="BubblePressure" parameter="BubblePressure"/>
+ <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+ <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode"/>
+ <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode"/>
+ <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention"/>
+ <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope"/>
+ <material_property name="MassExchangeCoefficient" parameter="MicroMacroPotentialExchangeCoefficient"/>
+ <material_property name="MacroViscosity" parameter="MacroPotentialViscosity"/>
+ <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro"/>
+ <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro"/>
+ <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid"/>
+ <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA"/>
+ <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB"/>
+ <material_property name="HamakerConstant" parameter="HamakerConstant"/>
+ <material_property name="SpecificSurface" parameter="SpecificSurface"/>
+ <material_property name="AreaFactorTuller" parameter="AreaFactorTuller"/>
+ <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller"/>
+ <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize"/>
+ <material_property name="SurfaceTension" parameter="SurfaceTension"/>
+ <material_property name="InitialPorosity" parameter="phi0"/>
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+ <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+ <state_variable name="n_l" parameter="n_l0"/>
+ <state_variable name="rho_LR" parameter="rho_LR0"/>
+ <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+ </initial_values>
+ </constitutive_relation>
+ </process>
+ </processes>
+ </process_variables>

```

```

+       <pressure>pressure</pressure>
+       <displacement>displacement</displacement>
+     </process_variables>
+     <secondary_variables>
+       <secondary_variable name="ElasticStrain"/>
+       <secondary_variable name="EquivalentPlasticStrain"/>
+       <secondary_variable name="PreConsolidationPressure"/>
+       <secondary_variable name="PlasticVolumetricStrain"/>
+       <secondary_variable name="sigma"/>
+       <secondary_variable name="swelling_stress"/>
+       <secondary_variable name="epsilon"/>
+       <secondary_variable name="velocity"/>
+       <secondary_variable name="saturation"/>
+       <secondary_variable name="VolumeRatio"/>
+       <secondary_variable name="transport_porosity"/>
+       <secondary_variable name="dry_density_solid"/>
+       <secondary_variable name="n_l"/>
+       <secondary_variable name="rho_LR"/>
+       <secondary_variable name="epsilon_sw"/>
+       <secondary_variable name="phi_m"/>
+       <secondary_variable name="phi_M"/>
+       <secondary_variable name="mu_LR"/>
+       <secondary_variable name="rho_hat"/>
+     </secondary_variables>
+     <specific_body_force>0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+   </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2780.0</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>1e-12</value>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.01</exponent>
+         <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.4</exponent>
+         <p_b>1e4</p_b>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+       </property>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e0 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>

```

```

+         </time_discretization>
+         <time_stepping>
+             <type>IterationNumberBasedTimeStepping</type>
+             <t_initial>0</t_initial>
+             <t_end>1000</t_end>
+             <initial_dt>1000</initial_dt>
+             <minimum_dt>100</minimum_dt>
+             <maximum_dt>1000</maximum_dt>
+             <number_iterations>1 5 10 15</number_iterations>
+             <multiplier>1.2 1.0 0.8 0.6</multiplier>
+         </time_stepping>
+     </process>
+ </processes>
+ <output>
+     <type>VTK</type>
+     <prefix>mfront_restart_part1_dsm_micromacro_mcc_bridge</prefix>
+     <suffix>_ts_{: timestep}_t_{: time}</suffix>
+     <timesteps>
+         <pair>
+             <repeat>1</repeat>
+             <each_steps>1</each_steps>
+         </pair>
+     </timesteps>
+     <variables>
+     </variables>
+ </output>
+ </time_loop>
+ <parameters>
+     <!--Modified Can clay parameters-->
+     <parameter>
+         <name>sigma0</name>
+         <type>Function</type>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>-5e3</expression>
+         <expression>0</expression>
+     </parameter>
+     <parameter>
+         <name>YoungModulus</name>
+         <type>Constant</type>
+         <value>52e6</value><!--Pa-->
+     </parameter>
+     <parameter>
+         <name>PoissonRatio</name>
+         <type>Constant</type>
+         <value>0.3</value>
+     </parameter>
+     <parameter>
+         <name>CriticalStateLineSlope</name>
+         <type>Constant</type>
+         <value>1.2</value>
+     </parameter>
+     <parameter>
+         <name>SwellingLineSlope</name>
+         <type>Constant</type>
+         <value>6.6e-3</value>
+     </parameter>
+     <parameter>
+         <name>VirginConsolidationLineSlope</name>
+         <type>Constant</type>
+         <value>7.7e-2</value>
+     </parameter>
+     <parameter>
+         <name>InitialPreConsolidationPressure</name>
+         <type>Constant</type>
+         <value>2e5</value>
+     </parameter>
+     <parameter>
+         <name>InitialVolumeRatio</name>
+         <type>Constant</type>
+         <value>1.78571428571428571429</value>
+     </parameter>
+     <parameter>
+         <name>ResidualLiquidSaturation</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>ResidualGasSaturation</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>BubblePressure</name>
+         <type>Constant</type>
+         <value>1e4</value>
+     </parameter>
+     <parameter>
+         <name>VanGenuchtenExponent_m</name>
+         <type>Constant</type>
+         <value>0.4</value>
+     </parameter>
+     <parameter>
+         <name>MacroSaturationLawMode</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>MicroStateLocalSolveMode</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>MicroPotentialConvention</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>MicroSwellingStrainSlope</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>
+     <parameter>
+         <name>MicroMacroPotentialExchangeCoefficient</name>
+         <type>Constant</type>
+         <value>0.0</value>
+     </parameter>

```



```

+ <parameter>
+   <name>MacroPotentialViscosity</name>
+   <type>Constant</type>
+   <value>1000.0</value>
+ </parameter>
+ <parameter>
+   <name>ReferenceLiquidDensityMacro</name>
+   <type>Constant</type>
+   <value>1000.0</value>
+ </parameter>
+ <parameter>
+   <name>ReferenceLiquidDensityMicro</name>
+   <type>Constant</type>
+   <value>1300.0</value>
+ </parameter>
+ <parameter>
+   <name>ReferenceDensitySolid</name>
+   <type>Constant</type>
+   <value>2470.0</value>
+ </parameter>
+ <parameter>
+   <name>MicroLiquidDensityA</name>
+   <type>Constant</type>
+   <value>1.3</value>
+ </parameter>
+ <parameter>
+   <name>MicroLiquidDensityB</name>
+   <type>Constant</type>
+   <value>1.0</value>
+ </parameter>
+ <parameter>
+   <name>HamakerConstant</name>
+   <type>Constant</type>
+   <value>-6e-20</value>
+ </parameter>
+ <parameter>
+   <name>SpecificSurface</name>
+   <type>Constant</type>
+   <value>100.0</value>
+ </parameter>
+ <parameter>
+   <name>AreaFactorTuller</name>
+   <type>Constant</type>
+   <value>1.0</value>
+ </parameter>
+ <parameter>
+   <name>PoreAreaShapeFactorTuller</name>
+   <type>Constant</type>
+   <value>0.8584073464182069</value>
+ </parameter>
+ <parameter>
+   <name>CharacteristicPoreSize</name>
+   <type>Constant</type>
+   <value>1e-5</value>
+ </parameter>
+ <parameter>
+   <name>SurfaceTension</name>
+   <type>Constant</type>
+   <value>0.0715</value>
+ </parameter>
+ <parameter>
+   <name>n_l0</name>
+   <type>Constant</type>
+   <value>0.012869019712402708</value>
+ </parameter>
+ <parameter>
+   <name>rho_LR0</name>
+   <type>Constant</type>
+   <value>2267.4495975433856</value>
+ </parameter>
+ <parameter>
+   <name>epsilon_sw0</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <!-- Initial and boundary values -->
+ <parameter>
+   <name>displacement0</name>
+   <type>Constant</type>
+   <values>0 0</values>
+ </parameter>
+ <parameter>
+   <name>zero</name>
+   <type>Constant</type>
+   <value>0.0</value>
+ </parameter>
+ <parameter>
+   <name>phi0</name>
+   <type>Constant</type>
+   <value>0.432</value> <!-- TODO 0.44 -->
+ </parameter>
+ <parameter>
+   <name>top_pressure</name>
+   <type>Constant</type>
+   <value>-1e5</value>
+ </parameter>
+ <parameter>
+   <name>pressure_ic</name>
+   <type>Constant</type>
+   <value>-5e3</value>
+ </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <!-- fix left in radial direction -->
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <!-- fix bottom in axial direction -->

```

```

+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_bottom</mesh>
+             <type>Dirichlet</type>
+             <component>1</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         <!--compression in axial direction -->
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_top</mesh>
+             <type>Neumann</type>
+             <component>1</component>
+             <parameter>top_pressure</parameter>
+         </boundary_condition>
+         <!--compression in -radial direction-->
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_right</mesh>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+     </boundary_conditions>
+ </process_variable>
+ <process_variables>
+     <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+             <boundary_condition>
+                 <mesh>square_1x1_quad_1e0_top</mesh>
+                 <type>Dirichlet</type>
+                 <parameter>pressure_ic</parameter>
+             </boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>60</max_iter>
+         <damping>0.1</damping>
+         <damping_reduction>20</damping_reduction>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+             <solver_type>SparselU</solver_type>
+             <scaling>true</scaling>
+         </eigen>
+     </linear_solver>
+ </linear_solvers>
+ <test_definition>
+     <vtkdiff>
+         <regex>mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_*.vtu</regex>
+         <field>displacement</field>
+         <absolute_tolerance>2e-14</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_*.vtu</regex>
+         <field>sigma</field>
+         <absolute_tolerance>1e-9</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_*.vtu</regex>
+         <field>epsilon</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_*.vtu</regex>
+         <field>pressure</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>1e-15</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_*.vtu</regex>
+         <field>velocity</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>1e-15</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_*.vtu</regex>
+         <field>MassFlowRate</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_*.vtu</regex>
+         <field>NodalForces</field>
+         <absolute_tolerance>1e-10</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+ </test_definition>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_tuller_bridge.prj b/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_tuller_br
idge.prj
new file mode 100644
index 0000000000..0a7040f85c
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_tuller_bridge.prj
@@ -0,0 +1,461 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+     <meshes>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+     </meshes>
+ </processes>

```

```

+
+ <process>
+   <name>RM</name>
+   <type>RICHARDS_MECHANICS</type>
+   <integration_order>2</integration_order>
+   <jacobian_assembler>
+     <type>Analytical</type>
+   </jacobian_assembler>
+   <constitutive_relation>
+     <type>MFrontRichardsMechanics</type>
+     <behaviour>RichardsMechanicsDSMMicroMacroBridge_MCC</behaviour>
+     <material_properties>
+       <material_property name="YoungModulus" parameter="YoungModulus"/>
+       <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+       <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+       <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+       <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+       <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+       <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+       <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+       <material_property name="BubblePressure" parameter="BubblePressure"/>
+       <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+       <material_property name="MacroSaturationLawMode" parameter="MacroSaturationLawMode"/>
+       <material_property name="MicroStateLocalSolveMode" parameter="MicroStateLocalSolveMode"/>
+       <material_property name="MicroPotentialConvention" parameter="MicroPotentialConvention"/>
+       <material_property name="SwellingSlope" parameter="MicroSwellingStrainSlope"/>
+       <material_property name="MassExchangeCoefficient" parameter="MicroMacroPotentialExchangeCoefficient"/>
+       <material_property name="MacroViscosity" parameter="MacroPotentialViscosity"/>
+       <material_property name="ReferenceLiquidDensityMacro" parameter="ReferenceLiquidDensityMacro"/>
+       <material_property name="ReferenceLiquidDensityMicro" parameter="ReferenceLiquidDensityMicro"/>
+       <material_property name="ReferenceDensitySolid" parameter="ReferenceDensitySolid"/>
+       <material_property name="MicroLiquidDensityA" parameter="MicroLiquidDensityA"/>
+       <material_property name="MicroLiquidDensityB" parameter="MicroLiquidDensityB"/>
+       <material_property name="HamakerConstant" parameter="HamakerConstant"/>
+       <material_property name="SpecificSurface" parameter="SpecificSurface"/>
+       <material_property name="AreaFactorTuller" parameter="AreaFactorTuller"/>
+       <material_property name="PoreAreaShapeFactorTuller" parameter="PoreAreaShapeFactorTuller"/>
+       <material_property name="CharacteristicPoreSize" parameter="CharacteristicPoreSize"/>
+       <material_property name="SurfaceTension" parameter="SurfaceTension"/>
+       <material_property name="InitialPorosity" parameter="phi0"/>
+     </material_properties>
+     <initial_values>
+       <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+       <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+       <state_variable name="n_l" parameter="n_l0"/>
+       <state_variable name="rho_LR" parameter="rho_LR0"/>
+       <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+     </initial_values>
+   </constitutive_relation>
+   <process_variables>
+     <pressure>pressure</pressure>
+     <displacement>displacement</displacement>
+   </process_variables>
+   <secondary_variables>
+     <secondary_variable name="ElasticStrain"/>
+     <secondary_variable name="EquivalentPlasticStrain"/>
+     <secondary_variable name="PreConsolidationPressure"/>
+     <secondary_variable name="PlasticVolumetricStrain"/>
+     <secondary_variable name="sigma"/>
+     <secondary_variable name="swelling_stress"/>
+     <secondary_variable name="epsilon"/>
+     <secondary_variable name="velocity"/>
+     <secondary_variable name="saturation"/>
+     <secondary_variable name="VolumeRatio"/>
+     <secondary_variable name="transport_porosity"/>
+     <secondary_variable name="dry_density_solid"/>
+     <secondary_variable name="n_l"/>
+     <secondary_variable name="rho_LR"/>
+     <secondary_variable name="epsilon_sw"/>
+     <secondary_variable name="phi_m"/>
+     <secondary_variable name="phi_M"/>
+     <secondary_variable name="mu_LR"/>
+     <secondary_variable name="rho_l_hat"/>
+   </secondary_variables>
+   <specific_body_force>0</specific_body_force>
+   <initial_stress>sigma0</initial_stress>
+   <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+   <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2780.0</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>1e-12</value>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>

```

```

+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+     </property>
+ <property>
+     <name>reference_temperature</name>
+     <type>Constant</type>
+     <value>293.15</value>
+ </property>
+ <property>
+     <name>relative_permeability</name>
+     <type>RelativePermeabilityVanGenuchten</type>
+     <residual_liquid_saturation>0.0</residual_liquid_saturation>
+     <residual_gas_saturation>0</residual_gas_saturation>
+     <exponent>0.01</exponent>
+     <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+ </property>
+ <property>
+     <name>saturation</name>
+     <type>SaturationTuller</type>
+     <area_factor_tuller>1.0</area_factor_tuller>
+     <pore_area_shape_factor_tuller>0.8584073464102069</pore_area_shape_factor_tuller>
+     <characteristic_pore_size>1e-5</characteristic_pore_size>
+     <surface_tension>0.0715</surface_tension>
+ </property>
+ <property>
+     <name>bishops_effective_stress</name>
+     <type>BishopsPowerLaw</type>
+     <exponent>1</exponent>
+ </property>
+ <property>
+     <name>biot_coefficient</name>
+     <type>Constant</type>
+     <value>1.0</value>
+ </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e0 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>1000</t_end>
+         <timesteps>
+           <pair>
+             <repeat>1000</repeat>
+             <delta_t>1</delta_t>
+           </pair>
+         </timesteps>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>#front_restart_part1_dsm_micromacro_mcc_tuller_bridge</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+       <pair>
+         <repeat>1</repeat>
+         <each_steps>1000</each_steps>
+       </pair>
+     </timesteps>
+     <variables>
+       </variables>
+     </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>sigma0</name>
+     <type>Function</type>
+     <expression>-5e3</expression>
+     <expression>-5e3</expression>
+     <expression>-5e3</expression>
+     <expression>0</expression>
+   </parameter>
+   <parameter>
+     <name>YoungModulus</name>
+     <type>Constant</type>
+     <value>52e6</value>
+   </parameter>
+   <parameter>
+     <name>PoissonRatio</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>CriticalStateLineSlope</name>
+     <type>Constant</type>
+     <value>1.2</value>
+   </parameter>
+   <parameter>
+     <name>SwellingLineSlope</name>
+     <type>Constant</type>
+     <value>6.6e-3</value>
+   </parameter>
+   <parameter>
+     <name>VirginConsolidationLineSlope</name>
+     <type>Constant</type>
+     <value>7.7e-2</value>
+   </parameter>
+   <parameter>
+     <name>InitialPreConsolidationPressure</name>
+     <type>Constant</type>
+     <value>2e5</value>
+   </parameter>
+ </parameters>

```

```

+      <name>InitialVolumeRatio</name>
+      <type>Constant</type>
+      <value>1.78571428571428571429</value>
+    </parameter>
+    <parameter>
+      <name>ResidualLiquidSaturation</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>ResidualGasSaturation</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>BubblePressure</name>
+      <type>Constant</type>
+      <value>1e4</value>
+    </parameter>
+    <parameter>
+      <name>VanGenuchtenExponent_m</name>
+      <type>Constant</type>
+      <value>0.4</value>
+    </parameter>
+    <parameter>
+      <name>MacroSaturationLawMode</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroStateLocalSolveMode</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroPotentialConvention</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroSwellingStrainSlope</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroMacroPotentialExchangeCoefficient</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>MacroPotentialViscosity</name>
+      <type>Constant</type>
+      <value>1000.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMacro</name>
+      <type>Constant</type>
+      <value>1000.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceLiquidDensityMicro</name>
+      <type>Constant</type>
+      <value>1300.0</value>
+    </parameter>
+    <parameter>
+      <name>ReferenceDensitySolid</name>
+      <type>Constant</type>
+      <value>2470.0</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityA</name>
+      <type>Constant</type>
+      <value>1.3</value>
+    </parameter>
+    <parameter>
+      <name>MicroLiquidDensityB</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>HamakerConstant</name>
+      <type>Constant</type>
+      <value>-6e-20</value>
+    </parameter>
+    <parameter>
+      <name>SpecificSurface</name>
+      <type>Constant</type>
+      <value>100.0</value>
+    </parameter>
+    <parameter>
+      <name>AreaFactorTuller</name>
+      <type>Constant</type>
+      <value>1.0</value>
+    </parameter>
+    <parameter>
+      <name>PoreAreaShapeFactorTuller</name>
+      <type>Constant</type>
+      <value>0.8584073464102069</value>
+    </parameter>
+    <parameter>
+      <name>CharacteristicPoreSize</name>
+      <type>Constant</type>
+      <value>1e-5</value>
+    </parameter>
+    <parameter>
+      <name>SurfaceTension</name>
+      <type>Constant</type>
+      <value>0.0715</value>
+    </parameter>
+    <parameter>
+      <name>n_l0</name>
+      <type>Constant</type>
+      <value>0.012069019712402708</value>
+    </parameter>
+    <parameter>
+      <name>rho_LR0</name>
+      <type>Constant</type>

```

```

+         <value>2267.449597543856</value>
+     </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_left</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_bottom</mesh>
+         <type>Dirichlet</type>
+         <component>1</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Neumann</type>
+         <component>1</component>
+         <parameter>top_pressure</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_right</mesh>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>zero</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+   <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <mesh>square_1x1_quad_1e0_top</mesh>
+         <type>Dirichlet</type>
+         <parameter>pressure_ic</parameter>
+       </boundary_condition>
+     </boundary_conditions>
+   </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+   <nonlinear_solver>
+     <name>basic_newton</name>
+     <type>Newton</type>
+     <max_iter>60</max_iter>
+     <damping>0.1</damping>
+     <damping_reduction>20</damping_reduction>
+     <linear_solver>general_linear_solver</linear_solver>
+   </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+   <linear_solver>
+     <name>general_linear_solver</name>
+     <eigen>
+       <solver_type>SparseLU</solver_type>
+       <scaling>true</scaling>
+     </eigen>
+   </linear_solver>
+ </linear_solvers>
+</OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_tuller_native.prj b/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_tuller_native.prj
new file mode 100644
index 0000000000..87ab3ec0a7
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_restart_part1_dsm_micromacro_mcc_tuller_native.prj
@@ -0,0 +1,309 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
  <meshes>
    <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
    <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
  </meshes>
  <processes>

```

```

+
+ <process>
+   <name>RM</name>
+   <type>RICHARDS_MECHANICS</type>
+   <integration_order>2</integration_order>
+   <jacobian_assembler>
+     <type>Analytical</type>
+   </jacobian_assembler>
+   <constitutive_relation>
+     <type>MFront</type>
+     <behaviour>ModCamClay_semiExpl_constE</behaviour>
+     <material_properties>
+       <material_property name="YoungModulus" parameter="YoungModulus"/>
+       <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+       <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+       <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+       <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+       <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+     </material_properties>
+     <initial_values>
+       <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+       <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+     </initial_values>
+   </constitutive_relation>
+   <process_variables>
+     <pressure>pressure</pressure>
+     <displacement>displacement</displacement>
+   </process_variables>
+   <secondary_variables>
+     <secondary_variable name="ElasticStrain"/>
+     <secondary_variable name="EquivalentPlasticStrain"/>
+     <secondary_variable name="PreConsolidationPressure"/>
+     <secondary_variable name="PlasticVolumetricStrain"/>
+     <secondary_variable name="sigma"/>
+     <secondary_variable name="swelling_stress"/>
+     <secondary_variable name="epsilon"/>
+     <secondary_variable name="velocity"/>
+     <secondary_variable name="saturation"/>
+     <secondary_variable name="VolumeRatio"/>
+     <secondary_variable name="transport_porosity"/>
+     <secondary_variable name="dry_density_solid"/>
+   </secondary_variables>
+   <specific_body_force>0 0</specific_body_force>
+   <initial_stress>sigma0</initial_stress>
+   <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+   <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2700.0</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>1e-12</value>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.01</exponent>
+         <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationTuller</type>
+         <area_factor_tuller>1.0</area_factor_tuller>
+         <pore_area_shape_factor_tuller>0.8584073464102069</pore_area_shape_factor_tuller>
+         <characteristic_pore_size>1e-5</characteristic_pore_size>
+         <surface_tension>0.0715</surface_tension>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+       </property>
+       <property>
+         <name>biot_coefficient</name>

```

```

+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="PM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e0 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>FixedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>1000</t_end>
+         <timesteps>
+           <pair>
+             <repeat>1000</repeat>
+             <delta_t>1</delta_t>
+           </pair>
+         </timesteps>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>#front_restart_part1_dsm_micromacro_mcc_tuller_native</prefix>
+     <suffix>_ts_{:timestep}_t_{:time}</suffix>
+     <timesteps>
+       <pair>
+         <repeat>1</repeat>
+         <each_steps>1000</each_steps>
+       </pair>
+     </timesteps>
+     <variables>
+       </variables>
+     </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>sigma0</name>
+     <type>Function</type>
+     <expression>-5e3</expression>
+     <expression>-5e3</expression>
+     <expression>-5e3</expression>
+     <expression>0</expression>
+   </parameter>
+   <parameter>
+     <name>YoungModulus</name>
+     <type>Constant</type>
+     <value>52e6</value>
+   </parameter>
+   <parameter>
+     <name>PoissonRatio</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>CriticalStateLineSlope</name>
+     <type>Constant</type>
+     <value>1.2</value>
+   </parameter>
+   <parameter>
+     <name>SwellingLineSlope</name>
+     <type>Constant</type>
+     <value>6.6e-3</value>
+   </parameter>
+   <parameter>
+     <name>VirginConsolidationLineSlope</name>
+     <type>Constant</type>
+     <value>7.7e-2</value>
+   </parameter>
+   <parameter>
+     <name>InitialPreConsolidationPressure</name>
+     <type>Constant</type>
+     <value>2e5</value>
+   </parameter>
+   <parameter>
+     <name>InitialVolumeRatio</name>
+     <type>Constant</type>
+     <value>1.78571428571428571429</value>
+   </parameter>
+   <parameter>
+     <name>displacement0</name>
+     <type>Constant</type>
+     <values>0 0</values>
+   </parameter>
+   <parameter>
+     <name>zero</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>phi0</name>
+     <type>Constant</type>
+     <value>0.432</value>
+   </parameter>
+   <parameter>
+     <name>top_pressure</name>
+     <type>Constant</type>
+     <value>-1e5</value>
+   </parameter>
+   <parameter>
+     <name>pressure_ic</name>
+     <type>Constant</type>
+     <value>-5e3</value>
+   </parameter>
+ </parameters>
+ <process_variables>
+   <process_variable>

```



```

+         <name>displacement</name>
+         <components>2</components>
+         <order>1</order>
+         <initial_condition>displacement0</initial_condition>
+         <boundary_conditions>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_left</mesh>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_bottom</mesh>
+             <type>Dirichlet</type>
+             <component>1</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_top</mesh>
+             <type>Neumann</type>
+             <component>1</component>
+             <parameter>top_pressure</parameter>
+         </boundary_condition>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_right</mesh>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+     <process_variable>
+         <name>pressure</name>
+         <components>1</components>
+         <order>1</order>
+         <initial_condition>pressure_ic</initial_condition>
+         <boundary_conditions>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_top</mesh>
+             <type>Dirichlet</type>
+             <parameter>pressure_ic</parameter>
+         </boundary_condition>
+         </boundary_conditions>
+     </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>60</max_iter>
+         <damping>0.1</damping>
+         <damping_reduction>20</damping_reduction>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+             <solver_type>SparseLU</solver_type>
+             <scaling>true</scaling>
+         </eigen>
+     </linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/mfront_restart_part1_rm_bridge.prj b/Tests/Data/RichardsMechanics/mfront_restart_part1_rm_bridge.prj
new file mode 100644
index 0000000000..f226e15e68
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/mfront_restart_part1_rm_bridge.prj
@@ -0,0 +1,382 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<OpenGeoSysProject>
+     <meshes>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_left.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_right.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_top.vtu</mesh>
+         <mesh axially_symmetric="true">square_1x1_quad_1e0_bottom.vtu</mesh>
+     </meshes>
+     <processes>
+         <process>
+             <name>RM</name>
+             <type>RICHARDS_MECHANICS</type>
+             <integration_order>2</integration_order>
+             <jacobian_assembler>
+                 <type>Analytical</type>
+             </jacobian_assembler>
+             <constitutive_relation>
+                 <type>MFrontRichardsMechanics</type>
+                 <behaviour>ModCamClay_semiExpl_constE_pressureCoupled</behaviour>
+             </constitutive_relation>
+             <material_properties>
+                 <material_property name="YoungModulus" parameter="YoungModulus"/>
+                 <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+                 <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+                 <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+                 <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+                 <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+                 <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+                 <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+                 <material_property name="BubblePressure" parameter="BubblePressure"/>
+                 <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+             </material_properties>
+             <initial_values>
+                 <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+                 <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+             </initial_values>
+             </constitutive_relation>
+             <process_variables>
+                 <pressure>pressure</pressure>
+                 <displacement>displacement</displacement>
+             </process_variables>
+             <secondary_variables>
+                 <secondary_variable name="ElasticStrain"/>
+                 <secondary_variable name="EquivalentPlasticStrain"/>
+                 <secondary_variable name="PreConsolidationPressure"/>
+                 <secondary_variable name="PlasticVolumetricStrain"/>

```

```

+         <secondary_variable name="sigma"/>
+         <secondary_variable name="swelling_stress"/>
+         <secondary_variable name="epsilon"/>
+         <secondary_variable name="velocity"/>
+         <secondary_variable name="saturation"/>
+         <secondary_variable name="VolumeRatio"/>
+         <secondary_variable name="transport_porosity"/>
+         <secondary_variable name="dry_density_solid"/>
+     </secondary_variables>
+     <specific_body_force>0 0</specific_body_force>
+     <initial_stress>sigma0</initial_stress>
+     <explicit_hm_coupling_in_unsaturated_zone>false</explicit_hm_coupling_in_unsaturated_zone>
+     <mass_lumping>true</mass_lumping>
+ </process>
+ </processes>
+ <media>
+   <medium>
+     <phases>
+       <phase>
+         <type>AqueousLiquid</type>
+         <properties>
+           <property>
+             <name>viscosity</name>
+             <type>Constant</type>
+             <value>1e-3</value>
+           </property>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>1e3</value>
+           </property>
+         </properties>
+       </phase>
+       <phase>
+         <type>Solid</type>
+         <properties>
+           <property>
+             <name>density</name>
+             <type>Constant</type>
+             <value>2700.0</value>
+           </property>
+         </properties>
+       </phase>
+     </phases>
+     <properties>
+       <property>
+         <name>permeability</name>
+         <type>Constant</type>
+         <value>1e-12</value>
+       </property>
+       <property>
+         <name>porosity</name>
+         <type>PorosityFromMassBalance</type>
+         <initial_porosity>phi0</initial_porosity>
+         <minimal_porosity>0</minimal_porosity>
+         <maximal_porosity>1</maximal_porosity>
+       </property>
+       <property>
+         <name>reference_temperature</name>
+         <type>Constant</type>
+         <value>293.15</value>
+       </property>
+       <property>
+         <name>relative_permeability</name>
+         <type>RelativePermeabilityVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.01</exponent>
+         <minimum_relative_permeability_liquid>1e-2</minimum_relative_permeability_liquid>
+       </property>
+       <property>
+         <name>saturation</name>
+         <type>SaturationVanGenuchten</type>
+         <residual_liquid_saturation>0.0</residual_liquid_saturation>
+         <residual_gas_saturation>0</residual_gas_saturation>
+         <exponent>0.4</exponent>
+         <p_b>1e4</p_b>
+       </property>
+       <property>
+         <name>bishops_effective_stress</name>
+         <type>BishopsPowerLaw</type>
+         <exponent>1</exponent>
+       </property>
+       <property>
+         <name>biot_coefficient</name>
+         <type>Constant</type>
+         <value>1.0</value>
+       </property>
+     </properties>
+   </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="RM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-10 1e0 1e-9</reltols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>IterationNumberBasedTimeStepping</type>
+         <t_initial>0</t_initial>
+         <t_end>1000</t_end>
+         <initial_dt>1000</initial_dt>
+         <minimum_dt>100</minimum_dt>
+         <maximum_dt>1000</maximum_dt>
+         <number_iterations>1 5 10 15</number_iterations>
+         <multiplier>1.2 1.0 0.8 0.6</multiplier>
+       </time_stepping>
+     </process>
+   </processes>
+ </output>
+   <type>VTK</type>

```

```

+      <prefix>mfront_restart_part1_rm_bridges</prefix>
+      <suffix>_ts_{:timestep}_t_{:time}</suffix>
+      <timesteps>
+        <pair>
+          <repeat>1</repeat>
+          <each_steps>1</each_steps>
+        </pair>
+      </timesteps>
+      <variables>
+      </variables>
+    </output>
+  </time_loop>
+  <parameters>
+    <!-- Modified Cam clay parameters -->
+    <parameter>
+      <name>sigma0</name>
+      <type>Function</type>
+      <expression>-5e3</expression>
+      <expression>-5e3</expression>
+      <expression>-5e3</expression>
+      <expression>0</expression>
+    </parameter>
+    <parameter>
+      <name>YoungModulus</name>
+      <type>Constant</type>
+      <value>52e6</value><!-- Pa -->
+    </parameter>
+    <parameter>
+      <name>PoissonRatio</name>
+      <type>Constant</type>
+      <value>0.3</value>
+    </parameter>
+    <parameter>
+      <name>CriticalStateLineSlope</name>
+      <type>Constant</type>
+      <value>1.2</value>
+    </parameter>
+    <parameter>
+      <name>SwellingLineSlope</name>
+      <type>Constant</type>
+      <value>6.6e-3</value>
+    </parameter>
+    <parameter>
+      <name>VirginConsolidationLineSlope</name>
+      <type>Constant</type>
+      <value>7.7e-2</value>
+    </parameter>
+    <parameter>
+      <name>InitialPreConsolidationPressure</name>
+      <type>Constant</type>
+      <value>2e5</value>
+    </parameter>
+    <parameter>
+      <name>InitialVolumeRatio</name>
+      <type>Constant</type>
+      <value>1.78571428571428571429</value>
+    </parameter>
+    <parameter>
+      <name>ResidualLiquidSaturation</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>ResidualGasSaturation</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>BubblePressure</name>
+      <type>Constant</type>
+      <value>1e4</value>
+    </parameter>
+    <parameter>
+      <name>VanGenuchtenExponent_m</name>
+      <type>Constant</type>
+      <value>0.4</value>
+    </parameter>
+    <!-- Initial and boundary values -->
+    <parameter>
+      <name>displacement0</name>
+      <type>Constant</type>
+      <values>0 0</values>
+    </parameter>
+    <parameter>
+      <name>zero</name>
+      <type>Constant</type>
+      <value>0.0</value>
+    </parameter>
+    <parameter>
+      <name>phi0</name>
+      <type>Constant</type>
+      <value>0.432</value> <!-- TODD 0.44 -->
+    </parameter>
+    <parameter>
+      <name>top_pressure</name>
+      <type>Constant</type>
+      <value>-1e5</value>
+    </parameter>
+    <parameter>
+      <name>pressure_ic</name>
+      <type>Constant</type>
+      <value>-5e3</value>
+    </parameter>
+  </parameters>
+  <process_variables>
+    <process_variable>
+      <name>displacement</name>
+      <components>2</components>
+      <order>1</order>
+      <initial_condition>displacement0</initial_condition>
+      <boundary_conditions>
+        <!-- fix left in radial direction -->
+        <boundary_condition>
+          <mesh>square_1x1_quad_1e0_left</mesh>
+          <type>Dirichlet</type>
+          <component>0</component>
+        </boundary_condition>
+      </boundary_conditions>
+    </process_variable>
+  </process_variables>
+  <parameters>zero</parameters>

```

```

+         </boundary_condition>
+         <!--fix bottom in axial direction-->
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_bottom</mesh>
+             <type>Dirichlet</type>
+             <component>1</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+         <!--compression in axial direction -->
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_top</mesh>
+             <type>Neumann</type>
+             <component>1</component>
+             <parameter>top_pressure</parameter>
+         </boundary_condition>
+         <!--compression in -radial direction-->
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_right</mesh>
+             <type>Dirichlet</type>
+             <component>0</component>
+             <parameter>zero</parameter>
+         </boundary_condition>
+     </boundary_conditions>
+ </process_variable>
+ <process_variable>
+     <name>pressure</name>
+     <components>1</components>
+     <order>1</order>
+     <initial_condition>pressure_ic</initial_condition>
+     <boundary_conditions>
+         <boundary_condition>
+             <mesh>square_1x1_quad_1e0_top</mesh>
+             <type>Dirichlet</type>
+             <parameter>pressure_ic</parameter>
+         </boundary_condition>
+     </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+     <nonlinear_solver>
+         <name>basic_newton</name>
+         <type>Newton</type>
+         <max_iter>60</max_iter>
+         <damping>0.1</damping>
+         <damping_reduction>20</damping_reduction>
+         <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+     <linear_solver>
+         <name>general_linear_solver</name>
+         <eigen>
+             <solver_type>SparselU</solver_type>
+             <scaling>true</scaling>
+         </eigen>
+     </linear_solver>
+ </linear_solvers>
+ <test_definition>
+     <vtkdiff>
+         <regex>mfront_restart_part1_rm_bridge_ts_.*_t_.*.vtu</regex>
+         <field>displacement</field>
+         <absolute_tolerance>2e-14</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_rm_bridge_ts_.*_t_.*.vtu</regex>
+         <field>sigma</field>
+         <absolute_tolerance>1e-9</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_rm_bridge_ts_.*_t_.*.vtu</regex>
+         <field>epsilon</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_rm_bridge_ts_.*_t_.*.vtu</regex>
+         <field>pressure</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>1e-15</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_rm_bridge_ts_.*_t_.*.vtu</regex>
+         <field>velocity</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>1e-15</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_rm_bridge_ts_.*_t_.*.vtu</regex>
+         <field>MassFlowRate</field>
+         <absolute_tolerance>1e-15</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+     <vtkdiff>
+         <regex>mfront_restart_part1_rm_bridge_ts_.*_t_.*.vtu</regex>
+         <field>NodalForces</field>
+         <absolute_tolerance>1e-10</absolute_tolerance>
+         <relative_tolerance>0</relative_tolerance>
+     </vtkdiff>
+ </test_definition>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/RichardsMechanics/plot_beacon_report_figures.py b/Tests/Data/RichardsMechanics/plot_beacon_report_figures.py
new file mode 100644
index 0000000000..1655884c84
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/plot_beacon_report_figures.py
@@ -0,0 +1,748 @@
#!/usr/bin/env python3
"""Generate BEACON-style comparison figures for native and MFront runs.

+This helper script re-runs selected BEACON cases (1a01, 1b, 1c), extracts
+stress and density profile indicators from VTK outputs, and writes report-like
+plots plus CSV/JSON summary artifacts under ``Tests/Data/RichardsMechanics``.
+"""
+
+from __future__ import annotations

```

```

+
+import argparse
+import csv
+import json
+import os
+import subprocess
+import tempfile
+import xml.etree.ElementTree as ET
+from dataclasses import dataclass
+from pathlib import Path
+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+
+import matplotlib
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+
+HERE = Path(__file__).resolve().parent
+NATIVE_DEFAULT = Path("/Users/vinaykumar/git/build/release-native-beacon/bin/ogs")
+MFRONT_DEFAULT = Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs")
+
+
+CASE_CONFIGS = {
+    "1a01": {
+        "native_project": HERE / "beacon_1a01_inflow_unstructured_batch.prj",
+        "bridge_project": HERE / "beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch.prj",
+        "native_prefix": "beacon_1a01_native_reportplot",
+        "bridge_prefix": "beacon_1a01_bridge_reportplot",
+        "t_end_s": 1.0e5,
+        "output_times_s": [
+            10.0,
+            30.0,
+            100.0,
+            300.0,
+            1.0e3,
+            3.0e3,
+            1.0e4,
+            3.0e4,
+            5.0e4,
+            7.5e4,
+            1.0e5,
+        ],
+        "profile_bins": 4,
+        "profile_height_mm": [2.5, 7.5, 12.5, 17.5],
+        "report_profile_kg_m3": [1466.0, 1454.0, 1427.0, 1353.0],
+        "report_axial_kpa": 604.0,
+        "report_radial_kpa": 994.0,
+    },
+    "1b": {
+        "native_project": HERE / "beacon_1b_unstructured_batch.prj",
+        "bridge_project": HERE / "beacon_1b_dsm_micromacro_mcc_unstructured_batch.prj",
+        "native_prefix": "beacon_1b_native_reportplot",
+        "bridge_prefix": "beacon_1b_bridge_reportplot",
+        "t_end_s": 4.32e7,
+        "output_times_s": [
+            1.08e5,
+            5.40e5,
+            1.08e6,
+            2.16e6,
+            5.40e6,
+            1.08e7,
+            1.62e7,
+            2.16e7,
+            3.24e7,
+            4.32e7,
+        ],
+        "profile_bins": 10,
+        "report_density_mean_kg_m3": 1520.0,
+        "report_density_note": "nonzero and stabilized after about 500 days",
+    },
+    "1c": {
+        "bridge_project": HERE / "beacon_1c_dsm_micromacro_mcc_bridge.prj",
+        "bridge_prefix": "beacon_1c_bridge_reportplot",
+        "t_end_s": 1.0e3,
+        "output_times_s": [50.0, 100.0, 150.0, 250.0, 400.0, 550.0, 700.0, 850.0, 1.0e3],
+        "profile_bins": 20,
+        "block_height_m": 0.0485,
+    },
+}
+
+
+@dataclass
+class BoundarySeries:
+    label: str
+    times_s: np.ndarray
+    axial_kpa: np.ndarray
+    radial_kpa: np.ndarray
+
+
+@dataclass
+class ZoneStressSeries:
+    label: str
+    times_s: np.ndarray
+    block_axial_kpa: np.ndarray
+    block_radial_kpa: np.ndarray
+    pellet_axial_kpa: np.ndarray
+    pellet_radial_kpa: np.ndarray
+
+
+def write_project_copy(source: Path, target: Path, prefix: str, output_times_s: list[float]) -> None:
+    """Write a temporary project with absolute mesh/geometry and custom output prefix/times."""
+    target.parent.mkdir(parents=True, exist_ok=True)
+    tree = ET.parse(source)
+    root = tree.getroot()
+
+    mesh = root.find("./mesh")
+    geometry = root.find("./geometry")
+    if mesh is None or geometry is None or mesh.text is None or geometry.text is None:
+        raise RuntimeError(f"Could not resolve mesh/geometry in {source}")
+
+    mesh.text = str((source.parent / mesh.text).resolve())
+    geometry.text = str((source.parent / geometry.text).resolve())

```

```

+
+ output = root.find("./time_loop/output")
+ if output is None:
+     raise RuntimeError(f"No output block found in {source}")
+
+ prefix_node = output.find("prefix")
+ if prefix_node is None:
+     prefix_node = ET.SubElement(output, "prefix")
+ prefix_node.text = prefix
+
+ fixed_times = output.find("fixed_output_times")
+ if fixed_times is None:
+     fixed_times = ET.SubElement(output, "fixed_output_times")
+ fixed_times.text = " ".join(f"{t:.6f}" for t in output_times_s)
+
+ tree.write(target, encoding="ISO-8859-1", xml_declaration=True)
+
+
+def run_ogs(ogs: Path, project: Path, output_dir: Path) -> None:
+    """Execute one OGS project and raise with captured logs on failure."""
+    output_dir.mkdir(parents=True, exist_ok=True)
+    result = subprocess.run(
+        [str(ogs), "-o", str(output_dir), str(project)],
+        cwd=HERE,
+        text=True,
+        capture_output=True,
+        check=False,
+    )
+    if result.returncode != 0:
+        raise RuntimeError(
+            f"ogs failed for '{project.name}' with '{ogs}'.\n"
+            f"stdout:\n{result.stdout}\n"
+            f"stderr:\n{result.stderr}"
+        )
+
+
+def time_from_filename(path: Path) -> float:
+    marker = "_t_"
+    if marker not in path.stem:
+        raise RuntimeError(f"Could not parse time from {path.name}")
+    return float(path.stem.split(marker, 1)[1])
+
+
+def load_vtu(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    grid = reader.GetOutput()
+    points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+    point_data = {
+        grid.GetPointData().GetArrayName(i): vtk_to_numpy(grid.GetPointData().GetArray(i))
+        for i in range(grid.GetPointData().GetNumberOfArrays())
+    }
+    return points, point_data
+
+
+def boundary_mean(points: np.ndarray, values: np.ndarray, axis: int, side: str) -> np.ndarray:
+    target = points[:, axis].max() if side == "max" else points[:, axis].min()
+    mask = np.isclose(points[:, axis], target, atol=1e-10)
+    return np.asarray(values[mask]).mean(axis=0)
+
+
+def binned_profile(
+    points: np.ndarray,
+    values: np.ndarray,
+    *,
+    n_bins: int,
+    y_max_override: float | None = None,
+) -> list[dict[str, float]]:
+    y = points[:, 1]
+    ymin = float(y.min())
+    ymax = float(y.max()) if y_max_override is None else y_max_override
+    edges = np.linspace(ymin, ymax, n_bins + 1)
+    out: list[dict[str, float]] = []
+    for i in range(n_bins):
+        lo = edges[i]
+        hi = edges[i + 1]
+        if i == n_bins - 1:
+            mask = (y >= lo) & (y <= hi)
+        else:
+            mask = (y >= lo) & (y < hi)
+        out.append(
+            {
+                "center_mm": 1000.0 * 0.5 * (lo + hi),
+                "mean": float(np.asarray(values[mask]).mean()),
+                "count": int(mask.sum()),
+            }
+        )
+    return out
+
+
+def collect_boundary_series(run_dir: Path, prefix: str, label: str) -> BoundarySeries:
+    """Read all VTU files for one prefix and return top/outer boundary stress time series."""
+    paths = sorted(run_dir.glob(f"{prefix}_t_*.vtu"), key=time_from_filename)
+    if not paths:
+        raise RuntimeError(f"No VTU outputs found for {prefix} in {run_dir}")
+
+    times_s = []
+    axial_kpa = []
+    radial_kpa = []
+    for path in paths:
+        points, data = load_vtu(path)
+        sigma_top = boundary_mean(points, data["sigma"], axis=1, side="max")
+        sigma_outer = boundary_mean(points, data["sigma"], axis=0, side="max")
+        times_s.append(time_from_filename(path))
+        axial_kpa.append(abs(float(sigma_top[1])) / 1000.0)
+        radial_kpa.append(abs(float(sigma_outer[0])) / 1000.0)
+
+    order = np.argsort(np.asarray(times_s))
+    return BoundarySeries(
+        label=label,
+        times_s=np.asarray(times_s)[order],
+        axial_kpa=np.asarray(axial_kpa)[order],
+        radial_kpa=np.asarray(radial_kpa)[order],
+    )

```

```

+def collect_density_profile(run_dir: Path, prefix: str, n_bins: int) -> list[dict[str, float]]:
+    paths = sorted(run_dir.glob(f"{prefix}_t_*.vtu"), key=time_from_filename)
+    if not paths:
+        raise RuntimeError(f"No VTU outputs found for {prefix} in {run_dir}")
+    points, data = load_vtu(paths[-1])
+    return binned_profile(points, data["dry_density_solid"], n_bins=n_bins)
+
+
+def collect_zone_stress_series(run_dir: Path, prefix: str, label: str, block_height_m: float) -> ZoneStressSeries:
+    paths = sorted(run_dir.glob(f"{prefix}_t_*.vtu"), key=time_from_filename)
+    if not paths:
+        raise RuntimeError(f"No VTU outputs found for {prefix} in {run_dir}")
+
+    times_s = []
+    block_axial_kpa = []
+    block_radial_kpa = []
+    pellet_axial_kpa = []
+    pellet_radial_kpa = []
+
+    for path in paths:
+        points, data = load_vtu(path)
+        sigma = np.asarray(data["sigma"])
+        y = points[:, 1]
+        block_mask = y <= (block_height_m + 1e-10)
+        pellet_mask = y >= (block_height_m - 1e-10)
+
+        times_s.append(time_from_filename(path))
+        block_axial_kpa.append(abs(float(np.asarray(sigma[block_mask, 1]).mean())) / 1000.0)
+        block_radial_kpa.append(abs(float(np.asarray(sigma[block_mask, 0]).mean())) / 1000.0)
+        pellet_axial_kpa.append(abs(float(np.asarray(sigma[pellet_mask, 1]).mean())) / 1000.0)
+        pellet_radial_kpa.append(abs(float(np.asarray(sigma[pellet_mask, 0]).mean())) / 1000.0)
+
+    order = np.argsort(np.asarray(times_s))
+    return ZoneStressSeries(
+        label=label,
+        times_s=np.asarray(times_s)[order],
+        block_axial_kpa=np.asarray(block_axial_kpa)[order],
+        block_radial_kpa=np.asarray(block_radial_kpa)[order],
+        pellet_axial_kpa=np.asarray(pellet_axial_kpa)[order],
+        pellet_radial_kpa=np.asarray(pellet_radial_kpa)[order],
+    )
+
+
+def collect_multi_profile(
+    run_dir: Path,
+    prefix: str,
+    n_bins: int,
+) -> list[dict[str, float]]:
+    paths = sorted(run_dir.glob(f"{prefix}_t_*.vtu"), key=time_from_filename)
+    if not paths:
+        raise RuntimeError(f"No VTU outputs found for {prefix} in {run_dir}")
+    points, data = load_vtu(paths[-1])
+    porosity_profile = binned_profile(points, data["porosity"], n_bins=n_bins)
+    transport_profile = binned_profile(points, data["transport_porosity"], n_bins=n_bins)
+    saturation_profile = binned_profile(points, data["saturation"], n_bins=n_bins)
+
+    rows = []
+    for i in range(n_bins):
+        rows.append(
+            {
+                "center_mm": porosity_profile[i]["center_mm"],
+                "porosity": porosity_profile[i]["mean"],
+                "transport_porosity": transport_profile[i]["mean"],
+                "saturation": saturation_profile[i]["mean"],
+            }
+        )
+    return rows
+
+
+def collect_profile_by_unique_height(run_dir: Path, prefix: str) -> list[dict[str, float]]:
+    paths = sorted(run_dir.glob(f"{prefix}_t_*.vtu"), key=time_from_filename)
+    if not paths:
+        raise RuntimeError(f"No VTU outputs found for {prefix} in {run_dir}")
+    points, data = load_vtu(paths[-1])
+    y = np.round(points[:, 1], decimals=10)
+    rows = []
+    for height in np.unique(y):
+        mask = np.isclose(y, height, atol=1e-10)
+        rows.append(
+            {
+                "center_mm": float(1000.0 * height),
+                "porosity": float(np.asarray(data["porosity"][mask]).mean()),
+                "transport_porosity": float(np.asarray(data["transport_porosity"][mask]).mean()),
+                "saturation": float(np.asarray(data["saturation"][mask]).mean()),
+            }
+        )
+    return rows
+
+
+def write_boundary_series_csv(path: Path, series_list: list[BoundarySeries]) -> None:
+    with path.open("w", newline="") as f:
+        writer = csv.writer(f)
+        writer.writerow(["series", "time_s", "time_h", "time_d", "axial_kpa", "radial_kpa"])
+        for series in series_list:
+            for row in zip(
+                series.times_s,
+                series.times_s / 3600.0,
+                series.times_s / 86400.0,
+                series.axial_kpa,
+                series.radial_kpa,
+                strict=True,
+            ):
+                writer.writerow([series.label, *row])
+
+
+def write_density_profile_csv(path: Path, report_rows: list[dict[str, float]] | None, native_rows: list[dict[str, float]] | None, bridge_rows: list[dict[str, float]] | None) -> No
ne:
+    with path.open("w", newline="") as f:
+        writer = csv.writer(f)
+        writer.writerow(["height_mm", "report_density_kg_m3", "native_density_kg_m3", "bridge_density_kg_m3"])
+        n_rows = 0
+        for rows in (report_rows, native_rows, bridge_rows):
+            if rows is not None:
+                n_rows = max(n_rows, len(rows))
+        for i in range(n_rows):
+            height = None
+            if report_rows is not None and i < len(report_rows):

```

```

+         height = report_rows[i]["center_mm"]
+     elif native_rows is not None and i < len(native_rows):
+         height = native_rows[i]["center_mm"]
+     elif bridge_rows is not None and i < len(bridge_rows):
+         height = bridge_rows[i]["center_mm"]
+     writer.writerow(
+         [
+             height,
+             None if report_rows is None else report_rows[i]["mean"],
+             None if native_rows is None else native_rows[i]["mean"],
+             None if bridge_rows is None else bridge_rows[i]["mean"],
+         ]
+     )
+
+
+def write_zone_series_csv(path: Path, series: ZoneStressSeries) -> None:
+    with path.open("w", newline="") as f:
+        writer = csv.writer(f)
+        writer.writerow(
+            [
+                "Series",
+                "time_s",
+                "time_min",
+                "block_axial_kpa",
+                "block_radial_kpa",
+                "pellet_axial_kpa",
+                "pellet_radial_kpa",
+            ]
+        )
+        for row in zip(
+            series.times_s,
+            series.times_s / 60.0,
+            series.block_axial_kpa,
+            series.block_radial_kpa,
+            series.pellet_axial_kpa,
+            series.pellet_radial_kpa,
+            strict=True,
+        ):
+            writer.writerow([series.label, *row])
+
+
+def write_zone_profile_csv(path: Path, rows: List[dict[str, float]]) -> None:
+    with path.open("w", newline="") as f:
+        writer = csv.writer(f)
+        writer.writerow(["height_mm", "porosity", "transport_porosity", "saturation"])
+        for row in rows:
+            writer.writerow([row["center_mm"], row["porosity"], row["transport_porosity"], row["saturation"]])
+
+
+def plot_1a01_panel(
+    path: Path,
+    series_list: List[BoundarySeries],
+    native_profile: List[dict[str, float]],
+    bridge_profile: List[dict[str, float]],
+    config: dict,
+) -> None:
+    fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(11.8, 4.6))
+
+    colors = {"Native OGS": "#d62728", "MFront OGS": "#1f77b4"}
+    for series in series_list:
+        color = colors.get(series.label, "#333333")
+        ax0.plot(series.times_s / 3600.0, series.axial_kpa, color=color, linewidth=2.0, label=f"{series.label} axial")
+        ax0.plot(
+            series.times_s / 3600.0,
+            series.radial_kpa,
+            color=color,
+            linewidth=1.8,
+            linestyle="--",
+            label=f"{series.label} radial",
+        )
+
+    ax0.axhline(config["report_axial_kpa"], color="#666666", linewidth=1.5, linestyle="-. ", label="BEACON axial target")
+    ax0.axhline(config["report_radial_kpa"], color="#999999", linewidth=1.5, linestyle=":", label="BEACON radial target")
+    ax0.set_title("1a01 Stage-1 Swelling Pressure")
+    ax0.set_xlabel("Time (h)")
+    ax0.set_ylabel("Swelling pressure (kPa)")
+    ax0.set_xlim(left=0.0)
+    ax0.set_ylim(bottom=0.0)
+    ax0.grid(True, alpha=0.3)
+    ax0.legend(frameon=True, fontsize=8)
+
+    report_heights = np.asarray(config["profile_height_mm"])
+    report_density = np.asarray(config["report_profile_kg_m3"])
+    ax1.plot(report_density, report_heights, color="#444444", linewidth=2.0, marker="s", label="BEACON post-mortem")
+    ax1.plot(
+        [row["mean"] for row in native_profile],
+        [row["center_mm"] for row in native_profile],
+        color=colors["Native OGS"],
+        linewidth=1.9,
+        marker="o",
+        label="Native OGS stage-1",
+    )
+    ax1.plot(
+        [row["mean"] for row in bridge_profile],
+        [row["center_mm"] for row in bridge_profile],
+        color=colors["MFront OGS"],
+        linewidth=1.9,
+        marker="^",
+        label="MFront OGS stage-1",
+    )
+    ax1.set_title("1a01 Dry-Density Profile")
+    ax1.set_xlabel(r"Dry density (kg/m3)")
+    ax1.set_ylabel("Height from bottom (mm)")
+    ax1.set_ylim(0.0, 20.0)
+    ax1.grid(True, alpha=0.3)
+    ax1.legend(frameon=True, fontsize=8, loc="lower left")
+
+    fig.tight_layout()
+    fig.savefig(path, dpi=220)
+    plt.close(fig)
+
+
+def plot_1b_panel(
+    path: Path,
+    series_list: List[BoundarySeries],
+    native_profile: List[dict[str, float]],
+    bridge_profile: List[dict[str, float]],

```



```

+ config: dict,
+) -> None:
+ fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(11.8, 4.6))
+
+ colors = {"Native OG5": "#d62728", "MFront OG5": "#1f77b4"}
+ for series in series_list:
+     color = colors.get(series.label, "#333333")
+     ax0.plot(series.times_s / 86400.0, series.axial_kpa, color=color, linewidth=2.0, label=f"{series.label} axial")
+     ax0.plot(
+         series.times_s / 86400.0,
+         series.radial_kpa,
+         color=color,
+         linewidth=1.8,
+         linestyle="--",
+         label=f"{series.label} radial",
+     )
+
+ ax0.set_title("1b Swelling Pressure Evolution")
+ ax0.set_xlabel("Time (days)")
+ ax0.set_ylabel("Swelling pressure (kPa)")
+ ax0.set_xlim(left=0.0, right=max(series_list[0].times_s.max() / 86400.0, 500.0))
+ ax0.set_ylim(bottom=0.0)
+ ax0.grid(True, alpha=0.3)
+ ax0.text(
+     0.02,
+     0.96,
+     "BEACON note: nonzero and stabilized after about 500 days",
+     transform=ax0.transAxes,
+     va="top",
+     ha="left",
+     fontsize=8,
+     bbox={"facecolor": "white", "edgecolor": "#cccccc", "alpha": 0.85},
+ )
+ ax0.legend(frameon=True, fontsize=8)
+
+ ax1.plot(
+     [row["mean"] for row in native_profile],
+     [row["center_mm"] for row in native_profile],
+     color=colors["Native OG5"],
+     linewidth=1.9,
+     marker="o",
+     label="Native OG5",
+ )
+ ax1.plot(
+     [row["mean"] for row in bridge_profile],
+     [row["center_mm"] for row in bridge_profile],
+     color=colors["MFront OG5"],
+     linewidth=1.9,
+     marker="^",
+     label="MFront OG5",
+ )
+ ax1.axvline(
+     config["report_density_mean_kg_m3"],
+     color="#666666",
+     linewidth=1.6,
+     linestyle="-.",
+     label="BEACON mean dry density",
+ )
+ ax1.set_title("1b Dry-Density Profile")
+ ax1.set_xlabel(r"Dry density (kg/m3s)")
+ ax1.set_ylabel("Height from bottom (mm)")
+ ax1.set_ylim(0.0, 105.15)
+ ax1.grid(True, alpha=0.3)
+ ax1.legend(frameon=True, fontsize=8, loc="lower right")
+
+ fig.tight_layout()
+ fig.savefig(path, dpi=220)
+ plt.close(fig)
+
+
+ def plot_1c_panel(
+     path: Path,
+     zone_series: ZoneStressSeries,
+     profile_rows: List[dict[str, float]],
+     block_height_m: float,
+) -> None:
+     fig, (ax0, ax1) = plt.subplots(1, 2, figsize=(11.8, 4.6))
+
+     time_min = zone_series.times_s / 60.0
+     ax0.plot(time_min, zone_series.block_axial_kpa, color="#2ca02c", linewidth=2.0, label="Block axial")
+     ax0.plot(time_min, zone_series.block_radial_kpa, color="#2ca02c", linewidth=1.8, linestyle="--", label="Block radial")
+     ax0.plot(time_min, zone_series.pellet_axial_kpa, color="#ff7f0e", linewidth=2.0, label="Pellet axial")
+     ax0.plot(time_min, zone_series.pellet_radial_kpa, color="#ff7f0e", linewidth=1.8, linestyle="--", label="Pellet radial")
+     ax0.set_title("1c Zone-Mean Total Stress")
+     ax0.set_xlabel("Time (min)")
+     ax0.set_ylabel("Mean total stress magnitude (kPa)")
+     ax0.set_xlim(left=0.0)
+     ax0.set_ylim(bottom=0.0)
+     ax0.grid(True, alpha=0.3)
+     ax0.legend(frameon=True, fontsize=8)
+
+     heights = [row["center_mm"] for row in profile_rows]
+     ax1.plot([row["porosity"] for row in profile_rows], heights, color="#1f77b4", linewidth=2.0, label="Porosity")
+     ax1.plot(
+         [row["transport_porosity"] for row in profile_rows],
+         heights,
+         color="#d62728",
+         linewidth=1.8,
+         linestyle="--",
+         label="Transport porosity",
+     )
+     ax1.plot(
+         [row["saturation"] for row in profile_rows],
+         heights,
+         color="#9467bd",
+         linewidth=1.8,
+         linestyle="-.",
+         label="Saturation",
+     )
+     ax1.axhline(1000.0 * block_height_m, color="#666666", linewidth=1.4, linestyle=":", label="Block/pellet interface")
+     ax1.set_title("1c Final Vertical Profile")
+     ax1.set_xlabel("Field value (-)")
+     ax1.set_ylabel("Height from bottom (mm)")
+     ax1.set_ylim(0.0, 100.0)
+     ax1.grid(True, alpha=0.3)
+     ax1.legend(frameon=True, fontsize=8, loc="lower right")
+
+

```

```

+ fig.tight_layout()
+ fig.savefig(path, dpi=220)
+ plt.close(fig)
+
+
+def run_uniform_case(
+    case: str,
+    config: dict,
+    native_ogs: Path,
+    bridge_ogs: Path,
+    out_dir: Path,
+    run_cases: bool,
+) -> dict:
+    native_dir = out_dir / case / "native"
+    bridge_dir = out_dir / case / "bridge"
+    native_copy = out_dir / case / f"{case}_native_reportplot.prj"
+    bridge_copy = out_dir / case / f"{case}_bridge_reportplot.prj"
+
+    write_project_copy(config["native_project"], native_copy, config["native_prefix"], config["output_times_s"])
+    write_project_copy(config["bridge_project"], bridge_copy, config["bridge_prefix"], config["output_times_s"])
+
+    if run_cases:
+        run_ogs(native_ogs, native_copy, native_dir)
+        run_ogs(bridge_ogs, bridge_copy, bridge_dir)
+
+    native_series = collect_boundary_series(native_dir, config["native_prefix"], "Native OGS")
+    bridge_series = collect_boundary_series(bridge_dir, config["bridge_prefix"], "MFront OGS")
+    native_profile = collect_density_profile(native_dir, config["native_prefix"], config["profile_bins"])
+    bridge_profile = collect_density_profile(bridge_dir, config["bridge_prefix"], config["profile_bins"])
+
+    panel_path = out_dir / f"beacon_{case}_report_style_panel.png"
+    series_csv = out_dir / f"beacon_{case}_series.csv"
+    profile_csv = out_dir / f"beacon_{case}_profile.csv"
+
+    write_boundary_series_csv(series_csv, [native_series, bridge_series])
+    report_rows = None
+    if case == "1a01":
+        report_rows = [
+            {"center_mm": h, "mean": v}
+            for h, v in zip(config["profile_height_mm"], config["report_profile_kg_m3"], strict=True)
+        ]
+    plot_1a01_panel(panel_path, [native_series, bridge_series], native_profile, bridge_profile, config)
+    elif case == "1b":
+        plot_1b_panel(panel_path, [native_series, bridge_series], native_profile, bridge_profile, config)
+    write_density_profile_csv(profile_csv, report_rows, native_profile, bridge_profile)
+
+    summary = {
+        "case": case,
+        "panel": str(panel_path),
+        "series_csv": str(series_csv),
+        "profile_csv": str(profile_csv),
+        "native_final": {
+            "time_s": float(native_series.times_s[-1]),
+            "axial_kpa": float(native_series.axial_kpa[-1]),
+            "radial_kpa": float(native_series.radial_kpa[-1]),
+            "dry_density_mean_kg_m3": float(np.mean([row["mean"] for row in native_profile])),
+        },
+        "bridge_final": {
+            "time_s": float(bridge_series.times_s[-1]),
+            "axial_kpa": float(bridge_series.axial_kpa[-1]),
+            "radial_kpa": float(bridge_series.radial_kpa[-1]),
+            "dry_density_mean_kg_m3": float(np.mean([row["mean"] for row in bridge_profile])),
+        },
+    },
+    return summary
+
+def run_1c_case(
+    config: dict,
+    bridge_ogs: Path,
+    out_dir: Path,
+    run_cases: bool,
+) -> dict:
+    case = "1c"
+    bridge_dir = out_dir / case / "bridge"
+    bridge_copy = out_dir / case / "1c_bridge_reportplot.prj"
+
+    write_project_copy(config["bridge_project"], bridge_copy, config["bridge_prefix"], config["output_times_s"])
+
+    if run_cases:
+        run_ogs(bridge_ogs, bridge_copy, bridge_dir)
+
+    zone_series = collect_zone_stress_series(bridge_dir, config["bridge_prefix"], "MFront OGS", config["block_height_m"])
+    profile_rows = collect_profile_by_unique_height(bridge_dir, config["bridge_prefix"])
+
+    panel_path = out_dir / "beacon_1c_report_style_panel.png"
+    series_csv = out_dir / "beacon_1c_zone_series.csv"
+    profile_csv = out_dir / "beacon_1c_profile.csv"
+
+    plot_1c_panel(panel_path, zone_series, profile_rows, config["block_height_m"])
+    write_zone_series_csv(series_csv, zone_series)
+    write_zone_profile_csv(profile_csv, profile_rows)
+
+    summary = {
+        "case": case,
+        "panel": str(panel_path),
+        "series_csv": str(series_csv),
+        "profile_csv": str(profile_csv),
+        "bridge_final": {
+            "time_s": float(zone_series.times_s[-1]),
+            "block_axial_kpa": float(zone_series.block_axial_kpa[-1]),
+            "block_radial_kpa": float(zone_series.block_radial_kpa[-1]),
+            "pellet_axial_kpa": float(zone_series.pellet_axial_kpa[-1]),
+            "pellet_radial_kpa": float(zone_series.pellet_radial_kpa[-1]),
+        },
+    },
+    return summary
+
+def main() -> None:
+    parser = argparse.ArgumentParser()
+    parser.add_argument("--cases", nargs="+", choices=["1a01", "1b", "1c"], default=["1a01", "1b", "1c"])
+    parser.add_argument("--native-ogs", type=Path, default=NATIVE_DEFAULT)
+    parser.add_argument("--bridge-ogs", type=Path, default=MFRONT_DEFAULT)
+    parser.add_argument(
+        "--out-dir",
+        type=Path,

```

```

+     default=HERE / "BEACON_report_figures",
+ )
+ parser.add_argument("--skip-run", action="store_true")
+ args = parser.parse_args()
+
+ out_dir = args.out_dir.resolve()
+ out_dir.mkdir(parents=True, exist_ok=True)
+
+ summaries = []
+ for case in args.cases:
+     if case in ("1a01", "1b"):
+         summaries.append(
+             run_uniform_case(
+                 case,
+                 CASE_CONFIGS[case],
+                 args.native_ogs.resolve(),
+                 args.bridge_ogs.resolve(),
+                 out_dir,
+                 run_cases=not args.skip_run,
+             )
+         )
+     else:
+         summaries.append(
+             run_1c_case(
+                 CASE_CONFIGS["1c"],
+                 args.bridge_ogs.resolve(),
+                 out_dir,
+                 run_cases=not args.skip_run,
+             )
+         )
+
+ summary_path = out_dir / "beacon_report_plot_summary.json"
+ summary_path.write_text(json.dumps(summaries, indent=2) + "\n")
+ print(json.dumps({"summary": str(summary_path), "cases": summaries}, indent=2))
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/plot_calibrated_benchmark_comparison.py b/Tests/Data/RichardsMechanics/plot_calibrated_benchmark_comparison.py
new file mode 100644
index 0000000000..95822c90b8
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/plot_calibrated_benchmark_comparison.py
@@ -0,0 +1,167 @@
+#!/usr/bin/env python3
+"""Plot consolidated native-vs-MFront calibrated benchmark comparisons.
+
+Inputs:
+- calibrated_benchmark_pairwise_deltas.csv produced by
+  run_calibrated_beacon_anchors_comparison.py
+
+Outputs:
+- calibrated_beacon_native_mfront_comparison.png
+- calibrated_anchors_native_mfront_parity.png
+
+These figures are meant for direct insertion into the transition note.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+from pathlib import Path
+
+import matplotlib
+import numpy as np
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+
+OUTPUT_ROOT = Path(__file__).resolve().parent / "_outputs" / "calibrated_native_mfront_comparison"
+
+
+def parse_args() -> argparse.Namespace:
+    parser = argparse.ArgumentParser()
+    parser.add_argument(
+        "--pairwise-csv",
+        type=Path,
+        default=OUTPUT_ROOT / "calibrated_benchmark_pairwise_deltas.csv",
+    )
+    parser.add_argument(
+        "--beacon-png",
+        type=Path,
+        default=OUTPUT_ROOT / "calibrated_beacon_native_mfront_comparison.png",
+    )
+    parser.add_argument(
+        "--anchors-png",
+        type=Path,
+        default=OUTPUT_ROOT / "calibrated_anchors_native_mfront_parity.png",
+    )
+    return parser.parse_args()
+
+
+def read_rows(path: Path) -> list[dict[str, str]]:
+    with path.open(newline="", encoding="utf-8") as f:
+        return list(csv.DictReader(f))
+
+
+def to_float(value: str) -> float:
+    try:
+        return float(value)
+    except Exception:
+        return float("nan")
+
+
+def plot_beacon(rows: list[dict[str, str]], out_png: Path) -> None:
+    case_order = ["1a01", "1b"]
+    by_case = {r["case_id"]: r for r in rows if r["benchmark_group"] == "BEACON_report"}
+    cases = [c for c in case_order if c in by_case]
+    if not cases:
+        raise RuntimeError("No BEACON rows found in pairwise CSV.")
+
+    native_ps = np.array([to_float(by_case[c]["native_final_swelling_pressure_mpa"]) for c in cases])
+    mfront_ps = np.array([to_float(by_case[c]["mfront_final_swelling_pressure_mpa"]) for c in cases])
+    native_mult = np.array([to_float(by_case[c]["native_multiplier"]) for c in cases])
+    mfront_mult = np.array([to_float(by_case[c]["mfront_multiplier"]) for c in cases])
+    delta_ax = np.array([to_float(by_case[c]["delta_axial_sigma_kpa_native_minus_mfront"]) for c in cases])

```

```

+ delta_rad = np.array([to_float(by_case[c]["delta_radial_sigma_kpa_native_minus_mfront"]) for c in cases])
+
+ x = np.arange(len(cases))
+ w = 0.34
+
+ fig, axes = plt.subplots(2, 2, figsize=(11.5, 8.0))
+
+ ax = axes[0, 0]
+ ax.bar(x - w / 2, native_ps, width=w, label="Native", color="#1f77b4")
+ ax.bar(x + w / 2, mfront_ps, width=w, label="MFront", color="#d62728")
+ ax.set_xticks(x, cases)
+ ax.set_ylabel("Final swelling pressure (MPa)")
+ ax.set_title("BEACON calibrated replay: final swelling pressure")
+ ax.grid(axis="y", alpha=0.3)
+ ax.legend(frameon=False)
+
+ ax = axes[0, 1]
+ ax.bar(x - w / 2, native_mult, width=w, label="Native", color="#1f77b4")
+ ax.bar(x + w / 2, mfront_mult, width=w, label="MFront", color="#d62728")
+ ax.set_yscale("log")
+ ax.set_xticks(x, cases)
+ ax.set_ylabel("Effective vdw multiplier (-)")
+ ax.set_title("Calibrated multipliers (log scale)")
+ ax.grid(axis="y", which="both", alpha=0.3)
+
+ ax = axes[1, 0]
+ ax.axhline(0.0, color="black", linewidth=1.0)
+ ax.bar(x - w / 2, delta_ax, width=w, label=r"$\Delta \sigma_{ax}$", color="#9467bd")
+ ax.bar(x + w / 2, delta_rad, width=w, label=r"$\Delta \sigma_{rad}$", color="#2ca02c")
+ ax.set_xticks(x, cases)
+ ax.set_ylabel("Native - MFront stress (kPa)")
+ ax.set_title("Boundary stress deltas")
+ ax.grid(axis="y", alpha=0.3)
+ ax.legend(frameon=False)
+
+ ax = axes[1, 1]
+ ratio = np.array([to_float(by_case[c]["native_to_mfront_multiplier_ratio"]) for c in cases])
+ ax.plot(cases, ratio, marker="o", linestyle="-", color="#ff7f0e")
+ ax.axhline(1.0, color="black", linewidth=1.0, linestyle="--")
+ ax.set_ylabel("Native / MFront multiplier ratio")
+ ax.set_title("Multiplier ratio by BEACON case")
+ ax.grid(alpha=0.3)
+
+ fig.tight_layout()
+ out_png.parent.mkdir(parents=True, exist_ok=True)
+ fig.savefig(out_png, dpi=220)
+ plt.close(fig)
+
+
+def plot_anchors(rows: List[dict[str, str]], out_png: Path) -> None:
+ anchors = [r for r in rows if r["benchmark_group"] == "ANCHORS_MS33_ModelI"]
+ if not anchors:
+     raise RuntimeError("No ANCHORS rows found in pairwise CSV.")
+
+ anchors = sorted(anchors, key=lambda r: to_float(r["dry_density_kg_m3"]))
+ dd = np.array([to_float(r["dry_density_kg_m3"]) for r in anchors])
+ ratio = np.array([to_float(r["native_to_mfront_multiplier_ratio"]) for r in anchors])
+ delta_ps_kpa = np.array(
+     [1e3 * to_float(r["delta_swelling_pressure_mpa_native_minus_mfront"]) for r in anchors]
+ )
+
+ fig, axes = plt.subplots(2, 1, figsize=(9.5, 7.0), sharex=True)
+
+ ax = axes[0]
+ ax.plot(dd, ratio, marker="o", markersize=3.0, linewidth=1.5, color="#1f77b4")
+ ax.axhline(1.0, color="black", linewidth=1.0, linestyle="--")
+ ax.set_ylabel("Native / MFront multiplier ratio")
+ ax.set_title("ANCHORS dense calibration parity")
+ ax.grid(alpha=0.3)
+
+ ax = axes[1]
+ ax.axhline(0.0, color="black", linewidth=1.0)
+ ax.plot(dd, delta_ps_kpa, marker="s", markersize=3.6, linewidth=1.3, color="#d62728")
+ ax.set_xlabel(r"Dry density $\rho_{ds}$ (kg/m$^3$)")
+ ax.set_ylabel(r"$\Delta p_{cal}$ (kPa), native - MFront")
+ ax.grid(alpha=0.3)
+
+ fig.tight_layout()
+ out_png.parent.mkdir(parents=True, exist_ok=True)
+ fig.savefig(out_png, dpi=220)
+ plt.close(fig)
+
+
+def main() -> None:
+ args = parse_args()
+ rows = read_rows(args.pairwise_csv)
+ plot_beacon(rows, args.beacon_png)
+ plot_anchors(rows, args.anchors_png)
+ print(args.beacon_png)
+ print(args.anchors_png)
+
+
+if __name__ == "__main__":
+ main()
diff --git a/Tests/Data/RichardsMechanics/run_beacon_native_bridge_report_batch.py b/Tests/Data/RichardsMechanics/run_beacon_native_bridge_report_batch.py
new file mode 100755
index 0000000000..e59e0e91e7
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/run_beacon_native_bridge_report_batch.py
@@ -0,0 +1,110 @@
+#!/usr/bin/env python3
+"""Run the BEACON report benchmarks for native and bridge implementations.
+
+The script executes one paired case, captures the final VTU files, and
+delegates comparison metrics to 'analyze_beacon_unstructured_batch'.
+"""
+
+from __future__ import annotations
+
+import argparse
+import json
+import subprocess
+from pathlib import Path
+
+from analyze_beacon_unstructured_batch import summarise
+
+

```

```

+CASE_CONFIGS = {
+    "1a01": {
+        "native_project": "beacon_1a01_inflow_unstructured_batch.prj",
+        "bridge_project": "beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch.prj",
+        "native_output_prefix": "beacon_1a01_inflow_unstructured_batch",
+        "bridge_output_prefix": "beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch",
+        "final_time": "100000.000000",
+    },
+    "1b": {
+        "native_project": "beacon_1b_unstructured_batch.prj",
+        "bridge_project": "beacon_1b_dsm_micromacro_mcc_unstructured_batch.prj",
+        "native_output_prefix": "beacon_1b_unstructured_batch",
+        "bridge_output_prefix": "beacon_1b_dsm_micromacro_mcc_unstructured_batch",
+        "final_time": "43200000.000000",
+    },
+}
+
+def run_ogs(executable: Path, project: Path, output_dir: Path, source_dir: Path) -> None:
+    """Execute OGS with a project file and capture a useful failure report."""
+    output_dir.mkdir(parents=True, exist_ok=True)
+    result = subprocess.run(
+        [str(executable), "-o", str(output_dir), str(project)],
+        cwd=source_dir,
+        text=True,
+        capture_output=True,
+        check=False,
+    )
+    if result.returncode != 0:
+        raise RuntimeError(
+            f"ogs failed for '{project.name}' using '{executable}'.\n"
+            f"stdout:\n{result.stdout}\n"
+            f"stderr:\n{result.stderr}"
+        )
+
+def expected_output_file(output_dir: Path, prefix: str, final_time: str) -> Path:
+    """Resolve the VTU file that should exist at the end of a case run."""
+    path = output_dir / f"{prefix}_t{final_time}.vtu"
+    if not path.exists():
+        raise FileNotFoundError(f"Expected output file not found: {path}")
+    return path
+
+def main() -> None:
+    """Run one BEACON case and emit the summary JSON."""
+    parser = argparse.ArgumentParser()
+    parser.add_argument("--case", choices=sorted(CASE_CONFIGS), required=True)
+    parser.add_argument("--native-ogs", type=Path, required=True)
+    parser.add_argument("--bridge-ogs", type=Path, required=True)
+    parser.add_argument("--source-dir", type=Path, default=Path(__file__).resolve().parent)
+    parser.add_argument("--output-root", type=Path, required=True)
+    parser.add_argument("--json-out", type=Path)
+    args = parser.parse_args()
+
+    config = CASE_CONFIGS[args.case]
+    source_dir = args.source_dir.resolve()
+    native_project = source_dir / config["native_project"]
+    bridge_project = source_dir / config["bridge_project"]
+
+    native_output_dir = (args.output_root / args.case / "native").resolve()
+    bridge_output_dir = (args.output_root / args.case / "bridge").resolve()
+
+    run_ogs(args.native_ogs.resolve(), native_project, native_output_dir, source_dir)
+    run_ogs(args.bridge_ogs.resolve(), bridge_project, bridge_output_dir, source_dir)
+
+    native_vtu = expected_output_file(
+        native_output_dir, config["native_output_prefix"], config["final_time"]
+    )
+    bridge_vtu = expected_output_file(
+        bridge_output_dir, config["bridge_output_prefix"], config["final_time"]
+    )
+
+    result = summarise(args.case, native_vtu, bridge_vtu)
+    result["artifacts"] = {
+        "native_project": str(native_project),
+        "bridge_project": str(bridge_project),
+        "native_vtu": str(native_vtu),
+        "bridge_vtu": str(bridge_vtu),
+    }
+    result["executables"] = {
+        "native_ogs": str(args.native_ogs.resolve()),
+        "bridge_ogs": str(args.bridge_ogs.resolve()),
+    }
+
+    text = json.dumps(result, indent=2, sort_keys=True)
+    if args.json_out:
+        args.json_out.write_text(text + "\n")
+    print(text)
+
+if __name__ == "__main__":
+    main()
+
+diff --git a/Tests/Data/RichardsMechanics/run_calibrated_beacon_anchors_comparison.py b/Tests/Data/RichardsMechanics/run_calibrated_beacon_anchors_comparison.py
+new file mode 100644
+index 0000000000..ef1178d784
+--- /dev/null
++++ b/Tests/Data/RichardsMechanics/run_calibrated_beacon_anchors_comparison.py
+@@ -0,0 +1,868 @@
+#!/usr/bin/env python3
+"""Run the dry-density calibrated native-vs-MFront benchmark comparison.
+
++The script reuses the ANCHORS calibration curves, reruns BEACON with
++interpolated calibrated multipliers, and writes combined CSV/JSON summaries
++plus pairwise delta tables for the transition note.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import json
+import math
+import shutil
+import subprocess
+import tempfile
+import xml.etree.ElementTree as ET

```

```

+from dataclasses import dataclass
+from pathlib import Path
+
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+
+ROOT = Path(__file__).resolve().parent
+ANCHORS_DIR = ROOT / "ANCHORS_MS33_ModelI"
+MFRONT_CALIBRATION_CSV = ANCHORS_DIR / "villar_dense_dd_calibration.csv"
+NATIVE_CALIBRATION_CSV = ANCHORS_DIR / "villar_dense_dd_native_dsm_micromacro_calibration.csv"
+HAMAKER_REFERENCE_J = 5.1e-21
+OUTPUT_ROOT = ROOT / "_outputs" / "calibrated_native_mfront_comparison"
+
+
+@dataclass(frozen=True)
+class BeaconCase:
+    case_id: str
+    native_project: Path
+    mfront_project: Path
+
+
+BEACON_CASES = [
+    BeaconCase(
+        case_id="1a01",
+        native_project=ROOT / "beacon_1a01_inflow_unstructured_batch.prj",
+        mfront_project=ROOT / "beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch.prj",
+    ),
+    BeaconCase(
+        case_id="1b",
+        native_project=ROOT / "beacon_1b_unstructured_batch.prj",
+        mfront_project=ROOT / "beacon_1b_dsm_micromacro_mcc_unstructured_batch.prj",
+    ),
+]
+
+
+def parse_args() -> argparse.Namespace:
+    """Parse command-line arguments for the comparison workflow."""
+    parser = argparse.ArgumentParser(
+        description=(
+            "Run BEACON native-vs-Mfront comparison with dry-density calibrated "
+            "vdw multipliers, then merge with ANCHORS DD calibration rows."
+        )
+    )
+    parser.add_argument(
+        "--native-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-native-transition-mfront/bin/ogs"),
+    )
+    parser.add_argument(
+        "--mfront-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs"),
+    )
+    parser.add_argument(
+        "--out-csv",
+        type=Path,
+        default=OUTPUT_ROOT / "calibrated_benchmark_runs_summary.csv",
+    )
+    parser.add_argument(
+        "--out-json",
+        type=Path,
+        default=OUTPUT_ROOT / "calibrated_benchmark_runs_summary.json",
+    )
+    parser.add_argument(
+        "--out-deltas-csv",
+        type=Path,
+        default=OUTPUT_ROOT / "calibrated_benchmark_pairwise_deltas.csv",
+    )
+    parser.add_argument(
+        "--skip-beacon-runs",
+        action="store_true",
+        help="Only merge existing ANCHORS calibration rows.",
+    )
+    parser.add_argument(
+        "--beacon-curve-mode",
+        choices=["implementation_specific", "native", "mfront"],
+        default="implementation_specific",
+        help=(
+            "Curve used for BEACON multiplier interpolation. "
+            "'implementation_specific' keeps native and Mfront curves separate; "
+            "'native' or 'mfront' forces a common curve for both implementations."
+        )
+    )
+    parser.add_argument(
+        "--assert-thresholds",
+        action="store_true",
+        help="Fail with exit code 2 if equivalence thresholds are exceeded.",
+    )
+    parser.add_argument(
+        "--beacon-max-abs-delta-swelling-pressure-mpa",
+        type=float,
+        default=1e-4,
+        help="Absolute threshold for |native-mfront| final swelling pressure in BEACON rows.",
+    )
+    parser.add_argument(
+        "--beacon-max-abs-delta-axial-sigma-kpa",
+        type=float,
+        default=0.1,
+        help="Absolute threshold for |native-mfront| axial total stress in BEACON rows.",
+    )
+    parser.add_argument(
+        "--beacon-max-abs-delta-radial-sigma-kpa",
+        type=float,
+        default=0.02,
+        help="Absolute threshold for |native-mfront| radial total stress in BEACON rows.",
+    )
+    parser.add_argument(
+        "--anchors-max-abs-delta-swelling-pressure-mpa",
+        type=float,
+        default=2e-3,
+        help="Absolute threshold for |native-mfront| ANCHORS swelling-pressure deltas.",
+    )
+    parser.add_argument(
+        "--anchors-max-abs-multiplier-ratio-minus-one",
+        type=float,

```

```

+         default=2e-3,
+         help="Absolute threshold for |native/mfront multiplier ratio - 1| in ANCHORS rows.",
+     )
+     return parser.parse_args()
+
+
+def parse_csv_rows(path: Path) -> list[dict[str, str]]:
+    """Load a CSV file into a list of row dictionaries."""
+    with path.open(newLine="", encoding="utf-8") as f:
+        return list(csv.DictReader(f))
+
+
+def load_multiplier_curve(path: Path) -> tuple[np.ndarray, np.ndarray]:
+    """Load the dry-density calibration curve as sorted arrays."""
+    rows = parse_csv_rows(path)
+    dd = np.array([float(r["dry_density_kg_m3"]) for r in rows], dtype=float)
+    mult = np.array([float(r["vdm_multiplier"]) for r in rows], dtype=float)
+    order = np.argsort(dd)
+    return dd[order], mult[order]
+
+
+def interpolate_multiplier(dd: float, dd_curve: np.ndarray, mult_curve: np.ndarray) -> float:
+    """Interpolate the effective multiplier for one dry density."""
+    return float(np.interp(dd, dd_curve, mult_curve, left=mult_curve[0], right=mult_curve[-1]))
+
+
+def find_parameter_value(root: ET.Element, name: str) -> str | None:
+    node = root.find(f"./parameters/parameter[name='{name}']/value")
+    if node is None or node.text is None:
+        return None
+    return node.text.strip()
+
+
+def extract_dd_from_project(project: Path) -> float:
+    """Infer the dry density from a project file's solid density and phi0."""
+    root = ET.parse(project).getroot()
+    phi0_text = find_parameter_value(root, "phi0")
+    if phi0_text is None:
+        raise RuntimeError(f"Could not find parameter phi0 in {project}")
+    rho_text = root.findtext("./media/medium/phases/phase[type='Solid']/properties/property[name='density']/value")
+    if rho_text is None:
+        raise RuntimeError(f"Could not find solid density in {project}")
+    phi0 = float(phi0_text)
+    rho_s = float(rho_text.strip())
+    return rho_s * (1.0 - phi0)
+
+
+def absolutize_mesh_and_geometry(root: ET.Element, base_dir: Path) -> None:
+    """Resolve mesh and geometry paths relative to the source project."""
+    mesh = root.find("./mesh")
+    if mesh is not None and mesh.text:
+        mesh.text = str((base_dir / mesh.text.strip()).resolve())
+    geometry = root.find("./geometry")
+    if geometry is not None and geometry.text:
+        geometry.text = str((base_dir / geometry.text.strip()).resolve())
+
+
+def set_output_prefix(root: ET.Element, prefix: str) -> None:
+    """Rewrite the VTK output prefix in a copied project file."""
+    node = root.find("./time_loop/output/prefix")
+    if node is None:
+        raise RuntimeError("Could not find <time_loop><output><prefix> in project.")
+    node.text = prefix
+
+
+def set_biot_coefficient(root: ET.Element, value: float) -> None:
+    """Force all medium biot_coefficient properties to a consistent scalar."""
+    found = False
+    for prop in root.findall("./property"):
+        name = (prop.findtext("name") or "").strip()
+        if name != "biot_coefficient":
+            continue
+        val = prop.find("value")
+        if val is None:
+            val = ET.SubElement(prop, "value")
+        val.text = f"{value:.16g}"
+        found = True
+    if not found:
+        raise RuntimeError("Could not find any biot_coefficient property in project.")
+
+
+def set_nonlinear_max_iter(root: ET.Element, value: int) -> None:
+    """Set Newton max_iter in copied benchmark projects."""
+    nodes = root.findall("./nonlinear_solvers/nonlinear_solver/max_iter")
+    if not nodes:
+        raise RuntimeError("Could not find nonlinear solver max_iter in project.")
+    for node in nodes:
+        node.text = str(value)
+
+
+def set_time_stepping_value(root: ET.Element, key: str, value: float) -> None:
+    """Set one time-stepping scalar entry on copied benchmark projects."""
+    nodes = root.findall(f"./time_loop/processes/process/time_stepping/{key}")
+    if not nodes:
+        raise RuntimeError(f"Could not find time_stepping/{key} in project.")
+    for node in nodes:
+        node.text = f"{value:.16g}"
+
+
+def relax_first_component_abstol(root: ET.Element, value: float) -> None:
+    """Relax first PerComponentDeltaX absolute tolerance in copied decks."""
+    nodes = root.findall("./convergence_criterion/abstols")
+    if not nodes:
+        raise RuntimeError("Could not find convergence_criterion/abstols in project.")
+    for node in nodes:
+        if node.text is None:
+            continue
+        parts = node.text.strip().split()
+        if not parts:
+            continue
+        parts[0] = f"{value:.16g}"
+        node.text = " ".join(parts)
+
+
+def set_native_hamaker(root: ET.Element, hamaker_j: float) -> None:
+    """Inject the calibrated Hamaker constant into a native project copy."""
+    node = root.find("./processes/process/potential_exchange/hamaker_constant")

```

```

+     if node is None:
+         raise RuntimeError("Could not find potential_exchange/hamaker_constant in native project.")
+     node.text = f"{{hamaker_j:.16g}}"
+
+
+
+def set_mfront_hamaker(root: ET.Element, hamaker_j: float) -> None:
+    """Inject the calibrated Hamaker constant into an MFront project copy."""
+    node = root.find("./parameters/parameter[name='HamakerConstant']/value")
+    if node is None:
+        raise RuntimeError("Could not find parameter HamakerConstant in MFront project.")
+    node.text = f"{{hamaker_j:.16g}}"
+
+
+
+def write_project_copy(
+    source: Path,
+    target: Path,
+    prefix: str,
+    hamaker_j: float,
+    implementation: str,
+    case_id: str,
+) -> None:
+    """Clone a project and patch its output prefix and vdW parameter."""
+    root = ET.parse(source).getroot()
+    absolutize_mesh_and_geometry(root, source.parent)
+    set_output_prefix(root, prefix)
+    set_biot_coefficient(root, 1.0)
+    # Current-density constitutive updates in the dense benchmark shell can
+    # require more global Newton iterations on early inflow steps.
+    set_nonlinear_max_iter(root, 60)
+    relax_first_component_abstol(root, 5e-7)
+    if case_id == "1a01":
+        # The DD-calibrated Hamaker values make BEACON 1a01 startup stiff in
+        # both native and MFront DSM shells. A very small startup dt avoids
+        # the first-step jump into an infeasible local constitutive state.
+        set_nonlinear_max_iter(root, 80)
+        set_time_stepping_value(root, "initial_dt", 1e-8)
+        set_time_stepping_value(root, "minimum_dt", 1e-12)
+        set_time_stepping_value(root, "maximum_dt", 2e3)
+    if implementation == "native":
+        set_native_hamaker(root, hamaker_j)
+    elif implementation == "mfront":
+        set_mfront_hamaker(root, hamaker_j)
+    else:
+        raise ValueError(implementation)
+    target.parent.mkdir(parents=True, exist_ok=True)
+    ET.ElementTree(root).write(target, encoding="ISO-8859-1", xml_declaration=True)
+
+
+def run_ogs(ogs_bin: Path, project: Path, output_dir: Path) -> None:
+    """Run OGS for one temporary project and fail with captured logs."""
+    output_dir.mkdir(parents=True, exist_ok=True)
+    result = subprocess.run(
+        [str(ogs_bin), "-o", str(output_dir), str(project)],
+        cwd=ROOT,
+        text=True,
+        capture_output=True,
+        check=False,
+    )
+    if result.returncode != 0:
+        raise RuntimeError(
+            f"ogs failed for {project.name} with {ogs_bin}\n"
+            f"stdout:\n{result.stdout}\n"
+            f"stderr:\n{result.stderr}"
+        )
+
+
+def summarize_error(exc: Exception) -> str:
+    """Extract a compact, informative error line from a raised exception."""
+    text = str(exc)
+    lines = [ln.strip() for ln in text.splitlines() if ln.strip()]
+    # Prefer explicit OGS physics-failure lines.
+    for ln in reversed(lines):
+        low = ln.lower()
+        if "biot-coefficient" in low or ("porosity" in low and low.startswith("error:")):
+            return ln[:500]
+    # Then prefer explicit OGS error lines.
+    for ln in reversed(lines):
+        low = ln.lower()
+        if low.startswith("error:") or "terminated with error" in low:
+            return ln[:500]
+    # Fall back to any line mentioning stderr/stdout details.
+    for ln in reversed(lines):
+        if "stderr" in ln.lower() or "stdout" in ln.lower():
+            continue
+        return ln[:500]
+    return text.strip()[:500]
+
+
+def parse_output_time(path: Path) -> float:
+    marker = "_t_"
+    if marker not in path.stem:
+        return -math.inf
+    return float(path.stem.split(marker, 1)[1])
+
+
+def latest_vtu(output_dir: Path, prefix: str) -> Path:
+    files = sorted(output_dir.glob(f"{{prefix}}_t_*.vtu"), key=parse_output_time)
+    if not files:
+        raise FileNotFoundError(f"No VTU output found for prefix {prefix} in {output_dir}")
+    return files[-1]
+
+
+def load_vtu(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    grid = reader.GetOutput()
+    points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+    data: dict[str, np.ndarray] = {}
+    for i in range(grid.GetPointData().GetNumberOfArrays()):
+        arr = grid.GetPointData().GetArray(i)
+        name = grid.GetPointData().GetArrayName(i)
+        if arr is None or name is None:
+            continue
+        data[name] = vtk_to_numpy(arr)
+    return points, data

```



```

+
+def boundary_mean(points: np.ndarray, values: np.ndarray, axis: int, side: str) -> np.ndarray:
+    target = points[:, axis].max() if side == "max" else points[:, axis].min()
+    mask = np.isclose(points[:, axis], target, atol=1e-10)
+    return np.asarray(values[mask]).mean(axis=0)
+
+
+def value_component(arr: np.ndarray, index: int, default: float = math.nan) -> float:
+    flat = np.asarray(arr).reshape(-1)
+    if flat.size == 0:
+        return default
+    return float(flat[index] if index < flat.size else flat[0])
+
+
+def evaluate_beacon_metrics(vtu_path: Path) -> dict[str, float]:
+    """Extract the final stresses, displacements, and density statistics."""
+    points, data = load_vtu(vtu_path)
+    sigma = data.get("sigma")
+    swelling = data.get("swelling_stress")
+    disp = data.get("displacement")
+    dry_density = data.get("dry_density_solid")
+
+    axial_sigma_kpa = radial_sigma_kpa = math.nan
+    axial_swelling_kpa = radial_swelling_kpa = math.nan
+    mean_total_stress_mpa = math.nan
+
+    if sigma is not None:
+        sigma_top = boundary_mean(points, sigma, axis=1, side="max")
+        sigma_side = boundary_mean(points, sigma, axis=0, side="max")
+        axial_sigma_kpa = abs(value_component(sigma_top, 1)) / 1e3
+        radial_sigma_kpa = abs(value_component(sigma_side, 0)) / 1e3
+
+        sig = np.asarray(sigma)
+        if sig.ndim == 2 and sig.shape[1] >= 3:
+            p_mean = -(sig[:, 0] + sig[:, 1] + sig[:, 2]) / 3.0).mean()
+            mean_total_stress_mpa = float(p_mean / 1e6)
+
+    if swelling is not None:
+        swelling_top = boundary_mean(points, swelling, axis=1, side="max")
+        swelling_side = boundary_mean(points, swelling, axis=0, side="max")
+        axial_swelling_kpa = abs(value_component(swelling_top, 1)) / 1e3
+        radial_swelling_kpa = abs(value_component(swelling_side, 0)) / 1e3
+
+    axial_disp_mm = radial_disp_mm = math.nan
+
+    if disp is not None:
+        disp_top = boundary_mean(points, disp, axis=1, side="max")
+        disp_side = boundary_mean(points, disp, axis=0, side="max")
+        axial_disp_mm = value_component(disp_top, 1) * 1e3
+        radial_disp_mm = value_component(disp_side, 0) * 1e3
+
+    return {
+        "axial_sigma_kpa": axial_sigma_kpa,
+        "radial_sigma_kpa": radial_sigma_kpa,
+        "axial_swelling_stress_kpa": axial_swelling_kpa,
+        "radial_swelling_stress_kpa": radial_swelling_kpa,
+        "axial_displacement_mm": axial_disp_mm,
+        "radial_displacement_mm": radial_disp_mm,
+        "dry_density_mean_kg_m3": float(np.asarray(dry_density).mean()) if dry_density is not None else math.nan,
+        "final_total_stress_mpa": mean_total_stress_mpa,
+        "final_swelling_pressure_mpa": axial_sigma_kpa / 1e3 if not math.isnan(axial_sigma_kpa) else math.nan,
+    }
+
+
+def nan_beacon_metrics() -> dict[str, float]:
+    """Return placeholder metrics for a failed BEACON run."""
+    return {
+        "axial_sigma_kpa": math.nan,
+        "radial_sigma_kpa": math.nan,
+        "axial_swelling_stress_kpa": math.nan,
+        "radial_swelling_stress_kpa": math.nan,
+        "axial_displacement_mm": math.nan,
+        "radial_displacement_mm": math.nan,
+        "dry_density_mean_kg_m3": math.nan,
+        "final_total_stress_mpa": math.nan,
+        "final_swelling_pressure_mpa": math.nan,
+    }
+
+
+def anchors_rows_from_calibrations() -> list[dict[str, object]]:
+    """Convert the ANCHORS calibration CSVs into the common comparison schema."""
+    mfront_rows = parse_csv_rows(MFRONT_CALIBRATION_CSV)
+    native_rows = parse_csv_rows(NATIVE_CALIBRATION_CSV)
+    native_by_dd = {float(r["dry_density_kg_m3"]): r for r in native_rows}
+
+    out: list[dict[str, object]] = []
+    for mr in mfront_rows:
+        dd = float(mr["dry_density_kg_m3"])
+        nr = native_by_dd.get(dd)
+        if nr is None:
+            continue
+
+        mfront_multiplier = float(mr["vdw_multiplier"])
+        native_multiplier = float(nr["vdw_multiplier"])
+        mfront_ps = float(mr["mfront_calibrated_MPa"])
+        native_ps = float(nr["native_calibrated_MPa"])
+
+        out.append(
+            {
+                "benchmark_group": "ANCHORS_MS33_ModelI",
+                "case_id": f"dd{int(dd)}",
+                "implementation": "mfront",
+                "dry_density_kg_m3": dd,
+                "multiplier_used": mfront_multiplier,
+                "hamaker_constant_3": float(mr["hamaker_effective_3"]),
+                "final_total_stress_mpa": mfront_ps,
+                "final_swelling_pressure_mpa": mfront_ps,
+                "axial_sigma_kpa": mfront_ps * 1e3,
+                "radial_sigma_kpa": math.nan,
+                "axial_swelling_stress_kpa": math.nan,
+                "radial_swelling_stress_kpa": math.nan,
+                "axial_displacement_mm": math.nan,
+                "radial_displacement_mm": math.nan,
+                "dry_density_mean_kg_m3": dd,
+                "run_status": "success",
+                "source": MFRONT_CALIBRATION_CSV.name,
+            }
+        )
+    out.append(

```

```

+         {
+             "benchmark_group": "ANCHORS_MS33_ModelI",
+             "case_id": f"dd{int(dd)}",
+             "implementation": "native",
+             "dry_density_kg_m3": dd,
+             "multiplier_used": native_multiplier,
+             "hamaker_constant_j": float(nr["hamaker_effective_j"]),
+             "final_total_stress_mpa": native_ps,
+             "final_swelling_pressure_mpa": native_ps,
+             "axial_sigma_kpa": native_ps * 1e3,
+             "radial_sigma_kpa": math.nan,
+             "axial_swelling_stress_kpa": math.nan,
+             "radial_swelling_stress_kpa": math.nan,
+             "axial_displacement_mm": math.nan,
+             "radial_displacement_mm": math.nan,
+             "dry_density_mean_kg_m3": dd,
+             "run_status": "success",
+             "source": NATIVE_CALIBRATION_CSV.name,
+         }
+     )
+     return out
+
+
+def run_beacon_rows(
+    native_ogs: Path,
+    mfront_ogs: Path,
+    native_curve: tuple[np.ndarray, np.ndarray],
+    mfront_curve: tuple[np.ndarray, np.ndarray],
+    curve_mode: str = "implementation_specific",
+) -> list[dict[str, object]]:
+    """Execute BEACON cases with calibrated multipliers and collect metrics."""
+    if curve_mode == "native":
+        dd_native, mult_native = native_curve
+        dd_mfront, mult_mfront = native_curve
+    elif curve_mode == "mfront":
+        dd_native, mult_native = mfront_curve
+        dd_mfront, mult_mfront = mfront_curve
+    elif curve_mode == "implementation_specific":
+        dd_native, mult_native = native_curve
+        dd_mfront, mult_mfront = mfront_curve
+    else:
+        raise ValueError(f"Unsupported curve_mode '{curve_mode}'.")
+
+    rows: list[dict[str, object]] = []
+    tmpdir = Path(tempfile.mkdtemp(prefix="calibrated-beacon-runs-"))
+    try:
+        for case in BEACON_CASES:
+            dd_native_case = extract_dd_from_project(case.native_project)
+            dd_mfront_case = extract_dd_from_project(case.mfront_project)
+
+            native_multiplier = interpolate_multiplier(dd_native_case, dd_native, mult_native)
+            mfront_multiplier = interpolate_multiplier(dd_mfront_case, dd_mfront, mult_mfront)
+            native_hamaker = HAMAKER_REFERENCE_J * native_multiplier
+            mfront_hamaker = HAMAKER_REFERENCE_J * mfront_multiplier
+
+            native_prefix = f"beacon_{case.case_id}_native_ddcal"
+            mfront_prefix = f"beacon_{case.case_id}_mfront_ddcal"
+            native_project = tmpdir / f"{native_prefix}.prj"
+            mfront_project = tmpdir / f"{mfront_prefix}.prj"
+            native_out = tmpdir / f"out_native_{case.case_id}"
+            mfront_out = tmpdir / f"out_mfront_{case.case_id}"
+
+            write_project_copy(
+                source=case.native_project,
+                target=native_project,
+                prefix=native_prefix,
+                hamaker_j=native_hamaker,
+                implementation="native",
+                case_id=case.case_id,
+            )
+            write_project_copy(
+                source=case.mfront_project,
+                target=mfront_project,
+                prefix=mfront_prefix,
+                hamaker_j=mfront_hamaker,
+                implementation="mfront",
+                case_id=case.case_id,
+            )
+
+            native_status = "success"
+            native_error = ""
+            mfront_status = "success"
+            mfront_error = ""
+
+            try:
+                run_ogs(native_ogs, native_project, native_out)
+                native_metrics = evaluate_beacon_metrics(latest_vtu(native_out, native_prefix))
+            except Exception as exc:
+                native_status = "failed"
+                native_error = summarize_error(exc)
+                native_metrics = nan_beacon_metrics()
+
+            try:
+                run_ogs(mfront_ogs, mfront_project, mfront_out)
+                mfront_metrics = evaluate_beacon_metrics(latest_vtu(mfront_out, mfront_prefix))
+            except Exception as exc:
+                mfront_status = "failed"
+                mfront_error = summarize_error(exc)
+                mfront_metrics = nan_beacon_metrics()
+
+            rows.append(
+                {
+                    "benchmark_group": "BEACON_report",
+                    "case_id": case.case_id,
+                    "implementation": "native",
+                    "dry_density_kg_m3": dd_native_case,
+                    "multiplier_used": native_multiplier,
+                    "hamaker_constant_j": native_hamaker,
+                    "run_status": native_status,
+                    "error_message": native_error,
+                    "source": case.native_project.name,
+                    **native_metrics,
+                }
+            )
+            rows.append(
+                {
+                    "benchmark_group": "BEACON_report",

```

```

+         "case_id": case.case_id,
+         "implementation": "mfront",
+         "dry_density_kg_m3": dd_mfront_case,
+         "multiplier_used": mfront_multiplier,
+         "hamaker_constant_j": mfront_hamaker,
+         "run_status": mfront_status,
+         "error_message": mfront_error,
+         "source": case.mfront_project.name,
+         **mfront_metrics,
+     }
+ )
+ finally:
+     shutil.rmtree(tmpdir, ignore_errors=True)
+ return rows
+
+
+def pairwise_deltas(rows: list[dict[str, object]]) -> list[dict[str, object]]:
+    """Compute native-minus-Mfront deltas for each benchmark case."""
+    by_key: dict[tuple[str, str], dict[str, dict[str, object]]] = {}
+    for row in rows:
+        key = (str(row["benchmark_group"]), str(row["case_id"]))
+        impl = str(row["implementation"])
+        by_key.setdefault(key, {})[impl] = row
+
+    out: list[dict[str, object]] = []
+    for (group, case_id), impls in sorted(by_key.items()):
+        if "native" not in impls or "mfront" not in impls:
+            continue
+        n = impls["native"]
+        m = impls["mfront"]
+
+        def f(row: dict[str, object], key: str) -> float:
+            value = row.get(key, math.nan)
+            try:
+                return float(value)
+            except Exception:
+                return math.nan
+
+        out.append(
+            {
+                "benchmark_group": group,
+                "case_id": case_id,
+                "dry_density_kg_m3": f(n, "dry_density_kg_m3"),
+                "native_multiplier": f(n, "multiplier_used"),
+                "mfront_multiplier": f(m, "multiplier_used"),
+                "native_to_mfront_multiplier_ratio": (
+                    f(n, "multiplier_used") / f(m, "multiplier_used")
+                    if abs(f(m, "multiplier_used")) > 0.0
+                    else math.nan
+                ),
+                "native_final_swelling_pressure_mpa": f(n, "final_swelling_pressure_mpa"),
+                "mfront_final_swelling_pressure_mpa": f(m, "final_swelling_pressure_mpa"),
+                "delta_swelling_pressure_mpa_native_minus_mfront": (
+                    f(n, "final_swelling_pressure_mpa") - f(m, "final_swelling_pressure_mpa")
+                ),
+                "native_axial_sigma_kpa": f(n, "axial_sigma_kpa"),
+                "mfront_axial_sigma_kpa": f(m, "axial_sigma_kpa"),
+                "delta_axial_sigma_kpa_native_minus_mfront": (
+                    f(n, "axial_sigma_kpa") - f(m, "axial_sigma_kpa")
+                ),
+                "native_radial_sigma_kpa": f(n, "radial_sigma_kpa"),
+                "mfront_radial_sigma_kpa": f(m, "radial_sigma_kpa"),
+                "delta_radial_sigma_kpa_native_minus_mfront": (
+                    f(n, "radial_sigma_kpa") - f(m, "radial_sigma_kpa")
+                ),
+                "native_axial_displacement_mm": f(n, "axial_displacement_mm"),
+                "mfront_axial_displacement_mm": f(m, "axial_displacement_mm"),
+                "delta_axial_displacement_mm_native_minus_mfront": (
+                    f(n, "axial_displacement_mm") - f(m, "axial_displacement_mm")
+                ),
+                "native_radial_displacement_mm": f(n, "radial_displacement_mm"),
+                "mfront_radial_displacement_mm": f(m, "radial_displacement_mm"),
+                "delta_radial_displacement_mm_native_minus_mfront": (
+                    f(n, "radial_displacement_mm") - f(m, "radial_displacement_mm")
+                ),
+            }
+        )
+    return out
+
+def _as_float(value: object) -> float:
+    try:
+        return float(value)
+    except Exception:
+        return math.nan
+
+def _is_finite(value: float) -> bool:
+    return math.isfinite(value)
+
+def collect_threshold_failures(
+    deltas: list[dict[str, object]], args: argparse.Namespace
+) -> list[str]:
+    """Check configured equivalence thresholds and return violation messages."""
+    failures: list[str] = []
+    for row in deltas:
+        group = str(row.get("benchmark_group", ""))
+        case_id = str(row.get("case_id", ""))
+
+        d_psw = abs(_as_float(row.get("delta_swelling_pressure_mpa_native_minus_mfront")))
+        d_ax = abs(_as_float(row.get("delta_axial_sigma_kpa_native_minus_mfront")))
+        d_rad = abs(_as_float(row.get("delta_radial_sigma_kpa_native_minus_mfront")))
+        ratio_dev = abs(_as_float(row.get("native_to_mfront_multiplier_ratio"))) - 1.0
+
+        if group == "BEACON_report":
+            if (not _is_finite(d_psw)) or (
+                d_psw > args.beacon_max_abs_delta_swelling_pressure_mpa
+            ):
+                failures.append(
+                    f"[BEACON {case_id}] Δ|Psw|={d_psw:.6g} MPa > "
+                    f"[args.beacon_max_abs_delta_swelling_pressure_mpa:.6g] MPa"
+                )
+            if (not _is_finite(d_ax)) or (d_ax > args.beacon_max_abs_delta_axial_sigma_kpa):
+                failures.append(
+                    f"[BEACON {case_id}] Δ|sigma_ax|={d_ax:.6g} kPa > "
+                    f"[args.beacon_max_abs_delta_axial_sigma_kpa:.6g] kPa"
+                )

```

```

+         )
+         if (not _is_finite(d_rad)) or (d_rad > args.beacon_max_abs_delta_radial_sigma_kpa):
+             failures.append(
+                 f"[BEACON {case_id}] Δ[sigma_rad]={d_rad:.6g} kPa > "
+                 f"{args.beacon_max_abs_delta_radial_sigma_kpa:.6g} kPa"
+             )
+
+     if group == "ANCHORS_MS33_ModelI":
+         if (not _is_finite(d_psw)) or (
+             d_psw > args.anchors_max_abs_delta_swelling_pressure_mpa
+         ):
+             failures.append(
+                 f"[ANCHORS {case_id}] Δ[Psw]={d_psw:.6g} MPa > "
+                 f"{args.anchors_max_abs_delta_swelling_pressure_mpa:.6g} MPa"
+             )
+         if (not _is_finite(ratio_dev)) or (
+             ratio_dev > args.anchors_max_abs_multiplier_ratio_minus_one
+         ):
+             failures.append(
+                 f"[ANCHORS {case_id}] [native/mfront multiplier ratio - 1]= "
+                 f"{ratio_dev:.6g} > {args.anchors_max_abs_multiplier_ratio_minus_one:.6g}"
+             )
+     return failures
+
+
+def write_rows_csv(rows: list[dict[str, object]], path: Path) -> None:
+    """Write a list of heterogeneous row dictionaries as a CSV table."""
+    keys = sorted(k for row in rows for k in row.keys())
+    path.parent.mkdir(parents=True, exist_ok=True)
+    with path.open("w", newline="", encoding="utf-8") as f:
+        writer = csv.DictWriter(f, fieldnames=keys)
+        writer.writeheader()
+        writer.writerows(rows)
+
+
+def git_hash(repo: Path) -> str:
+    """Return a short Git hash for provenance tracking."""
+    try:
+        return (
+            subprocess.check_output(["git", "rev-parse", "--short", "HEAD"], cwd=repo, text=True)
+            .strip()
+        )
+    except Exception:
+        return ""
+
+
+def main() -> None:
+    """Run the full comparison workflow and write CSV/JSON artifacts."""
+    args = parse_args()
+
+    if not MFRONT_CALIBRATION_CSV.exists() or not NATIVE_CALIBRATION_CSV.exists():
+        raise FileNotFoundError(
+            "Missing calibration CSVs.\n"
+            f"Expected: {MFRONT_CALIBRATION_CSV}\n"
+            f"Expected: {NATIVE_CALIBRATION_CSV}"
+        )
+
+    rows = anchors_rows_from_calibrations()
+    native_curve = load_multiplier_curve(NATIVE_CALIBRATION_CSV)
+    mfront_curve = load_multiplier_curve(MFRONT_CALIBRATION_CSV)
+
+    if not args.skip_beacon_runs:
+        beacon_rows = run_beacon_rows(
+            native_ogs=args.native_ogs.resolve(),
+            mfront_ogs=args.mfront_ogs.resolve(),
+            native_curve=native_curve,
+            mfront_curve=mfront_curve,
+            curve_mode=args.beacon_curve_mode,
+        )
+        rows.extend(beacon_rows)
+
+    rows.sort(key=lambda r: (str(r["benchmark_group"]), str(r["case_id"]), str(r["implementation"])))
+    deltas = pairwise_deltas(rows)
+    deltas.sort(key=lambda r: (str(r["benchmark_group"]), str(r["case_id"])))
+
+    write_rows_csv(rows, args.out_csv)
+    write_rows_csv(deltas, args.out_deltas_csv)
+
+    threshold_failures: list[str] = []
+    if args.assert_thresholds:
+        threshold_failures = collect_threshold_failures(deltas, args)
+
+    summary = {
+        "reference_hamaker_j": HAMAKER_REFERENCE_J,
+        "row_count": len(rows),
+        "pairwise_delta_count": len(deltas),
+        "beacon_curve_mode": args.beacon_curve_mode,
+        "thresholds": {
+            "asserted": bool(args.assert_thresholds),
+            "beacon_max_abs_delta_swelling_pressure_mpa": args.beacon_max_abs_delta_swelling_pressure_mpa,
+            "beacon_max_abs_delta_axial_sigma_kpa": args.beacon_max_abs_delta_axial_sigma_kpa,
+            "beacon_max_abs_delta_radial_sigma_kpa": args.beacon_max_abs_delta_radial_sigma_kpa,
+            "anchors_max_abs_delta_swelling_pressure_mpa": args.anchors_max_abs_delta_swelling_pressure_mpa,
+            "anchors_max_abs_delta_multiplier_ratio_minus_one": args.anchors_max_abs_delta_multiplier_ratio_minus_one,
+        },
+        "threshold_failures": threshold_failures,
+        "sources": {
+            "anchors_mfront_calibration_csv": str(MFRONT_CALIBRATION_CSV),
+            "anchors_native_calibration_csv": str(NATIVE_CALIBRATION_CSV),
+            "beacon_cases": [case.case_id for case in BEACON_CASES],
+        },
+        "git_hashes": {
+            "ogs_repo_hash": git_hash(ROOT.parents[2]),
+            "materialmodels_repo_hash": git_hash(Path("/Users/vinaykumar/Documents/GitHub/materialmodels")),
+        },
+        "notes": [
+            (
+                "Dry-density calibrated multipliers are implementation-specific: "
+                "native uses villar_dense_dd_native_dsm_micromacro_calibration.csv and "
+                "MFront uses villar_dense_dd_calibration.csv."
+            ),
+            (
+                "BEACON dry density is inferred from rho_s * (1 - phi0) in each "
+                "project file; multiplier is linearly interpolated on D0 and "
+                "clamped to curve bounds."
+            ),
+        ],
+    }

```

```

+         "For ANCHORS, swelling pressure values are taken from calibrated DD runs;"
+         "for BEACON, values come from fresh reruns with calibrated Hamaker values."
+     ),
+     (
+         "Biot coefficient is enforced as 1.0 in generated BEACON run copies "
+         "for both native and MFront implementations."
+     ),
+     (
+         "BEACON curve mode can be switched to native or mfront to isolate "
+         "implementation drift from calibration-curve differences."
+     ),
+ ],
+ "rows": rows,
+ "pairwise_deltas": deltas,
+ }
+ args.out_json.parent.mkdir(parents=True, exist_ok=True)
+ args.out_json.write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")
+
+ print(f"Wrote rows CSV: {args.out_csv}")
+ print(f"Wrote summary JSON: {args.out_json}")
+ print(f"Wrote pairwise deltas CSV: {args.out_deltas_csv}")
+
+ if threshold_failures:
+     print("Equivalence threshold violations detected:")
+     for failure in threshold_failures:
+         print(f" - {failure}")
+     raise SystemExit(2)
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/run_identical_parameter_benchmark_parity.py b/Tests/Data/RichardsMechanics/run_identical_parameter_benchmark_parity.py
new file mode 100644
index 0000000000..b05a2dd478
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/run_identical_parameter_benchmark_parity.py
@@ -0,0 +1,573 @@
+#!/usr/bin/env python3
+"""Run native-vs-MFront parity with one identical shared parameter set.
+
+This workflow enforces a single shared vdW multiplier curve for BOTH
+implementations, then reruns:
+1) BEACON benchmark cases (1a01, 1b) via the calibrated comparison runner.
+2) ANCHORS dense dry-density points via the existing one-point calibration
+   drivers (single run per dry density, no fitting).
+3) EPFL/BGR WP3 AB' benchmark with MCC + dsm_micromacro micro-potential path
+   for both native and MFront implementations.
+
+Outputs:
+- rows CSV
+- pairwise-deltas CSV
+- summary JSON
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import importlib.util
+import json
+import math
+import shutil
+import subprocess
+import sys
+import tempfile
+import xml.etree.ElementTree as ET
+from pathlib import Path
+
+import numpy as np
+
+ROOT = Path(__file__).resolve().parent
+ANCHORS_DIR = ROOT / "ANCHORS_MS33_ModelII"
+EPFL_DIR = ROOT / "BEACON_WP3_BGR_EPFL"
+EPFL_NATIVE_PROJECT = EPFL_DIR / "bgr_wp3_p2_1_abprime_native.prj"
+EPFL_MFRONT_PROJECT = EPFL_DIR / "bgr_wp3_p2_1_abprime_mfront.prj"
+DEFAULT_SHARED_CURVE_CSV = ANCHORS_DIR / "villar_dense_dd_calibration.csv"
+OUTPUT_ROOT = ROOT / "_outputs" / "identical_parameter_native_mfront_comparison_with_epfl"
+
+def load_module(module_path: Path, module_name: str):
+    spec = importlib.util.spec_from_file_location(module_name, module_path)
+    if spec is None or spec.loader is None:
+        raise RuntimeError(f"Could not load module: {module_path}")
+    module = importlib.util.module_from_spec(spec)
+    sys.modules[module_name] = module
+    spec.loader.exec_module(module)
+    return module
+
+def parse_args() -> argparse.Namespace:
+    parser = argparse.ArgumentParser(
+        description=(
+            "Run BEACON+ANCHORS native-vs-MFront parity with identical "
+            "dry-density multiplier curve."
+        )
+    )
+    parser.add_argument(
+        "--native-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-native-transition2/bin/ogs"),
+    )
+    parser.add_argument(
+        "--mfront-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs"),
+    )
+    parser.add_argument(
+        "--shared-curve-csv",
+        type=Path,
+        default=DEFAULT_SHARED_CURVE_CSV,
+        help="CSV with columns dry_density_kg_m3, vdW_multiplier.",
+    )
+    parser.add_argument(
+        "--out-csv",
+        type=Path,
+        default=OUTPUT_ROOT / "identical_parameter_benchmark_runs_summary.csv",

```

```

+ )
+ parser.add_argument(
+     "--out-deltas-csv",
+     type=Path,
+     default=OUTPUT_ROOT / "identical_parameter_benchmark_pairwise_deltas.csv",
+ )
+ parser.add_argument(
+     "--out-json",
+     type=Path,
+     default=OUTPUT_ROOT / "identical_parameter_benchmark_runs_summary.json",
+ )
+ return parser.parse_args()
+
+
+def load_multiplier_curve(path: Path) -> tuple[np.ndarray, np.ndarray]:
+    with path.open(newLine="", encoding="utf-8") as f:
+        rows = list(csv.DictReader(f))
+        dd = np.array([float(r["dry_density_kg_m3"]) for r in rows], dtype=float)
+        mult = np.array([float(r["vdm_multiplier"]) for r in rows], dtype=float)
+        order = np.argsort(dd)
+        return dd[order], mult[order]
+
+
+def interpolate_multiplier(dd: float, dd_curve: np.ndarray, mult_curve: np.ndarray) -> float:
+    return float(np.interp(dd, dd_curve, mult_curve, left=mult_curve[0], right=mult_curve[-1]))
+
+
+def get_parameter_value(root: ET.Element, name: str, default: float | None = None) -> float:
+    node = root.find(f"./parameters/parameter[name='{name}']/value")
+    if node is None or node.text is None:
+        if default is None:
+            raise RuntimeError(f"Missing parameter '{name}' in project.")
+        return default
+    return float(node.text.strip())
+
+
+def _get_property_node(root: ET.Element, name: str) -> ET.Element | None:
+    return root.find(f"./media/medium/properties/property[name='{name}']")
+
+
+def _get_property_float(node: ET.Element, tag: str, default: float) -> float:
+    child = node.find(tag)
+    if child is None or child.text is None:
+        return default
+    return float(child.text.strip())
+
+
+def estimate_initial_micro_water_content(root: ET.Element) -> float:
+    """Estimate n_l0 from EPFL-style native inputs when n_l0 is not explicit."""
+    phi0 = get_parameter_value(root, "phi0")
+    phi_tr0 = get_parameter_value(root, "phi_tr0", default=0.0)
+    micro_porosity0 = max(1e-12, phi0 - phi_tr0)
+    p_l0 = get_parameter_value(root, "pressure_ic", default=0.0)
+    p_cap = max(0.0, -p_l0)
+
+    sat_prop = _get_property_node(root, "saturation_micro")
+    if sat_prop is None:
+        return 0.99 * micro_porosity0
+
+    sat_type = sat_prop.find("type")
+    sat_type_text = sat_type.text.strip() if sat_type is not None and sat_type.text else ""
+    if sat_type_text != "SaturationVanGenuchten":
+        return 0.99 * micro_porosity0
+
+    s_lr = _get_property_float(sat_prop, "residual_liquid_saturation", 0.0)
+    s_gr = _get_property_float(sat_prop, "residual_gas_saturation", 0.0)
+    m = _get_property_float(sat_prop, "exponent", 0.5)
+    p_b = _get_property_float(sat_prop, "p_b", 1.0)
+    if p_cap <= 0.0:
+        s_eff = 1.0
+    else:
+        m = min(max(m, 1e-12), 1.0 - 1e-12)
+        p_b = max(p_b, 1e-12)
+        n = 1.0 / (1.0 - m)
+        s_eff = (1.0 + (p_cap / p_b) ** n) ** (-m)
+
+    sat = s_lr + (1.0 - s_lr - s_gr) * s_eff
+    sat = min(max(sat, 0.0), 1.0)
+    return micro_porosity0 * sat
+
+
+def set_or_create(parent: ET.Element, tag: str, value: str) -> ET.Element:
+    node = parent.find(tag)
+    if node is None:
+        node = ET.SubElement(parent, tag)
+    node.text = value
+    return node
+
+
+def ensure_native_epfl_potential_exchange(
+    root: ET.Element,
+    hamaker_j: float,
+    mfront_parameter_defaults: dict[str, float] | None = None,
+) -> None:
+    """Inject a native potential_exchange block mapped from EPFL MFront params."""
+    process = root.find("./processes/process")
+    if process is None:
+        raise RuntimeError("Missing process block in EPFL native project.")
+
+    defaults = mfront_parameter_defaults or {}
+
+    # Pull shared constitutive constants from parameters to keep a single set.
+    phi0 = get_parameter_value(root, "phi0")
+    rho_sr = get_parameter_value(
+        root,
+        "ReferenceDensitySolid",
+        default=defaults.get("ReferenceDensitySolid", 2780.0),
+    )
+    specific_surface = get_parameter_value(
+        root,
+        "SpecificSurface",
+        default=defaults.get("SpecificSurface", 523.0),
+    )
+    try:
+        n_l0 = get_parameter_value(root, "n_l0")
+    except RuntimeError:
+        n_l0 = defaults.get("n_l0", estimate_initial_micro_water_content(root))

```

```

+ micro_swelling_slope = get_parameter_value(
+     root,
+     "MicroSwellingStrainSlope",
+     default=defaults.get("MicroSwellingStrainSlope", 0.0),
+ )
+ mass_exchange_coeff = get_parameter_value(
+     root,
+     "MassExchangeCoefficient",
+     default=defaults.get("MassExchangeCoefficient", 5e-15),
+ )
+
+ # Keep mass exchange coefficient explicitly aligned with the shared set.
+ micro_porosity = process.find("micro_porosity")
+ if micro_porosity is None:
+     micro_porosity = ET.SubElement(process, "micro_porosity")
+ set_or_create(
+     micro_porosity,
+     "mass_exchange_coefficient",
+     f"({mass_exchange_coeff:.16g})",
+ )
+
+ potential_exchange = process.find("potential_exchange")
+ if potential_exchange is None:
+     potential_exchange = ET.SubElement(process, "potential_exchange")
+
+ set_or_create(potential_exchange, "enabled", "true")
+ set_or_create(potential_exchange, "mode", "full_potential")
+ set_or_create(potential_exchange, "pressure_tolerance", "1e-12")
+ set_or_create(potential_exchange, "hamaker_constant", f"({hamaker_j:.16g})")
+ set_or_create(potential_exchange, "specific_surface", f"({specific_surface:.16g})")
+ set_or_create(
+     potential_exchange,
+     "micro_solid_density_reference",
+     f"({rho_sr:.16g})",
+ )
+ # Reference split for native exchange path; use total-solid fraction.
+ set_or_create(
+     potential_exchange,
+     "micro_solid_volume_fraction_reference",
+     f"({max(1e-12, 1.0 - phi0):.16g})",
+ )
+ set_or_create(
+     potential_exchange,
+     "initial_micro_water_content",
+     f"({max(1e-12, n_l0):.16g})",
+ )
+
+ set_or_create(
+     potential_exchange,
+     "local_nonlinear_solve_mode",
+     "scalar_microstate_storage_mode",
+ )
+ set_or_create(potential_exchange, "fd_jacobian_for_exchange", "false")
+ set_or_create(
+     potential_exchange,
+     "micro_potential_convention",
+     "negative_attractive",
+ )
+ set_or_create(
+     potential_exchange,
+     "micro_solid_volume_fraction_mode",
+     "reference",
+ )
+ set_or_create(
+     potential_exchange,
+     "micro_water_content_swelling_slope",
+     f"({max(0.0, micro_swelling_slope):.16g})",
+ )
+
+
+ def run_epfl_rows(
+     compare_mod,
+     native_ogs: Path,
+     mfront_ogs: Path,
+     dd_curve: np.ndarray,
+     mult_curve: np.ndarray,
+ ) -> list[dict[str, object]]:
+     """Run EPFL native and Mfront decks with one shared DO-based multiplier."""
+     if not EPFL_NATIVE_PROJECT.exists() or not EPFL_MFRONT_PROJECT.exists():
+         raise FileNotFoundError("EPFL benchmark projects are missing.")
+
+     dd_native = compare_mod.extract_dd_from_project(EPFL_NATIVE_PROJECT)
+     dd_mfront = compare_mod.extract_dd_from_project(EPFL_MFRONT_PROJECT)
+     dd_epfl = 0.5 * (dd_native + dd_mfront)
+     multiplier = interpolate_multiplier(dd_epfl, dd_curve, mult_curve)
+     hamaker_j = compare_mod.HAMAKER_REFERENCE_J * multiplier
+
+     tmpdir = Path(tempfile.mkdtemp(prefix="identical-epfl-runs-"))
+     try:
+         native_prefix = "epfl_abprime_native_identical"
+         mfront_prefix = "epfl_abprime_mfront_identical"
+
+         native_prj = tmpdir / f"({native_prefix}).prj"
+         mfront_prj = tmpdir / f"({mfront_prefix}).prj"
+         native_out = tmpdir / native_prefix
+         mfront_out = tmpdir / mfront_prefix
+
+         mfront_root = ET.parse(EPFL_MFRONT_PROJECT).getroot()
+         mfront_defaults = {
+             "ReferenceDensitySolid": get_parameter_value(
+                 mfront_root, "ReferenceDensitySolid", default=2780.0
+             ),
+             "SpecificSurface": get_parameter_value(
+                 mfront_root, "SpecificSurface", default=523.0
+             ),
+             "MicroSwellingStrainSlope": get_parameter_value(
+                 mfront_root, "MicroSwellingStrainSlope", default=0.0
+             ),
+             "MassExchangeCoefficient": get_parameter_value(
+                 mfront_root, "MassExchangeCoefficient", default=5e-15
+             ),
+             "n_l0": get_parameter_value(mfront_root, "n_l0", default=1e-12),
+         }
+
+         native_root = ET.parse(EPFL_NATIVE_PROJECT).getroot()
+         compare_mod.absolutize_mesh_and_geometry(native_root, EPFL_NATIVE_PROJECT.parent)
+         compare_mod.set_output_prefix(native_root, native_prefix)
+         compare_mod.set_biot_coefficient(native_root, 1.0)

```

```

+     compare_mod.set_nonlinear_max_iter(native_root, 80)
+     compare_mod.relax_first_component_abstol(native_root, 5e-7)
+     ensure_native_epfl_potential_exchange(
+         native_root, hamaker_j, mfront_parameter_defaults=mfront_defaults
+     )
+     ET.ElementTree(native_root).write(
+         native_prj, encoding="ISO-8859-1", xml_declaration=True
+     )
+
+     compare_mod.absolutize_mesh_and_geometry(mfront_root, EPFL_MFRONT_PROJECT.parent)
+     compare_mod.set_output_prefix(mfront_root, mfront_prefix)
+     compare_mod.set_biot_coefficient(mfront_root, 1.0)
+     compare_mod.set_nonlinear_max_iter(mfront_root, 80)
+     compare_mod.relax_first_component_abstol(mfront_root, 5e-7)
+     compare_mod.set_mfront_hamaker(mfront_root, hamaker_j)
+     ET.ElementTree(mfront_root).write(
+         mfront_prj, encoding="ISO-8859-1", xml_declaration=True
+     )
+
+     compare_mod.run ogs(native_ogs, native_prj, native_out)
+     compare_mod.run ogs(mfront_ogs, mfront_prj, mfront_out)
+
+     native_metrics = compare_mod.evaluate_beacon_metrics(
+         compare_mod.latest_vtu(native_out, native_prefix)
+     )
+     mfront_metrics = compare_mod.evaluate_beacon_metrics(
+         compare_mod.latest_vtu(mfront_out, mfront_prefix)
+     )
+
+     common = {
+         "benchmark_group": "EPFL_BGR_MP3",
+         "case_id": "p2_l_abprime",
+         "dry_density_kg_m3": dd_epfl,
+         "multiplier_used": multiplier,
+         "hamaker_constant_j": hamaker_j,
+         "source": "BEACON_MP3_BGR_EPFL",
+         "run_status": "success",
+     }
+     native_row = {
+         **common,
+         "implementation": "native",
+         **native_metrics,
+     }
+     mfront_row = {
+         **common,
+         "implementation": "mfront",
+         **mfront_metrics,
+     }
+     return [native_row, mfront_row]
+ finally:
+     shutil.rmtree(tmpdir, ignore_errors=True)
+
+
+def write_rows_csv(rows: list[dict[str, object]], path: Path) -> None:
+    keys = sorted(k for row in rows for k in row.keys())
+    path.parent.mkdir(parents=True, exist_ok=True)
+    with path.open("w", newline="", encoding="utf-8") as f:
+        writer = csv.DictWriter(f, fieldnames=keys)
+        writer.writeheader()
+        writer.writerows(rows)
+
+
+def git_hash(repo: Path) -> str:
+    try:
+        return (
+            subprocess.check_output(["git", "rev-parse", "--short", "HEAD"], cwd=repo, text=True)
+            .strip()
+        )
+    except Exception:
+        return ""
+
+
+def anchors_rows_identical_multiplier(
+    native_mod,
+    mfront_mod,
+    native_ogs: Path,
+    mfront_ogs: Path,
+    dd_curve: np.ndarray,
+    mult_curve: np.ndarray,
+) -> list[dict[str, object]]:
+    rows: list[dict[str, object]] = []
+    for i, dd in enumerate(dd_curve):
+        multiplier = interpolate_multiplier(float(dd), dd_curve, mult_curve)
+
+        mfront_case = mfront_mod.Case(float(dd))
+        native_case = native_mod.Case(float(dd))
+        mfront_n10 = mfront_mod.n10_from_micro_suction(
+            mfront_case.phi0, mfront_mod.HAMAKER_LITERATURE
+        )
+        native_n10 = native_mod.n10_from_micro_suction(
+            native_case.phi0, native_mod.HAMAKER_LITERATURE
+        )
+
+        mfront_run = mfront_mod.run_mfront_case(
+            mfront_ogs,
+            mfront_case,
+            multiplier,
+            mfront_n10,
+            900 + i,
+        )
+        native_run = native_mod.run_native_dsm_micromacro_case(
+            native_ogs,
+            native_case,
+            multiplier,
+            native_n10,
+            900 + i,
+        )
+
+        mfront_ps = float(mfront_run["pressure_mpa"])
+        native_ps = float(native_run["pressure_mpa"])
+
+        rows.append(
+            {
+                "benchmark_group": "ANCHORS_MS33_ModelI",
+                "case_id": f"dd{int(dd)}",
+                "implementation": "mfront",
+                "dry_density_kg_m3": float(dd),

```



```

+         "multiplier_used": multiplier,
+         "hamaker_constant_3": float(mfront_run["hamaker_effective"]),
+         "final_total_stress_mpa": mfront_ps,
+         "final_swelling_pressure_mpa": mfront_ps,
+         "axial_sigma_kpa": mfront_ps * 1e3,
+         "radial_sigma_kpa": math.nan,
+         "axial_swelling_stress_kpa": math.nan,
+         "radial_swelling_stress_kpa": math.nan,
+         "axial_displacement_mm": math.nan,
+         "radial_displacement_mm": math.nan,
+         "dry_density_mean_kg_m3": float(dd),
+         "run_status": "success",
+         "source": "anchors_identical_parameter_rerun",
+     }
+ )
+ rows.append(
+     {
+         "benchmark_group": "ANCHORS_MS33_ModelI",
+         "case_id": f"dd(int(dd))",
+         "implementation": "native",
+         "dry_density_kg_m3": float(dd),
+         "multiplier_used": multiplier,
+         "hamaker_constant_3": float(native_run["hamaker_effective_3"]),
+         "final_total_stress_mpa": native_ps,
+         "final_swelling_pressure_mpa": native_ps,
+         "axial_sigma_kpa": native_ps * 1e3,
+         "radial_sigma_kpa": math.nan,
+         "axial_swelling_stress_kpa": math.nan,
+         "radial_swelling_stress_kpa": math.nan,
+         "axial_displacement_mm": math.nan,
+         "radial_displacement_mm": math.nan,
+         "dry_density_mean_kg_m3": float(dd),
+         "run_status": "success",
+         "source": "anchors_identical_parameter_rerun",
+     }
+ )
+ return rows
+
+def max_abs_rel_delta(deltas: list[dict[str, object]], group: str) -> dict[str, object]:
+    filtered = [d for d in deltas if str(d["benchmark_group"]) == group]
+    if not filtered:
+        return {}
+
+    def abs_delta(row: dict[str, object]) -> float:
+        return abs(float(row["delta_swelling_pressure_mpa_native_minus_mfront"]))
+
+    def rel_delta(row: dict[str, object]) -> float:
+        ref = abs(float(row["mfront_final_swelling_pressure_mpa"]))
+        return abs_delta(row) / max(ref, 1e-12)
+
+    max_abs_row = max(filtered, key=abs_delta)
+    max_rel_row = max(filtered, key=rel_delta)
+    return {
+        "max_abs_case_id": max_abs_row["case_id"],
+        "max_abs_delta_mpa": abs_delta(max_abs_row),
+        "max_rel_case_id": max_rel_row["case_id"],
+        "max_rel_delta": rel_delta(max_rel_row),
+    }
+
+def main() -> None:
+    args = parse_args()
+
+    compare_mod = load_module(
+        ROOT / "run_calibrated_beacon_anchors_comparison.py",
+        "run_calibrated_beacon_anchors_comparison_mod",
+    )
+    mfront_mod = load_module(
+        ANCHORS_DIR / "run_villar_dense_dd_calibration.py",
+        "run_villar_dense_dd_calibration_mod",
+    )
+    native_mod = load_module(
+        ANCHORS_DIR / "run_villar_dense_dd_native_dsm_micromacro_calibration.py",
+        "run_villar_dense_dd_native_dsm_micromacro_calibration_mod",
+    )
+
+    if not args.shared_curve_csv.exists():
+        raise FileNotFoundError(args.shared_curve_csv)
+    dd_curve, mult_curve = load_multiplier_curve(args.shared_curve_csv)
+
+    # BEACON: enforce identical multiplier curve in both implementations.
+    beacon_rows = compare_mod.run_beacon_rows(
+        native_ogs=args.native_ogs.resolve(),
+        mfront_ogs=args.mfront_ogs.resolve(),
+        native_curve=(dd_curve, mult_curve),
+        mfront_curve=(dd_curve, mult_curve),
+    )
+
+    # ANCHORS: rerun both implementations with the same multiplier per DD.
+    anchors_rows = anchors_rows_identical_multiplier(
+        native_mod=native_mod,
+        mfront_mod=mfront_mod,
+        native_ogs=args.native_ogs.resolve(),
+        mfront_ogs=args.mfront_ogs.resolve(),
+        dd_curve=dd_curve,
+        mult_curve=mult_curve,
+    )
+
+    # EPFL: rerun native and MFront with the same DD-derived multiplier and MCC setup.
+    epfl_rows = run_epfl_rows(
+        compare_mod=compare_mod,
+        native_ogs=args.native_ogs.resolve(),
+        mfront_ogs=args.mfront_ogs.resolve(),
+        dd_curve=dd_curve,
+        mult_curve=mult_curve,
+    )
+
+    rows = anchors_rows + beacon_rows + epfl_rows
+    rows.sort(key=lambda r: (str(r["benchmark_group"]), str(r["case_id"]), str(r["implementation"])))
+    deltas = compare_mod.pairwise_deltas(rows)
+    deltas.sort(key=lambda d: (str(d["benchmark_group"]), str(d["case_id"])))
+
+    write_rows_csv(rows, args.out_csv)
+    write_rows_csv(deltas, args.out_deltas_csv)
+
+    summary = {

```

```

+         "mode": "identical_parameter_shared_multiplier_curve",
+         "shared_curve_csv": str(args.shared_curve_csv.resolve()),
+         "row_count": len(rows),
+         "pairwise_delta_count": len(deltas),
+         "beacon_delta_stats": max_abs_rel_delta(deltas, "BEACON_report"),
+         "anchors_delta_stats": max_abs_rel_delta(deltas, "ANCHORS_MS33_ModelI"),
+         "epfl_delta_stats": max_abs_rel_delta(deltas, "EPFL_BGR_WP3"),
+         "git_hashes": {
+             "ogs_repo_hash": git_hash(ROOT.parents[2]),
+             "native_repo_hash": git_hash(Path("/Users/vinaykumar/git/ogs-native-dsm-transition")),
+             "materialmodels_repo_hash": "",
+         },
+         "rows": rows,
+         "pairwise_deltas": deltas,
+     }
+     args.out_json.parent.mkdir(parents=True, exist_ok=True)
+     args.out_json.write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")
+
+     print(f"Wrote rows CSV: {args.out_csv}")
+     print(f"Wrote pairwise deltas CSV: {args.out_deltas_csv}")
+     print(f"Wrote summary JSON: {args.out_json}")
+
+
+if __name__ == "__main__":
+    main()
+
diff --git a/Tests/Data/RichardsMechanics/run_linear_law_calibrated_dsm_comparison.py b/Tests/Data/RichardsMechanics/run_linear_law_calibrated_dsm_comparison.py
new file mode 100644
index 0000000000..90e4b699de
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/run_linear_law_calibrated_dsm_comparison.py
@@ -0,0 +1,759 @@
+#!/usr/bin/env python3
+"""Calibrate DSM to OGS linear swelling law, then compare benchmarks.
+
+Workflow:
+1) Build ANCHORS dry-density calibration curves for DSM native and DSM Mfront
+   against the OGS linear SaturationDependentSwelling reference response.
+2) Rerun BEACON + EPFL with three variants:
+   - linear swelling-law baseline (native project, potential_exchange disabled),
+   - DSM native (potential_exchange enabled, linear swelling disabled),
+   - DSM Mfront (bridge behaviour with calibrated Hamaker).
+3) Merge ANCHORS, BEACON, EPFL into one comparison table and summary.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import importlib.util
+import json
+import math
+import os
+import shutil
+import subprocess
+import sys
+import tempfile
+import xml.etree.ElementTree as ET
+from dataclasses import dataclass
+from pathlib import Path
+
+import numpy as np
+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+import matplotlib
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+
+
+ROOT = Path(__file__).resolve().parent
+ANCHORS_DIR = ROOT / "ANCHORS_MS33_ModelI"
+EPFL_DIR = ROOT / "BEACON_WP3_BGR_EPFL"
+OUTPUT_ROOT = ROOT / "_outputs" / "linear_law_calibrated_dsm_comparison"
+HAMAKER_REFERENCE = 5.1e-21
+
+
+@dataclass(frozen=True)
+class BenchmarkCase:
+    benchmark_group: str
+    case_id: str
+    native_project: Path
+    mfront_project: Path
+
+BEACON_AND_EPFL_CASES = [
+    BenchmarkCase(
+        benchmark_group="BEACON_report",
+        case_id="1a01",
+        native_project=ROOT / "beacon_1a01_inflow_unstructured_batch.prj",
+        mfront_project=ROOT / "beacon_1a01_dsm_micromacro_mcc_inflow_unstructured_batch.prj",
+    ),
+    BenchmarkCase(
+        benchmark_group="BEACON_report",
+        case_id="1b",
+        native_project=ROOT / "beacon_1b_unstructured_batch.prj",
+        mfront_project=ROOT / "beacon_1b_dsm_micromacro_mcc_unstructured_batch.prj",
+    ),
+    BenchmarkCase(
+        benchmark_group="EPFL_BGR_WP3",
+        case_id="p2_1_abprime",
+        native_project=EPFL_DIR / "bgr_wp3_p2_1_abprime_native.prj",
+        mfront_project=EPFL_DIR / "bgr_wp3_p2_1_abprime_mfront.prj",
+    ),
+]
+
+
+def load_module(module_path: Path, module_name: str):
+    spec = importlib.util.spec_from_file_location(module_name, module_path)
+    if spec is None or spec.loader is None:
+        raise RuntimeError(f"Could not load module: {module_path}")
+    module = importlib.util.module_from_spec(spec)
+    sys.modules[module_name] = module
+    spec.loader.exec_module(module)
+    return module

```

```

+
+def parse_args() -> argparse.Namespace:
+    parser = argparse.ArgumentParser(
+        description=(
+            "Calibrate DSM native + MFront against the OGS linear swelling-law "
+            "reference, then compare ANCHORS + BEACON + EPFL."
+        )
+    )
+    parser.add_argument(
+        "--native-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-native-transition2/bin/ogs"),
+    )
+    parser.add_argument(
+        "--mfront-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs"),
+    )
+    parser.add_argument(
+        "--linear-ref-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs"),
+        help="OGS binary used for Linear-law ANCHORS reference runs.",
+    )
+    parser.add_argument("--dd-min", type=float, default=1400.0)
+    parser.add_argument("--dd-max", type=float, default=1800.0)
+    parser.add_argument("--dd-step", type=float, default=25.0)
+    parser.add_argument("--rel-tol", type=float, default=0.02)
+    parser.add_argument(
+        "--out-root",
+        type=Path,
+        default=OUTPUT_ROOT,
+        help="Output directory for CSV/JSON/PNG artifacts.",
+    )
+    parser.add_argument(
+        "--assert-thresholds",
+        action="store_true",
+        help="Fail if ANCHORS calibration relative errors exceed rel-tol.",
+    )
+    return parser.parse_args()
+
+
+def git_short_hash(repo: Path) -> str:
+    try:
+        return (
+            subprocess.check_output(
+                ["git", "rev-parse", "--short", "HEAD"], cwd=repo, text=True
+            )
+            .strip()
+        )
+    except Exception:
+        return ""
+
+
+def write_rows_csv(rows: list[dict[str, object]], path: Path) -> None:
+    path.parent.mkdir(parents=True, exist_ok=True)
+    keys: list[str] = []
+    seen: set[str] = set()
+    for row in rows:
+        for k in row.keys():
+            if k not in seen:
+                seen.add(k)
+                keys.append(k)
+    with path.open("w", newline="", encoding="utf-8") as f:
+        writer = csv.DictWriter(f, fieldnames=keys)
+        writer.writeheader()
+        writer.writerows(rows)
+
+
+def interpolate_multiplier(dd: float, dd_curve: np.ndarray, mult_curve: np.ndarray) -> float:
+    return float(np.interp(dd, dd_curve, mult_curve, left=mult_curve[0], right=mult_curve[-1]))
+
+
+def remove_potential_exchange(root: ET.Element) -> None:
+    process = root.find("./processes/process")
+    if process is None:
+        return
+    pe = process.find("potential_exchange")
+    if pe is not None:
+        process.remove(pe)
+
+
+def zero_swelling_stress_rate(root: ET.Element) -> None:
+    for prop in root.findall("./property"):
+        name = (prop.findtext("name") or "").strip()
+        if name != "swelling_stress_rate":
+            continue
+        s = prop.find("swelling_pressures")
+        if s is not None:
+            s.text = "0 0 0"
+
+
+def find_existing_parameter(root: ET.Element, names: list[str], fallback: str) -> str:
+    existing = {
+        (node.findtext("name") or "").strip()
+        for node in root.findall("./parameters/parameter")
+    }
+    for n in names:
+        if n in existing:
+            return n
+    return fallback
+
+
+def set_linear_elastic_constitutive(root: ET.Element) -> None:
+    """Replace constitutive relation by native linear elastic isotropic law."""
+    cr = root.find("./processes/process/constitutive_relation")
+    if cr is None:
+        return
+    young_name = find_existing_parameter(root, ["YoungModulus", "E"], "YoungModulus")
+    poisson_name = find_existing_parameter(root, ["PoissonRatio", "nu"], "PoissonRatio")
+    for child in list(cr):
+        cr.remove(child)
+    ET.SubElement(cr, "type").text = "LinearElasticIsotropic"
+    ET.SubElement(cr, "youngs_modulus").text = young_name
+    ET.SubElement(cr, "poissons_ratio").text = poisson_name
+
+

```

```

+def configure_linear_reference_outputs(root: ET.Element) -> None:
+    """Limit outputs/secondary variables to fields available in linear reference."""
+    sec = root.find("./processes/process/secondary_variables")
+    if sec is not None:
+        for child in list(sec):
+            sec.remove(child)
+        for name in [
+            "sigma",
+            "swelling_stress",
+            "saturation",
+            "porosity",
+            "transport_porosity",
+            "dry_density_solid",
+            "epsilon",
+            "velocity",
+        ]:
+            sv = ET.SubElement(sec, "secondary_variable")
+            sv.set("name", name)
+
+    out_vars = root.find("./time_loop/output/variables")
+    if out_vars is not None:
+        for child in list(out_vars):
+            out_vars.remove(child)
+        for name in [
+            "pressure",
+            "displacement",
+            "saturation",
+            "swelling_stress",
+            "sigma",
+            "porosity",
+            "transport_porosity",
+            "dry_density_solid",
+        ]:
+            v = ET.SubElement(out_vars, "variable")
+            v.text = name
+
+def run_project_variant(
+    *,
+    root: ET.Element,
+    source_project: Path,
+    prefix: str,
+    project_path: Path,
+    output_dir: Path,
+    ogs_bin: Path,
+    compare_mod,
+    max_iter: int = 80,
+    first_abstol: float = 5e-7,
+) -> tuple[dict[str, float], str, str]:
+    compare_mod.absolutize_mesh_and_geometry(root, source_project.parent)
+    compare_mod.set_output_prefix(root, prefix)
+    compare_mod.set_biot_coefficient(root, 1.0)
+    compare_mod.set_nonlinear_max_iter(root, max_iter)
+    compare_mod.relax_first_component_abstol(root, first_abstol)
+    ET.ElementTree(root).write(project_path, encoding="ISO-8859-1", xml_declaration=True)
+    try:
+        compare_mod.run_ogs(ogs_bin, project_path, output_dir)
+        metrics = compare_mod.evaluate_beacon_metrics(compare_mod.latest_vtu(output_dir, prefix))
+        return metrics, "success", ""
+    except Exception as exc:
+        return compare_mod.nan_beacon_metrics(), "failed", compare_mod.summarize_error(exc)
+
+def calibration_rows_and_curves(args: argparse.Namespace, mfront_mod, native_mod) -> tuple[
+    list[dict[str, object]],
+    np.ndarray,
+    np.ndarray,
+    np.ndarray,
+    np.ndarray,
+]
+    dd_values = np.arange(args.dd_min, args.dd_max + 0.5 * args.dd_step, args.dd_step)
+    rows: list[dict[str, object]] = []
+    for dd in dd_values:
+        mcase = mfront_mod.Case(float(dd))
+        ncase = native_mod.Case(float(dd))
+
+        linear_target = mfront_mod.run_native_case(args.linear_ref_ogs, mcase)
+
+        mfront_cal = mfront_mod.calibrate_multiplier_for_case(
+            args.mfront_ogs, mcase, linear_target, rel_tol=args.rel_tol
+        )
+
+        n_l0_fixed = native_mod.n_l0_from_micro_suction(
+            ncase.phi0, native_mod.HAMAKER_LITERATURE
+        )
+        native_cal = native_mod.calibrate_multiplier_for_case(
+            args.native_ogs,
+            ncase,
+            n_l0_fixed,
+            linear_target,
+            rel_tol=args.rel_tol,
+        )
+
+        mfront_rel_err = abs(mfront_cal["pressure_mpa"] - linear_target) / max(
+            abs(linear_target), 1e-12
+        )
+        native_rel_err = abs(native_cal["pressure_mpa"] - linear_target) / max(
+            abs(linear_target), 1e-12
+        )
+
+        rows.append(
+            {
+                "dry_density_kg_m3": float(dd),
+                "linear_target_MPa": float(linear_target),
+                "mfront_calibrated_MPa": float(mfront_cal["pressure_mpa"]),
+                "native_calibrated_MPa": float(native_cal["pressure_mpa"]),
+                "mfront_vdw_multiplier": float(mfront_cal["multiplier"]),
+                "native_vdw_multiplier": float(native_cal["multiplier"]),
+                "mfront_hamaker_effective_J": float(mfront_cal["hamaker_effective_J"]),
+                "native_hamaker_effective_J": float(native_cal["hamaker_effective_J"]),
+                "mfront_relative_error": float(mfront_rel_err),
+                "native_relative_error": float(native_rel_err),
+            }
+        )
+    print(
+        f"ANCHORS linear calibration dd={dd:.0f}: "
+        f"linear={linear_target:.6f} MPa, "

```

```

+         f"native={native_cal['pressure_mpa']:.6f} MPa (m={native_cal['multiplier']:.6e}), "
+         f"mfront={mfront_cal['pressure_mpa']:.6f} MPa (m={mfront_cal['multiplier']:.6e})"
+     )
+
+ rows = sorted(rows, key=lambda r: float(r["dry_density_kg_m3"]))
+ dd_curve = np.array([float(r["dry_density_kg_m3"]) for r in rows], dtype=float)
+ native_curve = np.array([float(r["native_vdw_multiplier"]) for r in rows], dtype=float)
+ mfront_curve = np.array([float(r["mfront_vdw_multiplier"]) for r in rows], dtype=float)
+ return rows, dd_curve, native_curve, dd_curve, mfront_curve
+
+
+def anchors_triplet_rows(calibration_rows: list[dict[str, object]]) -> list[dict[str, object]]:
+ rows: list[dict[str, object]] = []
+ for r in calibration_rows:
+     dd = float(r["dry_density_kg_m3"])
+     linear = float(r["linear_target_MPa"])
+     native = float(r["native_calibrated_MPa"])
+     mfront = float(r["mfront_calibrated_MPa"])
+     rows.extend(
+         [
+             {
+                 "benchmark_group": "ANCHORS_MS33_ModelI",
+                 "case_id": f"dd(int(dd))",
+                 "implementation": "linear",
+                 "dry_density_kg_m3": dd,
+                 "multiplier_used": math.nan,
+                 "hamaker_constant_j": math.nan,
+                 "final_total_stress_mpa": linear,
+                 "final_swelling_pressure_mpa": linear,
+                 "axial_sigma_kpa": linear * 1e3,
+                 "radial_sigma_kpa": math.nan,
+                 "axial_swelling_stress_kpa": math.nan,
+                 "radial_swelling_stress_kpa": math.nan,
+                 "axial_displacement_mm": math.nan,
+                 "radial_displacement_mm": math.nan,
+                 "dry_density_mean_kg_m3": dd,
+                 "run_status": "success",
+                 "error_message": "",
+                 "source": "linear_reference_native_project",
+             },
+             {
+                 "benchmark_group": "ANCHORS_MS33_ModelI",
+                 "case_id": f"dd(int(dd))",
+                 "implementation": "native_dsm",
+                 "dry_density_kg_m3": dd,
+                 "multiplier_used": float(r["native_vdw_multiplier"]),
+                 "hamaker_constant_j": float(r["native_hamaker_effective_j"]),
+                 "final_total_stress_mpa": native,
+                 "final_swelling_pressure_mpa": native,
+                 "axial_sigma_kpa": native * 1e3,
+                 "radial_sigma_kpa": math.nan,
+                 "axial_swelling_stress_kpa": math.nan,
+                 "radial_swelling_stress_kpa": math.nan,
+                 "axial_displacement_mm": math.nan,
+                 "radial_displacement_mm": math.nan,
+                 "dry_density_mean_kg_m3": dd,
+                 "run_status": "success",
+                 "error_message": "",
+                 "source": "anchors_linear_calibration_native",
+             },
+             {
+                 "benchmark_group": "ANCHORS_MS33_ModelI",
+                 "case_id": f"dd(int(dd))",
+                 "implementation": "mfront_dsm",
+                 "dry_density_kg_m3": dd,
+                 "multiplier_used": float(r["mfront_vdw_multiplier"]),
+                 "hamaker_constant_j": float(r["mfront_hamaker_effective_j"]),
+                 "final_total_stress_mpa": mfront,
+                 "final_swelling_pressure_mpa": mfront,
+                 "axial_sigma_kpa": mfront * 1e3,
+                 "radial_sigma_kpa": math.nan,
+                 "axial_swelling_stress_kpa": math.nan,
+                 "radial_swelling_stress_kpa": math.nan,
+                 "axial_displacement_mm": math.nan,
+                 "radial_displacement_mm": math.nan,
+                 "dry_density_mean_kg_m3": dd,
+                 "run_status": "success",
+                 "error_message": "",
+                 "source": "anchors_linear_calibration_mfront",
+             },
+         ]
+     )
+ return rows
+
+
+def mfront_defaults_for_epfl(root: ET.Element, parity_mod) -> dict[str, float]:
+     return {
+         "ReferenceDensitySolid": parity_mod.get_parameter_value(
+             root, "ReferenceDensitySolid", default=2780.0
+         ),
+         "SpecificSurface": parity_mod.get_parameter_value(root, "SpecificSurface", default=523.0),
+         "MicroSwellingStrainSlope": parity_mod.get_parameter_value(
+             root, "MicroSwellingStrainSlope", default=0.0
+         ),
+         "MassExchangeCoefficient": parity_mod.get_parameter_value(
+             root, "MassExchangeCoefficient", default=5e-15
+         ),
+         "n_l0": parity_mod.get_parameter_value(root, "n_l0", default=1e-12),
+     }
+
+
+def run_beacon_epfl_triplet_rows(
+     *,
+     args: argparse.Namespace,
+     compare_mod,
+     parity_mod,
+     dd_curve_native: np.ndarray,
+     mult_curve_native: np.ndarray,
+     dd_curve_mfront: np.ndarray,
+     mult_curve_mfront: np.ndarray,
+) -> list[dict[str, object]]:
+ rows: list[dict[str, object]] = []
+ tmpdir = Path(tempfile.mkdtemp(prefix="linear-law-triplet-runs-"))
+ try:
+     for case in BEACON_AND_EPFL_CASES:
+         dd_native = compare_mod.extract_dd_from_project(case.native_project)
+         dd_mfront = compare_mod.extract_dd_from_project(case.mfront_project)

```

```

+ native_multiplier = interpolate_multiplier(
+     dd_native, dd_curve_native, mult_curve_native
+ )
+ mfront_multiplier = interpolate_multiplier(
+     dd_mfront, dd_curve_mfront, mult_curve_mfront
+ )
+ hamaker_native = HAMAKER_REFERENCE_J * native_multiplier
+ hamaker_mfront = HAMAKER_REFERENCE_J * mfront_multiplier
+
+ # 1) Linear baseline on native project with potential_exchange disabled.
+ linear_prefix = f"{case.benchmark_group}_{case.case_id}_linear".lower()
+ linear_project = tmpdir / f"{linear_prefix}.prj"
+ linear_out = tmpdir / f"out_{linear_prefix}"
+ linear_root = ET.parse(case.native_project).getroot()
+ remove_potential_exchange(linear_root)
+ set_linear_elastic_constitutive(linear_root)
+ configure_linear_reference_outputs(linear_root)
+ linear_metrics, linear_status, linear_error = run_project_variant(
+     root=linear_root,
+     source_project=case.native_project,
+     prefix=linear_prefix,
+     project_path=linear_project,
+     output_dir=linear_out,
+     ogs_bin=args.native_ogs,
+     compare_mod=compare_mod,
+     max_iter=200,
+     first_abstol=1e-5,
+ )
+
+ # 2) Native DSM with linear swelling disabled.
+ native_prefix = f"{case.benchmark_group}_{case.case_id}_native_dsm".lower()
+ native_project = tmpdir / f"{native_prefix}.prj"
+ native_out = tmpdir / f"out_{native_prefix}"
+ native_root = ET.parse(case.native_project).getroot()
+ pe = native_root.find("./processes/process/potential_exchange")
+ if pe is None:
+     epfl_mfront_root = ET.parse(case.mfront_project).getroot()
+     parity_mod.ensure_native_epfl_potential_exchange(
+         native_root,
+         hamaker_native,
+         mfront_parameter_defaults=mfront_defaults_for_epfl(
+             epfl_mfront_root, parity_mod
+         ),
+     )
+ else:
+     compare_mod.set_native_hamaker(native_root, hamaker_native)
+     zero_swelling_stress_rate(native_root)
+ native_metrics, native_status, native_error = run_project_variant(
+     root=native_root,
+     source_project=case.native_project,
+     prefix=native_prefix,
+     project_path=native_project,
+     output_dir=native_out,
+     ogs_bin=args.native_ogs,
+     compare_mod=compare_mod,
+ )
+
+ # 3) Mfront DSM.
+ mfront_prefix = f"{case.benchmark_group}_{case.case_id}_mfront_dsm".lower()
+ mfront_project = tmpdir / f"{mfront_prefix}.prj"
+ mfront_out = tmpdir / f"out_{mfront_prefix}"
+ mfront_root = ET.parse(case.mfront_project).getroot()
+ compare_mod.set_mfront_hamaker(mfront_root, hamaker_mfront)
+ mfront_metrics, mfront_status, mfront_error = run_project_variant(
+     root=mfront_root,
+     source_project=case.mfront_project,
+     prefix=mfront_prefix,
+     project_path=mfront_project,
+     output_dir=mfront_out,
+     ogs_bin=args.mfront_ogs,
+     compare_mod=compare_mod,
+ )
+
+ common = {
+     "benchmark_group": case.benchmark_group,
+     "case_id": case.case_id,
+     "dry_density_kg_m3": float(0.5 * (dd_native + dd_mfront)),
+ }
+ rows.extend(
+     [
+         {
+             **common,
+             "implementation": "linear",
+             "multiplier_used": math.nan,
+             "hamaker_constant_j": math.nan,
+             "run_status": linear_status,
+             "error_message": linear_error,
+             "source": case.native_project.name,
+             **linear_metrics,
+         },
+         {
+             **common,
+             "implementation": "native_dsm",
+             "multiplier_used": native_multiplier,
+             "hamaker_constant_j": hamaker_native,
+             "run_status": native_status,
+             "error_message": native_error,
+             "source": case.native_project.name,
+             **native_metrics,
+         },
+         {
+             **common,
+             "implementation": "mfront_dsm",
+             "multiplier_used": mfront_multiplier,
+             "hamaker_constant_j": hamaker_mfront,
+             "run_status": mfront_status,
+             "error_message": mfront_error,
+             "source": case.mfront_project.name,
+             **mfront_metrics,
+         },
+     ]
+ )
+ finally:
+     shutil.rmtree(tmpdir, ignore_errors=True)
+ return rows

```

```

+def triplet_deltas(rows: list[dict[str, object]]) -> list[dict[str, object]]:
+    by_case: dict[tuple[str, str], dict[str, dict[str, object]]] = {}
+    for row in rows:
+        by_case.setdefault((str(row["benchmark_group"]), str(row["case_id"])) , {})[
+            str(row["implementation"])
+        ] = row
+
+    out: list[dict[str, object]] = []
+    for (group, case_id), impl in sorted(by_case.items()):
+        linear = impl.get("linear")
+        native = impl.get("native_dsm")
+        mfront = impl.get("mfront_dsm")
+        if linear is None or native is None or mfront is None:
+            continue
+
+        def val(r: dict[str, object], key: str) -> float:
+            try:
+                return float(r.get(key, math.nan))
+            except Exception:
+                return math.nan
+
+        p_lin = val(linear, "final_swelling_pressure_mpa")
+        p_nat = val(native, "final_swelling_pressure_mpa")
+        p_mfr = val(mfront, "final_swelling_pressure_mpa")
+        out.append(
+            {
+                "benchmark_group": group,
+                "case_id": case_id,
+                "dry_density_kg_m3": val(linear, "dry_density_kg_m3"),
+                "linear_final_swelling_pressure_mpa": p_lin,
+                "native_dsm_final_swelling_pressure_mpa": p_nat,
+                "mfront_dsm_final_swelling_pressure_mpa": p_mfr,
+                "delta_native_minus_linear_mpa": p_nat - p_lin,
+                "delta_mfront_minus_linear_mpa": p_mfr - p_lin,
+                "delta_native_minus_mfront_mpa": p_nat - p_mfr,
+                "rel_native_vs_linear": abs(p_nat - p_lin) / max(abs(p_lin), 1e-12),
+                "rel_mfront_vs_linear": abs(p_mfr - p_lin) / max(abs(p_lin), 1e-12),
+                "native_multiplier": val(native, "multiplier_used"),
+                "mfront_multiplier": val(mfront, "multiplier_used"),
+                "native_to_mfront_multiplier_ratio": val(native, "multiplier_used")
+                / max(val(mfront, "multiplier_used"), 1e-300),
+                "native_status": str(native.get("run_status", "")),
+                "mfront_status": str(mfront.get("run_status", "")),
+                "linear_status": str(linear.get("run_status", "")),
+            }
+        )
+    return out
+
+def plot_triplet_summary(deltas: list[dict[str, object]], out_png: Path) -> None:
+    if not deltas:
+        return
+    rows = sorted(deltas, key=lambda r: (r["benchmark_group"], r["case_id"]))
+    labels = [f"{r['benchmark_group']}:{r['case_id']}" for r in rows]
+    y_lin = np.array([float(r["linear_final_swelling_pressure_mpa"]) for r in rows])
+    y_nat = np.array([float(r["native_dsm_final_swelling_pressure_mpa"]) for r in rows])
+    y_mfr = np.array([float(r["mfront_dsm_final_swelling_pressure_mpa"]) for r in rows])
+
+    x = np.arange(len(rows))
+    w = 0.28
+    fig, ax = plt.subplots(figsize=(1.1 * max(8, len(rows)), 4.8))
+    ax.bar(x - w, y_lin, width=w, label="Linear reference")
+    ax.bar(x, y_nat, width=w, label="DSM native")
+    ax.bar(x + w, y_mfr, width=w, label="DSM Mfront")
+    ax.set_xticks(x)
+    ax.set_xticklabels(labels, rotation=30, ha="right")
+    ax.set_ylabel("Final swelling pressure (MPa)")
+    ax.set_title("Linear-law-calibrated comparison: linear vs DSM native vs DSM Mfront")
+    ax.grid(True, axis="y", ls=":", alpha=0.5)
+    ax.legend(loc="best")
+    fig.tight_layout()
+    out_png.parent.mkdir(parents=True, exist_ok=True)
+    fig.savefig(out_png, dpi=200)
+    plt.close(fig)
+
+def fail_if_needed(
+    *,
+    args: argparse.Namespace,
+    calibration_rows: list[dict[str, object]],
+    all_rows: list[dict[str, object]],
+) -> None:
+    if not args.assert_thresholds:
+        return
+    failures: list[str] = []
+    for r in calibration_rows:
+        dd = float(r["dry_density_kg_m3"])
+        if float(r["native_relative_error"]) > args.rel_tol:
+            failures.append(
+                f"ANCHORS dd={dd:.0f}: native calibration rel err "
+                f"{float(r['native_relative_error']):.6g} > {args.rel_tol:.6g}"
+            )
+        if float(r["mfront_relative_error"]) > args.rel_tol:
+            failures.append(
+                f"ANCHORS dd={dd:.0f}: mfront calibration rel err "
+                f"{float(r['mfront_relative_error']):.6g} > {args.rel_tol:.6g}"
+            )
+    for r in all_rows:
+        if str(r.get("run_status", "success")) != "success":
+            failures.append(
+                f"{r.get('benchmark_group')}:{r.get('case_id')}:{r.get('implementation')} failed: "
+                f"{r.get('error_message', '')}"
+            )
+    if failures:
+        raise RuntimeError(
+            "Linear-law-calibrated comparison thresholds failed:\n- "
+            + "\n- ".join(failures)
+        )
+
+def main() -> None:
+    args = parse_args()
+    args.out_root.mkdir(parents=True, exist_ok=True)
+
+    compare_mod = load_module(
+        ROOT / "run_calibrated_beacon_anchors_comparison.py",
+        "run_calibrated_beacon_anchors_comparison_linear_mode",

```

```

+ )
+ parity_mod = load_module(
+     ROOT / "run_identical_parameter_benchmark_parity.py",
+     "run_identical_parameter_benchmark_parity_linear_mode",
+ )
+ mfront_mod = load_module(
+     ANCHORS_DIR / "run_villar_dense_dd_calibration.py",
+     "run_villar_dense_dd_calibration_linear_mode",
+ )
+ native_mod = load_module(
+     ANCHORS_DIR / "run_villar_dense_dd_native_dsm_micromacro_calibration.py",
+     "run_villar_dense_dd_native_dsm_micromacro_calibration_linear_mode",
+ )
+
+ calibration_rows, dd_native, mult_native, dd_mfront, mult_mfront = (
+     calibration_rows_and_curves(args, mfront_mod, native_mod)
+ )
+
+ anchors_rows = anchors_triplet_rows(calibration_rows)
+ benchmark_rows = run_beacon_epfl_triplet_rows(
+     args=args,
+     compare_mod=compare_mod,
+     parity_mod=parity_mod,
+     dd_curve_native=dd_native,
+     mult_curve_native=mult_native,
+     dd_curve_mfront=dd_mfront,
+     mult_curve_mfront=mult_mfront,
+ )
+
+ all_rows = anchors_rows + benchmark_rows
+ deltas = triplet_deltas(all_rows)
+
+ calibration_csv = args.out_root / "anchors_linear_law_calibration.csv"
+ rows_csv = args.out_root / "linear_law_triplet_runs_summary.csv"
+ deltas_csv = args.out_root / "linear_law_triplet_pairwise_deltas.csv"
+ summary_json = args.out_root / "linear_law_triplet_summary.json"
+ plot_png = args.out_root / "linear_law_triplet_swelling_pressure_comparison.png"
+
+ write_rows_csv(calibration_rows, calibration_csv)
+ write_rows_csv(all_rows, rows_csv)
+ write_rows_csv(deltas, deltas_csv)
+ plot_triplet_summary(deltas, plot_png)
+
+ summary = {
+     "mode": "linear_law_calibrated_triplet_comparison",
+     "native ogs": str(args.native_ogs.resolve()),
+     "mfront ogs": str(args.mfront_ogs.resolve()),
+     "linear_ref ogs": str(args.linear_ref_ogs.resolve()),
+     "dd_range": {
+         "min": args.dd_min,
+         "max": args.dd_max,
+         "step": args.dd_step,
+     },
+     "rel_tol": args.rel_tol,
+     "row_count": len(all_rows),
+     "delta_count": len(deltas),
+     "max_native_calibration_rel_error": max(
+         (float(r["native_relative_error"]) for r in calibration_rows), default=math.nan
+     ),
+     "max_mfront_calibration_rel_error": max(
+         (float(r["mfront_relative_error"]) for r in calibration_rows), default=math.nan
+     ),
+     "git_hashes": {
+         "ogs_repo_hash": git_short_hash(ROOT.parent.parent.parent),
+         "native_repo_hash": git_short_hash(
+             Path("/Users/vinaykumar/git/ogs-native-dsm-transition")
+         ),
+     },
+     "artifacts": {
+         "calibration_csv": str(calibration_csv.resolve()),
+         "rows_csv": str(rows_csv.resolve()),
+         "deltas_csv": str(deltas_csv.resolve()),
+         "plot_png": str(plot_png.resolve()),
+     },
+     "deltas": deltas,
+ }
+
+ summary_json.write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")
+
+ fail_if_needed(args=args, calibration_rows=calibration_rows, all_rows=all_rows)
+
+ print(f"Wrote calibration CSV: {calibration_csv}")
+ print(f"Wrote rows CSV: {rows_csv}")
+ print(f"Wrote deltas CSV: {deltas_csv}")
+ print(f"Wrote summary JSON: {summary_json}")
+ print(f"Wrote plot: {plot_png}")
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/run_linear_mcc_dd1700_dsm_comparison.py b/Tests/Data/RichardsMechanics/run_linear_mcc_dd1700_dsm_comparison.py
new file mode 100644
index 0000000000..91e78f832a
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/run_linear_mcc_dd1700_dsm_comparison.py
@@ -0,0 +1,623 @@
+#!/usr/bin/env python3
+"""DD=1700 comparison: linear swelling (MCC) vs native DSM vs MFront DSM.
+
+
+This script performs one focused comparison requested for the transition note:
+1) Calibrate native and MFront DSM vdw multipliers at DD=1700 kg/m3 using
+   the existing Villar-curve calibration drivers.
+2) Run three one-element confined hydration variants with MCC carrier:
+   - linear SaturationDependentSwelling with max swelling pressure = 11e6 Pa,
+   - native DSM + MCC with calibrated multiplier,
+   - MFront DSM + MCC with calibrated multiplier.
+3) Write CSV/JSON/PNG artifacts for documentation.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import importlib.util
+import json
+import math
+import os
+import re
+import shutil

```



```

+import subprocess
+import sys
+import tempfile
+import xml.etree.ElementTree as ET
+from dataclasses import dataclass
+from pathlib import Path
+
+
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+
+import matplotlib
+
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+
+
+
+
+ROOT = Path(__file__).resolve().parent
+ANCHORS_DIR = ROOT / "ANCHORS_MS33_ModelI"
+OUTPUT_ROOT = ROOT / "_outputs" / "linear_mcc_dd1700_dsm_comparison"
+
+
+
+@dataclass(frozen=True)
+class RunSpec:
+    implementation: str
+    project_path: Path
+    ogs_bin: Path
+    output_prefix: str
+
+
+
+def load_module(module_path: Path, module_name: str):
+    spec = importlib.util.spec_from_file_location(module_name, module_path)
+    if spec is None or spec.loader is None:
+        raise RuntimeError(f"Could not load module: {module_path}")
+    module = importlib.util.module_from_spec(spec)
+    sys.modules[module_name] = module
+    spec.loader.exec_module(module)
+    return module
+
+
+
+def parse_args() -> argparse.Namespace:
+    parser = argparse.ArgumentParser(
+        description=(
+            "Run DD=1700 linear(MCC, p_sw,max=11e6) vs native DSM vs MFront DSM "
+            "comparison with calibrated multipliers."
+        )
+    )
+    parser.add_argument(
+        "--native-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-native-transition2/bin/ogs"),
+    )
+    parser.add_argument(
+        "--mfront-ogs",
+        type=Path,
+        default=Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs"),
+    )
+    parser.add_argument("--dry-density", type=float, default=1700.0)
+    parser.add_argument(
+        "--max-swelling-pressure-pa",
+        type=float,
+        default=11e6,
+        help="Linear SaturationDependentSwelling isotropic max pressure (Pa).",
+    )
+    parser.add_argument(
+        "--rel-tol",
+        type=float,
+        default=0.02,
+        help="Relative tolerance used in the two calibration routines.",
+    )
+    parser.add_argument(
+        "--out-root",
+        type=Path,
+        default=OUTPUT_ROOT,
+        help="Directory for CSV/JSON/PNG outputs.",
+    )
+    parser.add_argument(
+        "--keep-runtime",
+        action="store_true",
+        help="Keep temporary project files in ANCHORS folder.",
+    )
+    return parser.parse_args()
+
+
+
+def git_short_hash(repo: Path) -> str:
+    try:
+        return (
+            subprocess.check_output(
+                ["git", "rev-parse", "--short", "HEAD"], cwd=repo, text=True
+            )
+            .strip()
+        )
+    except Exception:
+        return ""
+
+
+
+def set_output_prefix(root: ET.Element, prefix: str) -> None:
+    node = root.find("./time_loop/output/prefix")
+    if node is None:
+        raise RuntimeError("Output prefix node not found in project.")
+    node.text = prefix
+
+
+
+def limit_outputs_to_linear_core(root: ET.Element) -> None:
+    allowed = {
+        "pressure",
+        "displacement",
+        "sigma",
+        "swelling_stress",
+        "saturation",
+        "porosity",
+        "dry_density_solid",
+    }
+    sec = root.find("./processes/process/secondary_variables")

```

```

+     if sec is not None:
+         for child in list(sec):
+             name = child.get("name", "").strip()
+             if name not in allowed:
+                 sec.remove(child)
+
+     out_vars = root.find("./time_loop/output/variables")
+     if out_vars is not None:
+         for child in list(out_vars):
+             name = (child.text or "").strip()
+             if name not in allowed:
+                 out_vars.remove(child)
+
+
+def configure_linear_mcc_project(
+    source_project: Path, target_project: Path, prefix: str, max_swelling_pressure_pa: float
+) -> None:
+    root = ET.parse(source_project).getroot()
+    process = root.find("./processes/process")
+    if process is None:
+        raise RuntimeError("Missing process node in source project.")
+
+    pe = process.find("potential_exchange")
+    if pe is not None:
+        process.remove(pe)
+    mp = process.find("micro_porosity")
+    if mp is not None:
+        process.remove(mp)
+
+    sw = root.find(
+        "./media/medium/phases/phase[type='Solid']/properties/property[name='swelling_stress_rate']"
+    )
+    if sw is None:
+        raise RuntimeError("Solid swelling_stress_rate property not found.")
+    sw_type = sw.find("type")
+    if sw_type is None:
+        sw_type = ET.SubElement(sw, "type")
+        sw_type.text = "SaturationDependentSwelling"
+    pressures = sw.find("swelling_pressures")
+    if pressures is None:
+        pressures = ET.SubElement(sw, "swelling_pressures")
+        pressures.text = (
+            f"({max_swelling_pressure_pa:.16g}) "
+            f"({max_swelling_pressure_pa:.16g}) "
+            f"({max_swelling_pressure_pa:.16g})"
+        )
+
+    exponents = sw.find("exponents")
+    if exponents is None:
+        exponents = ET.SubElement(sw, "exponents")
+        exponents.text = "1 1 1"
+    smin = sw.find("lower_saturation_limit")
+    if smin is None:
+        smin = ET.SubElement(sw, "lower_saturation_limit")
+        smin.text = "0"
+    smax = sw.find("upper_saturation_limit")
+    if smax is None:
+        smax = ET.SubElement(sw, "upper_saturation_limit")
+        smax.text = "1"
+
+    set_output_prefix(root, prefix)
+    limit_outputs_to_linear_core(root)
+    ET.ElementTree(root).write(target_project, encoding="ISO-8859-1", xml_declaration=True)
+
+
+def parse_time_from_vtu(path: Path) -> float:
+    stem = path.stem
+    match = re.search(r"_t_[0-9eE.]+s", stem)
+    if not match:
+        raise RuntimeError(f"Cannot parse time from output filename {path.name}")
+    return float(match.group(1))
+
+
+def latest_vtu(output_dir: Path, prefix: str) -> Path:
+    files = sorted(output_dir.glob(f"{prefix}_ts_*_t_*_vtu"), key=parse_time_from_vtu)
+    if not files:
+        raise FileNotFoundError(f"No VTU outputs for prefix {prefix} in {output_dir}")
+    return files[-1]
+
+
+def load_vtu(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    grid = reader.GetOutput()
+    points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+    arrays: dict[str, np.ndarray] = {}
+
+    point_data = grid.GetPointData()
+    for i in range(point_data.GetNumberOfArrays()):
+        arr = point_data.GetArray(i)
+        if arr is None:
+            continue
+        name = point_data.GetArrayName(i)
+        if name is None:
+            continue
+        arrays[name] = vtk_to_numpy(arr)
+
+    cell_data = grid.GetCellData()
+    for i in range(cell_data.GetNumberOfArrays()):
+        arr = cell_data.GetArray(i)
+        if arr is None:
+            continue
+        name = cell_data.GetArrayName(i)
+        if name is None or name in arrays:
+            continue
+        arrays[name] = vtk_to_numpy(arr)
+    return points, arrays
+
+
+def boundary_mask(points: np.ndarray, axis: int, side: str) -> np.ndarray:
+    val = points[:, axis].max() if side == "max" else points[:, axis].min()
+    return np.isclose(points[:, axis], val, atol=1e-10)
+
+
+def mean_component(values: np.ndarray, mask: np.ndarray, idx: int) -> float:
+    arr = np.asarray(values)

```

```

+ if arr.ndim == 1:
+     return float(arr[mask].mean())
+ return float(arr[mask, idx].mean())
+
+
+def mean_isotropic_pressure_mpa(tensor: np.ndarray) -> float:
+    arr = np.asarray(tensor)
+    p_pa = float((-arr[:, 0] - arr[:, 1] - arr[:, 2]).mean() / 3.0)
+    return p_pa / 1e6
+
+
+def evaluate_metrics(vtu_path: Path) -> dict[str, float]:
+    points, arrays = load_vtu(vtu_path)
+    sigma = arrays.get("sigma")
+    swelling = arrays.get("swelling_stress")
+    displacement = arrays.get("displacement")
+    saturation = arrays.get("saturation")
+    porosity = arrays.get("porosity")
+    dry_density = arrays.get("dry_density_solid")
+
+    top = boundary_mask(points, axis=1, side="max")
+    side = boundary_mask(points, axis=0, side="max")
+
+    metrics: dict[str, float] = {
+        "final_total_stress_mpa": mean_isotropic_pressure_mpa(sigma)
+        if sigma is not None
+        else math.nan,
+        "final_swelling_pressure_mpa": mean_isotropic_pressure_mpa(swelling)
+        if swelling is not None
+        else math.nan,
+        "axial_sigma_kpa": abs(mean_component(sigma, top, 1)) / 1000.0
+        if sigma is not None
+        else math.nan,
+        "radial_sigma_kpa": abs(mean_component(sigma, side, 0)) / 1000.0
+        if sigma is not None
+        else math.nan,
+        "axial_swelling_stress_kpa": abs(mean_component(swelling, top, 1)) / 1000.0
+        if swelling is not None
+        else math.nan,
+        "radial_swelling_stress_kpa": abs(mean_component(swelling, side, 0)) / 1000.0
+        if swelling is not None
+        else math.nan,
+        "axial_displacement_mm": mean_component(displacement, top, 1) * 1000.0
+        if displacement is not None
+        else math.nan,
+        "radial_displacement_mm": mean_component(displacement, side, 0) * 1000.0
+        if displacement is not None
+        else math.nan,
+        "saturation_mean": float(np.asarray(saturation).mean())
+        if saturation is not None
+        else math.nan,
+        "porosity_mean": float(np.asarray(porosity).mean())
+        if porosity is not None
+        else math.nan,
+        "dry_density_mean_kg_m3": float(np.asarray(dry_density).mean())
+        if dry_density is not None
+        else math.nan,
+    }
+    return metrics
+
+
+def collect_history(output_dir: Path, prefix: str) -> list[dict[str, float]]:
+    files = sorted(output_dir.glob(f"{prefix}_ts_w_t*.vtu"), key=parse_time_from_vtu)
+    history: list[dict[str, float]] = []
+    for path in files:
+        t = parse_time_from_vtu(path)
+        metrics = evaluate_metrics(path)
+        history.append(
+            {
+                "time_s": t,
+                "time_d": t / 86400.0,
+                "total_stress_mpa": metrics["final_total_stress_mpa"],
+                "swelling_pressure_mpa": metrics["final_swelling_pressure_mpa"],
+            }
+        )
+    return history
+
+
+def run_ogs(ogs_bin: Path, project_path: Path, output_dir: Path) -> None:
+    output_dir.mkdir(parents=True, exist_ok=True)
+    result = subprocess.run(
+        [str(ogs_bin), "-o", str(output_dir), str(project_path)],
+        cwd=project_path.parent,
+        text=True,
+        capture_output=True,
+        check=False,
+    )
+    if result.returncode != 0:
+        raise RuntimeError(
+            f"OGS failed for {project_path.name} with {ogs_bin}\n"
+            f"stdout:\n{result.stdout}\n"
+            f"stderr:\n{result.stderr}"
+        )
+
+
+def write_rows_csv(rows: list[dict[str, object]], path: Path) -> None:
+    path.parent.mkdir(parents=True, exist_ok=True)
+    keys: list[str] = []
+    seen: set[str] = set()
+    for row in rows:
+        for key in row:
+            if key not in seen:
+                seen.add(key)
+                keys.append(key)
+    with path.open("w", newline="", encoding="utf-8") as f:
+        writer = csv.DictWriter(f, fieldnames=keys)
+        writer.writeheader()
+        writer.writerows(rows)
+
+
+def pairwise_deltas(rows: list[dict[str, object]]) -> list[dict[str, float]]:
+    by_impl = {str(r["implementation"]): r for r in rows}
+    lin = by_impl["linear_mcc"]
+    nat = by_impl["native_dsm_mcc"]
+    mfr = by_impl["mfront_dsm_mcc"]
+
+    def val(r: dict[str, object], key: str) -> float:

```

```

+         return float(r.get(key, math.nan))
+
+     p_lin = val(lin, "final_swelling_pressure_mpa")
+     p_nat = val(nat, "final_swelling_pressure_mpa")
+     p_mfr = val(mfr, "final_swelling_pressure_mpa")
+     s_lin = val(lin, "final_total_stress_mpa")
+     s_nat = val(nat, "final_total_stress_mpa")
+     s_mfr = val(mfr, "final_total_stress_mpa")
+
+     return [
+         {
+             "quantity": "swelling_pressure_mpa",
+             "linear_mcc": p_lin,
+             "native_dsm_mcc": p_nat,
+             "mfront_dsm_mcc": p_mfr,
+             "delta_native_minus_linear": p_nat - p_lin,
+             "delta_mfront_minus_linear": p_mfr - p_lin,
+             "delta_native_minus_mfront": p_nat - p_mfr,
+             "rel_native_vs_linear": abs(p_nat - p_lin) / max(abs(p_lin), 1e-12),
+             "rel_mfront_vs_linear": abs(p_mfr - p_lin) / max(abs(p_lin), 1e-12),
+             "rel_native_vs_mfront": abs(p_nat - p_mfr) / max(abs(p_mfr), 1e-12),
+         },
+         {
+             "quantity": "total_stress_mpa",
+             "linear_mcc": s_lin,
+             "native_dsm_mcc": s_nat,
+             "mfront_dsm_mcc": s_mfr,
+             "delta_native_minus_linear": s_nat - s_lin,
+             "delta_mfront_minus_linear": s_mfr - s_lin,
+             "delta_native_minus_mfront": s_nat - s_mfr,
+             "rel_native_vs_linear": abs(s_nat - s_lin) / max(abs(s_lin), 1e-12),
+             "rel_mfront_vs_linear": abs(s_mfr - s_lin) / max(abs(s_lin), 1e-12),
+             "rel_native_vs_mfront": abs(s_nat - s_mfr) / max(abs(s_mfr), 1e-12),
+         },
+     ]
+
+
+def plot_results(rows: List[dict[str, object]], histories: dict[str, List[dict[str, float]]], out_png: Path) -> None:
+    impl_order = ["linear_mcc", "native_dsm_mcc", "mfront_dsm_mcc"]
+    labels = ["Linear+MCC (11 MPa)", "Native DSM+MCC", "MFront DSM+MCC"]
+    p = [float(next(r for r in rows if r["implementation"] == impl))["final_swelling_pressure_mpa"] for impl in impl_order]
+
+    fig, axes = plt.subplots(1, 2, figsize=(12, 4.6))
+
+    x = np.arange(len(labels))
+    axes[0].bar(x, p, color=["#4e79a7", "#f28e2b", "#59a14f"])
+    axes[0].set_xticks(x, labels, rotation=12, ha="right")
+    axes[0].set_ylabel("Final swelling pressure (MPa)")
+    axes[0].set_title("D0=1700 final comparison")
+    axes[0].grid(True, axis="y", ls=":", alpha=0.4)
+
+    for impl, color, label in zip(
+        impl_order,
+        ["#4e79a7", "#f28e2b", "#59a14f"],
+        labels,
+    ):
+        hist = histories.get(impl, [])
+        if not hist:
+            continue
+        t = np.array([r["time_d"] for r in hist], dtype=float)
+        y = np.array([r["total_stress_mpa"] for r in hist], dtype=float)
+        axes[1].plot(t, y, marker="o", ms=3, lw=1.2, color=color, label=label)
+    axes[1].set_xlabel("Time (days)")
+    axes[1].set_ylabel("Mean total stress (MPa)")
+    axes[1].set_title("Stress evolution")
+    axes[1].grid(True, ls=":", alpha=0.4)
+    axes[1].legend(loc="best")
+
+    fig.tight_layout()
+    out_png.parent.mkdir(parents=True, exist_ok=True)
+    fig.savefig(out_png, dpi=220)
+    plt.close(fig)
+
+def main() -> None:
+    args = parse_args()
+    args.out_root.mkdir(parents=True, exist_ok=True)
+
+    mfront_mod = load_module(
+        ANCHORS_DIR / "run_villar_dense_dd_calibration.py",
+        "run_villar_dense_dd_calibration_dd1700_triplet",
+    )
+    native_mod = load_module(
+        ANCHORS_DIR / "run_villar_dense_dd_native_dsm_micromacro_calibration.py",
+        "run_villar_dense_dd_native_dsm_micromacro_calibration_dd1700_triplet",
+    )
+
+    case_m = mfront_mod.Case(float(args.dry_density))
+    case_n = native_mod.Case(float(args.dry_density))
+    target_mpa = float(case_m.villar_target_swelling_mpa)
+
+    mfront_cal = mfront_mod.calibrate_multiplier_for_case(
+        args.mfront_ogs, case_m, target_mpa, rel_tol=args.rel_tol
+    )
+    n_l0_native = native_mod.n_l0_from_micro_suction(case_n.phi0, native_mod.HAMAKER_LITERATURE)
+    native_cal = native_mod.calibrate_multiplier_for_case(
+        args.native_ogs, case_n, n_l0_native, target_mpa, rel_tol=args.rel_tol
+    )
+
+    runtime_prefix = f"dd(int(args.dry_density))_linear_mcc_triplet"
+    native_dsm_project = ANCHORS_DIR / f"{runtime_prefix}_native_dsm.prj"
+    mfront_dsm_project = ANCHORS_DIR / f"{runtime_prefix}_mfront_dsm.prj"
+    linear_mcc_project = ANCHORS_DIR / f"{runtime_prefix}_linear_mcc.prj"
+
+    native_prefix = f"{runtime_prefix}_native_dsm"
+    mfront_prefix = f"{runtime_prefix}_mfront_dsm"
+    linear_prefix = f"{runtime_prefix}_linear_mcc"
+
+    native_mod.write_native_dsm_micromacro_project(
+        case_n,
+        native_dsm_project,
+        float(native_cal["multiplier"]),
+        float(n_l0_native),
+    )
+    mfront_mod.write_mfront_project(
+        case_m,
+        float(mfront_cal["multiplier"]),

```

```

+         float(mfront_cal.get("n_l0", mfront_mod.n_l0_from_micro_suction(case_m.phi0, mfront_mod.HAMAKER_LITERATURE))),
+         mfront_dsm_project,
+     )
+     configure_linear_mcc_project(
+         native_dsm_project,
+         linear_mcc_project,
+         linear_prefix,
+         float(args.max_swelling_pressure_pa),
+     )
+
+     # Ensure output prefixes are unique.
+     native_root = ET.parse(native_dsm_project).getroot()
+     set_output_prefix(native_root, native_prefix)
+     ET.ElementTree(native_root).write(
+         native_dsm_project, encoding="ISO-8859-1", xml_declaration=True
+     )
+
+     mfront_root = ET.parse(mfront_dsm_project).getroot()
+     set_output_prefix(mfront_root, mfront_prefix)
+     ET.ElementTree(mfront_root).write(
+         mfront_dsm_project, encoding="ISO-8859-1", xml_declaration=True
+     )
+
+     run_specs = [
+         RunSpec("linear_mcc", linear_mcc_project, args.native_ogs, linear_prefix),
+         RunSpec("native_dsm_mcc", native_dsm_project, args.native_ogs, native_prefix),
+         RunSpec("mfront_dsm_mcc", mfront_dsm_project, args.mfront_ogs, mfront_prefix),
+     ]
+
+     rows: list[dict[str, object]] = []
+     histories: dict[str, list[dict[str, float]]] = {}
+     run_root = args.out_root / "runs"
+     for spec in run_specs:
+         out_dir = run_root / spec.implementation
+         run_ogs(spec.ogs_bin, spec.project_path, out_dir)
+         final_vtu = latest_vtu(out_dir, spec.output_prefix)
+         metrics = evaluate_metrics(final_vtu)
+         histories[spec.implementation] = collect_history(out_dir, spec.output_prefix)
+         rows.append(
+             {
+                 "implementation": spec.implementation,
+                 "dry_density_kg_m3": float(args.dry_density),
+                 "max_swelling_pressure_pa_linear": float(args.max_swelling_pressure_pa)
+                 if spec.implementation == "linear_mcc"
+                 else math.nan,
+                 "vdw_multiplier": math.nan
+                 if spec.implementation == "linear_mcc"
+                 else (
+                     float(native_cal["multiplier"])
+                     if spec.implementation == "native_dsm_mcc"
+                     else float(mfront_cal["multiplier"])
+                 ),
+                 "hamaker_effective_j": math.nan
+                 if spec.implementation == "linear_mcc"
+                 else (
+                     float(native_cal["hamaker_effective_j"])
+                     if spec.implementation == "native_dsm_mcc"
+                     else float(mfront_cal["hamaker_effective"])
+                 ),
+                 "calibration_target_mpa": target_mpa,
+                 "output_vtu": str(final_vtu.resolve()),
+                 **metrics,
+             }
+         )
+
+     deltas = pairwise_deltas(rows)
+
+     rows_csv = args.out_root / "dd1700_linear_mcc_dsm_runs_summary.csv"
+     deltas_csv = args.out_root / "dd1700_linear_mcc_dsm_pairwise_deltas.csv"
+     history_csv = args.out_root / "dd1700_linear_mcc_dsm_history.csv"
+     figure_png = args.out_root / "dd1700_linear_mcc_dsm_comparison.png"
+     summary_json = args.out_root / "dd1700_linear_mcc_dsm_summary.json"
+
+     write_rows_csv(rows, rows_csv)
+     write_rows_csv(deltas, deltas_csv)
+     flat_history = []
+     for impl, hist in histories.items():
+         for row in hist:
+             flat_history.append({"implementation": impl, **row})
+     write_rows_csv(flat_history, history_csv)
+     plot_results(rows, histories, figure_png)
+
+     summary = {
+         "mode": "dd1700_linear_mcc_vs_dsm_triplet",
+         "dry_density_kg_m3": float(args.dry_density),
+         "max_swelling_pressure_pa_linear": float(args.max_swelling_pressure_pa),
+         "calibration_target_mpa_villar": target_mpa,
+         "native_calibration": {
+             "multiplier": float(native_cal["multiplier"]),
+             "pressure_mpa": float(native_cal["pressure_mpa"]),
+             "hamaker_effective_j": float(native_cal["hamaker_effective_j"]),
+             "n_l0": float(native_cal["n_l0"]),
+         },
+         "mfront_calibration": {
+             "multiplier": float(mfront_cal["multiplier"]),
+             "pressure_mpa": float(mfront_cal["pressure_mpa"]),
+             "hamaker_effective_j": float(mfront_cal["hamaker_effective"]),
+             "n_l0": float(mfront_cal["n_l0"]),
+         },
+         "runs": rows,
+         "pairwise_deltas": deltas,
+         "git_hashes": {
+             "mfront_repo_hash": git_short_hash(ROOT.parent.parent.parent),
+             "native_repo_hash": git_short_hash(Path("/Users/vinaykumar/git/ogs-native-dsm-transition")),
+         },
+         "artifacts": {
+             "rows_csv": str(rows_csv.resolve()),
+             "deltas_csv": str(deltas_csv.resolve()),
+             "history_csv": str(history_csv.resolve()),
+             "figure_png": str(figure_png.resolve()),
+         },
+     }
+
+     summary_json.write_text(json.dumps(summary, indent=2) + "\n", encoding="utf-8")
+
+     print(f"Wrote rows CSV: {rows_csv}")
+     print(f"Wrote pairwise deltas CSV: {deltas_csv}")
+     print(f"Wrote history CSV: {history_csv}")

```

```

+ print(f"Wrote plot: {figure_png}")
+ print(f"Wrote summary JSON: {summary_json}")
+
+ if not args.keep_runtime:
+     for project in (native_dsm_project, mfront_dsm_project, linear_mcc_project):
+         project.unlink(missing_ok=True)
+     for p in ANCHORS_DIR.glob(f"({runtime_prefix})_*_pvd"):
+         p.unlink(missing_ok=True)
+     for p in ANCHORS_DIR.glob(f"({runtime_prefix})_*_ts_*_t_*_vtu"):
+         p.unlink(missing_ok=True)
+     shutil.rmtree(run_root, ignore_errors=True)
+
+
+if __name__ == "__main__":
+    main()
diff --git a/Tests/Data/RichardsMechanics/run_native_branch_dsm_micromacro_comparison.py b/Tests/Data/RichardsMechanics/run_native_branch_dsm_micromacro_comparison.py
new file mode 100644
index 0000000000..216a593fd9
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/run_native_branch_dsm_micromacro_comparison.py
@@ -0,0 +1,355 @@
#!/usr/bin/env python3
+"""Compare native dsm_micromacro branch and MFront bridge outputs on BEACON cases.
+
+The script executes paired projects, extracts final-state stress/density
+metrics from VTK outputs, computes field-wise maximum absolute differences, and
+writes CSV/JSON/PNG artifacts for parity tracking.
+"""
+
+from __future__ import annotations
+
+import argparse
+import csv
+import json
+import math
+import os
+import subprocess
+import tempfile
+import xml.etree.ElementTree as ET
+from dataclasses import dataclass
+from pathlib import Path
+
+import numpy as np
+import vtk
+from vtk.util.numpy_support import vtk_to_numpy
+
+os.environ.setdefault("MPLCONFIGDIR", tempfile.mkdtemp(prefix="mplconfig-"))
+os.environ.setdefault("XDG_CACHE_HOME", tempfile.mkdtemp(prefix="xdg-cache-"))
+
+import matplotlib
+
+matplotlib.use("Agg")
+import matplotlib.pyplot as plt
+
+
+HERE = Path(__file__).resolve().parent
+DEFAULT_NATIVE_SOURCE = Path("/Users/vinaykumar/git/ogs-native-dsm-transition")
+DEFAULT_MFRONT_SOURCE = HERE.parents[2]
+DEFAULT_NATIVE_OGS = Path("/Users/vinaykumar/git/build/release-native-beacon/bin/ogs")
+DEFAULT_MFRONT_OGS = Path("/Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs")
+
+
+@dataclass(frozen=True)
+class CaseConfig:
+    native_project: str
+    mfront_project: str
+
+
+CASE_CONFIGS: dict[str, CaseConfig] = {
+    "1a01_smoke": CaseConfig(
+        native_project="beacon_1a01_dsm_micromacro_smoke.prj",
+        mfront_project="beacon_1a01_dsm_micromacro_mcc_bridge.prj",
+    ),
+    "1b_smoke": CaseConfig(
+        native_project="beacon_1b_dsm_micromacro_smoke.prj",
+        mfront_project="beacon_1b_dsm_micromacro_mcc_bridge.prj",
+    ),
+    "1c_smoke": CaseConfig(
+        native_project="beacon_1c_dsm_micromacro_smoke.prj",
+        mfront_project="beacon_1c_dsm_micromacro_mcc_bridge.prj",
+    ),
+    "1a01_inflow": CaseConfig(
+        native_project="beacon_1a01_dsm_micromacro_inflow.prj",
+        mfront_project="beacon_1a01_dsm_micromacro_mcc_inflow_bridge.prj",
+    ),
+}
+
+
+def run_ogs(executable: Path, project: Path, output_dir: Path) -> None:
+    """Run OGS for one project and capture logs for reproducible failure diagnosis."""
+    output_dir.mkdir(parents=True, exist_ok=True)
+    result = subprocess.run(
+        [str(executable), "-o", str(output_dir), str(project)],
+        cwd=project.parent,
+        text=True,
+        capture_output=True,
+        check=False,
+    )
+    if result.returncode != 0:
+        raise RuntimeError(
+            f"ogs failed for {project} with {executable}\n"
+            f"stdout:\n{result.stdout}\n"
+            f"stderr:\n{result.stderr}"
+        )
+
+
+def project_output_prefix(project: Path) -> str:
+    root = ET.parse(project).getroot()
+    prefix = root.findtext("./time_loop/output/prefix")
+    if prefix is None:
+        raise RuntimeError(f"No output prefix in {project}")
+    return prefix.strip()
+
+
+def parse_time_from_vtu(path: Path) -> float:
+    marker = "_t_"
+    stem = path.stem

```

```

+     if marker not in stem:
+         raise RuntimeError(f"Cannot parse output time from {path.name}")
+     return float(stem.split(marker, 1)[1])
+
+
+def latest_vtu(output_dir: Path, prefix: str) -> Path:
+    files = sorted(output_dir.glob(f"{prefix}_t*.vtu"), key=parse_time_from_vtu)
+    if not files:
+        raise FileNotFoundError(f"No VTU outputs in {output_dir} for prefix {prefix}")
+    return files[-1]
+
+
+def load_vtu(path: Path) -> tuple[np.ndarray, dict[str, np.ndarray]]:
+    reader = vtk.vtkXMLUnstructuredGridReader()
+    reader.SetFileName(str(path))
+    reader.Update()
+    grid = reader.GetOutput()
+    points = np.array([grid.GetPoint(i) for i in range(grid.GetNumberOfPoints())])
+    arrays: dict[str, np.ndarray] = {}
+    for i in range(grid.GetPointData().GetNumberOfArrays()):
+        array = grid.GetPointData().GetArray(i)
+        if array is None:
+            continue
+        name = grid.GetPointData().GetArrayName(i)
+        if name is None:
+            continue
+        arrays[name] = vtk_to_numpy(array)
+    return points, arrays
+
+
+def boundary_mean(points: np.ndarray, values: np.ndarray, axis: int, side: str) -> np.ndarray:
+    boundary_value = points[:, axis].max() if side == "max" else points[:, axis].min()
+    mask = np.isclose(points[:, axis], boundary_value, atol=1e-10)
+    return np.asarray(values[mask]).mean(axis=0)
+
+
+def component_or_scalar(arr: np.ndarray, preferred_index: int) -> float:
+    values = np.asarray(arr).reshape(-1)
+    if values.size == 0:
+        return float("nan")
+    idx = preferred_index if preferred_index < values.size else 0
+    return float(values[idx])
+
+
+def evaluate_metrics(points: np.ndarray, arrays: dict[str, np.ndarray]) -> dict[str, float]:
+    """Extract comparable boundary stress and scalar-field means from final VTU data."""
+    sigma = arrays.get("sigma")
+    swelling = arrays.get("swelling_stress")
+    dry_density = arrays.get("dry_density_solid")
+
+    sigma_top = boundary_mean(points, sigma, axis=1, side="max") if sigma is not None else np.array([np.nan, np.nan])
+    sigma_side = boundary_mean(points, sigma, axis=0, side="max") if sigma is not None else np.array([np.nan, np.nan])
+    swell_top = (
+        boundary_mean(points, swelling, axis=1, side="max")
+        if swelling is not None
+        else np.array([np.nan, np.nan])
+    )
+    swell_side = (
+        boundary_mean(points, swelling, axis=0, side="max")
+        if swelling is not None
+        else np.array([np.nan, np.nan])
+    )
+
+    metrics: dict[str, float] = {
+        "axial_sigma_kpa": abs(component_or_scalar(sigma_top, 1)) / 1000.0,
+        "radial_sigma_kpa": abs(component_or_scalar(sigma_side, 0)) / 1000.0,
+        "axial_swelling_stress_kpa": abs(component_or_scalar(swell_top, 1)) / 1000.0,
+        "radial_swelling_stress_kpa": abs(component_or_scalar(swell_side, 0)) / 1000.0,
+        "dry_density_mean_kg_m3": float(np.asarray(dry_density).mean()) if dry_density is not None else float("nan"),
+    }
+
+    optional_scalar_fields = [
+        "pressure",
+        "saturation",
+        "micro_pressure",
+        "micro_saturation",
+        "micro_water_content",
+        "micro_porosity",
+        "micro_exchange_source",
+        "phi_m",
+        "phi_M",
+        "n_l",
+    ]
+    for field in optional_scalar_fields:
+        data = arrays.get(field)
+        if data is not None:
+            metrics[f"{field}_mean"] = float(np.asarray(data).mean())
+    return metrics
+
+
+def max_abs_diff(a: np.ndarray, b: np.ndarray) -> float:
+    return float(np.abs(np.asarray(a) - np.asarray(b)).max())
+
+
+def compare_fields(
+    native_points: np.ndarray,
+    native_data: dict[str, np.ndarray],
+    mfront_points: np.ndarray,
+    mfront_data: dict[str, np.ndarray],
+) -> dict[str, float]:
+    """Return max-abs field differences on shared point ordering."""
+    if not np.allclose(native_points, mfront_points):
+        raise RuntimeError("Point coordinates differ between native and MFront outputs.")
+
+    diffs: dict[str, float] = {}
+    candidate_fields = [
+        "pressure",
+        "saturation",
+        "sigma",
+        "swelling_stress",
+        "dry_density_solid",
+    ]
+    for field in candidate_fields:
+        if field in native_data and field in mfront_data:
+            diffs[field] = max_abs_diff(native_data[field], mfront_data[field])
+    return diffs

```

```

+
+def write_csv(rows: List[dict[str, object]], path: Path) -> None:
+    columns = [
+        "case",
+        "native_axial_sigma_kpa",
+        "mfront_axial_sigma_kpa",
+        "native_radial_sigma_kpa",
+        "mfront_radial_sigma_kpa",
+        "native_dry_density_mean_kg_m3",
+        "mfront_dry_density_mean_kg_m3",
+        "native_pressure_mean",
+        "mfront_pressure_mean",
+        "max_abs_diff_pressure",
+        "max_abs_diff_sigma",
+        "max_abs_diff_swelling_stress",
+    ]
+    with path.open("w", newline="", encoding="utf-8") as f:
+        writer = csv.DictWriter(f, fieldnames=columns)
+        writer.writeheader()
+        for row in rows:
+            writer.writerow(row)
+
+
+def make_stress_plot(rows: List[dict[str, object]], output_png: Path) -> None:
+    cases = [str(r["case"]) for r in rows]
+    x = np.arange(len(cases))
+    width = 0.35
+
+    native_axial = np.array([float(r["native_axial_sigma_kpa"]) for r in rows])
+    mfront_axial = np.array([float(r["mfront_axial_sigma_kpa"]) for r in rows])
+    native_radial = np.array([float(r["native_radial_sigma_kpa"]) for r in rows])
+    mfront_radial = np.array([float(r["mfront_radial_sigma_kpa"]) for r in rows])
+
+    fig, axes = plt.subplots(2, 1, figsize=(9, 7), sharex=True)
+
+    axes[0].bar(x - width / 2, native_axial, width=width, label="native branch")
+    axes[0].bar(x + width / 2, mfront_axial, width=width, label="Mfront bridge branch")
+    axes[0].set_ylabel("axial stress [kPa]")
+    axes[0].grid(axis="y", alpha=0.3)
+    axes[0].legend()
+
+    axes[1].bar(x - width / 2, native_radial, width=width, label="native branch")
+    axes[1].bar(x + width / 2, mfront_radial, width=width, label="Mfront bridge branch")
+    axes[1].set_ylabel("radial stress [kPa]")
+    axes[1].set_xticks(x, cases, rotation=15, ha="right")
+    axes[1].grid(axis="y", alpha=0.3)
+
+    fig.tight_layout()
+    output_png.parent.mkdir(parents=True, exist_ok=True)
+    fig.savefig(output_png, dpi=200)
+    plt.close(fig)
+
+
+def main() -> None:
+    parser = argparse.ArgumentParser()
+    parser.add_argument("--native-source-dir", type=Path, default=DEFAULT_NATIVE_SOURCE)
+    parser.add_argument("--mfront-source-dir", type=Path, default=DEFAULT_MFRONT_SOURCE)
+    parser.add_argument("--native-ogs", type=Path, default=DEFAULT_NATIVE_OGS)
+    parser.add_argument("--mfront-ogs", type=Path, default=DEFAULT_MFRONT_OGS)
+    parser.add_argument(
+        "--output-root",
+        type=Path,
+        required=True,
+        help="Transient runtime output folder for VTU files.",
+    )
+    parser.add_argument("--json-out", type=Path, required=True)
+    parser.add_argument("--csv-out", type=Path, required=True)
+    parser.add_argument("--plot-out", type=Path, required=True)
+    args = parser.parse_args()
+
+    native_data_dir = args.native_source_dir / "Tests" / "Data" / "RichardsMechanics"
+    mfront_data_dir = args.mfront_source_dir / "Tests" / "Data" / "RichardsMechanics"
+
+    rows: List[dict[str, object]] = []
+    report: dict[str, object] = {
+        "native_source_dir": str(args.native_source_dir.resolve()),
+        "mfront_source_dir": str(args.mfront_source_dir.resolve()),
+        "native_ogs": str(args.native_ogs.resolve()),
+        "mfront_ogs": str(args.mfront_ogs.resolve()),
+        "cases": {},
+    }
+
+    for case_name, case in CASE_CONFIGS.items():
+        native_project = native_data_dir / case.native_project
+        mfront_project = mfront_data_dir / case.mfront_project
+        native_run_dir = args.output_root / case_name / "native"
+        mfront_run_dir = args.output_root / case_name / "mfront"
+
+        run_ogs(args.native_ogs, native_project, native_run_dir)
+        run_ogs(args.mfront_ogs, mfront_project, mfront_run_dir)
+
+        native_prefix = project_output_prefix(native_project)
+        mfront_prefix = project_output_prefix(mfront_project)
+        native_vtu = latest_vtu(native_run_dir, native_prefix)
+        mfront_vtu = latest_vtu(mfront_run_dir, mfront_prefix)
+
+        native_points, native_arrays = load_vtu(native_vtu)
+        mfront_points, mfront_arrays = load_vtu(mfront_vtu)
+
+        native_metrics = evaluate_metrics(native_points, native_arrays)
+        mfront_metrics = evaluate_metrics(mfront_points, mfront_arrays)
+        diffs = compare_fields(native_points, native_arrays, mfront_points, mfront_arrays)
+
+        report_case = {
+            "native_project": str(native_project),
+            "mfront_project": str(mfront_project),
+            "native_vtu": str(native_vtu),
+            "mfront_vtu": str(mfront_vtu),
+            "native_metrics": native_metrics,
+            "mfront_metrics": mfront_metrics,
+            "max_abs_diff": diffs,
+        }
+        casted_cases = report["cases"]
+        assert isinstance(casted_cases, dict)
+        casted_cases[case_name] = report_case
+
+    rows.append(

```



```

+
+    {
+        "case": case_name,
+        "native_axial_sigma_kpa": native_metrics["axial_sigma_kpa"],
+        "mfront_axial_sigma_kpa": mfront_metrics["axial_sigma_kpa"],
+        "native_radial_sigma_kpa": native_metrics["radial_sigma_kpa"],
+        "mfront_radial_sigma_kpa": mfront_metrics["radial_sigma_kpa"],
+        "native_dry_density_mean_kg_m3": native_metrics["dry_density_mean_kg_m3"],
+        "mfront_dry_density_mean_kg_m3": mfront_metrics["dry_density_mean_kg_m3"],
+        "native_pressure_mean": native_metrics.get("pressure_mean", math.nan),
+        "mfront_pressure_mean": mfront_metrics.get("pressure_mean", math.nan),
+        "max_abs_diff_pressure": diffs.get("pressure", math.nan),
+        "max_abs_diff_sigma": diffs.get("sigma", math.nan),
+        "max_abs_diff_swelling_stress": diffs.get("swelling_stress", math.nan),
+    }
+
+
+    args.json_out.write_text(json.dumps(report, indent=2, sort_keys=True) + "\n", encoding="utf-8")
+    write_csv(rows, args.csv_out)
+    make_stress_plot(rows, args.plot_out)
+
+    print(json.dumps({"json_out": str(args.json_out), "csv_out": str(args.csv_out), "plot_out": str(args.plot_out)}, indent=2))
+
+
+if __name__ == "__main__":
+    main()
+
diff --git a/Tests/Data/RichardsMechanics/test_run_table.csv b/Tests/Data/RichardsMechanics/test_run_table.csv
new file mode 100644
index 0000000000..bf25e12eed
--- /dev/null
+++ b/Tests/Data/RichardsMechanics/test_run_table.csv
@@ -0,0 +1,6 @@
+suite,artifact_folder,tracked_artifacts,project_files,generator_scripts,ogs_repo_hash,materialmodels_repo_hash,run_date_utc,notes
+BEACON_report_figures,Tests/Data/RichardsMechanics/BEACON_report_figures,"beacon_1a01_report_style_panel.png;beacon_1b_report_style_panel.png;beacon_1c_report_style_panel.png;beac
+on_1a01_series.csv;beacon_1b_series.csv;beacon_1c_zone_series.csv;beacon_1a01_profile.csv;beacon_1b_profile.csv;beacon_1c_profile.csv;beacon_report_plot_summary.json","1a01/1a01_na
+tive_reportplot.prj;1a01/1a01_bridge_reportplot.prj;1b/1b_native_reportplot.prj;1b/1b_bridge_reportplot.prj;1c/1c_bridge_reportplot.prj","Tests/Data/RichardsMechanics/run_beacon_na
+tive_bridge_report_batch.py;Tests/Data/RichardsMechanics/plot_beacon_report_figures.py",1adfb44269,3d1eefa,2026-04-01,"First benchmark set rerun with native binary /Users/vinaykuma
+r/git/build/release-native-beacon/bin/ogs and bridge binary /Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs."
+BEACON_native_branch_notebook_validation,Tests/Data/RichardsMechanics/BEACON_report_figures,"native_branch_notebook_comparison_report.json;native_branch_notebook_comparison_summar
+y.csv;native_branch_notebook_stress_compare.png","native branch: beacon_1a01_notebook_smoke.prj;beacon_1b_notebook_smoke.prj;beacon_1c_notebook_smoke.prj;beacon_1a01_notebook_infl
+o.w.prj | mfront branch: beacon_1a01_notebook_mcc_bridge.prj;beacon_1b_notebook_mcc_bridge.prj;beacon_1c_notebook_mcc_bridge.prj;beacon_1a01_notebook_mcc_inflow_bridge.prj","Tests/Da
+ta/RichardsMechanics/run_native_branch_notebook_comparison.py",f0a453fb89,3d1eefa,2026-04-01,"Native notebook branch hash d46e11ac00 tested with /Users/vinaykumar/git/build/release
+native-beacon/bin/ogs against MFront branch hash f0a453fb89 with /Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs; smoke decks match at zero stress, inflow deck shows -604 k
+Pa stress mismatch."
+BEACON_wp3_BGR_EPFL,Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL,"bgr_wp3_p2_1_abprime_fig4_5b_like.png;bgr_wp3_p2_1_abprime_fig4_6_like.png;bgr_wp3_p2_1_abprime_report_style
+panel.png;bgr_wp3_p2_1_abprime_report_style_series.csv;bgr_wp3_p2_1_abprime_report.json","bgr_wp3_p2_1_abprime_native.prj;bgr_wp3_p2_1_abprime_mfront.prj","Tests/Data/RichardsMecha
+nics/BEACON_WP3_BGR_EPFL/analyze_bgr_wp3_abprime.py;Tests/Data/RichardsMechanics/BEACON_WP3_BGR_EPFL/plot_bgr_wp3_epfl_report_figures.py",1adfb44269,3d1eefa,2026-04-01,"First bench
+mark set rerun with /Users/vinaykumar/git/build/release-mfront-tpm/bin/ogs for both native and MFront EPFL decks (native-beacon binary lacks MFront support)."
+ANCHORS_MS33_ModelI,Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI,"ms33_model_i_history.csv;ms33_model_i_summary.json;ms33_vs_ciemat_swelling_pressure_vs_dry_density.csv;ms33_v
+s_ciemat_swelling_pressure_vs_dry_density.png;villar_dense_dd_calibration.csv;villar_dense_dd_calibration_summary.json;villar_dense_dd_swelling_pressure_comparison.png;villar_dense
+dd_vdw_multiplier.png;villar_dense_dd_native_notebook_branch.csv;villar_dense_dd_native_notebook_branch_summary.json;villar_dense_dd_native_notebook_branch_vs_villar.png;villar_de
+nse_dd_native_notebook_calibration.csv;villar_dense_dd_native_notebook_calibration_summary.json;villar_dense_dd_native_notebook_calibration_comparison.png;villar_dense_dd_native_no
+tebook_vdw_multiplier.png;villar_dense_dd_native_vs_mfront_calibrated_comparison.csv;villar_dense_dd_native_vs_mfront_calibrated_comparison_summary.json","ms33_model_i_dd1400.prj;m
+s33_model_i_dd1600.prj;ms33_model_i_dd1800.prj","Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/summarize_ms33_model_i.py;Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/plot_ms
+33_vs_ciemat_swelling_pressure.dd.py;Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_calibration.py;Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar
+_dense_dd_native_notebook_branch.py;Tests/Data/RichardsMechanics/ANCHORS_MS33_ModelI/run_villar_dense_dd_native_notebook_calibration.py",a6431b12c6,aa696a8,2026-04-01,"MS33 scaffold
+plus dense (25 kg/m3 spacing) CIEMAT/Villar sweeps rerun after native notebook mass-storage alignment (native branch hash d46e11ac00). Native baseline mean relative error 98.69%;
+native calibrated mean relative error 1.26% (max 1.83%). Native-vs-MFront calibrated pressure gap: mean 0.135 MPa, max 0.467 MPa. Parameter-level multiplier scale mismatch remains
+."
+ANCHORS_embedded_linear,Tests/Data/Mechanics/Linear/EmbeddedAnchorSourceTerm,"ms33_one_anchor.json","ms33_no_anchor.prj;ms33_one_anchor.prj","manual project setup",1adfb44269,3d1e
+efa,2026-04-01,"Minimal anchor source-term setup used as mechanical anchor scaffold."
diff --git a/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/.gitignore b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/.gitignore
new file mode 100644
index 0000000000..dcffccb5b5
--- /dev/null
+++ b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/.gitignore
@@ -0,0 +1,6 @@
+ogs.log
+out/
+saturation_mtest.res
+src/*.d
+src/*.o
+src/*.so
diff --git a/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/include/MFront/GenericBehaviour/ModCamClay_semiExpl_coupled-generic.hxx b/Tests/Data/ThermoRichardsMechanics/
MFront/TRM_MCC_coupled/include/MFront/GenericBehaviour/ModCamClay_semiExpl_coupled-generic.hxx
new file mode 100644
index 0000000000..37976eff0f
--- /dev/null
+++ b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/include/MFront/GenericBehaviour/ModCamClay_semiExpl_coupled-generic.hxx
@@ -0,0 +1,96 @@
+/*!
+
+ * \file ModCamClay_semiExpl_coupled-generic.hxx
+ * \brief This file declares the umat interface for the ModCamClay_semiExpl_coupled behaviour law
+ * \author Aqeel Afzal Chaudhry , Christian Silbermann , Eric Simo , Miguel Mánica , Thomas Helfer , Thomas Nagel
+ * \date 01 / 09 / 2025
+
+ */
+
+#ifndef LTB_GENERIC_MODCAMCLAY_SEMIEXPL_COUPLED_HXX
+#define LTB_GENERIC_MODCAMCLAY_SEMIEXPL_COUPLED_HXX
+
+#include "TFEL/Config/TFELConfig.hxx"
+#include "MFront/GenericBehaviour/BehaviourData.h"
+
+#ifndef _WIN32
+#ifndef NOMINMAX
+#define NOMINMAX
+#endif /* NOMINMAX */
+#include <windows.h>
+#ifndef small
+#undef small
+#endif /* small */
+#endif /* _WIN32 */
+
+#ifndef MFRONT_SHARED0BJ
+#define MFRONT_SHARED0BJ TFEL_VISIBILITY_EXPORT
+#endif /* MFRONT_SHARED0BJ */
+
+#ifndef MFRONT_EXPORT_SYMBOL
+#define MFRONT_EXPORT_SYMBOL(TYPE, NAME, VALUE) \
+    MFRONT_SHARED0BJ extern TYPE NAME; \
+    MFRONT_SHARED0BJ TYPE NAME = VALUE
+#endif /* MFRONT_EXPORT_SYMBOL */
+
+#ifndef MFRONT_EXPORT_ARRAY_ARGUMENTS
+#define MFRONT_EXPORT_ARRAY_ARGUMENTS(...) __VA_ARGS__
+#endif /* MFRONT_EXPORT_ARRAY_ARGUMENTS */
+
+#ifndef MFRONT_EXPORT_ARRAY_OF_SYMBOLS

```



```

+double theta;
+double epsilon;
+unsigned short iterMax;
+double minimal_time_step_scaling_factor;
+double maximal_time_step_scaling_factor;
+double numerical_jacobian_epsilon;
+
+void set(const char* const,const double);
+
+void set(const char* const,const unsigned short);
+
+/*!
+ * \brief convert a string to double
+ * \param[in] p : parameter
+ * \param[in] v : value
+ */
+static double getDouble(const std::string&,const std::string&);
+/*!
+ * \brief convert a string to unsigned short
+ * \param[in] p : parameter
+ * \param[in] v : value
+ */
+static unsigned short getUnsignedShort(const std::string&,const std::string&);
+private :
+
+ModCamClay_semiExpl_coupledParametersInitializer();
+
+ModCamClay_semiExpl_coupledParametersInitializer(const ModCamClay_semiExpl_coupledParametersInitializer&);
+
+ModCamClay_semiExpl_coupledParametersInitializer&
+operator=(const ModCamClay_semiExpl_coupledParametersInitializer&);
+/*!
+ * \brief read the parameters from the given file
+ * \param[out] pi : parameters initializer
+ * \param[in] fn : file name
+ */
+static void readParameters(ModCamClay_semiExpl_coupledParametersInitializer&,const char* const);
+};
+
+///! \brief forward declaration
+template<ModellingHypothesis::Hypothesis, typename NumericType, bool use_qt>
+struct ModCamClay_semiExpl_coupled;
+
+///! \brief forward declaration
+template<ModellingHypothesis::Hypothesis hypothesis, typename NumericType>
+std::ostream&
+operator <<(std::ostream&,const ModCamClay_semiExpl_coupled-hypothesis, NumericType, false&);
+
+/*!
+ * \class ModCamClay_semiExpl_coupled
+ * \brief This class implements the ModCamClay_semiExpl_coupled behaviour.
+ * \tparam hypothesis: modelling hypothesis.
+ * \tparam NumericType: numerical type.
+ * \author Aqeel Afzal Chaudhry , Christian Silbermann , Eric Simo , Miguel Mánica , Thomas Helfer , Thomas Nagel
+ * \date 01 / 09 / 2025
+ * The modified cam - clay model according to Callari ( 1998 ) :
+ * A finite-strain cam-clay model in the framework of multiplicative elasto-plasticity
+ * but here in a consistent geometrically linear form ( linearized volume ratio evolution )
+ * semi - explicit due to explicit volume ratio update at the end of time step ,
+ * nonlinear hypoelastic behavior : pressure - dependent bulk modulus , constant Poisson ratio ,
+ * incremental formulation assuming constant elastic parameters over the time step ,
+ * normalized plastic flow direction , lower limit for a minimal pre - consolidation pressure ,
+ * with hydraulic and thermal coupling , saturation dependent swelling ,
+ */
+template<ModellingHypothesis::Hypothesis hypothesis,typename NumericType>
+struct ModCamClay_semiExpl_coupled-hypothesis, NumericType, false> final
+: public MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>,
+public ModCamClay_semiExpl_coupledBehaviourData-hypothesis, NumericType, false>,
+public ModCamClay_semiExpl_coupledIntegrationData-hypothesis, NumericType, false>,
+public tfel::math::TinyNewtonRaphsonSolver<2*ModellingHypothesisToTensorSize-hypothesis::value+2, NumericType, ModCamClay_semiExpl_coupled-hypothesis, NumericType, false>>{
+
+static constexpr unsigned short N = ModellingHypothesisToSpaceDimension-hypothesis::value;
+
+static_assert(N==1||N==2||N==3);
+static_assert(tfel::traits::IsFundamental<NumericType-NumericType>::cond);
+static_assert(tfel::traits::IsReal<NumericType>::cond);
+
+friend std::ostream& operator<< <<(std::ostream&,const ModCamClay_semiExpl_coupled&);
+
+friend struct tfel::math::TinyNonLinearSolverBase<2*ModellingHypothesisToTensorSize-hypothesis::value+2, NumericType, ModCamClay_semiExpl_coupled>;
+static constexpr unsigned short TVectorSize = N;
+typedef tfel::math::TensorDimToSize<N> TensorDimToSize;
+static constexpr unsigned short TensorSize = TensorDimToSize::value;
+typedef tfel::math::TensorDimToSize<N> TensorDimToSize;
+static constexpr unsigned short TensorSize = TensorDimToSize::value;
+
+using ushort = unsigned short;
+using Types = tfel::config::Types<N, NumericType, false>;
+using Type = NumericType;
+using numeric_type = typename Types::numeric_type;
+using real = typename Types::real;
+using time = typename Types::time;
+using length = typename Types::length;
+using frequency = typename Types::frequency;
+using speed = typename Types::speed;
+using stress = typename Types::stress;
+using strain = typename Types::strain;
+using strainrate = typename Types::strainrate;
+using stressrate = typename Types::stressrate;
+using temperature = typename Types::temperature;
+using thermalexpansion = typename Types::thermalexpansion;
+using thermalconductivity = typename Types::thermalconductivity;
+using massdensity = typename Types::massdensity;
+using energydensity = typename Types::energydensity;
+using TVector = typename Types::TVector;
+using DisplacementTVector = typename Types::DisplacementTVector;
+using ForceVector = typename Types::ForceTVector;
+using HeatFlux = typename Types::HeatFlux;
+using TemperatureGradient = typename Types::TemperatureGradient;
+using Stensor = typename Types::Stensor;
+using StressStensor = typename Types::StressStensor;
+using StressRateStensor = typename Types::StressRateStensor;
+using StrainStensor = typename Types::StrainStensor;
+using StrainRateStensor = typename Types::StrainRateStensor;
+using FrequencyStensor = typename Types::FrequencyStensor;
+using Tensor = typename Types::Tensor;
+using DeformationGradientTensor = typename Types::DeformationGradientTensor;

```

```

+using DeformationGradientRateTensor = typename Types::DeformationGradientRateTensor;
+using StressTensor = typename Types::StressTensor;
+using StiffnessTensor = typename Types::StiffnessTensor;
+using Tensor4 = typename Types::Tensor4;
+using TangentOperator = tfel::math::tvector<(TensorSize)*(TensorSize)*(TensorSize)*(1)+(TensorSize)*(1)*(1)*(1),real>;
+using PhysicalConstants = tfel::PhysicalConstants<NumericType, false>;
+
+public :
+
+typedef ModCamClay_semiExpL_coupledBehaviourData-hypothesis, NumericType, false> BehaviourData;
+typedef ModCamClay_semiExpL_coupledIntegrationData-hypothesis, NumericType, false> IntegrationData;
+typedef typename MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::SMFlag SMFlag;
+typedef typename MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::SMTYPE SMTYPE;
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::ELASTIC;
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::SECANTOPERATOR;
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::TANGENTOPERATOR;
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::CONSISTENTTANGENTOPERATOR;
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::NOSTIFFNESSREQUESTED;
+using IntegrationResult = typename MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::IntegrationResult;
+
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::SUCCESS;
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::FAILURE;
+using MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::UNRELIABLE_RESULTS;
+
+private :
+
+
+
+#line 45 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+strain rpc;
+
+#line 36 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+tfel::math::View<StressTensor> dtfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7;
+#line 39 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+tfel::math::View<StrainTensor> dtfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1;
+#line 42 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real& dtfel_unicode_mangling_039Btfel_unicode_mangling_209A;
+#line 45 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real& drpc;
+
+#line 105 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real K;
+#line 106 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real tfel_unicode_mangling_03C6;
+#line 107 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real tfel_unicode_mangling_03C6tfel_unicode_mangling_2080;
+#line 108 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real tfel_unicode_mangling_2202tfel_unicode_mangling_03C6tfel_unicode_mangling_2215tfel_unicode_mangling_2202tfel_unicode_mangling_1D38;
+#line 109 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real tfel_unicode_mangling_0394tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7;
+#line 110 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real tfel_unicode_mangling_03C7;
+#line 111 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real tfel_unicode_mangling_03C7tfel_unicode_mangling_2080;
+#line 112 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real tfel_unicode_mangling_2202tfel_unicode_mangling_03C7tfel_unicode_mangling_2215tfel_unicode_mangling_2202tfel_unicode_mangling_1D38;
+#line 113 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+StressTensor tfel_unicode_mangling_03C3tfel_unicode_mangling_2080;
+#line 114 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+StiffnessTensor tfel_unicode_mangling_2202tfel_unicode_mangling_03C3tfel_unicode_mangling_2215tfel_unicode_mangling_2202tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_un
icode_mangling_02E1;
+#line 115 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+bool withinElasticRange;
+#line 116 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real Mtfel_unicode_mangling_00B2;
+#line 117 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real young;
+#line 118 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real pc_min;
+#line 119 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+real rpc_min;
+
+
+NumericType theta;
+NumericType epsilon;
+ushort iterMax;
+real minimal_time_step_scaling_factor;
+real maximal_time_step_scaling_factor;
+NumericType numerical_jacobian_epsilon;
+
+
+//! Tangent operator;
+TangentOperator Dt;
+tfel::math::View<tfel::math::derivative_type<StressTensor,StrainTensor>> dtfel_unicode_mangling_03C3_ddtfel_unicode_mangling_03B5tfel_unicode_mangling_1D57tfel_unicode_mangling_
1D52;
+tfel::math::View<tfel::math::derivative_type<StressTensor,temperature>> dtfel_unicode_mangling_03C3_ddt;
+tfel::math::View<tfel::math::derivative_type<StressTensor,real>> dtfel_unicode_mangling_03C3_ddtfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F;
+tfel::math::derivative_type<real,real>& dStfel_unicode_mangling_1D38_ddtfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F;
+// updating material properties, in mandatory
+TFEL_HOST_DEVICE void updateMaterialPropertiesDependantOnStateVariables(){
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+}
+
+
+SMTYPE stiffness_matrix_type;
+TFEL_HOST_DEVICE void computeNumericalJacobian(tfel::math::tmatrix<2*TensorSize+2,2*TensorSize+2, NumericType>& njacobian)
+{
+using namespace std;
+using namespace tfel::math;
+vector<2*TensorSize+2, NumericType> tzeros(this->zeros);
+vector<2*TensorSize+2, NumericType> tfzeros(this->fzeros);
+tmatrix<2*TensorSize+2,2*TensorSize+2, NumericType> tjacobian(this->jacobian);
+for(ushort mfront_idx = 0; mfront_idx != 2*TensorSize+2; ++mfront_idx){
+this->zeros(mfront_idx) -= this->numerical_jacobian_epsilon;
+this->computeThermodynamicForces();
+this->computeeDF(true);
+this->zeros = tzeros;
+vector<2*TensorSize+2, NumericType> tfzeros2(this->fzeros);
+this->zeros(mfront_idx) += this->numerical_jacobian_epsilon;
+this->computeThermodynamicForces();
+this->computeeDF(true);
+this->fzeros = (this->fzeros-tfzeros2) / (2 * (this->numerical_jacobian_epsilon));
+for(ushort mfront_idx2 = 0; mfront_idx2!= 2*TensorSize+2; ++mfront_idx2){
+njacobian(mfront_idx2,mfront_idx) = this->fzeros(mfront_idx2);
+}
+this->zeros = tzeros;
+this->fzeros = tfzeros;
+}
+if(&njacobian)!=(&this->jacobian)){

```

```

+this->jacobian = tjacobian;
+}
+}
+
+[[nodiscard]] TFE_HOST_DEVICE bool
+computePartialJacobianInvert(const tfel::math::TinyPermutation<2*StensorSize+2*> jacobian_permutation, tfel::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_
tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7)
+{
+using namespace tfel::math;
+for(ushort mfront_idx=0; mfront_idx !=StensorSize; ++mfront_idx){
+vector<2*StensorSize+2, NumericType> vect_e(NumericType(0));
+vect_e(mfront_idx) = NumericType(1);
+if(!TinyMatrixSolve<2*StensorSize+2, NumericType, false>::back_substitute(this->jacobian, jacobian_permutation, vect_e)){
+return false;
+}
+for(ushort mfront_idx2=0; mfront_idx2!=StensorSize; ++mfront_idx2){
+partial_jacobian_tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7(mfront_idx2,mfront_idx)=vect_e(mfront_idx2);
+}
+}
+return true;
+}
+
+[[nodiscard]] TFE_HOST_DEVICE bool
+computePartialJacobianInvert(const tfel::math::TinyPermutation<2*StensorSize+2*> jacobian_permutation, tfel::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_
tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7,
+tfel::math::derivative_type<StrainTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1)
+{
+using namespace tfel::math;
+for(ushort mfront_idx=0; mfront_idx !=StensorSize; ++mfront_idx){
+vector<2*StensorSize+2, NumericType> vect_e(NumericType(0));
+vect_e(mfront_idx) = NumericType(1);
+if(!TinyMatrixSolve<2*StensorSize+2, NumericType, false>::back_substitute(this->jacobian, jacobian_permutation, vect_e)){
+return false;
+}
+for(ushort mfront_idx2=0; mfront_idx2!=StensorSize; ++mfront_idx2){
+partial_jacobian_tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7(mfront_idx2,mfront_idx)=vect_e(mfront_idx2);
+}
+for(ushort mfront_idx2=StensorSize; mfront_idx2!=2*StensorSize; ++mfront_idx2){
+partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1(mfront_idx2,mfront_idx)=vect_e(mfront_idx2);
+}
+}
+return true;
+}
+
+[[nodiscard]] TFE_HOST_DEVICE bool
+computePartialJacobianInvert(const tfel::math::TinyPermutation<2*StensorSize+2*> jacobian_permutation, tfel::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_
tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7,
+tfel::math::derivative_type<StrainTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1,
+tfel::math::derivative_type<real, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B8tfel_unicode_mangling_209A)
+{
+using namespace tfel::math;
+for(ushort mfront_idx=0; mfront_idx !=StensorSize; ++mfront_idx){
+vector<2*StensorSize+2, NumericType> vect_e(NumericType(0));
+vect_e(mfront_idx) = NumericType(1);
+if(!TinyMatrixSolve<2*StensorSize+2, NumericType, false>::back_substitute(this->jacobian, jacobian_permutation, vect_e)){
+return false;
+}
+for(ushort mfront_idx2=0; mfront_idx2!=StensorSize; ++mfront_idx2){
+partial_jacobian_tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7(mfront_idx2,mfront_idx)=vect_e(mfront_idx2);
+}
+for(ushort mfront_idx2=StensorSize; mfront_idx2!=2*StensorSize; ++mfront_idx2){
+partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1(mfront_idx2,mfront_idx)=vect_e(mfront_idx2);
+}
+partial_jacobian_tfel_unicode_mangling_03B8tfel_unicode_mangling_209A(mfront_idx)=vect_e(2*StensorSize);
+}
+return true;
+}
+
+[[nodiscard]] TFE_HOST_DEVICE bool
+computePartialJacobianInvert(const tfel::math::TinyPermutation<2*StensorSize+2*> jacobian_permutation, tfel::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_
tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7,
+tfel::math::derivative_type<StrainTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1,
+tfel::math::derivative_type<real, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B8tfel_unicode_mangling_209A,
+tfel::math::derivative_type<strain, StrainTensor>& partial_jacobian_rpc)
+{
+using namespace tfel::math;
+for(ushort mfront_idx=0; mfront_idx !=StensorSize; ++mfront_idx){
+vector<2*StensorSize+2, NumericType> vect_e(NumericType(0));
+vect_e(mfront_idx) = NumericType(1);
+if(!TinyMatrixSolve<2*StensorSize+2, NumericType, false>::back_substitute(this->jacobian, jacobian_permutation, vect_e)){
+return false;
+}
+for(ushort mfront_idx2=0; mfront_idx2!=StensorSize; ++mfront_idx2){
+partial_jacobian_tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7(mfront_idx2,mfront_idx)=vect_e(mfront_idx2);
+}
+for(ushort mfront_idx2=StensorSize; mfront_idx2!=2*StensorSize; ++mfront_idx2){
+partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1(mfront_idx2,mfront_idx)=vect_e(mfront_idx2);
+}
+partial_jacobian_tfel_unicode_mangling_03B8tfel_unicode_mangling_209A(mfront_idx)=vect_e(2*StensorSize);
+partial_jacobian_rpc(mfront_idx)=vect_e(2*StensorSize+1);
+}
+return true;
+}
+
+TFEL_HOST_DEVICE void additionalConvergenceChecks(bool& converged, const NumericType& error){
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+const auto smt = this->stiffness_matrix_type;
+#line 299 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+if (converged)
+#line 300 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+{
+#line 301 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+if (!this->withinElasticRange)
+#line 302 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+{
+#line 303 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+if (this->dtfel_unicode_mangling_03B8tfel_unicode_mangling_209A < 0)
+#line 304 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+{
+#line 305 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+converged = false;
+#line 306 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+this->withinElasticRange = true;
+#line 307 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+}
+}

```

```

+ #line 308 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+}
+ #line 309 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+}
+static_cast<void>(converged);
+static_cast<void>(error);
+static_cast<void>(smt);
+} // end of additionalConvergenceChecks
+
+TFEL_HOST_DEVICE void computeThermodynamicForces(){
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+ #line 222 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfe_l_unicode_mangling_03C3 = this->tfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_2080 + theta * this->tfe_l_unicode_mangling_2202tfe_l_unicode_mangling_03C3tfe_l_unicode_mang
ling_2215tfe_l_unicode_mangling_2202tfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode_mangling_02E1 * this->dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l
unicode_mangling_02E1 - 3 * this->K * this->tfe_l_unicode_mangling_03B1 * this->dT * Stensor::Id() - this->tfe_l_unicode_mangling_0394ptfe_l_unicode_mangling_02E2tfe_l_unicode_mangling
_02B7 * Stensor::Id();
+} // end of ModCamClay_semiExpl_coupled::computeThermodynamicForces
+
+TFEL_HOST_DEVICE void computeFinalThermodynamicForces(){
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+ #line 285 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfe_l_unicode_mangling_03C3 = this->tfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_2080 + this->tfe_l_unicode_mangling_2202tfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_221
5tfe_l_unicode_mangling_2202tfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode_mangling_02E1 * this->dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode
_mangling_02E1 - 3 * this->K * this->tfe_l_unicode_mangling_03B1 * this->dT * Stensor::Id() - this->tfe_l_unicode_mangling_03B1tfe_l_unicode_mangling_1047 * this->tfe_l_unicode_mangling
_03C7 * (this->ptfe_l_unicode_mangling_1038tfe_l_unicode_mangling_103F+this->dpdtfe_l_unicode_mangling_1038tfe_l_unicode_mangling_103F) * Stensor::Id() - this->tfe_l_unicode_mangling_039
4ptfe_l_unicode_mangling_02E2tfe_l_unicode_mangling_02B7 * Stensor::Id();
+} // end of ModCamClay_semiExpl_coupled::computeFinalThermodynamicForces
+
+/*!
+ ** \brief Update internal variables at end of integration
+ **/
+TFEL_HOST_DEVICE void updateIntegrationVariables(){
+this->rpc += this->drpc;
+}
+
+/*!
+ ** \brief Update internal variables at end of integration
+ **/
+TFEL_HOST_DEVICE void updateStateVariables(){
+this->tfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_02E2tfe_l_unicode_mangling_02B7 += this->dtfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_02E2tfe_l_unicode_mangling_02B7;
+this->tfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode_mangling_02E1 += this->dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode_mangling_02E1;
+this->tfe_l_unicode_mangling_039Btfe_l_unicode_mangling_209A += this->dtfe_l_unicode_mangling_039Btfe_l_unicode_mangling_209A;
+}
+
+/*!
+ ** \brief Update auxiliary state variables at end of integration
+ **/
+TFEL_HOST_DEVICE void updateAuxiliaryStateVariables(){
+using namespace std;
+using namespace tfel::math;
+ #line 291 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_02E2tfe_l_unicode_mangling_02B7 -= this->tfe_l_unicode_mangling_0394ptfe_l_unicode_mangling_02E2tfe_l_unicode_mangling_02B7 * Ste
nsor::Id();
+ #line 292 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->pc += this->drpc * this->young;
+ #line 293 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_105Btfe_l_unicode_mangling_209Atfe_l_unicode_mangling_2097 += trace(this->dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1057
tfe_l_unicode_mangling_1052 - this->dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode_mangling_02E1 - this->tfe_l_unicode_mangling_03B1 * this->dT * Stensor::Id());
+ #line 294 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->vtfe_l_unicode_mangling_02B3 += this->vtfe_l_unicode_mangling_02B3tfe_l_unicode_mangling_2070 * trace(this->dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1057tfe_l_unicode_ma
ngling_1052 - this->tfe_l_unicode_mangling_03B1 * this->dT * Stensor::Id());
+}
+
+///! \brief Default constructor (disabled)
+ModCamClay_semiExpl_coupled() =delete ;
+///! \brief Copy constructor (disabled)
+ModCamClay_semiExpl_coupled(const ModCamClay_semiExpl_coupled&) = delete;
+///! \brief Assignment operator (disabled)
+ModCamClay_semiExpl_coupled& operator = (const ModCamClay_semiExpl_coupled&) = delete;
+
+public:
+
+/*!
+ ** \brief Constructor
+ **/
+ModCamClay_semiExpl_coupled(const ModCamClay_semiExpl_coupledBehaviourData-hypothesis, NumericType, false& src1,
+const ModCamClay_semiExpl_coupledIntegrationData-hypothesis, NumericType, false& src2)
+: ModCamClay_semiExpl_coupledBehaviourData-hypothesis, NumericType, false&(src1),
+ModCamClay_semiExpl_coupledIntegrationData-hypothesis, NumericType, false&(src2),
+dtfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_02E2tfe_l_unicode_mangling_02B7(tfel::math::map<StressTensor, 0>(this->zeros)),
+dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode_mangling_02E1(tfel::math::map<StrainTensor, TensorSize>(this->zeros)),
+dtfe_l_unicode_mangling_039Btfe_l_unicode_mangling_209A(this->zeros(2*TensorSize)),
+drpc(this->zeros(2*TensorSize+1)),
+dtfe_l_unicode_mangling_03C3_ddtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1057tfe_l_unicode_mangling_1052(Dt.begin()),
+dtfe_l_unicode_mangling_03C3_ddT(Dt.begin()+TensorSize+TensorSize),
+dtfe_l_unicode_mangling_03C3_ddpdtfe_l_unicode_mangling_1038tfe_l_unicode_mangling_103F(Dt.begin()+TensorSize+TensorSize+TensorSize),
+d5tfe_l_unicode_mangling_1038_ddpdtfe_l_unicode_mangling_1038tfe_l_unicode_mangling_103F(Dt(TensorSize+TensorSize+2*TensorSize))
+{
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+this->theta = ModCamClay_semiExpl_coupledParametersInitializer::get().theta;
+this->epsilon = ModCamClay_semiExpl_coupledParametersInitializer::get().epsilon;
+this->iterMax = ModCamClay_semiExpl_coupledParametersInitializer::get().iterMax;
+this->minima_l_time_step_scaling_factor = ModCamClay_semiExpl_coupledParametersInitializer::get().minima_l_time_step_scaling_factor;
+this->maxima_l_time_step_scaling_factor = ModCamClay_semiExpl_coupledParametersInitializer::get().maxima_l_time_step_scaling_factor;
+this->numerical_jacobian_epsilon = ModCamClay_semiExpl_coupledParametersInitializer::get().numerical_jacobian_epsilon;
+}
+
+/*
+ * * \brief constructor for the Generic interface
+ * * \param[in] mgb_d: behaviour data
+ * */
+ModCamClay_semiExpl_coupled(const mfront::gb:BehaviourData& mgb_d)
+: ModCamClay_semiExpl_coupledBehaviourData-hypothesis, NumericType, false&(mgb_d),
+ModCamClay_semiExpl_coupledIntegrationData-hypothesis, NumericType, false&(mgb_d),
+dtfe_l_unicode_mangling_03C3tfe_l_unicode_mangling_02E2tfe_l_unicode_mangling_02B7(tfel::math::map<StressTensor, 0>(this->zeros)),
+dtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1049tfe_l_unicode_mangling_02E1(tfel::math::map<StrainTensor, TensorSize>(this->zeros)),
+dtfe_l_unicode_mangling_039Btfe_l_unicode_mangling_209A(this->zeros(2*TensorSize)),
+drpc(this->zeros(2*TensorSize+1)),
+dtfe_l_unicode_mangling_03C3_ddtfe_l_unicode_mangling_03B5tfe_l_unicode_mangling_1057tfe_l_unicode_mangling_1052(Dt.begin()),
+dtfe_l_unicode_mangling_03C3_ddT(Dt.begin()+TensorSize+TensorSize),
+dtfe_l_unicode_mangling_03C3_ddpdtfe_l_unicode_mangling_1038tfe_l_unicode_mangling_103F(Dt.begin()+TensorSize+TensorSize+TensorSize),

```

```

+Stfel_unicode_mangling_1038_dtpfel_unicode_mangling_1038tfel_unicode_mangling_103F(Dt(StensorSize+StensorSize+2+StensorSize))
+{
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+this->theta = ModCamClay_semiExpl_coupledParametersInitializer::get().theta;
+this->epsilon = ModCamClay_semiExpl_coupledParametersInitializer::get().epsilon;
+this->iterMax = ModCamClay_semiExpl_coupledParametersInitializer::get().iterMax;
+this->minimal_time_step_scaling_factor = ModCamClay_semiExpl_coupledParametersInitializer::get().minimal_time_step_scaling_factor;
+this->maximal_time_step_scaling_factor = ModCamClay_semiExpl_coupledParametersInitializer::get().maximal_time_step_scaling_factor;
+this->numerical_jacobian_epsilon = ModCamClay_semiExpl_coupledParametersInitializer::get().numerical_jacobian_epsilon;
+this-> tfel_unicode_mangling_0385tfel_unicode_mangling_1057tfel_unicode_mangling_1052 = tfel::math::map<StrainTensor>(mgb_d.s0.gradients);
+tfel::fsa[go::transform<StensorSize>::exe(mgb_d.s1.gradients,mgb_d.s0.gradients,this->dtfel_unicode_mangling_0385tfel_unicode_mangling_1057tfel_unicode_mangling_1052.begin()),std::minus<real>());
+this->ptfel_unicode_mangling_1038tfel_unicode_mangling_103F = mgb_d.s0.gradients[StensorSize];
+this->dptfel_unicode_mangling_1038tfel_unicode_mangling_103F = mgb_d.s1.gradients[StensorSize] - mgb_d.s0.gradients[StensorSize];
+}
+
+/*!
+ * * \ brief initialize the behaviour with user code
+ * */
+{[nodiscard]} TFEL_HOST_DEVICE bool initialize(){
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+#line 127 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const auto Stfel_unicode_mangling_2091 = [&](const auto ptfel_unicode_mangling_109Ctfel_unicode_mangling_1043tfel_unicode_mangling_1056) {
+#line 128 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+return std::pow(
+#line 129 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+std::pow(std::max(ptfel_unicode_mangling_109Ctfel_unicode_mangling_1043tfel_unicode_mangling_1056, 0.0) / this->ptfel_unicode_mangling_1047, 1.0 / (1.0 - this->mtfel_unicode_mangling_270tfel_unicode_mangling_1033)) + 1.0, -this->mtfel_unicode_mangling_270tfel_unicode_mangling_1033);
+#line 130 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+});
+#line 132 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const double Stfel_unicode_mangling_2080 = Stfel_unicode_mangling_2091(-(this->ptfel_unicode_mangling_1038tfel_unicode_mangling_103F)) * (1.0 - this->Stfel_unicode_mangling_1033tfel_unicode_mangling_02B3 - this->Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3) + this->Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3;
+#line 133 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const double Stfel_unicode_mangling_1038tfel_unicode_mangling_2080 = std::clamp(Stfel_unicode_mangling_2080, this->Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3, 1.0 - this->Stfel_unicode_mangling_1033tfel_unicode_mangling_02B3);
+#line 135 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const double S = Stfel_unicode_mangling_2091(-(this->ptfel_unicode_mangling_1038tfel_unicode_mangling_103F + this->dptfel_unicode_mangling_1038tfel_unicode_mangling_103F)) * (1.0 - this->Stfel_unicode_mangling_1033tfel_unicode_mangling_02B3 - this->Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3) + this->Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3;
+#line 136 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->Stfel_unicode_mangling_1038 = std::clamp(S, this->Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3, 1.0 - this->Stfel_unicode_mangling_1033tfel_unicode_mangling_02B3);
+#line 139 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const int model_type = static_cast<int>(this->bishop_type);
+#line 140 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+if (model_type == 0) {
+#line 141 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C7 = std::pow(this->Stfel_unicode_mangling_1038, this->bishop_value);
+#line 142 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_2202tfel_unicode_mangling_03C7tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1038 = this->bishop_value * std::pow(this->Stfel_unicode_mangling_1038, this->bishop_value - 1);
+#line 143 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C7tfel_unicode_mangling_2080 = std::pow(Stfel_unicode_mangling_1038tfel_unicode_mangling_2080, this->bishop_value);
+#line 144 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+}
+#line 145 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+else if (model_type == 1) {
+#line 146 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const double S_cut = this->bishop_value;
+#line 147 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+if (this->Stfel_unicode_mangling_1038 < S_cut) {
+#line 148 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C7 = 0.0;
+#line 149 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_2202tfel_unicode_mangling_03C7tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1038 = 0.0;
+#line 150 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+} else if (this->Stfel_unicode_mangling_1038 > 1.0) {
+#line 151 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C7 = 1.0;
+#line 152 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_2202tfel_unicode_mangling_03C7tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1038 = 0.0;
+#line 153 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+} else {
+#line 154 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const double denom = std::max(1e-15, 1.0 - S_cut);
+#line 155 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C7 = (this->Stfel_unicode_mangling_1038 - S_cut) / denom;
+#line 156 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_2202tfel_unicode_mangling_03C7tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1038 = 1.0 / denom;
+#line 157 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+}
+#line 158 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C7tfel_unicode_mangling_2080 = (Stfel_unicode_mangling_1038tfel_unicode_mangling_2080 < S_cut) ? 0.0 :
+#line 159 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+(Stfel_unicode_mangling_1038tfel_unicode_mangling_2080 > 1.0) ? 1.0 :
+#line 160 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+(Stfel_unicode_mangling_1038tfel_unicode_mangling_2080 - S_cut) / std::max(1e-15, 1.0 - S_cut);
+#line 161 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+}
+#line 162 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+else {
+#line 163 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+throw std::runtime_error("Invalid BishopsModeType: must be 0 (power law) or 1 (cutoff).");
+#line 164 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+}
+#line 167 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+if (this->Stfel_unicode_mangling_1038 <= this->Stfel_unicode_mangling_1038tfel_unicode_mangling_2097)
+#line 168 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+{
+#line 169 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C6 = 0.0;
+#line 170 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_2202tfel_unicode_mangling_03C6tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1038 = 0.0;
+#line 171 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+}
+#line 172 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+else if (this->Stfel_unicode_mangling_1038 >= this->Stfel_unicode_mangling_1038tfel_unicode_mangling_2095)
+#line 173 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+{
+#line 174 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_03C6 = 1.0;
+#line 175 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+this->tfel_unicode_mangling_2202tfel_unicode_mangling_03C6tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1038 = 0.0;

```

```

+ #line 176 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ }
+ #line 177 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ else
+ #line 178 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ {
+ #line 179 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const double normalized = (this->Stfel_uniocode_mangling_1038 - this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2097) / (this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2095 - this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2097);
+ #line 180 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->tfe_uniocode_mangling_03C6 = std::pow(normalized, this->mtfe_uniocode_mangling_02E2tfe_uniocode_mangling_02B7);
+ #line 181 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->tfe_uniocode_mangling_2202tfe_uniocode_mangling_03C6tfe_uniocode_mangling_2215tfe_uniocode_mangling_22025tfe_uniocode_mangling_1038 = this->mtfe_uniocode_mangling_02E2tfe_uniocode_mangling_02B7 / (this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2095 - this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2097) * std::pow(normalized, this->mtfe_uniocode_mangling_02E2tfe_uniocode_mangling_02B7 - 1);
+ #line 182 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ }
+ #line 184 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->tfe_uniocode_mangling_03C6tfe_uniocode_mangling_2080 = std::pow((Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2080 - this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2097) / (this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2095 - this->Stfel_uniocode_mangling_1038tfe_uniocode_mangling_2097), this->mtfe_uniocode_mangling_02E2tfe_uniocode_mangling_02B7);
+ #line 187 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->tfe_uniocode_mangling_0394tfe_uniocode_mangling_02E2tfe_uniocode_mangling_02B7 = this->ptfe_uniocode_mangling_02E2tfe_uniocode_mangling_02B7 * (this->tfe_uniocode_mangling_03C6 - this->tfe_uniocode_mangling_03C6tfe_uniocode_mangling_2080);
+ #line 189 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ tfe::raise_if(this->la < this->ka, "Invalid parameters: la<ka");
+ #line 190 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->mtfe_uniocode_mangling_00B2 = this->M * this->M;
+ #line 193 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->tfe_uniocode_mangling_03C3tfe_uniocode_mangling_2080 = this->tfe_uniocode_mangling_03C3 + this->tfe_uniocode_mangling_03B1tfe_uniocode_mangling_1047 * this->tfe_uniocode_mangling_03C7tfe_uniocode_mangling_2080 * this->ptfe_uniocode_mangling_1038tfe_uniocode_mangling_103F * Stensor::Id();
+ #line 196 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->pc_min = 0.5e-8 * this->pc_char;
+ #line 197 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto p = -trace(this->tfe_uniocode_mangling_03C3tfe_uniocode_mangling_2080) / 3;
+ #line 198 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->K = this->vtfe_uniocode_mangling_02B3tfe_uniocode_mangling_2070 / this->ka * max(p, this->pc_char);
+ #line 199 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto E = 3.0 * this->K * (1.0 - 2 * this->tfe_uniocode_mangling_03B0);
+ #line 201 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->young = E;
+ #line 202 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->rpc = this->pc / this->young;
+ #line 203 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->rpc_min = this->pc_min / this->young;
+ #line 206 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->tfe_uniocode_mangling_2202tfe_uniocode_mangling_03C3tfe_uniocode_mangling_2215tfe_uniocode_mangling_2202tfe_uniocode_mangling_03B5tfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 = E / (1.0 + this->tfe_uniocode_mangling_03B0) * Stensor4::K() + this->K * Stensor4::IXI();
+ #line 209 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto tfe_uniocode_mangling_03C3tfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 = this->tfe_uniocode_mangling_03C3tfe_uniocode_mangling_2080 + this->tfe_uniocode_mangling_2202tfe_uniocode_mangling_03C3tfe_uniocode_mangling_2215tfe_uniocode_mangling_2202tfe_uniocode_mangling_03B5tfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 * this->dtfe_uniocode_mangling_03B5tfe_uniocode_mangling_1057tfe_uniocode_mangling_1052 - 3 * this->K * this->tfe_uniocode_mangling_03B1 * this->dt * Stensor::Id() - this->tfe_uniocode_mangling_0394tfe_uniocode_mangling_02E2tfe_uniocode_mangling_02B7 * Stensor::Id();
+ #line 212 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto stfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 = deviator(tfe_uniocode_mangling_03C3tfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1);
+ #line 213 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto qtfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 = sqrt(1.5 * stfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 | stfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1);
+ #line 214 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto ptfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 = -trace(tfe_uniocode_mangling_03C3tfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1) / 3;
+ #line 216 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto pctfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 = this->pc;
+ #line 217 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ const auto ftfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 = qtfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 * qtfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 + this->mtfe_uniocode_mangling_00B2 * ptfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 * (ptfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 - pctfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1);
+ #line 218 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+ #this->withinElasticRange = ftfe_uniocode_mangling_1049tfe_uniocode_mangling_02E1 < 0;
+ return true;
+ }
+
+ /*!
+ * \brief set the policy for "out of bounds" conditions
+ */
+ TFEL_HOST_DEVICE void
+ setOutOfBoundsPolicy(const OutOfBoundsPolicy policy_value){
+     + this->policy = policy_value;
+ } // end of setOutOfBoundsPolicy
+
+ /*!
+ ** \return the modelling hypothesis
+ **/
+ constexpr ModellingHypothesis: Hypothesis
+ getModellingHypothesis() const{
+     return hypothesis;
+ } // end of getModellingHypothesis
+
+
+ /*! \brief check physical bounds and standard bounds
+ TFEL_HOST_DEVICE void checkBounds() const{
+ } // end of checkBounds
+
+
+ [[nodiscard]] IntegrationResult computePredictionOperator(const SMFlag, const SMType) override{
+ tfe::raise("ModCamClay_semiExpl_coupled::computePredictionOperator: "
+ "unsupported prediction operator flag");
+ }
+
+ TFEL_HOST_DEVICE real getMinimalTimeStepScalingFactor() const noexcept override{
+     return this->minimal_time_step_scaling_factor;
+ }
+
+ TFEL_HOST_DEVICE std::pair<bool, real>
+ computeAPrioriTimeStepScalingFactor(const real current_time_step_scaling_factor) const override{
+ const auto time_scaling_factor = this->computeAPrioriTimeStepScalingFactorII();
+ return {time_scaling_factor.first,
+         std::min(std::min(std::max(time_scaling_factor.second,
+                                     this->minimal_time_step_scaling_factor),
+                                     this->maximal_time_step_scaling_factor),
+                 current_time_step_scaling_factor)};
+ }
+
+ /*!
+ * \brief Integrate behaviour law over the time step
+ */
+ [[nodiscard]] TFEL_HOST_DEVICE IntegrationResult integrate(const SMFlag smflag, const SMType smt) override{
+ using namespace std;
+ using namespace tfe::math;
+ #this->stiffness_matrix_type = smt;

```



```

+tfel::raise_if(smflag!=MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::STANDARDTANGENTOPERATOR,
+"invalid tangent operator flag");
+tfel::math::TinyNewtonRaphsonSolver<2*ModellingHypothesisToStensorSize<hypothesis::value+2, NumericType, ModCamClay_semiExpl_coupled<hypothesis, NumericType, false>::epsilon = t
his->epsilon;
+tfel::math::TinyNewtonRaphsonSolver<2*ModellingHypothesisToStensorSize<hypothesis::value+2, NumericType, ModCamClay_semiExpl_coupled<hypothesis, NumericType, false>::iterMax = t
his->iterMax;
+if(!this->solveNonLinearSystem()){
+return MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::FAILURE;
+}
+if (this->stiffness_matrix_type != NOSTIFFNESSREQUESTED){
+this->updateOrCheckJacobian();
+}
+this->updateIntegrationVariables();
+this->updateStateVariables();
+this->computeFinalThermodynamicForces();
+this->updateAuxiliaryStateVariables();
+if(smt!=NOSTIFFNESSREQUESTED){
+if(!this->computeConsistentTangentOperator(smt)){
+return MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::FAILURE;
+}
+}
+return MechanicalBehaviour<MechanicalBehaviourBase::GENERALBEHAVIOUR,hypothesis, NumericType, false>::SUCCESS;
+} // end of ModCamClay_semiExpl_coupled::integrate
+
+TFEL_HOST_DEVICE bool computeResidual(){
+this->computeThermodynamicForces();
+return this->computeFd(false);
+}
+
+//! \return the norm of the residual
+TFEL_HOST_DEVICE NumericType computeResidualNorm() {
+return tfel::math::norm(this->fzeros) / (2*StensorSize+2);
+}
+
+//!
+ * * \brief check the convergence of the method
+ * * \param[in] error: current error
+ * *
+TFEL_HOST_DEVICE bool checkConvergence(const NumericType error) {
+auto converged = error < this->epsilon;
+auto mfront_internals_converged = converged;
+this->additionalConvergenceChecks(converged, error);
+if((mfront_internals_converged) && (!converged)){
+this->is_delta_zeros_defined = false;
+}
+return converged;
+}
+
+//!
+ * * \brief update the jacobian matrix if required.
+ * * \param[in] converged: boolean stating if the method is
+ * * called after the convergence of the algorithm or before
+ * * the computation of the next correction.
+ * *
+ * * This method can be used to compute the jacobian or part
+ * * of the jacobian numerically. If the jacobian was computed
+ * * in 'computeResidual', this method can be used to compare it
+ * * to a numerical approximation.
+ * *
+TFEL_HOST_DEVICE void updateOrCheckJacobian(){
+}
+
+//!
+ * * \brief method meant to set bounds on some components
+ * * of the current Newton correction or to implement a line
+ * * search.
+ * *
+TFEL_HOST_DEVICE void processNewCorrection()
+{
+}
+
+//!
+ * * \brief method meant to process the new estimate.
+ * *
+ * * This method may be called to apply bounds on the
+ * * new estimate and update material properties
+ * * depending on the current state.
+ * *
+TFEL_HOST_DEVICE void processNewEstimate(){
+this->updateMaterialPropertiesDependantOnStateVariables();
+}
+
+//!
+ * * \brief solve linear system
+ * * \return true on success
+ * * \param[in] m: matrix
+ * * \param[in,out] v: right hand side on input, solution on output
+ * *
+TFEL_HOST_DEVICE bool solveLinearSystem(tfel::math::tmatrix<2*StensorSize+2, 2*StensorSize+2, NumericType>& mfront_matrix,tfel::math::tvector<2*StensorSize+2, NumericType>& mfront
_vector)const noexcept{
+auto mfront_success = true;
+mfront_success = tfel::math::TinyNewtonRaphsonSolver<2*ModellingHypothesisToStensorSize<hypothesis::value+2, NumericType, ModCamClay_semiExpl_coupled<hypothesis, NumericType, fal
se>::solveLinearSystem(mfront_matrix, mfront_vector);
+return mfront_success;
+}
+
+//!
+ * * \brief compute fzeros and jacobian
+ * *
+TFEL_HOST_DEVICE bool computeFd(const bool perturbedSystemEvaluation){
+using namespace std;
+using namespace tfel::math;
+using std::vector;
+// silent compiler warning
+static_cast<void>(perturbedSystemEvaluation);
+constexpr auto tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7_offset = 0;
+constexpr auto tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1_offset = StensorSize;
+constexpr auto tfel_unicode_mangling_03B8tfel_unicode_mangling_209A_offset = 2*StensorSize;
+constexpr auto rpc_offset = 2*StensorSize+1;
+auto tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7 = tfel::math::map<StressStensor, 0>(this->fzeros);
+auto tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1 = tfel::math::map<StrainStensor, StensorSize>(this->fzeros);
+real6 tfel_unicode_mangling_03B8tfel_unicode_mangling_209A(this->fzeros(2*StensorSize));
+real6 frpc(this->fzeros(2*StensorSize+1));
+// derivative of residual tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7 with respect to variable tfel_unicode_mangling_03C3tfel_unicode_mangling_
02E2tfel_unicode_mangling_02B7
+auto&& ddtfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7_ddtfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7 = tfel::ma
th::map_derivative<0, 0, StressStensor, StressStensor>(this->jacobian);
+// derivative of residual tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7 with respect to variable tfel_unicode_mangling_03B5tfel_unicode_mangling_
1D49tfel_unicode_mangling_02E1
+auto&& ddtfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7_ddtfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1 = tfel::ma
th::map_derivative<0, StensorSize, StressStensor, StrainStensor>(this->jacobian);
+// derivative of residual tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7 with respect to variable tfel_unicode_mangling_03B8tfel_unicode_mangling_
209A
+auto&& ddtfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7_ddtfel_unicode_mangling_03B8tfel_unicode_mangling_209A = tfel::math::map_derivative<0, 2*St

```

```

+#!line 267 "/home/aaqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"

```

```

+const auto tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394rpc = (Stensor::Id()) + tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202p * n / norm) * (this->Mtfel_uniCode_mangling_0002) / (3 * norm) * theta * (this->young);
+#line 268 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+const auto tfeL_uniCode_mangling_2202frpctfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1056tfeL_uniCode_mangling_02E1tfeL_uniCode_mangling_105B = the * (rpc_new - (this->rpc_min));
+#line 271 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+dtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_ddtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1 += (this->dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A) * tfeL_uniCode_mangling_2202ntfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1;
+#line 272 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+dtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A = n;
+#line 273 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+dtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_ddrpc = (this->dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A) * tfeL_uniCode_mangling_2202ntfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394rpc;
+#line 275 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_ddtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1 = (tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_03C3 | (this->tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1)) * theta / fchar;
+#line 276 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A = strain(0);
+#line 277 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_ddrpc = -(this->Mtfel_uniCode_mangling_0002) * p * theta / fchar * (this->young);
+#line 279 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+drpc_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A = tfeL_uniCode_mangling_2202frpctfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1056tfeL_uniCode_mangling_02E1tfeL_uniCode_mangling_105B * ntr;
+#line 280 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+drpc_ddtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1 = tfeL_uniCode_mangling_2202frpctfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1056tfeL_uniCode_mangling_02E1tfeL_uniCode_mangling_105B * (this->dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A) * (Stensor::Id()) | tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1;
+#line 281 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpL_coupled.mfront"
+drpc_ddrpc = 1 + tfeL_uniCode_mangling_0394tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1056tfeL_uniCode_mangling_02E1tfeL_uniCode_mangling_105B * the * theta + tfeL_uniCode_mangling_2202frpctfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1056tfeL_uniCode_mangling_02E1tfeL_uniCode_mangling_105B * (this->dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A) * trace(tfeL_uniCode_mangling_2202ntfeL_uniCode_mangling_2215tfeL_uniCode_mangling_2202tfeL_uniCode_mangling_0394rpc);
+static_cast<void>(dtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7_ddtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7_ddtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7_ddrpc); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_ddtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_ddtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_ddrpc); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_ddtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_ddtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1); /* suppress potential warnings */
+static_cast<void>(dtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A); /* suppress potential warnings */
+static_cast<void>(drpc_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_ddrpc); /* suppress potential warnings */
+static_cast<void>(drpc_ddtfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7); /* suppress potential warnings */
+static_cast<void>(drpc_ddtfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7); /* suppress potential warnings */
+static_cast<void>(drpc_ddtfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1); /* suppress potential warnings */
+static_cast<void>(drpc_ddtfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A); /* suppress potential warnings */
+static_cast<void>(drpc_ddrpc); /* suppress potential warnings */
+static_cast<void>(tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7_offset);
+static_cast<void>(tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1_offset);
+static_cast<void>(tfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A_offset);
+static_cast<void>(rpc_offset);
+return true;
+}
+
+TFEL_HOST_DEVICE std::pair<bool, real>
+computeAPosteriorITimeStepScalingFactor(const real current_time_step_scaling_factor) const override{
+const auto time_scaling_factor = this->computeAPosteriorITimeStepScalingFactorII();
+return {time_scaling_factor.first,
+        std::min(std::min(std::max(time_scaling_factor.second,
+                                    this->minimal_time_step_scaling_factor),
+                                    this->maximal_time_step_scaling_factor),
+                current_time_step_scaling_factor)};
+}
+
+/*!
+** \brief Update the internal energy at end of the time step
+** \param[in] Psi_s: internal energy at end of the time step
+**/
+TFEL_HOST_DEVICE void computeInternalEnergy(stress& Psi_s) const
+{
+Psi_s=stress(0);
+}
+
+/*!
+** \brief Update the dissipated energy at end of the time step
+** \param[in] Psi_d: dissipated energy at end of the time step
+**/
+TFEL_HOST_DEVICE void computeDissipatedEnergy(stress& Psi_d) const
+{
+Psi_d=stress(0);
+}
+
+TFEL_HOST_DEVICE bool computeConsistentTangentOperator(const SMType smt){
+using namespace std;
+using namespace tfeL::math;
+using std::vector;
+auto mfront_success = true;
+TinyPermutation<2*StensorSize> jacobian_permutation;
+if(!ITinyMatrixSolve<2*StensorSize+2, NumericType, false>::decomp(this->jacobian, jacobian_permutation)){
+return false;
+}
+struct TFEL_VISIBILITY_LOCAL GetPartialJacobianInvert{
+TFEL_HOST_DEVICE GetPartialJacobianInvert(ModCamClay_semiExpL_coupled& mfront_behaviour_argument,
+const tfeL::math::TinyPermutation<2*StensorSize>& mfront_permutation_argument,
+bool& mfront_success_argument)
+{
+mfront_get_partial_jacobian_invert_behaviour(mfront_behaviour_argument),
+mfront_get_partial_jacobian_invert_permutation(mfront_permutation_argument),
+mfront_success_reference(mfront_success_argument)}
+TFEL_HOST_DEVICE void operator()(tfeL::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7){
+if(!this->mfront_get_partial_jacobian_invert_behaviour.computePartialJacobianInvert(this->mfront_get_partial_jacobian_invert_permutation, partial_jacobian_tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_02B7)){
+this->mfront_success_reference = false;
+}
+}
+}

```

```

+TFEL_HOST_DEVICE void operator()(tfel::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangl
ing_02B7,
+tfel::math::derivative_type<StrainTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1){
+if((this->mfront_get_partial_jacobian_invert_behaviour.computePartialJacobianInvert(this->mfront_get_partial_jacobian_invert_permutation, partial_jacobian_tfel_unicode_mangling_03
C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7,
+partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1)){
+this->mfront_success_reference = false;
+}
+)
+)
+TFEL_HOST_DEVICE void operator()(tfel::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangl
ing_02B7,
+tfel::math::derivative_type<StrainTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1,
+tfel::math::derivative_type<real, StrainTensor>& partial_jacobian_tfel_unicode_mangling_0398tfel_unicode_mangling_209A){
+if((this->mfront_get_partial_jacobian_invert_behaviour.computePartialJacobianInvert(this->mfront_get_partial_jacobian_invert_permutation, partial_jacobian_tfel_unicode_mangling_03
C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7,
+partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1,
+partial_jacobian_tfel_unicode_mangling_0398tfel_unicode_mangling_209A)){
+this->mfront_success_reference = false;
+}
+)
+)
+TFEL_HOST_DEVICE void operator()(tfel::math::derivative_type<StressTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangl
ing_02B7,
+tfel::math::derivative_type<StrainTensor, StrainTensor>& partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1,
+tfel::math::derivative_type<real, StrainTensor>& partial_jacobian_tfel_unicode_mangling_0398tfel_unicode_mangling_209A,
+tfel::math::derivative_type<strain, StrainTensor>& partial_jacobian_rpc){
+if((this->mfront_get_partial_jacobian_invert_behaviour.computePartialJacobianInvert(this->mfront_get_partial_jacobian_invert_permutation, partial_jacobian_tfel_unicode_mangling_03
C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7,
+partial_jacobian_tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1,
+partial_jacobian_tfel_unicode_mangling_0398tfel_unicode_mangling_209A,
+partial_jacobian_rpc)){
+this->mfront_success_reference = false;
+}
+)
+)
+private:
+ModCamClay_semiExpl_coupled& mfront_get_partial_jacobian_invert_behaviour;
+const tfel::math::TinyPermutation<2*TensorSize+2>& mfront_get_partial_jacobian_invert_permutation;
+bool& mfront_success_reference;
+}; // end of struct GetPartialJacobianInvert
+GetPartialJacobianInvert getPartialJacobianInvert(*this, jacobian_permutation, mfront_success);
+#line 314 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+const auto tfel_unicode_mangling_22025tfel_unicode_mangling_1D38tfel_unicode_mangling_2215tfel_unicode_mangling_2202tfel_unicode_mangling_1D9Ctfel_unicode_mangling_1D43tfel_unico
de_mangling_1D56 = [&](const auto ptfel_unicode_mangling_1D9Ctfel_unicode_mangling_1D43tfel_unicode_mangling_1D56) {
+#line 315 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+return (1.0 - (this->Stfel_unicode_mangling_1D33tfel_unicode_mangling_02B3) - (this->Stfel_unicode_mangling_1D38tfel_unicode_mangling_02B3))*((this->Stfel_unicode_mangling_1D38) <
(1 - (this->Stfel_unicode_mangling_1D33tfel_unicode_mangling_02B3)) && (this->Stfel_unicode_mangling_1D38) > (this->Stfel_unicode_mangling_1D38tfel_unicode_mangling_02B3)
+#line 316 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+? -(this->mtfel_unicode_mangling_2C7Dtfel_unicode_mangling_1D33) / ((this->ptfel_unicode_mangling_1D47) * (1 - (this->mtfel_unicode_mangling_2C7Dtfel_unicode_mangling_1D33))) * st
d::pow(
+#line 317 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+std::pow(std::max(ptfel_unicode_mangling_1D9Ctfel_unicode_mangling_1D43tfel_unicode_mangling_1D56, 0.0) / (this->ptfel_unicode_mangling_1D47),
+#line 318 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+1.0 / (1.0 - (this->mtfel_unicode_mangling_2C7Dtfel_unicode_mangling_1D33))) +
+#line 319 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+1.0,
+#line 320 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+-(this->mtfel_unicode_mangling_2C7Dtfel_unicode_mangling_1D33) - 1) * std::pow(std::max(ptfel_unicode_mangling_1D9Ctfel_unicode_mangling_1D43tfel_unicode_mangling_1D56, 0.0) / (th
is->ptfel_unicode_mangling_1D47),
+#line 321 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+1.0 / (1.0 - (this->mtfel_unicode_mangling_2C7Dtfel_unicode_mangling_1D33)) - 1.0)
+#line 322 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+); 0.);
+#line 323 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+);
+#line 324 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+dtfel_unicode_mangling_03C3_dptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F = -(this->tfeunicode_mangling_03B1tfel_unicode_mangling_1D47) * ((this->tfeunicode_mangli
ng_03C7) - ((this->ptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F) + (this->sdptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F)) * (this->tfeunicode_mangling_2202tfel_
unicode_mangling_03C7tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1D38) * tfeunicode_mangling_22025tfel_unicode_mangling_1D38tfel_unicode_mangling_2
215tfel_unicode_mangling_2202ptfel_unicode_mangling_1D9Ctfel_unicode_mangling_1D43tfel_unicode_mangling_1D56(-(this->ptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F) + (this
->sdptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F))) * Tensor::Id() + (this->ptfel_unicode_mangling_02E2tfel_unicode_mangling_02B7) * (this->tfeunicode_mangling_2202tfel
unicode_mangling_03C6tfel_unicode_mangling_2215tfel_unicode_mangling_22025tfel_unicode_mangling_1D38) * tfeunicode_mangling_22025tfel_unicode_mangling_1D38tfel_unicode_mangling_
2215tfel_unicode_mangling_2202ptfel_unicode_mangling_1D9Ctfel_unicode_mangling_1D43tfel_unicode_mangling_1D56(-(this->ptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F) + (thi
s->sdptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F))) * Tensor::Id();
+#line 325 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+dtfel_unicode_mangling_03C3_ddT = -3 * (this->K) * (this->tfeunicode_mangling_03B1) * Tensor::Id();
+#line 326 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+((this->Stfel_unicode_mangling_1D38_dptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F) = - tfeunicode_mangling_22025tfel_unicode_mangling_1D38tfel_unicode_mangling_2215tfel_
unicode_mangling_2202ptfel_unicode_mangling_1D9Ctfel_unicode_mangling_1D43tfel_unicode_mangling_1D56(-(this->ptfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F) + (this->dp
tfel_unicode_mangling_1D38tfel_unicode_mangling_1D3F)));
+#line 328 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+if ((smt == ELASTIC) || (smt == SECANTOPERATOR))
+#line 329 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+{
+#line 330 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+((this->dtfel_unicode_mangling_03C3_ddtfel_unicode_mangling_03B5tfel_unicode_mangling_1D57tfel_unicode_mangling_1D52) = (this->tfeunicode_mangling_2202tfel_unicode_mangling_03C3t
fel_unicode_mangling_2215tfel_unicode_mangling_2202tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1);
+#line 331 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+);
+#line 332 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+else if (smt == CONSISTENTTANGENTOPERATOR)
+#line 333 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+{
+#line 334 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+Stensor4 Je;
+#line 335 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+getPartialJacobianInvert(Je);
+#line 336 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+((this->dtfel_unicode_mangling_03C3_ddtfel_unicode_mangling_03B5tfel_unicode_mangling_1D57tfel_unicode_mangling_1D52) = (this->tfeunicode_mangling_2202tfel_unicode_mangling_03C3t
fel_unicode_mangling_2215tfel_unicode_mangling_2202tfel_unicode_mangling_03B5tfel_unicode_mangling_1D49tfel_unicode_mangling_02E1) * Je;
+#line 337 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+);
+#line 338 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+else
+#line 339 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+{
+#line 340 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+return false;
+#line 341 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+);
+return mfront_success;
+}
+)
+
+/*!
+** \brief compute the sound velocity
+** \param[in] rho_m0: mass density in the reference configuration
+**/
+TFEL_HOST_DEVICE speed computeSpeedOfSound(const massdensity&) const {
+return speed(0);

```

```

+
+}
+
+TFEL_HOST_DEVICE const TangentOperator& getTangentOperator() const{
+return this->Dt;
+}
+
+TFEL_HOST_DEVICE void updateExternalStateVariables(){
+this->tfeL_unicode_mangling_0385tfeL_unicode_mangling_1057tfeL_unicode_mangling_1052 += this->dtfeL_unicode_mangling_0385tfeL_unicode_mangling_1057tfeL_unicode_mangling_1052;
+this->ptfeL_unicode_mangling_1038tfeL_unicode_mangling_103F += this->dtfeL_unicode_mangling_1038tfeL_unicode_mangling_103F;
+this->T += this->Dt;
+}
+
+
+//!
+ModCamClay_semiExpl_coupled()
+ override = default;
+
+private:
+
+TFEL_HOST_DEVICE std::pair<bool, real> computeAPrioriTimeStepScalingFactorII() const{
+return (true, this->maximal_time_step_scaling_factor);
+}
+
+TFEL_HOST_DEVICE std::pair<bool, real> computeAPosterioriTimeStepScalingFactorII() const{
+return (true, this->maximal_time_step_scaling_factor);
+}
+
+
+//! policy for treating out of bounds conditions
+OutOfBoundsPolicy policy = tfeL::material::None;
+
+}; // end of ModCamClay_semiExpl_coupled class
+
+
+template<ModellingHypothesis:Hypothesis hypothesis, typename NumericType>
+std::ostream&
+operator <<(std::ostream& os, const ModCamClay_semiExpl_coupled<hypothesis, NumericType, false>& b)
+{
+os << e"" : " << b.tfeL_unicode_mangling_0385tfeL_unicode_mangling_1057tfeL_unicode_mangling_1052 << '\n';
+os << Ae"" : " << b.dtfeL_unicode_mangling_0385tfeL_unicode_mangling_1057tfeL_unicode_mangling_1052 << '\n';
+os << o" : " << b.tfeL_unicode_mangling_03C3 << '\n';
+os << "p : " << b.ptfeL_unicode_mangling_1038tfeL_unicode_mangling_103F << '\n';
+os << A"p : " << b.dptfeL_unicode_mangling_1038tfeL_unicode_mangling_103F << '\n';
+os << "S : " << b.StfeL_unicode_mangling_1038 << '\n';
+os << A"t : " << b.dt << '\n';
+os << v" : " << b.tfeL_unicode_mangling_03B0 << '\n';
+os << M : " << b.M << '\n';
+os << ka : " << b.ka << '\n';
+os << la : " << b.la << '\n';
+os << pc_char : " << b.pc_char << '\n';
+os << "v : " << b.vtfeL_unicode_mangling_02B3tfeL_unicode_mangling_2070 << '\n';
+os << o" : " << b.tfeL_unicode_mangling_03B1 << '\n';
+os << o" : " << b.tfeL_unicode_mangling_03B1tfeL_unicode_mangling_1047 << '\n';
+os << "S : " << b.StfeL_unicode_mangling_1038tfeL_unicode_mangling_02B3 << '\n';
+os << "S : " << b.StfeL_unicode_mangling_1033tfeL_unicode_mangling_02B3 << '\n';
+os << "p : " << b.ptfeL_unicode_mangling_1047 << '\n';
+os << "v"m : " << b.mtfeL_unicode_mangling_2C7DtfeL_unicode_mangling_1033 << '\n';
+os << "bishop_type : " << b.bishop_type << '\n';
+os << "bishop_value : " << b.bishop_value << '\n';
+os << "p : " << b.ptfeL_unicode_mangling_02E2tfeL_unicode_mangling_02B7 << '\n';
+os << "m : " << b.mtfeL_unicode_mangling_02E2tfeL_unicode_mangling_02B7 << '\n';
+os << "[] S : " << b.StfeL_unicode_mangling_1038tfeL_unicode_mangling_2097 << '\n';
+os << "[] S : " << b.StfeL_unicode_mangling_1038tfeL_unicode_mangling_2095 << '\n';
+os << o"" : " << b.tfeL_unicode_mangling_03C3tfeL_unicode_mangling_02E2tfeL_unicode_mangling_02B7 << '\n';
+os << Ao"" : " << b.dtfeL_unicode_mangling_03C3tfeL_unicode_mangling_02E2tfeL_unicode_mangling_02B7 << '\n';
+os << e"" : " << b.tfeL_unicode_mangling_0385tfeL_unicode_mangling_1049tfeL_unicode_mangling_02E1 << '\n';
+os << Ae"" : " << b.dtfeL_unicode_mangling_0385tfeL_unicode_mangling_1049tfeL_unicode_mangling_02E1 << '\n';
+os << A[] : " << b.tfeL_unicode_mangling_03B8tfeL_unicode_mangling_209A << '\n';
+os << A[] : " << b.dtfeL_unicode_mangling_03B8tfeL_unicode_mangling_209A << '\n';
+os << pc : " << b.pc << '\n';
+os << e[] : " << b.tfeL_unicode_mangling_0385tfeL_unicode_mangling_1058tfeL_unicode_mangling_209AtfeL_unicode_mangling_2097 << '\n';
+os << v : " << b.vtfeL_unicode_mangling_02B3 << '\n';
+os << T : " << b.T << '\n';
+os << A"T : " << b.dt << '\n';
+os << o" : " << b.tfeL_unicode_mangling_03C3tfeL_unicode_mangling_2080 << '\n';
+os << o/o"" : " << b.tfeL_unicode_mangling_2202tfeL_unicode_mangling_03C3tfeL_unicode_mangling_2202tfeL_unicode_mangling_0385tfeL_unicode_mangling_104
9tfeL_unicode_mangling_02E1 << '\n';
+os << o" : " << b.theta << '\n';
+os << e" : " << b.epsilon << '\n';
+os << iterMax : " << b.iterMax << '\n';
+os << "minimal_time_step_scaling_factor : " << b.minimal_time_step_scaling_factor << '\n';
+os << "maximal_time_step_scaling_factor : " << b.maximal_time_step_scaling_factor << '\n';
+os << "numerical_jacobian_epsilon : " << b.numerical_jacobian_epsilon << '\n';
+return os;
+}
+
+
+//!
+// Partial specialisation for ModCamClay_semiExpl_coupled.
+//
+template<ModellingHypothesis:Hypothesis hypothesis, typename NumericType>
+class MechanicalBehaviourTraits<ModCamClay_semiExpl_coupled<hypothesis, NumericType, false> >
+{
+using size_type = unsigned short;
+static constexpr unsigned short N = ModellingHypothesisToSpaceDimension<hypothesis>::value;
+static constexpr unsigned short TVectorSize = N;
+typedef tfeL::math::StensorDimeToSize<N> StensorDimeToSize;
+static constexpr unsigned short StensorSize = StensorDimeToSize::value;
+typedef tfeL::math::TensorDimeToSize<N> TensorDimeToSize;
+static constexpr unsigned short TensorSize = TensorDimeToSize::value;
+public:
+static constexpr bool is_defined = true;
+static constexpr bool use_quantities = false;
+static constexpr bool hasStressFreeExpansion = false;
+static constexpr bool handlesThermalExpansion = false;
+static constexpr unsigned short dimension = N;
+static constexpr size_type material_properties_nb = 18;
+static constexpr size_type internal_variables_nb = 2*StensorSize+4;
+static constexpr size_type external_variables_nb = 1;
+static constexpr size_type external_variables_nb2 = 0;
+static constexpr bool hasConsistentTangentOperator = true;
+static constexpr bool isConsistentTangentOperatorSymmetric = false;
+static constexpr bool hasPredictionOperator = false;
+static constexpr bool hasAPrioriTimeStepScalingFactor = false;
+static constexpr bool hasComputeInternalEnergy = false;
+static constexpr bool hasComputeDissipatedEnergy = false;
+//!
+// \return the name of the class.
+//
+static const char* getName(){

```

```

return "ModCamClay_semiExpl_coupled";
}
+
+};
+
+
+} // end of namespace tfel::material
+
+endif /* LIB_TFELMATERIAL_MODCAMCLAY_SEMIEXPLOUPL_COUPLED_HXX */
diff --git a/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/include/TFEL/Material/ModCamClay_semiExpl_coupledBehaviourData.hxx b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/include/TFEL/Material/ModCamClay_semiExpl_coupledBehaviourData.hxx
new file mode 100644
index 0000000000..f361ffc8b0
--- /dev/null
+++ b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/include/TFEL/Material/ModCamClay_semiExpl_coupledBehaviourData.hxx
@@ -0,0 +1,348 @@
+/*!
+ * \file TFEL/Material/ModCamClay_semiExpl_coupledBehaviourData.hxx
+ * \brief this file implements the ModCamClay_semiExpl_coupledBehaviourData class.
+ *
+ * File generated by tfel version 4.2.4-dev
+ * \author Aqeel Afzal Chaudhry, Christian Silbermann, Eric Simo, Miguel Mánica, Thomas Helfer, Thomas Nagel
+ * \date 01 / 09 / 2025
+ */
+
+#ifndef LIB_TFELMATERIAL_MODCAMCLAY_SEMIEXPLOUPL_BEHAVIOUR_DATA_HXX
+#define LIB_TFELMATERIAL_MODCAMCLAY_SEMIEXPLOUPL_BEHAVIOUR_DATA_HXX
+
+#include<limits>
+#include<string>
+#include<sstream>
+#include<iostream>
+#include<stdexcept>
+#include<algorithm>
+
+#include"TFEL/Raise.hxx"
+#include"TFEL/PhysicalConstants.hxx"
+#include"TFEL/Config/TFELConfig.hxx"
+#include"TFEL/Config/TFELTypes.hxx"
+#include"TFEL/TypeTraits/IsFundamentalNumericType.hxx"
+#include"TFEL/TypeTraits/IsReal.hxx"
+#include"TFEL/Math/General/Abs.hxx"
+#include"TFEL/Math/General/IEEE754.hxx"
+#include"TFEL/Math/Array/ViewsArray1D.hxx"
+#include"TFEL/Math/Array/fsarray1D.hxx"
+#include"TFEL/Math/Array/runtime_array1D.hxx"
+#include"TFEL/Math/fsarray.hxx"
+#include"TFEL/Math/runtime_array.hxx"
+#include"TFEL/Math/tmatrix.hxx"
+#include"TFEL/Math/Matrix/tmatrix1D.hxx"
+#include"TFEL/Math/stensor.hxx"
+#include"TFEL/Math/Stensor/StensorConcept1D.hxx"
+#include"TFEL/Math/st2to2.hxx"
+#include"TFEL/Math/ST2toST2/ST2toST2Concept1D.hxx"
+#include"TFEL/Math/ST2toST2/ConvertToTangentModuli.hxx"
+#include"TFEL/Math/ST2toST2/ConvertSpatialModuliToKirchhoffJaumanRateModuli.hxx"
+#include"TFEL/Material/FiniteStrainBehaviourTangentOperator.hxx"
+#include"TFEL/Material/ModellingHypothesis.hxx"
+
+#include "MFront/GenericBehaviour/State.hxx"
+#include "MFront/GenericBehaviour/BehaviourData.hxx"
+namespace tfel::material{
+
+//!! \brief forward declaration
+template<ModellingHypothesis::Hypothesis hypothesis,typename,bool>
+class ModCamClay_semiExpl_coupledBehaviourData;
+
+//!! \brief forward declaration
+template<ModellingHypothesis::Hypothesis hypothesis, typename NumericType,bool use_gt>
+class ModCamClay_semiExpl_coupledIntegrationData;
+
+//!! \brief forward declaration
+template<ModellingHypothesis::Hypothesis hypothesis,typename NumericType>
+std::ostream&
+operator <<(std::ostream&,const ModCamClay_semiExpl_coupledBehaviourData<hypothesis, NumericType,false>);
+
+template<ModellingHypothesis::Hypothesis hypothesis, typename NumericType>
+class ModCamClay_semiExpl_coupledBehaviourData<hypothesis, NumericType,false>
+{
+
+static constexpr unsigned short N = ModellingHypothesis::ToSpaceDimension<hypothesis>::value;
+static_assert(N==1||N==2||N==3);
+static_assert(tfel::typetraits::IsFundamentalNumericType<NumericType>::cond);
+static_assert(tfel::typetraits::IsReal<NumericType>::cond);
+
+friend std::ostream& operator<< <<(std::ostream&,const ModCamClay_semiExpl_coupledBehaviourData&);
+
+/* integration data is declared friend to access driving variables at the beginning of the time step */
+friend class ModCamClay_semiExpl_coupledIntegrationData<hypothesis, NumericType, false>;
+
+static constexpr unsigned short TVectorSize = N;
+using StensorDimToSize = tfel::math::StensorDimToSize<N>;
+static constexpr unsigned short StensorSize = StensorDimToSize::value;
+using TensorDimToSize = tfel::math::TensorDimToSize<N>;
+static constexpr unsigned short TensorSize = TensorDimToSize::value;
+
+using ushort = unsigned short;
+using Types = tfel::config::Types<N, NumericType, false>;
+using Type = NumericType;
+using numeric_type = typename Types::numeric_type;
+using real = typename Types::real;
+using time = typename Types::time;
+using length = typename Types::length;
+using frequency = typename Types::frequency;
+using speed = typename Types::speed;
+using stress = typename Types::stress;
+using strain = typename Types::strain;
+using strainrate = typename Types::strainrate;
+using stressrate = typename Types::stressrate;
+using temperature = typename Types::temperature;
+using thermalexpansion = typename Types::thermalexpansion;
+using thermalconductivity = typename Types::thermalconductivity;
+using massdensity = typename Types::massdensity;
+using energydensity = typename Types::energydensity;
+using TVector = typename Types::TVector;
+using DisplacementVector = typename Types::DisplacementVector;
+using ForceVector = typename Types::ForceVector;
+using HeatFlux = typename Types::HeatFlux;
+using TemperatureGradient = typename Types::TemperatureGradient;

```

```

+using Stensor = typename Types::Stensor;
+using StressStensor = typename Types::StressStensor;
+using StressRateStensor = typename Types::StressRateStensor;
+using StrainStensor = typename Types::StrainStensor;
+using StrainRateStensor = typename Types::StrainRateStensor;
+using FrequencyStensor = typename Types::FrequencyStensor;
+using Tensor = typename Types::Tensor;
+using DeformationGradientTensor = typename Types::DeformationGradientTensor;
+using DeformationGradientRateTensor = typename Types::DeformationGradientRateTensor;
+using StressTensor = typename Types::StressTensor;
+using StiffnessTensor = typename Types::StiffnessTensor;
+using Stensor4 = typename Types::Stensor4;
+using TangentOperator = tfel::math::tvector<(StensorSize)*(StensorSize)*(StensorSize)*(1)+(StensorSize)*(1)+(1)*(1),real>;
+using PhysicalConstants = tfel::PhysicalConstants<NumericType, false>;
+
+protected:
+
+
+StrainStensor tfel_unicode_mangling_0385tfel_unicode_mangling_1057tfel_unicode_mangling_1052;
+
+StressStensor tfel_unicode_mangling_03C3;
+
+real ptfel_unicode_mangling_1038tfel_unicode_mangling_103F;
+
+real Stfel_unicode_mangling_1038;
+
+#line 57 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real tfel_unicode_mangling_0380;
+#line 60 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real M;
+#line 63 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real ka;
+#line 66 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real la;
+#line 69 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+stress pc_char;
+#line 72 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real vtfel_unicode_mangling_0283tfel_unicode_mangling_2070;
+#line 75 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real tfel_unicode_mangling_03B1;
+#line 77 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real tfel_unicode_mangling_03B1tfel_unicode_mangling_1047;
+#line 79 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3;
+#line 81 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real Stfel_unicode_mangling_1033tfel_unicode_mangling_02B3;
+#line 83 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real ptfel_unicode_mangling_1047;
+#line 85 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real mtfel_unicode_mangling_2C70tfel_unicode_mangling_1033;
+#line 88 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real bishop_type;
+#line 91 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real bishop_value;
+#line 95 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real ptfel_unicode_mangling_02E2tfel_unicode_mangling_02B7;
+#line 97 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real mtfel_unicode_mangling_02E2tfel_unicode_mangling_02B7;
+#line 99 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real Stfel_unicode_mangling_1038tfel_unicode_mangling_2097;
+#line 101 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real Stfel_unicode_mangling_1038tfel_unicode_mangling_2095;
+
+#line 36 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+StressStensor tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7;
+#line 39 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+StrainStensor tfel_unicode_mangling_0385tfel_unicode_mangling_1049tfel_unicode_mangling_02E1;
+#line 42 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real tfel_unicode_mangling_039Btfel_unicode_mangling_209A;
+#line 47 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+stress pc;
+#line 50 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real tfel_unicode_mangling_0385tfel_unicode_mangling_1058tfel_unicode_mangling_209Atfel_unicode_mangling_2097;
+#line 53 "/home/aqeel/ogs/MaterialLib/SolidModels/MFront/ModCamClay_semiExpl_coupled.mfront"
+real vtfel_unicode_mangling_02B3;
+temperature T;
+
+public:
+
+
+/*!
+** \brief Default constructor
+**/
+ModCamClay_semiExpl_coupledBehaviourData()
+{}
+
+/*!
+** \brief copy constructor
+**/
+ModCamClay_semiExpl_coupledBehaviourData(const ModCamClay_semiExpl_coupledBehaviourData& src)
+: tfel_unicode_mangling_0385tfel_unicode_mangling_1057tfel_unicode_mangling_1052(src.tfel_unicode_mangling_0385tfel_unicode_mangling_1057tfel_unicode_mangling_1052),
+tfel_unicode_mangling_03C3(src.tfel_unicode_mangling_03C3),
+ptfel_unicode_mangling_1038tfel_unicode_mangling_103F(src.ptfel_unicode_mangling_1038tfel_unicode_mangling_103F),
+Stfel_unicode_mangling_1038(src.Stfel_unicode_mangling_1038),
+tfel_unicode_mangling_0380(src.tfel_unicode_mangling_0380),
+M(src.M),
+ka(src.ka),
+la(src.la),
+pc_char(src.pc_char),
+vtfel_unicode_mangling_0283tfel_unicode_mangling_2070(src.vtfel_unicode_mangling_0283tfel_unicode_mangling_2070),
+tfel_unicode_mangling_03B1(src.tfel_unicode_mangling_03B1),
+tfel_unicode_mangling_03B1tfel_unicode_mangling_1047(src.tfel_unicode_mangling_03B1tfel_unicode_mangling_1047),
+Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3(src.Stfel_unicode_mangling_1038tfel_unicode_mangling_02B3),
+Stfel_unicode_mangling_1033tfel_unicode_mangling_02B3(src.Stfel_unicode_mangling_1033tfel_unicode_mangling_02B3),
+ptfel_unicode_mangling_1047(src.ptfel_unicode_mangling_1047),
+mtfel_unicode_mangling_2C70tfel_unicode_mangling_1033(src.mtfel_unicode_mangling_2C70tfel_unicode_mangling_1033),
+bishop_type(src.bishop_type),
+bishop_value(src.bishop_value),
+ptfel_unicode_mangling_02E2tfel_unicode_mangling_02B7(src.ptfel_unicode_mangling_02E2tfel_unicode_mangling_02B7),
+mtfel_unicode_mangling_02E2tfel_unicode_mangling_02B7(src.mtfel_unicode_mangling_02E2tfel_unicode_mangling_02B7),
+Stfel_unicode_mangling_1038tfel_unicode_mangling_2097(src.Stfel_unicode_mangling_1038tfel_unicode_mangling_2097),
+Stfel_unicode_mangling_1038tfel_unicode_mangling_2095(src.Stfel_unicode_mangling_1038tfel_unicode_mangling_2095),
+tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7(src.tfel_unicode_mangling_03C3tfel_unicode_mangling_02E2tfel_unicode_mangling_02B7),
+tfel_unicode_mangling_0385tfel_unicode_mangling_1049tfel_unicode_mangling_02E1(src.tfel_unicode_mangling_0385tfel_unicode_mangling_1049tfel_unicode_mangling_02E1),
+tfel_unicode_mangling_039Btfel_unicode_mangling_209A(src.tfel_unicode_mangling_039Btfel_unicode_mangling_209A),
+pc(src.pc),
+tfel_unicode_mangling_0385tfel_unicode_mangling_1058tfel_unicode_mangling_209Atfel_unicode_mangling_2097(src.tfel_unicode_mangling_0385tfel_unicode_mangling_1058tfel_unicode_mangl
ing_209Atfel_unicode_mangling_2097),
+vtfel_unicode_mangling_02B3(src.vtfel_unicode_mangling_02B3),

```

```

+T(src.T)
+{}
+
+/*
+ * \brief constructor for the Generic interface
+ * \param[in] mgb_d: behaviour data
+ */
+ModCamClay_semiExpl_coupledBehaviourData(const mfront::gb::BehaviourData& mgb_d)
+: tfeL_uniCode_mangling_0380(mgb_d.s1.material_properties[0]),
+M(mgb_d.s1.material_properties[1]),
+ka(mgb_d.s1.material_properties[2]),
+la(mgb_d.s1.material_properties[3]),
+pc_char(mgb_d.s1.material_properties[4]),
+vtfeL_uniCode_mangling_0283tfeL_uniCode_mangling_2070(mgb_d.s1.material_properties[5]),
+tfeL_uniCode_mangling_0381(mgb_d.s1.material_properties[6]),
+tfeL_uniCode_mangling_0381tfeL_uniCode_mangling_1047(mgb_d.s1.material_properties[7]),
+StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_0283(mgb_d.s1.material_properties[8]),
+StfeL_uniCode_mangling_1033tfeL_uniCode_mangling_0283(mgb_d.s1.material_properties[9]),
+ptfeL_uniCode_mangling_1047(mgb_d.s1.material_properties[10]),
+mtfeL_uniCode_mangling_2C70tfeL_uniCode_mangling_1033(mgb_d.s1.material_properties[11]),
+bishop_type(mgb_d.s1.material_properties[12]),
+bishop_value(mgb_d.s1.material_properties[13]),
+ptfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287(mgb_d.s1.material_properties[14]),
+mtfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287(mgb_d.s1.material_properties[15]),
+StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_2097(mgb_d.s1.material_properties[16]),
+StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_2095(mgb_d.s1.material_properties[17]),
+tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287(tfeL::math::map<StressTensor>(mgb_d.s0.internal_state_variables + 0)),
+tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1(tfeL::math::map<StrainTensor>(mgb_d.s0.internal_state_variables + TensorSize)),
+tfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A(mgb_d.s0.internal_state_variables[2+TensorSize]),
+pc(mgb_d.s0.internal_state_variables[2+TensorSize+1]),
+tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1058tfeL_uniCode_mangling_209AtfeL_uniCode_mangling_2097(mgb_d.s0.internal_state_variables[2+TensorSize+2]),
+vtfeL_uniCode_mangling_0283(mgb_d.s0.internal_state_variables[2+TensorSize+3]),
+T(mgb_d.s0.external_state_variables[0])
+{
+this->tfeL_uniCode_mangling_03C3 = tfeL::math::map<StressTensor>(mgb_d.s0.thermodynamic_forces);
+this->StfeL_uniCode_mangling_1038 = mgb_d.s0.thermodynamic_forces[TensorSize];
+}
+
+
+/*
+ * \brief Assignment operator
+ */
+ModCamClay_semiExpl_coupledBehaviourData&
+operator=(const ModCamClay_semiExpl_coupledBehaviourData& src){
+this->tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1057tfeL_uniCode_mangling_1052 = src.tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1057tfeL_uniCode_mangling_1052;
+this->tfeL_uniCode_mangling_03C3 = src.tfeL_uniCode_mangling_03C3;
+this->ptfeL_uniCode_mangling_1038tfeL_uniCode_mangling_103F = src.ptfeL_uniCode_mangling_1038tfeL_uniCode_mangling_103F;
+this->StfeL_uniCode_mangling_1038 = src.StfeL_uniCode_mangling_1038;
+this->tfeL_uniCode_mangling_0380 = src.tfeL_uniCode_mangling_0380;
+this->M = src.M;
+this->ka = src.ka;
+this->la = src.la;
+this->pc_char = src.pc_char;
+this->vtfeL_uniCode_mangling_0283tfeL_uniCode_mangling_2070 = src.vtfeL_uniCode_mangling_0283tfeL_uniCode_mangling_2070;
+this->tfeL_uniCode_mangling_0381 = src.tfeL_uniCode_mangling_0381;
+this->tfeL_uniCode_mangling_0381tfeL_uniCode_mangling_1047 = src.tfeL_uniCode_mangling_0381tfeL_uniCode_mangling_1047;
+this->StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_0283 = src.StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_0283;
+this->StfeL_uniCode_mangling_1033tfeL_uniCode_mangling_0283 = src.StfeL_uniCode_mangling_1033tfeL_uniCode_mangling_0283;
+this->ptfeL_uniCode_mangling_1047 = src.ptfeL_uniCode_mangling_1047;
+this->mtfeL_uniCode_mangling_2C70tfeL_uniCode_mangling_1033 = src.mtfeL_uniCode_mangling_2C70tfeL_uniCode_mangling_1033;
+this->bishop_type = src.bishop_type;
+this->bishop_value = src.bishop_value;
+this->ptfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287 = src.ptfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287;
+this->mtfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287 = src.mtfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287;
+this->StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_2097 = src.StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_2097;
+this->StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_2095 = src.StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_2095;
+this->tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287 = src.tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287;
+this->tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1 = src.tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1;
+this->tfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A = src.tfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A;
+this->pc = src.pc;
+this->tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1058tfeL_uniCode_mangling_209AtfeL_uniCode_mangling_2097 = src.tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1058tfeL_uniCode_mangling_209AtfeL_uniCode_mangling_2097;
+de_mangling_209AtfeL_uniCode_mangling_2097;
+this->vtfeL_uniCode_mangling_0283 = src.vtfeL_uniCode_mangling_0283;
+this->T = src.T;
+return *this;
+}
+
+
+TFEL_HOST_DEVICE void exportStateData(mfront::gb::State& mgb_s1) const
+{
+using namespace tfeL::math;
+tfeL::math::map<StressTensor>(mgb_s1.thermodynamic_forces) = this->tfeL_uniCode_mangling_03C3;
+mgb_s1.thermodynamic_forces[TensorSize] = this->StfeL_uniCode_mangling_1038;
+tfeL::math::map<StressTensor>(mgb_s1.internal_state_variables) = this->tfeL_uniCode_mangling_03C3tfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287;
+tfeL::math::map<StrainTensor>(mgb_s1.internal_state_variables + TensorSize) = this->tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1049tfeL_uniCode_mangling_02E1;
+mgb_s1.internal_state_variables[2+TensorSize] = this->tfeL_uniCode_mangling_0398tfeL_uniCode_mangling_209A;
+mgb_s1.internal_state_variables[2+TensorSize+1] = this->pc;
+mgb_s1.internal_state_variables[2+TensorSize+2] = this->tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1058tfeL_uniCode_mangling_209AtfeL_uniCode_mangling_2097;
+mgb_s1.internal_state_variables[2+TensorSize+3] = this->vtfeL_uniCode_mangling_0283;
+} // end of exportStateData
+
+}; // end of ModCamClay_semiExpl_coupledBehaviourDataclass
+
+
+template<ModellingHypothesis:Hypothesis hypothesis, typename NumericType>
+std::ostream&
+operator <<(std::ostream& os,const ModCamClay_semiExpl_coupledBehaviourData&hypothesis, NumericType, false& b)
+{
+os << "t" << " b.tfeL_uniCode_mangling_0385tfeL_uniCode_mangling_1057tfeL_uniCode_mangling_1052 << '\n';
+os << "0" << " b.tfeL_uniCode_mangling_03C3 << '\n';
+os << "p" << " b.ptfeL_uniCode_mangling_1038tfeL_uniCode_mangling_103F << '\n';
+os << "S" << " b.StfeL_uniCode_mangling_1038 << '\n';
+os << "V" << " b.tfeL_uniCode_mangling_0380 << '\n';
+os << "M" << " b.M << '\n';
+os << "ka" << " b.ka << '\n';
+os << "la" << " b.la << '\n';
+os << "pc_char" << " b.pc_char << '\n';
+os << "v" << " b.vtfeL_uniCode_mangling_0283tfeL_uniCode_mangling_2070 << '\n';
+os << "0" << " b.tfeL_uniCode_mangling_0381 << '\n';
+os << "0" << " b.tfeL_uniCode_mangling_0381tfeL_uniCode_mangling_1047 << '\n';
+os << "s" << " b.StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_0283 << '\n';
+os << "s" << " b.StfeL_uniCode_mangling_1033tfeL_uniCode_mangling_0283 << '\n';
+os << "p" << " b.ptfeL_uniCode_mangling_1047 << '\n';
+os << "v" << " b.mtfeL_uniCode_mangling_2C70tfeL_uniCode_mangling_1033 << '\n';
+os << "bishop_type" << " b.bishop_type << '\n';
+os << "bishop_value" << " b.bishop_value << '\n';
+os << "p" << " b.ptfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287 << '\n';
+os << "m" << " b.mtfeL_uniCode_mangling_02E2tfeL_uniCode_mangling_0287 << '\n';
+os << "[] S" << " b.StfeL_uniCode_mangling_1038tfeL_uniCode_mangling_2097 << '\n';

```



```

+ "import time\n",
+ "import math\n",
+ "from ogs6py import ogs\n",
+ "import vtkIO\n",
+ "from ogs6py import ogs\n",
+ "import itertools\n",
+ "import pyvista as pv\n",
+ "\n",
+ "# Some plot settings\n",
+ "#plt.style.use('seaborn-deep')\n",
+ "plt.rcParams[\"lines.linewidth\"] = 2.0\n",
+ "plt.rcParams[\"lines.color\"] = 'black'\n",
+ "plt.rcParams[\"figure.facecolor\"] = 'white'\n",
+ "plt.rcParams[\"legend.frameon\"] = True\n",
+ "plt.rcParams[\"figure.figsize\"] = (10, 6)\n",
+ "plt.rcParams[\"font.family\"] = 'serif'\n",
+ "plt.rcParams[\"legend.fontsize\"] = 12\n",
+ "plt.rcParams[\"legend.framealpha\"] = 0.4\n",
+ "plt.rcParams[\"font.size\"] = 18\n",
+ "plt.rcParams[\"axes.spines.right\"] = False\n",
+ "plt.rcParams[\"axes.spines.top\"] = False\n",
+ "plt.rcParams[\"axes.spines.left\"] = True\n",
+ "plt.rcParams[\"axes.spines.bottom\"] = True\n",
+ "plt.rcParams[\"axes.axisbelow\"] = True
+ ]
+ },
+ {
+ "cell_type": "code",
+ "execution_count": 2,
+ "metadata": {},
+ "outputs": [],
+ "source": [
+ "def compute_chi(S_L, bishop_type, bishop_value):\n",
+ "    S_L = np.asarray(S_L)\n",
+ "    if bishop_type == 0:\n",
+ "        chi = S_L*bishop_value\n",
+ "    elif bishop_type == 1:\n",
+ "        S_cut = bishop_value\n",
+ "        chi = np.zeros_like(S_L, dtype=float)\n",
+ "        chi = np.where(S_L < S_cut, 0.0, chi)\n",
+ "        chi = np.where(S_L > 1.0, 1.0, chi)\n",
+ "        mask = (S_L >= S_cut) & (S_L <= 1.0)\n",
+ "        chi = np.where(mask, (S_L - S_cut) / max(1e-12, 1.0 - S_cut), chi)\n",
+ "    else:\n",
+ "        raise ValueError(\"bishop_type must be 0 (power-law) or 1 (cutoff)\")\n",
+ "    return chi\n",
+ "\n",
+ "\n",
+ "def compute_initial_values(sigma_pre, bishop_type, bishop_value, alpha_B, p_b, n_S, S_L_res, S_G_res,\n",
+ "    S_L: float = None,\n",
+ "    p0: float = None):\n",
+ "    if (S_L is None) == (p0 is None):\n",
+ "        raise ValueError(\"Must provide exactly one of 'S_L' or 'p0', not both or neither.\")\n",
+ "\n",
+ "    if S_L is not None:\n",
+ "        S_e = (S_L - S_L_res) / (1 - S_L_res - S_G_res)\n",
+ "        p_cap = p_b * ((S_e)*(-1/m_S) - 1)*((1-m_S)\n",
+ "        p0 = -p_cap\n",
+ "\n",
+ "    else:\n",
+ "        p_cap = -p0\n",
+ "        f = (p_cap / p_b)*((1/(1-m_S))\n",
+ "        S_e = (1 + f)*((-m_S)\n",
+ "        S_L = S_e * (1 - S_L_res - S_G_res) + S_L_res\n",
+ "\n",
+ "    chi = compute_chi(S_L, bishop_type, bishop_value)\n",
+ "\n",
+ "    sigma_total = sigma_pre - alpha_B * chi * p0\n",
+ "\n",
+ "    print(f\"initial stress (total): {np.round(sigma_total,8)}\")\n",
+ "    print(f\"initial fluid pressure: {np.round(p0,2)}\")\n",
+ "    print(f\"initial liquid saturation: {np.round(S_L,4)}\")\n",
+ "\n",
+ "    return sigma_total, p0, S_L\n",
+ ]
+ },
+ {
+ "cell_type": "code",
+ "execution_count": null,
+ "metadata": {},
+ "outputs": [],
+ "source": []
+ },
+ {
+ "cell_type": "code",
+ "execution_count": 37,
+ "metadata": {},
+ "outputs": [
+ {
+ "name": "stdout",
+ "output_type": "stream",
+ "text": [
+ "initial stress (total): -5000000.0\n",
+ "initial fluid pressure: -4000000.0\n",
+ "initial liquid saturation: 0.50000000000002\n",
+ "1515151515.151515\n",
+ ]
+ }
+ ],
+ "source": [
+ "# only strain driven for the time being\n",
+ "# Caution: uniaxial : works for both Bishops Power law and cutoff\n",
+ "# triaxial : works with Cutoff only for the time being\n",
+ "test_type = \"triaxial\"\n",
+ "\n",
+ "# Bishops model settings\n",
+ "bishops_model_type = 1 # only 0: power law model or 1: cutoff model are allowed\n",
+ "bishops_model_value = 1\n",
+ "\n",
+ "assert test_type in (\"uniaxial\", \"triaxial\"), f\"Invalid test_type: {test_type}\"\n",
+ "assert (test_type != \"triaxial\" or (bishops_model_type != 0), \"Triaxial test is not set up to work with Bishop's Power Law yet.\"\n",
+ "\n",
+ "temperature_ic = 293\n",
+ "temperature_bc = 100 + temperature_ic\n",
+ "# swelling stress settings\n",
+ "pmax = 4e6\n",
+ "exp_p = 1\n",

```

```

+ "Smin_p = 0\n",
+ "Smax_p = 1\n",
+ "\n",
+ "alpha = 1e-5\n",
+ "intrinsic_permeability = 1e-21\n",
+ "\n",
+ "# Biot-willis coefficient\n",
+ "alpha_B = 1\n",
+ "\n",
+ "# van-Genuchten saturation settings (relative_permeability settings need to be changed in the project file directly)\n",
+ "p_l = -40e6*1 ; p_b = 16e6; m_5 = 0.4014347581153\n",
+ "\n",
+ "S_L_res = 0.0\n",
+ "S_G_res = 0.0\n",
+ "\n",
+ "nu = 0.3 # Poisson ratio\n",
+ "la = 7.7e-2 # Slope of the virgin consolidation line\n",
+ "ka = 6.6e-3 # Slope of the swelling line\n",
+ "M = 1.2 # Slope of the critical state line (CSL)\n",
+ "phi = 0.4\n",
+ "v0 = 1/(1-phi) # Initial volume ratio\n",
+ "\n",
+ "pc0 = 5e6 #Initial pre-consolidation pressure in Pa\n",
+ "\n",
+ "sig_pre_eff = -5e6\n",
+ "\n",
+ "\n",
+ "initial_stress, pressure_ic, S_L = compute_initial_values(sig_pre_eff, bishops_model_type, bishops_model_value, alpha_B, p_b, m_5, S_L_res, S_G_res, \n",
+ "                p0 = p_l)\n",
+ "pressure_final = 0\n",
+ "\n",
+ "days = 100\n",
+ "interval = 2\n",
+ "t_list = [86400.0*t for t in np.arange(0, days + 1, interval)]\n",
+ "t_end = t_list[-1]\n",
+ "init_dt = 1#86400\n",
+ "\n",
+ "E = 3 * (1 - 2 * nu) / (1 - phi) * pc0 / ka\n",
+ "\n",
+ "e_axi = 0.02\n",
+ "print(E)\n",
+ " "
+ ]
+ },
+ {
+ "cell_type": "code",
+ "execution_count": null,
+ "metadata": {},
+ "outputs": [],
+ "source": []
+ },
+ {
+ "cell_type": "code",
+ "execution_count": 38,
+ "metadata": {
+ "scrolled": true
+ },
+ "outputs": [],
+ "source": [
+ "f_mfront = \"saturation.prj\"\n",
+ "f_mfront_coupled = \"saturation_coupled.prj\"\n",
+ "def modify_prj():\n",
+ "    model = ogs.OGS(INPUT_FILE=f_mfront, PROJECT_FILE=f_mfront)\n",
+ "    model.remove_element(\"./media/medium/properties/property[name='saturation']\")\n",
+ "    model.add_block(\"property\", parent_xpath=\"./media/medium/properties\", \n",
+ "                    taglist=[\"name\", \"type\", \"p_b\", \"exponent\", \"residual_liquid_saturation\", \"residual_gas_saturation\"], \n",
+ "                    textlist=[\"saturation\", \"SaturationVanGenuchten\", p_b, m_5, S_L_res, S_G_res])\n",
+ "\n",
+ "    model.remove_element(\"./media/medium/properties/property[name='bishops_effective_stress']\")\n",
+ "    if bishops_model_type == 0:\n",
+ "        model.add_block(\"property\", parent_xpath=\"./media/medium/properties\", \n",
+ "                        taglist=[\"name\", \"type\", \"exponent\"], textlist=[\"bishops_effective_stress\", \"BishopsPowerLaw\", bishops_model_value])\n",
+ "    elif bishops_model_type == 1:\n",
+ "        model.add_block(\"property\", parent_xpath=\"./media/medium/properties\", \n",
+ "                        taglist=[\"name\", \"type\", \"cutoff_value\"], textlist=[\"bishops_effective_stress\", \"BishopsSaturationCutoff\", bishops_model_value])\n",
+ "    else:\n",
+ "        raise ValueError(\"bishop_type must be 0 (power-law) or 1 (cutoff)\")\n",
+ "\n",
+ "    model.remove_element(\"./media/medium/phases/phase[type='Solid']/properties/property[name='swelling_stress_rate']\")\n",
+ "    model.add_block(\"property\", parent_xpath=\"./media/medium/phases/phase[type='Solid']/properties\", \n",
+ "                    taglist=[\"name\", \"type\", \"swelling_pressures\", \"exponents\", \"lower_saturation_limit\", \"upper_saturation_limit\"], \n",
+ "                    textlist=[\"swelling_stress_rate\", \"SaturationDependentSwelling\", f\"{pmax} {pmax} {pmax}\", f\"{exp_p} {exp_p} {exp_p}\", f\"{Smin_p}\", f\"{Smax_p}\"])\n",
+ "\n",
+ "    model.replace_parameter(\"temperature_ic\", \"Constant\", [\"value\"], [temperature_ic])\n",
+ "    model.replace_parameter(\"alpha\", \"Constant\", [\"value\"], [alpha])\n",
+ "    model.replace_parameter(\"alpha_B\", \"Constant\", [\"value\"], [alpha_B])\n",
+ "    model.replace_parameter(\"bishops_model_type\", \"Constant\", [\"value\"], [bishops_model_type])\n",
+ "    model.replace_parameter(\"bishops_model_value\", \"Constant\", [\"value\"], [bishops_model_value])\n",
+ "\n",
+ "    model.replace_parameter(\"E\", \"Constant\", [\"value\"], [E])\n",
+ "    model.replace_parameter(\"nu\", \"Constant\", [\"value\"], [nu])\n",
+ "    model.replace_parameter(\"M\", \"Constant\", [\"value\"], [M])\n",
+ "    model.replace_parameter(\"ka\", \"Constant\", [\"value\"], [ka])\n",
+ "    model.replace_parameter(\"la\", \"Constant\", [\"value\"], [la])\n",
+ "    model.replace_parameter(\"pc0\", \"Constant\", [\"value\"], [pc0])\n",
+ "    model.replace_parameter(\"v0\", \"Constant\", [\"value\"], [v0])\n",
+ "    model.replace_parameter(\"phi\", \"Constant\", [\"value\"], [phi])\n",
+ "\n",
+ "    model.replace_parameter(\"intrinsic_permeability\", \"Constant\", [\"value\"], [intrinsic_permeability])\n",
+ "    model.replace_parameter(\"p_b\", \"Constant\", [\"value\"], [p_b])\n",
+ "    model.replace_parameter(\"m_5\", \"Constant\", [\"value\"], [m_5])\n",
+ "    model.replace_parameter(\"S_L_res\", \"Constant\", [\"value\"], [S_L_res])\n",
+ "    model.replace_parameter(\"S_G_res\", \"Constant\", [\"value\"], [S_G_res])\n",
+ "\n",
+ "    model.replace_parameter(\"initial_stress\", \"Constant\", [\"values\"], [f\"{initial_stress} {initial_stress} {initial_stress} 0.0\"])\n",
+ "    model.replace_parameter(\"pressure_ic\", \"Constant\", [\"value\"], [pressure_ic])\n",
+ "    model.replace_parameter(\"disp_value\", \"Constant\", [\"value\"], [-e_axi])\n",
+ "\n",
+ "    model.remove_element(\"./curves/curve[name='pressure_ramp_curve']\")\n",
+ "    model.add_block(\"curve\", parent_xpath=\"./curves\", \n",
+ "                    taglist=[\"name\", \"coords\", \"values\"], \n",
+ "                    textlist=[\"pressure_ramp_curve\", f\"0.0 {t_end}\", f\"1.0 {pressure_final/pressure_ic if pressure_ic!=0 else 0}\"])\n",
+ "    model.remove_element(\"./curves/curve[name='temperature_curve']\")\n",
+ "    model.add_block(\"curve\", parent_xpath=\"./curves\", \n",
+ "                    taglist=[\"name\", \"coords\", \"values\"], \n",
+ "                    textlist=[\"temperature_curve\", f\"0.0 {t_end}\", f\"{temperature_ic} {temperature_bc}\"])\n",

```

```

+ "
+ "    \n",
+ "    model.remove_element(\"./curves/curve[name='ax_disp_curve']\")\n",
+ "    model.add_block(\"curve\", parent_xpath=\"./curves\", \n",
+ "        taglist=[\"name\", \"coords\", \"values\"], \n",
+ "        textlist=[\"ax_disp_curve\", f\"0.0 {t_end}\", \"0 1\"])\n",
+ "
+ "    \n",
+ "    \n",
+ "    model.remove_element(\"./curves/curve[name='pre_loading_curve']\")\n",
+ "    model.add_block(\"curve\", parent_xpath=\"./curves\", \n",
+ "        taglist=[\"name\", \"coords\", \"values\"], \n",
+ "        textlist=[\"pre_loading_curve\", f\"0.0 {t_end}\", f\"{initial_stress} {initial_stress}\"])\n",
+ "
+ "    \n",
+ "    \n",
+ "    model.remove_element(\"./process_variables/process_variable[name='displacement']/boundary_conditions/boundary_condition[geometry='right']\")\n",
+ "    bc_type = \"Dirichlet\"; bc_parameter = \"dirichlet0\"\n",
+ "    if test_type == \"triaxial\":\n",
+ "        bc_type = \"Neumann\"; bc_parameter = \"confining_pressure\"\n",
+ "    model.add_block(\"boundary_condition\", parent_xpath=\"./process_variables/process_variable[name='displacement']/boundary_conditions\", \n",
+ "        taglist=[\"geometrical_set\", \"geometry\", \"component\", \"type\", \"parameter\"], \n",
+ "        textlist=[\"square_1x1_geometry\", \"right\", \"0\", bc_type, bc_parameter])\n",
+ "
+ "    \n",
+ "    \n",
+ "    model.replace_text(f_mfront[:-4], xpath=\"./time_loop/output/prefix\")\n",
+ "    model.replace_text(t_end, xpath=\"./time_loop/processes/process/time_stepping/t_end\")\n",
+ "    model.replace_text(init_dt, xpath=\"./time_loop/processes/process/time_stepping/initial_dt\")\n",
+ "    model.replace_text(t_end*interval/days, xpath=\"./time_loop/processes/process/time_stepping/maximum_dt\")\n",
+ "    model.replace_text(\" \".join(str(t) for t in t_list), xpath=\"./time_loop/output/fixed_output_times\")\n",
+ "
+ "    \n",
+ "    model.write_input()\n",
+ "
+ "    \n",
+ "    # second .prj\n",
+ "    model = ogs.OGS(INPUT_FILE=f_mfront, PROJECT_FILE=f_mfront_coupled)\n",
+ "
+ "    \n",
+ "    m_path = \"./processes/process/constitutive_relation/material_properties\"\n",
+ "    props = [\"InitialVolumeRatio\", \"ThermalExpansion\", \"BiotCoefficient\", \"BishopsModelType\", \"BishopsModelValue\", \n",
+ "        \"ResidualLiquidSaturation\", \"ResidualGasSaturation\", \"BubblePressure\", \"VanGenuchtenExponent_m\", \n",
+ "        \"SwellingPressures\", \"Exponents\", \"LowerSaturationLimit\", \"UpperSaturationLimit\"]\n",
+ "    params = [\"v0\", \"alpha\", \"alpha_B\", \"bishops_model_type\", \"bishops_model_value\", \"S_L_res\", \"S_G_res\", \"p_b\", \"m_S\", \"pmax\", \"exp_p\", \"Smin_p\", \n",
+ "        Smax_p\"]\n",
+ "
+ "    \n",
+ "    model.remove_element(m_path+\"/material_property[@name='YoungModulus']\")\n",
+ "    model.remove_element(m_path+\"/material_property[@name='InitialVolumeRatio']\")\n",
+ "    for i in range(len(props)):\n",
+ "        model.add_block(blocktag=\"material_property\", block_attrb={\"name\": props[i], \"parameter\": params[i], parent_xpath=m_path})\n",
+ "
+ "    \n",
+ "    model.processes.add_secondary_variable(internal_name=\"sigma_total\", output_name=\"sigma_total\")\n",
+ "    model.remove_element(\"./processes/process/secondary_variables/secondary_variable[@internal_name='sigma']\")\n",
+ "    model.replace_text(\"StressSaturationStrainPressureTemperature\", xpath=\"./processes/process/subtype\")\n",
+ "    model.replace_text(\"ModCamClay_semiExpL_coupled\", xpath=\"./processes/process/constitutive_relation/behaviour\")\n",
+ "    model.replace_text(f_mfront_coupled[:-4], xpath=\"./time_loop/output/prefix\")\n",
+ "    model.deactivate_property(name=\"swelling_stress_rate\", mediumid=0, phase=\"Solid\")\n",
+ "    model.parameters.add_parameter(name=\"pmax\", type=\"Constant\", value=pmax)\n",
+ "    model.parameters.add_parameter(name=\"exp_p\", type=\"Constant\", value=exp_p)\n",
+ "    model.parameters.add_parameter(name=\"Smin_p\", type=\"Constant\", value=Smin_p)\n",
+ "    model.parameters.add_parameter(name=\"Smax_p\", type=\"Constant\", value=Smax_p)\n",
+ "
+ "    model.write_input()\n",
+ "
+ "    modify_prj()\n",
+ "
+ "    \n",
+ "    \"\"Idiff saturation.prj saturation_coupled.prj\"
+ "
+ "    ]
+ "
+ "    },
+ "
+ "    {
+ "
+ "        \"cell_type\": \"markdown\",
+ "        \"metadata\": {},
+ "        \"source\": [
+ "
+ "            \"## Run OGS simulations\"
+ "
+ "        ],
+ "
+ "    },
+ "
+ "    {
+ "
+ "        \"cell_type\": \"code\",
+ "        \"execution_count\": 39,
+ "        \"metadata\": {},
+ "        \"outputs\": [
+ "
+ "            {
+ "
+ "                \"name\": \"stdout\",
+ "                \"output_type\": \"stream\",
+ "                \"text\": [
+ "
+ "                    \"MFront run time = 00:00:00(h:m:s)\\n\",
+ "                    \"MFront run time = 00:00:20(h:m:s)\\n\"
+ "
+ "                ]
+ "
+ "            },
+ "
+ "        ],
+ "
+ "        \"source\": [
+ "
+ "            \"run_ogs = True\\n\",
+ "            \"ogs_source_dir = os.getcwd()\\n\",
+ "            \" : -len('Tests/Data/ThermoRichardsMechanics/MFront/TRM_JMC_coupled')\\n\",
+ "            \"]\\n\",
+ "            \"assert os.path.exists(ogs_source_dir)\\n\",
+ "            \"ogs_build_dir = os.path.join(ogs_source_dir, 'build')\\n\",
+ "            \"assert os.path.exists(ogs_build_dir)\\n\",
+ "            \"ogs_path = os.path.realpath(os.path.join(ogs_build_dir, 'bin/ogs'))\\n\",
+ "            \"assert os.path.exists(ogs_path)\\n\",
+ "
+ "            \"\\n\",
+ "
+ "            \"if run_ogs:\\n\",
+ "            \"    !rm out/*\\n\",
+ "            \"    #\\\"\\\"\\\"\\n\",
+ "            \"    start = time.time()\\n\",
+ "            \"    assert (\\n\",
+ "            \"        os.system(\\n\",
+ "            \"            f'\\\"{ogs_path} -o out {f_mfront} > ogs.log 2>1\\\"\\n\",
+ "            \"        )\\n\",
+ "            \"        == 0\\n\",
+ "            \"    )\\n\",
+ "            \"    e = int(time.time() - start)\\n\",
+ "            \"    print('MFront run time = {:02d}:{:02d}:{:02d}(h:m:s)'.format(e // 3600, (e % 3600 // 60), e % 60))\\n\",
+ "            \"    #\\\"\\\"\\\"\\n\",
+ "            \"    \\n\",
+ "            \"    start = time.time()\\n\",
+ "            \"    assert (\\n\",
+ "            \"        os.system(\\n\",
+ "            \"            f'\\\"{ogs_path} -o out {f_mfront_coupled} > ogs.log 2>1\\\"\\n\",
+ "            \"        )\\n\",
+ "            \"        == 0\\n\",
+ "            \"    )\\n\",
+ "            \"    e = int(time.time() - start)\\n\",
+ "            \"    print('MFront run time = {:02d}:{:02d}:{:02d}(h:m:s)'.format(e // 3600, (e % 3600 // 60), e % 60))\"
+ "
+ "        ]
+ "
+ "    }
+ "

```

```

+ },
+ {
+   "cell_type": "code",
+   "execution_count": 40,
+   "metadata": {
+     "scrolled": true
+   },
+   "outputs": [],
+   "source": [
+     "behaviors_dir = \"../MaterialLib/SolidModels/Mfront/ModCamClay_semiExp_coupled.mfront\"\n",
+     "MCC_coupled_path = ogs_source_dir + behaviors_dir\n",
+     "# Uncomment below line once and if there is a change in mfront model (which may require restart of the notebook)\n",
+     "#!rm -rf include src && mfront --obuild --interface-generic $MCC_coupled_path"
+   ]
+ },
+ {
+   "cell_type": "markdown",
+   "metadata": {},
+   "source": [
+     "## MTest"
+   ]
+ },
+ {
+   "cell_type": "code",
+   "execution_count": 42,
+   "metadata": {},
+   "outputs": [
+     {
+       "name": "stdout",
+       "output_type": "stream",
+       "text": [
+         "MTest done!\n"
+       ]
+     }
+   ],
+   "source": [
+     "def saturation_mtest(): \n",
+     "    #test.setVerboseMode(mtest.VerboseLevel.VERBOSE_LEVEL3)\n",
+     "    mtest.setVerboseMode(mtest.VerboseLevel.VERBOSE_QUIET)\n",
+     "    m = mtest.MTest()\n",
+     "    m.setMaximumNumberOfSubSteps(1)\n",
+     "    m.setModelLingHypothesis(\"Axisymmetrical\")\n",
+     "    m.setBehaviour(\"generic\", \"../src/LibBehaviour.so\", \"ModCamClay_semiExp_coupled\")\n",
+     "    m.setGradientsInitialValues([0,0,0, pressure_ic])\n",
+     "    m.setThermodynamicForcesInitialValues([initial_stress, initial_stress, initial_stress, 0.0, S_L]) #check\n",
+     "\n",
+     "    m.setImposedGradient(\"StrainRZ\", 0.0)\n",
+     "\n",
+     "    m.setImposedGradient(\"StrainZZ\", {0: 0, t_end: -e_axi})\n",
+     "    \n",
+     "    if test_type == \"triaxial\": \n",
+     "        m.setImposedThermodynamicForce(\"StressRR\", initial_stress)\n",
+     "        m.setImposedThermodynamicForce(\"StressTT\", initial_stress)\n",
+     "    else:\n",
+     "        m.setImposedGradient(\"StrainRR\", 0)\n",
+     "        m.setImposedGradient(\"StrainTT\", 0)\n",
+     "    \n",
+     "    \n",
+     "    m.setImposedGradient(\"LiquidPressure\", {0: pressure_ic, t_end: pressure_final})\n",
+     "\n",
+     "    m.setExternalStateVariable(\"Temperature\", {0: temperature_ic, t_end: temperature_bc})\n",
+     "    m.setMaterialProperty(\"PoissonRatio\", nu)\n",
+     "    m.setMaterialProperty(\"CriticalStateLineSlope\", M)\n",
+     "    m.setMaterialProperty(\"SwellingLineSlope\", ka)\n",
+     "    m.setMaterialProperty(\"VirginConsolidationLineSlope\", la)\n",
+     "    m.setMaterialProperty(\"CharacteristicPreConsolidationPressure\", pc0)\n",
+     "    m.setMaterialProperty(\"InitialVolumeRatio\", v0)\n",
+     "\n",
+     "    m.setInternalStateVariableInitialValue(\"PreConsolidationPressure\", pc0)\n",
+     "    m.setInternalStateVariableInitialValue(\"VolumeRatio\", v0)\n",
+     "    \n",
+     "    m.setMaterialProperty(\"ThermalExpansion\", alpha)\n",
+     "    m.setMaterialProperty(\"BiotCoefficient\", alpha_B)\n",
+     "    m.setMaterialProperty(\"BishopsModelType\", bishops_model_type)\n",
+     "    m.setMaterialProperty(\"BishopsModelValue\", bishops_model_value)\n",
+     "    m.setMaterialProperty(\"ResidualLiquidSaturation\", S_L_res)\n",
+     "    m.setMaterialProperty(\"ResidualGasSaturation\", S_G_res)\n",
+     "    m.setMaterialProperty(\"BubblePressure\", p_b)\n",
+     "    m.setMaterialProperty(\"VanGenuchtenExponent_m\", m_S)\n",
+     "    \n",
+     "    m.setMaterialProperty(\"SwellingPressures\", pmax)\n",
+     "    m.setMaterialProperty(\"Exponents\", exp_p)\n",
+     "    m.setMaterialProperty(\"LowerSaturationLimit\", Smin_p)\n",
+     "    m.setMaterialProperty(\"UpperSaturationLimit\", Smax_p)\n",
+     "    m.setTimes(t_list)\n",
+     "    out = f\"saturation_mtest.res\"\n",
+     "    m.setOutputFileName(out)\n",
+     "    m.execute()\n",
+     "    print(f\"MTest done!\")\n",
+     "\n",
+     "saturation_mtest()"
+   ]
+ },
+ {
+   "cell_type": "code",
+   "execution_count": null,
+   "metadata": {},
+   "outputs": [],
+   "source": []
+ },
+ {
+   "cell_type": "code",
+   "execution_count": 43,
+   "metadata": {
+     "scrolled": false
+   },
+   "outputs": [
+     {
+       "data": {
+         "image/png": "iVBORw0KgGooAAANSUHEUGAABHYAAPKCAAYAAADHXX0H0AAA0nRfWHRtb2Z8d2FyZ0BhYXRwG9B6GL1IHZ1cnRpb24zLjEwLjEgSH08dHBz018vbW90cGxvdGxpY15vcnVzZC2/+5QAAALwSF1ZAAALwAACwAIAAGBAJGACGAABABJREFUeJzs3Xd4U9UbwPFvku43pYUyS1uGD0Cke7dMq2xky1S0XQ0ERECCGqDg+AGyQYKynawCgKComwA1I6KIWDRpXmtzfhSHY0B2aaGkp7+d5eL Tnnvz31ZpU57z3tU1q1oCCGEEII1Y00ggghBBCS1K6sANQ0ggghBBCCGEEII1Cs2QxLl0ggghBBCCGEEII1YTNuKsCCGEEII1Y00ggghBBA11ySx1I000ggghBBCCGEEIXJPEghBBCCGEEII1Y000ggghBBCCGEEyDVJLAgghBD1u0Np0yZK1CjBkFDcuwTFRXF0KFDKvU2LXVL62ly5YcDXLkQcUoHBBCCCGEEIUVZJYEEI1rZ1yq1p49ezJxzg14uJy7j0kIC3tzdxrLwhICCA8PBwN31vdq1a4e/v/9TjYIIY000ggghCh6LEghBBC10fGoEQDqY5Mgc0Hh0v4ULF3Lp8iVw1r10Ns7Mz2aRw9957jzp16By5EgMjK6UsRCCCGEEI1TUTR14kFY0Q0j3Vq1axSeffIKlpM0FRFYE3atV5VoZaWuab0RoweBgYecPny40EMV0ggghBBCGGLKsPMDX17ew0x0BCCCFEA1l0cctj+1y/fp3bt29T03btLnvq1K6dWNGjRx9SHB1TCCGEECKy/LhCCFEUS0dH0uRUV0ggghBBCGEXRr14Fo6zS1m21Sx00b16498jyphBBCCFEFpXhBBC1KJGegtCCGEEhncX9T2xsYmy7b7bGxsU8zJCGEEII1Y000gkX+WAhBBCCGK11WrVrFq15oAIjMjCzka1Y000ggghBBC1YhMBSGEEK1Tb0H0FI7kT0su1z2+0-d0x+FD1nTS/m90nTuL14FFy00ggghBBCCFEIS1W1Yw5dAChR1RqV5FHYo00ggghhEvVpCAHfu3Mym7f7bt2uB0Vr1qYkHBBCCCGEEI1U3w6FNLo8a0Jj4/P12MpiKJ8FDxarRZUFuLmElkhqUwtzChAC8H0MCFEAAPSp0rLypXj7+/yYLTfupj88TPUzAWhpZ5UnodLon0o40xYwG0Hh1ZLSBvCDkUI1Y000g1rC80zdB11YAgJo2LhP6d0eULj5Cno1KpPFwq1FzUFYX1f1r10TKZ1vbwg5HCKGLZVksZtvssMH37TpuUqXm"
+       }
+     }
+   ],
+   "source": [
+     "BAJGACGAABABJREFUeJzs3Xd4U9UbwPFvku43pYUyS1uGD0Cke7dMq2xky1S0XQ0ERECCGqDg+AGyQYKynawCgKComwA1I6KIWDRpXmtzfhSHY0B2aaGkp7+d5eL Tnnvz31ZpU57z3tU1q1oCCGEEII1Y00ggghBBCS1K6sANQ0ggghBBCCGEEII1Cs2QxLl0ggghBBCCGEEII1YTNuKsCCGEEII1Y00ggghBBA11ySx1I000ggghBBCCGEEIXJPEghBBCCGEEII1Y000ggghBBCCGEEyDVJLAgghBD1u0Np0yZK1CjBkFDcuwTFRXF0KFDKvU2LXVL62ly5YcDXLkQcUoHBBCCCGEEIUVZJYEEI1rZ1yq1p49ezJxzg14uJy7j0kIC3tzdxrLwhICCA8PBwN31vdq1a4e/v/9TjYIIY000ggghCh6LEghBBC10fGoEQDqY5Mgc0Hh0v4ULF3Lp8iVw1r10Ns7Mz2aRw9957jzp16By5EgMjK6UsRCCCGEEI1TUTR14kFY0Q0j3Vq1axSeffIKlpM0FRFYE3atV5VoZaWuab0RoweBgYecPny40EMV0ggghBBCGGLKsPMDX17ew0x0BCCCFEA1l0cctj+1y/fp3bt29T03btLnvq1K6dWNGjRx9SHB1TCCGEECKy/LhCCFEUS0dH0uRUV0ggghBBCGEXRr14Fo6zS1m21Sx00b16498jyphBBCCFEFpXhBBC1KJGegtCCGEEhncX9T2xsYmy7b7bGxsU8zJCGEEII1Y000gkX+WAhBBCCGK11WrVrFq15oAIjMjCzka1Y000ggghBBC1YhMBSGEEK1Tb0H0FI7kT0su1z2+0-d0x+FD1nTS/m90nTuL14FFy00ggghBBCCFEIS1W1Yw5dAChR1RqV5FHYo00ggghhEvVpCAHfu3Mym7f7bt2uB0Vr1qYkHBBCCCGEEI1U3w6FNLo8a0Jj4/P12MpiKJ8FDxarRZUFuLmElkhqUwtzChAC8H0MCFEAAPSp0rLypXj7+/yYLTfupj88TPUzAWhpZ5UnodLon0o40xYwG0Hh1ZLSBvCDkUI1Y000g1rC80zdB11YAgJo2LhP6d0eULj5Cno1KpPFwq1FzUFYX1f1r10TKZ1vbwg5HCKGLZVksZtvssMH37TpuUqXm"
+   ]
+ }

```


[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]


```

+ " fig, ((ax0,ax1,ax2),(ax3,ax4,ax5),(ax6,ax7,ax8)) = plt.subplots(3,3,figsize=(22,14))\n",
+ "\n",
+ " for idx, pvd in enumerate(pvd_files):\n",
+ "     label = \"OGS + MCC (M)\"\n",
+ "     if \"coupled\" in os.path.basename(pvd)[-4:]\n",
+ "         label = \"OGS + MCC (THM)\"\n",
+ "     t, eps, sig, pc, eps_eq, vr, p_LR, S_L, sig_sw, T = read_ogs(pvd)\n",
+ "     t = t/86400\n",
+ "     eps_mean = np.mean(eps, axis=1)\n",
+ "     sig_mean = np.mean(sig, axis=1)\n",
+ "     dev_eps = eps - eps_mean[:, None]\n",
+ "     dev_sig = sig - sig_mean[:, None]\n",
+ "     eps_dev = np.sqrt(1.5 * np.sum(dev_eps**2, axis=1))\n",
+ "     q = np.sqrt(1.5 * np.sum(dev_sig**2, axis=1))\n",
+ "     p = -sig_mean\n",
+ "     \n",
+ "     ##### plotting starts here\n",
+ "     # q vs epsilon_dev\n",
+ "     ax0.plot(eps_dev, q*1e-6, label=label)\n",
+ "     #ax0.set_prop_cycle(None)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         ax0.plot(m_eps_dev, m_q*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label="MTTest")\n",
+ "\n",
+ "     # p-q locus + quiver\n",
+ "     ax1.plot(p*1e-6, q*1e-6, label=f"(p,q), {label})")\n",
+ "     #ax1.set_prop_cycle(None)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         ax1.plot(m_p*1e-6, m_q*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=f"(p,q), MTTest")\n",
+ "         #ax1.quiver(p[:-1], q[:-1], np.diff(p), np.diff(q), angles='xy', scale_units='xy', scale=1, width=0.005)\n",
+ "\n",
+ "     # Time-series: p, q, p_c\n",
+ "     ax2.plot(t, p*1e-6, linestyle='--' if idx else '-', label=f"$p$, {label})")\n",
+ "     ax2.plot(t, q*1e-6, linestyle='--' if idx else '-', label=f"$q$, {label})")\n",
+ "     ax2.plot(t, pc*1e-6, linestyle='--' if idx else '-', label=f"$p_c$, {label})")\n",
+ "     ax2.set_xlabel("$t$ / d")\n",
+ "     ax2.set_ylabel("stress / MPa")\n",
+ "     ax2.set_prop_cycle(None)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         ax2.plot(time, m_p*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=f"$p$, MTTest")\n",
+ "         ax2.plot(time, m_q*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=f"$q$, MTTest")\n",
+ "         ax2.plot(time, m_pc*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=f"$p_c$, MTTest")\n",
+ "         ax2.legend()\n",
+ "\n",
+ "     # Stress components\n",
+ "     ax3.plot(t, sig[:, 0]*1e-6, linestyle='--' if idx else '-', label=r"$\sigma_{rr}$, {" + label})\n",
+ "     ax3.plot(t, sig[:, 2]*1e-6, linestyle='--' if idx else '-', label=r"$\sigma_{\phi\phi}$, {" + label})\n",
+ "     ax3.plot(t, sig[:, 1]*1e-6, linestyle='--' if idx else '-', label=r"$\sigma_{zz}$, {" + label})\n",
+ "     ax3.set_xlabel("$t$ / d")\n",
+ "     ax3.set_ylabel("stress / MPa")\n",
+ "     ax3.set_prop_cycle(None)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         ax3.plot(time, m_sig[:, 0]*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\sigma_{rr}$, MTTest")\n",
+ "         ax3.plot(time, m_sig[:, 2]*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\sigma_{\phi\phi}$, MTTest")\n",
+ "         ax3.plot(time, m_sig[:, 1]*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\sigma_{zz}$, MTTest")\n",
+ "         ax3.legend()\n",
+ "\n",
+ "     # Strain components\n",
+ "     ax4.plot(t, eps[:, 0], linestyle='--' if idx else '-', label=r"$\epsilon_{rr}$, {" + label})\n",
+ "     ax4.plot(t, eps[:, 2], linestyle='--' if idx else '-', label=r"$\epsilon_{\phi\phi}$, {" + label})\n",
+ "     ax4.plot(t, eps[:, 1], linestyle='--' if idx else '-', label=r"$\epsilon_{zz}$, {" + label})\n",
+ "     ax4.set_xlabel("$t$ / d")\n",
+ "     ax4.set_ylabel("strain")\n",
+ "     ax4.set_prop_cycle(None)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         ax4.plot(time, m_eps[:, 0], marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\epsilon_{rr}$, MTTest")\n",
+ "         ax4.plot(time, m_eps[:, 2], marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\epsilon_{\phi\phi}$, MTTest")\n",
+ "         ax4.plot(time, m_eps[:, 1], marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\epsilon_{zz}$, MTTest")\n",
+ "         ax4.legend(loc='lower left')\n",
+ "     \n",
+ "     # Volumetric change\n",
+ "     vol = np.sum(eps, axis=1)\n",
+ "     ax5.plot(t, 1 - 1/vr - (1 - 1/v0), linestyle='--' if idx else '-', label=r"$\phi - \phi_0$, {" + label})\n",
+ "     ax5.plot(t, vol, linestyle='--' if idx else '-', label=r"$\epsilon_{vol}$, {" + label})\n",
+ "     ax5.plot(t, eps_eq, linestyle='--' if idx else '-', label=r"$\epsilon_{eq}$, {" + label})\n",
+ "     ax5.set_xlabel("$t$ / d")\n",
+ "     ax5.set_prop_cycle(None)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         m_vol = np.sum(m_eps, axis=1)\n",
+ "         ax5.plot(time, 1 - 1/m_vr - (1 - 1/v0), marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\phi - \phi_0$, MTTest")\n",
+ "         ax5.plot(time, m_vol, marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\epsilon_{vol}$, MTTest")\n",
+ "         ax5.plot(time, m_eps_eq, marker='o', markersize= m_ms, linestyle=m_ls, label=r"$\epsilon_{eq}$, MTTest")\n",
+ "         ax5.legend()\n",
+ "     \n",
+ "     ax6.plot(p_LR*1e-6, S_L, label=label)\n",
+ "     #ax6.set_prop_cycle(None)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         ax6.plot(m_p_LR*1e-6, m_S_L, marker='o', markersize= m_ms, linestyle=m_ls, label="MTTest")\n",
+ "         ax6.set_xlabel("$p_{LR}$ / MPa")\n",
+ "         ax6.set_ylabel("$S_{\mathcal{L}}$ / -")\n",
+ "         ax6.legend()\n",
+ "     \n",
+ "     ax7.plot(t, sig_sw[:, 1]*1e-6, linestyle='--' if idx else '-', label= label)\n",
+ "     if plt_mtest == True and idx:\n",
+ "         ax7.plot(time, m_sig_sw[:, 1]*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label="MTTest")\n",
+ "         ax7.set_xlabel("$t$ / d")\n",
+ "         ax7.set_ylabel("$\sigma_{sw}$ / MPa")\n",
+ "         ax7.legend()\n",
+ "     \n",
+ "     ax8.plot(t, T, linestyle='--' if idx else '-', label= label)\n",
+ "     ax8.set_xlabel("$t$ / d")\n",
+ "     ax8.set_ylabel("$T$ / $^{\circ}$C")\n",
+ "     ax8.legend()\n",
+ "     \n",
+ "     #np.savetxt('OGS_{idx}.txt', np.c_[t/86400, S_L, p_LR, sig_sw[:, 1], sig, p, q, pc, eps_eq, vr], fmt='%12.8g', header='time S_L p_LR swellingZZ sigmaRR sigmaZZ sigm
+ "ATT p q pc eps_eq v_r', comments='') \n",
+ "\n",
+ "     # Critical state line and ellipses on p-q\n",
+ "     p_range = np.linspace(0, np.max(pc_0), n_points)\n",
+ "     ax1.plot(p_range*1e-6, M * p_range*1e-6, 'k', label='CSL')\n",
+ "     for i in np.linspace(1, len(pc_0) - 2, n_ellipse, dtype=int):\n",
+ "         pr = pc_0[i] * np.linspace(0, 1, n_points)\n",
+ "         qr = M * np.sqrt((pc_0[i] - pr))\n",
+ "         ax1.plot(pr*1e-6, qr*1e-6, '-', alpha=0.5)\n",
+ "\n",
+ "     ax0.set_xlabel(r"$\epsilon_{q5}$")\n",
+ "     ax0.set_ylabel("$q$ / MPa")\n",
+ "     ax0.legend()\n",

```

```

+ "    ax1.set_xlabel("$ps / MPa")\n",
+ "    ax1.set_ylabel("$qs / MPa")\n",
+ "    ax1.legend()\n",
+ "\n",
+ "    plt.tight_layout()\n",
+ "    save_fig = False\n",
+ "    if save_fig:\n",
+ "        fig.savefig(f'results_MCC.pdf')\n",
+ "    plt.show()\n",
+ "plt_mtest = False\n",
+ "plt_mtest = True\n",
+ "pvd_files = ['out/saturation.pvd']\n",
+ "pvd_files = ['out/saturation.pvd', 'out/saturation_coupled.pvd']\n",
+ "#pvd_files = ['out/saturation_coupled.pvd']\n",
+ "plot_mtest_results_vtu(pvd_files, plt_mtest)"
+ ]
+ },
+ {
+ "cell_type": "markdown",
+ "metadata": {},
+ "source": [
+ "### Selective plots (same as above, just fewer)"
+ ]
+ },
+ {
+ "cell_type": "code",
+ "execution_count": 45,
+ "metadata": {},
+ "outputs": [],
+ "source": [
+ "def plot_mtest_results_vtu_digben(pvd_files, plt_mtest = False, n_ellipse=5, n_points=100):\n",
+ "    data = np.loadtxt("$saturation_mtest.res")\n",
+ "    time = data[:,0]/86400\n",
+ "    m_eps = data[:,1:4]\n",
+ "    m_p_LR = data[:,5]\n",
+ "    m_sig = data[:,6:9]\n",
+ "    m_S_L = data[:,10]\n",
+ "    m_sig_sw = data[:,11:14]\n",
+ "    m_eps_eq = data[:,19]\n",
+ "    m_pc = data[:,20]\n",
+ "    m_vr = data[:,22]\n",
+ "    \n",
+ "    m_ls = 'None' # line style for mtest\n",
+ "    m_ms = 4 # marker size for mtest\n",
+ "    \n",
+ "    m_chi = compute_chi(m_S_L, bishops_model_type, bishops_model_value)\n",
+ "    for comp in range(3):\n",
+ "        m_sig[:, comp] += alpha_B * m_chi * m_p_LR\n",
+ "    \n",
+ "    m_eps_mean = np.mean(m_eps,axis=1)\n",
+ "    m_sig_mean = np.mean(m_sig,axis=1)\n",
+ "    m_dev_eps = m_eps - m_eps_mean[:,None]\n",
+ "    m_dev_sig = m_sig - m_sig_mean[:,None]\n",
+ "    m_eps_dev = np.sqrt(1.5*np.sum(m_dev_eps**2,axis=1))\n",
+ "    m_q = np.sqrt(1.5*np.sum(m_dev_sig**2,axis=1))\n",
+ "    m_p = -m_sig_mean\n",
+ "    \n",
+ "    # Load reference dataset\n",
+ "    t_0, eps_0, sig_0, pc_0, eps_eq_0, vr_0, p_LR_0, S_L_0, sig_sw_0, T0 = read ogs(pvd_files[0])\n",
+ "    \n",
+ "    eps_mean_0 = np.mean(eps_0, axis=1)\n",
+ "    sig_mean_0 = np.mean(sig_0, axis=1)\n",
+ "    dev_eps_0 = eps_0 - eps_mean_0[:,None]\n",
+ "    dev_sig_0 = sig_0 - sig_mean_0[:,None]\n",
+ "    eps_dev_0 = np.sqrt(1.5 * np.sum(dev_eps_0**2, axis=1))\n",
+ "    q_0 = np.sqrt(1.5 * np.sum(dev_sig_0**2, axis=1))\n",
+ "    p_0 = -sig_mean_0\n",
+ "    \n",
+ "    #np.savetxt('MTest.txt', np.c_[time, m_S_L, m_p_LR, m_sig_sw[:, 1], m_sig, m_p, m_q, m_pc, m_eps_eq, m_vr], fmt='%12.8g', header='time S_L p_LR swellingZZ sigmaRR sigmaZZ\n",
+ "sigmaTT p q pc eps_eq v_r', comments='')\n",
+ "    \n",
+ "    fig, ((ax0,ax1),(ax2,ax3)) = plt.subplots(2,2,figsize=(16,10))\n",
+ "    \n",
+ "    for idx, pvd in enumerate(pvd_files):\n",
+ "        label = "$OGS + MCC (M)" \n",
+ "        if "coupled" in os.path.basename(pvd)[-4:]:\n",
+ "            label = "$OGS + MCC (TH)" \n",
+ "        t, eps, sig, pc, eps_eq, vr, p_LR, S_L, sig_sw, T = read_ogs(pvd)\n",
+ "        t = t/86400\n",
+ "        eps_mean = np.mean(eps, axis=1)\n",
+ "        sig_mean = np.mean(sig, axis=1)\n",
+ "        dev_eps = eps - eps_mean[:,None]\n",
+ "        dev_sig = sig - sig_mean[:,None]\n",
+ "        eps_dev = np.sqrt(1.5 * np.sum(dev_eps**2, axis=1))\n",
+ "        q = np.sqrt(1.5 * np.sum(dev_sig**2, axis=1))\n",
+ "        p = -sig_mean\n",
+ "        \n",
+ "        ##### plotting starts here\n",
+ "        # q vs epsilon_dev\n",
+ "        ax0.plot(eps_dev, q*1e-6, label=label)\n",
+ "        #ax0.set_prop_cycle(None)\n",
+ "        if plt_mtest == True and idx:\n",
+ "            ax0.plot(m_eps_dev, m_q*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label="$MTest")\n",
+ "    \n",
+ "    # p-q locus + quiver\n",
+ "    ax1.plot(p*1e-6, q*1e-6, label=f"(p,q), {label}")\n",
+ "    #ax1.set_prop_cycle(None)\n",
+ "    if plt_mtest == True and idx:\n",
+ "        ax1.plot(m_p*1e-6, m_q*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label=f"(p,q), MTest")\n",
+ "    #ax1.quiver(p[:-1], q[:-1], np.diff(p), np.diff(q), angles='xy', scale_units='xy', scale=1, width=0.005)\n",
+ "    \n",
+ "    # swelling\n",
+ "    ax2.plot(t, sig_sw[:, 1]*1e-6, linestyle='--' if idx else '-', label= label)\n",
+ "    if plt_mtest == True and idx:\n",
+ "        ax2.plot(time, m_sig_sw[:, 1]*1e-6, marker='o', markersize= m_ms, linestyle=m_ls, label="$MTest")\n",
+ "    ax2.set_xlabel("$t$ / d")\n",
+ "    ax2.set_ylabel("$\\sigma_{sw}\\mathrm{zz}$ / MPa")\n",
+ "    ax2.legend()\n",
+ "    \n",
+ "    ax3.plot(p_LR*1e-6, S_L, label=label)\n",
+ "    #ax3.set_prop_cycle(None)\n",
+ "    if plt_mtest == True and idx:\n",
+ "        ax3.plot(m_p_LR*1e-6, m_S_L, marker='o', markersize= m_ms, linestyle=m_ls, label="$MTest")\n",
+ "    ax3.set_xlabel(r"$p_{LR}\\mathrm{LR}$ / MPa")\n",
+ "    \n",
+ "    ax3.set_ylabel("$S_L\\mathrm{L}$ / -")\n",
+ "    \n",

```



```

+ "      ax3.legend()\n",
+ "      \n",
+ "      \n",
+ "      #np.savetxt(f'0G5_{idx}.txt', np.c_[t_0/86400, S_L, p_LR,sig_sw[:, 1], sig, p, q, pc, eps_eq, vr], fmt='%12.8g', header='time S_L p_LR swellingZZ sigmaRR sigmaZZ sig
aTT p q pc eps_eq v_r', comments='')      \n",
+ "\n",
+ "      # Critical state line and ellipses on p-q\n",
+ "      p_range = np.linspace(0, np.max(pc_0), n_points)\n",
+ "      ax1.plot(p_range*1e-6, M * p_range*1e-6, ':k', label='CSL')\n",
+ "      for i in np.linspace(1, len(pc_0) - 2, n_ellipse, dtype=int):\n",
+ "          pr = pc_0[i] * np.linspace(0, 1, n_points)\n",
+ "          qr = M * np.sqrt(pr * (pc_0[i] - pr))\n",
+ "          ax1.plot(pr*1e-6, qr*1e-6, '-', alpha=0.5)\n",
+ "\n",
+ "      ax0.set_xlabel(r"$\epsilon$")\n",
+ "      ax0.set_ylabel("$q$ / MPa")\n",
+ "      ax0.legend()\n",
+ "      ax1.set_xlabel("$p$ / MPa")\n",
+ "      ax1.set_ylabel("$q$ / MPa")\n",
+ "      ax1.legend()\n",
+ "\n",
+ "      plt.tight_layout()\n",
+ "      save_fig = False\n",
+ "      if save_fig:\n",
+ "          fig.savefig(f'results_MCC_digben.pdf')\n",
+ "      plt.show()\n",
+ "plt_mtest = False\n",
+ "plt_mtest = True\n",
+ "pvd_files = ['out/saturation.pvd']\n",
+ "pvd_files = ['out/saturation.pvd', 'out/saturation_coupled.pvd']\n",
+ "#pvd_files = ['out/saturation_coupled.pvd']\n",
+ "#plot_mtest_results_vtu_digben(pvd_files, plt_mtest)"
+ ]
+ },
+ {
+ "cell_type": "code",
+ "execution_count": null,
+ "metadata": {},
+ "outputs": [],
+ "source": []
+ }
+ },
+ "metadata": {
+ "kernelspec": {
+ "display_name": "Python 3 (ipykernel)",
+ "language": "python",
+ "name": "python3"
+ },
+ "language_info": {
+ "codemirror_mode": {
+ "name": "python",
+ "version": 3
+ },
+ "file_extension": ".py",
+ "mimetype": "text/x-python",
+ "name": "python",
+ "nbconvert_exporter": "python",
+ "pygments_lexer": "ipython3",
+ "version": "3.10.12"
+ }
+ },
+ "nbformat": 4,
+ "nbformat_minor": 4
+ }
diff --git a/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/saturation.prj b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/saturation.prj
new file mode 100644
index 0000000000..4507824cc9
--- /dev/null
+++ b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/saturation.prj
@@ -0,0 +1,452 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+ <mesh axially_symmetric="true">square_1x1_quad_le0.vtu</mesh>
+ <geometry>square_1x1.gml</geometry>
+ <processes>
+ <process>
+ <name>TRM</name>
+ <type>THERMO_RICHARDS_MECHANICS</type>
+ <subtype>Stress_StrainTemperature</subtype>
+ <integration_order>2</integration_order>
+ <jacobian_assembler>
+ <type>Analytical</type>
+ </jacobian_assembler>
+ <constitutive_relation>
+ <type>MFront</type>
+ <behaviour>ModCamClay_semiExpl_constE</behaviour>
+ <material_properties>
+ <material_property name="YoungModulus" parameter="E"/>
+ <!--material_property name="InitialVolumeRatio" parameter="v0"/-->
+ <material_property name="PoissonRatio" parameter="nu"/>
+ <material_property name="CriticalStateLineSlope" parameter="M"/>
+ <material_property name="SwellingLineSlope" parameter="ka"/>
+ <material_property name="VirginConsolidationLineSlope" parameter="la"/>
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="pc0"/>
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="pc0"/>
+ <state_variable name="VolumeRatio" parameter="v0"/>
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+ <temperature>temperature</temperature>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable internal_name="EquivalentPlasticStrain" output_name="EquivalentPlasticStrain"/>
+ <secondary_variable internal_name="PreConsolidationPressure" output_name="PreConsolidationPressure"/>
+ <secondary_variable internal_name="PlasticVolumetricStrain" output_name="PlasticVolumetricStrain"/>
+ <secondary_variable internal_name="VolumeRatio" output_name="VolumeRatio"/>
+ <secondary_variable internal_name="ElasticStrain" output_name="ElasticStrain"/>
+ <secondary_variable internal_name="sigma" output_name="sigma"/>
+ <secondary_variable internal_name="swelling_stress" output_name="swelling_stress"/>
+ <secondary_variable internal_name="epsilon" output_name="epsilon"/>
+ <secondary_variable internal_name="velocity" output_name="velocity"/>
+ <secondary_variable internal_name="saturation" output_name="saturation"/>
+ </secondary_variables>

```

```

+         <specific_body_force>0 0</specific_body_force>
+         <initial_stress type="total">initial_stress</initial_stress>
+         <mass_lumping>false</mass_lumping>
+     </process>
+ </processes>
+ <media>
+     <medium>
+         <phases>
+             <phase>
+                 <type>AqueousLiquid</type>
+                 <properties>
+                     <property>
+                         <name>specific_heat_capacity</name>
+                         <type>Constant</type>
+                         <value>4280.0</value>
+                     </property>
+                     <property>
+                         <name>thermal_conductivity</name>
+                         <type>Constant</type>
+                         <value>0.6</value>
+                     </property>
+                     <property>
+                         <name>thermal_expansivity</name>
+                         <type>Constant</type>
+                         <value>1e-5</value>
+                     </property>
+                     <property>
+                         <name>viscosity</name>
+                         <type>Constant</type>
+                         <value>1e-3</value>
+                     </property>
+                     <property>
+                         <name>density</name>
+                         <type>Constant</type>
+                         <value>1e3</value>
+                     </property>
+                 </properties>
+             </phase>
+             <phase>
+                 <type>Solid</type>
+                 <properties>
+                     <property>
+                         <name>density</name>
+                         <type>Constant</type>
+                         <value>2780.0</value>
+                     </property>
+                     <property>
+                         <name>thermal_conductivity</name>
+                         <type>Constant</type>
+                         <value>1.838</value>
+                     </property>
+                     <property>
+                         <name>specific_heat_capacity</name>
+                         <type>Constant</type>
+                         <value>917.654</value>
+                     </property>
+                     <property>
+                         <name>thermal_expansivity</name>
+                         <type>Parameter</type>
+                         <parameter_name>alpha</parameter_name>
+                     </property>
+                     <property>
+                         <name>swelling_stress_rate</name>
+                         <type>SaturationDependentSwelling</type>
+                         <swelling_pressures>4000000.0 4000000.0 4000000.0</swelling_pressures>
+                         <exponents>1 1</exponents>
+                         <lower_saturation_limit>0</lower_saturation_limit>
+                         <upper_saturation_limit>1</upper_saturation_limit>
+                     </property>
+                 </properties>
+             </phase>
+         </phases>
+         <properties>
+             <property>
+                 <name>biot_coefficient</name>
+                 <type>Parameter</type>
+                 <parameter_name>alpha_B</parameter_name>
+             </property>
+             <property>
+                 <name>permeability</name>
+                 <type>Parameter</type>
+                 <parameter_name>intrinsic_permeability</parameter_name>
+             </property>
+             <property>
+                 <name>porosity</name>
+                 <type>Parameter</type>
+                 <parameter_name>phi</parameter_name>
+             </property>
+             <property>
+                 <name>relative_permeability</name>
+                 <type>RelativePermeabilityVanGenuchten</type>
+                 <residual_liquid_saturation>0.0</residual_liquid_saturation>
+                 <residual_gas_saturation>0</residual_gas_saturation>
+                 <exponent>0.4</exponent>
+                 <minimum_relative_permeability_liquid>1e-15</minimum_relative_permeability_liquid>
+             </property>
+             <property>
+                 <name>thermal_conductivity</name>
+                 <type>EffectiveThermalConductivityPorosityMixing</type>
+             </property>
+             <property>
+                 <name>saturation</name>
+                 <type>SaturationVanGenuchten</type>
+                 <p_b>16000000.0</p_b>
+                 <exponent>0.4014347581153</exponent>
+                 <residual_liquid_saturation>0.0</residual_liquid_saturation>
+                 <residual_gas_saturation>0.0</residual_gas_saturation>
+             </property>
+             <property>
+                 <name>bishops_effective_stress</name>
+                 <type>BishopsSaturationCutoff</type>
+                 <cutoff_value>1</cutoff_value>
+             </property>
+         </properties>
+     </medium>
+ </media>
+ <time_loop>

```

```

+
+ <processes>
+   <process ref="TRM">
+     <nonlinear_solver>basic_newton</nonlinear_solver>
+     <convergence_criterion>
+       <type>PerComponentDeltaX</type>
+       <norm_type>NORM2</norm_type>
+       <reltols>1e-6 1e-6 1e-6 1e-6</reltols>
+       <abstols>1e-6 1e-3 1e-8 1e-8</abstols>
+     </convergence_criterion>
+     <time_discretization>
+       <type>BackwardEuler</type>
+     </time_discretization>
+     <time_stepping>
+       <type>IterationNumberBasedTimeStepping</type>
+       <t_initial>0.0</t_initial>
+       <t_end>8640000.0</t_end>
+       <initial_dt>1</initial_dt>
+       <minimum_dt>1</minimum_dt>
+       <maximum_dt>172800.0</maximum_dt>
+       <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+       <multiplier> 1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+     </time_stepping>
+   </process>
+ </processes>
+
+ <output>
+   <type>VTK</type>
+   <prefix>saturation</prefix>
+   <suffix>_{:time}</suffix>
+   <variables>
+   </variables>
+   <output_iteration_results>false</output_iteration_results>
+   <fixed_output_times>0.0 172800.0 345600.0 518400.0 691200.0 864000.0 1036800.0 1209600.0 1382400.0 1555200.0 1728000.0 1900800.0 2073600.0 2246400.0 2419200.0 2592000.
0 2764800.0 2937600.0 3110400.0 3283200.0 3456000.0 3628800.0 3801600.0 3974400.0 4147200.0 4320000.0 4492800.0 4665600.0 4838400.0 5011200.0 5184000.0 5356800.0 5529600.0 5702400.
0 5875200.0 6048000.0 6220800.0 6393600.0 6566400.0 6739200.0 6912000.0 7084800.0 7257600.0 7430400.0 7603200.0 7776000.0 7948800.0 8121600.0 8294400.0 8467200.0 8640000.0</fixed_o
utput_times>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>p_b</name>
+     <type>Constant</type>
+     <value>16000000.0</value>
+   </parameter>
+   <parameter>
+     <name>m_S</name>
+     <type>Constant</type>
+     <value>0.4014347581153</value>
+   </parameter>
+   <parameter>
+     <name>S_L_res</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>S_G_res</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>alpha</name>
+     <type>Constant</type>
+     <value>1e-05</value>
+   </parameter>
+   <parameter>
+     <name>alpha_B</name>
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>intrinsic_permeability</name>
+     <type>Constant</type>
+     <value>1e-21</value>
+   </parameter>
+   <parameter>
+     <name>bishops_model_type</name>
+     <!-- 0: power law, 1: cutoff -->
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>bishops_model_value</name>
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>E</name>
+     <type>Constant</type>
+     <value>1515151515.151515</value>
+   </parameter>
+   <parameter>
+     <name>nuc</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>M</name>
+     <type>Constant</type>
+     <value>1.2</value>
+   </parameter>
+   <parameter>
+     <name>lac</name>
+     <type>Constant</type>
+     <value>0.077</value>
+   </parameter>
+   <parameter>
+     <name>ka</name>
+     <type>Constant</type>
+     <value>0.006</value>
+   </parameter>
+   <parameter>
+     <name>pc0</name>
+     <type>Constant</type>
+     <value>5000000.0</value>
+   </parameter>
+   <parameter>
+     <name>v0</name>
+     <type>Constant</type>

```

```

+      <value>1.666666666666667</value>
+    </parameter>
+  <parameter>
+    <name>phi</name>
+    <type>Constant</type>
+    <value>0.4</value>
+  </parameter>
+  <parameter>
+    <name>displacement0</name>
+    <type>Constant</type>
+    <values>0 0</values>
+  </parameter>
+  <parameter>
+    <name>dirichlet0</name>
+    <type>Constant</type>
+    <value>0</value>
+  </parameter>
+  <parameter>
+    <name>disp_value</name>
+    <type>Constant</type>
+    <value>-0.02</value>
+  </parameter>
+  <parameter>
+    <name>axial_displacement</name>
+    <type>CurveScaled</type>
+    <curve>ax_disp_curve</curve>
+    <parameter>disp_value</parameter>
+  </parameter>
+  <parameter>
+    <name>pressure_ic</name>
+    <type>Constant</type>
+    <value>-40000000.0</value>
+  </parameter>
+  <parameter>
+    <name>temperature_ic</name>
+    <type>Constant</type>
+    <value>293</value>
+  </parameter>
+  <parameter>
+    <name>temperature_bc</name>
+    <type>CurveScaled</type>
+    <curve>temperature_curve</curve>
+    <parameter>loading_value</parameter>
+  </parameter>
+  <parameter>
+    <name>initial_stress</name>
+    <type>Constant</type>
+    <values>-5000000.0 -5000000.0 -5000000.0 0.0</values>
+  </parameter>
+  <parameter>
+    <name>loading_values</name>
+    <type>Constant</type>
+    <value>1</value>
+  </parameter>
+  <parameter>
+    <name>confining_pressure</name>
+    <type>CurveScaled</type>
+    <curve>pre_loading_curve</curve>
+    <parameter>loading_value</parameter>
+  </parameter>
+  <parameter>
+    <name>pressure_ramp</name>
+    <type>CurveScaled</type>
+    <curve>pressure_ramp_curve</curve>
+    <parameter>pressure_ic</parameter>
+  </parameter>
+ </parameters>
+ <curves>
+   <curve>
+     <name>pressure_ramp_curve</name>
+     <coords>0.0 8640000.0</coords>
+     <values>1.0 -0.0</values>
+   </curve>
+   <curve>
+     <name>temperature_curve</name>
+     <coords>0.0 8640000.0</coords>
+     <values>293 393</values>
+   </curve>
+   <curve>
+     <name>ax_disp_curve</name>
+     <coords>0.0 8640000.0</coords>
+     <values>0 1</values>
+   </curve>
+   <curve>
+     <name>pre_loading_curve</name>
+     <coords>0.0 8640000.0</coords>
+     <values>-5000000.0 -5000000.0</values>
+   </curve>
+ </curves>
+ <process_variables>
+   <process_variable>
+     <name>displacement</name>
+     <components>2</components>
+     <order>1</order>
+     <initial_condition>displacement0</initial_condition>
+     <boundary_conditions>
+       <boundary_condition>
+         <geometrical_set>square_1x1_geometry</geometrical_set>
+         <geometry>left</geometry>
+         <type>Dirichlet</type>
+         <component>0</component>
+         <parameter>dirichlet0</parameter>
+       </boundary_condition>
+       <boundary_condition>
+         <geometrical_set>square_1x1_geometry</geometrical_set>
+         <geometry>top</geometry>
+         <component>1</component>
+         <type>Dirichlet</type>
+         <parameter>axial_displacement</parameter>
+         <!--type=Neumann</type>
+         <parameter>axial_stress</parameter-->
+       </boundary_condition>
+       <boundary_condition>
+         <geometrical_set>square_1x1_geometry</geometrical_set>
+         <geometry>bottom</geometry>
+         <type>Dirichlet</type>
+         <component>1</component>

```

```

+         <parameter>dirichlet0</parameter>
+     </boundary_condition>
+ </boundary_conditions>
+ <geometrical_set>square_1x1</geometrical_set>
+ <geometry>right</geometry>
+ <component>0</component>
+ <type>Neumann</type>
+ <parameter>confining_pressure</parameter>
+ </boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ <process_variable>
+ <name>pressure</name>
+ <components>1</components>
+ <order>1</order>
+ <initial_condition>pressure_ic</initial_condition>
+ <boundary_conditions>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0</mesh>
+ <type>Dirichlet</type>
+ <component>0</component>
+ <parameter>pressure_ramp</parameter>
+ </boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ <process_variable>
+ <name>temperature</name>
+ <components>1</components>
+ <order>1</order>
+ <initial_condition>temperature_ic</initial_condition>
+ <boundary_conditions>
+ <boundary_condition>
+ <mesh>square_1x1_quad_1e0</mesh>
+ <type>Dirichlet</type>
+ <component>0</component>
+ <parameter>temperature_bc</parameter>
+ </boundary_condition>
+ </boundary_conditions>
+ </process_variable>
+ </process_variables>
+ <nonlinear_solvers>
+ <nonlinear_solver>
+ <name>basic_newton</name>
+ <type>Newton</type>
+ <max_iter>30</max_iter>
+ <linear_solver>general_linear_solver</linear_solver>
+ </nonlinear_solver>
+ </nonlinear_solvers>
+ <linear_solvers>
+ <linear_solver>
+ <name>general_linear_solver</name>
+ <eigen>
+ <solver_type>PardisoLU</solver_type>
+ </eigen>
+ </linear_solver>
+ </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/saturation_coupled.prj b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/saturation_coupled.prj
new file mode 100644
index 0000000000..a451e56160
--- /dev/null
+++ b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/saturation_coupled.prj
@@ -0,0 +1,485 @@
<?xml version='1.0' encoding='ISO-8859-1'?>
<OpenGeoSysProject>
+ <mesh axially_symmetric="true">square_1x1_quad_1e0.vtu</mesh>
+ <geometry>square_1x1.gml</geometry>
+ <processes>
+ <process>
+ <name>TRM</name>
+ <type>THERMO_RICHARDS_MECHANICS</type>
+ <subtype>StressSaturation_StrainPressureTemperature</subtype>
+ <integration_order>2</integration_order>
+ <jacobian_assembler>
+ <type>Analytical</type>
+ </jacobian_assembler>
+ <constitutive_relation>
+ <type>MFront</type>
+ <behaviour>ModCamClay_semiExpl_coupled</behaviour>
+ <material_properties>
+ <!--material_property name="InitialVolumeRatio" parameter="v0"/-->
+ <material_property name="PoissonRatio" parameter="nu"/>
+ <material_property name="CriticalStateLineSlope" parameter="M"/>
+ <material_property name="SwellingLineSlope" parameter="ka"/>
+ <material_property name="VirginConsolidationLineSlope" parameter="la"/>
+ <material_property name="CharacteristicPreConsolidationPressure" parameter="pc0"/>
+ <material_property name="InitialVolumeRatio" parameter="v0"/>
+ <material_property name="ThermalExpansion" parameter="alpha"/>
+ <material_property name="BiotCoefficient" parameter="alpha_B"/>
+ <material_property name="BishopsModelType" parameter="bishops_model_type"/>
+ <material_property name="BishopsModelValue" parameter="bishops_model_value"/>
+ <material_property name="ResidualLiquidSaturation" parameter="S_L_res"/>
+ <material_property name="ResidualGasSaturation" parameter="S_G_res"/>
+ <material_property name="BubblePressure" parameter="p_b"/>
+ <material_property name="VanGenuchtenExponent_m" parameter="m_S"/>
+ <material_property name="SwellingPressures" parameter="pmax"/>
+ <material_property name="Exponents" parameter="exp_p"/>
+ <material_property name="LowerSaturationLimit" parameter="Smin_p"/>
+ <material_property name="UpperSaturationLimit" parameter="Smax_p"/>
+ </material_properties>
+ <initial_values>
+ <state_variable name="PreConsolidationPressure" parameter="pc0"/>
+ <state_variable name="VolumeRatio" parameter="v0"/>
+ </initial_values>
+ </constitutive_relation>
+ <process_variables>
+ <temperature>temperature</temperature>
+ <pressure>pressure</pressure>
+ <displacement>displacement</displacement>
+ </process_variables>
+ <secondary_variables>
+ <secondary_variable internal_name="EquivalentPlasticStrain" output_name="EquivalentPlasticStrain"/>
+ <secondary_variable internal_name="PreConsolidationPressure" output_name="PreConsolidationPressure"/>
+ <secondary_variable internal_name="PlasticVolumetricStrain" output_name="PlasticVolumetricStrain"/>
+ <secondary_variable internal_name="VolumeRatio" output_name="VolumeRatio"/>
+ <secondary_variable internal_name="ElasticStrain" output_name="ElasticStrain"/>
+ <secondary_variable internal_name="swelling_stress" output_name="swelling_stress"/>

```

```

+      <secondary_variable internal_name="epsilon" output_name="epsilon"/>
+      <secondary_variable internal_name="velocity" output_name="velocity"/>
+      <secondary_variable internal_name="saturation" output_name="saturation"/>
+      <secondary_variable internal_name="sigma_total" output_name="sigma_total"/>
+    </secondary_variables>
+    <specific_body_force>0 0</specific_body_force>
+    <initial_stress type="total">initial_stress</initial_stress>
+    <mass_lumping>false</mass_lumping>
+  </process>
+</processes>
+<media>
+  <medium>
+    <phases>
+      <phase>
+        <type>AqueousLiquid</type>
+        <properties>
+          <property>
+            <name>specific_heat_capacity</name>
+            <type>Constant</type>
+            <value>4280.0</value>
+          </property>
+          <property>
+            <name>thermal_conductivity</name>
+            <type>Constant</type>
+            <value>0.6</value>
+          </property>
+          <property>
+            <name>thermal_expansivity</name>
+            <type>Constant</type>
+            <value>1e-5</value>
+          </property>
+          <property>
+            <name>viscosity</name>
+            <type>Constant</type>
+            <value>1e-3</value>
+          </property>
+          <property>
+            <name>density</name>
+            <type>Constant</type>
+            <value>1e3</value>
+          </property>
+        </properties>
+      </phase>
+      <phase>
+        <type>Solid</type>
+        <properties>
+          <property>
+            <name>density</name>
+            <type>Constant</type>
+            <value>2780.0</value>
+          </property>
+          <property>
+            <name>thermal_conductivity</name>
+            <type>Constant</type>
+            <value>1.838</value>
+          </property>
+          <property>
+            <name>specific_heat_capacity</name>
+            <type>Constant</type>
+            <value>917.654</value>
+          </property>
+          <property>
+            <name>thermal_expansivity</name>
+            <type>Parameter</type>
+            <parameter_name>alpha</parameter_name>
+          </property>
+          <!--<property>
+            <name>swelling_stress_rate</name>
+            <type>SaturationDependentSwelling</type>
+            <swelling_pressures>4000000.0 4000000.0 4000000.0</swelling_pressures>
+            <exponents>1 1 1</exponents>
+            <lower_saturation_limit>0</lower_saturation_limit>
+            <upper_saturation_limit>1</upper_saturation_limit>
+          </property>
+          -->
+        </properties>
+      </phase>
+    </phases>
+    <properties>
+      <property>
+        <name>biot_coefficient</name>
+        <type>Parameter</type>
+        <parameter_name>alpha_B</parameter_name>
+      </property>
+      <property>
+        <name>permeability</name>
+        <type>Parameter</type>
+        <parameter_name>intrinsic_permeability</parameter_name>
+      </property>
+      <property>
+        <name>porosity</name>
+        <type>Parameter</type>
+        <parameter_name>phi</parameter_name>
+      </property>
+      <property>
+        <name>relative_permeability</name>
+        <type>RelativePermeabilityVanGenuchten</type>
+        <residual_liquid_saturation>0.0</residual_liquid_saturation>
+        <residual_gas_saturation>0</residual_gas_saturation>
+        <exponent>0.4</exponent>
+        <minimum_relative_permeability_liquid>1e-15</minimum_relative_permeability_liquid>
+      </property>
+      <property>
+        <name>thermal_conductivity</name>
+        <type>EffectiveThermalConductivityPorosityMixing</type>
+      </property>
+      <property>
+        <name>saturation</name>
+        <type>SaturationVanGenuchten</type>
+        <p_b>16000000.0</p_b>
+        <exponent>0.4014347581153</exponent>
+        <residual_liquid_saturation>0.0</residual_liquid_saturation>
+        <residual_gas_saturation>0.0</residual_gas_saturation>
+      </property>
+      <property>
+        <name>bishops_effective_stress</name>
+        <type>BishopsSaturationCutoff</type>

```

```

+         <cutoff_value>1</cutoff_value>
+     </property>
+ </properties>
+ </medium>
+ </media>
+ <time_loop>
+   <processes>
+     <process ref="TRM">
+       <nonlinear_solver>basic_newton</nonlinear_solver>
+       <convergence_criterion>
+         <type>PerComponentDeltaX</type>
+         <norm_type>NORM2</norm_type>
+         <reltols>1e-6 1e-6 1e-6 1e-6</reltols>
+         <abstols>1e-6 1e-3 1e-8 1e-8</abstols>
+       </convergence_criterion>
+       <time_discretization>
+         <type>BackwardEuler</type>
+       </time_discretization>
+       <time_stepping>
+         <type>IterationNumberBasedTimeStepping</type>
+         <t_initial>0.0</t_initial>
+         <t_end>8640000.0</t_end>
+         <initial_dt>1</initial_dt>
+         <minimum_dt>1</minimum_dt>
+         <maximum_dt>172800.0</maximum_dt>
+         <number_iterations>1 2 3 4 5 6 7 8 9 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25</number_iterations>
+         <multiplier> 1.225 1.2 1.175 1.15 1.125 1.1 1.075 1.05 1.025 0.975 0.95 0.925 0.9 0.875 0.85 0.825 0.8 0.775 0.75 0.725 0.7 0.675 0.65 0.625</multiplier>
+       </time_stepping>
+     </process>
+   </processes>
+   <output>
+     <type>VTK</type>
+     <prefix>saturation_coupled</prefix>
+     <suffix>_{:time}</suffix>
+     <variables>
+     </variables>
+     <output_iteration_results>false</output_iteration_results>
+     <fixed_output_times>0.0 172800.0 345600.0 518400.0 691200.0 864000.0 1036800.0 1209600.0 1382400.0 1555200.0 1728000.0 1900800.0 2073600.0 2246400.0 2419200.0 2592000.
0 2764800.0 2937600.0 3110400.0 3283200.0 3456000.0 3628800.0 3801600.0 3974400.0 4147200.0 4320000.0 4492800.0 4665600.0 4838400.0 5011200.0 5184000.0 5356800.0 5529600.0 5702400.
0 5875200.0 6048000.0 6220800.0 6393600.0 6566400.0 6739200.0 6912000.0 7084800.0 7257600.0 7430400.0 7603200.0 7776000.0 7948800.0 8121600.0 8294400.0 8467200.0 8640000.0</fixed_o
utput_times>
+   </output>
+ </time_loop>
+ <parameters>
+   <parameter>
+     <name>p_b</name>
+     <type>Constant</type>
+     <value>16000000.0</value>
+   </parameter>
+   <parameter>
+     <name>m_S</name>
+     <type>Constant</type>
+     <value>0.4014347581153</value>
+   </parameter>
+   <parameter>
+     <name>S_l_res</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>S_G_res</name>
+     <type>Constant</type>
+     <value>0.0</value>
+   </parameter>
+   <parameter>
+     <name>alpha</name>
+     <type>Constant</type>
+     <value>1e-05</value>
+   </parameter>
+   <parameter>
+     <name>alpha_B</name>
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>intrinsic_permeability</name>
+     <type>Constant</type>
+     <value>1e-21</value>
+   </parameter>
+   <parameter>
+     <name>bishops_model_type</name>
+     <!-- 0: power law, 1: cutoff -->
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>bishops_model_value</name>
+     <type>Constant</type>
+     <value>1</value>
+   </parameter>
+   <parameter>
+     <name>E</name>
+     <type>Constant</type>
+     <value>1515151515.151515</value>
+   </parameter>
+   <parameter>
+     <name>nuc</name>
+     <type>Constant</type>
+     <value>0.3</value>
+   </parameter>
+   <parameter>
+     <name>M</name>
+     <type>Constant</type>
+     <value>1.2</value>
+   </parameter>
+   <parameter>
+     <name>la</name>
+     <type>Constant</type>
+     <value>0.077</value>
+   </parameter>
+   <parameter>
+     <name>ka</name>
+     <type>Constant</type>
+     <value>0.006</value>
+   </parameter>
+   <parameter>
+     <name>pc0</name>

```

```

+         <type>Constant</type>
+         <value>5000000.0</value>
+     </parameter>
+     <parameter>
+         <name>v0</name>
+         <type>Constant</type>
+         <value>1.666666666666667</value>
+     </parameter>
+     <parameter>
+         <name>phi</name>
+         <type>Constant</type>
+         <value>0.4</value>
+     </parameter>
+     <parameter>
+         <name>displacement0</name>
+         <type>Constant</type>
+         <values>0 0</values>
+     </parameter>
+     <parameter>
+         <name>dirichlet0</name>
+         <type>Constant</type>
+         <value>0</value>
+     </parameter>
+     <parameter>
+         <name>disp_value</name>
+         <type>Constant</type>
+         <value>-0.02</value>
+     </parameter>
+     <parameter>
+         <name>axial_displacement</name>
+         <type>CurveScaled</type>
+         <curve>ax_disp_curve</curve>
+         <parameter>disp_value</parameter>
+     </parameter>
+     <parameter>
+         <name>pressure_ic</name>
+         <type>Constant</type>
+         <value>-40000000.0</value>
+     </parameter>
+     <parameter>
+         <name>temperature_ic</name>
+         <type>Constant</type>
+         <value>293</value>
+     </parameter>
+     <parameter>
+         <name>temperature_bc</name>
+         <type>CurveScaled</type>
+         <curve>temperature_curve</curve>
+         <parameter>loading_value</parameter>
+     </parameter>
+     <parameter>
+         <name>initial_stress</name>
+         <type>Constant</type>
+         <values>-5000000.0 -5000000.0 -5000000.0 0.0</values>
+     </parameter>
+     <parameter>
+         <name>loading_values</name>
+         <type>Constant</type>
+         <value>1</value>
+     </parameter>
+     <parameter>
+         <name>confining_pressure</name>
+         <type>CurveScaled</type>
+         <curve>pre_loading_curve</curve>
+         <parameter>loading_value</parameter>
+     </parameter>
+     <parameter>
+         <name>pressure_ramp</name>
+         <type>CurveScaled</type>
+         <curve>pressure_ramp_curve</curve>
+         <parameter>pressure_ic</parameter>
+     </parameter>
+     <parameter>
+         <name>pmax</name>
+         <type>Constant</type>
+         <value>4000000.0</value>
+     </parameter>
+     <parameter>
+         <name>exp_p</name>
+         <type>Constant</type>
+         <value>1</value>
+     </parameter>
+     <parameter>
+         <name>Smin_p</name>
+         <type>Constant</type>
+         <value>0</value>
+     </parameter>
+     <parameter>
+         <name>Smax_p</name>
+         <type>Constant</type>
+         <value>1</value>
+     </parameter>
+ </parameters>
+ <curves>
+     <curve>
+         <name>pressure_ramp_curve</name>
+         <coords>0.0 8640000.0</coords>
+         <values>1.0 -0.0</values>
+     </curve>
+     <curve>
+         <name>temperature_curve</name>
+         <coords>0.0 8640000.0</coords>
+         <values>293 393</values>
+     </curve>
+     <curve>
+         <name>ax_disp_curve</name>
+         <coords>0.0 8640000.0</coords>
+         <values>0 1</values>
+     </curve>
+     <curve>
+         <name>pre_loading_curve</name>
+         <coords>0.0 8640000.0</coords>
+         <values>-5000000.0 -5000000.0</values>
+     </curve>
+ </curves>
+ <process_variables>
+     <process_variable>

```



```

+       <name>displacement</name>
+       <components>2</components>
+       <order>1</order>
+       <initial_condition>displacement0</initial_condition>
+       <boundary_conditions>
+         <boundary_condition>
+           <geometrical_set>square_1x1_geometry</geometrical_set>
+           <geometry>left</geometry>
+           <type>Dirichlet</type>
+           <component>0</component>
+           <parameter>dirichlet0</parameter>
+         </boundary_condition>
+         <boundary_condition>
+           <geometrical_set>square_1x1_geometry</geometrical_set>
+           <geometry>top</geometry>
+           <component>1</component>
+           <type>Dirichlet</type>
+           <parameter>axial_displacement</parameter>
+           <!--type=Neumann</type>
+           <parameter>axial_stress</parameter-->
+         </boundary_condition>
+         <boundary_condition>
+           <geometrical_set>square_1x1_geometry</geometrical_set>
+           <geometry>bottom</geometry>
+           <type>Dirichlet</type>
+           <component>1</component>
+           <parameter>dirichlet0</parameter>
+         </boundary_condition>
+         <boundary_condition>
+           <geometrical_set>square_1x1_geometry</geometrical_set>
+           <geometry>right</geometry>
+           <component>0</component>
+           <type>Neumann</type>
+           <parameter>confining_pressure</parameter>
+         </boundary_condition>
+       </boundary_conditions>
+     </process_variable>
+     <process_variable>
+       <name>pressure</name>
+       <components>1</components>
+       <order>1</order>
+       <initial_condition>pressure_ic</initial_condition>
+       <boundary_conditions>
+         <boundary_condition>
+           <mesh>square_1x1_quad_1e0</mesh>
+           <type>Dirichlet</type>
+           <component>0</component>
+           <parameter>pressure_ramp</parameter>
+         </boundary_condition>
+       </boundary_conditions>
+     </process_variable>
+     <process_variable>
+       <name>temperature</name>
+       <components>1</components>
+       <order>1</order>
+       <initial_condition>temperature_ic</initial_condition>
+       <boundary_conditions>
+         <boundary_condition>
+           <mesh>square_1x1_quad_1e0</mesh>
+           <type>Dirichlet</type>
+           <component>0</component>
+           <parameter>temperature_bc</parameter>
+         </boundary_condition>
+       </boundary_conditions>
+     </process_variable>
+   </process_variables>
+   <nonlinear_solvers>
+     <nonlinear_solver>
+       <name>basic_newton</name>
+       <type>Newton</type>
+       <max_iter>30</max_iter>
+       <linear_solver>general_linear_solver</linear_solver>
+     </nonlinear_solver>
+   </nonlinear_solvers>
+   <linear_solvers>
+     <linear_solver>
+       <name>general_linear_solver</name>
+       <eigen>
+         <solver_type>PardisoLU</solver_type>
+       </eigen>
+     </linear_solver>
+   </linear_solvers>
+ </OpenGeoSysProject>
diff --git a/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/square_1x1.gml b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/square_1x1.gml
new file mode 100644
index 00000000000..2f637aa9f9
--- /dev/null
+++ b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/square_1x1.gml
@@ -0,0 +1,31 @@
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-style sheet type="text/xml" href="OpenGeoSysGLI.xsl"?>
+
+<OpenGeoSysGLI xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:ogs="http://www.opengeosys.org">
+  <name>square_1x1_geometry</name>
+  <points>
+    <point id="0" x="0" y="0" z="0"/>
+    <point id="1" x="1" y="1" z="0"/>
+    <point id="2" x="1" y="0" z="0"/>
+    <point id="3" x="0" y="1" z="0"/>
+  </points>
+  <polylines>
+    <polyline id="0" name="left">
+      <pnt>0</pnt>
+      <pnt>1</pnt>
+    </polyline>
+    <polyline id="1" name="right">
+      <pnt>2</pnt>
+      <pnt>3</pnt>
+    </polyline>
+    <polyline id="2" name="bottom">
+      <pnt>0</pnt>
+      <pnt>2</pnt>
+    </polyline>
+    <polyline id="3" name="top">
+      <pnt>1</pnt>
+      <pnt>3</pnt>

```



```

+
+#ifndef MFRONT_SHARED0B3
+#define MFRONT_SHARED0B3 TFEL_VISIBILITY_EXPORT
+#endif /* MFRONT_SHARED0B3 */
+
+#ifndef MFRONT_EXPORT_SYMBOL
+#define MFRONT_EXPORT_SYMBOL(TYPE, NAME, VALUE) \
+ MFRONT_SHARED0B3 extern TYPE NAME; \
+ MFRONT_SHARED0B3 TYPE NAME = VALUE
+#endif /* MFRONT_EXPORT_SYMBOL */
+
+#ifndef MFRONT_EXPORT_ARRAY_ARGUMENTS
+#define MFRONT_EXPORT_ARRAY_ARGUMENTS(...) __VA_ARGS__
+#endif /* MFRONT_EXPORT_ARRAY_ARGUMENTS */
+
+#ifndef MFRONT_EXPORT_ARRAY_OF_SYMBOLS
+#define MFRONT_EXPORT_ARRAY_OF_SYMBOLS(TYPE, NAME, SIZE, VALUE) \
+ MFRONT_SHARED0B3 extern TYPE NAME[SIZE]; \
+ MFRONT_SHARED0B3 TYPE NAME[SIZE] = {VALUE}
+#endif /* MFRONT_EXPORT_ARRAY_OF_SYMBOLS */
+
+#include<iostream>
+#include<cstdlib>
+#include"TFEL/Material/OutOfBoundsPolicy.hxx"
+#include"TFEL/Math/t2tot2.hxx"
+#include"TFEL/Math/t2tot2.hxx"
+#include"TFEL/Material/ModCamClay_semiExpl_coupled.hxx"
+#include"MFront/GenericBehaviour/GenericBehaviourTraits.hxx"
+#include"MFront/GenericBehaviour/Integrate.hxx"
+#include"MFront/GenericBehaviour/ModCamClay_semiExpl_coupled-generic.hxx"
+
+static tfel::material::OutOfBoundsPolicy&
+ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy(){
+static auto policy = tfel::material::None;
+return policy;
+}
+
+namespace mfront::gb{
+
+template<>
+struct GenericBehaviourTraits<tfel::material::ModCamClay_semiExpl_coupled<tfel::material::ModellingHypothesis::AXISYMETRICALGENERALISEDPLANESTRAIN, real, false>>{
+static constexpr auto hypothesis = tfel::material::ModellingHypothesis::AXISYMETRICALGENERALISEDPLANESTRAIN;
+static constexpr auto N = tfel::material::ModellingHypothesis::ToSpaceDimension-hypothesis::value;
+static constexpr auto StensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+static constexpr auto TensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+};
+
+template<>
+struct GenericBehaviourTraits<tfel::material::ModCamClay_semiExpl_coupled<tfel::material::ModellingHypothesis::AXISYMETRICALGENERALISEDPLANESTRESS, real, false>>{
+static constexpr auto hypothesis = tfel::material::ModellingHypothesis::AXISYMETRICALGENERALISEDPLANESTRESS;
+static constexpr auto N = tfel::material::ModellingHypothesis::ToSpaceDimension-hypothesis::value;
+static constexpr auto StensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+static constexpr auto TensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+};
+
+template<>
+struct GenericBehaviourTraits<tfel::material::ModCamClay_semiExpl_coupled<tfel::material::ModellingHypothesis::AXISYMETRICAL, real, false>>{
+static constexpr auto hypothesis = tfel::material::ModellingHypothesis::AXISYMETRICAL;
+static constexpr auto N = tfel::material::ModellingHypothesis::ToSpaceDimension-hypothesis::value;
+static constexpr auto StensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+static constexpr auto TensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+};
+
+template<>
+struct GenericBehaviourTraits<tfel::material::ModCamClay_semiExpl_coupled<tfel::material::ModellingHypothesis::PLANESTRESS, real, false>>{
+static constexpr auto hypothesis = tfel::material::ModellingHypothesis::PLANESTRESS;
+static constexpr auto N = tfel::material::ModellingHypothesis::ToSpaceDimension-hypothesis::value;
+static constexpr auto StensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+static constexpr auto TensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+};
+
+template<>
+struct GenericBehaviourTraits<tfel::material::ModCamClay_semiExpl_coupled<tfel::material::ModellingHypothesis::PLANESTRAIN, real, false>>{
+static constexpr auto hypothesis = tfel::material::ModellingHypothesis::PLANESTRAIN;
+static constexpr auto N = tfel::material::ModellingHypothesis::ToSpaceDimension-hypothesis::value;
+static constexpr auto StensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+static constexpr auto TensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+};
+
+template<>
+struct GenericBehaviourTraits<tfel::material::ModCamClay_semiExpl_coupled<tfel::material::ModellingHypothesis::GENERALISEDPLANESTRAIN, real, false>>{
+static constexpr auto hypothesis = tfel::material::ModellingHypothesis::GENERALISEDPLANESTRAIN;
+static constexpr auto N = tfel::material::ModellingHypothesis::ToSpaceDimension-hypothesis::value;
+static constexpr auto StensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+static constexpr auto TensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+};
+
+template<>
+struct GenericBehaviourTraits<tfel::material::ModCamClay_semiExpl_coupled<tfel::material::ModellingHypothesis::TRIDIMENSIONAL, real, false>>{
+static constexpr auto hypothesis = tfel::material::ModellingHypothesis::TRIDIMENSIONAL;
+static constexpr auto N = tfel::material::ModellingHypothesis::ToSpaceDimension-hypothesis::value;
+static constexpr auto StensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+static constexpr auto TensorSize = tfel::material::ModellingHypothesis::ToTensorSize-hypothesis::value;
+};
+
+} // end of namespace mfront::gb
+
+#ifdef __cplusplus
+extern "C"{
+#endif /* __cplusplus */
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_author, "Aqeel Afzal Chaudhry , Christian Silberman , Eric Simo , Miguel Mnica , Thomas Helfer , Thomas Nagel");
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_date, "01 / 09 / 2025");
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_description, "The modified cam - clay model according to Callari ( 1998 ) :\nA finite-strain cam-clay model in the fr
amework of multiplicative elasto-plasticity\nbut here in a consistent geometrically linear form ( linearized volume ratio evolution )\nsemi - explicit due to explicit volume ratio up
date at the end of time step ,\nnonlinear hypoelastic behavior : pressure - dependent bulk modulus , constant Poisson ratio ,\nincremental formulation assuming constant elastic p
arameters over the time step ,\nnormalized plastic flow direction , lower limit for a minimal pre - consolidation pressure ,\nwith hydraulic and thermal coupling , saturation depen
dent swelling ,");
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_validator, "");
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_build_id, "");
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_mfront_ept, "ModCamClay_semiExpl_coupled");
+
+

```

```

+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_tfel_version, "4.2.4-dev");
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_unit_system, "");
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_mfront_mkt, 1u);
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_mfront_interface, "Generic");
+
+MFRONT_EXPORT_SYMBOL(const char*, ModCamClay_semiExpl_coupled_src, "ModCamClay_semiExpl_coupled.mfront");
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nModellingHypotheses, 7u);
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(const char *, ModCamClay_semiExpl_coupled_ModellingHypotheses, 7, MFRONT_EXPORT_ARRAY_ARGUMENTS("AxisymmetricalGeneralisedPlaneStrain",
+"AxisymmetricalGeneralisedPlaneStress", "Axisymmetrical", "PlaneStress", "PlaneStrain", "GeneralisedPlaneStrain",
+"Tridimensional"));
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nMainVariables, 2u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nGradients, 2u);
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(int, ModCamClay_semiExpl_coupled_GradientsTypes, 2, MFRONT_EXPORT_ARRAY_ARGUMENTS(1,
+0));
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(const char *, ModCamClay_semiExpl_coupled_Gradients, 2, MFRONT_EXPORT_ARRAY_ARGUMENTS("Strain",
+"LiquidPressure"));
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nThermodynamicForces, 2u);
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(int, ModCamClay_semiExpl_coupled_ThermodynamicForcesTypes, 2, MFRONT_EXPORT_ARRAY_ARGUMENTS(1,
+0));
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(const char *, ModCamClay_semiExpl_coupled_ThermodynamicForces, 2, MFRONT_EXPORT_ARRAY_ARGUMENTS("Stress",
+"Saturation"));
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nTangentOperatorBlocks, 8u);
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(const char *, ModCamClay_semiExpl_coupled_TangentOperatorBlocks, 8, MFRONT_EXPORT_ARRAY_ARGUMENTS("Stress",
+"Strain", "Stress", "Temperature", "Stress", "LiquidPressure",
+"Saturation", "LiquidPressure"));
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_BehaviourType, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_BehaviourKinematic, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_SymmetryType, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_ElasticSymmetryType, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_api_version, 1u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_TemperatureRemovedFromExternalStateVariables, 1u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_UsableInPurelyImplicitResolution, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nMaterialProperties, 18u);
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(const char *, ModCamClay_semiExpl_coupled_MaterialProperties, 18, MFRONT_EXPORT_ARRAY_ARGUMENTS("PoissonRatio",
+"CriticalStateLineSlope", "SwellingLineSlope", "VirginConsolidationLineSlope", "CharacteristicPreConsolidationPressure", "InitialVolumeRatio",
+"ThermalExpansion", "BiotCoefficient", "ResidualLiquidSaturation", "ResidualGasSaturation", "BubblePressure",
+"VanGenuchtenExponent_m", "BishopsModelType", "BishopsModelValue", "SwellingPressures", "Exponents",
+"LowerSaturationLimit", "UpperSaturationLimit"));
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nInternalStateVariables, 6u);
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(const char *, ModCamClay_semiExpl_coupled_InternalStateVariables, 6, MFRONT_EXPORT_ARRAY_ARGUMENTS("swelling_stress",
+"ElasticStrain", "EquivalentPlasticStrain", "PreConsolidationPressure", "PlasticVolumetricStrain", "VolumeRatio"));
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(int, ModCamClay_semiExpl_coupled_InternalStateVariablesTypes, 6, MFRONT_EXPORT_ARRAY_ARGUMENTS(1,1,0,0,0,0));
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nExternalStateVariables, 0u);
+MFRONT_EXPORT_SYMBOL(const char * const *, ModCamClay_semiExpl_coupled_ExternalStateVariables, nullptr);
+
+MFRONT_EXPORT_SYMBOL(const int *, ModCamClay_semiExpl_coupled_ExternalStateVariablesTypes, nullptr);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nParameters, 6u);
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(const char *, ModCamClay_semiExpl_coupled_Parameters, 6, MFRONT_EXPORT_ARRAY_ARGUMENTS("theta",
+"epsilon", "iterMax", "minimal_time_step_scaling_factor", "maximal_time_step_scaling_factor", "numerical_jacobian_epsilon"));
+
+MFRONT_EXPORT_ARRAY_OF_SYMBOLS(int, ModCamClay_semiExpl_coupled_ParametersTypes, 6, MFRONT_EXPORT_ARRAY_ARGUMENTS(0,0,2,0,0,0));
+
+MFRONT_EXPORT_SYMBOL(double, ModCamClay_semiExpl_coupled_theta_ParameterDefaultValue, 1);
+
+MFRONT_EXPORT_SYMBOL(double, ModCamClay_semiExpl_coupled_epsilon_ParameterDefaultValue, 1e-14);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_iterMax_ParameterDefaultValue, 100);
+
+MFRONT_EXPORT_SYMBOL(double, ModCamClay_semiExpl_coupled_minimal_time_step_scaling_factor_ParameterDefaultValue, 0.1);
+
+MFRONT_EXPORT_SYMBOL(double, ModCamClay_semiExpl_coupled_maximal_time_step_scaling_factor_ParameterDefaultValue, 1.7976931348623e+308);
+
+MFRONT_EXPORT_SYMBOL(double, ModCamClay_semiExpl_coupled_numerical_jacobian_epsilon_ParameterDefaultValue, 1e-15);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_requiresStiffnessTensor, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_requiresThermalExpansionCoefficientTensor, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nInitializeFunctions, 0u);
+
+MFRONT_EXPORT_SYMBOL(const char * const *, ModCamClay_semiExpl_coupled_InitializeFunctions, nullptr);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_nPostProcessings, 0u);
+
+MFRONT_EXPORT_SYMBOL(const char * const *, ModCamClay_semiExpl_coupled_PostProcessings, nullptr);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_ComputesInternalEnergy, 0u);
+
+MFRONT_EXPORT_SYMBOL(unsigned short, ModCamClay_semiExpl_coupled_ComputesDissipatedEnergy, 0u);
+
+MFRONT_SHARED0BJ void
+ModCamClay_semiExpl_coupled_setOutOfBoundsPolicy(const int p){
+    if(p==0){
+        ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy() = tfel::material::None;
+    } else if(p==1){
+        ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy() = tfel::material::Warning;
+    } else if(p==2){

```

```

+ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy() = tfel::material::Strict;
+} else {
+std::cerr << "ModCamClay_semiExpl_coupled_setOutOfBoundsPolicy: invalid argument\n";
+}
+}
+
+MFRONT_SHARED0BJ int
+ModCamClay_semiExpl_coupled_setParameter(const char *const key,const double value){
+using tfel::material::ModCamClay_semiExpl_coupledParametersInitializer;
+auto& i = ModCamClay_semiExpl_coupledParametersInitializer::get();
+try{
+i.set(key,value);
+} catch(std::runtime_error& e){
+std::cerr << e.what() << std::endl;
+return 0;
+}
+return 1;
+}
+
+MFRONT_SHARED0BJ int
+ModCamClay_semiExpl_coupled_setUnsignedShortParameter(const char *const key,const unsigned short value){
+using tfel::material::ModCamClay_semiExpl_coupledParametersInitializer;
+auto& i = ModCamClay_semiExpl_coupledParametersInitializer::get();
+try{
+i.set(key,value);
+} catch(std::runtime_error& e){
+std::cerr << e.what() << std::endl;
+return 0;
+}
+return 1;
+}
+
+MFRONT_SHARED0BJ int ModCamClay_semiExpl_coupled_AxisymmetricalGeneralisedPlaneStrain(mfront_gb_BehaviourData* const d){
+using namespace tfel::material;
+using real = mfront::gb::real;
+constexpr auto h = ModellingHypothesis::AXISYMMETRICALGENERALISEDPLANESTRAIN;
+using Behaviour = ModCamClay_semiExpl_coupled-h,real,false>;
+const auto r = mfront::gb::integrate<Behaviour>(<d, Behaviour::STANDARDTANGENTOPERATOR, ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy());
+return r;
+} // end of ModCamClay_semiExpl_coupled_AxisymmetricalGeneralisedPlaneStrain
+
+MFRONT_SHARED0BJ int ModCamClay_semiExpl_coupled_AxisymmetricalGeneralisedPlaneStress(mfront_gb_BehaviourData* const d){
+using namespace tfel::material;
+using real = mfront::gb::real;
+constexpr auto h = ModellingHypothesis::AXISYMMETRICALGENERALISEDPLANESTRESS;
+using Behaviour = ModCamClay_semiExpl_coupled-h,real,false>;
+const auto r = mfront::gb::integrate<Behaviour>(<d, Behaviour::STANDARDTANGENTOPERATOR, ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy());
+return r;
+} // end of ModCamClay_semiExpl_coupled_AxisymmetricalGeneralisedPlaneStress
+
+MFRONT_SHARED0BJ int ModCamClay_semiExpl_coupled_Axisymmetrical(mfront_gb_BehaviourData* const d){
+using namespace tfel::material;
+using real = mfront::gb::real;
+constexpr auto h = ModellingHypothesis::AXISYMMETRICAL;
+using Behaviour = ModCamClay_semiExpl_coupled-h,real,false>;
+const auto r = mfront::gb::integrate<Behaviour>(<d, Behaviour::STANDARDTANGENTOPERATOR, ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy());
+return r;
+} // end of ModCamClay_semiExpl_coupled_Axisymmetrical
+
+MFRONT_SHARED0BJ int ModCamClay_semiExpl_coupled_PlaneStress(mfront_gb_BehaviourData* const d){
+using namespace tfel::material;
+using real = mfront::gb::real;
+constexpr auto h = ModellingHypothesis::PLANESTRESS;
+using Behaviour = ModCamClay_semiExpl_coupled-h,real,false>;
+const auto r = mfront::gb::integrate<Behaviour>(<d, Behaviour::STANDARDTANGENTOPERATOR, ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy());
+return r;
+} // end of ModCamClay_semiExpl_coupled_PlaneStress
+
+MFRONT_SHARED0BJ int ModCamClay_semiExpl_coupled_PlaneStrain(mfront_gb_BehaviourData* const d){
+using namespace tfel::material;
+using real = mfront::gb::real;
+constexpr auto h = ModellingHypothesis::PLANESTRAIN;
+using Behaviour = ModCamClay_semiExpl_coupled-h,real,false>;
+const auto r = mfront::gb::integrate<Behaviour>(<d, Behaviour::STANDARDTANGENTOPERATOR, ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy());
+return r;
+} // end of ModCamClay_semiExpl_coupled_PlaneStrain
+
+MFRONT_SHARED0BJ int ModCamClay_semiExpl_coupled_GeneralisedPlaneStrain(mfront_gb_BehaviourData* const d){
+using namespace tfel::material;
+using real = mfront::gb::real;
+constexpr auto h = ModellingHypothesis::GENERALISEDPLANESTRAIN;
+using Behaviour = ModCamClay_semiExpl_coupled-h,real,false>;
+const auto r = mfront::gb::integrate<Behaviour>(<d, Behaviour::STANDARDTANGENTOPERATOR, ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy());
+return r;
+} // end of ModCamClay_semiExpl_coupled_GeneralisedPlaneStrain
+
+MFRONT_SHARED0BJ int ModCamClay_semiExpl_coupled_Tridimensional(mfront_gb_BehaviourData* const d){
+using namespace tfel::material;
+using real = mfront::gb::real;
+constexpr auto h = ModellingHypothesis::TRIDIMENSIONAL;
+using Behaviour = ModCamClay_semiExpl_coupled-h,real,false>;
+const auto r = mfront::gb::integrate<Behaviour>(<d, Behaviour::STANDARDTANGENTOPERATOR, ModCamClay_semiExpl_coupled_getOutOfBoundsPolicy());
+return r;
+} // end of ModCamClay_semiExpl_coupled_Tridimensional
+
+#ifdef __cplusplus
+}
+#endif /* __cplusplus */
+
+diff --git a/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/src/ModCamClay_semiExpl_coupled.cxx b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/src/ModCamClay_semiExpl_coupled.cxx
new file mode 100644
index 0000000000..58a832d5b2
--- /dev/null
+++ b/Tests/Data/ThermoRichardsMechanics/MFront/TRM_MCC_coupled/src/ModCamClay_semiExpl_coupled.cxx
@@ -0,0 +1,150 @@
+/*
+ * \file ModCamClay_semiExpl_coupled.cxx
+ * \brief this file implements the ModCamClay_semiExpl_coupled Behaviour.
+ * File generated by tfel version 4.2.4-dev
+ * \author Ageel Afzal Chaudhry , Christian Silbermann , Eric Simo , Miguel Mánica , Thomas Helfer , Thomas Nagel
+ * \date 01 / 09 / 2025
+ */
+
+#include<string>
+#include<cstring>
+#include<sstream>

```

```

#include<fstream>
#include<stdexcept>
+
#include"TFEL/Raise.hxx"
#include"TFEL/Material/ModCamClay_semiExpl_coupledBehaviourData.hxx"
#include"TFEL/Material/ModCamClay_semiExpl_coupledIntegrationData.hxx"
#include"TFEL/Material/ModCamClay_semiExpl_coupled.hxx"
+
namespace tfel::material{
+
ModCamClay_semiExpl_coupledParametersInitializer&
ModCamClay_semiExpl_coupledParametersInitializer::get()
+{
+static ModCamClay_semiExpl_coupledParametersInitializer i;
+return i;
+}
+
+ModCamClay_semiExpl_coupledParametersInitializer::ModCamClay_semiExpl_coupledParametersInitializer()
+{
+this->theta = 1;
+this->epsilon = 1e-14;
+this->iterMax = 100;
+this->minimal_time_step_scaling_factor = 0.1;
+this->maximal_time_step_scaling_factor = 1.7976931348623e+308;
+this->numerical_jacobian_epsilon = 1e-15;
+// Reading parameters from a file
+ModCamClay_semiExpl_coupledParametersInitializer::readParameters(*this,"ModCamClay_semiExpl_coupled-parameters.txt");
+}
+
+void
+ModCamClay_semiExpl_coupledParametersInitializer::set(const char* const key,
+const double v){
+using namespace std;
+if(!::strcmp("theta",key)==0){
+this->theta = v;
+} else if(!::strcmp("epsilon",key)==0){
+this->epsilon = v;
+} else if(!::strcmp("minimal_time_step_scaling_factor",key)==0){
+this->minimal_time_step_scaling_factor = v;
+} else if(!::strcmp("maximal_time_step_scaling_factor",key)==0){
+this->maximal_time_step_scaling_factor = v;
+} else if(!::strcmp("numerical_jacobian_epsilon",key)==0){
+this->numerical_jacobian_epsilon = v;
+} else {
+tfel::raise("ModCamClay_semiExpl_coupledParametersInitializer::set: "
+" no parameter named '"+std::string(key)+"'");
+}
+}
+
+void
+ModCamClay_semiExpl_coupledParametersInitializer::set(const char* const key,
+const unsigned short v){
+using namespace std;
+if(!::strcmp("iterMax",key)==0){
+this->iterMax = v;
+} else {
+tfel::raise("ModCamClay_semiExpl_coupledParametersInitializer::set: "
+"no parameter named '"+std::string(key)+"'");
+}
+}
+
+double
+ModCamClay_semiExpl_coupledParametersInitializer::getDouble(const std::string& n,
+const std::string& v)
+{
+double value;
+std::stringstream converter(v);
+converter >> value;
+tfel::raise_if(!converter||(!converter.eof()),
+"ModCamClay_semiExpl_coupledParametersInitializer::getDouble: "
+"can't convert '"+v+"' to double for parameter '"+n+"'");
+return value;
+}
+
+unsigned short
+ModCamClay_semiExpl_coupledParametersInitializer::getUnsignedShort(const std::string& n,
+const std::string& v)
+{
+unsigned short value;
+std::stringstream converter(v);
+converter >> value;
+tfel::raise_if(!converter||(!converter.eof()),
+"ModCamClay_semiExpl_coupledParametersInitializer::getUnsignedShort: "
+"can't convert '"+v+"' to unsigned short for parameter '"+n+"'");
+return value;
+}
+
+void
+ModCamClay_semiExpl_coupledParametersInitializer::readParameters(ModCamClay_semiExpl_coupledParametersInitializer& pi,const char* const fn){
+auto tokenize = [](const std::string& line){
+std::stringstream tokenizer(line);
+std::vector<std::string> tokens;
+std::copy(std::istream_iterator<std::string>(tokenizer),
+std::istream_iterator<std::string>(),
+std::back_inserter(tokens));
+return tokens;
+};
+std::ifstream f(fn);
+if(!f){
+return;
+}
+size_t ln = 1u;
+auto line = std::string();
+while(std::getline(f,line)){
+auto tokens = tokenize(line);
+auto throw_if = [line_number = ln, fn](const bool mfront_c,
+const std::string& mfront_error_message){
+tfel::raise_if(mfront_c,"ModCamClay_semiExpl_coupledParametersInitializer::readParameters: "
+"error at line '"+std::to_string(line_number)+"' "
+"while reading parameter file '"+std::string(fn)+"' "
+"("mfront_error_message)");
+};
+if(tokens.empty()){
+continue;
+}
+if(tokens[0][0]!='#'){
+continue;

```



```

+ {
+     parameters.push_back(
+         std::make_unique<ParameterLib::ConstantParameter<double>>(name,
+             value));
+     });
+
+     add_param("YoungModulus", 52e6);
+     add_param("PoissonRatio", 0.3);
+     add_param("CriticalStateLineSlope", 1.2);
+     add_param("SwellingLineSlope", 6.6e-3);
+     add_param("VirginConsolidationLineSlope", 7.7e-2);
+     add_param("InitialPreConsolidationPressure", 2e5);
+     add_param("InitialVolumeRatio", initial_volume_ratio);
+     add_param("ResidualLiquidSaturation", 0.0);
+     add_param("ResidualGasSaturation", 0.0);
+     add_param("BubblePressure", 1e4);
+     add_param("VanGenuchtenExponent_m", 0.4);
+
+     return parameters;
+ }
+
+ double mcbBridgeInitialVolumeRatioFromDryDensity(
+     double const dry_density,
+     double const solid_density = 2470.0)
+ {
+     return solid_density / dry_density;
+ }
+
+ std::unique_ptr<MB> createMCCBridgeModelFromXml(
+     std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+     char const* const xml)
+ {
+     auto local_coordinate_system = std::nullopt;
+
+     auto ptree = Tests::readXml(xml);
+     BaseLib::ConfigTree config_tree(std::move(ptree), "FILENAME",
+         &BaseLib::ConfigTree::onerror,
+         &BaseLib::ConfigTree::onwarning);
+
+     return MaterialLib::Solids::createConstitutiveRelation<3>(
+         parameters, local_coordinate_system, config_tree);
+ }
+
+ std::unique_ptr<MB> createMCCBridgeNativeModel(
+     std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters)
+ {
+     char const* xml = R"XML(
+     <type>MFront</type>
+     <behaviour>ModCamClay_semiExpL_constE</behaviour>
+     <library path_is_relative_to_prj_file="false">LibOgsMFrontBehaviour</library>
+     <material_properties>
+         <material_property name="YoungModulus" parameter="YoungModulus"/>
+         <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+         <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+         <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+         <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+         <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+     </material_properties>
+     <initial_values>
+         <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+         <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+     </initial_values>
+     )XML";
+
+     return createMCCBridgeModelFromXml(parameters, xml);
+ }
+
+ std::unique_ptr<MB> createMCCBridgePressureCoupledModel(
+     std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters)
+ {
+     char const* xml = R"XML(
+     <type>MFrontRichardsMechanics</type>
+     <behaviour>ModCamClay_semiExpL_constE_pressureCoupled</behaviour>
+     <library path_is_relative_to_prj_file="false">LibOgsMFrontBehaviour</library>
+     <material_properties>
+         <material_property name="YoungModulus" parameter="YoungModulus"/>
+         <material_property name="PoissonRatio" parameter="PoissonRatio"/>
+         <material_property name="CriticalStateLineSlope" parameter="CriticalStateLineSlope"/>
+         <material_property name="SwellingLineSlope" parameter="SwellingLineSlope"/>
+         <material_property name="VirginConsolidationLineSlope" parameter="VirginConsolidationLineSlope"/>
+         <material_property name="CharacteristicPreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+         <material_property name="ResidualLiquidSaturation" parameter="ResidualLiquidSaturation"/>
+         <material_property name="ResidualGasSaturation" parameter="ResidualGasSaturation"/>
+         <material_property name="BubblePressure" parameter="BubblePressure"/>
+         <material_property name="VanGenuchtenExponent_m" parameter="VanGenuchtenExponent_m"/>
+     </material_properties>
+     <initial_values>
+         <state_variable name="PreConsolidationPressure" parameter="InitialPreConsolidationPressure"/>
+         <state_variable name="VolumeRatio" parameter="InitialVolumeRatio"/>
+     </initial_values>
+     )XML";
+
+     return createMCCBridgeModelFromXml(parameters, xml);
+ }
+
+ void initializeMCCBridgeState(MB const& model,
+     MB::MaterialStateVariables& state)
+ {
+     ParameterLib::SpatialPosition x();
+     model.initializeInternalStateVariables(0.0, x, state);
+ }
+
+ double getMCCBridgeInternalScalar(MB const& model,
+     MB::MaterialStateVariables const& state,
+     std::string const& name)
+ {
+     auto const internal_variables = model.getInternalVariables();
+     auto const it = std::find_if(
+         internal_variables.begin(), internal_variables.end(),
+         [&name](auto const& internal_variable)
+         { return internal_variable.name == name; });
+
+     if (it == internal_variables.end())
+     {
+         return std::numeric_limits<double>::quiet_NaN();
+     }
+ }

```



```

+ std::vector<double> cache;
+ auto const& values = it->getter(state, cache);
+ if (values.size() != 1)
+ {
+     return std::numeric_limits<double>::quiet_NaN();
+ }
+
+ return values[0];
+}
+
+std::vector<double> getMCCBridgeInternalVector(
+ MB const& model,
+ MB::MaterialStateVariables const& state,
+ std::string const& name)
+{
+ auto const internal_variables = model.getInternalVariables();
+ auto const it = std::find_if(
+     internal_variables.begin(), internal_variables.end(),
+     [&name](auto const& internal_variable)
+     { return internal_variable.name == name; });
+
+ if (it == internal_variables.end())
+ {
+     return {};
+ }
+
+ std::vector<double> cache;
+ auto const& values = it->getter(state, cache);
+ return {values.begin(), values.end()};
+}
+
+KV mcbBridgeIsotropicStress(double const value)
+{
+ KV sigma = KV::Zero();
+ sigma[0] = value;
+ sigma[1] = value;
+ sigma[2] = value;
+ return sigma;
+}
+
+KV mcbBridgeIsotropicStrainFromVolumetric(double const epsilon_v)
+{
+ KV eps = KV::Zero();
+ eps[0] = epsilon_v / 3.0;
+ eps[1] = epsilon_v / 3.0;
+ eps[2] = epsilon_v / 3.0;
+ return eps;
+}
+
+double mcbBridgeSaturationVG(double const p_cap,
+ double const residual_liquid_saturation,
+ double const residual_gas_saturation,
+ double const exponent_m,
+ double const bubble_pressure)
+{
+ double const saturation_max = 1.0 - residual_gas_saturation;
+ if (p_cap <= 0.0)
+ {
+     return saturation_max;
+ }
+
+ double const exponent_n = 1.0 / (1.0 - exponent_m);
+ double const p = p_cap / bubble_pressure;
+ double const p_to_n = std::pow(p, exponent_n);
+ double const s_eff = std::pow(p_to_n + 1.0, -exponent_m);
+ double const s =
+     s_eff * (saturation_max - residual_liquid_saturation) +
+     residual_liquid_saturation;
+ return std::clamp(s, residual_liquid_saturation, saturation_max);
+}
+
+double mcbBridgeDSaturationVGdLiquidPressure(
+ double const liquid_pressure,
+ double const residual_liquid_saturation,
+ double const residual_gas_saturation,
+ double const exponent_m,
+ double const bubble_pressure)
+{
+ double const p_cap = -liquid_pressure;
+ if (p_cap <= 0.0)
+ {
+     return 0.0;
+ }
+
+ double const exponent_n = 1.0 / (1.0 - exponent_m);
+ double const saturation_max = 1.0 - residual_gas_saturation;
+ double const p = p_cap / bubble_pressure;
+ double const p_to_n = std::pow(p, exponent_n);
+ double const s_eff = std::pow(p_to_n + 1.0, -exponent_m);
+ double const ds_eff_dp_cap =
+     -exponent_m * exponent_n * p_to_n * s_eff /
+     (p_cap * (p_to_n + 1.0));
+
+ return -ds_eff_dp_cap *
+     (saturation_max - residual_liquid_saturation);
+}
+
+bool copyMCCBridgeInternalVariables(MB const& source_model,
+ MB::MaterialStateVariables const& source,
+ MB const& target_model,
+ MB::MaterialStateVariables& target)
+{
+ auto const source_internal_variables = source_model.getInternalVariables();
+ auto const target_internal_variables = target_model.getInternalVariables();
+
+ for (auto const& source_variable : source_internal_variables)
+ {
+     auto const target_it = std::find_if(
+         target_internal_variables.begin(), target_internal_variables.end(),
+         [&source_variable](auto const& target_variable)
+         { return target_variable.name == source_variable.name; });
+
+     if (target_it == target_internal_variables.end())
+     {
+         return false;
+     }
+ }
+}

```

```

+         std::vector<double> cache;
+         auto const& source_values = source_variable.getter(source, cache);
+         auto target_values = target_it->reference(target);
+         if (target_values.size() != source_values.size())
+         {
+             return false;
+         }
+
+         std::copy(source_values.begin(), source_values.end(),
+                 target_values.begin());
+     }
+
+     return true;
+ }
+
+void setMCCBridgeThermodynamicForces(MB::MaterialStateVariables& state,
+        KV const& stress,
+        double const saturation)
+{
+    auto* const mfront_state =
+        dynamic_cast<MSM::MaterialStateVariablesMFront<3>>>(&state);
+    if (mfront_state == nullptr)
+    {
+        return;
+    }
+
+    auto& thermodynamic_forces =
+        mfront_state->behaviour_data.sl.thermodynamic_forces;
+    thermodynamic_forces[0] = stress[0];
+    thermodynamic_forces[1] = stress[1];
+    thermodynamic_forces[2] = stress[2];
+    thermodynamic_forces[3] = stress[3];
+    thermodynamic_forces[4] = stress[4];
+    thermodynamic_forces[5] = stress[5];
+    if (thermodynamic_forces.size() > 6u)
+    {
+        thermodynamic_forces[6] = saturation;
+    }
+}
+
+struct NativeMCCOneStepResult
+{
+    KV stress = KV::Zero();
+    KM tangent = KM::Zero();
+};
+
+std::optional<NativeMCCOneStepResult> runMCCBridgeNativeOneStep(
+    double const epsilon_v,
+    double const liquid_pressure)
+{
+    auto parameters = createMCCBridgeParameters();
+    auto native = createMCCBridgeNativeModel(parameters);
+    auto state = native->createMaterialStateVariables();
+    initializeMCCBridgeState(&native, &state);
+
+    MPL::VariableArray previous;
+    previous.mechanical_strain.emplace<KV>(KV::Zero());
+    previous.stress.emplace<KV>(mccBridgeIsotropicStress(-5e3));
+    previous.liquid_phase_pressure = -5e3;
+    previous.temperature = 293.15;
+
+    MPL::VariableArray current = previous;
+    current.mechanical_strain.emplace<KV>({
+        mccBridgeIsotropicStrainFromVolumetric(epsilon_v)});
+    current.liquid_phase_pressure = liquid_pressure;
+
+    ParameterLib::SpatialPosition x{};
+    auto const solution =
+        native->integrateStress(previous, current, 1.0, x, 1.0, &state);
+    if (!solution)
+    {
+        return std::nullopt;
+    }
+
+    auto const& [stress, state_new, tangent] = *solution;
+    (void)state_new;
+    return NativeMCCOneStepResult{stress, tangent};
+}
+
+struct PressureCoupledMCCOneStepResult
+{
+    KV stress = KV::Zero();
+    KM dStress_dStrain = KM::Zero();
+    KV dStress_dLiquidPressure = KV::Zero();
+};
+
+std::optional<PressureCoupledMCCOneStepResult> runMCCBridgePressureCoupledOneStep(
+    double const epsilon_v,
+    double const liquid_pressure)
+{
+    auto parameters = createMCCBridgeParameters();
+    auto bridge = createMCCBridgePressureCoupledModel(parameters);
+    auto state = bridge->createMaterialStateVariables();
+    initializeMCCBridgeState(&bridge, &state);
+
+    MPL::VariableArray previous;
+    previous.mechanical_strain.emplace<KV>(KV::Zero());
+    previous.stress.emplace<KV>(mccBridgeIsotropicStress(-5e3));
+    previous.liquid_phase_pressure = -5e3;
+    previous.temperature = 293.15;
+
+    MPL::VariableArray current = previous;
+    current.mechanical_strain.emplace<KV>({
+        mccBridgeIsotropicStrainFromVolumetric(epsilon_v)});
+    current.liquid_phase_pressure = liquid_pressure;
+
+    ParameterLib::SpatialPosition x{};
+    auto const response = bridge->integrateStressPressureCoupled(
+        previous, current, 1.0, x, 1.0, &state);
+    if (!response)
+    {
+        return std::nullopt;
+    }
+
+    return PressureCoupledMCCOneStepResult{response->stress,
+        response->dStress_dStrain,
+        response->dStress_dLiquidPressure};
+}

```

```

*}
*} // namespace
+
+TEST(MaterialLib_RMBridgeMFront_MCC,
+    PressureCoupledConstEMatchesNativeMechanicalPath)
+{
+    auto parameters = createMCCBridgeParameters();
+    auto native = createMCCBridgeNativeModel(parameters);
+    auto bridge = createMCCBridgePressureCoupledModel(parameters);
+
+    auto native_state = native->createMaterialStateVariables();
+    auto bridge_state = bridge->createMaterialStateVariables();
+    initializeMCCBridgeState(*native, *native_state);
+    initializeMCCBridgeState(*bridge, *bridge_state);
+
+    ParameterLib::SpatialPosition x{};
+    double const liquid_pressure_prev = -5e3;
+    double const residual_liquid_saturation = 0.0;
+    double const residual_gas_saturation = 0.0;
+    double const exponent_m = 0.4;
+    double const bubble_pressure = 1e4;
+
+    MPL::VariableArray native_prev;
+    native_prev.mechanical_strain.emplace<KV>(KV::Zero());
+    native_prev.stress.emplace<KV>(mccBridgeIsotropicStress(-5e3));
+    native_prev.liquid_phase_pressure = liquid_pressure_prev;
+    native_prev.temperature = 293.15;
+
+    MPL::VariableArray bridge_prev = native_prev;
+
+    struct Step
+    {
+        double t;
+        double eps_v;
+        double liquid_pressure;
+    };
+
+    std::vector<Step> const steps = {
+        {1.0, -5e-4, -2e4},
+        {2.0, -1.5e-3, -4e4},
+        {3.0, -3.5e-3, -7e4},
+        {4.0, -6.5e-3, -1e5},
+    };
+
+    for (std::size_t i = 0; i < steps.size(); ++i)
+    {
+        MPL::VariableArray native_current = native_prev;
+        native_current.mechanical_strain.emplace<KV>{
+            mccBridgeIsotropicStrainFromVolumetric(steps[i].eps_v)};
+        native_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+        MPL::VariableArray bridge_current = bridge_prev;
+        bridge_current.mechanical_strain.emplace<KV>{
+            mccBridgeIsotropicStrainFromVolumetric(steps[i].eps_v)};
+        bridge_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+        double const dt = i == 0 ? steps[i].t : steps[i].t - steps[i - 1].t;
+
+        auto native_solution = native->integrateStress(
+            native_prev, native_current, steps[i].t, x, dt, *native_state);
+        ASSERT_TRUE(native_solution);
+
+        auto bridge_solution = bridge->integrateStressPressureCoupled(
+            bridge_prev, bridge_current, steps[i].t, x, dt, *bridge_state);
+        ASSERT_TRUE(bridge_solution);
+
+        auto& [native_stress, native_state_new, native_tangent] =
+            *native_solution;
+        auto& bridge_response = *bridge_solution;
+
+        EXPECT_TRUE(
+            (native_stress - bridge_response.stress).isZero(5e-11));
+        EXPECT_TRUE((native_tangent - bridge_response.dStress_dStrain)
+            .isZero(1e-8));
+
+        EXPECT_TRUE(bridge_response.dStress_dLiquidPressure.isZero(1e-12));
+        EXPECT_TRUE(bridge_response.dSaturation_dStrain.isZero(1e-12));
+
+        auto const expected_saturation = mccBridgeSaturationVG(
+            -steps[i].liquid_pressure, residual_liquid_saturation,
+            residual_gas_saturation, exponent_m, bubble_pressure);
+        auto const expected_dS_dp = mccBridgeDSaturationVGdLiquidPressure(
+            steps[i].liquid_pressure, residual_liquid_saturation,
+            residual_gas_saturation, exponent_m, bubble_pressure);
+
+        EXPECT_NEAR(expected_saturation, bridge_response.saturation, 1e-15);
+        EXPECT_NEAR(expected_dS_dp, bridge_response.dSaturation_dLiquidPressure,
+            1e-15);
+
+        EXPECT_NEAR(
+            getMCCBridgeInternalScalar(*native, *native_state_new,
+                "EquivalentPlasticStrain"),
+            getMCCBridgeInternalScalar(*bridge, *bridge_response.state,
+                "EquivalentPlasticStrain"),
+            1e-14);
+        EXPECT_NEAR(
+            getMCCBridgeInternalScalar(*native, *native_state_new,
+                "PreConsolidationPressure"),
+            getMCCBridgeInternalScalar(*bridge, *bridge_response.state,
+                "PreConsolidationPressure"),
+            1e-8);
+        EXPECT_NEAR(
+            getMCCBridgeInternalScalar(*native, *native_state_new,
+                "PlasticVolumetricStrain"),
+            getMCCBridgeInternalScalar(*bridge, *bridge_response.state,
+                "PlasticVolumetricStrain"),
+            1e-14);
+        EXPECT_NEAR(getMCCBridgeInternalScalar(*native, *native_state_new,
+            "VolumeRatio"),
+            getMCCBridgeInternalScalar(*bridge,
+                *bridge_response.state,
+                "VolumeRatio"),
+            1e-14);
+
+        auto const native_elastic_strain = getMCCBridgeInternalVector(
+            *native, *native_state_new, "ElasticStrain");
+        auto const bridge_elastic_strain = getMCCBridgeInternalVector(

```

```

+         *bridge, *bridge_response.state, "ElasticStrain");
+         ASSERT_EQ(native_elastic_strain.size(), bridge_elastic_strain.size());
+         for (std::size_t k = 0; k < native_elastic_strain.size(); ++k)
+         {
+             EXPECT_NEAR(native_elastic_strain[k], bridge_elastic_strain[k],
+                           1e-14);
+         }
+
+         native_state = std::move(native_state_new);
+         bridge_state = std::move(bridge_response.state);
+
+         native_prev = native_current;
+         native_prev.stress.emplace<KV>(native_stress);
+         bridge_prev = bridge_current;
+         bridge_prev.stress.emplace<KV>(bridge_response.stress);
+     }
+ }
+
+ *TEST(MaterialLib_RMBCBridgeMF_Front_MCC, PressureCoupledConstEFactoryCreationWorks)
+ {
+     auto parameters = createMCCBridgeParameters();
+     auto bridge = createMCCBridgePressureCoupledModel(parameters);
+     ASSERT_TRUE(bridge);
+
+     auto state = bridge->createMaterialStateVariables();
+     initializeMCCBridgeState(*bridge, *state);
+
+     MPL::VariableArray previous;
+     previous.mechanical_strain.emplace<KV>(KV::Zero());
+     previous.stress.emplace<KV>(mccBridgeIsotropicStress(-5e3));
+     previous.liquid_phase_pressure = -5e3;
+     previous.temperature = 293.15;
+
+     MPL::VariableArray current = previous;
+     current.mechanical_strain.emplace<KV>({
+         mccBridgeIsotropicStrainFromVolumetric(-1e-3)});
+     current.liquid_phase_pressure = -2e4;
+
+     ParameterLib::SpatialPosition x{};
+     auto response = bridge->integrateStressPressureCoupled(previous, current,
+                                                             1.0, x, 1.0,
+                                                             *state);
+     ASSERT_TRUE(response);
+     EXPECT_TRUE(response->stress.allFinite());
+     EXPECT_TRUE(response->dStress_dStrain.allFinite());
+     EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+     EXPECT_TRUE(std::isfinite(response->saturation));
+     EXPECT_TRUE(std::isfinite(response->dSaturation_dLiquidPressure));
+     EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+ }
+
+ *TEST(MaterialLib_RMBCBridgeMF_Front_MCC,
+       PressureCoupledConstEMatchesNativeAcrossDryDensitySweep)
+ {
+     struct DryDensityCase
+     {
+         double dry_density;
+     };
+
+     std::array<DryDensityCase, 4> const dry_density_cases{{
+         {1350.0},
+         {1450.0},
+         {1550.0},
+         {1650.0},
+     }};
+
+     ParameterLib::SpatialPosition x{};
+     double const residual_liquid_saturation = 0.0;
+     double const residual_gas_saturation = 0.0;
+     double const exponent_m = 0.4;
+     double const bubble_pressure = 1e4;
+
+     struct Step
+     {
+         double t;
+         double eps_v;
+         double liquid_pressure;
+     };
+
+     std::vector<Step> const steps = {
+         {1.0, -5e-4, -2e4},
+         {2.0, -1.5e-3, -4e4},
+         {3.0, -3.5e-3, -7e4},
+     };
+
+     for (auto const& c : dry_density_cases)
+     {
+         SCOPED_TRACE(::testing::Message()
+                       << "dry_density=" << c.dry_density << " kg/m^3");
+
+         auto parameters = createMCCBridgeParameters(
+             mccBridgeInitialVolumeRatioFromDryDensity(c.dry_density));
+         auto native = createMCCBridgeNativeModel(parameters);
+         auto bridge = createMCCBridgePressureCoupledModel(parameters);
+
+         auto native_state = native->createMaterialStateVariables();
+         auto bridge_state = bridge->createMaterialStateVariables();
+         initializeMCCBridgeState(*native, *native_state);
+         initializeMCCBridgeState(*bridge, *bridge_state);
+
+         MPL::VariableArray native_prev;
+         native_prev.mechanical_strain.emplace<KV>(KV::Zero());
+         native_prev.stress.emplace<KV>(mccBridgeIsotropicStress(-5e3));
+         native_prev.liquid_phase_pressure = -5e3;
+         native_prev.temperature = 293.15;
+
+         MPL::VariableArray bridge_prev = native_prev;
+
+         for (std::size_t i = 0; i < steps.size(); ++i)
+         {
+             MPL::VariableArray native_current = native_prev;
+             native_current.mechanical_strain.emplace<KV>({
+                 mccBridgeIsotropicStrainFromVolumetric(steps[i].eps_v)});
+             native_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+             MPL::VariableArray bridge_current = bridge_prev;
+             bridge_current.mechanical_strain.emplace<KV>({

```

```

+         mcbBridgeIsotropicStrainFromVolumetric(steps[i].eps_v));
+         bridge_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+         double const dt =
+         i == 0 ? steps[i].t : steps[i].t - steps[i - 1].t;
+
+         auto native_solution = native->integrateStress(
+             native_prev, native_current, steps[i].t, x, dt, *native_state);
+         auto bridge_solution = bridge->integrateStressPressureCoupled(
+             bridge_prev, bridge_current, steps[i].t, x, dt, *bridge_state);
+
+         ASSERT_TRUE(native_solution);
+         ASSERT_TRUE(bridge_solution);
+
+         auto& [native_stress, native_state_new, native_tangent] =
+             *native_solution;
+         auto& bridge_response = *bridge_solution;
+
+         EXPECT_TRUE((native_stress - bridge_response.stress).isZero(5e-11));
+         EXPECT_TRUE((native_tangent - bridge_response.dStress_dStrain)
+             .isZero(1e-8));
+         EXPECT_TRUE(bridge_response.dStress_dLiquidPressure.isZero(1e-12));
+
+         auto const expected_saturation = mcbBridgeSaturationVG(
+             steps[i].liquid_pressure, residual_liquid_saturation,
+             residual_gas_saturation, exponent_m, bubble_pressure);
+         auto const expected_dS_dp = mcbBridgeSaturationVGdLiquidPressure(
+             steps[i].liquid_pressure, residual_liquid_saturation,
+             residual_gas_saturation, exponent_m, bubble_pressure);
+
+         EXPECT_NEAR(expected_saturation, bridge_response.saturation, 1e-15);
+         EXPECT_NEAR(expected_dS_dp,
+             bridge_response.dSaturation_dLiquidPressure, 1e-15);
+
+         for (auto const& name :
+             {"EquivalentPlasticStrain", "PreConsolidationPressure",
+              "PlasticVolumetricStrain", "VolumeRatio"})
+         {
+             double const native_value =
+                 getMCCBridgeInternalScalar(*native, *native_state_new, name);
+             double const bridge_value = getMCCBridgeInternalScalar(
+                 *bridge, *bridge_response.state, name);
+             EXPECT_NEAR(native_value, bridge_value,
+                 name == std::string("PreConsolidationPressure")
+                     ? 1e-8
+                     : 1e-14)
+                 << name;
+         }
+
+         native_state = std::move(native_state_new);
+         bridge_state = std::move(bridge_response.state);
+         native_prev = native_current;
+         native_prev.stress.emplace<KV>(native_stress);
+         bridge_prev = bridge_current;
+         bridge_prev.stress.emplace<KV>(bridge_response.stress);
+     }
+ }
+}
+
+TEST(MaterialLib_RMBridgeMFront_MCC,
+     PressureCoupledConstETangentConsistentNearPlasticOnset)
+{
+     constexpr double eps_v = -1.5e-3;
+     constexpr double liquid_pressure = -4e4;
+     constexpr double deps_v = 1e-7;
+
+     auto const native_base = runMCCBridgeNativeOneStep(eps_v, liquid_pressure);
+     auto const native_plus =
+         runMCCBridgeNativeOneStep(eps_v + deps_v, liquid_pressure);
+     auto const native_minus =
+         runMCCBridgeNativeOneStep(eps_v - deps_v, liquid_pressure);
+     auto const bridge_base =
+         runMCCBridgePressureCoupledOneStep(eps_v, liquid_pressure);
+     auto const bridge_plus =
+         runMCCBridgePressureCoupledOneStep(eps_v + deps_v, liquid_pressure);
+     auto const bridge_minus =
+         runMCCBridgePressureCoupledOneStep(eps_v - deps_v, liquid_pressure);
+
+     ASSERT_TRUE(native_base);
+     ASSERT_TRUE(native_plus);
+     ASSERT_TRUE(native_minus);
+     ASSERT_TRUE(bridge_base);
+     ASSERT_TRUE(bridge_plus);
+     ASSERT_TRUE(bridge_minus);
+
+     auto const dSigma_dEps_v_native_fd =
+         (native_plus->stress - native_minus->stress) / (2.0 * deps_v);
+     auto const dSigma_dEps_v_bridge_fd =
+         (bridge_plus->stress - bridge_minus->stress) / (2.0 * deps_v);
+
+     auto const dEps_dEps_v = mcbBridgeIsotropicStrainFromVolumetric(1.0);
+     auto const dSigma_dEps_v_native_tangent =
+         native_base->tangent * dEps_dEps_v;
+     auto const dSigma_dEps_v_bridge_tangent =
+         bridge_base->dStress_dStrain * dEps_dEps_v;
+
+     for (Eigen::Index i = 0; i < KV::RowsAtCompileTime; ++i)
+     {
+         auto const native_fd = dSigma_dEps_v_native_fd[i];
+         auto const native_tangent = dSigma_dEps_v_native_tangent[i];
+         auto const bridge_fd = dSigma_dEps_v_bridge_fd[i];
+         auto const bridge_tangent = dSigma_dEps_v_bridge_tangent[i];
+
+         double const native_tol =
+             1e2 + 5e-3 * std::max(std::abs(native_fd), std::abs(native_tangent));
+         double const bridge_tol =
+             1e2 + 5e-3 * std::max(std::abs(bridge_fd), std::abs(bridge_tangent));
+         double const parity_tol = 1e2 + 1e-4 * std::max(
+             std::abs(native_tangent),
+             std::abs(bridge_tangent));
+
+         EXPECT_NEAR(native_fd, native_tangent, native_tol);
+         EXPECT_NEAR(bridge_fd, bridge_tangent, bridge_tol);
+         EXPECT_NEAR(native_tangent, bridge_tangent, parity_tol);
+     }
+
+     EXPECT_TRUE((native_base->stress - bridge_base->stress).isZero(5e-11));

```

```

+ EXPECT_TRUE((native_base->tangent - bridge_base->dStress_dStrain)
+             .isZero(1e-8));
+ EXPECT_TRUE(bridge_base->dStress_dLiquidPressure.isZero(1e-12));
+ }
+
+ *TEST(MaterialLib_RMBridgeMFront_MCC,
+       PressureCoupledConstERestartStateTransferMatchesContinuousPath)
+ {
+     auto parameters = createMCCBridgeParameters();
+     auto native = createMCCBridgeNativeModel(parameters);
+     auto bridge = createMCCBridgePressureCoupledModel(parameters);
+
+     auto native_state = native->createMaterialStateVariables();
+     auto bridge_state = bridge->createMaterialStateVariables();
+     initializeMCCBridgeState(*native, *native_state);
+     initializeMCCBridgeState(*bridge, *bridge_state);
+
+     ParameterLib::SpatialPosition x{};
+
+     struct Step
+     {
+         double t;
+         double eps_v;
+         double liquid_pressure;
+     };
+
+     std::vector<Step> const steps = {
+         {1.0, -5e-4, -2e4},
+         {2.0, -1.5e-3, -4e4},
+         {3.0, -3.5e-3, -7e4},
+     };
+
+     MPL::VariableArray native_prev;
+     native_prev.mechanical_strain.emplace<KV>(KV::Zero());
+     native_prev.stress.emplace<KV>(mccBridgeIsotropicStress(-5e3));
+     native_prev.liquid_phase_pressure = -5e3;
+     native_prev.temperature = 293.15;
+
+     MPL::VariableArray bridge_prev = native_prev;
+
+     // Advance to a checkpoint (step index 1).
+     for (std::size_t i = 0; i < 2; ++i)
+     {
+         MPL::VariableArray native_current = native_prev;
+         native_current.mechanical_strain.emplace<KV>({
+             mccBridgeIsotropicStrainFromVolumetric(steps[i].eps_v)});
+         native_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+         MPL::VariableArray bridge_current = bridge_prev;
+         bridge_current.mechanical_strain.emplace<KV>({
+             mccBridgeIsotropicStrainFromVolumetric(steps[i].eps_v)});
+         bridge_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+         double const dt =
+             i == 0 ? steps[i].t : steps[i].t - steps[i - 1].t;
+
+         auto native_solution = native->integrateStress(
+             native_prev, native_current, steps[i].t, x, dt, *native_state);
+         auto bridge_solution = bridge->integrateStressPressureCoupled(
+             bridge_prev, bridge_current, steps[i].t, x, dt, *bridge_state);
+
+         ASSERT_TRUE(native_solution);
+         ASSERT_TRUE(bridge_solution);
+
+         auto& [native_stress, native_state_new, native_tangent] =
+             *native_solution;
+         (void)native_tangent;
+         auto& bridge_response = *bridge_solution;
+
+         native_state = std::move(native_state_new);
+         bridge_state = std::move(bridge_response.state);
+
+         native_prev = native_current;
+         native_prev.stress.emplace<KV>(native_stress);
+         bridge_prev = bridge_current;
+         bridge_prev.stress.emplace<KV>(bridge_response.stress);
+     }
+
+     // Continuous continuation for step index 2.
+     MPL::VariableArray native_current_cont = native_prev;
+     native_current_cont.mechanical_strain.emplace<KV>({
+         mccBridgeIsotropicStrainFromVolumetric(steps[2].eps_v)});
+     native_current_cont.liquid_phase_pressure = steps[2].liquid_pressure;
+
+     MPL::VariableArray bridge_current_cont = bridge_prev;
+     bridge_current_cont.mechanical_strain.emplace<KV>({
+         mccBridgeIsotropicStrainFromVolumetric(steps[2].eps_v)});
+     bridge_current_cont.liquid_phase_pressure = steps[2].liquid_pressure;
+
+     double const dt_cont = steps[2].t - steps[1].t;
+
+     auto native_continuous = native->integrateStress(
+         native_prev, native_current_cont, steps[2].t, x, dt_cont, *native_state);
+     auto bridge_continuous = bridge->integrateStressPressureCoupled(
+         bridge_prev, bridge_current_cont, steps[2].t, x, dt_cont, *bridge_state);
+     ASSERT_TRUE(native_continuous);
+     ASSERT_TRUE(bridge_continuous);
+
+     // Restart from checkpointed internal variables.
+     auto restart_native = createMCCBridgeNativeModel(parameters);
+     auto restart_bridge = createMCCBridgePressureCoupledModel(parameters);
+     auto restart_native_state = restart_native->createMaterialStateVariables();
+     auto restart_bridge_state = restart_bridge->createMaterialStateVariables();
+     initializeMCCBridgeState(*restart_native, *restart_native_state);
+     initializeMCCBridgeState(*restart_bridge, *restart_bridge_state);
+
+     ASSERT_TRUE(copyMCCBridgeInternalVariables(
+         *native, *native_state, *restart_native, *restart_native_state));
+     ASSERT_TRUE(copyMCCBridgeInternalVariables(
+         *bridge, *bridge_state, *restart_bridge, *restart_bridge_state));
+
+     auto const checkpoint_native_stress = std::get<KV>(native_prev.stress);
+     auto const checkpoint_bridge_stress = std::get<KV>(bridge_prev.stress);
+     double const checkpoint_saturation = mccBridgeSaturationVG(
+         -bridge_prev.liquid_phase_pressure, 0.0, 0.0, 0.4, 1e4);
+
+     setMCCBridgeThermodynamicForces(*restart_native_state,

```



```

+namespace MPL = MaterialPropertyLib;
+using KV = MathLib::KelvinVector::KelvinVectorType<3>;
+using KV2 = MathLib::KelvinVector::KelvinVectorType<2>;
+using MB = MaterialLib::Solids::MechanicsBase<3>;
+using MB2 = MaterialLib::Solids::MechanicsBase<2>;
+
+static auto createParameters()
+{
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> parameters;
+
+    auto add_param = [&parameters](char const* name, double const value)
+    {
+        parameters.push_back(
+            std::make_unique<ParameterLib::ConstantParameter<double>>(name,
+                                                                    value));
+    };
+
+    add_param("E", 1e10);
+    add_param("nu", 0.25);
+    add_param("swelling_slope", 0.1);
+    add_param("mass_exchange_coefficient", 1.0);
+    add_param("rho_LR_ref", 1000.0);
+    add_param("rho_l0", 1300.0);
+    add_param("rho_LR0", 1300.0);
+    add_param("rho_SR", 2470.0);
+    add_param("density_a", 1.3);
+    add_param("density_b", 1.0);
+    add_param("hamaker_constant", -6e-20);
+    add_param("specific_surface", 100.0);
+    add_param("phi0", 0.2);
+    add_param("area_factor_tuller", 1.0);
+    add_param("pore_area_shape_factor_tuller", 0.8584073464102069);
+    add_param("characteristic_pore_size", 1e-5);
+    add_param("surface_tension", 0.0715);
+    add_param("n_l0", 0.1);
+    add_param("epsilon_sw0", 0.0);
+
+    return parameters;
+}
+
+static auto createBenchmarkParameters()
+{
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> parameters;
+
+    auto add_param = [&parameters](char const* name, double const value)
+    {
+        parameters.push_back(
+            std::make_unique<ParameterLib::ConstantParameter<double>>(name,
+                                                                    value));
+    };
+
+    add_param("E", 52e6);
+    add_param("nu", 0.3);
+    add_param("swelling_slope", 0.1);
+    add_param("mass_exchange_coefficient", 1.0);
+    add_param("rho_LR_ref", 1000.0);
+    add_param("rho_l0", 1300.0);
+    add_param("rho_LR0", 1300.0);
+    add_param("rho_SR", 2470.0);
+    add_param("density_a", 1.3);
+    add_param("density_b", 1.0);
+    add_param("hamaker_constant", -6e-20);
+    add_param("specific_surface", 100.0);
+    add_param("phi0", 0.432);
+    add_param("area_factor_tuller", 1.0);
+    add_param("pore_area_shape_factor_tuller", 0.8584073464102069);
+    add_param("characteristic_pore_size", 1e-5);
+    add_param("surface_tension", 0.0715);
+    add_param("n_l0", 0.1);
+    add_param("epsilon_sw0", 0.0);
+
+    return parameters;
+}
+
+static auto createBenchmarkPressureConsistentParameters()
+{
+    auto parameters = createBenchmarkParameters();
+
+    auto set_param = [&parameters](char const* name, double const value)
+    {
+        auto it = std::find_if(parameters.begin(), parameters.end(),
+                                [name](auto const& parameter)
+                                { return parameter->name == name; });
+        EXPECT_TRUE(it != parameters.end());
+        *it = std::make_unique<ParameterLib::ConstantParameter<double>>(name,
+                                                                    value);
+    };
+
+    set_param("n_l0", 0.012069019712402708);
+    set_param("rho_LR0", 2267.4495975433856);
+
+    return parameters;
+}
+
+static void overwriteParameter(
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>>& parameters,
+    char const* name,
+    double const value)
+{
+    auto it = std::find_if(parameters.begin(), parameters.end(),
+                            [name](auto const& parameter)
+                            { return parameter->name == name; });
+    ASSERT_TRUE(it != parameters.end()) << name;
+    *it = std::make_unique<ParameterLib::ConstantParameter<double>>(name,
+                                                                    value);
+}
+
+static std::string stripQuotes(std::string value)
+{
+    value.erase(0, value.find_first_not_of(" \t\r\n\""));
+    value.erase(value.find_last_not_of(" \t\r\n\"") + 1);
+    return value;
+}
+
+static std::vector<std::string> splitCommaLine(std::string const& line)
+{
+    std::stringstream ss(line);

```



```

+ std::string field;
+ std::vector<std::string> fields;
+ while (std::getline(ss, field, ','))
+ {
+     fields.push_back(stripQuotes(field));
+ }
+ return fields;
+}
+
+struct BaselineRow
+{
+    int step = 0;
+    double pressure = 0.0;
+    double saturation = 0.0;
+    double mu_LR = 0.0;
+    double n_l = 0.0;
+    double phi_m = 0.0;
+    double phi_M = 0.0;
+    double rho_LR = 0.0;
+    double mu_LR = 0.0;
+    double rho_l_hat = 0.0;
+    double epsilon_v_total = 0.0;
+    double delta_epsilon_v = 0.0;
+    double epsilon_sw = 0.0;
+    double stress_xx = 0.0;
+};
+
+static std::vector<BaselineRow> loadBaselineRows(std::string const& filename)
+{
+    std::ifstream in(filename);
+    EXPECT_TRUE(in.good()) << filename;
+
+    std::string header_line;
+    std::getline(in, header_line);
+    auto const headers = splitCommaLine(header_line);
+
+    std::unordered_map<std::string, std::size_t> column;
+    for (std::size_t i = 0; i < headers.size(); ++i)
+    {
+        column[headers[i]] = i;
+    }
+
+    auto const get_value = [&](std::vector<std::string> const& fields,
+                               std::string const& key) -> double
+    {
+        return std::stod(fields.at(column.at(key)));
+    };
+
+    auto const get_optional_value = [&](std::vector<std::string> const& fields,
+                                         std::string const& key,
+                                         double const default_value) -> double
+    {
+        auto const it = column.find(key);
+        if (it == column.end())
+        {
+            return default_value;
+        }
+        return std::stod(fields.at(it->second));
+    };
+
+    std::vector<BaselineRow> rows;
+    std::string line;
+    while (std::getline(in, line))
+    {
+        if (line.empty())
+        {
+            continue;
+        }
+
+        auto const fields = splitCommaLine(line);
+        BaselineRow row;
+        row.step = static_cast<int>(get_value(fields, "step"));
+        row.pressure = get_value(fields, "pressure");
+        row.saturation = get_value(fields, "S_L");
+        row.mu_LR = get_value(fields, "mu_LR");
+        row.epsilon_v_total =
+            get_optional_value(fields, "epsilon_v_total", 0.0);
+        row.delta_epsilon_v =
+            get_optional_value(fields, "delta_epsilon_v", 0.0);
+        row.n_l = get_value(fields, "n_l");
+        row.phi_m = get_value(fields, "phi_m");
+        row.phi_M = get_value(fields, "phi_M");
+        row.rho_LR = get_value(fields, "rho_LR");
+        row.mu_LR = get_value(fields, "mu_LR");
+        row.rho_l_hat = get_value(fields, "rho_l_hat");
+        row.epsilon_sw = get_value(fields, "epsilon_sw");
+        row.stress_xx = get_value(fields, "sigma_5_xx");
+        rows.push_back(row);
+    }
+
+    return rows;
+}
+
+static double scaledTolerance(double const value,
+                               double const relative = 1e-8,
+                               double const absolute = 1e-12)
+{
+    return absolute + relative * std::max(1.0, std::abs(value));
+}
+
+static KV isotropicStrainFromVolumetric(double const epsilon_v)
+{
+    KV eps = KV::Zero();
+    eps[0] = epsilon_v / 3.0;
+    eps[1] = epsilon_v / 3.0;
+    eps[2] = epsilon_v / 3.0;
+    return eps;
+}
+
+static auto createBridgeModel(
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters)
+{
+    auto local_coordinate_system = std::nullopt;
+
+    const char* xml = R"XML(
+    <type>MFrontRichardsMechanics</type>
+    <behaviour>RichardsMechanicsDSMMicroMacroBridge</behaviour>

```

```

+ <library path_is_relative_to_prj_file="false">lib0gsmFrontBehaviour</library>
+ <material_properties>
+   <material_property name="YoungModulus" parameter="E"/>
+   <material_property name="PoissonRatio" parameter="nu"/>
+   <material_property name="SwellingSlope" parameter="swelling_slope"/>
+   <material_property name="MassExchangeCoefficient" parameter="mass_exchange_coefficient"/>
+   <material_property name="ReferenceLiquidDensityMacro" parameter="rho_LR_ref"/>
+   <material_property name="ReferenceLiquidDensityMicro" parameter="rho_L0"/>
+   <material_property name="ReferenceDensitySolid" parameter="rho_SR"/>
+   <material_property name="MicroLiquidDensityA" parameter="density_a"/>
+   <material_property name="MicroLiquidDensityB" parameter="density_b"/>
+   <material_property name="HamakerConstant" parameter="hamaker_constant"/>
+   <material_property name="SpecificSurface" parameter="specific_surface"/>
+   <material_property name="InitialPorosity" parameter="phi0"/>
+   <material_property name="AreaFactorTuller" parameter="area_factor_tuller"/>
+   <material_property name="PoreAreaShapeFactorTuller" parameter="pore_area_shape_factor_tuller"/>
+   <material_property name="CharacteristicPoreSize" parameter="characteristic_pore_size"/>
+   <material_property name="SurfaceTension" parameter="surface_tension"/>
+ </material_properties>
+ <initial_values>
+   <state_variable name="n_l" parameter="n_l0"/>
+   <state_variable name="rho_LR" parameter="rho_LR0"/>
+   <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+ </initial_values>
+ )XML";
+
+ auto ptree = Tests::readXml(xml);
+ BaseLib::ConfigTree config_tree(std::move(ptree), "FILENAME",
+   &BaseLib::ConfigTree::onerror,
+   &BaseLib::ConfigTree::onwarning);
+
+ return MSM::createMFrontGeneric<
+   3, boost::mpl::mp_list<MSM::Strain, MSM::LiquidPressure>,
+   boost::mpl::mp_list<MSM::Stress, MSM::Saturation>,
+   boost::mpl::mp_list<MSM::Temperature>>{
+   parameters, local_coordinate_system, config_tree;
+ }
+
+ static auto createBridgeModelThroughFactory(
+   std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters)
+ {
+   auto local_coordinate_system = std::nullopt;
+
+   const char* xml = R"XML(
+   <type>MFrontRichardsMechanics</type>
+   <behaviour>RichardsMechanicsDSMMicroMacroBridge</behaviour>
+   <library path_is_relative_to_prj_file="false">lib0gsmFrontBehaviour</library>
+   <material_properties>
+     <material_property name="YoungModulus" parameter="E"/>
+     <material_property name="PoissonRatio" parameter="nu"/>
+     <material_property name="SwellingSlope" parameter="swelling_slope"/>
+     <material_property name="MassExchangeCoefficient" parameter="mass_exchange_coefficient"/>
+     <material_property name="ReferenceLiquidDensityMacro" parameter="rho_LR_ref"/>
+     <material_property name="ReferenceLiquidDensityMicro" parameter="rho_L0"/>
+     <material_property name="ReferenceDensitySolid" parameter="rho_SR"/>
+     <material_property name="MicroLiquidDensityA" parameter="density_a"/>
+     <material_property name="MicroLiquidDensityB" parameter="density_b"/>
+     <material_property name="HamakerConstant" parameter="hamaker_constant"/>
+     <material_property name="SpecificSurface" parameter="specific_surface"/>
+     <material_property name="InitialPorosity" parameter="phi0"/>
+     <material_property name="AreaFactorTuller" parameter="area_factor_tuller"/>
+     <material_property name="PoreAreaShapeFactorTuller" parameter="pore_area_shape_factor_tuller"/>
+     <material_property name="CharacteristicPoreSize" parameter="characteristic_pore_size"/>
+     <material_property name="SurfaceTension" parameter="surface_tension"/>
+   </material_properties>
+   <initial_values>
+     <state_variable name="n_l" parameter="n_l0"/>
+     <state_variable name="rho_LR" parameter="rho_LR0"/>
+     <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+   </initial_values>
+   )XML";
+
+   auto ptree = Tests::readXml(xml);
+   BaseLib::ConfigTree config_tree(std::move(ptree), "FILENAME",
+     &BaseLib::ConfigTree::onerror,
+     &BaseLib::ConfigTree::onwarning);
+
+   return MaterialLib::Solids::createConstitutiveRelation<3>{
+     parameters, local_coordinate_system, config_tree;
+   }
+ }
+
+ static auto createBridgeModelThroughFactoryPlaneStrain(
+   std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters)
+ {
+   auto local_coordinate_system = std::nullopt;
+
+   const char* xml = R"XML(
+   <type>MFrontRichardsMechanics</type>
+   <behaviour>RichardsMechanicsDSMMicroMacroBridge</behaviour>
+   <library path_is_relative_to_prj_file="false">lib0gsmFrontBehaviour</library>
+   <material_properties>
+     <material_property name="YoungModulus" parameter="E"/>
+     <material_property name="PoissonRatio" parameter="nu"/>
+     <material_property name="SwellingSlope" parameter="swelling_slope"/>
+     <material_property name="MassExchangeCoefficient" parameter="mass_exchange_coefficient"/>
+     <material_property name="ReferenceLiquidDensityMacro" parameter="rho_LR_ref"/>
+     <material_property name="ReferenceLiquidDensityMicro" parameter="rho_L0"/>
+     <material_property name="ReferenceDensitySolid" parameter="rho_SR"/>
+     <material_property name="MicroLiquidDensityA" parameter="density_a"/>
+     <material_property name="MicroLiquidDensityB" parameter="density_b"/>
+     <material_property name="HamakerConstant" parameter="hamaker_constant"/>
+     <material_property name="SpecificSurface" parameter="specific_surface"/>
+     <material_property name="InitialPorosity" parameter="phi0"/>
+     <material_property name="AreaFactorTuller" parameter="area_factor_tuller"/>
+     <material_property name="PoreAreaShapeFactorTuller" parameter="pore_area_shape_factor_tuller"/>
+     <material_property name="CharacteristicPoreSize" parameter="characteristic_pore_size"/>
+     <material_property name="SurfaceTension" parameter="surface_tension"/>
+   </material_properties>
+   <initial_values>
+     <state_variable name="n_l" parameter="n_l0"/>
+     <state_variable name="rho_LR" parameter="rho_LR0"/>
+     <state_variable name="epsilon_sw" parameter="epsilon_sw0"/>
+   </initial_values>
+   )XML";
+
+   auto ptree = Tests::readXml(xml);
+   BaseLib::ConfigTree config_tree(std::move(ptree), "FILENAME",

```

```

+         &BaseLib::ConfigTree::onerror,
+         &BaseLib::ConfigTree::onwarning);
+
+ return MaterialLib::Solids::createConstitutiveRelation<2>({
+     parameters, local_coordinate_system, config_tree);
+ })
+
+template <typename Model, typename StateVariables>
+static void initializeState(Model const& model, StateVariables& state)
+{
+    ParameterLib::SpatialPosition x();
+    model.initializeInternalStateVariables(0.0, x, state);
+}
+
+template <typename Model, typename State>
+static double getInternalVariable(Model const& model,
+    State const& state,
+    std::string const& name)
+{
+    auto const internal_variables = model.getInternalVariables();
+    auto const it = std::find_if(
+        internal_variables.begin(), internal_variables.end(),
+        [&name](auto const& internal_variable)
+        { return internal_variable.name == name; });
+    if (it == internal_variables.end())
+    {
+        return std::numeric_limits<double>::quiet_NaN();
+    }
+
+    std::vector<double> cache;
+    auto const& values = it->getter(state, cache);
+    if (values.size() != 1)
+    {
+        return std::numeric_limits<double>::quiet_NaN();
+    }
+    return values[0];
+}
+
+template <typename Model, typename State>
+static void setInternalVariable(Model const& model,
+    State& state,
+    std::string const& name,
+    double const value)
+{
+    auto const internal_variables = model.getInternalVariables();
+    auto const it = std::find_if(
+        internal_variables.begin(), internal_variables.end(),
+        [&name](auto const& internal_variable)
+        { return internal_variable.name == name; });
+    ASSERT_TRUE(it != internal_variables.end()) << name;
+    auto values = it->reference(state);
+    ASSERT_EQ(values.size(), 1);
+    values[0] = value;
+}
+
+static void setThermodynamicForces(MB::MaterialStateVariables& state,
+    KV const& stress,
+    double const saturation)
+{
+    auto* const mfront_state =
+        dynamic_cast<MSM::MaterialStateVariablesMFront<3>>(&state);
+    ASSERT_TRUE(mfront_state != nullptr);
+
+    auto& thermodynamic_forces = mfront_state->behaviour_data.sl.thermodynamic_forces;
+    thermodynamic_forces[0] = stress[0];
+    thermodynamic_forces[1] = stress[1];
+    thermodynamic_forces[2] = stress[2];
+    thermodynamic_forces[3] = stress[3];
+    thermodynamic_forces[4] = stress[4];
+    thermodynamic_forces[5] = stress[5];
+    thermodynamic_forces[6] = saturation;
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSM1MacroBridgeMFront,
+    OneStepPressureResponse)
+{
+    auto const baseline_rows = loadBaselineRows(
+        TestInfoLib::TestInfo::data_path +
+        "/MaterialLib/MFront/RichardsMechanicsDSM1MacroBridge_overlap_transfer_baseline.csv");
+    ASSERT_EQ(baseline_rows.size(), 5);
+    auto const& saturated_anchor = baseline_rows.front();
+
+    auto const parameters = createParameters();
+    auto model = createBridgeModel(parameters);
+    ASSERT_TRUE(model != nullptr);
+
+    auto state = model->createMaterialStateVariables();
+    ASSERT_TRUE(state != nullptr);
+    initializeState(*model, *state);
+    EXPECT_NEAR(getInternalVariable(*model, *state, "n_l"), 0.1, 1e-15);
+    EXPECT_NEAR(getInternalVariable(*model, *state, "rho_LR"), 1300.0, 1e-12);
+
+    MPL::VariableArray variable_array_prev;
+    variable_array_prev.stress.template emplace<KV>(KV::Zero());
+    variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+    variable_array_prev.liquid_phase_pressure = 0.0;
+    variable_array_prev.liquid_saturation = 1.0;
+    variable_array_prev.temperature = 0.0;
+
+    MPL::VariableArray variable_array = variable_array_prev;
+    variable_array.liquid_phase_pressure = 1000.0;
+
+    ParameterLib::SpatialPosition x();
+    auto solution = model->integrateStress(variable_array_prev,
+        variable_array,
+        1.0,
+        x,
+        1.0,
+        *state);
+    ASSERT_TRUE(solution);
+
+    auto& [forces_data, new_state, tangent_matrix] = *solution;
+    ASSERT_TRUE(new_state != nullptr);
+
+    MSM::OGSMFrontThermodynamicForcesView<
+        3, boost::mpl::list<MSM::Stress, MSM::Saturation>>
+        view;

```

```

+ auto const stress = view.block(MSM::stress, forces_data);
+ auto const saturation = view.block(MSM::saturation, forces_data);
+
+ EXPECT_NEAR(saturation, saturated_anchor.saturation,
+   scaledTolerance(saturated_anchor.saturation));
+ EXPECT_NEAR(stress[0], saturated_anchor.stress_xx,
+   scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+ EXPECT_NEAR(stress[1], saturated_anchor.stress_xx,
+   scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+ EXPECT_NEAR(stress[2], saturated_anchor.stress_xx,
+   scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+ EXPECT_NEAR(stress[3], 0.0, 1e-12);
+ EXPECT_NEAR(stress[4], 0.0, 1e-12);
+ EXPECT_NEAR(stress[5], 0.0, 1e-12);
+
+ ASSERT_FALSE(tangent_matrix.data.empty());
+ EXPECT_TRUE(std::all_of(
+   tangent_matrix.data.begin(), tangent_matrix.data.end(),
+   [](double const v) { return std::isfinite(v); }));
+)
+
+*TEST(MaterialLib_RichardsMechanicsDSMIMicroMacroBridgeMFront,
+  FactoryPathOneStepPressureResponse)
+{
+  auto const baseline_rows = loadBaselineRows(
+    TestInfoLib::TestInfo::data_path +
+    "/MaterialLib/MFront/RichardsMechanicsDSMIMicroMacroBridge_overlap_transfer_baseline.csv");
+  ASSERT_EQ(baseline_rows.size(), 5);
+  auto const& saturated_anchor = baseline_rows.front();
+
+  auto const parameters = createParameters();
+  auto model = createBridgeModelThroughFactory(parameters);
+  ASSERT_TRUE(model != nullptr);
+
+  auto state = model->createMaterialStateVariables();
+  ASSERT_TRUE(state != nullptr);
+  initializeState(*model, *state);
+
+  MPL::VariableArray variable_array_prev;
+  variable_array_prev.stress.template emplace<KV>(KV::Zero());
+  variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+  variable_array_prev.liquid_phase_pressure = 0.0;
+  variable_array_prev.liquid_saturation = 1.0;
+  variable_array_prev.temperature = 0.0;
+
+  MPL::VariableArray variable_array = variable_array_prev;
+  variable_array.liquid_phase_pressure = 1000.0;
+
+  ParameterLib::SpatialPosition x{};
+  auto solution = model->integrateStress(variable_array_prev,
+    variable_array,
+    1.0,
+    x,
+    1.0,
+    *state);
+  ASSERT_TRUE(solution);
+
+  auto& [stress, new_state, tangent_matrix] = *solution;
+  ASSERT_TRUE(new_state != nullptr);
+
+  EXPECT_NEAR(stress[0], saturated_anchor.stress_xx,
+    scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+  EXPECT_NEAR(stress[1], saturated_anchor.stress_xx,
+    scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+  EXPECT_NEAR(stress[2], saturated_anchor.stress_xx,
+    scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+  EXPECT_NEAR(stress[3], 0.0, 1e-12);
+  EXPECT_NEAR(stress[4], 0.0, 1e-12);
+  EXPECT_NEAR(stress[5], 0.0, 1e-12);
+  EXPECT_TRUE(tangent_matrix.allFinite());
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMIMicroMacroBridgeMFront,
+  FactoryPathPressureCoupledBlocks)
+{
+  auto const baseline_rows = loadBaselineRows(
+    TestInfoLib::TestInfo::data_path +
+    "/MaterialLib/MFront/RichardsMechanicsDSMIMicroMacroBridge_overlap_transfer_baseline.csv");
+  ASSERT_EQ(baseline_rows.size(), 5);
+  auto const& saturated_anchor = baseline_rows.front();
+
+  auto const parameters = createParameters();
+  auto model = createBridgeModelThroughFactory(parameters);
+  ASSERT_TRUE(model != nullptr);
+
+  auto state = model->createMaterialStateVariables();
+  ASSERT_TRUE(state != nullptr);
+  initializeState(*model, *state);
+
+  MPL::VariableArray variable_array_prev;
+  variable_array_prev.stress.template emplace<KV>(KV::Zero());
+  variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+  variable_array_prev.liquid_phase_pressure = 0.0;
+  variable_array_prev.liquid_saturation = 1.0;
+  variable_array_prev.temperature = 0.0;
+
+  MPL::VariableArray variable_array = variable_array_prev;
+  variable_array.liquid_phase_pressure = 1000.0;
+
+  ParameterLib::SpatialPosition x{};
+  auto response = model->integrateStressPressureCoupled(
+    variable_array_prev, variable_array, 1.0, x, 1.0, *state);
+  ASSERT_TRUE(response);
+  ASSERT_TRUE(response->state != nullptr);
+
+  EXPECT_NEAR(response->saturation, saturated_anchor.saturation,
+    scaledTolerance(saturated_anchor.saturation));
+  EXPECT_NEAR(response->stress[0], saturated_anchor.stress_xx,
+    scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+  EXPECT_NEAR(response->stress[1], saturated_anchor.stress_xx,
+    scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+  EXPECT_NEAR(response->stress[2], saturated_anchor.stress_xx,
+    scaledTolerance(saturated_anchor.stress_xx, 5e-3, 1e-6));
+  EXPECT_NEAR(response->stress[3], 0.0, 1e-12);
+  EXPECT_NEAR(response->stress[4], 0.0, 1e-12);
+  EXPECT_NEAR(response->stress[5], 0.0, 1e-12);
+  EXPECT_NEAR(response->dStress_dLiquidPressure[0], 0.0, 1e-12);

```

```

+ EXPECT_NEAR(response->dStress_dLiquidPressure[1], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[2], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[3], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[4], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[5], 0.0, 1e-12);
+ EXPECT_NEAR(response->dSaturation_dLiquidPressure, 0.0, 1e-12);
+ EXPECT_TRUE(response->dStress_dStrain.allFinite());
+ for (int i = 0; i < response->dSaturation_dStrain.size(); ++i)
+ {
+     EXPECT_NEAR(response->dSaturation_dStrain[i], 0.0, 1e-12);
+ }
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMIMicroMacroBridgeMFront,
+     PressureHistoryResponse)
+{
+    auto const baseline_rows = loadBaselineRows(
+        TestInfoLib::TestInfo::data_path +
+        "/MaterialLib/MFront/RichardsMechanicsDSMIMicroMacroBridge_overlap_transfer_baseline.csv");
+    ASSERT_EQ(baseline_rows.size(), 5);
+
+    auto const parameters = createParameters();
+    auto model = createBridgeModel(parameters);
+    ASSERT_TRUE(model != nullptr);
+
+    auto state = model->createMaterialStateVariables();
+    ASSERT_TRUE(state != nullptr);
+    initializeState(*model, *state);
+
+    MPL::VariableArray variable_array_prev;
+    variable_array_prev.stress.template emplace<KV>(KV::Zero());
+    variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+    variable_array_prev.liquid_phase_pressure = 0.0;
+    variable_array_prev.liquid_saturation = 1.0;
+    variable_array_prev.temperature = 0.0;
+
+    ParameterLib::SpatialPosition x{};
+
+    MSM::OGSMFrontThermodynamicForcesView<
+        3, boost::mpl::list<MSM::Stress, MSM::Saturation>>
+        view;
+
+    for (auto const& row : baseline_rows)
+    {
+        auto variable_array = variable_array_prev;
+        variable_array.liquid_phase_pressure = row.pressure;
+
+        auto solution = model->integrateStress(variable_array_prev,
+            variable_array,
+            static_cast<double>(row.step + 1),
+            x,
+            1.0,
+            *state);
+
+        ASSERT_TRUE(solution);
+
+        auto& [forces_data, new_state, tangent_matrix] = *solution;
+        ASSERT_TRUE(new_state != nullptr);
+
+        auto const stress = view.block(MSM::stress, forces_data);
+        auto const saturation = view.block(MSM::saturation, forces_data);
+
+        EXPECT_NEAR(saturation, row.saturation, scaledTolerance(row.saturation));
+        EXPECT_NEAR(stress[0], row.stress_xx, scaledTolerance(row.stress_xx, 5e-3, 1e-6));
+        EXPECT_NEAR(stress[1], row.stress_xx, scaledTolerance(row.stress_xx, 5e-3, 1e-6));
+        EXPECT_NEAR(stress[2], row.stress_xx, scaledTolerance(row.stress_xx, 5e-3, 1e-6));
+        EXPECT_NEAR(stress[3], 0.0, 1e-12);
+        EXPECT_NEAR(stress[4], 0.0, 1e-12);
+        EXPECT_NEAR(stress[5], 0.0, 1e-12);
+
+        ASSERT_FALSE(tangent_matrix.data.empty());
+        EXPECT_TRUE(std::all_of(
+            tangent_matrix.data.begin(), tangent_matrix.data.end(),
+            [](double const v) { return std::isfinite(v); }));
+
+        state = std::move(new_state);
+        state->pushBackState();
+        variable_array_prev = variable_array;
+        variable_array_prev.stress.template emplace<KV>(stress);
+        variable_array_prev.liquid_saturation = saturation;
+    }
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMIMicroMacroBridgeMFront,
+     MicrostateHistoryResponse)
+{
+    auto const baseline_rows = loadBaselineRows(
+        TestInfoLib::TestInfo::data_path +
+        "/MaterialLib/MFront/RichardsMechanicsDSMIMicroMacroBridge_overlap_transfer_baseline.csv");
+    ASSERT_EQ(baseline_rows.size(), 5);
+
+    auto const parameters = createParameters();
+    auto model = createBridgeModelThroughFactory(parameters);
+    ASSERT_TRUE(model != nullptr);
+
+    auto state = model->createMaterialStateVariables();
+    ASSERT_TRUE(state != nullptr);
+    initializeState(*model, *state);
+
+    EXPECT_NEAR(getInternalVariable(*model, *state, "n_l"), 0.1, 1e-12);
+    EXPECT_NEAR(getInternalVariable(*model, *state, "rho_LR"), 1300.0, 1e-12);
+    EXPECT_NEAR(getInternalVariable(*model, *state, "epsilon_sv"), 0.0, 1e-12);
+
+    MPL::VariableArray variable_array_prev;
+    variable_array_prev.stress.template emplace<KV>(KV::Zero());
+    variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+    variable_array_prev.liquid_phase_pressure = 0.0;
+    variable_array_prev.liquid_saturation = 1.0;
+    variable_array_prev.temperature = 0.0;
+
+    ParameterLib::SpatialPosition x{};
+
+    for (auto const& row : baseline_rows)
+    {
+        auto variable_array = variable_array_prev;
+        variable_array.liquid_phase_pressure = row.pressure;
+
+        auto response = model->integrateStressPressureCoupled(

```

```

+         variable_array_prev,
+         variable_array,
+         static_cast<double>(row.step + 1),
+         x,
+         1.0,
+         *state);
+ ASSERT_TRUE(response);
+ ASSERT_TRUE(response->state != nullptr);
+
+     auto const n_l_value = getInternalVariable(*model, *response->state, "n_l");
+     auto const phi_m_value = getInternalVariable(*model, *response->state, "phi_m");
+     auto const phi_M_value = getInternalVariable(*model, *response->state, "phi_M");
+     auto const rho_LR_value = getInternalVariable(*model, *response->state, "rho_LR");
+     auto const mu_LR_value = getInternalVariable(*model, *response->state, "mu_LR");
+     auto const rho_L_hat_value = getInternalVariable(*model, *response->state, "rho_L_hat");
+     auto const epsilon_sw_value =
+         getInternalVariable(*model, *response->state, "epsilon_sw");
+
+     EXPECT_NEAR(response->saturation, row.saturation, scaledTolerance(row.saturation));
+     EXPECT_NEAR(n_l_value, row.n_l, scaledTolerance(row.n_l, 5e-3, 1e-8));
+     EXPECT_NEAR(phi_m_value, row.phi_m, scaledTolerance(row.phi_m, 5e-3, 1e-8));
+     EXPECT_NEAR(phi_M_value, row.phi_M, scaledTolerance(row.phi_M, 5e-3, 1e-8));
+     EXPECT_NEAR(rho_LR_value, row.rho_LR, scaledTolerance(row.rho_LR, 1e-3, 1e-6));
+     EXPECT_NEAR(mu_LR_value, row.mu_LR, scaledTolerance(row.mu_LR, 1e-2, 1e-8));
+     EXPECT_NEAR(rho_L_hat_value, row.rho_L_hat, scaledTolerance(row.rho_L_hat, 1e-2, 1e-8));
+     EXPECT_NEAR(epsilon_sw_value, row.epsilon_sw, scaledTolerance(row.epsilon_sw, 5e-3, 1e-8));
+     EXPECT_NEAR(phi_m_value, n_l_value, scaledTolerance(row.n_l, 5e-3, 1e-8));
+
+     state = std::move(response->state);
+     state->pushBackState();
+     variable_array_prev = variable_array;
+     variable_array_prev.stress.template emplace<KV>(response->stress);
+     variable_array_prev.liquid_saturation = response->saturation;
+ }
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFfront,
+     StrainCoupledOverlapHistoryResponse)
+{
+     auto const overlap_rows = loadBaselineRows(
+         TestInfoLib::TestInfo::data_path +
+         "/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv");
+     auto const strain_rows = loadBaselineRows(
+         TestInfoLib::TestInfo::data_path +
+         "/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_strain_coupled_overlap_baseline.csv");
+     ASSERT_EQ(overlap_rows.size(), 5);
+     ASSERT_EQ(strain_rows.size(), 5);
+     auto const& anchor = overlap_rows.back();
+
+     auto const parameters = createParameters();
+     auto model = createBridgeModelThroughFactory(parameters);
+     ASSERT_TRUE(model != nullptr);
+
+     auto state = model->createMaterialStateVariables();
+     ASSERT_TRUE(state != nullptr);
+     initializeState(*model, *state);
+     setInternalVariable(*model, *state, "n_l", anchor.n_l);
+     setInternalVariable(*model, *state, "rho_LR", anchor.rho_LR);
+     setInternalVariable(*model, *state, "epsilon_sw", anchor.epsilon_sw);
+
+     MPL::VariableArray variable_array_prev;
+     KV anchor_stress = KV::Zero();
+     anchor_stress[0] = anchor.stress_xx;
+     anchor_stress[1] = anchor.stress_xx;
+     anchor_stress[2] = anchor.stress_xx;
+     setThermodynamicForces(*state, anchor_stress, anchor.saturation);
+     state->pushBackState();
+
+     variable_array_prev.stress.template emplace<KV>(anchor_stress);
+     variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+     variable_array_prev.liquid_phase_pressure = anchor.pressure;
+     variable_array_prev.liquid_saturation = anchor.saturation;
+     variable_array_prev.temperature = 0.0;
+
+     ParameterLib::SpatialPosition x();
+
+     auto const& expr strain_stress_relative_tolerance = 7e-3;
+
+     for (auto const& row : strain_rows)
+     {
+         auto variable_array = variable_array_prev;
+         variable_array.liquid_phase_pressure = row.pressure;
+         variable_array.mechanical_strain.template emplace<KV>({
+             isotropicStrainFromVolumetric(row.epsilon_v_tot)});
+
+         auto response = model->integrateStressPressureCoupled(
+             variable_array_prev,
+             variable_array,
+             static_cast<double>(row.step + 1),
+             x,
+             1.0,
+             *state);
+         ASSERT_TRUE(response);
+         ASSERT_TRUE(response->state != nullptr);
+
+         auto const n_l_value = getInternalVariable(*model, *response->state, "n_l");
+         auto const phi_m_value = getInternalVariable(*model, *response->state, "phi_m");
+         auto const phi_M_value = getInternalVariable(*model, *response->state, "phi_M");
+         auto const rho_LR_value = getInternalVariable(*model, *response->state, "rho_LR");
+         auto const mu_LR_value = getInternalVariable(*model, *response->state, "mu_LR");
+         auto const rho_L_hat_value = getInternalVariable(*model, *response->state, "rho_L_hat");
+         auto const epsilon_sw_value =
+             getInternalVariable(*model, *response->state, "epsilon_sw");
+
+         EXPECT_NEAR(response->saturation, row.saturation, scaledTolerance(row.saturation));
+         EXPECT_NEAR(response->stress[0], row.stress_xx, scaledTolerance(row.stress_xx, strain_stress_relative_tolerance, 1e-6));
+         EXPECT_NEAR(response->stress[1], row.stress_xx, scaledTolerance(row.stress_xx, strain_stress_relative_tolerance, 1e-6));
+         EXPECT_NEAR(response->stress[2], row.stress_xx, scaledTolerance(row.stress_xx, strain_stress_relative_tolerance, 1e-6));
+         EXPECT_NEAR(response->stress[3], 0.0, 1e-12);
+         EXPECT_NEAR(response->stress[4], 0.0, 1e-12);
+         EXPECT_NEAR(response->stress[5], 0.0, 1e-12);
+         EXPECT_NEAR(response->dSaturation_dLiquidPressure, 0.0, 1e-12);
+         EXPECT_NEAR(n_l_value, row.n_l, scaledTolerance(row.n_l, 5e-3, 1e-8));
+         EXPECT_NEAR(phi_m_value, row.phi_m, scaledTolerance(row.phi_m, 5e-3, 1e-8));
+         EXPECT_NEAR(phi_M_value, row.phi_M, scaledTolerance(row.phi_M, 5e-3, 1e-8));
+         EXPECT_NEAR(rho_LR_value, row.rho_LR, scaledTolerance(row.rho_LR, 1e-3, 1e-6));
+         EXPECT_NEAR(mu_LR_value, row.mu_LR, scaledTolerance(row.mu_LR, 1e-2, 1e-8));

```

```

+ EXPECT_NEAR(rho_l_hat_value, row.rho_l_hat, scaledTolerance(row.rho_l_hat, 1e-2, 1e-8));
+ EXPECT_NEAR(epsilon_sw_value, row.epsilon_sw, scaledTolerance(row.epsilon_sw, 5e-3, 1e-8));
+ EXPECT_NEAR(phi_m_value, n_l_value, scaledTolerance(row.n_l, 5e-3, 1e-8));
+ EXPECT_TRUE(response->dStress_dStrain.allFinite());
+
+ state = std::move(response->state);
+ state->pushBackState();
+ variable_array_prev = variable_array;
+ variable_array_prev.stress.template emplace<KV>(response->stress);
+ variable_array_prev.liquid_saturation = response->saturation;
+ }
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFfront,
+ DMMMicroMacroOverlapTransferBaselineHistory)
+{
+ auto const baseline_rows = loadBaselineRows(
+ TestInfoLib::TestInfo::data_path +
+ "/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv");
+ ASSERT_EQ(baseline_rows.size(), 5);
+
+ auto const parameters = createParameters();
+ auto model = createBridgeModelThroughFactory(parameters);
+ ASSERT_TRUE(model != nullptr);
+
+ auto state = model->createMaterialStateVariables();
+ ASSERT_TRUE(state != nullptr);
+ initializeState(*model, *state);
+
+ MPL::VariableArray variable_array_prev;
+ variable_array_prev.stress.template emplace<KV>(KV::Zero());
+ variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+ variable_array_prev.liquid_phase_pressure = 0.0;
+ variable_array_prev.liquid_saturation = 1.0;
+ variable_array_prev.temperature = 0.0;
+
+ ParameterLib::SpatialPosition x{};
+
+ for (auto const& row : baseline_rows)
+ {
+ auto variable_array = variable_array_prev;
+ variable_array.liquid_phase_pressure = row.pressure;
+
+ auto response = model->integrateStressPressureCoupled(
+ variable_array_prev,
+ variable_array,
+ static_cast<double>(row.step + 1),
+ x,
+ 1.0,
+ *state);
+ ASSERT_TRUE(response);
+ ASSERT_TRUE(response->state != nullptr);
+
+ auto const n_l_value = getInternalVariable(*model, *response->state, "n_l");
+ auto const phi_m_value = getInternalVariable(*model, *response->state, "phi_m");
+ auto const phi_M_value = getInternalVariable(*model, *response->state, "phi_M");
+ auto const rho_LR_value = getInternalVariable(*model, *response->state, "rho_LR");
+ auto const mu_LR_value = getInternalVariable(*model, *response->state, "mu_LR");
+ auto const rho_l_hat_value = getInternalVariable(*model, *response->state, "rho_l_hat");
+ auto const epsilon_sw_value =
+ getInternalVariable(*model, *response->state, "epsilon_sw");
+
+ EXPECT_NEAR(response->saturation, row.saturation, scaledTolerance(row.saturation));
+ EXPECT_NEAR(response->stress[0], row.stress_xx, scaledTolerance(row.stress_xx, 5e-3, 1e-6));
+ EXPECT_NEAR(response->stress[1], row.stress_xx, scaledTolerance(row.stress_xx, 5e-3, 1e-6));
+ EXPECT_NEAR(response->stress[2], row.stress_xx, scaledTolerance(row.stress_xx, 5e-3, 1e-6));
+ EXPECT_NEAR(response->stress[3], 0.0, 1e-12);
+ EXPECT_NEAR(response->stress[4], 0.0, 1e-12);
+ EXPECT_NEAR(response->stress[5], 0.0, 1e-12);
+ EXPECT_NEAR(response->dSaturation_dLiquidPressure, 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[0], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[1], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[2], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[3], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[4], 0.0, 1e-12);
+ EXPECT_NEAR(response->dStress_dLiquidPressure[5], 0.0, 1e-12);
+ EXPECT_NEAR(n_l_value, row.n_l, scaledTolerance(row.n_l, 5e-3, 1e-8));
+ EXPECT_NEAR(phi_m_value, row.phi_m, scaledTolerance(row.phi_m, 5e-3, 1e-8));
+ EXPECT_NEAR(phi_M_value, row.phi_M, scaledTolerance(row.phi_M, 5e-3, 1e-8));
+ EXPECT_NEAR(rho_LR_value, row.rho_LR, scaledTolerance(row.rho_LR, 1e-3, 1e-6));
+ EXPECT_NEAR(mu_LR_value, row.mu_LR, scaledTolerance(row.mu_LR, 1e-2, 1e-8));
+ EXPECT_NEAR(rho_l_hat_value, row.rho_l_hat, scaledTolerance(row.rho_l_hat, 1e-2, 1e-8));
+ EXPECT_NEAR(epsilon_sw_value, row.epsilon_sw, scaledTolerance(row.epsilon_sw, 5e-3, 1e-8));
+
+ state = std::move(response->state);
+ state->pushBackState();
+ variable_array_prev = variable_array;
+ variable_array_prev.stress.template emplace<KV>(response->stress);
+ variable_array_prev.liquid_saturation = response->saturation;
+ }
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFfront,
+ DMMMicroMacroOverlapCarrierHistory)
+{
+ auto const baseline_rows = loadBaselineRows(
+ TestInfoLib::TestInfo::data_path +
+ "/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv");
+ ASSERT_EQ(baseline_rows.size(), 5);
+
+ auto const parameters = createParameters();
+ auto model = createBridgeModelThroughFactory(parameters);
+ ASSERT_TRUE(model != nullptr);
+
+ auto state = model->createMaterialStateVariables();
+ ASSERT_TRUE(state != nullptr);
+ initializeState(*model, *state);
+
+ MPL::VariableArray variable_array_prev;
+ variable_array_prev.stress.template emplace<KV>(KV::Zero());
+ variable_array_prev.mechanical_strain.template emplace<KV>(KV::Zero());
+ variable_array_prev.liquid_phase_pressure = 0.0;
+ variable_array_prev.liquid_saturation = 1.0;
+ variable_array_prev.temperature = 0.0;
+
+ ParameterLib::SpatialPosition x{};

```



```

+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 0.0, x, 0.0, *state);
+   ASSERT_TRUE(response);
+   ASSERT_TRUE(response->state != nullptr);
+
+   EXPECT_NEAR(response->saturation, 1.0, 1e-12);
+   EXPECT_TRUE(response->stress.allFinite());
+   EXPECT_NEAR(response->stress[0], response->stress[1], 1e-9);
+   EXPECT_NEAR(response->stress[1], response->stress[2], 1e-9);
+   EXPECT_NEAR(response->stress[3], 0.0, 1e-12);
+   EXPECT_NEAR(response->dSaturation_dLiquidPressure, 0.0, 1e-12);
+   EXPECT_TRUE(response->dStress_dStrain.allFinite());
+   EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+   EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+}
+
+
+*TEST(MaterialLib_RichardsMechanicsDSMMacroBridgeMFront,
+   PlaneStrainFactoryPathZeroDtBenchmarkParameterResponse)
+{
+   auto const parameters = createBenchmarkParameters();
+   auto model = createBridgeModelThroughFactoryPlaneStrain(parameters);
+   ASSERT_TRUE(model != nullptr);
+
+   auto state = model->createMaterialStateVariables();
+   ASSERT_TRUE(state != nullptr);
+   initializeState(*model, *state);
+
+   MPL::VariableArray variable_array_prev;
+   variable_array_prev.stress.template emplace<KV2>(KV2::Zero());
+   variable_array_prev.mechanical_strain.template emplace<KV2>(KV2::Zero());
+   variable_array_prev.liquid_phase_pressure = 0.0;
+   variable_array_prev.liquid_saturation = 1.0;
+   variable_array_prev.temperature = 293.15;
+
+   MPL::VariableArray variable_array = variable_array_prev;
+
+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 0.0, x, 0.0, *state);
+   ASSERT_TRUE(response);
+   ASSERT_TRUE(response->state != nullptr);
+
+   EXPECT_NEAR(response->saturation, 1.0, 1e-12);
+   EXPECT_TRUE(response->stress.allFinite());
+   EXPECT_TRUE(response->dStress_dStrain.allFinite());
+   EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+   EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMMacroBridgeMFront,
+   PlaneStrainFactoryPathZeroDtBenchmarkPressureConsistentNegativePressureResponse)
+{
+   auto const parameters = createBenchmarkPressureConsistentParameters();
+   auto model = createBridgeModelThroughFactoryPlaneStrain(parameters);
+   ASSERT_TRUE(model != nullptr);
+
+   auto state = model->createMaterialStateVariables();
+   ASSERT_TRUE(state != nullptr);
+   initializeState(*model, *state);
+
+   MPL::VariableArray variable_array_prev;
+   KV2 sigma0 = KV2::Zero();
+   sigma0[0] = -5e3;
+   sigma0[1] = -5e3;
+   sigma0[2] = -5e3;
+   variable_array_prev.stress.template emplace<KV2>(sigma0);
+   variable_array_prev.mechanical_strain.template emplace<KV2>(KV2::Zero());
+   variable_array_prev.liquid_phase_pressure = -5e3;
+   variable_array_prev.liquid_saturation = 1.0;
+   variable_array_prev.temperature = 293.15;
+
+   MPL::VariableArray variable_array = variable_array_prev;
+
+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 0.0, x, 0.0, *state);
+   ASSERT_TRUE(response);
+   ASSERT_TRUE(response->state != nullptr);
+
+   EXPECT_TRUE(response->stress.allFinite());
+   EXPECT_TRUE(response->dStress_dStrain.allFinite());
+   EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+   EXPECT_TRUE(std::isfinite(response->saturation));
+   EXPECT_TRUE(std::isfinite(response->dSaturation_dLiquidPressure));
+   EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+}
+
+*TEST(MaterialLib_RichardsMechanicsDSMMacroBridgeMFront,
+   PlaneStrainFactoryPathZeroDtBenchmarkPressureConsistentRMeffectiveStressResponse)
+{
+   auto const parameters = createBenchmarkPressureConsistentParameters();
+   auto model = createBridgeModelThroughFactoryPlaneStrain(parameters);
+   ASSERT_TRUE(model != nullptr);
+
+   auto state = model->createMaterialStateVariables();
+   ASSERT_TRUE(state != nullptr);
+   initializeState(*model, *state);
+
+   MPL::VariableArray variable_array_prev;
+   KV2 sigma_eff0 = KV2::Zero();
+   sigma_eff0[0] = -1e4;
+   sigma_eff0[1] = -1e4;
+   sigma_eff0[2] = -1e4;
+   variable_array_prev.stress.template emplace<KV2>(sigma_eff0);
+   variable_array_prev.mechanical_strain.template emplace<KV2>(KV2::Zero());
+   variable_array_prev.liquid_phase_pressure = -5e3;
+   variable_array_prev.liquid_saturation = 1.0;
+   variable_array_prev.temperature = 293.15;
+
+   MPL::VariableArray variable_array = variable_array_prev;
+
+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 0.0, x, 0.0, *state);
+   ASSERT_TRUE(response);

```

```

+ ASSERT_TRUE(response->state != nullptr);
+
+ EXPECT_TRUE(response->stress.allFinite());
+ EXPECT_TRUE(response->dStress_dStrain.allFinite());
+ EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+ EXPECT_TRUE(std::isfinite(response->saturation));
+ EXPECT_TRUE(std::isfinite(response->dSaturation_dLiquidPressure));
+ EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+)
+
+TEST(MaterialLib_RichardsMechanicsDSMMacroBridgeMFront,
+   PlaneStrainFactoryPathZeroDtBenchmarkPressureConsistentExactRMSStateResponse)
+{
+   auto const parameters = createBenchmarkPressureConsistentParameters();
+   auto model = createBridgeModelThroughFactoryPlaneStrain(parameters);
+   ASSERT_TRUE(model != nullptr);
+
+   auto state = model->createMaterialStateVariables();
+   ASSERT_TRUE(state != nullptr);
+   initializeState(*model, *state);
+
+   MPL::VariableArray variable_array_prev;
+   KV2 sigma_eff0 = KV2::Zero();
+   sigma_eff0[0] = -9481.282375677227;
+   sigma_eff0[1] = -9481.282375677227;
+   sigma_eff0[2] = -9481.282375677227;
+   variable_array_prev.stress.template emplace<KV2>(sigma_eff0);
+   variable_array_prev.mechanical_strain.template emplace<KV2>(KV2::Zero());
+   variable_array_prev.liquid_phase_pressure = -5e3;
+   variable_array_prev.liquid_saturation = 0.8962564751354456;
+   variable_array_prev.temperature = 293.15;
+
+   MPL::VariableArray variable_array = variable_array_prev;
+
+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 0.0, x, 0.0, *state);
+   ASSERT_TRUE(response);
+   ASSERT_TRUE(response->state != nullptr);
+
+   EXPECT_TRUE(response->stress.allFinite());
+   EXPECT_TRUE(response->dStress_dStrain.allFinite());
+   EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+   EXPECT_TRUE(std::isfinite(response->saturation));
+   EXPECT_TRUE(std::isfinite(response->dSaturation_dLiquidPressure));
+   EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+)
+
+TEST(MaterialLib_RichardsMechanicsDSMMacroBridgeMFront,
+   PlaneStrainFactoryPathZeroDtElasticNegativePressureSaturationResponse)
+{
+   auto parameters = createBenchmarkParameters();
+   overwriteParameter(parameters, "swelling_slope", 0.0);
+   overwriteParameter(parameters, "mass_exchange_coefficient", 0.0);
+
+   auto model = createBridgeModelThroughFactoryPlaneStrain(parameters);
+   ASSERT_TRUE(model != nullptr);
+
+   auto state = model->createMaterialStateVariables();
+   ASSERT_TRUE(state != nullptr);
+   initializeState(*model, *state);
+
+   MPL::VariableArray variable_array_prev;
+   variable_array_prev.stress.template emplace<KV2>(KV2::Zero());
+   variable_array_prev.mechanical_strain.template emplace<KV2>(KV2::Zero());
+   variable_array_prev.liquid_phase_pressure = -2.5e5;
+   variable_array_prev.liquid_saturation = 1.0;
+   variable_array_prev.temperature = 293.15;
+
+   MPL::VariableArray variable_array = variable_array_prev;
+
+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 0.0, x, 0.0, *state);
+   ASSERT_TRUE(response);
+   ASSERT_TRUE(response->state != nullptr);
+
+   auto constexpr area_factor_tuller = 1.0;
+   auto constexpr pore_area_shape_factor_tuller = 0.8584073464102069;
+   auto constexpr characteristic_pore_size = 1e-5;
+   auto constexpr surface_tension = 0.0715;
+   auto constexpr liquid_pressure = -2.5e5;
+   auto const capillary_prefactor =
+       4.0 * pore_area_shape_factor_tuller * surface_tension *
+       surface_tension /
+       (area_factor_tuller * characteristic_pore_size *
+        characteristic_pore_size);
+   auto const expected_saturation =
+       1.0 -
+       std::exp(-capillary_prefactor /
+               (liquid_pressure * liquid_pressure));
+
+   EXPECT_NEAR(response->saturation,
+               expected_saturation,
+               scaledTolerance(expected_saturation, 1e-12, 1e-15));
+   EXPECT_LT(response->saturation, 1e-2);
+   EXPECT_TRUE(std::isfinite(response->dSaturation_dLiquidPressure));
+)
+
+TEST(MaterialLib_RichardsMechanicsDSMMacroBridgeMFront,
+   PlaneStrainFactoryPathBenchmarkPressureConsistentExactRMSStateFirstStepResponse)
+{
+   auto const parameters = createBenchmarkPressureConsistentParameters();
+   auto model = createBridgeModelThroughFactoryPlaneStrain(parameters);
+   ASSERT_TRUE(model != nullptr);
+
+   auto state = model->createMaterialStateVariables();
+   ASSERT_TRUE(state != nullptr);
+   initializeState(*model, *state);
+
+   MPL::VariableArray variable_array_prev;
+   KV2 sigma_eff0 = KV2::Zero();
+   sigma_eff0[0] = -9481.282375677227;
+   sigma_eff0[1] = -9481.282375677227;
+   sigma_eff0[2] = -9481.282375677227;
+   variable_array_prev.stress.template emplace<KV2>(sigma_eff0);
+   variable_array_prev.mechanical_strain.template emplace<KV2>(KV2::Zero());

```

```

+   variable_array_prev.liquid_phase_pressure = -5e3;
+   variable_array_prev.liquid_saturation = 0.8962564751354456;
+   variable_array_prev.temperature = 293.15;
+
+   MPL::VariableArray variable_array = variable_array_prev;
+
+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 1000.0, x, 1000.0, *state);
+   ASSERT_TRUE(response);
+   ASSERT_TRUE(response->state != nullptr);
+
+   EXPECT_TRUE(response->stress.allFinite());
+   EXPECT_TRUE(response->dStress_dStrain.allFinite());
+   EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+   EXPECT_TRUE(std::isfinite(response->saturation));
+   EXPECT_TRUE(std::isfinite(response->dSaturation_dLiquidPressure));
+   EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+}
+
+TEST(MaterialLib_RichardsMechanicsDSMMicroMacroBridgeMFront,
+     PlaneStrainFactoryPathBenchmarkPressureConsistentProcessFailureStateResponse)
+{
+   auto const parameters = createBenchmarkPressureConsistentParameters();
+   auto model = createBridgeModelThroughFactoryPlaneStrain(parameters);
+   ASSERT_TRUE(model != nullptr);
+
+   auto state = model->createMaterialStateVariables();
+   ASSERT_TRUE(state != nullptr);
+   initializeState(*model, *state);
+
+   setInternalVariable(*model, *state, "n_l", 0.012069019712402708);
+   setInternalVariable(*model, *state, "rho_LR", 2267.4495975433856);
+   setInternalVariable(*model, *state, "epsilon_sw", 0.0);
+
+   MPL::VariableArray variable_array_prev;
+   KV2 sigma_eff = KV2::Zero();
+   sigma_eff[0] = -48189.3;
+   sigma_eff[1] = -105123.0;
+   sigma_eff[2] = -48189.3;
+   sigma_eff[3] = 3.51498e-09;
+   variable_array_prev.stress.template emplace<KV2>(sigma_eff);
+
+   KV2 eps_m = KV2::Zero();
+   eps_m[0] = 0.0;
+   eps_m[1] = -0.00310187;
+   eps_m[2] = 0.0;
+   eps_m[3] = 2.39782e-13;
+   variable_array_prev.mechanical_strain.template emplace<KV2>(eps_m);
+   variable_array_prev.liquid_phase_pressure = -411262.7142312391;
+   variable_array_prev.liquid_saturation = 0.0838622053846748;
+   variable_array_prev.temperature = 293.15;
+
+   MPL::VariableArray variable_array = variable_array_prev;
+
+   ParameterLib::SpatialPosition x{};
+   auto response = model->integrateStressPressureCoupled(
+       variable_array_prev, variable_array, 1000.0, x, 1000.0, *state);
+   ASSERT_TRUE(response);
+   ASSERT_TRUE(response->state != nullptr);
+
+   EXPECT_TRUE(response->stress.allFinite());
+   EXPECT_TRUE(response->dStress_dStrain.allFinite());
+   EXPECT_TRUE(response->dStress_dLiquidPressure.allFinite());
+   EXPECT_TRUE(std::isfinite(response->saturation));
+   EXPECT_TRUE(std::isfinite(response->dSaturation_dLiquidPressure));
+   EXPECT_TRUE(response->dSaturation_dStrain.allFinite());
+}
+
+#endif // OGS_USE_MFRONT
diff --git a/Tests/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridgeMCC.cpp b/Tests/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridgeMCC.cpp
new file mode 100644
index 0000000000..61592fb64b
--- /dev/null
+++ b/Tests/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridgeMCC.cpp
@@ -0,0 +1,1335 @@
+/**
+ * \file
+ * \copyright
+ * * Copyright (c) 2012-2025, OpenGeoSys Community (http://www.opengeosys.org)
+ * * Distributed under a Modified BSD License.
+ * * See accompanying file LICENSE.txt or
+ * * http://www.opengeosys.org/project/license
+ */
+
+#ifdef OGS_USE_MFRONT
+
+#include <algorithm>
+#include <cmath>
+#include <fstream>
+#include <limits>
+#include <memory>
+#include <optional>
+#include <sstream>
+#include <string>
+#include <unordered_map>
+#include <vector>
+
+#include <gtest/gtest.h>
+
+#include "BaseLib/ConfigTree.h"
+#include "InfoLib/TestInfo.h"
+#include "MaterialLib/SolidModels/CreateConstitutiveRelation.h"
+#include "MaterialLib/SolidModels/MFront/MFrontGeneric.h"
+#include "ParameterLib/ConstantParameter.h"
+#include "Tests/TestTools.h"
+
+namespace MPL = MaterialPropertyLib;
+namespace MSM = MaterialLib::Solids::MFront;
+using KV = MathLib::KelvinVector::KelvinVectorType<3>;
+using MB = MaterialLib::Solids::MechanicsBase<3>;
+
+namespace
+{
+   std::vector<std::unique_ptr<ParameterLib::ParameterBase>>
+   createDSMMicroMacroMCCParameters(double const swelling_slope = 0.0,
+       double const mass_exchange_coefficient = 0.0,

```

```

+         double const n_l0 = 0.1,
+         double const rho_lR0 = 1300.0,
+         double const epsilon_sw0 = 0.0,
+         double const macro_saturation_law_mode = 0.0,
+         double const microstate_local_solve_mode = 0.0,
+         double const micro_potential_convention = 0.0,
+         double const hamaker_constant = -6e-20,
+         double const macro_viscosity = 1000.0)
+{
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> parameters;
+
+    auto add_param = [&parameters](char const* name, double const value)
+    {
+        parameters.push_back(
+            std::make_unique<ParameterLib::ConstantParameter<double>>(name,
+                                                                    value));
+    };
+
+    add_param("YoungModulus", 52e6);
+    add_param("PoissonRatio", 0.3);
+    add_param("CriticalStateLineSlope", 1.2);
+    add_param("SwellingLineSlope", 6.6e-3);
+    add_param("VirginConsolidationLineSlope", 7.7e-2);
+    add_param("InitialPreConsolidationPressure", 2e5);
+    add_param("InitialVolumeRatio", 1.7857142857142858);
+    add_param("ResidualLiquidSaturation", 0.0);
+    add_param("ResidualGasSaturation", 0.0);
+    add_param("BubblePressure", 1e4);
+    add_param("VanGenuchtenExponent_m", 0.4);
+    add_param("MacroSaturationLawMode", macro_saturation_law_mode);
+    add_param("MicroStateLocalSolveMode", microstate_local_solve_mode);
+    add_param("MicroPotentialConvention", micro_potential_convention);
+
+    // Neutral first-step dsm_micromacro coupling: state is updated and visible, but
+    // the verified MCC stress/saturation surface stays unchanged.
+    add_param("SwellingSlope", swelling_slope);
+    add_param("MassExchangeCoefficient", mass_exchange_coefficient);
+    add_param("MacroViscosity", macro_viscosity);
+    add_param("ReferenceLiquidDensityMacro", 1000.0);
+    add_param("ReferenceLiquidDensityMicro", 1300.0);
+    add_param("ReferenceDensitySolid", 2470.0);
+    add_param("MicroLiquidDensityA", 1.3);
+    add_param("MicroLiquidDensityB", 1.0);
+    add_param("HamakerConstant", hamaker_constant);
+    add_param("SpecificSurface", 100.0);
+    add_param("AreaFactorTuller", 1.0);
+    add_param("PoreAreaShapeFactorTuller", 0.8584073464102069);
+    add_param("CharacteristicPoreSize", 1e-5);
+    add_param("SurfaceTension", 0.0715);
+    add_param("InitialPorosity", 0.432);
+    add_param("n_l0", n_l0);
+    add_param("rho_lR0", rho_lR0);
+    add_param("epsilon_sw0", epsilon_sw0);
+
+    return parameters;
+}
+
+std::vector<std::unique_ptr<ParameterLib::ParameterBase>>
+createDSMMicroMacroSupportParameters(double const macro_saturation_law_mode = 0.0,
+                                     double const microstate_local_solve_mode = 0.0,
+                                     double const micro_potential_convention = 0.0,
+                                     double const hamaker_constant = -6e-20)
+{
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> parameters;
+
+    auto add_param = [&parameters](char const* name, double const value)
+    {
+        parameters.push_back(
+            std::make_unique<ParameterLib::ConstantParameter<double>>(name,
+                                                                    value));
+    };
+
+    // Use the original dsm_micromacro support-state settings for the auxiliary
+    // surface, while keeping the MCC carrier mechanically stable.
+    add_param("YoungModulus", 1e10);
+    add_param("PoissonRatio", 0.25);
+    add_param("CriticalStateLineSlope", 1.2);
+    add_param("SwellingLineSlope", 6.6e-3);
+    add_param("VirginConsolidationLineSlope", 7.7e-2);
+    add_param("InitialPreConsolidationPressure", 1e9);
+    add_param("InitialVolumeRatio", 1.7857142857142858);
+    add_param("ResidualLiquidSaturation", 0.0);
+    add_param("ResidualGasSaturation", 0.0);
+    add_param("BubblePressure", 1e4);
+    add_param("VanGenuchtenExponent_m", 0.4);
+    add_param("MacroSaturationLawMode", macro_saturation_law_mode);
+    add_param("MicroStateLocalSolveMode", microstate_local_solve_mode);
+    add_param("MicroPotentialConvention", micro_potential_convention);
+
+    add_param("SwellingSlope", 0.1);
+    add_param("MassExchangeCoefficient", 1.0);
+    add_param("MacroViscosity", 1000.0);
+    add_param("ReferenceLiquidDensityMacro", 1000.0);
+    add_param("ReferenceLiquidDensityMicro", 1300.0);
+    add_param("ReferenceDensitySolid", 2470.0);
+    add_param("MicroLiquidDensityA", 1.3);
+    add_param("MicroLiquidDensityB", 1.0);
+    add_param("HamakerConstant", hamaker_constant);
+    add_param("SpecificSurface", 100.0);
+    add_param("AreaFactorTuller", 1.0);
+    add_param("PoreAreaShapeFactorTuller", 0.8584073464102069);
+    add_param("CharacteristicPoreSize", 1e-5);
+    add_param("SurfaceTension", 0.0715);
+    add_param("InitialPorosity", 0.2);
+    add_param("n_l0", 0.1);
+    add_param("rho_lR0", 1300.0);
+    add_param("epsilon_sw0", 0.0);
+
+    return parameters;
+}
+
+std::unique_ptr<MB> createModelFromXml(
+    std::vector<std::unique_ptr<ParameterLib::ParameterBase>> const& parameters,
+    char const* const xml)
+{
+    auto local_coordinate_system = std::nullopt;
+
+

```



```

+std::vector<double> getInternalVector(MB const& model,
+                                     MB::MaterialStateVariables const& state,
+                                     std::string const& name)
+{
+    auto const internal_variables = model.getInternalVariables();
+    auto const it = std::find_if(
+        internal_variables.begin(), internal_variables.end(),
+        [&name](auto const& internal_variable)
+        { return internal_variable.name == name; });
+
+    if (it == internal_variables.end())
+    {
+        return {};
+    }
+
+    std::vector<double> cache;
+    auto const& values = it->getter(state, cache);
+    return {values.begin(), values.end()};
+}
+
+void setInternalScalar(MB const& model,
+                      MB::MaterialStateVariables& state,
+                      std::string const& name,
+                      double const value)
+{
+    auto const internal_variables = model.getInternalVariables();
+    auto const it = std::find_if(
+        internal_variables.begin(), internal_variables.end(),
+        [&name](auto const& internal_variable)
+        { return internal_variable.name == name; });
+
+    ASSERT_TRUE(it != internal_variables.end()) << name;
+    auto values = it->reference(state);
+    ASSERT_EQ(values.size(), 1u);
+    values[0] = value;
+}
+
+void setDSMMicroMacroMCCThermodynamicForces(MB::MaterialStateVariables& state,
+                                             KV const& stress,
+                                             double const saturation)
+{
+    auto* const mfront_state =
+        dynamic_cast<MSM::MaterialStateVariablesMFront<3>>&(state);
+    ASSERT_TRUE(mfront_state != nullptr);
+
+    auto& thermodynamic_forces =
+        mfront_state->behaviour_data.sl.thermodynamic_forces;
+    thermodynamic_forces[0] = stress[0];
+    thermodynamic_forces[1] = stress[1];
+    thermodynamic_forces[2] = stress[2];
+    thermodynamic_forces[3] = stress[3];
+    thermodynamic_forces[4] = stress[4];
+    thermodynamic_forces[5] = stress[5];
+    thermodynamic_forces[6] = saturation;
+}
+
+KV dsm_micromacroMCCIIsotropicStress(double const value)
+{
+    KV sigma = KV::Zero();
+    sigma[0] = value;
+    sigma[1] = value;
+    sigma[2] = value;
+    return sigma;
+}
+
+KV dsm_micromacroMCCIIsotropicStrainFromVolumetric(double const epsilon_v)
+{
+    KV eps = KV::Zero();
+    eps[0] = epsilon_v / 3.0;
+    eps[1] = epsilon_v / 3.0;
+    eps[2] = epsilon_v / 3.0;
+    return eps;
+}
+
+double dsm_micromacroTullerSaturation(double const liquid_pressure)
+{
+    if (liquid_pressure >= -1e-12)
+    {
+        return 1.0;
+    }
+
+    constexpr double area_factor_tuller = 1.0;
+    constexpr double pore_area_shape_factor_tuller = 0.8584073464102069;
+    constexpr double characteristic_pore_size = 1e-5;
+    constexpr double surface_tension = 0.0715;
+
+    double const prefactor =
+        4.0 * pore_area_shape_factor_tuller * surface_tension * surface_tension /
+        (area_factor_tuller * characteristic_pore_size *
+         characteristic_pore_size);
+    return 1.0 - std::exp(-prefactor / (liquid_pressure * liquid_pressure));
+}
+
+double dsm_micromacroTullerDSaturationDLiquidPressure(double const liquid_pressure)
+{
+    if (liquid_pressure >= -1e-12)
+    {
+        return 0.0;
+    }
+
+    constexpr double area_factor_tuller = 1.0;
+    constexpr double pore_area_shape_factor_tuller = 0.8584073464102069;
+    constexpr double characteristic_pore_size = 1e-5;
+    constexpr double surface_tension = 0.0715;
+
+    double const prefactor =
+        4.0 * pore_area_shape_factor_tuller * surface_tension * surface_tension /
+        (area_factor_tuller * characteristic_pore_size *
+         characteristic_pore_size);
+    double const inv_p = 1.0 / liquid_pressure;
+    double const exp_term = std::exp(-prefactor * inv_p * inv_p);
+    return -2.0 * prefactor * exp_term * inv_p * inv_p * inv_p;
+}
+
+std::string dsm_micromacroMCCStripQuotes(std::string value)
+{
+    value.erase(0, value.find_first_not_of("\t\r\n"));
+}

```

```

+   value.erase(value.find_last_not_of(" \t\r\n") + 1);
+   return value;
+}
+
+std::vector<std::string> dsm_micromacroMCCSplitCommaLine(std::string const& line)
+{
+   std::stringstream ss(line);
+   std::string field;
+   std::vector<std::string> fields;
+   while (std::getline(ss, field, ','))
+   {
+       fields.push_back(dsm_micromacroMCCStripQuotes(field));
+   }
+   return fields;
+}
+
+struct DSMMicroMacroSupportBaselineRow
+{
+   int step = 0;
+   double pressure = 0.0;
+   double epsilon_v_total = 0.0;
+   double saturation = 0.0;
+   double n_l = 0.0;
+   double phi_m = 0.0;
+   double phi_M = 0.0;
+   double phi = 0.0;
+   double n_S = 0.0;
+   double n_L = 0.0;
+   double rho_LR = 0.0;
+   double rho_LR = 0.0;
+   double omega_l = 0.0;
+   double mu_LR = 0.0;
+   double rho_l_hat = 0.0;
+   double delta_epsilon_sw = 0.0;
+   double epsilon_sw = 0.0;
+   double sigma_S_xx = 0.0;
+};
+
+std::vector<DSMMicroMacroSupportBaselineRow> loadDSMMicroMacroSupportBaselineRows(
+   std::string const& filename)
+{
+   std::ifstream in(filename);
+   EXPECT_TRUE(in.good()) << filename;
+
+   std::string header_line;
+   std::getline(in, header_line);
+   auto const headers = dsm_micromacroMCCSplitCommaLine(header_line);
+
+   std::unordered_map<std::string, std::size_t> column;
+   for (std::size_t i = 0; i < headers.size(); ++i)
+   {
+       column[headers[i]] = i;
+   }
+
+   auto const get_value = [&](std::vector<std::string> const& fields,
+                               std::string const& key) -> double
+   { return std::stod(fields.at(column.at(key))); };
+
+   auto const get_optional_value = [&](std::vector<std::string> const& fields,
+                                       std::string const& key,
+                                       double const default_value) -> double
+   {
+       auto const it = column.find(key);
+       if (it == column.end())
+       {
+           return default_value;
+       }
+       return std::stod(fields.at(it->second));
+   };
+
+   std::vector<DSMMicroMacroSupportBaselineRow> rows;
+   std::string line;
+   while (std::getline(in, line))
+   {
+       if (line.empty())
+       {
+           continue;
+       }
+
+       auto const fields = dsm_micromacroMCCSplitCommaLine(line);
+       DSMMicroMacroSupportBaselineRow row;
+       row.step = static_cast<int>(get_value(fields, "step"));
+       row.pressure = get_value(fields, "pressure");
+       row.epsilon_v_total =
+           get_optional_value(fields, "epsilon_v_total", 0.0);
+       row.saturation = get_value(fields, "S_L");
+       row.n_l = get_value(fields, "n_L");
+       row.phi_m = get_value(fields, "phi_m");
+       row.phi_M = get_value(fields, "phi_M");
+       row.phi = get_value(fields, "phi");
+       row.n_S = get_value(fields, "n_S");
+       row.n_L = get_value(fields, "n_L");
+       row.rho_LR = get_value(fields, "rho_LR");
+       row.rho_LR = get_value(fields, "rho_LR");
+       row.omega_l = get_value(fields, "omega_l");
+       row.mu_LR = get_value(fields, "mu_LR");
+       row.rho_l_hat = get_value(fields, "rho_l_hat");
+       row.delta_epsilon_sw = get_value(fields, "delta_epsilon_sw");
+       row.epsilon_sw = get_value(fields, "epsilon_sw");
+       row.sigma_S_xx = get_value(fields, "sigma_S_xx");
+       rows.push_back(row);
+   }
+
+   return rows;
+}
+
+double dsm_micromacroMCCScaledTolerance(double const value,
+                                         double const relative = 1e-8,
+                                         double const absolute = 1e-12)
+{
+   return absolute + relative * std::max(1.0, std::abs(value));
+}
+
+// namespace
+
+TEST(MaterialLib_RMBridgeMFront_DSMMicroMacroMCC,
+   NeutralDSMMicroMacroStateMatchesVerifiedMCCBridge)
+{

```

```

+ auto parameters = createDSMMicroMacroMCCParameters();
+ auto reference = createReferenceMCCModel(parameters);
+ auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+ auto reference_state = reference->createMaterialStateVariables();
+ auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+ initializeState(*reference, *reference_state);
+ initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+
+ ParameterLib::SpatialPosition x();
+
+ MPL::VariableArray previous;
+ previous.mechanical_strain.emplace<KV>(KV::Zero());
+ previous.stress.emplace<KV>(dsm_micromacroMCCIsotropicStress(-5e3));
+ previous.liquid_phase_pressure = -5e3;
+ previous.temperature = 293.15;
+
+ auto reference_previous = previous;
+ auto dsm_micromacro_previous = previous;
+
+ struct Step
+ {
+     double t;
+     double eps_v;
+     double liquid_pressure;
+ };
+
+ std::vector<Step> const steps = {
+     {1.0, -5e-4, -2e4},
+     {2.0, -1.5e-3, -4e4},
+     {3.0, -3.5e-3, -7e4},
+     {4.0, -6.5e-3, -1e5},
+ };
+
+ for (std::size_t i = 0; i < steps.size(); ++i)
+ {
+     MPL::VariableArray reference_current = reference_previous;
+     reference_current.mechanical_strain.emplace<KV>(
+         dsm_micromacroMCCIsotropicStrainFromVolumetric(steps[i].eps_v));
+     reference_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+     MPL::VariableArray dsm_micromacro_current = dsm_micromacro_previous;
+     dsm_micromacro_current.mechanical_strain.emplace<KV>(
+         dsm_micromacroMCCIsotropicStrainFromVolumetric(steps[i].eps_v));
+     dsm_micromacro_current.liquid_phase_pressure = steps[i].liquid_pressure;
+
+     double const dt = i == 0 ? steps[i].t : steps[i].t - steps[i - 1].t;
+
+     auto reference_response = reference->integrateStressPressureCoupled(
+         reference_previous, reference_current, steps[i].t, x, dt,
+         *reference_state);
+     auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+         dsm_micromacro_previous, dsm_micromacro_current, steps[i].t, x, dt,
+         *dsm_micromacro_state);
+
+     ASSERT_TRUE(reference_response);
+     ASSERT_TRUE(dsm_micromacro_response);
+     ASSERT_TRUE(reference_response->state);
+     ASSERT_TRUE(dsm_micromacro_response->state);
+
+     EXPECT_TRUE((reference_response->stress - dsm_micromacro_response->stress)
+         .isZero(5e-11));
+
+     EXPECT_TRUE(
+         (reference_response->dStress_dStrain -
+          dsm_micromacro_response->dStress_dStrain)
+         .isZero(1e-8));
+     EXPECT_TRUE((reference_response->dStress_dLiquidPressure -
+         dsm_micromacro_response->dStress_dLiquidPressure)
+         .isZero(1e-12));
+     EXPECT_TRUE((reference_response->dSaturation_dStrain -
+         dsm_micromacro_response->dSaturation_dStrain)
+         .isZero(1e-12));
+     EXPECT_NEAR(reference_response->saturation, dsm_micromacro_response->saturation,
+         1e-15);
+     EXPECT_NEAR(reference_response->dSaturation_dLiquidPressure,
+         dsm_micromacro_response->dSaturation_dLiquidPressure, 1e-15);
+
+     for (auto const& name :
+         {"EquivalentPlasticStrain", "PreConsolidationPressure",
+          "PlasticVolumetricStrain", "VolumeRatio"})
+     {
+         auto const reference_value =
+             getInternalScalar(*reference, *reference_response->state, name);
+         auto const dsm_micromacro_value =
+             getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+                             name);
+         EXPECT_NEAR(reference_value, dsm_micromacro_value,
+             name == std::string("PreConsolidationPressure") ? 1e-8
+                                                         : 1e-14)
+             << name;
+     }
+
+     auto const reference_elastic_strain = getInternalVector(
+         *reference, *reference_response->state, "ElasticStrain");
+     auto const dsm_micromacro_elastic_strain = getInternalVector(
+         *dsm_micromacro_mcc, *dsm_micromacro_response->state, "ElasticStrain");
+     ASSERT_EQ(reference_elastic_strain.size(), dsm_micromacro_elastic_strain.size());
+     for (std::size_t k = 0; k < reference_elastic_strain.size(); ++k)
+     {
+         EXPECT_NEAR(reference_elastic_strain[k], dsm_micromacro_elastic_strain[k],
+             1e-14);
+     }
+
+     auto const n_l =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "n_l");
+     auto const rho_LR =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+                             "rho_LR");
+     auto const epsilon_sw = getInternalScalar(*dsm_micromacro_mcc,
+         *dsm_micromacro_response->state,
+         "epsilon_sw");
+
+     auto const phi_m =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi_m");
+     auto const phi_M =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi_M");
+     auto const mu_LR = getInternalScalar(*dsm_micromacro_mcc,
+         *dsm_micromacro_response->state,

```



```

+         "mu_LR");
+
+     auto const rho_l_hat =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+             "rho_l_hat");
+
+     EXPECT_TRUE(std::isfinite(n_l));
+     EXPECT_TRUE(std::isfinite(rho_LR));
+     EXPECT_TRUE(std::isfinite(epsilon_sw));
+     EXPECT_TRUE(std::isfinite(phi_m));
+     EXPECT_TRUE(std::isfinite(phi_M));
+     EXPECT_TRUE(std::isfinite(mu_LR));
+     EXPECT_TRUE(std::isfinite(rho_l_hat));
+     EXPECT_GT(n_l, 0.0);
+     EXPECT_GT(rho_LR, 1000.0);
+     EXPECT_NEAR(phi_m, n_l, 1e-14);
+     EXPECT_NEAR(phi_M, 0.432 - n_l, 1e-14);
+     EXPECT_NEAR(epsilon_sw, 0.0, 1e-14);
+     EXPECT_NEAR(rho_l_hat, 0.0, 1e-14);
+
+     reference_state = std::move(reference_response->state);
+     dsm_micromacro_state = std::move(dsm_micromacro_response->state);
+
+     reference_previous = reference_current;
+     reference_previous.stress.emplace<KV>(reference_response->stress);
+     dsm_micromacro_previous = dsm_micromacro_current;
+     dsm_micromacro_previous.stress.emplace<KV>(dsm_micromacro_response->stress);
+ }
+*)
+
+*TEST(MaterialLib_RMBridgeMFront_DSMMicroMacroMCC,
+    SwellingFeedbackChangesStressButKeepsCarrierSaturation)
+*{
+    auto parameters = createDSMMicroMacroMCCParameters(2.0, 1e-4, 0.1, 1300.0, 0.0);
+    auto reference = createReferenceMCCModel(parameters);
+    auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+    auto reference_state = reference->createMaterialStateVariables();
+    auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+    initializeState(*reference, *reference_state);
+    initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+
+    ParameterLib::SpatialPosition x();
+
+    MPL::VariableArray previous;
+    previous.mechanical_strain.emplace<KV>(KV::Zero());
+    previous.stress.emplace<KV>(dsm_micromacroMCCIsotropicStress(-5e3));
+    previous.liquid_phase_pressure = -5e3;
+    previous.temperature = 293.15;
+
+    constexpr double t = 1.0;
+    constexpr double dt = 1.0;
+    constexpr double eps_v = -2e-5;
+    constexpr double liquid_pressure = -4e4;
+
+    MPL::VariableArray current = previous;
+    current.mechanical_strain.emplace<KV>({
+        dsm_micromacroMCCIsotropicStrainFromVolumetric(eps_v)});
+    current.liquid_phase_pressure = liquid_pressure;
+
+    auto reference_response = reference->integrateStressPressureCoupled(
+        previous, current, t, x, dt, *reference_state);
+    auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+        previous, current, t, x, dt, *dsm_micromacro_state);
+
+    ASSERT_TRUE(reference_response);
+    ASSERT_TRUE(dsm_micromacro_response);
+    ASSERT_TRUE(reference_response->state);
+    ASSERT_TRUE(dsm_micromacro_response->state);
+
+    auto const epsilon_sw =
+        getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+            "epsilon_sw");
+    EXPECT_GT(std::abs(epsilon_sw), 1e-12);
+
+    double const bulk_modulus = 52e6 / (3.0 * (1.0 - 2.0 * 0.3));
+    auto const expected_stress =
+        reference_response->stress -
+        dsm_micromacroMCCIsotropicStress(bulk_modulus * epsilon_sw);
+    EXPECT_TRUE((expected_stress - dsm_micromacro_response->stress).isZero(5e-8));
+
+    auto const sigma_S_values =
+        getInternalVector(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "sigma_S");
+    ASSERT_EQ(sigma_S_values.size(), static_cast<std::size_t>(KV::RowsAtCompileTime));
+    KV sigma_S = KV::Zero();
+    for (Eigen::Index i = 0; i < KV::RowsAtCompileTime; ++i)
+    {
+        sigma_S[i] = sigma_S_values[static_cast<std::size_t>(i)];
+    }
+
+    auto const added_swelling_stress =
+        dsm_micromacro_response->stress - reference_response->stress;
+    EXPECT_TRUE((added_swelling_stress - sigma_S).isZero(5e-8));
+
+    EXPECT_NEAR(reference_response->saturation, dsm_micromacro_response->saturation,
+        1e-15);
+    EXPECT_TRUE((reference_response->dSaturation_dStrain -
+        dsm_micromacro_response->dSaturation_dStrain)
+        .isZero(1e-12));
+    EXPECT_NEAR(reference_response->dSaturation_dLiquidPressure,
+        dsm_micromacro_response->dSaturation_dLiquidPressure, 1e-15);
+
+    auto evaluate_dsm_micromacro_response =
+        [&](double const eps_v_value, double const pressure_value)
+    {
+        auto local_parameters =
+            createDSMMicroMacroMCCParameters(2.0, 1e-4, 0.1, 1300.0, 0.0);
+        auto local_model = createDSMMicroMacroMCCModel(local_parameters);
+        auto local_state = local_model->createMaterialStateVariables();
+        initializeState(*local_model, *local_state);
+
+        MPL::VariableArray local_current = previous;
+        local_current.mechanical_strain.emplace<KV>({
+            dsm_micromacroMCCIsotropicStrainFromVolumetric(eps_v_value)});
+        local_current.liquid_phase_pressure = pressure_value;
+
+        return local_model->integrateStressPressureCoupled(
+            previous, local_current, t, x, dt, *local_state);
+    };
+
+};

```

```

+
+ double const dp_fd = std::max(1e-6, std::abs(liquid_pressure) * 1e-6);
+ auto dsm_micromacro_p_plus = evaluate_dsm_micromacro_response(eps_v, liquid_pressure + dp_fd);
+ auto dsm_micromacro_p_minus = evaluate_dsm_micromacro_response(eps_v, liquid_pressure - dp_fd);
+ ASSERT_TRUE(dsm_micromacro_p_plus);
+ ASSERT_TRUE(dsm_micromacro_p_minus);
+ auto const dsigma_dp_fd =
+   (dsm_micromacro_p_plus->stress - dsm_micromacro_p_minus->stress) / (2.0 * dp_fd);
+ EXPECT_TRUE((dsm_micromacro_response->dStress_dLiquidPressure - dsigma_dp_fd)
+   .isZero(5e-4));
+
+}
+
+*TEST(MaterialLib_RMBCBridgeMFron_DSMMicroMacroMCC,
+  MicroOnlySwellingKeepsMacroExchangeOffAndPreservesCarrierSaturation)
+{
+  auto parameters = createDSMMicroMacroMCCParameters(2.0, 0.0, 0.1, 1300.0, 0.0);
+  auto reference = createReferenceMCCModel(parameters);
+  auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+  auto reference_state = reference->createMaterialStateVariables();
+  auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+  initializeState(*reference, *reference_state);
+  initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+
+  ParameterLib::SpatialPosition x{};
+
+  MPL::VariableArray previous;
+  previous.mechanical_strain.emplace<KV>(KV::Zero());
+  previous.stress.emplace<KV>(dsm_micromacroMCCIIsotropicStress(-5e3));
+  previous.liquid_phase_pressure = -5e3;
+  previous.temperature = 293.15;
+
+  constexpr double t = 1.0;
+  constexpr double dt = 1.0;
+  constexpr double eps_v = -2e-5;
+  constexpr double liquid_pressure = -4e4;
+
+  MPL::VariableArray current = previous;
+  current.mechanical_strain.emplace<KV>{
+    dsm_micromacroMCCIIsotropicStrainFromVolumetric(eps_v)};
+  current.liquid_phase_pressure = liquid_pressure;
+
+  auto reference_response = reference->integrateStressPressureCoupled(
+    previous, current, t, x, dt, *reference_state);
+  auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+    previous, current, t, x, dt, *dsm_micromacro_state);
+
+  ASSERT_TRUE(reference_response);
+  ASSERT_TRUE(dsm_micromacro_response);
+  ASSERT_TRUE(dsm_micromacro_response->state);
+
+  auto const epsilon_sw =
+    getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+      "epsilon_sw");
+  EXPECT_GT(std::abs(epsilon_sw), 1e-12);
+
+  EXPECT_NEAR(reference_response->liquid_mass_exchange_source, 0.0, 1e-18);
+  EXPECT_NEAR(dsm_micromacro_response->liquid_mass_exchange_source, 0.0, 1e-18);
+
+  EXPECT_NEAR(reference_response->saturation, dsm_micromacro_response->saturation,
+    1e-15);
+  EXPECT_TRUE((reference_response->dSaturation_dStrain -
+    dsm_micromacro_response->dSaturation_dStrain)
+    .isZero(1e-12));
+  EXPECT_NEAR(reference_response->dSaturation_dLiquidPressure,
+    dsm_micromacro_response->dSaturation_dLiquidPressure, 1e-15);
+
+  auto const sigma_S_values =
+    getInternalVector(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "sigma_S");
+  ASSERT_EQ(sigma_S_values.size(), static_cast<std::size_t>(KV::RowsAtCompileTime));
+  KV sigma_S = KV::Zero();
+  for (Eigen::Index i = 0; i < KV::RowsAtCompileTime; ++i)
+  {
+    sigma_S[i] = sigma_S_values[static_cast<std::size_t>(i)];
+  }
+
+  auto const added_swelling_stress =
+    dsm_micromacro_response->stress - reference_response->stress;
+  EXPECT_TRUE((added_swelling_stress - sigma_S).isZero(5e-8));
+}
+
+*TEST(MaterialLib_RMBCBridgeMFron_DSMMicroMacroMCC,
+  NativeAlignedStageOneStressGapIsMicroSupportStress)
+{
+  auto parameters = createDSMMicroMacroMCCParameters(
+    0.1, 1e-13, 0.1, 2095.322465784393, 0.0, 0.0, 0.0, 1.0, 6e-20, 1e-3);
+  auto reference = createReferenceMCCModel(parameters);
+  auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+  auto reference_state = reference->createMaterialStateVariables();
+  auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+  initializeState(*reference, *reference_state);
+  initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+
+  ParameterLib::SpatialPosition x{};
+
+  MPL::VariableArray previous;
+  previous.mechanical_strain.emplace<KV>(KV::Zero());
+  previous.stress.emplace<KV>(dsm_micromacroMCCIIsotropicStress(0.0));
+  previous.liquid_phase_pressure = -1e6;
+  previous.temperature = 293.15;
+
+  MPL::VariableArray current = previous;
+  current.liquid_phase_pressure = 2e3;
+
+  constexpr double t = 1e3;
+  constexpr double dt = 1e3;
+
+  auto reference_response = reference->integrateStressPressureCoupled(
+    previous, current, t, x, dt, *reference_state);
+  auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+    previous, current, t, x, dt, *dsm_micromacro_state);
+
+  ASSERT_TRUE(reference_response);
+  ASSERT_TRUE(dsm_micromacro_response);
+  ASSERT_TRUE(reference_response->state);
+  ASSERT_TRUE(dsm_micromacro_response->state);

```

```

+
+ auto const sigma_S_values =
+   getInternalVector(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "sigma_S");
+   ASSERT_EQ(sigma_S_values.size(), static_cast<std::size_t>(KV::RowsAtCompileTime));
+   KV sigma_S = KV::Zero();
+   for (Eigen::Index i = 0; i < KV::RowsAtCompileTime; ++i)
+   {
+     sigma_S[i] = sigma_S_values[static_cast<std::size_t>(i)];
+   }
+
+
+ auto const added_swelling_stress =
+   dsm_micromacro_response->stress - reference_response->stress;
+   EXPECT_GT(sigma_S.norm(), 1e-3);
+   EXPECT_TRUE(added_swelling_stress - sigma_S.isZero(1e-6));
+   EXPECT_NEAR(reference_response->saturation, dsm_micromacro_response->saturation,
+     1e-15);
+
+
+*TEST(MaterialLib_RMBridgeMFront_DSMMicroMacroMCC,
+  PressureCoupledResponseExposesMacroExchangeSource)
+{
+  auto parameters = createDSMMicroMacroMCCParameters(
+    0.1, 1e-13, 0.01, 2276.031917690513, 0.0, 0.0, 1.0, 5.1e-21, 1e-3);
+  auto reference = createReferenceMCCModel(parameters);
+  auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+  auto reference_state = reference->createMaterialStateVariables();
+  auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+  initializeState(*reference, *reference_state);
+  initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+
+  ParameterLib::SpatialPosition x{};
+
+  MPL::VariableArray previous;
+  previous.mechanical_strain.emplace<KV>(KV::Zero());
+  previous.stress.emplace<KV>(KV::Zero());
+  previous.liquid_phase_pressure = 2e3;
+  previous.temperature = 293.15;
+
+  MPL::VariableArray current = previous;
+
+  constexpr double t = 1e5;
+  constexpr double dt = 1e5;
+
+  auto reference_response = reference->integrateStressPressureCoupled(
+    previous, current, t, x, dt, *reference_state);
+  auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+    previous, current, t, x, dt, *dsm_micromacro_state);
+
+  ASSERT_TRUE(reference_response);
+  ASSERT_TRUE(dsm_micromacro_response);
+  ASSERT_TRUE(dsm_micromacro_response->state);
+
+  auto const rho_l_hat =
+    getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+      "rho_l_hat");
+  auto const sigma_S_values =
+    getInternalVector(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "sigma_S");
+  KV sigma_S = KV::Zero();
+  for (Eigen::Index i = 0; i < KV::RowsAtCompileTime; ++i)
+  {
+    sigma_S[i] = sigma_S_values[static_cast<std::size_t>(i)];
+  }
+
+  EXPECT_NEAR(reference_response->liquid_mass_exchange_source, 0.0, 1e-18);
+  EXPECT_GT(rho_l_hat, 0.0);
+  EXPECT_NEAR(dsm_micromacro_response->liquid_mass_exchange_source, -rho_l_hat,
+    std::max(1e-18, 1e-12 * std::abs(rho_l_hat)));
+  EXPECT_TRUE(
+    (dsm_micromacro_response->swelling_stress - sigma_S).isZero(1e-12));
+
+
+*TEST(MaterialLib_RMBridgeMFront_DSMMicroMacroMCC,
+  NegativeAttractiveConventionMatchesLegacyNegativeHamakerPath)
+{
+  auto legacy_parameters = createDSMMicroMacroMCCParameters(
+    8.0, 1e-13, 0.1, 1300.0, 0.0, 0.0, 0.0, -6e-20);
+  auto native_aligned_parameters = createDSMMicroMacroMCCParameters(
+    8.0, 1e-13, 0.1, 1300.0, 0.0, 0.0, 0.0, 1.0, 6e-20);
+
+  auto legacy_model = createDSMMicroMacroMCCModel(legacy_parameters);
+  auto native_aligned_model =
+    createDSMMicroMacroMCCModel(native_aligned_parameters);
+
+  auto legacy_state = legacy_model->createMaterialStateVariables();
+  auto native_aligned_state =
+    native_aligned_model->createMaterialStateVariables();
+  initializeState(*legacy_model, *legacy_state);
+  initializeState(*native_aligned_model, *native_aligned_state);
+
+  ParameterLib::SpatialPosition x{};
+
+  MPL::VariableArray previous;
+  previous.mechanical_strain.emplace<KV>(KV::Zero());
+  previous.stress.emplace<KV>(dsm_micromacroMCCIsotropicStress(0.0));
+  previous.liquid_phase_pressure = -1e6;
+  previous.temperature = 293.15;
+
+  MPL::VariableArray current = previous;
+  current.liquid_phase_pressure = 2e3;
+
+  constexpr double t = 1e5;
+  constexpr double dt = 1e5;
+
+  auto legacy_response = legacy_model->integrateStressPressureCoupled(
+    previous, current, t, x, dt, *legacy_state);
+  auto native_aligned_response =
+    native_aligned_model->integrateStressPressureCoupled(
+      previous, current, t, x, dt, *native_aligned_state);
+
+  ASSERT_TRUE(legacy_response);
+  ASSERT_TRUE(native_aligned_response);
+  ASSERT_TRUE(legacy_response->state);
+  ASSERT_TRUE(native_aligned_response->state);
+
+  EXPECT_TRUE((legacy_response->stress - native_aligned_response->stress)
+    .isZero(1e-8));
+
+

```

```

+ EXPECT_TRUE(
+   (legacy_response->dStress_dStrain -
+    native_aligned_response->dStress_dStrain)
+   .isZero(1e-8));
+ EXPECT_TRUE((legacy_response->dStress_dLiquidPressure -
+   native_aligned_response->dStress_dLiquidPressure)
+   .isZero(1e-8));
+ EXPECT_NEAR(legacy_response->saturation, native_aligned_response->saturation,
+   1e-15);
+ EXPECT_NEAR(legacy_response->dSaturation_dLiquidPressure,
+   native_aligned_response->dSaturation_dLiquidPressure, 1e-15);
+
+ auto const legacy_epsilon_sw =
+   getInternalScalar(*legacy_model, *legacy_response->state, "epsilon_sw");
+ auto const aligned_epsilon_sw = getInternalScalar(
+   *native_aligned_model, *native_aligned_response->state, "epsilon_sw");
+ EXPECT_GT(std::abs(legacy_epsilon_sw, 1e-8);
+ EXPECT_NEAR(legacy_epsilon_sw, aligned_epsilon_sw, 1e-12);
+)
+
+*TEST(MaterialLib_RMBridgeMFront_DSMMicroMacroMCC,
+   DSMMicroMacroSaturationModeMatchesTullerLawAndKeepsStressSurface)
+{
+   auto parameters = createDSMMicroMacroMCCParameters(0.0, 0.0, 0.1, 1300.0, 0.0,
+   1.0);
+   auto reference = createReferenceMCCModel(parameters);
+   auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+   auto reference_state = reference->createMaterialStateVariables();
+   auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+   initializeState(*reference, *reference_state);
+   initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+
+   ParameterLib::SpatialPosition x();
+
+   MPL::VariableArray previous;
+   previous.mechanical_strain.emplace<KV>(KV::Zero());
+   previous.stress.emplace<KV>(dsm_micromacroMCCIIsotropicStress(-5e3));
+   previous.liquid_phase_pressure = -5e3;
+   previous.temperature = 293.15;
+
+   constexpr double t = 1.0;
+   constexpr double dt = 1.0;
+   constexpr double eps_v = -2e-5;
+   constexpr double liquid_pressure = -4e4;
+
+   MPL::VariableArray current = previous;
+   current.mechanical_strain.emplace<KV>(
+   dsm_micromacroMCCIIsotropicStrainFromVolumetric(eps_v));
+   current.liquid_phase_pressure = liquid_pressure;
+
+   auto reference_response = reference->integrateStressPressureCoupled(
+   previous, current, t, x, dt, *reference_state);
+   auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+   previous, current, t, x, dt, *dsm_micromacro_state);
+
+   ASSERT_TRUE(reference_response);
+   ASSERT_TRUE(dsm_micromacro_response);
+   ASSERT_TRUE(reference_response->state);
+   ASSERT_TRUE(dsm_micromacro_response->state);
+
+   EXPECT_TRUE((reference_response->stress - dsm_micromacro_response->stress)
+   .isZero(5e-11));
+   EXPECT_TRUE((reference_response->dStress_dStrain -
+   dsm_micromacro_response->dStress_dStrain)
+   .isZero(1e-8));
+   EXPECT_TRUE((reference_response->dStress_dLiquidPressure -
+   dsm_micromacro_response->dStress_dLiquidPressure)
+   .isZero(1e-12));
+
+   EXPECT_NEAR(dsm_micromacro_response->saturation,
+   dsm_micromacroTullerSaturation(liquid_pressure), 1e-15);
+   EXPECT_NEAR(dsm_micromacro_response->dSaturation_dLiquidPressure,
+   dsm_micromacroTullerDSaturationDLiquidPressure(liquid_pressure),
+   1e-15);
+   EXPECT_TRUE(dsm_micromacro_response->dSaturation_dStrain.isZero(1e-12));
+
+   auto const epsilon_sw =
+   getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+   "epsilon_sw");
+   EXPECT_NEAR(epsilon_sw, 0.0, 1e-14);
+
+   MPL::VariableArray saturated_current = previous;
+   saturated_current.mechanical_strain.emplace<KV>(
+   dsm_micromacroMCCIIsotropicStrainFromVolumetric(eps_v));
+   saturated_current.liquid_phase_pressure = 1e3;
+
+   auto saturated_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+   previous, saturated_current, t, x, dt, *dsm_micromacro_state);
+   ASSERT_TRUE(saturated_response);
+   EXPECT_NEAR(saturated_response->saturation, 1.0, 1e-15);
+   EXPECT_NEAR(saturated_response->dSaturation_dLiquidPressure, 0.0, 1e-15);
+   EXPECT_TRUE(saturated_response->dSaturation_dStrain.isZero(1e-12));
+}
+
+*TEST(MaterialLib_RMBridgeMFront_DSMMicroMacroMCC,
+   DSMMicroMacroSupportStateMatchesOverlapTransferBaseline)
+{
+   auto const baseline_rows = loadDSMMicroMacroSupportBaselineRows(
+   TestInfoLib::TestInfo::data_path +
+   "/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv");
+   ASSERT_EQ(baseline_rows.size(), 5);
+
+   auto parameters = createDSMMicroMacroSupportParameters();
+   auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+   auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+   initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+
+   ParameterLib::SpatialPosition x();
+
+   MPL::VariableArray previous;
+   previous.mechanical_strain.emplace<KV>(KV::Zero());
+   previous.stress.emplace<KV>(KV::Zero());
+   previous.liquid_phase_pressure = 0.0;
+   previous.liquid_saturation = 1.0;
+   previous.temperature = 293.15;

```

```

+
+ for (auto const& row : baseline_rows)
+ {
+     MPL::VariableArray current = previous;
+     current.liquid_phase_pressure = row.pressure;
+
+     auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+         previous, current, static_cast<double>(row.step + 1), x, 1.0,
+         *dsm_micromacro_state);
+
+     ASSERT_TRUE(dsm_micromacro_response);
+     ASSERT_TRUE(dsm_micromacro_response->state);
+
+     auto const n_l =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "n_l");
+     auto const phi_m =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi_m");
+     auto const phi_M =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi_M");
+     auto const phi =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi");
+     auto const n_S =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "n_S");
+     auto const n_L =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "n_L");
+     auto const rho_LR =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+             "rho_LR");
+     auto const rho_LR =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+             "rho_LR");
+     auto const omega_L =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+             "omega_L");
+     auto const mu_LR =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+             "mu_LR");
+     auto const rho_L_hat =
+         getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+             "rho_L_hat");
+     auto const delta_epsilon_sw = getInternalScalar(
+         *dsm_micromacro_mcc, *dsm_micromacro_response->state, "delta_epsilon_sw");
+     auto const epsilon_sw = getInternalScalar(
+         *dsm_micromacro_mcc, *dsm_micromacro_response->state, "epsilon_sw");
+     auto const sigma_S =
+         getInternalVector(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+             "sigma_S");
+
+     EXPECT_NEAR(dsm_micromacro_response->saturation, row.saturation,
+         dsm_micromacroMCCScaledTolerance(row.saturation));
+     EXPECT_NEAR(n_l, row.n_l,
+         dsm_micromacroMCCScaledTolerance(row.n_l, 5e-3, 1e-8));
+     EXPECT_NEAR(phi_m, row.phi_m,
+         dsm_micromacroMCCScaledTolerance(row.phi_m, 5e-3, 1e-8));
+     EXPECT_NEAR(phi_M, row.phi_M,
+         dsm_micromacroMCCScaledTolerance(row.phi_M, 5e-3, 1e-8));
+     EXPECT_NEAR(phi, row.phi,
+         dsm_micromacroMCCScaledTolerance(row.phi, 1e-12, 1e-12));
+     EXPECT_NEAR(n_S, row.n_S,
+         dsm_micromacroMCCScaledTolerance(row.n_S, 1e-12, 1e-12));
+     EXPECT_NEAR(n_L, row.n_L,
+         dsm_micromacroMCCScaledTolerance(row.n_L, 5e-3, 1e-8));
+     EXPECT_NEAR(rho_LR, row.rho_LR,
+         dsm_micromacroMCCScaledTolerance(row.rho_LR, 1e-3, 1e-6));
+     EXPECT_NEAR(rho_LR, row.rho_LR,
+         dsm_micromacroMCCScaledTolerance(row.rho_LR, 1e-12, 1e-12));
+     EXPECT_NEAR(omega_L, row.omega_L,
+         dsm_micromacroMCCScaledTolerance(row.omega_L, 5e-3, 1e-8));
+     EXPECT_NEAR(mu_LR, row.mu_LR,
+         dsm_micromacroMCCScaledTolerance(row.mu_LR, 1e-2, 1e-8));
+     EXPECT_NEAR(rho_L_hat, row.rho_L_hat,
+         dsm_micromacroMCCScaledTolerance(row.rho_L_hat, 1e-2, 1e-8));
+     EXPECT_NEAR(delta_epsilon_sw, row.delta_epsilon_sw,
+         dsm_micromacroMCCScaledTolerance(row.delta_epsilon_sw, 5e-3,
+             1e-8));
+     EXPECT_NEAR(epsilon_sw, row.epsilon_sw,
+         dsm_micromacroMCCScaledTolerance(row.epsilon_sw, 5e-3, 1e-8));
+     ASSERT_EQ(sigma_S.size(), 6u);
+     EXPECT_NEAR(sigma_S[0], row.sigma_S_xx,
+         dsm_micromacroMCCScaledTolerance(row.sigma_S_xx, 5e-3, 1e-6));
+     EXPECT_NEAR(sigma_S[1], row.sigma_S_xx,
+         dsm_micromacroMCCScaledTolerance(row.sigma_S_xx, 5e-3, 1e-6));
+     EXPECT_NEAR(sigma_S[2], row.sigma_S_xx,
+         dsm_micromacroMCCScaledTolerance(row.sigma_S_xx, 5e-3, 1e-6));
+     EXPECT_NEAR(sigma_S[3], 0.0, 1e-12);
+     EXPECT_NEAR(sigma_S[4], 0.0, 1e-12);
+     EXPECT_NEAR(sigma_S[5], 0.0, 1e-12);
+
+     dsm_micromacro_state = std::move(dsm_micromacro_response->state);
+     dsm_micromacro_state->pushBackState();
+
+     previous = current;
+     previous.stress.emplace<KV>(dsm_micromacro_response->stress);
+     previous.liquid_saturation = dsm_micromacro_response->saturation;
+ }
+ }
+
+ TEST(MaterialLib_RMFBridgeMFront_DSMMicroMacroMCC,
+     DSMMicroMacroSupportStateMatchesStrainCoupledBaseline)
+ {
+     auto const overlap_rows = loadDSMMicroMacroSupportBaselineRows(
+         TestInfoLib::TestInfo::data_path +
+         "/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_overlap_transfer_baseline.csv");
+     auto const strain_rows = loadDSMMicroMacroSupportBaselineRows(
+         TestInfoLib::TestInfo::data_path +
+         "/MaterialLib/MFront/RichardsMechanicsDSMMicroMacroBridge_strain_coupled_overlap_baseline.csv");
+     ASSERT_EQ(overlap_rows.size(), 5);
+     ASSERT_EQ(strain_rows.size(), 5);
+     auto const& anchor = overlap_rows.back();
+
+     auto parameters = createDSMMicroMacroSupportParameters();
+     auto dsm_micromacro_mcc = createDSMMicroMacroMCCModel(parameters);
+
+     auto dsm_micromacro_state = dsm_micromacro_mcc->createMaterialStateVariables();
+     initializeState(*dsm_micromacro_mcc, *dsm_micromacro_state);
+     setInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_state, "n_l", anchor.n_l);
+     setInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_state, "rho_LR", anchor.rho_LR);

```

```

+ setInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_state, "epsilon_sw",
+   anchor.epsilon_sw);
+
+
+ KV anchor_stress = KV::Zero();
+ anchor_stress[0] = anchor.sigma_S_xx;
+ anchor_stress[1] = anchor.sigma_S_xx;
+ anchor_stress[2] = anchor.sigma_S_xx;
+ setDSMicroMacroMCCThermodynamicForces(*dsm_micromacro_state, anchor_stress,
+   anchor.saturation);
+ dsm_micromacro_state->pushBackState();
+
+ ParameterLib::SpatialPosition x();
+
+
+ MPL::VariableArray previous;
+ previous.mechanical_strain.emplace<KV>(KV::Zero());
+ previous.stress.emplace<KV>(anchor_stress);
+ previous.liquid_phase_pressure = anchor.pressure;
+ previous.liquid_saturation = anchor.saturation;
+ previous.temperature = 293.15;
+
+ for (auto const& row : strain_rows)
+ {
+   MPL::VariableArray current = previous;
+   current.liquid_phase_pressure = row.pressure;
+   current.mechanical_strain.emplace<KV>({
+     dsm_micromacroMCCIsotropicStrainFromVolumetric(row.epsilon_v_total));
+
+   auto dsm_micromacro_response = dsm_micromacro_mcc->integrateStressPressureCoupled(
+     previous, current, static_cast<double>(row.step + 1), x, 1.0,
+     *dsm_micromacro_state);
+
+   ASSERT_TRUE(dsm_micromacro_response);
+   ASSERT_TRUE(dsm_micromacro_response->state);
+
+   auto const n_l =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "n_L");
+   auto const phi_m =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi_m");
+   auto const phi_M =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi_M");
+   auto const phi =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "phi");
+   auto const n_S =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "n_S");
+   auto const n_L =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state, "n_L");
+   auto const rho_LR =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+       "rho_LR");
+   auto const rho_LR =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+       "rho_LR");
+   auto const omega_l =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+       "omega_l");
+   auto const mu_LR =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+       "mu_LR");
+   auto const rho_l_hat =
+     getInternalScalar(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+       "rho_l_hat");
+   auto const delta_epsilon_sw = getInternalScalar(
+     *dsm_micromacro_mcc, *dsm_micromacro_response->state, "delta_epsilon_sw");
+   auto const epsilon_sw = getInternalScalar(
+     *dsm_micromacro_mcc, *dsm_micromacro_response->state, "epsilon_sw");
+   auto const sigma_S =
+     getInternalVector(*dsm_micromacro_mcc, *dsm_micromacro_response->state,
+       "sigma_S");
+
+   EXPECT_NEAR(dsm_micromacro_response->saturation, row.saturation,
+     dsm_micromacroMCCScaledTolerance(row.saturation));
+   EXPECT_NEAR(n_l, row.n_l,
+     dsm_micromacroMCCScaledTolerance(row.n_l, 5e-3, 1e-8));
+   EXPECT_NEAR(phi_m, row.phi_m,
+     dsm_micromacroMCCScaledTolerance(row.phi_m, 5e-3, 1e-8));
+   EXPECT_NEAR(phi_M, row.phi_M,
+     dsm_micromacroMCCScaledTolerance(row.phi_M, 5e-3, 1e-8));
+   EXPECT_NEAR(phi, row.phi,
+     dsm_micromacroMCCScaledTolerance(row.phi, 1e-12, 1e-12));
+   EXPECT_NEAR(n_S, row.n_S,
+     dsm_micromacroMCCScaledTolerance(row.n_S, 1e-12, 1e-12));
+   EXPECT_NEAR(n_L, row.n_L,
+     dsm_micromacroMCCScaledTolerance(row.n_L, 5e-3, 1e-8));
+   EXPECT_NEAR(rho_LR, row.rho_LR,
+     dsm_micromacroMCCScaledTolerance(row.rho_LR, 1e-3, 1e-6));
+   EXPECT_NEAR(rho_LR, row.rho_LR,
+     dsm_micromacroMCCScaledTolerance(row.rho_LR, 1e-12, 1e-12));
+   EXPECT_NEAR(omega_l, row.omega_l,
+     dsm_micromacroMCCScaledTolerance(row.omega_l, 5e-3, 1e-8));
+   EXPECT_NEAR(mu_LR, row.mu_LR,
+     dsm_micromacroMCCScaledTolerance(row.mu_LR, 1e-2, 1e-8));
+   EXPECT_NEAR(rho_l_hat, row.rho_l_hat,
+     dsm_micromacroMCCScaledTolerance(row.rho_l_hat, 1e-2, 1e-8));
+   EXPECT_NEAR(delta_epsilon_sw, row.delta_epsilon_sw,
+     dsm_micromacroMCCScaledTolerance(row.delta_epsilon_sw, 5e-3,
+       1e-8));
+   EXPECT_NEAR(epsilon_sw, row.epsilon_sw,
+     dsm_micromacroMCCScaledTolerance(row.epsilon_sw, 5e-3, 1e-8));
+   double const bulk_modulus = 1e10 / (3.0 * (1.0 - 2.0 * 0.25));
+   double const expected_sigma_S_xx = -bulk_modulus * row.epsilon_sw;
+   ASSERT_EQ(sigma_S.size(), 6u);
+   EXPECT_NEAR(sigma_S[0], expected_sigma_S_xx,
+     dsm_micromacroMCCScaledTolerance(expected_sigma_S_xx, 5e-3,
+       1e-6));
+   EXPECT_NEAR(sigma_S[1], expected_sigma_S_xx,
+     dsm_micromacroMCCScaledTolerance(expected_sigma_S_xx, 5e-3,
+       1e-6));
+   EXPECT_NEAR(sigma_S[2], expected_sigma_S_xx,
+     dsm_micromacroMCCScaledTolerance(expected_sigma_S_xx, 5e-3,
+       1e-6));
+   EXPECT_NEAR(sigma_S[3], 0.0, 1e-12);
+   EXPECT_NEAR(sigma_S[4], 0.0, 1e-12);
+   EXPECT_NEAR(sigma_S[5], 0.0, 1e-12);
+
+   dsm_micromacro_state = std::move(dsm_micromacro_response->state);
+   dsm_micromacro_state->pushBackState();
+ }

```



```

+ using MechanicsBase =
+   MaterialLib::Solids::MechanicsBase<DisplacementDim>;
+ using KelvinMatrix = typename MechanicsBase::KelvinMatrix;
+ using KelvinVector = typename MechanicsBase::KelvinVector;
+ using SaturationStrainJacobian =
+   typename MechanicsBase::SaturationStrainJacobian;
+
+ typename MechanicsBase::PressureCoupledResponse response(
+   KelvinVector::Zero(),
+   0.625,
+   KelvinMatrix::Identity(),
+   KelvinVector::Constant(2.5),
+   SaturationStrainJacobian::Constant(1.5),
+   4.0,
+   KelvinVector::Constant(-3.0),
+   0.0,
+   std::make_unique<typename MechanicsBase::MaterialStateVariables>());
+
+ auto const mapped =
+   ProcessLib::RichardsMechanics::makePressureCoupledSolidData<
+     DisplacementDim>(response);
+
+ EXPECT_TRUE(mapped.is_active);
+ EXPECT_DOUBLE_EQ(mapped.saturation, 0.625);
+ EXPECT_DOUBLE_EQ(mapped.dS_L_dp_cap, -4.0);
+ EXPECT_TRUE(
+   mapped.dS_L_dStrain.isApprox(SaturationStrainJacobian::Constant(1.5)));
+ EXPECT_TRUE(mapped.dSigma_dLiquidPressure.isApprox(
+   KelvinVector::Constant(2.5)));
+ EXPECT_TRUE(mapped.swelling_stress.isApprox(KelvinVector::Constant(-3.0)));
+ EXPECT_DOUBLE_EQ(mapped.Liquid_mass_exchange_source, 0.0);
+)
diff --git a/mfront/tests/behaviours/CheckParamPassing1.mfront b/mfront/tests/behaviours/CheckParamPassing1.mfront
new file mode 100644
index 0000000000..da9e8c68e5
--- /dev/null
+++ b/mfront/tests/behaviours/CheckParamPassing1.mfront
@@ -0,0 +1,86 @@
+// vim: ft=cpp sw=2 ts=2 sts=2 et
+
+
+@OSL DefaultGenericBehaviour;
+@Behaviour CheckParamPassing1;
+@Author Christoph Lehmann;
+@Date 29 / 07 / 2022;
+@Description { Check parameter passing between OGS and MFront }
+
+
+@Gradient StrainTensor eps_total;
+eps_total.setGlossaryName("Strain");
+
+
+@Gradient real p_LR;
+p_LR.setEntryName("LiquidPressure");
+
+
+@Flux StressTensor sigma;
+sigma.setGlossaryName("Stress");
+
+
+@Flux real S_L;
+S_L.setEntryName("Saturation");
+
+
+@MaterialProperty stress E;
+E.setGlossaryName("YoungModulus");
+@MaterialProperty real nu;
+nu.setGlossaryName("PoissonRatio");
+@MaterialProperty real alpha;
+alpha.setGlossaryName("ThermalExpansion");
+
+
+@ProvidesTangentOperator;
+@Integrator {
+  // TODO restrict to 3D!
+
+  StressTensor const sigma_prev = sigma;
+  real const S_L_prev = S_L;
+
+  sigma(0) = 2301;
+  sigma(1) = 2302;
+  sigma(2) = 2303;
+  sigma(3) = 2304;
+  sigma(5) = 2305;
+  sigma(4) = 2306;
+  S_L = 2307;
+  (void) computeTangentOperator;
+
+  // error checks
+  // report parameter errors by adding 10 000
+  // report gradient errors by adding 100 000
+  // report gradient increment errors by adding 1 000 000
+  // report prev tdyn forces errors by adding 10 000 000
+  // report external state variable errors by adding 100 000 000
+  const double param_error = 10000;
+  const double grad_error = 100000;
+  const double dgrad_error = 1000000;
+  const double prev_force_error = 10000000;
+  const double ext_state_error = 100000000;
+
+  if (E != 101) { sigma(0) += param_error; }
+  if (nu != 102) { sigma(1) += param_error; }
+  if (alpha != 103) { sigma(2) += param_error; }
+
+  if (eps_total(0) != 1201) { sigma(0) += grad_error; }
+  if (eps_total(1) != 1202) { sigma(1) += grad_error; }
+  if (eps_total(2) != 1203) { sigma(2) += grad_error; }
+  if (eps_total(3) != 1204) { sigma(3) += grad_error; }
+  if (eps_total(5) != 1205) { sigma(5) += grad_error; }
+  if (eps_total(4) != 1206) { sigma(4) += grad_error; }
+  if (p_LR != 1207) { S_L += grad_error; }
+
+  if (deps_total(0) != 1000) { sigma(0) += dgrad_error; }
+  if (deps_total(1) != 1000) { sigma(1) += dgrad_error; }
+  if (deps_total(2) != 1000) { sigma(2) += dgrad_error; }
+  if (deps_total(3) != 1000) { sigma(3) += dgrad_error; }
+  if (deps_total(5) != 1000) { sigma(5) += dgrad_error; }
+  if (deps_total(4) != 1000) { sigma(4) += dgrad_error; }
+  if (dp_LR != 1000) { S_L += dgrad_error; }
+
+  if (sigma_prev(0) != 1301) { sigma(0) += prev_force_error; }
+  if (sigma_prev(1) != 1302) { sigma(1) += prev_force_error; }
+  if (sigma_prev(2) != 1303) { sigma(2) += prev_force_error; }
+  if (sigma_prev(3) != 1304) { sigma(3) += prev_force_error; }

```



```

+ if (sigma_prev(5) != 1305) { sigma(5) += prev_force_error; }
+ if (sigma_prev(4) != 1306) { sigma(4) += prev_force_error; }
+ if (S_L_prev != 1307) { S_L += prev_force_error; }
+
+ if (T != 1401) { sigma(0) += ext_state_error; }
+ if (dT != 1000) { sigma(1) += ext_state_error; }
+ )
diff --git a/mfront/tests/behaviours/CheckStiffnessMatrixAllBlocks.mfront b/mfront/tests/behaviours/CheckStiffnessMatrixAllBlocks.mfront
new file mode 100644
index 0000000000..398c933ecd
--- /dev/null
+++ b/mfront/tests/behaviours/CheckStiffnessMatrixAllBlocks.mfront
@@ -0,0 +1,100 @@
// vim: ft=cpp sw=2 ts=2 sts=2 et
+
+@DSL DefaultGenericBehaviour;
+@Behaviour CheckStiffnessMatrixAllBlocks;
+@Author Christoph Lehmann;
+@Date 01 / 08 / 2022;
+@Description {
+ Check passing the stiffness matrix between OGS and Mfront.
+ In this behaviour all usual tangent operator blocks are computed
+ (except derivatives w.r.t. T).
+}
+
+@Gradient StrainTensor eps_total;
+eps_total.setGlossaryName("Strain");
+
+@Gradient real p_LR;
+p_LR.setEntryName("LiquidPressure");
+
+@Flux StressTensor sigma;
+sigma.setGlossaryName("Stress");
+
+@Flux real S_L;
+S_L.setEntryName("Saturation");
+
+// @TangentOperatorBlocks(dsigma_ddeps_total, dsigma_ddp_LR, dsigma_ddT);
+
+@ProvidesTangentOperator;
+@Integrator {
+ // TODO restrict to 3D!
+
+ sigma(0) = 2301;
+ sigma(1) = 2302;
+ sigma(2) = 2303;
+ sigma(3) = 2304;
+ sigma(5) = 2305;
+ sigma(4) = 2306;
+ S_L = 2307;
+
+ if (computeTangentOperator_)
+ {
+ dsigma_ddeps_total(0,0) = 1001;
+ dsigma_ddeps_total(0,1) = 1002;
+ dsigma_ddeps_total(0,2) = 1003;
+ dsigma_ddeps_total(0,3) = 1004;
+ dsigma_ddeps_total(0,5) = 1005;
+ dsigma_ddeps_total(0,4) = 1006;
+
+ dsigma_ddeps_total(1,0) = 1007;
+ dsigma_ddeps_total(1,1) = 1008;
+ dsigma_ddeps_total(1,2) = 1009;
+ dsigma_ddeps_total(1,3) = 1010;
+ dsigma_ddeps_total(1,5) = 1011;
+ dsigma_ddeps_total(1,4) = 1012;
+
+ dsigma_ddeps_total(2,0) = 1013;
+ dsigma_ddeps_total(2,1) = 1014;
+ dsigma_ddeps_total(2,2) = 1015;
+ dsigma_ddeps_total(2,3) = 1016;
+ dsigma_ddeps_total(2,5) = 1017;
+ dsigma_ddeps_total(2,4) = 1018;
+
+ dsigma_ddeps_total(3,0) = 1019;
+ dsigma_ddeps_total(3,1) = 1020;
+ dsigma_ddeps_total(3,2) = 1021;
+ dsigma_ddeps_total(3,3) = 1022;
+ dsigma_ddeps_total(3,5) = 1023;
+ dsigma_ddeps_total(3,4) = 1024;
+
+ dsigma_ddeps_total(5,0) = 1025;
+ dsigma_ddeps_total(5,1) = 1026;
+ dsigma_ddeps_total(5,2) = 1027;
+ dsigma_ddeps_total(5,3) = 1028;
+ dsigma_ddeps_total(5,5) = 1029;
+ dsigma_ddeps_total(5,4) = 1030;
+
+ dsigma_ddeps_total(4,0) = 1031;
+ dsigma_ddeps_total(4,1) = 1032;
+ dsigma_ddeps_total(4,2) = 1033;
+ dsigma_ddeps_total(4,3) = 1034;
+ dsigma_ddeps_total(4,5) = 1035;
+ dsigma_ddeps_total(4,4) = 1036;
+
+ dsigma_ddp_LR(0) = 2001;
+ dsigma_ddp_LR(1) = 2002;
+ dsigma_ddp_LR(2) = 2003;
+ dsigma_ddp_LR(3) = 2004;
+ dsigma_ddp_LR(5) = 2005;
+ dsigma_ddp_LR(4) = 2006;
+
+ dS_L_ddeps_total(0) = 3001;
+ dS_L_ddeps_total(1) = 3002;
+ dS_L_ddeps_total(2) = 3003;
+ dS_L_ddeps_total(3) = 3004;
+ dS_L_ddeps_total(5) = 3005;
+ dS_L_ddeps_total(4) = 3006;
+
+ dS_L_ddp_LR = 4001;
+ }
+ }
diff --git a/mfront/tests/behaviours/CheckStiffnessMatrixAllBlocksReordered.mfront b/mfront/tests/behaviours/CheckStiffnessMatrixAllBlocksReordered.mfront
new file mode 100644
index 0000000000..d8fc8ba9b9
--- /dev/null

```

```

+++ b/mfront/tests/behaviours/CheckStiffnessMatrixAllBlocksReordered.mfront
@@ -0,0 +1,101 @@
+// vim: ft=cpp sw=2 ts=2 sts=2 et
+
+@DSL DefaultGenericBehaviour;
+@Behaviour CheckStiffnessMatrixAllBlocksReordered;
+@Author Christoph Lehmann;
+@Date 01 / 08 / 2022;
+@Description {
+  + Check passing the stiffness matrix between OGS and MFront.
+  + In this behaviour all usual tangent operator blocks are computed
+  + (except derivatives w.r.t. T).
+  + The blocks have been reordered compared to the base case.
+}
+
+@Gradient StrainTensor eps_total;
+eps_total.setGlossaryName("Strain");
+
+@Gradient real p_LR;
+p_LR.setEntryName("LiquidPressure");
+
+@Flux StressTensor sigma;
+sigma.setGlossaryName("Stress");
+
+@Flux real S_L;
+S_L.setEntryName("Saturation");
+
+@TangentOperatorBlocks{dS_L_ddeps_total, dsigma_ddp_LR, dS_L_ddp_LR, dsigma_ddeps_total};
+
+@ProvidesTangentOperator;
+@Integrator {
+  + // TODO restrict to 3D!
+  +
+  + sigma(0) = 2301;
+  + sigma(1) = 2302;
+  + sigma(2) = 2303;
+  + sigma(3) = 2304;
+  + sigma(5) = 2305;
+  + sigma(4) = 2306;
+  + S_L = 2307;
+  +
+  + if (computeTangentOperator_)
+  + {
+  +   dsigma_ddeps_total(0,0) = 1001;
+  +   dsigma_ddeps_total(0,1) = 1002;
+  +   dsigma_ddeps_total(0,2) = 1003;
+  +   dsigma_ddeps_total(0,3) = 1004;
+  +   dsigma_ddeps_total(0,5) = 1005;
+  +   dsigma_ddeps_total(0,4) = 1006;
+  +
+  +   dsigma_ddeps_total(1,0) = 1007;
+  +   dsigma_ddeps_total(1,1) = 1008;
+  +   dsigma_ddeps_total(1,2) = 1009;
+  +   dsigma_ddeps_total(1,3) = 1010;
+  +   dsigma_ddeps_total(1,5) = 1011;
+  +   dsigma_ddeps_total(1,4) = 1012;
+  +
+  +   dsigma_ddeps_total(2,0) = 1013;
+  +   dsigma_ddeps_total(2,1) = 1014;
+  +   dsigma_ddeps_total(2,2) = 1015;
+  +   dsigma_ddeps_total(2,3) = 1016;
+  +   dsigma_ddeps_total(2,5) = 1017;
+  +   dsigma_ddeps_total(2,4) = 1018;
+  +
+  +   dsigma_ddeps_total(3,0) = 1019;
+  +   dsigma_ddeps_total(3,1) = 1020;
+  +   dsigma_ddeps_total(3,2) = 1021;
+  +   dsigma_ddeps_total(3,3) = 1022;
+  +   dsigma_ddeps_total(3,5) = 1023;
+  +   dsigma_ddeps_total(3,4) = 1024;
+  +
+  +   dsigma_ddeps_total(5,0) = 1025;
+  +   dsigma_ddeps_total(5,1) = 1026;
+  +   dsigma_ddeps_total(5,2) = 1027;
+  +   dsigma_ddeps_total(5,3) = 1028;
+  +   dsigma_ddeps_total(5,5) = 1029;
+  +   dsigma_ddeps_total(5,4) = 1030;
+  +
+  +   dsigma_ddeps_total(4,0) = 1031;
+  +   dsigma_ddeps_total(4,1) = 1032;
+  +   dsigma_ddeps_total(4,2) = 1033;
+  +   dsigma_ddeps_total(4,3) = 1034;
+  +   dsigma_ddeps_total(4,5) = 1035;
+  +   dsigma_ddeps_total(4,4) = 1036;
+  +
+  +   dsigma_ddp_LR(0) = 2001;
+  +   dsigma_ddp_LR(1) = 2002;
+  +   dsigma_ddp_LR(2) = 2003;
+  +   dsigma_ddp_LR(3) = 2004;
+  +   dsigma_ddp_LR(5) = 2005;
+  +   dsigma_ddp_LR(4) = 2006;
+  +
+  +   dS_L_ddeps_total(0) = 3001;
+  +   dS_L_ddeps_total(1) = 3002;
+  +   dS_L_ddeps_total(2) = 3003;
+  +   dS_L_ddeps_total(3) = 3004;
+  +   dS_L_ddeps_total(5) = 3005;
+  +   dS_L_ddeps_total(4) = 3006;
+  +
+  +   dS_L_ddp_LR = 4001;
+  + }
+}
+
diff --git a/mfront/tests/behaviours/CheckStiffnessMatrixExcessBlocks.mfront b/mfront/tests/behaviours/CheckStiffnessMatrixExcessBlocks.mfront
new file mode 100644
index 0000000000..7f9e2d62da
--- /dev/null
+++ b/mfront/tests/behaviours/CheckStiffnessMatrixExcessBlocks.mfront
@@ -0,0 +1,107 @@
+// vim: ft=cpp sw=2 ts=2 sts=2 et
+
+@DSL DefaultGenericBehaviour;
+@Behaviour CheckStiffnessMatrixExcessBlocks;
+@Author Christoph Lehmann;
+@Date 01 / 08 / 2022;
+@Description {
+  + Check passing the stiffness matrix between OGS and MFront.

```

```

+ In this behaviour all usual tangent operator blocks are computed.
+ It also includes a derivative w.r.t. T.
+}
+
+@Gradient StrainTensor eps_total;
+eps_total.setGlossaryName("Strain");
+
+@Gradient real p_LR;
+p_LR.setEntryName("LiquidPressure");
+
+@Flux StressTensor sigma;
+sigma.setGlossaryName("Stress");
+
+@Flux real S_L;
+S_L.setEntryName("Saturation");
+
+@TangentOperatorBlocks{dS_L_ddeps_total, dsigma_ddp_LR, dS_L_ddp_LR, dsigma_ddT, dsigma_ddeps_total};
+
+@ProvidesTangentOperator;
+@Integrator {
+ // TODO restrict to 3D!
+
+
+ sigma(0) = 2301;
+ sigma(1) = 2302;
+ sigma(2) = 2303;
+ sigma(3) = 2304;
+ sigma(5) = 2305;
+ sigma(4) = 2306;
+ S_L = 2307;
+
+
+ if (computeTangentOperator_)
+ {
+
+   dsigma_ddeps_total(0,0) = 1001;
+   dsigma_ddeps_total(0,1) = 1002;
+   dsigma_ddeps_total(0,2) = 1003;
+   dsigma_ddeps_total(0,3) = 1004;
+   dsigma_ddeps_total(0,5) = 1005;
+   dsigma_ddeps_total(0,4) = 1006;
+
+
+   dsigma_ddeps_total(1,0) = 1007;
+   dsigma_ddeps_total(1,1) = 1008;
+   dsigma_ddeps_total(1,2) = 1009;
+   dsigma_ddeps_total(1,3) = 1010;
+   dsigma_ddeps_total(1,5) = 1011;
+   dsigma_ddeps_total(1,4) = 1012;
+
+
+   dsigma_ddeps_total(2,0) = 1013;
+   dsigma_ddeps_total(2,1) = 1014;
+   dsigma_ddeps_total(2,2) = 1015;
+   dsigma_ddeps_total(2,3) = 1016;
+   dsigma_ddeps_total(2,5) = 1017;
+   dsigma_ddeps_total(2,4) = 1018;
+
+
+   dsigma_ddeps_total(3,0) = 1019;
+   dsigma_ddeps_total(3,1) = 1020;
+   dsigma_ddeps_total(3,2) = 1021;
+   dsigma_ddeps_total(3,3) = 1022;
+   dsigma_ddeps_total(3,5) = 1023;
+   dsigma_ddeps_total(3,4) = 1024;
+
+
+   dsigma_ddeps_total(5,0) = 1025;
+   dsigma_ddeps_total(5,1) = 1026;
+   dsigma_ddeps_total(5,2) = 1027;
+   dsigma_ddeps_total(5,3) = 1028;
+   dsigma_ddeps_total(5,5) = 1029;
+   dsigma_ddeps_total(5,4) = 1030;
+
+
+   dsigma_ddeps_total(4,0) = 1031;
+   dsigma_ddeps_total(4,1) = 1032;
+   dsigma_ddeps_total(4,2) = 1033;
+   dsigma_ddeps_total(4,3) = 1034;
+   dsigma_ddeps_total(4,5) = 1035;
+   dsigma_ddeps_total(4,4) = 1036;
+
+
+   dsigma_ddp_LR(0) = 2001;
+   dsigma_ddp_LR(1) = 2002;
+   dsigma_ddp_LR(2) = 2003;
+   dsigma_ddp_LR(3) = 2004;
+   dsigma_ddp_LR(5) = 2005;
+   dsigma_ddp_LR(4) = 2006;
+
+
+   dS_L_ddeps_total(0) = 3001;
+   dS_L_ddeps_total(1) = 3002;
+   dS_L_ddeps_total(2) = 3003;
+   dS_L_ddeps_total(3) = 3004;
+   dS_L_ddeps_total(5) = 3005;
+   dS_L_ddeps_total(4) = 3006;
+
+
+   dS_L_ddp_LR = 4001;
+
+
+   dsigma_ddT(0) = 5001;
+   dsigma_ddT(1) = 5002;
+   dsigma_ddT(2) = 5003;
+   dsigma_ddT(3) = 5004;
+   dsigma_ddT(5) = 5005;
+   dsigma_ddT(4) = 5006;
+
+ }
+
+}
+
+diff --git a/mfront/tests/behaviours/CheckStiffnessMatrixMissingBlocks.mfront b/mfront/tests/behaviours/CheckStiffnessMatrixMissingBlocks.mfront
+new file mode 100644
+index 0000000000..a8e2c1320c
+--- /dev/null
++++ b/mfront/tests/behaviours/CheckStiffnessMatrixMissingBlocks.mfront
+@ -0,0 +1,54 @@
+// vim: ft=cpp sw=2 ts=2 sts=2 et
+
+
+@DSL DefaultGenericBehaviour;
+@Behaviour CheckStiffnessMatrixMissingBlocks;
+@Author Christoph Lehmann;
+@Date 01 / 08 / 2022;
+@Description {
+
+ Check passing the stiffness matrix between OGS and Mfront.
+
+ In this behaviour some tangent operator blocks are missing.
+
+}
+
+
+@Gradient StrainTensor eps_total;

```

```

+eps_total.setGlossaryName("Strain");
+
+@Gradient real p_LR;
+p_LR.setEntryName("LiquidPressure");
+
+@Flux StressTensor sigma;
+sigma.setGlossaryName("Stress");
+
+@Flux real S_L;
+S_L.setEntryName("Saturation");
+
+@TangentOperatorBlocks{dsigma_ddp_LR, dS_L_ddeps_total};
+
+@ProvidesTangentOperator;
+@Integrator {
+ // TODO restrict to 3D!
+
+ sigma(0) = 2301;
+ sigma(1) = 2302;
+ sigma(2) = 2303;
+ sigma(3) = 2304;
+ sigma(5) = 2305;
+ sigma(4) = 2306;
+ S_L = 2307;
+
+ if (computeTangentOperator_)
+ {
+ dsigma_ddp_LR(0) = 2001;
+ dsigma_ddp_LR(1) = 2002;
+ dsigma_ddp_LR(2) = 2003;
+ dsigma_ddp_LR(3) = 2004;
+ dsigma_ddp_LR(5) = 2005;
+ dsigma_ddp_LR(4) = 2006;
+
+ dS_L_ddeps_total(0) = 3001;
+ dS_L_ddeps_total(1) = 3002;
+ dS_L_ddeps_total(2) = 3003;
+ dS_L_ddeps_total(3) = 3004;
+ dS_L_ddeps_total(5) = 3005;
+ dS_L_ddeps_total(4) = 3006;
+ }
+}
diff --git a/scripts/cmake/Dependencies.cmake b/scripts/cmake/Dependencies.cmake
index 327103b0ba..0227fe5cb9 100644
--- a/scripts/cmake/Dependencies.cmake
+++ b/scripts/cmake/Dependencies.cmake
@@ -19,6 +19,13 @@ if(OGS_BUILD_TESTING)
)
add_library(autocheck INTERFACE IMPORTED)
else()
+ if(CONDA_BUILD)
+ find_program(VTKDIFF_TOOL vtkdiff REQUIRED)
+ add_executable(vtkdiff IMPORTED GLOBAL)
+ set_target_properties(vtkdiff PROPERTIES
+ IMPORTED_LOCATION "${VTKDIFF_TOOL}")
+ )
+ endif()
CPMAddPackage(
NAME googletest
GITHUB_REPOSITORY google/googletest
@@ -365,6 +372,9 @@ endif()
# ~~~
if(GUIX_BUILD OR CONDA_BUILD)
add_library(exprtk INTERFACE IMPORTED)
+ if(CONDA_BUILD AND DEFINED exprtk_SOURCE_DIR)
+ target_include_directories(exprtk SYSTEM INTERFACE ${exprtk_SOURCE_DIR})
+ endif()
else()
CPMAddPackage(
NAME exprtk
diff --git a/scripts/cmake/DependenciesExternalProject.cmake b/scripts/cmake/DependenciesExternalProject.cmake
index 8a7cfc13f..7f84da8d1f 100644
--- a/scripts/cmake/DependenciesExternalProject.cmake
+++ b/scripts/cmake/DependenciesExternalProject.cmake
@@ -52,6 +52,75 @@ if(OGS_USE_MFRONT)
endif()
find_package(TFEL)

+ # The installed TFEL package exposes imported targets, while the OGS
+ # MFront test helpers still rely on the TFEL CMake module that defines
+ # the behaviour-check helper functions. When building from a CPM source
+ # cache, include that module explicitly and provide the plural wrapper
+ # used by the OGS test CMakeLists.
+ if(NOT COMMAND mfront_behaviour_check_library)
+ if(NOT TARGET check)
+ add_custom_target(check)
+ endif()
+ if(MFRONT AND NOT TARGET mfront)
+ add_executable(mfront IMPORTED GLOBAL)
+ set_target_properties(mfront PROPERTIES
+ IMPORTED_LOCATION "${MFRONT}")
+ )
+ endif()
+ foreach(_tfel_lib
+ TFELMaterial TFELMath TFELUtilities TFEException
+ MFrontProfiling)
+ if(NOT TARGET ${_tfel_lib})
+ find_library(_${_tfel_lib}_LIBRARY ${_tfel_lib}
+ HINTS ${TFELHOME} PATH_SUFFIXES lib REQUIRED)
+ add_library(${_tfel_lib} STATIC IMPORTED GLOBAL)
+ set_target_properties(${_tfel_lib} PROPERTIES
+ IMPORTED_LOCATION "${${_tfel_lib}_LIBRARY}")
+ INTERFACE_INCLUDE_DIRECTORIES "${TFELHOME}/include"
+ )
+ endif()
+ endforeach()
+ file(GLOB _tfel_cmake_modules
+ "${CPM_SOURCE_CACHE}/_ext/TFEL/*src/TFEL/cmake/modules/tfel.cmake")
+ list(SORT _tfel_cmake_modules)
+ list(REVERSE _tfel_cmake_modules)
+ list(GET _tfel_cmake_modules 0 _tfel_cmake_module)
+ if(EXISTS "${_tfel_cmake_module}")
+ include("${_tfel_cmake_module}")
+ else()
+ message(FATAL_ERROR
+ "Could not locate TFEL CMake module tfel.cmake under "
+ "${CPM_SOURCE_CACHE}/_ext/TFEL; MFront behaviour check ")

```

```

+         "helpers are unavailable."
+     )
+ endif()
+ get_filename_component(_tfel_include_dir "${_tfel_cmake_module}"
+     DIRECTORY)
+ get_filename_component(_tfel_include_dir "${_tfel_include_dir}"
+     DIRECTORY)
+ get_filename_component(_tfel_include_dir "${_tfel_include_dir}"
+     DIRECTORY)
+ get_filename_component(_tfel_include_dir "${_tfel_include_dir}"
+     DIRECTORY)
+ get_filename_component(_tfel_include_dir "${_tfel_include_dir}"
+     DIRECTORY)
+ set(_tfel_include_dir "${_tfel_include_dir}/include")
+ foreach(_tfel_lib
+     TFELEMaterial TFELEMath TFELEUtilities TFELEException
+     MFrontProfiling)
+     if(TARGET ${_tfel_lib})
+         set_target_properties(${_tfel_lib} PROPERTIES
+             INTERFACE_INCLUDE_DIRECTORIES "${_tfel_include_dir}")
+     endif()
+ endforeach()
+ endif()
+ if(COMMAND mfront_behaviour_check_library
+     AND NOT COMMAND mfront_behaviours_check_library)
+     function(mfront_behaviours_check_library lib)
+         mfront_behaviour_check_library(${lib} generic ${ARGN})
+     endfunction()
+ endif()

+ if(MFRONT AND APPLE)
+     # TODO: check for version
+     # ~~~

diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCBenchmarkParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCBenchmarkParity.cmake
new file mode 100644
index 0000000000..317bc8c7bf
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCBenchmarkParity.cmake
@@ -0,0 +1,112 @@
+if(NOT DEFINED OGS_EXE)
+    message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+if(NOT DEFINED SOURCE_PATH)
+    message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+if(NOT DEFINED VTCDIFF_EXE)
+    message(FATAL_ERROR "VTCDIFF_EXE is required.")
+endif()
+
+if(NOT DEFINED BINARY_PATH)
+    message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+    execute_process(
+        COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+        WORKING_DIRECTORY "${SOURCE_PATH}"
+        RESULT_VARIABLE result
+        OUTPUT_VARIABLE stdout
+        ERROR_VARIABLE stderr
+    )
+
+    if(NOT result EQUAL 0)
+        message(
+            FATAL_ERROR
+            "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+            "stdout:\n${stdout}\n"
+            "stderr:\n${stderr}"
+        )
+    endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+    execute_process(
+        COMMAND
+        "${VTCDIFF_EXE}"
+        -a "${array_a}"
+        -b "${array_b}"
+        --abs "${abs_tol}"
+        --rel "${rel_tol}"
+        "${file_a}"
+        "${file_b}"
+        RESULT_VARIABLE result
+        OUTPUT_VARIABLE stdout
+        ERROR_VARIABLE stderr
+    )
+
+    if(NOT result EQUAL 0)
+        message(
+            FATAL_ERROR
+            "vtkdiff failed for '${array_a}' comparing\n"
+            "  ${file_a}\n"
+            "  ${file_b}\n"
+            "stdout:\n${stdout}\n"
+            "stderr:\n${stderr}"
+        )
+    endif()
+endfunction()
+
+run_ogs(mfront_restart_part1.prj)
+run_ogs(mfront_restart_part1_dsm_micromacro_mcc_bridge.prj)
+
+set(step_indices 0 1)
+set(step_times 0.000000 1000.000000)
+
+foreach(list_index RANGE 0 1)
+    list(GET step_indices ${list_index} ts)
+    list(GET step_times ${list_index} time)
+    set(
+        native_file
+        "${BINARY_PATH}/mfront_restart_part1_ts_${ts}_t_${time}.vtu"
+    )

```

```

+ set(
+   bridge_file
+   "${BINARY_PATH}/mfront_restart_part1_dsm_micromacro_mcc_bridge_ts_${ts}_t_${time}.vtu"
+ )
+
+ run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-15 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 5e-12 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 1e-10 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-15 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-15 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-15 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" ElasticStrain ElasticStrain 1e-15 0)
+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" EquivalentPlasticStrain
+   EquivalentPlasticStrain 1e-15 0
+ )
+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" PreConsolidationPressure
+   PreConsolidationPressure 1e-15 0
+ )
+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" PlasticVolumetricStrain
+   PlasticVolumetricStrain 1e-15 0
+ )
+ run_vtkdiff("${native_file}" "${bridge_file}" VolumeRatio VolumeRatio 1e-15 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" swelling_stress swelling_stress 1e-15 0)
+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" transport_porosity
+   transport_porosity 1e-15 0
+ )
+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" dry_density_solid dry_density_solid
+   1e-15 0
+ )
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCParity.cmake
new file mode 100644
index 0000000000..8e70c6dddf
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCParity.cmake
@@ -0,0 +1,89 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+      "${VTKDIFF_EXE}"
+      -a "${array_a}"
+      -b "${array_b}"
+      --abs "${abs_tol}"
+      --rel "${rel_tol}"
+      "${file_a}"
+      "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_parity_1element_mcc_native.prj)
+run_ogs(mfront_parity_1element_dsm_micromacro_mcc_bridge.prj)
+
+foreach(ts RANGE 0 4)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_parity_1element_mcc_native_ts_${ts}_t_${ts}.000000.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_parity_1element_dsm_micromacro_mcc_bridge_ts_${ts}_t_${ts}.000000.vtu"
+  )

```

```

+
+ run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 5e-11 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" ElasticStrain ElasticStrain 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" EquivalentPlasticStrain EquivalentPlasticStrain 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" PreConsolidationPressure PreConsolidationPressure 1e-8 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" PlasticVolumetricStrain PlasticVolumetricStrain 1e-14 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" VolumeRatio VolumeRatio 1e-14 0)
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCtullerBenchmarkParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCtullerB
enchmarkParity.cmake
new file mode 100644
index 0000000000..d777cd22c8
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCtullerBenchmarkParity.cmake
@@ -0,0 +1,112 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+      "${VTKDIFF_EXE}"
+      -a "${array_a}"
+      -b "${array_b}"
+      --abs "${abs_tol}"
+      --rel "${rel_tol}"
+      "${file_a}"
+      "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_restart_part1_dsm_micromacro_mcc_tuller_native.prj)
+run_ogs(mfront_restart_part1_dsm_micromacro_mcc_tuller_bridge.prj)
+
+set(step_indices 0 1000)
+set(step_times 0.000000 1000.000000)
+
+foreach(list_index RANGE 0 1)
+  list(GET step_indices ${list_index} ts)
+  list(GET step_times ${list_index} time)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_restart_part1_dsm_micromacro_mcc_tuller_native_ts_${ts}_t_${time}.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_restart_part1_dsm_micromacro_mcc_tuller_bridge_ts_${ts}_t_${time}.vtu"
+  )
+
+  run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-12 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 1e-6 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 1e-9 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-12 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-12 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" ElasticStrain ElasticStrain 1e-15 0)
+  run_vtkdiff(
+    "${native_file}" "${bridge_file}" EquivalentPlasticStrain
+    EquivalentPlasticStrain 1e-15 0
+  )
+  run_vtkdiff(
+    "${native_file}" "${bridge_file}" PreConsolidationPressure
+    PreConsolidationPressure 1e-15 0
+  )

```

```

+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" PlasticVolumetricStrain
+   PlasticVolumetricStrain 1e-12 0
+ )
+ run_vtkdiff("${native_file}" "${bridge_file}" VolumeRatio VolumeRatio 1e-15 0)
+ run_vtkdiff("${native_file}" "${bridge_file}" swelling_stress swelling_stress 1e-15 0)
+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" transport_porosity
+   transport_porosity 1e-15 0
+ )
+ run_vtkdiff(
+   "${native_file}" "${bridge_file}" dry_density_solid dry_density_solid
+   1e-9 0
+ )
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCtullerParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCtullerParity.cmake
ke
new file mode 100644
index 0000000000..68a8f57b20
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontDSMMicroMacroMCCtullerParity.cmake
@@ -0,0 +1,89 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs_project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+      "${VTKDIFF_EXE}"
+      -a "${array_a}"
+      -b "${array_b}"
+      --abs "${abs_tol}"
+      --rel "${rel_tol}"
+      "${file_a}"
+      "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_parity_1element_dsm_micromacro_mcc_tuller_native.prj)
+run_ogs(mfront_parity_1element_dsm_micromacro_mcc_tuller_bridge.prj)
+
+foreach(ts RANGE 0 4)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_parity_1element_dsm_micromacro_mcc_tuller_native_ts_${ts}_t_${ts}.000000.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_parity_1element_dsm_micromacro_mcc_tuller_bridge_ts_${ts}_t_${ts}.000000.vtu"
+  )
+
+  run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 2e-10 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 2e-10 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" ElasticStrain ElasticStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" EquivalentPlasticStrain EquivalentPlasticStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" PreConsolidationPressure PreConsolidationPressure 1e-8 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" PlasticVolumetricStrain PlasticVolumetricStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" VolumeRatio VolumeRatio 1e-14 0)
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontElasticParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontElasticParity.cmake
new file mode 100644
index 0000000000..c1270da750
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontElasticParity.cmake
@@ -0,0 +1,84 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs_project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+      "${VTKDIFF_EXE}"
+      -a "${array_a}"
+      -b "${array_b}"
+      --abs "${abs_tol}"
+      --rel "${rel_tol}"
+      "${file_a}"
+      "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_elastic_1element_dsm_micromacro_mcc_tuller_native.prj)
+run_ogs(mfront_elastic_1element_dsm_micromacro_mcc_tuller_bridge.prj)
+
+foreach(ts RANGE 0 4)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_elastic_1element_dsm_micromacro_mcc_tuller_native_ts_${ts}_t_${ts}.000000.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_elastic_1element_dsm_micromacro_mcc_tuller_bridge_ts_${ts}_t_${ts}.000000.vtu"
+  )
+
+  run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 2e-10 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 2e-10 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" ElasticStrain ElasticStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" EquivalentPlasticStrain EquivalentPlasticStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" PreConsolidationPressure PreConsolidationPressure 1e-8 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" PlasticVolumetricStrain PlasticVolumetricStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" VolumeRatio VolumeRatio 1e-14 0)
+endforeach()

```



```

+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+      "${VTKDIFF_EXE}"
+      -a "${array_a}"
+      -b "${array_b}"
+      --abs "${abs_tol}"
+      --rel "${rel_tol}"
+      "${file_a}"
+      "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_parity_1element_elastic_native.prj)
+run_ogs(mfront_parity_1element_elastic_bridge.prj)
+
+foreach(ts RANGE 0 4)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_parity_1element_elastic_native_ts_${ts}_t_${ts}.000000.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_parity_1element_elastic_bridge_ts_${ts}_t_${ts}.000000.vtu"
+  )
+
+  run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-15 0)
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCBenchmarkParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCBenchmarkParity.cmake
new file mode 100644
index 00000000000..28895f79fb
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCBenchmarkParity.cmake
@@ -0,0 +1,112 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )

```

```

+ if(NOT result EQUAL 0)
+   message(
+     FATAL_ERROR
+     "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+     "stdout:\n${stdout}\n"
+     "stderr:\n${stderr}"
+   )
+ endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+    "${VTKDIFF_EXE}"
+    -a "${array_a}"
+    -b "${array_b}"
+    --abs "${abs_tol}"
+    --rel "${rel_tol}"
+    "${file_a}"
+    "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_restart_part1.prj)
+run_ogs(mfront_restart_part1_rm_bridge.prj)
+
+set(step_indices 0 1)
+set(step_times 0.000000 1000.000000)
+
+foreach(list_index RANGE 0 1)
+  list(GET step_indices ${list_index} ts)
+  list(GET step_times ${list_index} time)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_restart_part1_ts_${ts}_t_${time}.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_restart_part1_rm_bridge_ts_${ts}_t_${time}.vtu"
+  )
+
+  run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 5e-12 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 1e-10 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" ElasticStrain ElasticStrain 1e-15 0)
+  run_vtkdiff(
+    "${native_file}" "${bridge_file}" EquivalentPlasticStrain
+    EquivalentPlasticStrain 1e-15 0
+  )
+  run_vtkdiff(
+    "${native_file}" "${bridge_file}" PreConsolidationPressure
+    PreConsolidationPressure 1e-15 0
+  )
+  run_vtkdiff(
+    "${native_file}" "${bridge_file}" PlasticVolumetricStrain
+    PlasticVolumetricStrain 1e-15 0
+  )
+  run_vtkdiff("${native_file}" "${bridge_file}" VolumeRatio VolumeRatio 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" swelling_stress swelling_stress 1e-15 0)
+  run_vtkdiff(
+    "${native_file}" "${bridge_file}" transport_porosity
+    transport_porosity 1e-15 0
+  )
+  run_vtkdiff(
+    "${native_file}" "${bridge_file}" dry_density_solid dry_density_solid
+    1e-15 0
+  )
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCParity.cmake
new file mode 100644
index 0000000000..0fda92a3d3
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontMCCParity.cmake
@@ -0,0 +1,89 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )

```

```

+
+ if(NOT result EQUAL 0)
+   message(
+     FATAL_ERROR
+     "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+     "stdout:\n${stdout}\n"
+     "stderr:\n${stderr}"
+   )
+ endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+    "${VTKDIFF_EXE}"
+    -a "${array_a}"
+    -b "${array_b}"
+    --abs "${abs_tol}"
+    --rel "${rel_tol}"
+    "${file_a}"
+    "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_parity_1element_mcc_native.prj)
+run_ogs(mfront_parity_1element_mcc_bridge.prj)
+
+foreach(ts RANGE 0 4)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_parity_1element_mcc_native_ts_${ts}_t_${ts}.000000.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_parity_1element_mcc_bridge_ts_${ts}_t_${ts}.000000.vtu"
+  )
+
+  run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 5e-11 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" ElasticStrain ElasticStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" EquivalentPlasticStrain EquivalentPlasticStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" PreConsolidationPressure PreConsolidationPressure 1e-8 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" PlasticVolumetricStrain PlasticVolumetricStrain 1e-14 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" VolumeRatio VolumeRatio 1e-14 0)
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontParity.cmake
new file mode 100644
index 0000000000..109443cb74
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontParity.cmake
@@ -0,0 +1,86 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+    "${VTKDIFF_EXE}"
+    -a "${array_a}"
+    -b "${array_b}"
+    --abs "${abs_tol}"
+    --rel "${rel_tol}"
+    "${file_a}"
+    "${file_b}"
+    RESULT_VARIABLE result

```

```

+     OUTPUT_VARIABLE stdout
+     ERROR_VARIABLE stderr
+ )
+
+ if(NOT result EQUAL 0)
+   message(
+     FATAL_ERROR
+     "vtkdiff failed for '${array_a}' comparing\n"
+     "  ${file_a}\n"
+     "  ${file_b}\n"
+     "stdout:\n${stdout}\n"
+     "stderr:\n${stderr}"
+   )
+ endif()
+endfunction()
+
+run_ogs(mfront_parity_1element_native.prj)
+run_ogs(mfront_parity_1element_bridge.prj)
+
+foreach(ts RANGE 0 4)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_parity_1element_native_ts_${ts}_t_${ts}.000000.vtu"
+  )
+  set(
+    bridge_file
+    "${BINARY_PATH}/mfront_parity_1element_bridge_ts_${ts}_t_${ts}.000000.vtu"
+  )
+
+  run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 1e-15 0)
+  # sigma still differs at machine precision because the native and bridge
+  # paths do not round identically.
+  run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 5e-12 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-15 0)
+  run_vtkdiff("${native_file}" "${bridge_file}" swelling_stress swelling_stress 1e-15 0)
+endforeach()
diff --git a/scripts/cmake/test/CompareRichardsMechanicsMFrontUnsaturatedElasticParity.cmake b/scripts/cmake/test/CompareRichardsMechanicsMFrontUnsaturatedElasticParity.cmake
new file mode 100644
index 0000000000..ea311e7028
--- /dev/null
+++ b/scripts/cmake/test/CompareRichardsMechanicsMFrontUnsaturatedElasticParity.cmake
@@ -0,0 +1,84 @@
+if(NOT DEFINED OGS_EXE)
+  message(FATAL_ERROR "OGS_EXE is required.")
+endif()
+
+if(NOT DEFINED SOURCE_PATH)
+  message(FATAL_ERROR "SOURCE_PATH is required.")
+endif()
+
+if(NOT DEFINED VTKDIFF_EXE)
+  message(FATAL_ERROR "VTKDIFF_EXE is required.")
+endif()
+
+if(NOT DEFINED BINARY_PATH)
+  message(FATAL_ERROR "BINARY_PATH is required.")
+endif()
+
+file(MAKE_DIRECTORY "${BINARY_PATH}")
+
+function(run_ogs project_file)
+  execute_process(
+    COMMAND "${OGS_EXE}" -o "${BINARY_PATH}" "${project_file}"
+    WORKING_DIRECTORY "${SOURCE_PATH}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "ogs failed for '${project_file}' in '${SOURCE_PATH}'.\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+function(run_vtkdiff file_a file_b array_a array_b abs_tol rel_tol)
+  execute_process(
+    COMMAND
+    "${VTKDIFF_EXE}"
+    -a "${array_a}"
+    -b "${array_b}"
+    --abs "${abs_tol}"
+    --rel "${rel_tol}"
+    "${file_a}"
+    "${file_b}"
+    RESULT_VARIABLE result
+    OUTPUT_VARIABLE stdout
+    ERROR_VARIABLE stderr
+  )
+
+  if(NOT result EQUAL 0)
+    message(
+      FATAL_ERROR
+      "vtkdiff failed for '${array_a}' comparing\n"
+      "  ${file_a}\n"
+      "  ${file_b}\n"
+      "stdout:\n${stdout}\n"
+      "stderr:\n${stderr}"
+    )
+  endif()
+endfunction()
+
+run_ogs(mfront_parity_1element_unsat_native.prj)
+run_ogs(mfront_parity_1element_unsat_bridge.prj)
+
+foreach(ts RANGE 0 4)
+  set(
+    native_file
+    "${BINARY_PATH}/mfront_parity_1element_unsat_native_ts_${ts}_t_${ts}.000000.vtu"
+  )
+  set(

```

```

+         bridge_file
+         "${BINARY_PATH}/mfront_parity_element_unsat_bridge_ts_${ts}_t_${ts}.000000.vtu"
+     )
+
+     run_vtkdiff("${native_file}" "${bridge_file}" displacement displacement 1e-15 0)
+     run_vtkdiff("${native_file}" "${bridge_file}" pressure pressure 1e-15 0)
+     run_vtkdiff("${native_file}" "${bridge_file}" sigma sigma 1e-15 0)
+     run_vtkdiff("${native_file}" "${bridge_file}" epsilon epsilon 1e-15 0)
+     run_vtkdiff("${native_file}" "${bridge_file}" saturation saturation 1e-15 0)
+     run_vtkdiff("${native_file}" "${bridge_file}" velocity velocity 1e-15 0)
+endforeach()

```